

From Communication Between Two Neurons to a Learned Decision Policy

Andrea Corti --- University of Pavia (TLB / Physics)

September 2026

Contents

1. Introduction, motivation, and theoretical foundations	4
1.1 Project motivation	4
1.2 Why the Izhikevich model	5
1.3 Development environment and tools	6
1.4 Structure of the document	7
2. Experimental journey: from the single neuron to the network	7
2.1 Basic communication between two neurons	7
2.2 Synaptic threshold	9
2.3 Short-term synaptic plasticity (facilitation)	11
2.4 Propagation along a chain of neurons	14
2.5 Temporal summation (convergence)	16
2.6 STDP — Spike-Timing-Dependent Plasticity	18
2.7 Pattern classification via STDP	21
2.8 Balanced excitation-inhibition network (300 neurons)	24
2.9 Response to a stimulus and homeostatic recovery	27
2.10 Modular architecture: two connected regions	30
2.11 Statistical verification: is the observed effect real or noise?	33
3. Scaling toward the size of a mouse brain	37
3.1 The reference number and the memory constraint	37
3.2 The key technical change: fixed-degree connectivity	37
3.3 Scaling data, measured on the MacBook Air M5	38
3.4 Effect of thermal throttling	39
3.5 Progressive scaling script: methodology	39
3.6 Final experiment at 40 million neurons	42
4. Beyond raw scale: spatial structure and attractors	45
4.1 Distance-dependent connectivity (small-world)	45
4.2 3D brain shape, and scaling to 1 million neurons	48

4.3 Attempt at an attractor via learning (STDP on a spatial network)	
— negative result	50
4.4 Ring attractor: a real neural attractor, built from scratch	51
4.5 Where the scientists have gotten to: the real state of the art (September 2026)	55
5. Toward a decision policy: chains, competition, learning	57
5.1 A chain of two attractors: activation of one determines the other . . .	57
5.2 Competition between alternatives: B "chooses" among several can- didates	60
5.3 Reinforcement learning: the network learns which alternative to choose	63
5.4 Contextual policy: different responses for different contexts	67
5.5 Learned bidirectional communication: from the fixed channel to the synapse that learns	71
5.6 Toward the spiking domain: a real impulse-based ring attractor . .	175
5.7 Learned communication between two spiking rings, with true STDP	180
5.8 Attempt at spiking bidirectionality: the same asymmetry as the rate model	185
5.9 Spiking bidirectionality: isolating the confound and a definitive negative conclusion	191
5.10 More messages in the spiking domain: no interference, unlike the rate model	204
5.11 Monte Carlo verification of the multi-message spiking result . . .	209
5.12 Spiking bidirectionality: a final conclusion after eight independent attempts	212
5.13 Transmission switch and Monte Carlo verification: the definitive, corrected conclusion	226
5.14 Homeostatic normalization: a real improvement, verified but not perfect	241
5.15 More reactive homeostasis: 7/8 stable seeds and the first clean bidirectional growth signal	252
5.16 Emergency switch: 7/8 clean seeds, the stubborn seed improves 40x	264
5.17 Spiking bidirectionality solved: 8/8 stable seeds, bidirectional growth in every single run	279
5.18 More bidirectional messages: completing the spiking dialogue . .	292
5.19 Three-message stress test: the architecture holds under greater load	305
5.20 Beyond the pair: a three-population relay (A->B->C->A)	317
5.21 Outside Brian2: physical impact of CNAO ion beams on the ring architecture (separate project)	326
5.22 From physical dose to function: knockout of the highest-dose neurons	328
5.23 Monte Carlo verification of the knockout: robust across 8 inde- pendent seeds	334

5.24	Negative control: a random knockout does not produce the same effect	339
5.25	The true Bragg peak, and a methodological artifact discovered while correcting for it	344
5.26	Re-verifying the knockout with the corrected dose map: the mechanism is confirmed, reversed	345
5.27	Unified project: complete dataset at the Bragg peak and a dose-response gradient	351
5.28	The dose-response gradient holds for protons too: four independent points	356
5.29	Functional recovery after damage: residual plasticity compensates, it does not repair	361
5.30	The point of no return: when the lesion is complete, recovery disappears	368
5.31	Physical dose or biological dose? Chiba/NIRS's RBE model applied to the neuron map	374
5.32	Limitation discovered: the per-neuron dose map is not reproducible run to run	379
5.33	The gradient on solid statistical grounds: a minimum threshold emerges	381
5.34	The real Kase/NIRS MKM model, validated against published data	387
5.35	The MKM model across all six CNAO clinical configurations . . .	389
5.36	Verified, not hypothesized: what causes the 150 MeV proton's anomalous RBE peak	390
5.37	More statistics help, but are not enough: quantifying the cost of stabilizing the map	392
5.38	The knockout size is not magic: the mechanism generalizes beyond "exactly 10"	393
5.39	The nanodosimetric link, first step: the secondary electron spectrum	403
5.40	Nanodosimetric link completed: real ICSD from CNAO secondary electrons	408
5.41	Monte Carlo verification of nanoICSD: unlike the dose map, the ICSDs are stable	410
5.42	Beyond ICSD: real DNA damage (SSB/DSB) from CNAO secondary electrons	410
5.43	DNA damage weighted over the entire real spectrum, not just two points	412
5.44	The physics-neuroscience bridge closed down to the DNA: physical dose remains a robust criterion	413
5.45	The C->A loop does not close: the three-ring relay is not self-sustaining	416
6.	Biophysical realism and synaptic density: comparison with the Allen Institute	422
6.1	The fundamental constraint: memory as a shared budget	422
6.2	Conductance-based synapses: a second axis of realism	422

6.3 First Allen-density network: 118,000 neurons, random connectivity	423
6.4 Synthesis: spatial structure + biophysical realism -- first unstable attempt	423
6.5 Retuning and verification: the successful correction	424
6.6 3D visualization of the final network	426
6.7 Final comparison with the Allen Institute	430
7. Discussion	431
7.1 Summary of the complete path	431
7.2 The two most important methodological lessons	431
7.3 What is missing for real brain-like functionality	432
8. Limitations of the work	433
9. Possible future developments	434
10. Glossary of technical terms used in the document	436
11. Materials and reproducibility	437

From communication between two neurons to a learned decision policy

A complete computational neural simulation journey: from the scale of a single neuron up to 40 million, from single-signal propagation up to a rudiment of reinforcement learning

Andrea Corti

September 2026

Personal project — Python 3.14, Brian2 2.10.1, MacBook Air M5 (24 GB RAM)

1. Introduction, motivation, and theoretical foundations

1.1 Project motivation

The project starts from a question that is easy to state but hard to answer in full: is it possible, with tools accessible to a university student — a laptop, the Python language, publicly available scientific literature — to build, step by step, a system that reproduces some of the fundamental computational principles of a biological neural circuit? And if so, how far can one get before running into the real limits — computational, theoretical, or both — that even professional research labs encounter?

The answer, as this document shows in detail, is nuanced: one can get surprisingly far in raw scale (tens of millions of simulated neurons on a personal laptop)

and in some basic computational phenomena (communication, thresholding, local learning, collective activity, even a rudiment of reinforcement learning) — but one also runs into, concretely and reproducibly, the same conceptual obstacles that make the full simulation of a real brain an open research problem even for labs with dedicated supercomputers.

One methodological principle guided the entire journey, and it is arguably as important a contribution of the project as the numerical results themselves: every claim had to be verified by code that was actually run, never assumed or eyeballed from a plot. When a result looked interesting but had not been checked with adequate statistical rigor (the clearest case is the propagation between two network modules, Section 2.11), the project stopped to verify it with the right tools — even when that meant discovering that the apparent effect was not real, or that an attempt to build a mechanism (the STDP-based attractor, Section 4.2) failed completely. This discipline recurs throughout the document, and it is deliberate: a rigorously verified negative result is, from a scientific standpoint, worth more than a positive result that was observed but never checked.

1.2 Why the Izhikevich model

Choosing the mathematical model to represent a single neuron is not trivial, and it was the project's first technical decision. In increasing order of biophysical detail and computational cost, several families of models exist:

- Integrate-and-Fire (IF) model: the simplest possible, a single linear differential equation for the membrane potential with a fixed threshold. Computationally almost free, but it reproduces neither the real shape of the action potential nor the rich variety of firing patterns observed in real neurons
- Izhikevich model (2003): two coupled differential equations (one for the membrane potential v , one for a recovery variable u). With the right choice of four parameters (a , b , c , d) it qualitatively reproduces nearly all firing patterns observed in real cortical neurons — regular spiking, fast spiking, bursting, chattering — at a computational cost only slightly higher than Integrate-and-Fire. This is the model chosen for the entire project
- Hodgkin-Huxley model (1952): four differential equations with a real biophysical basis (sodium and potassium ionic currents across the membrane, modeled explicitly). Extremely accurate and biologically interpretable, but computationally heavy — impractical for networks of millions of neurons on personal hardware within the timeframe of a project of this scope

For a project that needed to scale to tens of millions of neurons while keeping reasonable behavioral detail, the Izhikevich model represented the right compromise between realism and computational tractability — the same choice made by much of the computational neuroscience literature whenever simulation scale is a primary constraint.

The two equations used in every single script of the project (with minor parameterization variations discussed case by case) are, in Izhikevich's original form:

$$dv/dt = 0.04 v^2 + 5 v + 140 - u + I \text{ (} v \text{ in millivolts, } t \text{ in milliseconds)}$$

$$du/dt = a (b v - u)$$

reset condition: if $v \geq 30$ mV then $v \rightarrow c$, $u \rightarrow u + d$

The variable v represents the membrane potential (the analogue of the real electrical voltage across a neuron's cell membrane); u is a "recovery" variable that collectively represents potassium-channel activation and sodium-channel inactivation — the biological mechanism responsible for the refractory period that follows every spike (the brief interval during which a neuron cannot fire again, no matter how strong the stimulus).

The four parameters (a , b , c , d) determine the neuron's "type" of behavior. This project used, almost everywhere, the standard parameter set for "regular spiking" (RS) neurons, the most common type in real cerebral cortex:

Parameter	Value used	Meaning
a	0.02 /ms	recovery speed of variable u
b	0.2 /ms	sensitivity of u to sub-threshold fluctuations of v
c	-65 mV	value v is reset to after a spike
d	6–8 mV/ms (depending on the experiment)	increment of u after a spike (adaptation)

1.3 Development environment and tools

All experiments were developed and run on a MacBook Air with an Apple M5 chip and 24 GB of RAM. The main simulation software is Brian2 (version 2.10.1), a Python library specialized in simulating spiking neural networks, widely used in academic computational neuroscience. Brian2 compiles the user-specified differential equations into optimized C++/Cython code "behind the scenes," allowing efficient simulation of even fairly large networks while keeping a high-level, readable Python interface.

For the largest-scale experiments (beyond one million neurons, Section 3 and Section 4.1) it was necessary to abandon Brian2's standard probabilistic-connectivity syntax — which, conceptually, evaluates the connection probability for every possible pair of neurons, at a computational cost that grows as the square of the neuron count ($O(N^2)$) — in favor of an explicit, efficient construction of connection indices via NumPy and SciPy (in particular the KD-tree

data structure for spatial neighbor queries), then passed to Brian2 through the low-level `connect(i=..., j=...)` interface. The full details of this technical transition are discussed in Section 3.4.

Other tools used: SciPy (in particular `scipy.spatial.cKDTree` for efficient spatial queries and `scipy.stats` for statistical tests), NumPy for general numerical computation, Matplotlib for all static visualizations, and Three.js (a JavaScript library) for the two interactive 3D visualizations developed in Section 4.1.

1.4 Structure of the document

The document follows the real chronological order in which the project was developed, not a topic order reorganized after the fact. This choice is deliberate: it preserves the actual logic behind every decision, including course corrections and failed attempts, which are as much a part of the work's methodological value as the final positive results.

- Section 2: the basic experimental journey, from communication between two neurons to the rigorous statistical verification of propagation between network modules (11 experiments)
- Section 3: scaling the network up to 40 million neurons, with an empirical characterization of the MacBook Air M5's real hardware limits
- Section 4: beyond raw scale — spatial connectivity, brain-shaped 3D visualizations, attempts (one failed, one successful) at building a real neural attractor, and the real state of the art in professional research on the topic
- Section 5: chains of attractors, competition between alternatives, reinforcement learning, and a context-dependent decision policy — the most advanced part of the project
- Sections 6-9: discussion, limitations, future developments, materials and reproducibility

2. Experimental journey: from the single neuron to the network

This section walks through the project's eleven basic experiments in detail, each one built on the previous by adding only a single new element of complexity at a time — a deliberate methodological choice, so that any newly observed phenomenon can be attributed with certainty to the change just introduced, instead of conflating several changes at once.

2.1 Basic communication between two neurons

The project's first experiment checks the most elementary thing possible: can two neurons really "communicate" through a simulated synapse? Neuron A

receives a constant external current (the stimulus that makes it fire regularly); Neuron B receives no stimulus of its own — the only way it can become active is through the synapse connecting it to A.

The synapse is modeled in the simplest possible way in Brian2: when A crosses threshold and generates a spike, after a synaptic delay of 1.5 milliseconds (realistic for real chemical transmission, which requires time for the neurotransmitter to diffuse across the synaptic cleft) B's membrane potential receives an instantaneous increment of 20 mV.

Result: over 200 milliseconds of simulation, A produces 10 spikes (dictated by its constant input current), and B responds with 3 spikes, always and only after the set synaptic delay relative to the corresponding spike of A. This is the first concrete verification, numerically measured and not just eyeballed, that synaptic communication works as expected.

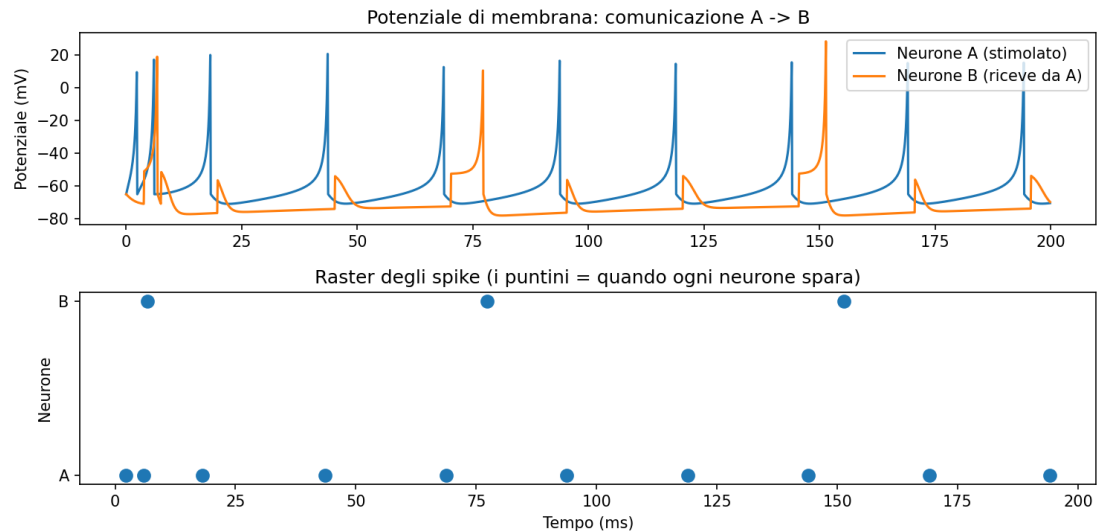


Fig. 1 — Membrane potential and raster: A→B communication.

Source code: `due_neuroni.py`

```
1 """
2 A -> B, sinapsi eccitatoria singola, Izhikevich RS.
3 Sanity check di base sulla toolchain prima di passare a reti piu' grandi.
4 """
5
6 from brian2 import *
7
8 # parametri Izhikevich, regime RS
9 a = 0.02/ms
10 b = 0.2/ms
11 c = -65*mV
12 d = 6*mV/ms
13
14 # I in mV/ms, come nei tutorial Brian2 standard (niente C/R espliciti)
15 eqs = '''
```

```

16 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
17 du/dt = a*(b*v - u) : volt/second
18 I : volt/second
19 '''
20
21 neurons = NeuronGroup(2, eqs, threshold='v > 30*mV', reset='v = c; u += d',
22                       method='euler')
23 neurons.v = -65*mV
24 neurons.u = b * neurons.v
25
26 # solo A (idx 0) riceve corrente esterna; B dipende solo dalla sinapsi
27 neurons.I = [15*mV/ms, 0*mV/ms]
28
29 syn = Synapses(neurons, neurons, on_pre='v_post += 20*mV')
30 syn.connect(i=0, j=1)
31 syn.delay = 1.5*ms
32
33 mon = StateMonitor(neurons, 'v', record=True)
34 spikes = SpikeMonitor(neurons)
35
36 run(200*ms)
37
38 figure(figsize=(10, 5))
39
40 subplot(2, 1, 1)
41 plot(mon.t/ms, mon.v[0]/mV, label='Neurone A (stimolato)')
42 plot(mon.t/ms, mon.v[1]/mV, label='Neurone B (riceve da A)')
43 ylabel('Potenziale (mV)')
44 legend(loc='upper right')
45 title('Potenziale di membrana: comunicazione A -> B')
46
47 subplot(2, 1, 2)
48 plot(spikes.t/ms, spikes.i, 'o', markersize=8)
49 yticks([0, 1], ['A', 'B'])
50 xlabel('Tempo (ms)')
51 ylabel('Neurone')
52 title('Raster degli spike (i puntini = quando ogni neurone spara)')
53
54 tight_layout()
55 savefig('due_neuroni_output.png', dpi=150)
56 print("Simulazione completata. Grafico salvato in due_neuroni_output.png")
57 print(f"Numero di spike del Neurone A: {sum(spikes.i == 0)}")
58 print(f"Numero di spike del Neurone B: {sum(spikes.i == 1)}")

```

2.2 Synaptic threshold

The second experiment systematically explores a parameter that was set arbitrarily in the first experiment: how strong does a synapse need to be for B to respond? Varying the synaptic strength between 2 and 30 millivolts (keeping everything else identical), a sharp, nonlinear transition is observed:

Synaptic strength (mV)	Spikes of A	Spikes of B
2	10	0
5	10	0
10	10	0
20	10	3
30	10	7

Below 10-15 millivolts neuron B never responds, no matter how many times A fires; above this threshold, B responds with an almost monotonically increasing frequency. This concretely demonstrates the concept of an activation threshold — an intrinsically nonlinear phenomenon, not a simple proportional effect (doubling the synaptic strength from 5 to 10 mV produces no measurable change; doubling it from 10 to 20 mV produces a drastic change, from zero response to substantial response).

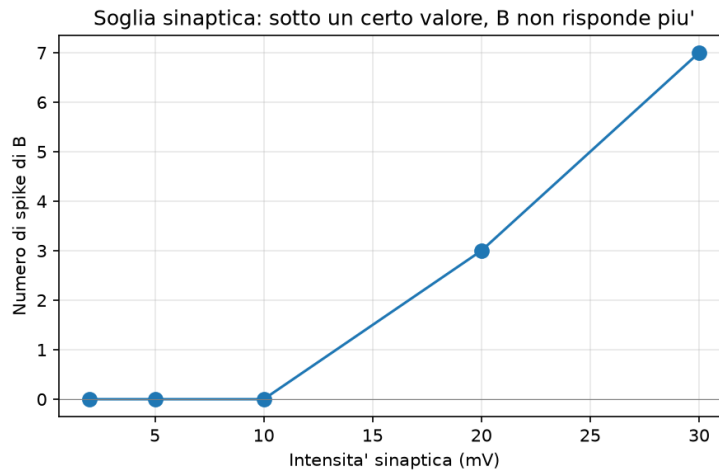


Fig. 2 — Synaptic threshold: a sharp transition between 10 and 20 mV.

Source code: *esplorazione_parametri.py*

```

1  """
2  Sweep su delay sinaptico e intensita' (A -> B): effetto sul timing di B
3  e curva soglia (intensita' minima perche' B risponda).
4  """
5
6  from brian2 import *
7
8  a = 0.02/ms
9  b = 0.2/ms
10 c = -65*mV
11 d = 6*mV/ms
12
13 eqs = '''
14 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
15 du/dt = a*(b*v - u) : volt/second
16 I : volt/second
17 '''
18
19 def run_simulation(delay_ms, synapse_strength_mV):
20     """Ritorna tempi/indici degli spike (A e B) per un dato delay/intensita'."""
21     start_scope()
22
23     neurons = NeuronGroup(2, eqs, threshold='v > 30*mV', reset='v = c; u += d',
24                           method='euler')
25     neurons.v = -65*mV
26     neurons.u = b * neurons.v
27     neurons.I = [15*mV/ms, 0*mV/ms]
28 
```

```

29     syn = Synapses(neurons, neurons, on_pre=f'v_post += {synapse_strength_mV}*mV')
30     syn.connect(i=0, j=1)
31     syn.delay = delay_ms*ms
32
33     spikes = SpikeMonitor(neurons)
34     run(200*ms)
35
36     return spikes.t/ms, spikes.i, sum(spikes.i == 0), sum(spikes.i == 1)
37
38
39 # sweep 1: delay sinaptico, intensita' fissa a 20 mV
40 delays_to_test = [0.5, 1.5, 5.0, 10.0]
41
42 figure(figsize=(10, 6))
43 for idx, delay in enumerate(delays_to_test):
44     t, i, n_a, n_b = run_simulation(delay, 20)
45     subplot(len(delays_to_test), 1, idx+1)
46     plot(t[i == 0], i[i == 0], 'o', color='C0', label='A' if idx == 0 else None)
47     plot(t[i == 1], i[i == 1]+0.1, 'o', color='C1', label='B' if idx == 0 else None)
48     yticks([0, 1.1], ['A', 'B'])
49     ylabel(f'delay={delay}ms', fontsize=9)
50     if idx == 0:
51         legend(loc='upper right', fontsize=8)
52 xlabel('Tempo (ms)')
53 suptitle('Effetto del delay sinaptico sulla risposta di B')
54 tight_layout()
55 savefig('esperimento_delay.png', dpi=150)
56 print("Grafico 1 salvato: esperimento_delay.png")
57
58 # sweep 2: intensita' sinaptica, delay fisso a 1.5 ms
59 strengths_to_test = [2, 5, 10, 20, 30]
60
61 print("\nEsperimento intensita' sinaptica:")
62 print(f"{'Intensita (mV)':>15} | {'Spike di A':>10} | {'Spike di B':>10}")
63 print("-" * 42)
64
65 results = []
66 for strength in strengths_to_test:
67     t, i, n_a, n_b = run_simulation(1.5, strength)
68     results.append((strength, n_b))
69     print(f"{strength:>15} | {n_a:>10} | {n_b:>10}")
70
71 figure(figsize=(7, 4))
72 strengths, n_b_values = zip(*results)
73 plot(strengths, n_b_values, 'o-', markersize=8)
74 xlabel('Intensita' sinaptica (mV)')
75 ylabel('Numero di spike di B')
76 title('Soglia sinaptica: sotto un certo valore, B non risponde piu')
77 axhline(0, color='gray', linewidth=0.5)
78 grid(alpha=0.3)
79 savefig('esperimento_soglia.png', dpi=150)
80 print("\nGrafico 2 salvato: esperimento_soglia.png")

```

2.3 Short-term synaptic plasticity (facilitation)

Up to this point, synaptic strength was a fixed value, decided once and for all at the start of the simulation. The third experiment introduces the first element of dynamics: a synapse whose strength **CHANGES** over time depending on how it is used — the first step, however elementary, toward the concept of synaptic “history.”

The mechanism implemented is called short-term facilitation: every time A generates a spike, the weight of the synapse to B increases by a fixed increment;

between one spike and the next, the weight decays exponentially toward a baseline value, with a time constant of 50 milliseconds. The result is a synaptic weight that, in the presence of closely spaced spikes, climbs progressively until it reaches a dynamic equilibrium between reinforcement and decay — a "saw-tooth" pattern clearly visible in the central panel of the figure below.

Over the course of the simulation, A produces 17 spikes and B responds with 5 — but unlike the second experiment, A's earliest spikes (when the synaptic weight is still at its baseline, sub-threshold value) do not trigger B; only after the weight has strengthened enough through repetition does B start responding.

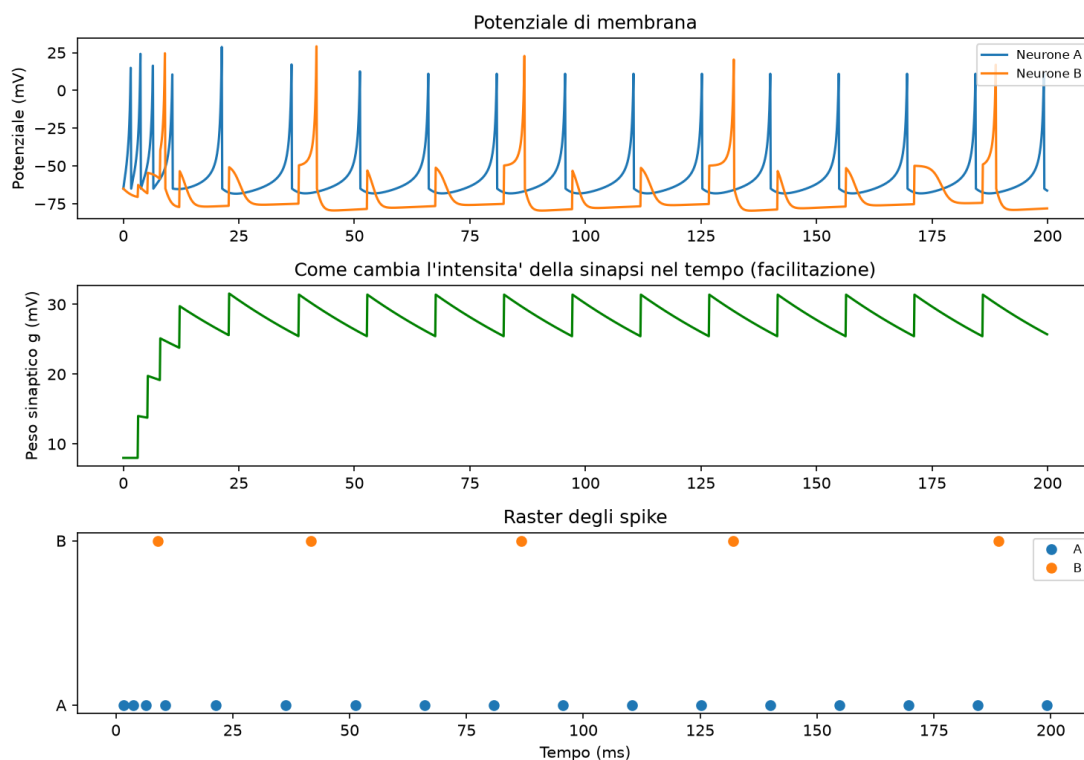


Fig. 3 — Evolution of the synaptic weight under short-term facilitation.

Source code: `plasticita_sinaptica.py`

```
1 """
2 Facilitazione sinaptica a breve termine (STF): g si incrementa ad ogni
3 spike di A e decade con tau_facilitation. A -> B, stimolo a raffica su A.
4 """
5
6 from brian2 import *
7
8 a = 0.02/ms
9 b = 0.2/ms
10 c = -65*mV
11 d = 6*mV/ms
```



```

12 eqs = '''
13 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
14 du/dt = a*(b*v - u) : volt/second
15 I : volt/second
16 '''
17
18
19 start_scope()
20
21 neurons = NeuronGroup(2, eqs, threshold='v > 30*mV', reset='v = c; u += d',
22                       method='euler')
23 neurons.v = -65*mV
24 neurons.u = b * neurons.v
25
26 # corrente su A piu' alta del solito -> raffica di spike, effetto facilitazione visibile
27 neurons.I = [25*mV/ms, 0*mV/ms]
28
29 # g: peso corrente della sinapsi, rilassa verso g_base con costante tau_facilitation
30 synapse_eqs = '''
31 dg/dt = (g_base - g) / tau_facilitation : volt (clock-driven)
32 g_base : volt
33 tau_facilitation : second
34 facilitation_increment : volt
35 '''
36
37 syn = Synapses(neurons, neurons, model=synapse_eqs,
38               on_pre='',
39               v_post += g
40               g += facilitation_increment
41               '',
42               method='euler')
43 syn.connect(i=0, j=1)
44 syn.delay = 1.5*ms
45 syn.g = 8*mV # sotto soglia da solo
46 syn.g_base = 8*mV
47 syn.tau_facilitation = 50*ms
48 syn.facilitation_increment = 6*mV
49
50 mon = StateMonitor(neurons, 'v', record=True)
51 g_mon = StateMonitor(syn, 'g', record=0)
52 spikes = SpikeMonitor(neurons)
53
54 run(200*ms)
55
56 figure(figsize=(10, 7))
57
58 subplot(3, 1, 1)
59 plot(mon.t/ms, mon.v[0]/mV, label='Neurone A', color='C0')
60 plot(mon.t/ms, mon.v[1]/mV, label='Neurone B', color='C1')
61 ylabel('Potenziale (mV)')
62 legend(loc='upper right', fontsize=8)
63 title('Potenziale di membrana')
64
65 subplot(3, 1, 2)
66 plot(g_mon.t/ms, g_mon.g[0]/mV, color='green')
67 ylabel('Peso sinaptico g (mV)')
68 title('Come cambia l\'intensita\' della sinapsi nel tempo (facilitazione)')
69
70 subplot(3, 1, 3)
71 plot(spikes.t[spikes.i == 0]/ms, [0]*sum(spikes.i == 0), 'o', color='C0', label='A')
72 plot(spikes.t[spikes.i == 1]/ms, [1]*sum(spikes.i == 1), 'o', color='C1', label='B')
73 yticks([0, 1], ['A', 'B'])
74 xlabel('Tempo (ms)')
75 title('Raster degli spike')
76 legend(loc='upper right', fontsize=8)
77
78 tight_layout()
79 savefig('plasticita_sinaptica.png', dpi=150)

```

```

80 print("Simulazione completata. Grafico salvato in plasticita_sinaptica.png")
81 print(f"Spike di A: {sum(spikes.i == 0)}")
82 print(f"Spike di B: {sum(spikes.i == 1)}")
83 print(f"Peso sinaptico iniziale: {syn.g[0]/mV:.1f} mV (vedi andamento nel grafico centrale)"
84       )
85 print("\nI primi spike di A da soli non bastano a far sparare B; dopo alcuni")
86 print("spike ravvicinati la sinapsi facilitata fa scattare B.")

```

2.4 Propagation along a chain of neurons

The fourth experiment moves from two neurons to eight, connected in a linear sequence ($N0 \rightarrow N1 \rightarrow N2 \rightarrow \dots \rightarrow N7$, each connected only to the next). Only $N0$ receives an external stimulus; the signal must cross the entire chain through synapses identical to those already verified.

The main result is the measurement of the propagation delay: each hop from one neuron to the next introduces about 3-4 milliseconds of delay (due to the combination of synaptic delay and the receiving neuron's integration time), for a total accumulated delay of 25.9 milliseconds from the first to the last neuron in the chain. The simulation's raster plot shows three distinct waves, well separated in time, each corresponding to a burst of $N0$ spikes propagating cleanly along the whole chain, with a diagonal pattern visible to the naked eye.

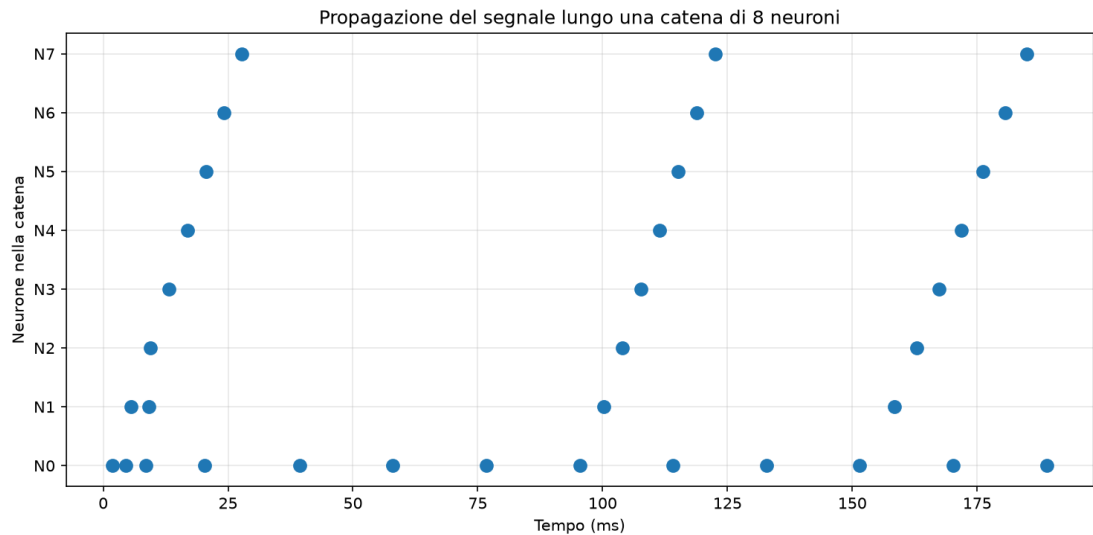


Fig. 4 — Wave propagation along a chain of 8 neurons.

Source code: `rete_catena.py`

```

1 """
2 Catena N0->N1->...->N7, solo N0 stimolato esternamente.
3 Verifica: accumulo del delay lungo la catena, e condizioni sotto cui
4 il segnale si estingue prima di raggiungere N7.
5 """

```

```

6
7 from brian2 import *
8
9 a = 0.02/ms
10 b = 0.2/ms
11 c = -65*mV
12 d = 6*mV/ms
13
14 eqs = '''
15 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
16 du/dt = a*(b*v - u) : volt/second
17 I : volt/second
18 '''
19
20 N = 8
21
22 start_scope()
23
24 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
25                       method='euler')
26 neurons.v = -65*mV
27 neurons.u = b * neurons.v
28
29 neurons.I = 0*mV/ms
30 neurons.I[0] = 20*mV/ms
31
32 # catena i -> i+1
33 syn = Synapses(neurons, neurons, on_pre='v_post += 22*mV')
34 syn.connect(i=arange(N-1), j=arange(1, N))
35 syn.delay = 1.5*ms
36
37 spikes = SpikeMonitor(neurons)
38 run(200*ms)
39
40 figure(figsize=(10, 5))
41 plot(spikes.t/ms, spikes.i, 'o', markersize=8)
42 yticks(range(N), [f'N{i}' for i in range(N)])
43 xlabel('Tempo (ms)')
44 ylabel('Neurone nella catena')
45 title(f'Propagazione del segnale lungo una catena di {N} neuroni')
46 grid(alpha=0.3)
47 tight_layout()
48 savefig('rete_catena.png', dpi=150)
49
50 print(f"Simulazione completata con {N} neuroni in catena.")
51 print(f"Grafico salvato in rete_catena.png\n")
52
53 print(f"{'Neurone':>10} | {'N. spike':>10} | {'Primo spike (ms)':>18}")
54 print("-" * 45)
55 for idx in range(N):
56     spike_times = spikes.t[spikes.i == idx]/ms
57     n_spikes = len(spike_times)
58     first = f"{spike_times[0]:.1f}" if n_spikes > 0 else "mai"
59     print(f"{'N'+str(idx):>10} | {n_spikes:>10} | {first:>18}")
60
61 if len(spikes.t[spikes.i == N-1]) == 0:
62     print(f"\nIl segnale NON e' arrivato fino all'ultimo neurone (N{N-1}).")
63     print("Da alzare l'intensita' sinaptica (22*mV -> es. 26*mV) se serve arrivi in fondo.")
64 else:
65     delay_totale = spikes.t[spikes.i == N-1][0]/ms - spikes.t[spikes.i == 0][0]/ms
66     print(f"\nIl segnale e' arrivato fino a N{N-1}.")
67     print(f"Ritardo totale accumulato dal primo all'ultimo neurone: {delay_totale:.1f} ms")

```

2.5 Temporal summation (convergence)

The fifth experiment reverses the topology: instead of a linear chain, three independent neurons (N0, N1, N2) converge onto a fourth, target neuron (N3). Each of the three source neurons fires exactly once, with the exact same synaptic strength (10 mV) onto N3 in both tested scenarios.

In the "synchronous" scenario, the three spikes arrive at the same instant ($t=10\text{ms}$): N3 receives all the input together, its potential jumps and crosses threshold — N3 fires. In the "staggered" scenario, the same three spikes arrive 5 milliseconds apart from one another ($t=10, 15, 20\text{ms}$): N3's potential rises in small steps but decays between one input and the next, never reaching threshold — N3 never fires, despite having received exactly the same total amount of stimulation.

This demonstrates the principle of temporal summation, or "coincidence detection": a neuron counts not only HOW MANY inputs it receives, but HOW CLOSE TOGETHER they are in time. It is one of the most fundamental computational mechanisms by which a single neuron can act as a detector of correlations among incoming signals.

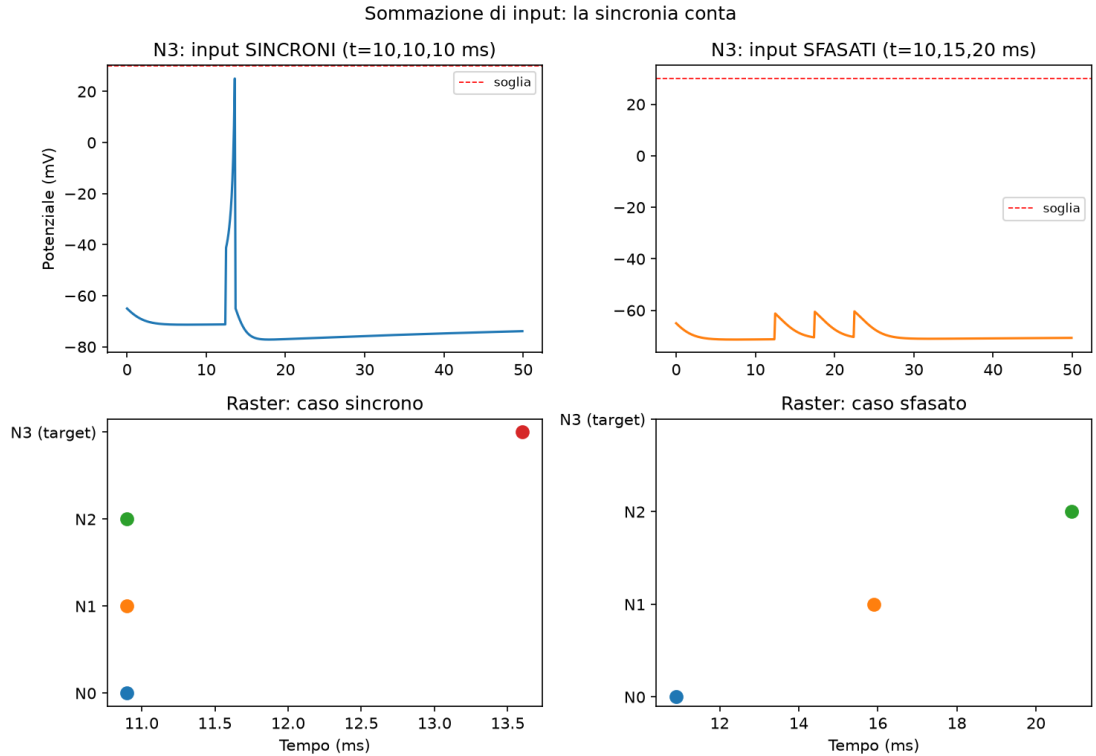


Fig. 5 — Temporal summation: synchronous vs. staggered inputs.

Source code: *rete_convergente.py*

```

1  """
2  Convergenza 3:1 (N0,N1,N2 -> N3): confronto input sincroni vs sfasati,
3  per isolare l'effetto di somministrazione temporale sulla risposta di N3.
4  """
5
6  from brian2 import *
7
8  a = 0.02/ms
9  b = 0.2/ms
10 c = -65*mV
11 d = 6*mV/ms
12
13 eqs = '''
14 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
15 du/dt = a*(b*v - u) : volt/second
16 I : volt/second
17 '''
18
19 def run_convergent(offsets_ms, synapse_strength_mV, label):
20     """N0,N1,N2 sparano una volta ciascuno ai tempi offsets_ms, convergono su N3."""
21     start_scope()
22
23     neurons = NeuronGroup(4, eqs, threshold='v > 30*mV', reset='v = c; u += d',
24                           method='euler')
25     neurons.v = -65*mV
26     neurons.u = b * neurons.v
27     neurons.I = 0*mV/ms
28
29     # SpikeGeneratorGroup per fissare i tempi esatti di N0,N1,N2
30     driver = SpikeGeneratorGroup(3, indices=[0, 1, 2],
31                                 times=[t*ms for t in offsets_ms])
32
33     drive_syn = Synapses(driver, neurons, on_pre='v_post += 35*mV')
34     drive_syn.connect(i=[0, 1, 2], j=[0, 1, 2]) # driver -> N0,N1,N2
35
36     converge_syn = Synapses(neurons, neurons,
37                             on_pre=f'v_post += {synapse_strength_mV}*mV')
38     converge_syn.connect(i=[0, 1, 2], j=[3, 3, 3]) # N0,N1,N2 -> N3
39     converge_syn.delay = 1.5*ms
40
41     spikes = SpikeMonitor(neurons)
42     mon = StateMonitor(neurons, 'v', record=3)
43
44     run(50*ms)
45
46     n_target_spikes = sum(spikes.i == 3)
47     print(f"{label}: N3 ha sparato {n_target_spikes} volte "
48           f"(input a t={offsets_ms} ms)")
49
50     return mon.t/ms, mon.v[0]/mV, spikes.t/ms, spikes.i
51
52
53 t_sync, v_sync, spk_t_sync, spk_i_sync = run_convergent(
54     [10, 10, 10], 10, "SINCRONO")
55
56 t_async, v_async, spk_t_async, spk_i_async = run_convergent(
57     [10, 15, 20], 10, "SFASATO")
58
59 figure(figsize=(10, 7))
60
61 subplot(2, 2, 1)
62 plot(t_sync, v_sync, color='C0')
63 axhline(30, color='red', linestyle='--', linewidth=0.8, label='soglia')
64 title('N3: input SINCRONI (t=10,10,10 ms)')
65 ylabel('Potenziale (mV)')
66 legend(fontsize=8)

```

```

67 subplot(2, 2, 2)
68 plot(t_async, v_async, color='C1')
69 axhline(30, color='red', linestyle='--', linewidth=0.8, label='soglia')
70 title('N3: input SFASATI (t=10,15,20 ms)')
71 legend(fontsize=8)
72
73 subplot(2, 2, 3)
74 for idx in range(4):
75     mask = spk_i_sync == idx
76     plot(spk_t_sync[mask], spk_i_sync[mask], 'o', markersize=8)
77 yticks(range(4), ['N0', 'N1', 'N2', 'N3 (target)'])
78 xlabel('Tempo (ms)')
79 title('Raster: caso sincrono')
80
81 subplot(2, 2, 4)
82 for idx in range(4):
83     mask = spk_i_async == idx
84     plot(spk_t_async[mask], spk_i_async[mask], 'o', markersize=8)
85 yticks(range(4), ['N0', 'N1', 'N2', 'N3 (target)'])
86 xlabel('Tempo (ms)')
87 title('Raster: caso sfasato')
88
89
90 suptitle('Sommazione di input: la sincronia conta')
91 tight_layout()
92 savefig('rete_convergente.png', dpi=150)
93 print("\nGrafico salvato: rete_convergente.png")

```

2.6 STDP — Spike-Timing-Dependent Plasticity

The sixth experiment replaces the simple short-term facilitation of experiment 2.3 with a much more realistic learning rule, and one far more widely used in the computational neuroscience literature: Spike-Timing-Dependent Plasticity (STDP), often summarized by the mnemonic "neurons that fire together, wire together" (the temporal version of Hebb's rule, 1949).

The principle: if the presynaptic neuron (A) systematically fires SHORTLY BEFORE the postsynaptic neuron (B), the synapse $A \rightarrow B$ is STRENGTHENED (potentiation, LTP — Long-Term Potentiation), because A appears to "predict" B, a potentially causal relationship. If instead A fires SHORTLY AFTER B, the synapse is WEAKENED (depression, LTD). The Brian2 implementation uses two exponential trace variables (apre, apost) that accumulate at every spike and decay with a time constant of 20 milliseconds — the standard mechanism for implementing STDP efficiently without having to explicitly compare every pair of past spikes.

Verified over 5 repetitions of a fixed pattern, "A fires 5 milliseconds before B": the synaptic weight rises monotonically, in steps, from an initial value of 5.00 mV to 8.36 mV after the 5 repetitions — a net strengthening of 3.36 mV entirely attributable to the STDP rule.

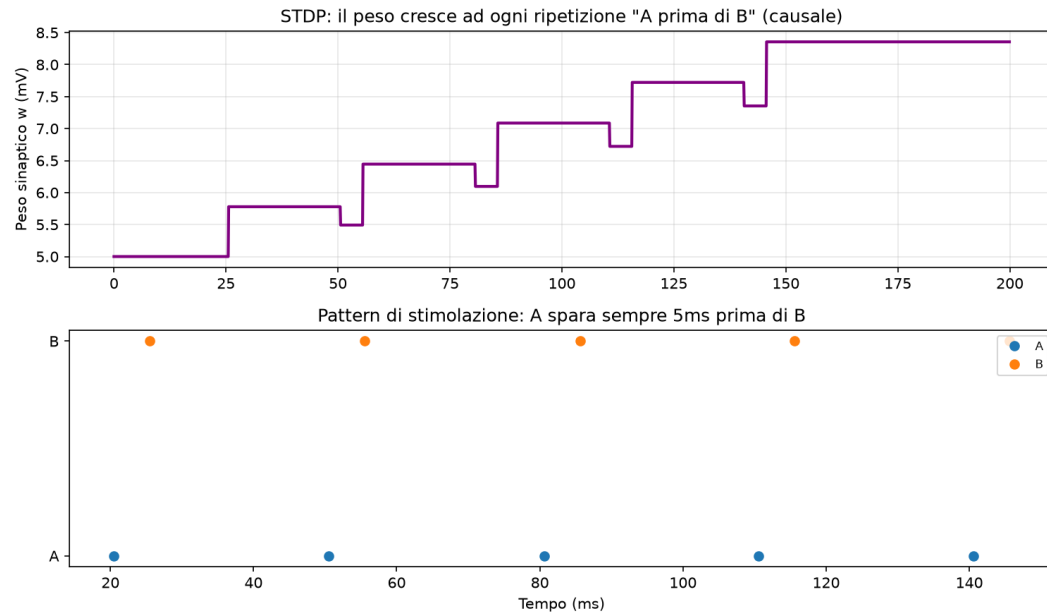


Fig. 6 — Strengthening of the synaptic weight via STDP over causal repetitions.

Source code: *stdp.py*

```

1  """
2  STDP pair-based, traccia esponenziale pre/post (Song & Abbott 2001).
3  Pattern: 5 ripetizioni di A a t, B a t+5ms (A causa B), verifica che w salga
4  monotonicamente. Da invertire manualmente (B prima di A) per il caso LTD.
5  """
6
7  from brian2 import *
8
9  a = 0.02/ms
10 b = 0.2/ms
11 c = -65*mV
12 d = 6*mV/ms
13
14 eqs = '''
15 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
16 du/dt = a*(b*v - u) : volt/second
17 I : volt/second
18 '''
19
20 # parametri STDP
21 tau_stdp = 20*ms
22 A_ltp = 1.0*mV
23 A_ltd = 1.0*mV
24 w_max = 40*mV
25 w_min = 0*mV
26
27 start_scope()
28
29 neurons = NeuronGroup(2, eqs, threshold='v > 30*mV', reset='v = c; u += d',
30                       method='euler')
31 neurons.v = -65*mV
32 neurons.u = b * neurons.v
33 neurons.I = 0*mV/ms

```

```

34 # SpikeGeneratorGroup per fissare i tempi di spike di A/B, indipendenti
35 # dalla dinamica del neurone -- 5 ripetizioni "A a t, B a t+5ms"
36
37 pattern_causale = []
38 pattern_indices = []
39 for rep in range(5):
40     t0 = 20 + rep*30
41     pattern_causale += [t0, t0+5]
42     pattern_indices += [0, 1] # 0 = A, 1 = B
43
44 driver = SpikeGeneratorGroup(2, indices=pattern_indices,
45                               times=[t*ms for t in pattern_causale])
46
47 # forza gli spike di A/B direttamente (impulso fortissimo) per tempi certi
48 force_syn = Synapses(driver, neurons, on_pre='v_post += 50*mV')
49 force_syn.connect(j='i')
50
51 # sinapsi plastica A->B
52 stdp_eqs = '''
53 w : volt
54 dapre/dt = -apre / tau_stdp : volt (event-driven)
55 dapost/dt = -apost / tau_stdp : volt (event-driven)
56 '''
57
58 syn = Synapses(neurons, neurons, model=stdp_eqs,
59               on_pre='''
60                 apre += A_ltp
61                 w = clip(w + apost, w_min, w_max)
62                 ''',
63               on_post='''
64                 apost -= A_ltd
65                 w = clip(w + apre, w_min, w_max)
66                 ''',
67               method='euler')
68 syn.connect(i=0, j=1)
69 syn.w = 5*mV
70 initial_w = syn.w[0]
71
72 w_mon = StateMonitor(syn, 'w', record=0)
73 spikes = SpikeMonitor(neurons)
74
75 run(200*ms)
76
77 figure(figsize=(10, 6))
78
79 subplot(2, 1, 1)
80 plot(w_mon.t/ms, w_mon.w[0]/mV, color='purple', linewidth=2)
81 ylabel('Peso sinaptico w (mV)')
82 title('STDP: il peso cresce ad ogni ripetizione "A prima di B" (causale)')
83 grid(alpha=0.3)
84
85 subplot(2, 1, 2)
86 plot(spikes.t[spikes.i == 0]/ms, [0]*sum(spikes.i == 0), 'o', color='C0', label='A')
87 plot(spikes.t[spikes.i == 1]/ms, [1]*sum(spikes.i == 1), 'o', color='C1', label='B')
88 yticks([0, 1], ['A', 'B'])
89 xlabel('Tempo (ms)')
90 title('Pattern di stimolazione: A spara sempre 5ms prima di B')
91 legend(loc='upper right', fontsize=8)
92
93 tight_layout()
94 savefig('stdp.png', dpi=150)
95
96 print("Simulazione completata. Grafico salvato in stdp.png")
97 print(f"Peso sinaptico iniziale: {initial_w/mV:.2f} mV")
98 print(f"Peso sinaptico finale: {w_mon.w[0][-1]/mV:.2f} mV")
99 print(f"Rafforzamento totale dopo 5 ripetizioni 'A causa B': "
100       f"{(w_mon.w[0][-1] - initial_w)/mV:.2f} mV")
101 print("\nCaso LTD (B prima di A) da testare invertendo l'ordine nel pattern.")

```


2.7 Pattern classification via STDP

The seventh experiment is the project's first major conceptual leap: no longer just observing how a SINGLE synaptic weight changes, but checking whether an entire small circuit can LEARN to distinguish two different input patterns, without anyone explicitly writing the rule for telling them apart.

Structure: 6 input neurons, split into two groups of 3 (Group A and Group B, representing two distinct "patterns"). An output neuron O receives from all 6 via plastic STDP synapses, with initial weights all equal and low (6 mV). During training (10 repetitions), every time Pattern A is presented, a small external "teacher" forces O to fire right afterward — like a teacher guiding the correct response. When Pattern B is presented, no external help is given.

In the TEST phase, with no external help at all: Pattern A presented alone makes O fire; Pattern B presented alone does NOT make it fire. The final weights confirm the mechanism: Group A's synapses have risen to saturate at the maximum allowed value (15.00 mV), while Group B's have stayed close to their initial value, if anything slightly weakened (5.67 mV). The discriminative behavior — responding selectively to one pattern and not the other — EMERGED from the STDP rule applied repeatedly; it was never written explicitly anywhere in the code as an "if pattern == A" condition.

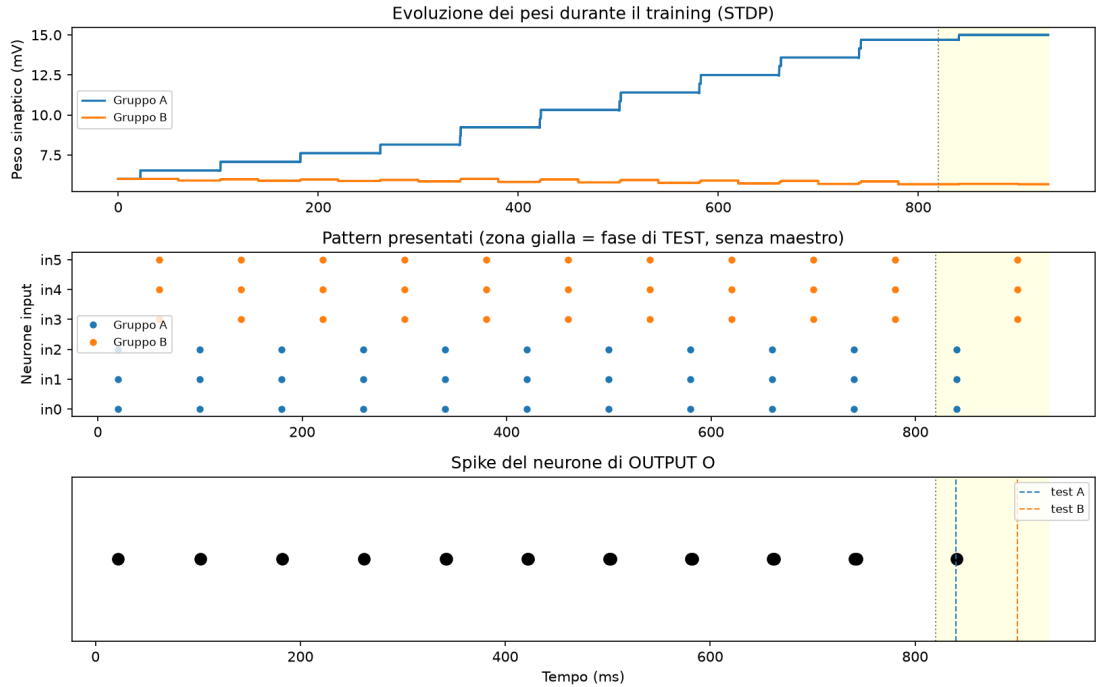


Fig. 7 — Selective learning: response to Pattern A only, after training.

Source code: *classificazione_stdp.py*

```

1  """
2  Classificazione a 1 neurone via STDP supervisionato (teacher signal).
3  6 input (Gruppo A=0,1,2 / Gruppo B=3,4,5) -> 1 output 0, sinapsi plastiche.
4  Training: 10 rep, teacher forza 0 a sparare solo dopo il Pattern A.
5  Test: nessun teacher -- verifica se 0 risponde selettivamente ad A.
6  """
7
8  from brian2 import *
9
10 # Izhikevich RS, come negli script precedenti
11 a = 0.02/ms
12 b = 0.2/ms
13 c = -65*mV
14 d = 6*mV/ms
15
16 eqs = '''
17 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
18 du/dt = a*(b*v - u) : volt/second
19 I : volt/second
20 '''
21
22 # parametri STDP
23 tau_stdp = 20*ms
24 A_ltp = 0.6*mV
25 A_ltd = 0.6*mV
26 w_max = 15*mV
27 w_min = 0*mV
28
29 start_scope()
30
31 out = NeuronGroup(1, eqs, threshold='v > 30*mV', reset='v = c; u += d',
32                  method='euler')
33 out.v = -65*mV
34 out.u = b * out.v[:]
35 out.I = 0*mV/ms
36
37 # protocollo: 10 rep training (A+teacher, B senza), poi test
38 indices, times, teacher_times = [], [], []
39
40 t = 20
41 N_TRAIN_REPS = 10
42 for rep in range(N_TRAIN_REPS):
43     for k in [0, 1, 2]: # Pattern A
44         indices.append(k); times.append(t)
45         teacher_times.append(t + 2) # il "maestro" fa sparare 0
46         t += 40
47     for k in [3, 4, 5]: # Pattern B (nessun aiuto)
48         indices.append(k); times.append(t)
49         t += 40
50
51 train_end = t
52
53 # Fase di TEST: pattern A da solo, poi pattern B da solo, senza maestro
54 test_A_time = train_end + 20
55 test_B_time = train_end + 80
56 for k in [0, 1, 2]:
57     indices.append(k); times.append(test_A_time)
58 for k in [3, 4, 5]:
59     indices.append(k); times.append(test_B_time)
60
61 total_duration = test_B_time + 30
62
63 driver = SpikeGeneratorGroup(6, indices=indices, times=[x*ms for x in times])
64 teacher = SpikeGeneratorGroup(1, indices=[0]*len(teacher_times),
65                               times=[x*ms for x in teacher_times])
66

```

```

67 # il teacher forza lo spike di 0 solo durante il training di A
68 teacher_syn = Synapses(teacher, out, on_pre='v_post += 50*mV')
69 teacher_syn.connect()
70
71 stdp_eqs = '''
72 w : volt
73 dapre/dt = -apre / tau_stdp : volt (event-driven)
74 dapost/dt = -apost / tau_stdp : volt (event-driven)
75 '''
76 syn = Synapses(driver, out, model=stdp_eqs,
77               on_pre='''
78                 v_post += w
79                 apre += A_ltp
80                 w = clip(w + apost, w_min, w_max)
81                 ''',
82               on_post='''
83                 apost -= A_ltd
84                 w = clip(w + apre, w_min, w_max)
85                 ''',
86               method='euler')
87 syn.connect(i=range(6), j=0)
88 syn.w = 6*mV # pesi iniziali uguali e bassi
89
90 w_mon = StateMonitor(syn, 'w', record=range(6))
91 spikes = SpikeMonitor(out)
92 driver_spikes = SpikeMonitor(driver)
93
94 run(total_duration*ms)
95
96 print("=" * 55)
97 print("PESI SINAPTICI FINALI (dopo il training)")
98 print("=" * 55)
99 for k in range(6):
100     grp = "A" if k < 3 else "B"
101     print(f" input {k} (gruppo {grp}): {syn.w[k]/mV:.2f} mV")
102
103 spikes_near_A = sum((spikes.t/ms > test_A_time) & (spikes.t/ms < test_A_time+15))
104 spikes_near_B = sum((spikes.t/ms > test_B_time) & (spikes.t/ms < test_B_time+15))
105
106 print("\n" + "=" * 55)
107 print("RISULTATO DEL TEST (nessun aiuto esterno)")
108 print("=" * 55)
109 print(f"Pattern A presentato da solo a t={test_A_time}ms -> "
110       f"0 ha sparato: {'SI' if spikes_near_A > 0 else 'NO'}")
111 print(f"Pattern B presentato da solo a t={test_B_time}ms -> "
112       f"0 ha sparato: {'SI' if spikes_near_B > 0 else 'NO'}")
113
114 if spikes_near_A > 0 and spikes_near_B == 0:
115     print("\n>>> Il neurone ha imparato a riconoscere selettivamente il Pattern A. <<<")
116
117 figure(figsize=(11, 7))
118
119 subplot(3, 1, 1)
120 colors = ['C0', 'C0', 'C0', 'C1', 'C1', 'C1']
121 for k in range(6):
122     plot(w_mon.t/ms, w_mon.w[k]/mV, color=colors[k],
123          label=f'Gruppo A' if k == 0 else (f'Gruppo B' if k == 3 else None))
124 axvline(train_end, color='gray', linestyle=':', linewidth=1)
125 axvspan(train_end, total_duration, color='yellow', alpha=0.1)
126 ylabel('Peso sinaptico (mV)')
127 title('Evoluzione dei pesi durante il training (STDP)')
128 legend(loc='center left', fontsize=8)
129
130 subplot(3, 1, 2)
131 plot(driver_spikes.t[driver_spikes.i < 3]/ms,
132      driver_spikes.i[driver_spikes.i < 3], 'o', color='C0', markersize=4, label='Gruppo A')
133 plot(driver_spikes.t[driver_spikes.i >= 3]/ms,
134      driver_spikes.i[driver_spikes.i >= 3], 'o', color='C1', markersize=4, label='Gruppo B')

```

```

135 axvline(train_end, color='gray', linestyle=':', linewidth=1)
136 axvspan(train_end, total_duration, color='yellow', alpha=0.1)
137 yticks(range(6), [f'in{i}' for i in range(6)])
138 ylabel('Neurone input')
139 legend(loc='center left', fontsize=8)
140 title('Pattern presentati (zona gialla = fase di TEST, senza maestro)')
141
142 subplot(3, 1, 3)
143 plot(spikes.t/ms, [0]*len(spikes.t), 'ko', markersize=8)
144 axvline(train_end, color='gray', linestyle=':', linewidth=1)
145 axvspan(train_end, total_duration, color='yellow', alpha=0.1)
146 axvline(test_A_time, color='C0', linestyle='--', linewidth=1, label='test A')
147 axvline(test_B_time, color='C1', linestyle='--', linewidth=1, label='test B')
148 yticks([])
149 xlabel('Tempo (ms)')
150 title('Spike del neurone di OUTPUT 0')
151 legend(loc='upper right', fontsize=8)
152
153 tight_layout()
154 savefig('classificazione_stdp.png', dpi=150)
155 print("\nGrafico salvato in classificazione_stdp.png")

```

2.8 Balanced excitation-inhibition network (300 neurons)

The eighth experiment is the project's first real jump in scale: from a handful of neurons to 300, organized according to the standard architecture from the computational neuroscience literature for networks of this size — the "balanced" model inspired by the work of Nicolas Brunel (2000).

The network is made of 240 excitatory neurons (80%, which push target neurons toward spiking) and 60 inhibitory neurons (20%, which suppress activity), with sparse, random 10% connectivity (each pair of neurons has a 10% probability of being connected, independently of every other pair). A preliminary attempt with excitatory synapses only (not shown in detail in this document, but verified during development) makes the network either explode into a synchronized discharge of the whole population, or shut down entirely — neither behavior is interesting or realistic.

Introducing inhibition in the correct proportion instead produces a state of activity known as "asynchronous irregular": neurons fire at an average frequency of about 15.6 Hz, in a statistically structured way but without all synchronizing together — qualitatively similar to the activity observed in real cerebral cortex during wakefulness.

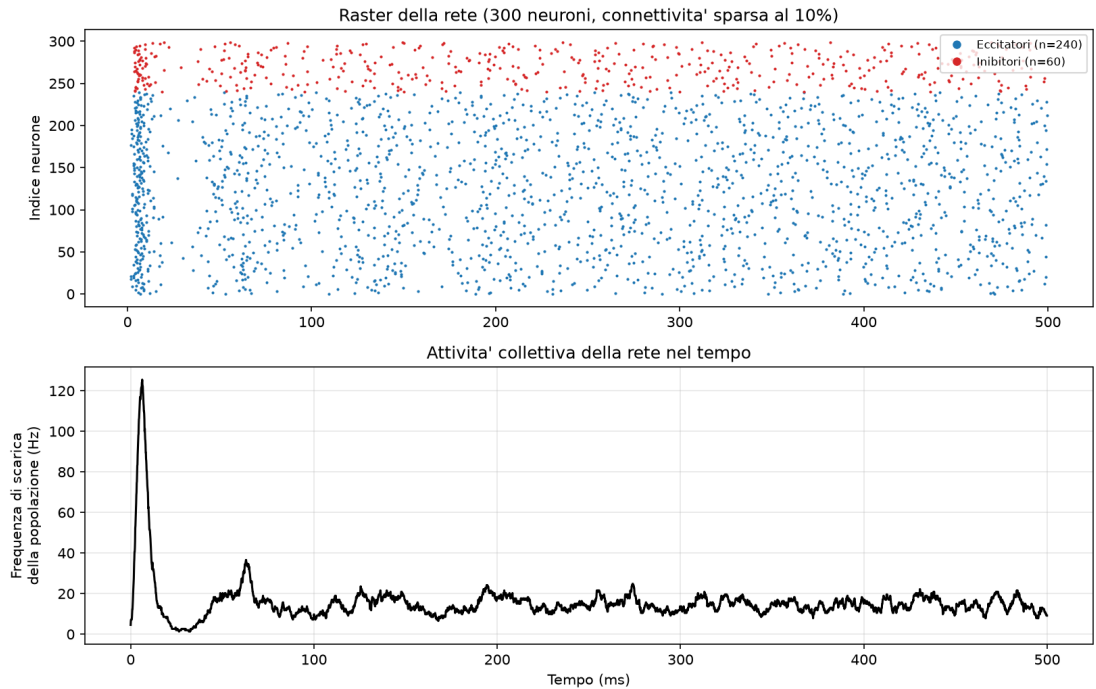


Fig. 8 — Raster and collective frequency of the 300-neuron network.

Source code: `rete_300_neuroni.py`

```

1  """
2  Rete E/I bilanciata, 300 neuroni, stile Brunel: 80/20 E/I, connettività
3  sparsa casuale p=0.1, w_inh > w_exc per tenere il regime asincrono-irregolare
4  senza saturare o spegnersi. Rumore di fondo Poisson su tutti i neuroni.
5  """
6
7  from brian2 import *
8
9  start_scope()
10
11  N_E = 240
12  N_I = 60
13  N = N_E + N_I
14
15  a = 0.02/ms
16  b = 0.2/ms
17  c = -65*mV
18  d = 6*mV/ms
19
20  eqs = '''
21  dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
22  du/dt = a*(b*v - u) : volt/second
23  I : volt/second
24  '''
25
26  neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
27                        method='euler')
28  neurons.v = -65*mV + 5*mV*randn(N) # rompe simmetrie iniziali
29  neurons.u = b * neurons.v[:]
30  neurons.I = 0*mV/ms

```

```

31 E = neurons[:N_E]
32 I_pop = neurons[N_E:]
33
34 background = PoissonGroup(N, rates=1500*Hz)
35 bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
36 bg_syn.connect(j='i')
37
38
39 p_conn = 0.1
40 w_exc = 1.0*mV
41 w_inh = 4.0*mV
42
43 syn_ee = Synapses(E, E, on_pre='v_post += w_exc')
44 syn_ee.connect(condition='i!=j', p=p_conn)
45
46 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
47 syn_ei.connect(p=p_conn)
48
49 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
50 syn_ie.connect(p=p_conn)
51
52 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
53 syn_ii.connect(condition='i!=j', p=p_conn)
54
55 spikes = SpikeMonitor(neurons)
56 rate_mon = PopulationRateMonitor(neurons)
57
58 run(500*ms)
59
60 n_spikes_total = sum(spikes.count)
61 rate_medio = n_spikes_total / N / 0.5 # Hz su 500ms
62
63 spikes_E = sum(spikes.count[:N_E])
64 spikes_I = sum(spikes.count[N_E:])
65 rate_E = spikes_E / N_E / 0.5
66 rate_I = spikes_I / N_I / 0.5
67
68 print("=" * 55)
69 print(f"Rete: {N_E} eccitatori + {N_I} inibitori = {N} neuroni totali")
70 print(f"Connettività: {p_conn*100:.0f}% (sparsa e casuale)")
71 print("=" * 55)
72 print(f"Spike totali in 500ms: {n_spikes_total}")
73 print(f"Frequenza media della rete: {rate_medio:.2f} Hz")
74 print(f"Frequenza media eccitatori: {rate_E:.2f} Hz")
75 print(f"Frequenza media inibitori: {rate_I:.2f} Hz")
76
77 figure(figsize=(11, 7))
78
79 subplot(2, 1, 1)
80 exc_mask = spikes.i < N_E
81 plot(spikes.t[exc_mask]/ms, spikes.i[exc_mask], '.', color='C0',
82      markersize=2, label=f'Eccitatori (n={N_E})')
83 plot(spikes.t[-exc_mask]/ms, spikes.i[-exc_mask], '.', color='C3',
84      markersize=2, label=f'Inibitori (n={N_I})')
85 ylabel('Indice neurone')
86 title(f'Raster della rete ({N} neuroni, connettività sparsa al {p_conn*100:.0f}%)')
87 legend(loc='upper right', markerscale=5, fontsize=8)
88
89 subplot(2, 1, 2)
90 plot(rate_mon.t/ms, rate_mon.smooth_rate(window='flat', width=5*ms)/Hz,
91      color='black')
92 xlabel('Tempo (ms)')
93 ylabel('Frequenza di scarica della popolazione (Hz)')
94 title('Attività collettiva della rete nel tempo')
95 grid(alpha=0.3)
96
97 tight_layout()
98 savefig('rete_300_neuroni.png', dpi=150)

```

99 | `print("\nGrafico salvato in rete_300_neuroni.png")`

2.9 Response to a stimulus and homeostatic recovery

The ninth experiment checks how the 300-neuron network, already stable in its spontaneous-activity state, responds to an external perturbation. A strong current pulse is applied to just 20 out of 300 neurons (6.7% of the network) for 20 milliseconds, then removed entirely.

The network's collective frequency jumps sharply from about 14.9 Hz (baseline) to a peak of about 68.6 Hz during and right after the stimulus, then spontaneously returns to the starting level (15.6 Hz) with no external intervention. An interesting detail, clearly visible in the plot: after the peak, the frequency does not glide gently back to baseline but briefly dips **BELOW** the starting level (an inhibitory "undershoot") before restabilizing — a behavior typical of negative-feedback systems, where the inhibitory neurons activated by the excitatory wave stay active a moment longer than strictly necessary.

This behavior — a proportionate response to a stimulus, followed by a spontaneous return to equilibrium — is the project's first appearance of something resembling HOMEOSTASIS, a fundamental property of any functioning nervous system: the ability to respond without staying "damaged" or entering a pathological regime.

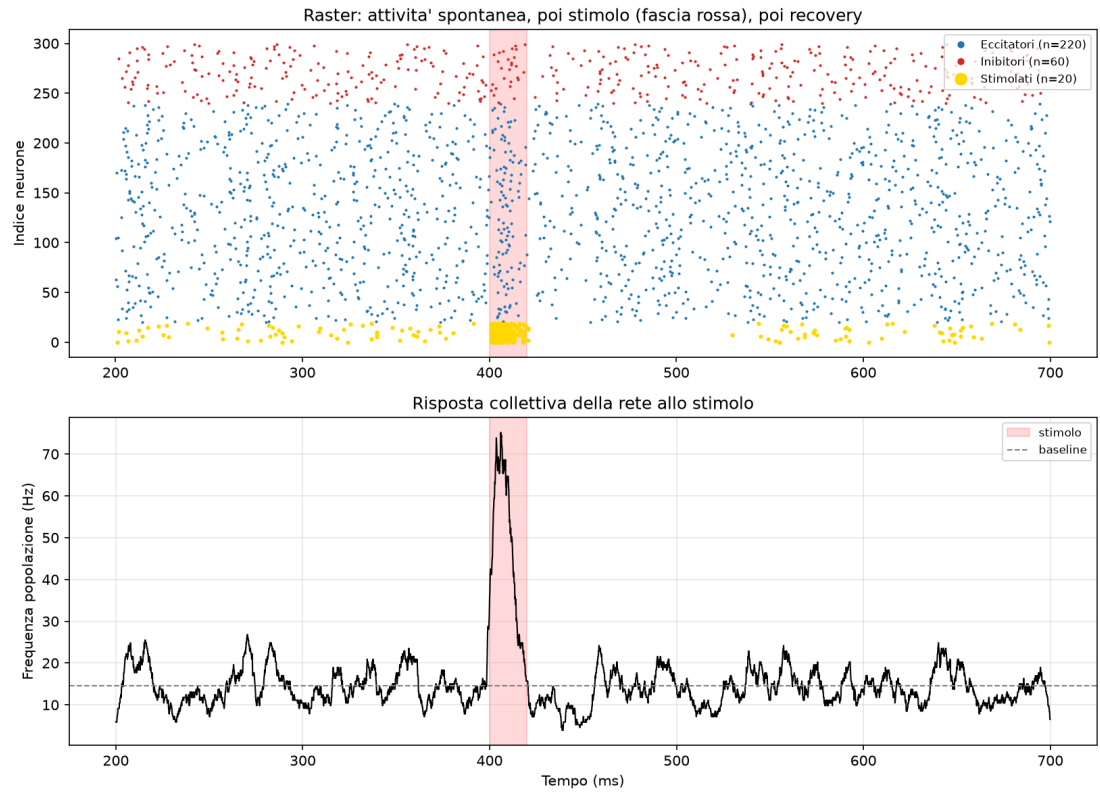


Fig. 9 — Homeostatic response and recovery after a targeted stimulus.

Source code: `rete_300_stimolo.py`

```

1  """
2  Rete E/I a 300 neuroni con protocollo burn-in / baseline / stimolo / recovery.
3  Burn-in scarta il transitorio sincronizzato iniziale; poi impulso su 20
4  eccitatori e verifica se la frequenza di popolazione torna al baseline.
5  """
6
7  from brian2 import *
8
9  start_scope()
10
11  N_E = 240
12  N_I = 60
13  N = N_E + N_I
14
15  a = 0.02/ms
16  b = 0.2/ms
17  c = -65*mV
18  d = 6*mV/ms
19
20  eqs = '''
21  dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
22  du/dt = a*(b*v - u) : volt/second
23  I : volt/second
24  '''
25

```



```

26 | neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
27 |                       method='euler')
28 | neurons.v = -65*mV + 15*mV*rand(N) - 5*mV # dispersione iniziale piu' ampia
29 | neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
30 | neurons.I = 0*mV/ms
31 |
32 | E = neurons[:N_E]
33 | I_pop = neurons[N_E:]
34 |
35 | background = PoissonGroup(N, rates=1500*Hz)
36 | bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
37 | bg_syn.connect(j='i')
38 |
39 | p_conn = 0.1
40 | w_exc = 1.0*mV
41 | w_inh = 4.0*mV
42 |
43 | syn_ee = Synapses(E, E, on_pre='v_post += w_exc')
44 | syn_ee.connect(condition='i!=j', p=p_conn)
45 | syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
46 | syn_ei.connect(p=p_conn)
47 | syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
48 | syn_ie.connect(p=p_conn)
49 | syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
50 | syn_ii.connect(condition='i!=j', p=p_conn)
51 |
52 | # burn-in: 200ms senza monitor, scarta il transitorio iniziale
53 | run(200*ms)
54 | print("Burn-in completato (200ms, non registrato).")
55 |
56 | # baseline: monitor collegati, 200ms di attivita' spontanea pre-stimolo
57 | rate_mon = PopulationRateMonitor(neurons)
58 | spikes = SpikeMonitor(neurons)
59 |
60 | run(200*ms)
61 | print("Baseline registrata (200ms).")
62 |
63 | # stimolo: impulso di 20ms su 20 eccitatori (su 300)
64 | N_STIM = 20
65 | stim_group = E[:N_STIM]
66 | stim_group.I = 40*mV/ms
67 | stim_time = 400*ms # 200 burn-in + 200 baseline
68 |
69 | run(20*ms)
70 | stim_group.I = 0*mV/ms
71 | print(f"Stimolo applicato a {N_STIM} neuroni eccitatori (20ms di impulso).")
72 |
73 | # recovery
74 | run(280*ms)
75 | print("Fase di recovery completata.")
76 |
77 | r = array(rate_mon.smooth_rate(window='flat', width=5*ms)/Hz)
78 | t = array(rate_mon.t/ms)
79 |
80 | baseline_rate = r[(t > 200) & (t < 400)].mean()
81 | peak_rate = r[(t >= 400) & (t < 450)].max()
82 | recovery_rate = r[t > 650].mean()
83 |
84 | print("\n" + "=" * 55)
85 | print("RISULTATI")
86 | print("=" * 55)
87 | print(f"Frequenza baseline (prima dello stimolo): {baseline_rate:.2f} Hz")
88 | print(f"Picco di frequenza dopo lo stimolo: {peak_rate:.2f} Hz")
89 | print(f"Frequenza finale (dopo il recovery): {recovery_rate:.2f} Hz")
90 | if abs(recovery_rate - baseline_rate) < 3:
91 |     print(">>> La rete e' tornata al livello di attivita' di partenza. <<<")
92 |
93 | figure(figsize=(11, 8))

```

```

94 subplot(2, 1, 1)
95 exc_mask = spikes.i < N_E
96 stim_mask = spikes.i < N_STIM # sottoinsieme stimolato di E
97
98
99 plot(spikes.t[exc_mask & ~stim_mask]/ms, spikes.i[exc_mask & ~stim_mask], '.',
100      color='C0', markersize=2, label=f'Eccitatori (n={N_E-N_STIM})')
101 plot(spikes.t[~exc_mask]/ms, spikes.i[~exc_mask], '.',
102      color='C3', markersize=2, label=f'Inibitori (n={N_I})')
103 plot(spikes.t[stim_mask]/ms, spikes.i[stim_mask], '.',
104      color='gold', markersize=4, label=f'Stimolati (n={N_STIM})')
105
106 axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15)
107 ylabel('Indice neurone')
108 title('Raster: attivita\' spontanea, poi stimolo (fascia rossa), poi recovery')
109 legend(loc='upper right', markerscale=4, fontsize=8)
110
111 subplot(2, 1, 2)
112 plot(t, r, color='black', linewidth=1)
113 axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15, label='stimolo')
114 axhline(baseline_rate, color='gray', linestyle='--', linewidth=1, label='baseline')
115 xlabel('Tempo (ms)')
116 ylabel('Frequenza popolazione (Hz)')
117 title('Risposta collettiva della rete allo stimolo')
118 legend(loc='upper right', fontsize=8)
119 grid(alpha=0.3)
120
121 tight_layout()
122 savefig('rete_300_stimolo.png', dpi=150)
123 print("\nGrafico salvato in rete_300_stimolo.png")

```

2.10 Modular architecture: two connected regions

The tenth experiment introduces, for the first time in the project, the concept of distinct REGIONS, rather than a single undifferentiated mass of neurons. The 300-neuron network is split into two modules of 150 each (identical internal E/I structure, dense 10% connectivity as before), but connected to each other only weakly: a 2% connection probability, and ONLY excitatory synapses — a choice that mirrors how long-range cortico-cortical connections actually work in the real brain, typically excitatory and relatively sparse compared to the connection density within a single area.

Stimulating ONLY Module 1 (20 neurons, as in experiment 2.9), a single trial shows an apparent increase in activity even in Module 2, which received no direct stimulus: +88 Hz in Module 1 (directly stimulated) and +13 Hz in Module 2 (attenuated but apparently present). At first glance, this would seem to demonstrate that the signal propagates between the two regions through the few connections linking them.



Fig. 10 — Apparent propagation between modules (single trial).

Source code: `due_moduli.py`

```

1  """
2  Due moduli da 150 neuroni (120E+30I ciascuno), connettivita' intra-modulo
3  densa (10%) e inter-modulo debole (2%, solo E->E) -- analogo alle connessioni
4  a lungo raggio della corteccia. Stimolo solo su Modulo 1, verifica se e quanto
5  si propaga al Modulo 2 non stimolato.
6  """
7
8  from brian2 import *
9
10 start_scope()
11
12 N_E_mod = 120
13 N_I_mod = 30
14 N_mod = N_E_mod + N_I_mod # 150 per modulo
15 N = N_mod * 2 # 300 totali
16
17 a = 0.02/ms
18 b = 0.2/ms
19 c = -65*mV
20 d = 6*mV/ms
21
22 eqs = '''
23 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
24 du/dt = a*(b*v - u) : volt/second
25 I : volt/second

```

```

26  '''
27
28  neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
29                      method='euler')
30  neurons.v = -65*mV + 15*mV*rand(N) - 5*mV
31  neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
32  neurons.I = 0*mV/ms
33
34  # Modulo 1 = [0:150], Modulo 2 = [150:300]; dentro ogni modulo E poi I
35  M1_E = neurons[0:120]
36  M1_I = neurons[120:150]
37  M2_E = neurons[150:270]
38  M2_I = neurons[270:300]
39
40  background = PoissonGroup(N, rates=1500*Hz)
41  bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
42  bg_syn.connect(j='i')
43
44  # connettivita' intra-modulo, densa
45  p_local = 0.1
46  w_exc = 1.0*mV
47  w_inh = 4.0*mV
48
49  def connect_module(E, I):
50      see = Synapses(E, E, on_pre='v_post += w_exc')
51      see.connect(condition='i!=j', p=p_local)
52      sei = Synapses(E, I, on_pre='v_post += w_exc')
53      sei.connect(p=p_local)
54      sie = Synapses(I, E, on_pre='v_post -= w_inh')
55      sie.connect(p=p_local)
56      sii = Synapses(I, I, on_pre='v_post -= w_inh')
57      sii.connect(condition='i!=j', p=p_local)
58      return see, sei, sie, sii
59
60  syn1 = connect_module(M1_E, M1_I)
61  syn2 = connect_module(M2_E, M2_I)
62
63  # connettivita' inter-modulo, debole e solo eccitatoria
64  p_inter = 0.02
65  w_inter = 1.0*mV
66
67  syn_12 = Synapses(M1_E, M2_E, on_pre='v_post += w_inter')
68  syn_12.connect(p=p_inter)
69  syn_21 = Synapses(M2_E, M1_E, on_pre='v_post += w_inter')
70  syn_21.connect(p=p_inter)
71
72  run(200*ms)
73  print("Burn-in completato.")
74
75  rate_mon1 = PopulationRateMonitor(neurons[0:150]) # Modulo 1
76  rate_mon2 = PopulationRateMonitor(neurons[150:300]) # Modulo 2
77  spikes = SpikeMonitor(neurons)
78
79  run(200*ms)
80  print("Baseline registrata.")
81
82  # stimolo: 20 neuroni eccitatori del solo Modulo 1
83  N_STIM = 20
84  stim_group = M1_E[:N_STIM]
85  stim_time = 400*ms
86  stim_group.I = 40*mV/ms
87  run(20*ms)
88  stim_group.I = 0*mV/ms
89  print(f"Stimolo applicato a {N_STIM} neuroni del Modulo 1 (il Modulo 2 non riceve nulla).")
90
91  run(280*ms)
92  print("Recovery completato.")
93

```

```

94 r1 = array(rate_mon1.smooth_rate(window='flat', width=5*ms)/Hz)
95 r2 = array(rate_mon2.smooth_rate(window='flat', width=5*ms)/Hz)
96 t = array(rate_mon1.t/ms)
97
98 base1 = r1[(t > 200) & (t < 400)].mean()
99 base2 = r2[(t > 200) & (t < 400)].mean()
100 peak1 = r1[(t >= 400) & (t < 450)].max()
101 peak2 = r2[(t >= 400) & (t < 450)].max()
102
103 print("\n" + "=" * 55)
104 print("RISULTATI")
105 print("=" * 55)
106 print(f"MODULO 1 (stimolato direttamente):")
107 print(f" baseline: {base1:.2f} Hz -> picco: {peak1:.2f} Hz "
108       f"({peak1-base1:.2f} Hz)")
109 print(f"MODULO 2 (NON stimolato direttamente):")
110 print(f" baseline: {base2:.2f} Hz -> picco: {peak2:.2f} Hz "
111       f"({peak2-base2:.2f} Hz)")
112 if peak2 - base2 > 3:
113     print("\n>>> Il segnale si e' propagato al Modulo 2, "
114           "anche se attenuato rispetto al Modulo 1. <<<")
115 else:
116     print("\n>>> Il segnale non si e' propagato in modo significativo "
117           "al Modulo 2. <<<")
118
119 figure(figsize=(11, 8))
120
121 subplot(2, 1, 1)
122 mod1_mask = spikes.i < 150
123 plot(spikes.t[mod1_mask]/ms, spikes.i[mod1_mask], '.', color='C0',
124       markersize=2, label='Modulo 1 (stimolato)')
125 plot(spikes.t[~mod1_mask]/ms, spikes.i[~mod1_mask], '.', color='C2',
126       markersize=2, label='Modulo 2 (non stimolato)')
127 axhline(150, color='black', linewidth=0.8, linestyle='-')
128 axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15)
129 ylabel('Indice neurone')
130 title('Raster: i due moduli (linea nera = confine tra Modulo 1 e Modulo 2)')
131 legend(loc='upper right', markerscale=4, fontsize=8)
132
133 subplot(2, 1, 2)
134 plot(t, r1, color='C0', label='Modulo 1 (stimolato)', linewidth=1.2)
135 plot(t, r2, color='C2', label='Modulo 2 (non stimolato)', linewidth=1.2)
136 axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15, label='stimolo')
137 xlabel('Tempo (ms)')
138 ylabel('Frequenza popolazione (Hz)')
139 title('Confronto della risposta: propagazione del segnale tra moduli')
140 legend(loc='upper right', fontsize=8)
141 grid(alpha=0.3)
142
143 tight_layout()
144 savefig('due_moduli.png', dpi=150)
145 print("\nGrafico salvato in due_moduli.png")

```

2.11 Statistical verification: is the observed effect real or noise?

The eleventh experiment is, from a methodological standpoint, probably the most important in the entire project — not for the positive result it produces, but for what it **TEACHES** about the scientific rigor needed before drawing conclusions.

Looking closely at the plot from experiment 2.10, the small increase in Mod-

ule 2 activity (+13 Hz) turned out to be comparable in size to the network's normal spontaneous fluctuations, visible elsewhere in the same plot even with no stimulus underway. A single trial does not allow one to distinguish a real propagation effect from a statistical coincidence.

The solution adopted is the standard technique used in experimental neuroscience for exactly this problem: the PSTH (Peristimulus Time Histogram). The identical experiment is repeated 25 times, each trial is aligned to the exact moment of the stimulus, and the AVERAGE is computed across all trials — if a real effect exists, it clearly emerges above the noise in the average (which tends to cancel out when averaging over many independent repetitions); if no real effect exists, the average stays flat.

Result of the check: the average effect measured in Module 2, averaged over 25 repetitions, came out to just +0.89 Hz — clearly within the background noise (the standard deviation of a single trial, measured over the same time window, is about 4 Hz, nearly 5 times larger than the observed average effect).

Honest conclusion: with these connectivity parameters (2% probability, excitatory synapses only), it is NOT possible to confidently claim that the signal propagates meaningfully between the two modules. What looked like a clear effect in the single trial was, in all likelihood, largely random statistical noise.

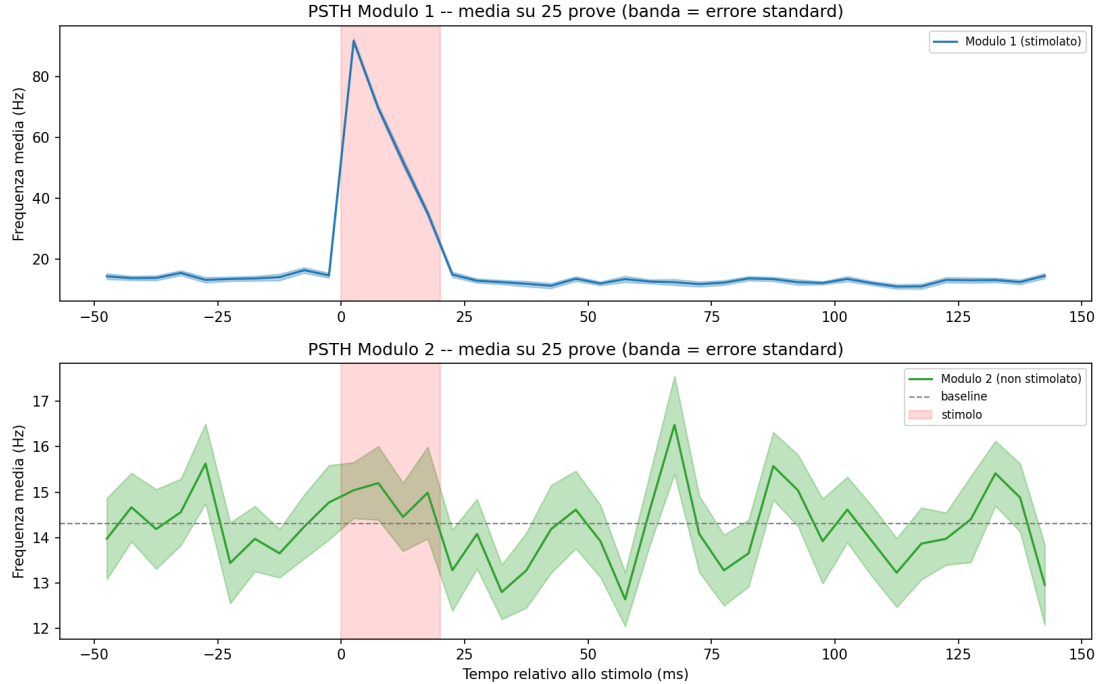


Fig. 11 — PSTH over 25 trials: the average effect falls within the noise.

This negative result, obtained with the correct statistical tool, is probably the most solid contribution of the entire project from the standpoint of scientific method: it shows the concrete difference between a result that SEEMS interesting at first glance and a result VERIFIED with the appropriate statistical tools — a distinction that recurs, in different forms, later in the document as well (Section 4.2, Section 5.3).

Source code: *psth_moduli.py*

```

1  """
2  PSTH su 25 ripetizioni dello stimolo al Modulo 1, per capire se la risposta
3  osservata del Modulo 2 (script precedente, singola prova) e' un effetto reale
4  o rientra nella variabilita' spontanea della rete. Media + SEM allineate
5  allo stimolo, finestra [-50,+150]ms, confronto con la deviazione standard
6  di una singola prova.
7  """
8
9  from brian2 import *
10 import numpy as np
11
12 start_scope()
13
14 # struttura identica allo script dei due moduli
15 N_E_mod = 120
16 N_I_mod = 30
17 N_mod = N_E_mod + N_I_mod
18 N = N_mod * 2
19
20 a = 0.02/ms
21 b = 0.2/ms
22 c = -65*mV
23 d = 6*mV/ms
24
25 eqs = '''
26 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
27 du/dt = a*(b*v - u) : volt/second
28 I : volt/second
29 '''
30
31 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
32                       method='euler')
33 neurons.v = -65*mV + 15*mV*rand(N) - 5*mV
34 neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
35 neurons.I = 0*mV/ms
36
37 M1_E = neurons[0:120]; M1_I = neurons[120:150]
38 M2_E = neurons[150:270]; M2_I = neurons[270:300]
39
40 background = PoissonGroup(N, rates=1500*Hz)
41 bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
42 bg_syn.connect(j='i')
43
44 p_local = 0.1
45 w_exc = 1.0*mV
46 w_inh = 4.0*mV
47
48 def connect_module(E, I):
49     see = Synapses(E, E, on_pre='v_post += w_exc')
50     see.connect(condition='i!=j', p=p_local)
51     sei = Synapses(E, I, on_pre='v_post += w_exc')
52     sei.connect(p=p_local)
53     sie = Synapses(I, E, on_pre='v_post -= w_inh')
54     sie.connect(p=p_local)
55     sii = Synapses(I, I, on_pre='v_post -= w_inh')
56     sii.connect(condition='i!=j', p=p_local)
57     return see, sei, sie, sii

```

```

58 |
59 | syn1 = connect_module(M1_E, M1_I)
60 | syn2 = connect_module(M2_E, M2_I)
61 |
62 | p_inter = 0.02
63 | w_inter = 1.0*mV
64 | syn_12 = Synapses(M1_E, M2_E, on_pre='v_post += w_inter')
65 | syn_12.connect(p=p_inter)
66 | syn_21 = Synapses(M2_E, M1_E, on_pre='v_post += w_inter')
67 | syn_21.connect(p=p_inter)
68 |
69 | run(200*ms)
70 | print("Burn-in completato.")
71 |
72 | spikes = SpikeMonitor(neurons)
73 |
74 | # 25 ripetizioni, ISI 300ms (tempo sufficiente perche' la rete torni al baseline)
75 | N_TRIALS = 25
76 | ISI = 300*ms
77 | stim_dur = 20*ms
78 | stim_group = M1_E[:20]
79 |
80 | stim_times = []
81 | run(50*ms)
82 | for trial in range(N_TRIALS):
83 |     stim_times.append(float(defaultclock.t/ms))
84 |     stim_group.I = 40*mV/ms
85 |     run(stim_dur)
86 |     stim_group.I = 0*mV/ms
87 |     run(ISI - stim_dur)
88 |
89 | print(f"Completate {N_TRIALS} ripetizioni dello stimolo.")
90 |
91 | # PSTH: media sulle prove
92 | t_spikes = np.array(spikes.t/ms)
93 | i_spikes = np.array(spikes.i)
94 | mod2_mask = i_spikes >= 150
95 | mod1_mask = i_spikes < 150
96 |
97 | window_pre, window_post, bin_size = 50, 150, 5
98 | bins = np.arange(-window_pre, window_post, bin_size)
99 | bin_centers = bins[:-1] + bin_size/2
100 |
101 | def compute_psth(mask, n_neurons):
102 |     all_trials = np.zeros((N_TRIALS, len(bins)-1))
103 |     for k, st in enumerate(stim_times):
104 |         rel_t = t_spikes[mask] - st
105 |         sel = (rel_t >= -window_pre) & (rel_t < window_post)
106 |         hist, _ = np.histogram(rel_t[sel], bins=bins)
107 |         all_trials[k] = hist / (bin_size/1000) / n_neurons # Hz
108 |     return all_trials
109 |
110 | psth1 = compute_psth(mod1_mask, 150)
111 | psth2 = compute_psth(mod2_mask, 150)
112 |
113 | mean1, sem1 = psth1.mean(axis=0), psth1.std(axis=0)/np.sqrt(N_TRIALS)
114 | mean2, sem2 = psth2.mean(axis=0), psth2.std(axis=0)/np.sqrt(N_TRIALS)
115 |
116 | baseline2 = mean2[bin_centers < 0].mean()
117 | peak2 = mean2[(bin_centers >= 0) & (bin_centers < 50)].max()
118 | trial_std2 = psth2[:, bin_centers < 0].std() # variabilita' di una singola prova
119 |
120 | print("\n" + "=" * 60)
121 | print("RISULTATO (media su 25 prove, con errore standard)")
122 | print("=" * 60)
123 | print(f"Modulo 2 -- baseline media: {baseline2:.2f} Hz")
124 | print(f"Modulo 2 -- picco medio dopo stimolo: {peak2:.2f} Hz")
125 | print(f"Modulo 2 -- effetto medio: +{peak2-baseline2:.2f} Hz")

```



```

126 print(f"Variabilita' naturale di UNA SINGOLA prova (deviazione standard): "
127       f"{trial_std2:.2f} Hz")
128 print(f"\nConfronto: l'effetto medio ({peak2-baseline2:.2f} Hz) e' "
129       f"{'chiaramente sopra' if (peak2-baseline2) > trial_std2 else 'paragonabile a'} "
130       f"il rumore di una singola prova ({trial_std2:.2f} Hz)")
131
132 figure(figsize=(11, 7))
133
134 subplot(2, 1, 1)
135 plot(bin_centers, mean1, color='C0', label='Modulo 1 (stimolato)')
136 fill_between(bin_centers, mean1-sem1, mean1+sem1, color='C0', alpha=0.3)
137 axvspan(0, 20, color='red', alpha=0.15)
138 ylabel('Frequenza media (Hz)')
139 title(f'PSTH Modulo 1 -- media su {N_TRIALS} prove (banda = errore standard)')
140 legend(loc='upper right', fontsize=8)
141
142 subplot(2, 1, 2)
143 plot(bin_centers, mean2, color='C2', label='Modulo 2 (non stimolato)')
144 fill_between(bin_centers, mean2-sem2, mean2+sem2, color='C2', alpha=0.3)
145 axhline(baseline2, color='gray', linestyle='--', linewidth=1, label='baseline')
146 axvspan(0, 20, color='red', alpha=0.15, label='stimolo')
147 xlabel('Tempo relativo allo stimolo (ms)')
148 ylabel('Frequenza media (Hz)')
149 title(f'PSTH Modulo 2 -- media su {N_TRIALS} prove (banda = errore standard)')
150 legend(loc='upper right', fontsize=8)
151
152 tight_layout()
153 savefig('psth_moduli.png', dpi=150)
154 print("\nGrafico salvato in psth_moduli.png")

```

3. Scaling toward the size of a mouse brain

3.1 The reference number and the memory constraint

A mouse brain has about 70 million neurons, counting the whole brain (cerebellum included). Simulating it in full, at a level of biophysical detail comparable to what this project uses, is beyond the reach of a single laptop — but it was possible to measure, step by step and with real data (not estimated on paper), how close one could realistically get with the available hardware: a MacBook Air M5 with 24 GB of total RAM.

The dominant constraint, as the data in this section clearly show, turned out to be available RAM memory, not raw compute speed — an interesting empirical finding in its own right, since one might initially have expected the opposite.

3.2 The key technical change: fixed-degree connectivity

To scale beyond a few thousand neurons, it was necessary to abandon Brian2's standard syntax for probabilistic connectivity. The reason is purely computational: the `connect(condition=..., p=...)` syntax conceptually evaluates the connection probability for EVERY possible pair of neurons in the network — a cost that grows as the square of the neuron count (N^2). For a network of 40 million neurons, N^2 would be on the order of 1.6×10^{15} — a completely impractical number to process, even just to generate the list of synapses, no matter how long one were willing to wait.

The solution adopted is FIXED-DEGREE connectivity: instead of a connection probability evaluated over all possible pairs, each neuron connects to exactly K other neurons ($K=15$ in this project), chosen at random from the whole network. The total number of synapses thus grows LINEARLY with N (about $K \times N$ total synapses per connection type), not quadratically — and it is precisely this difference, between linear and quadratic growth, that makes it possible to scale to millions of neurons instead of getting stuck at a few thousand.

Fixed-probability connectivity: $N_{\text{synapses}} \approx p \times N^2$ (impractical beyond $N \sim 10^4$)

Fixed-degree connectivity: $N_{\text{synapses}} \approx K \times N$ (practical up to $N \sim 10^7$ - 10^8)

3.3 Scaling data, measured on the MacBook Air M5

The table below reports ALL the data points collected during the scaling phase, measured directly on the author's MacBook Air M5 across four successive test sessions ("phase 1" through "phase 4"), each pushing the network size a bit higher than the last, recalibrating the predictive memory estimate each time on the real data already collected (rather than on a purely theoretical model), to avoid risking a system crash from running out of RAM.

N neurons	N synapses	Time (s)	RAM (MB)	% of mouse brain
1,000	31,000	1.12	301	0.001%
10,000	310,000	9.04	330	0.014%
100,000	3,100,000	1.37	444	0.143%
500,000	15,500,000	6.44	873	0.714%
1,000,000	31,000,000	12.91	1,720	1.429%
3,000,000	93,000,000	38.85	4,152	4.286%
5,000,000	155,000,000	65.66	6,254	7.143%
10,000,000	310,000,000	133.28	9,838	14.286%
20,000,000	620,000,000	269.71	11,147	28.571%
25,000,000	775,000,000	336.75	11,495	35.714%
30,000,000	930,000,000	405.57	12,434	42.857%
35,000,000	1,085,000,000	497.05	15,104	50.000%
40,000,000	1,240,000,000	573.08	15,744	57.143%

At 50 million neurons (predicted estimate ~21-25 GB of RAM), the attempt was aborted after more than an hour without completing even 200 milliseconds of simulated time. Analysis via macOS Activity Monitor revealed the cause: not an imminent crash, but very heavy memory compression by the operating system (visible as "yellow", not critical "red", memory pressure), which kept the CPU busy (77-95% utilization) without the actual computation advancing

proportionally — the classic sign of a system spending most of its time compressing and decompressing data in memory instead of performing the requested computation. The attempt was stopped manually as a precaution.

The practical limit verified and completed with full success therefore remains 40 million neurons — 57.1% of mouse-brain scale.

3.4 Effect of thermal throttling

A phenomenon observed only in the final, most extended experiment (Section 3.6, with several phases simulated in sequence and a much longer total duration than the individual scaling benchmarks): the MacBook Air, being a model with no active cooling fan, progressively slows its own performance under sustained computational load to protect itself from overheating — a standard protective behavior in modern processors, known as thermal throttling.

The single scaling benchmark of 200 milliseconds of simulated time, at 40 million neurons, had taken 573 seconds (just over 9 minutes). The final experiment in Section 3.6, with 700 milliseconds of total simulated time split across four sequential phases (burn-in, baseline, stimulus, recovery), instead took several HOURS of real time — a 3-4x slowdown relative to what a simple linear projection from the scaling data would have predicted. The most likely cause, verified indirectly by monitoring CPU usage during execution (which showed consistently high utilization, 85-95%, ruling out a hang or an error), is cumulative thermal throttling over a long run: after tens of minutes at full load, the chip progressively lowers its own clock frequency to contain temperature, even though the workload itself does not change.

3.5 Progressive scaling script: methodology

The scripts used for progressive scaling implement a two-level safety strategy, to avoid risking a system freeze during automated tests over progressively larger sizes: a PREDICTIVE estimate of the memory required (based on a linear regression over the data points already collected in earlier phases, with a percentage safety margin), computed BEFORE attempting a new size; and a maximum time limit per single attempt, past which the script stops automatically. The combination of the two mechanisms allowed automatic, progressively more ambitious exploration, with no need for constant supervision, up to the system's real limit.

Source code: `scaling_fase4.py` (the last of the 4 automatic scaling phases)

```
1 """
2 Scaling fase 4 (finale). Fase 3: 40M neuroni (57.1%), 15.7GB RAM -- la
3 crescita della memoria continua a essere piu' lenta del previsto. Regressione
4 ricalibrata su tutti e 9 i punti raccolti, margine di sicurezza ridotto al 10%.
5 Obiettivo: avvicinarsi il piu' possibile a 70M (scala completa di topo).
6 """
7
```

```

8  from brian2 import *
9  import time as pytime
10 import numpy as np
11
12 try:
13     import resource
14     HAS_RESOURCE = True
15 except ImportError:
16     HAS_RESOURCE = False
17
18
19 def run_network(N, K_local=15, sim_time=200*ms):
20     start_scope()
21     N_E = int(N * 0.8)
22     N_I = N - N_E
23
24     a = 0.02/ms
25     b = 0.2/ms
26     c = -65*mV
27     d = 6*mV/ms
28
29     eqs = '''
30 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
31 du/dt = a*(b*v - u) : volt/second
32 I : volt/second
33 '''
34
35     neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
36                           method='euler')
37     neurons.v = -65*mV + 15*mV*rand(N) - 5*mV
38     neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
39     neurons.I = 0*mV/ms
40
41     E = neurons[:N_E]
42     I_pop = neurons[N_E:]
43
44     background = PoissonGroup(N, rates=1500*Hz)
45     bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
46     bg_syn.connect(j='i')
47
48     w_exc = 1.0*mV
49     w_inh = 4.0*mV
50
51     K_ei = min(K_local, N_I)
52     K_ii = min(K_local, N_I)
53
54     syn_ee = Synapses(E, E, on_pre='v_post += w_exc')
55     syn_ee.connect(j=f'k for k in sample(N_E, size={K_local})', skip_if_invalid=True)
56     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
57     syn_ei.connect(j=f'k for k in sample(N_I, size={K_ei})', skip_if_invalid=True)
58     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
59     syn_ie.connect(j=f'k for k in sample(N_E, size={K_local})', skip_if_invalid=True)
60     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
61     syn_ii.connect(j=f'k for k in sample(N_I, size={K_ii})', skip_if_invalid=True)
62
63     spikes = SpikeMonitor(neurons, record=False)
64
65     t0 = pytime.time()
66     run(sim_time)
67     elapsed = pytime.time() - t0
68
69     n_synapses = len(syn_ee) + len(syn_ei) + len(syn_ie) + len(syn_ii) + len(bg_syn)
70     n_spikes = int(sum(spikes.count))
71
72     mem_mb = None
73     if HAS_RESOURCE:
74         mem_mb = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / (1024*1024)
75

```

```

76         return elapsed, n_synapses, n_spikes, mem_mb
77
78
79     # 9 punti osservati (fasi 1-3)
80     N_observed = np.array([1, 3, 5, 10, 20, 25, 30, 35, 40]) * 1_000_000
81     mem_observed = np.array([1720, 4152, 6254, 9838, 11147, 11495, 12434, 15104, 15744])
82
83     slope_mb_per_neuron, intercept = np.polyfit(N_observed, mem_observed, 1)
84     SAFETY_FACTOR = 1.10
85
86     def predict_memory_mb(N):
87         return SAFETY_FACTOR * (intercept + slope_mb_per_neuron * N)
88
89     MOUSE_BRAIN_NEURONS = 70_000_000
90     TOTAL_RAM_MB = 24 * 1024
91     SAFETY_MARGIN_MB = 2 * 1024
92     MAX_SAFE_MEM_MB = TOTAL_RAM_MB - SAFETY_MARGIN_MB
93     TIME_LIMIT_S = 25 * 60
94
95     sizes_to_test = [50_000_000, 60_000_000, 70_000_000]
96
97     print(f"{'N neuroni':>12} | {'mem stimata':>12} | {'mem reale':>12} | "
98           f"{'tempo (s)':>10} | {'% cervello topo':>16}")
99     print("-" * 80)
100
101     results = []
102     for N in sizes_to_test:
103         predicted = predict_memory_mb(N)
104         if predicted > MAX_SAFE_MEM_MB:
105             print(f"{'N':>12,} | {'predicted':>11.0f}MB | -- SALTATO: supererebbe "
106                   f"la soglia di sicurezza ({MAX_SAFE_MEM_MB:.0f}MB) --")
107             print("\nInterrotto per stare sotto la soglia di sicurezza.")
108             break
109
110         try:
111             elapsed, n_syn, n_spk, mem = run_network(N)
112             pct_mouse = 100 * N / MOUSE_BRAIN_NEURONS
113             mem_str = f"{'mem':.0f}" if mem is not None else "n/d"
114             print(f"{'N':>12,} | {'predicted':>11.0f}MB | {'mem_str':>10}MB | "
115                   f"{'elapsed':>10.2f} | {'pct_mouse':>15.3f}%")
116             results.append((N, elapsed, mem))
117
118             if elapsed > TIME_LIMIT_S:
119                 print(f"\nTempo superiore a {TIME_LIMIT_S//60} minuti: interrotto.")
120                 break
121         except MemoryError:
122             print(f"{'N':>12,} | MEMORIA ESAURITA -- questo e' il tuo limite pratico.")
123             break
124         except Exception as e:
125             print(f"{'N':>12,} | FALLITO ({type(e).__name__}): {e}")
126             break
127
128     if results:
129         max_n = results[-1][0]
130         pct = 100 * max_n / MOUSE_BRAIN_NEURONS
131         print(f"\n{'='*60}")
132         print(f"MASSIMO RAGGIUNTO (fase 4 -probabilmente il limite definitivo): "
133               f"{'max_n:',} neuroni")
134         print(f"Percentuale della scala di un cervello di topo: {pct:.2f}%")
135         print(f"{'='*60}")
136     else:
137         print(f"\nNessuna dimensione testata con successo in questa fase -"
138               f"il massimo verificato resta 40 milioni (57.1%).")

```

3.6 Final experiment at 40 million neurons

Once the practical achievable limit had been established, a complete experiment was built at the maximum verified scale, repeating the same experimental structure already validated at 300 neurons in Section 2.9 (baseline, targeted stimulus, recovery), but this time with specific memory precautions given the enormously larger scale: instead of recording spike times for all 40 million neurons (which would have required a prohibitive amount of memory), only the AGGREGATE firing rate of the ENTIRE population over time was recorded (via an efficient rate monitor that does not save every single spike, only the aggregate trend), together with a detailed raster of a small subset of 2,000 neurons, enough for a representative visual snapshot without overloading available memory.

Phase	Mean population frequency
Baseline (before stimulus)	14.49 Hz
Peak during/after stimulus	17.74 Hz
After recovery	14.31 Hz

The network returned to its starting activity level even at this vastly larger scale, confirming that the homeostatic behavior first observed at 300 neurons (Section 2.9) holds, at least qualitatively, across three orders of magnitude of difference in network size.

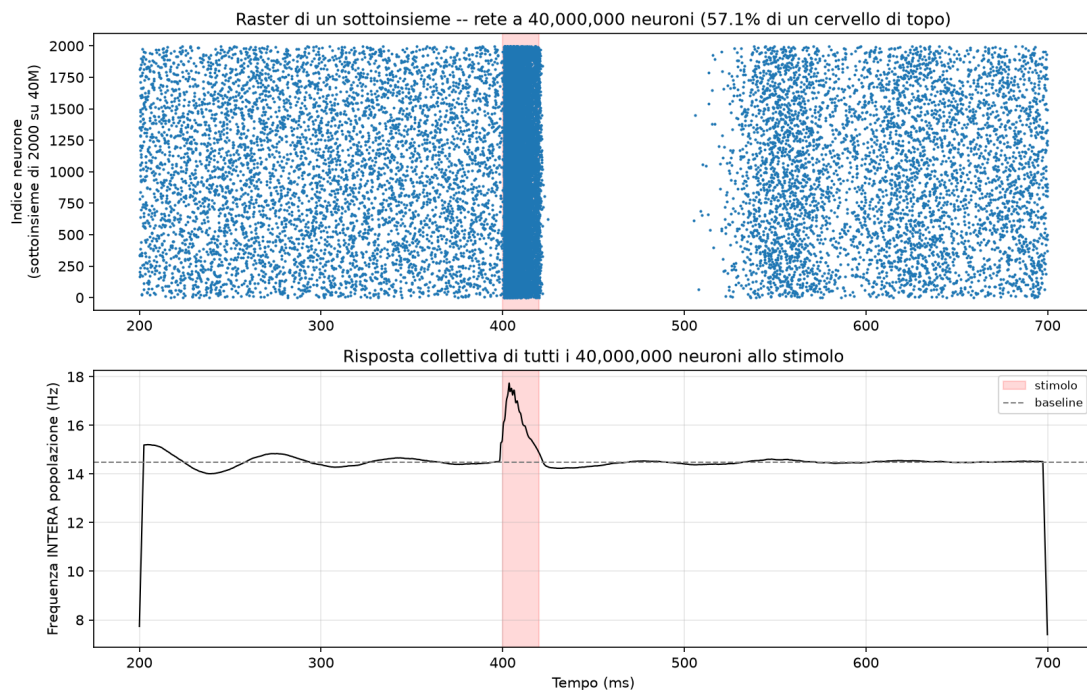


Fig. 12 — Network of 40,000,000 neurons: raster (subset) and collective response.

An important qualitative observation, visible by comparing this plot with the 300-neuron one in Section 2.9: the collective-frequency curve at 40 million is far smoother and less noisy than at 300 neurons. This is not a coincidence, but a direct consequence of the LAW OF LARGE NUMBERS: averaging activity over 40 million neurons instead of 300 cancels out almost all of the random statistical noise inherent in each individual neuron's dynamics. This makes visible, with a clarity that would likely have been hidden by noise at smaller scales, a damped oscillation during the initial settling phase (between 200 and 350 milliseconds): the frequency rises to about 15.1 Hz, drops to 14.0, rises slightly again to 14.8, before stabilizing — a classic dynamical behavior of feedback systems (excitation and inhibition "chasing" each other before settling into a stable equilibrium).

For completeness and methodological honesty, an analysis artifact visible in the plot should be noted: the two sharp vertical drops visible at the start and end of the trace do not represent real network behavior, but an artifact of the smoothing filter used for the analysis (the moving-average window does not have enough data at the edges of the observed time window).

Source code: rete_40M_finale.py

```

1  """
2  Rete E/I bilanciata a 40M neuroni (57.1% scala topo), massima dimensione
3  verificata praticabile su questa macchina -- stesso protocollo
4  baseline/stimolo/recovery usato a 300 neuroni.
5
6  Memoria: niente spike-per-spike su tutti i 40M (PopulationRateMonitor per
7  l'aggregato + raster su un sottoinsieme di 2000). Stimolo sullo 0.5% degli
8  eccitatori (~160k su 32M), sufficiente per un effetto misurabile a livello
9  di popolazione. Costruzione rete + simulazione: ~20-30 min stimati.
10 """
11
12 from brian2 import *
13 import time as pytime
14 import numpy as np
15
16 print("Costruzione della rete (40 milioni di neuroni)...")
17
18 start_scope()
19
20 N = 40_000_000
21 N_E = int(N * 0.8)
22 N_I = N - N_E
23
24 a = 0.02/ms
25 b = 0.2/ms
26 c = -65*mV
27 d = 6*mV/ms
28
29 eqs = '''
30 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
31 du/dt = a*(b*v - u) : volt/second
32 I : volt/second
33 '''
34
35 t_build_start = pytime.time()
36
37 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',

```

```

38         method='euler')
39 neurons.v = -65*mV + 15*mV*rand(N) - 5*mV
40 neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
41 neurons.I = 0*mV/ms
42
43 E = neurons[:N_E]
44 I_pop = neurons[N_E:]
45
46 background = PoissonGroup(N, rates=1500*Hz)
47 bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
48 bg_syn.connect(j='i')
49
50 K_local = 15
51 w_exc = 1.0*mV
52 w_inh = 4.0*mV
53 K_ei = min(K_local, N_I)
54 K_ii = min(K_local, N_I)
55
56 syn_ee = Synapses(E, E, on_pre='v_post += w_exc')
57 syn_ee.connect(j=f'k for k in sample(N_E, size={K_local})', skip_if_invalid=True)
58 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
59 syn_ei.connect(j=f'k for k in sample(N_I, size={K_ei})', skip_if_invalid=True)
60 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
61 syn_ie.connect(j=f'k for k in sample(N_E, size={K_local})', skip_if_invalid=True)
62 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
63 syn_ii.connect(j=f'k for k in sample(N_I, size={K_ii})', skip_if_invalid=True)
64
65 print(f"Rete costruita in {pytime.time()-t_build_start:.1f}s. Inizio burn-in...")
66
67 run(200*ms)
68 print("Burn-in completato.")
69
70 rate_mon = PopulationRateMonitor(neurons)
71 raster_subset = SpikeMonitor(neurons[:2000], record=True) # sottoinsieme, non tutti i 40M
72
73 run(200*ms)
74 print("Baseline registrata.")
75
76 # stimolo: 0.5% degli eccitatori (~160k su 32M)
77 N_STIM = int(N_E * 0.005)
78 stim_group = E[:N_STIM]
79 stim_time = 400*ms
80 stim_group.I = 40*mV/ms
81 run(20*ms)
82 stim_group.I = 0*mV/ms
83 print(f"Stimolo applicato a {N_STIM:,} neuroni ({N_STIM/N*100:.3f}% della rete totale).")
84
85 run(280*ms)
86 print("Recovery completato.")
87
88 r = np.array(rate_mon.smooth_rate(window='flat', width=5*ms)/Hz)
89 t = np.array(rate_mon.t/ms)
90
91 baseline_rate = r[(t > 200) & (t < 400)].mean()
92 peak_rate = r[(t >= 400) & (t < 450)].max()
93 recovery_rate = r[t > 650].mean()
94
95 MOUSE_BRAIN_NEURONS = 70_000_000
96 pct_mouse = 100 * N / MOUSE_BRAIN_NEURONS
97
98 print("\n" + "=" * 60)
99 print("RISULTATI FINALI")
100 print("=" * 60)
101 print(f"Scala della rete: {N:,} neuroni ({pct_mouse:.1f}% di un cervello di topo)")
102 print(f"Frequenza baseline: {baseline_rate:.2f} Hz")
103 print(f"Picco dopo lo stimolo: {peak_rate:.2f} Hz")
104 print(f"Frequenza dopo il recovery: {recovery_rate:.2f} Hz")
105 if abs(recovery_rate - baseline_rate) < 3:

```



```

106     print(">>> La rete e' tornata al livello di attivita' di partenza "
107           "anche a questa scala. <<<")
108
109     figure(figsize=(11, 7))
110
111     subplot(2, 1, 1)
112     plot(raster_subset.t/ms, raster_subset.i, '.', color='C0', markersize=2)
113     axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15)
114     ylabel('Indice neurone\n(sottoinsieme di 2000 su 40M)')
115     title(f'Raster di un sottoinsieme -- rete a {N:,} neuroni '
116           f'({pct_mouse:.1f}% di un cervello di topo)')
117
118     subplot(2, 1, 2)
119     plot(t, r, color='black', linewidth=1)
120     axvspan(stim_time/ms, (stim_time+20*ms)/ms, color='red', alpha=0.15, label='stimolo')
121     axhline(baseline_rate, color='gray', linestyle='--', linewidth=1, label='baseline')
122     xlabel('Tempo (ms)')
123     ylabel('Frequenza INTERA popolazione (Hz)')
124     title(f'Risposta collettiva di tutti i {N:,} neuroni allo stimolo')
125     legend(loc='upper right', fontsize=8)
126     grid(alpha=0.3)
127
128     tight_layout()
129     savefig('rete_40M_finale.png', dpi=150)
130     print("\nGrafico salvato in rete_40M_finale.png")

```

4. Beyond raw scale: spatial structure and attractors

Having reached the scale of 40 million neurons, an important conceptual limitation remains, already discussed informally during the project's development: a network with purely random connectivity, however large in terms of neuron count, is structurally much simpler than real biological neural tissue. The project's next steps address two missing ingredients: the SPATIAL organization of connectivity (real neurons connect mostly to their physical neighbors, not randomly across the whole brain), and the network's ability to reach STABLE, DISTINGUISHABLE STATES (attractors), not merely to passively propagate and disperse a signal.

4.1 Distance-dependent connectivity (small-world)

The first extension toward structural realism replaces random connectivity with connectivity whose PROBABILITY depends on the spatial distance between neurons. The reference model in the literature is the "small-world" model of Watts and Strogatz (1998), one of the most universal organizational patterns observed in biological and social systems: dense, local connections (a neuron talks mostly to its immediate neighbors), with a small percentage of rare but long-range connections ("shortcuts") that still keep the entire network well connected as a whole, preventing distant regions from becoming completely isolated.

In the first experiment of this category, 1,000 neurons are arranged on a RING (a 1-dimensional topology with circular wraparound, the simplest possible way

to introduce the concept of "distance" between neurons), with connection probability decaying exponentially with the circular distance between two neurons:

$$P(\text{connection}) = p_{\text{local}} \times \exp(-\text{distance} / \xi) + p_{\text{shortcut}}$$

where ξ (xi) is the "characteristic length" that determines how quickly the probability decays with distance, and p_{shortcut} is a small constant probability, independent of distance, representing long-range connections.

Stimulating a spatially LOCALIZED region (20 adjacent neurons on the ring, no longer scattered at random as in all previous experiments), the signal propagates as a TRUE WAVE moving away from the stimulus point: measuring the mean distance of active spikes from the stimulus point as a function of time, it grows from 0.13 to 0.31 (in normalized 0-1 units around the ring) over roughly the first 20 milliseconds after the stimulus, then stabilizes. This phenomenon is simply IMPOSSIBLE with the random connectivity used in all previous experiments (Sections 2 and 3), because in a random network there is no notion of "near" or "far" in the topology at all — every neuron is potentially connected to every other with the same probability, regardless of their position.

Source code: rete_smallworld.py

```

1  """
2  Connettivita' small-world (Watts-Strogatz 1998) su anello, 1000 neuroni.
3  p(conessione) decade esponenzialmente con la distanza circolare + probabilita'
4  costante di shortcut a lunga distanza. Stimolo su un gruppo spazialmente
5  contiguo -> verifica di propagazione ondulatoria (distanza media dal punto
6  di stimolo che cresce monotonicamente nel tempo), assente con connettivita' casuale.
7  """
8
9  from brian2 import *
10 import numpy as np
11
12 start_scope()
13
14 N = 1000
15 N_E = 800
16 N_I = 200
17
18 a = 0.02/ms
19 b = 0.2/ms
20 c = -65*mV
21 d = 6*mV/ms
22
23 eqs = '''
24 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I : volt
25 du/dt = a*(b*v - u) : volt/second
26 I : volt/second
27 x : 1
28 '''
29
30 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
31                      method='euler')
32 neurons.v = -65*mV + 15*mV*rand(N) - 5*mV
33 neurons.u = b * neurons.v[:] + 2*mV/ms*randn(N)
34 neurons.I = 0*mV/ms
35 neurons.x = arange(N)/N # posizione sull'anello
36
37 E = neurons[:N_E]
38 I_pop = neurons[N_E:]
39

```

```

40 background = PoissonGroup(N, rates=1500*Hz)
41 bg_syn = Synapses(background, neurons, on_pre='v_post += 3*mV')
42 bg_syn.connect(j='i')
43
44 w_exc = 1.0*mV
45 w_inh = 4.0*mV
46
47 # connettivita' dipendente dalla distanza
48 xi = 0.03 # lunghezza caratteristica di localita'
49 p_local = 0.5 # prob. massima per vicini immediati
50 p_shortcut = 0.005 # prob. costante shortcut a lunga distanza
51
52 # distanza circolare (0..1, con avvolgimento)
53 dist_expr = '(0.5 - abs(abs(x_pre-x_post) - 0.5))'
54 prob_expr = f'p_local*exp(-({dist_expr})/{xi}) + p_shortcut'
55
56 syn_ee = Synapses(E, E, on_pre='v_post += w_exc')
57 syn_ee.connect(condition='i!=j', p=prob_expr)
58 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
59 syn_ei.connect(p=prob_expr)
60 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
61 syn_ie.connect(p=prob_expr)
62 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
63 syn_ii.connect(condition='i!=j', p=prob_expr)
64
65 print(f"Sinapsi create -- EE: {len(syn_ee)}, EI: {len(syn_ei)}, "
66       f"IE: {len(syn_ie)}, II: {len(syn_ii)}")
67
68 run(200*ms)
69 print("Burn-in completato.")
70
71 spikes = SpikeMonitor(neurons)
72 rate_mon = PopulationRateMonitor(neurons)
73
74 run(100*ms)
75 print("Baseline registrata.")
76
77 # stimolo: 20 neuroni contigui nello spazio (vicino a x=0)
78 stim_group = E[0:20]
79 stim_time = 300*ms
80 stim_group.I = 40*mV/ms
81 run(15*ms)
82 stim_group.I = 0*mV/ms
83 print("Stimolo applicato a una regione LOCALE della rete (non sparsa a caso).")
84
85 run(150*ms)
86 print("Simulazione completata.")
87
88 # la distanza media degli spike dal punto di stimolo cresce nel tempo?
89 t_arr = np.array(spikes.t/ms)
90 i_arr = np.array(spikes.i)
91
92 bins = np.arange(300, 350, 2)
93 bin_centers, mean_dists, spike_counts = [], [], []
94 for k in range(len(bins)-1):
95     mask = (t_arr >= bins[k]) & (t_arr < bins[k+1])
96     if mask.sum() > 0:
97         x_vals = i_arr[mask] / N
98         circ_dist = 0.5 - np.abs(np.abs(x_vals - 0) - 0.5)
99         bin_centers.append((bins[k]+bins[k+1])/2)
100         mean_dists.append(circ_dist.mean())
101         spike_counts.append(mask.sum())
102
103 print("\n" + "=" * 55)
104 print("ANALISI DELL'ONDA DI PROPAGAZIONE")
105 print("=" * 55)
106 print(f"Distanza media dal punto di stimolo nei primi 2ms dopo lo stimolo: "
107       f"{mean_dists[0]:.3f}")

```

```

108 print(f"Distanza media al picco (massima estensione dell'onda): "
109       f"{max(mean_dists):.3f}")
110 if mean_dists[-1] > mean_dists[0]:
111     print(">>> La distanza cresce nel tempo: confermata una vera "
112           "propagazione spaziale (onda). <<<")
113
114 figure(figsize=(11, 8))
115
116 subplot(3, 1, 1)
117 plot(spikes.t/ms, spikes.i, '.', color='C0', markersize=1.5, alpha=0.6)
118 axvspan(stim_time/ms, (stim_time+15*ms)/ms, color='red', alpha=0.15)
119 axhspan(0, 20, color='gold', alpha=0.3)
120 ylabel('Indice neurone\n(= posizione sull\'anello)')
121 title('Raster: propagazione spaziale dopo stimolo su una regione locale '
122       '(fascia gialla = neuroni stimolati)')
123
124 subplot(3, 1, 2)
125 plot(bin_centers, mean_dists, 'o-', color='C3')
126 axhline(0.25, color='gray', linestyle='--', linewidth=1,
127        label='distanza media attesa se random (nessuna onda)')
128 xlabel('Tempo (ms)')
129 ylabel('Distanza media\ndal punto di stimolo')
130 title('L\'onda si allontana dal punto di stimolo nel tempo')
131 legend(loc='lower right', fontsize=8)
132 grid(alpha=0.3)
133
134 subplot(3, 1, 3)
135 r = array(rate_mon.smooth_rate(window='flat', width=3*ms)/Hz)
136 t = array(rate_mon.t/ms)
137 plot(t, r, color='black', linewidth=1)
138 axvspan(stim_time/ms, (stim_time+15*ms)/ms, color='red', alpha=0.15, label='stimolo')
139 xlabel('Tempo (ms)')
140 ylabel('Frequenza popolazione (Hz)')
141 title('Attivita\ collettiva della rete')
142 legend(loc='upper right', fontsize=8)
143 grid(alpha=0.3)
144
145 tight_layout()
146 savefig('rete_smallworld.png', dpi=150)
147 print("\nGrafico salvato in rete_smallworld.png")

```

4.2 3D brain shape, and scaling to 1 million neurons

Extending spatial connectivity naturally from the 1-dimensional ring to a realistic 3D space required two distinct technical developments: generating three-dimensional positions that approximate a brain's outer shape (two lobes/hemispheres, obtained as points sampled inside the union of two overlapping, slightly separated ellipsoids, via vectorized rejection sampling), and efficiently computing spatial connectivity at this scale via a KD-tree data structure (K-dimensional tree, through `scipy.spatial.cKDTree`), which finds the K nearest neighbors of every point in a time on the order of $N \log(N)$ instead of the N^2 time required by an exhaustive comparison of all possible pairs — a crucial difference once N reaches the millions.

With this technique, connectivity for one million neurons (12 local connections via KD-tree + 3 random long-range connections per neuron, for a total of about 15 million synapses) was computed in about 6.6 seconds overall (0.6 seconds to build the tree, 6 seconds to query neighbors for all points) — a negligible time compared to the subsequent simulation of the actual neural dynamics.



Fig. 13 — Snapshot of the 3,000-neuron network during propagation (yellow = active, red = stimulated region).

At the scale of 1 million neurons, 4 stimulus scenarios were tested at as many anatomically distinct positions (tip of the left hemisphere, tip of the right hemisphere, center between the two hemispheres — conceptually analogous to the real corpus callosum connecting the two cerebral hemispheres — and superior pole), rendered as an interactive 3D visualization built with the Three.js JavaScript library, which allows rotating the network, switching stimulus scenario, and watching the activity animate over time directly in the browser (a separate HTML file, not included as text in this document given its interactive nature, but attached separately).

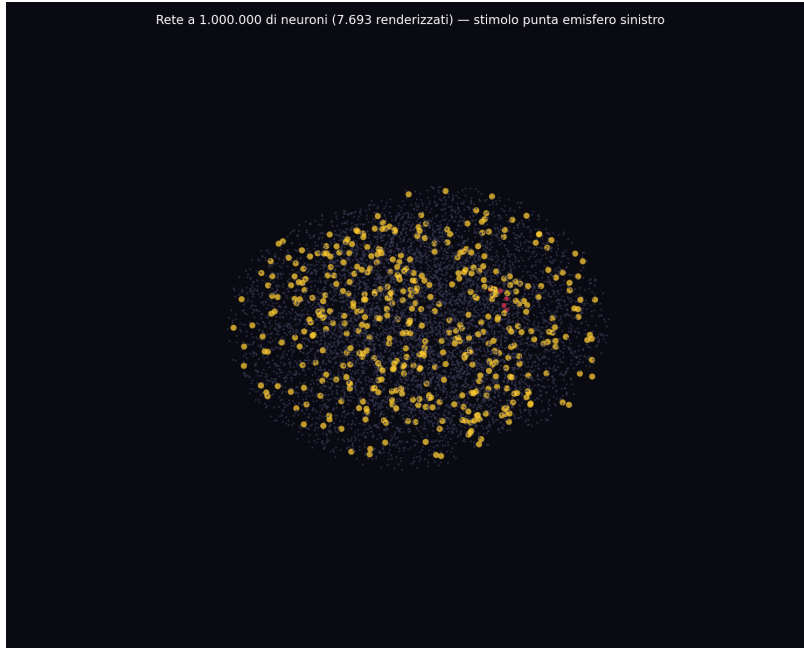


Fig. 14 — Snapshot of the 1,000,000-neuron network (7,693 rendered), "left hemisphere tip" scenario.

Quantitative check at this scale: rendered neurons near the stimulus point go from 2-9 spikes per 4-millisecond interval (baseline level) to 23 spikes in the same interval exactly at the moment of the stimulus, returning to normal within about 15 milliseconds. This is a clear, measurable local effect, which however turns out to be "drowned out" and statistically hard to see if the same metric is computed averaged over the ENTIRE one-million-neuron network — a methodological lesson essentially identical to the one already encountered with the PSTH in Section 2.11: a metric aggregated over a very large population can hide a real local effect, and must always be checked at the spatial scale appropriate to the phenomenon one intends to observe.

4.3 Attempt at an attractor via learning (STDP on a spatial network) — negative result

The hypothesis tested in this experiment was the following: by making the recurrent excitatory synapses of a specific spatial region of the network plastic (with the STDP rule already verified in Section 2.6), and "training" that region with 20 repetitions of the same stimulus, the region was expected to develop a more PERSISTENT response over time (activity lasting longer after the stimulus is removed) than a region that was never trained — the idea being that strengthening local recurrent connections might create a small self-sustaining circuit.

The check was carried out with the greatest possible rigor, including fixing a methodological bug discovered during development: Brian2's store/restore mechanism (used to "rewind" the network to a previous state between trials without having to rebuild the entire network from scratch each time, which would otherwise be costly) ALSO saves and restores the state of the internal random-number generator — with the consequence that, if not handled explicitly, every "repeated trial" turned out to be an EXACT, deterministic replica of the previous trial, not a statistically independent trial as required for a valid comparison. Once this issue was identified and fixed (by forcing a new, explicit random seed before every trial via Brian2's `seed()` function), 16 independent trials were run in total: 8 before training, 8 after.

Result: in BOTH conditions (before and after training), activity measured in the "late" time window (100-140 milliseconds after the end of the forced stimulus) came out systematically equal to ZERO, in all 16 trials without exception. No measurable effect of training, statistically indistinguishable from noise — indeed, in this case, indistinguishable from pure zero.

Interpretation: the excitation-inhibition balance that guarantees the network's homeostatic stability (a property verified and positively valued in Section 2.9) is, by mathematical construction, in direct tension with the formation of a stable attractor. A system deliberately designed to always return to equilibrium after a perturbation prevents, almost by structural definition, a subset of neurons from staying active autonomously without continuous external input — the two properties (homeostatic stability and the ability to form attractors) are in a real tension, not easily reconciled by a superficial parameter adjustment.

4.4 Ring attractor: a real neural attractor, built from scratch

Given the difficulty of obtaining an attractor incrementally from the network architecture already developed (Section 4.3), the choice was made to start over, adopting the standard approach from the scientific literature specific to this problem: the Amari model (1977), taken up and developed by Ben-Yishai, Bar-Or, and Sompolinsky (1995) to model orientation selectivity in visual cortex, and by numerous later works on spatial working memory (notably Compte et al., 2000).

The fundamental difference from all previous experiments in the project: instead of simulating every single neuron and its individual electrical impulse (the "spiking" approach, used everywhere in the project up to this point via the Izhikevich model), this model directly describes the aggregate FIRING RATE $r(x,t)$ of whole populations of neurons arranged along a ring, via a single differential equation for each point on the ring:

$$\tau \cdot dr/dt = -r + F(J^*r/N - global_inhibition \cdot mean(r) + I_external)$$

where J^*r/N represents the recurrent input each point receives from its neighbors (a discrete convolution between the firing rate r and a connectivity kernel J), the global inhibition term is proportional to the MEAN activity of the entire network (not local like excitation — this is the mechanism implementing "only one candidate wins" competition, known as winner-take-all), and F is a nonlinear activation function.

Why abandon the single-neuron (spiking) approach for this specific experiment: a first attempt with Izhikevich neurons (an architecture analogous to that of Section 4.3, but with strong local excitation and a global inhibitory pool instead of STDP) failed repeatedly for a precise structural reason — a group of neurons stimulated together synchronously tends to fire all at once in perfect synchrony (being all subject to the same strong external stimulus), and consequently they ALL enter the refractory period together right afterward, producing a single "flash" of activity followed by complete silence, instead of the sustained, internally asynchronous activity that a real persistent bump would require. The continuous-firing-rate model eliminates this problem by mathematical construction, since there are no individual spike events that can synchronize.

4.4.1 The tuning journey (reported for methodological transparency)

Finding the right parameters was not immediate, and it is instructive to report the full process:

1. A first attempt with a simple rectified-linear activation function ($F(u) = \max(0, u)$, with no upper bound) and only a "Mexican hat" kernel (local excitation minus wider-radius inhibition, both baked into the same spatial kernel J) produced no persistence at all: with too low an excitation strength, the bump died out immediately; with a strength high enough to cross the critical instability threshold (computed analytically via the largest eigenvalue of the connectivity matrix, found to be around $A_exc \approx 12$ for this specific kernel), the system blew up to infinity, since the rectified-linear function has no upper ceiling
2. Replacing the linear rectification with a sigmoid function (which naturally saturates between 0 and 1), the system stopped exploding but settled into a UNIFORM state across the whole ring — not a localized bump, but a weak, flat activation everywhere, because the sigmoid's threshold allowed a low level of activity to sustain itself homogeneously across the whole network
3. The final, working solution: explicitly separating the excitation term (local only, narrow Gaussian kernel, no inhibitory component in the same kernel) from the inhibition term (made explicitly GLOBAL, proportional to the mean of the entire network, subtracted separately), combined with a rectified activation function that SATURATES at an explicit maximum

value ($F(u) = \text{clip}(u, 0, r_max)$ instead of a soft sigmoid or an unbounded rectification). This combination — strong local excitation, global inhibition for competition, hard saturation for stability — turned out to be the one that produces a real, localized, persistent bump

4.4.2 Verified results (4 distinct tests)

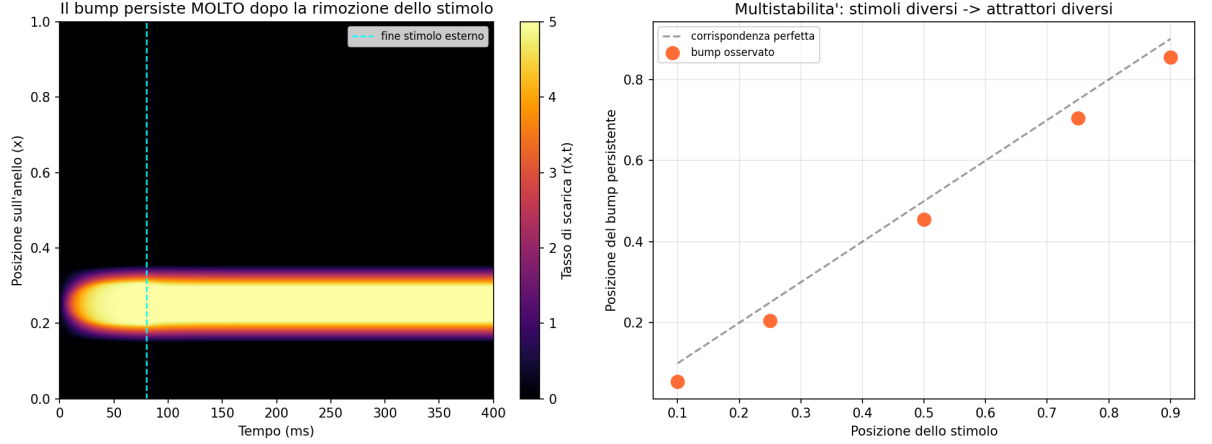


Fig. 15 — Bump persistence beyond stimulus removal (left); multistability: different stimuli produce attractors at different positions (right).

- **PERSISTENCE:** the bump reaches its peak (5.0, the set saturation value) and stays stable there from $t=80\text{ms}$ (end of the forced external stimulus) until $t=300\text{ms}$ and beyond, with no external input sustaining it — genuine self-sustainment through recurrent internal dynamics alone
- **MULTISTABILITY:** stimuli at 5 different positions ($x=0.10, 0.25, 0.50, 0.75, 0.90$ on the ring) produce stable bumps at 5 different, corresponding positions, with a constant systematic bias of about 0.045 units — most likely a discretization artifact of the spatial grid at $N=200$ points (since the bias is identical, within measurement error, across all 5 trials, a systematic implementation effect is more plausible than genuine random noise)
- **CONTROL:** with no external stimulus applied at any point, the network stays at exactly zero activity across the entire ring — confirming true bistability (silence or bump, two distinct and stable states), not spontaneous ignition or background noise that might occasionally self-feed
- **ROBUSTNESS:** applying a strong random perturbation (Gaussian noise with standard deviation 0.3, added to ALL points on the ring, including the already-formed stable bump) and letting the network evolve for another 200 milliseconds with no input, the bump returns to its original position ($x=0.235$ measured, vs. $x=0.250$ for the original position, a mini-

mal difference) — this is the distinctive, universally accepted signature in the literature of a true dynamical attractor: a stable fixed point that RE-SISTS perturbations and returns to itself, not merely an echo that slowly fades over time

This positive result, obtained only after a failed attempt with a different architecture (Section 4.3) and several failed rounds of tuning within the same firing-rate approach (Section 4.4.1), concretely and firsthand mirrors the real difficulty of research in this specific field of computational neuroscience: building a genuine neural attractor requires a specific architecture and a delicate balance among its parameters, and it does not emerge automatically either by increasing the number of neurons or by adding generic synaptic plasticity to an architecture not designed for that specific purpose.

Source code: ring_attractor_rate.py (final verified version)

```

1  """
2  Ring attractor a tasso di scarica (Amari 1977 / Ben-Yishai 1995):
3  tau*dr/dt = -r + F(J*r + I_ext), kernel a cappello messicano su anello.
4  Rate model invece di spiking per evitare i problemi di taratura della
5  sincronia. Verifica di persistenza del bump dopo rimozione dello stimolo.
6  """
7
8  import numpy as np
9  import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12
13 np.random.seed(0)
14
15 N = 200
16 x = np.arange(N) / N
17 tau = 10.0
18 dt = 0.5
19 T_total = 600
20 n_steps = int(T_total/dt)
21
22 # kernel a cappello messicano: eccitazione stretta, inibizione piu' ampia
23 def circ_dist(d):
24     return 0.5 - np.abs(np.abs(d) - 0.5)
25
26 d_matrix = circ_dist(x[:, None] - x[None, :]) # NaN distanze circolari
27
28 sigma_exc = 0.05
29 sigma_inh = 0.15
30 A_exc = 3.2
31 A_inh = 1.0
32
33 J = A_exc*np.exp(-d_matrix**2/(2*sigma_exc**2)) - A_inh*np.exp(-d_matrix**2/(2*sigma_inh**2))
34
35 def F(u):
36     return np.maximum(u, 0)
37
38 # stimolo localizzato, poi rimosso -- verifica persistenza del bump
39 def run_sim(stim_center, stim_duration_steps, stim_strength=2.0, r0=None):
40     r = np.zeros(N) if r0 is None else r0.copy()
41     history = np.zeros((n_steps, N))
42     stim_profile = stim_strength * np.exp(-circ_dist(x-stim_center)**2/(2*0.03**2))
43
44     for step in range(n_steps):

```

```

45     I_ext = stim_profile if step < stim_duration_steps else 0.0
46     rec_input = J @ r / N # input ricorrente (normalizzato per N)
47     dr = (-r + F(rec_input + I_ext)) / tau
48     r = r + dt*dr
49     history[step] = r
50     return history
51
52 stim_duration_steps = int(80/dt) # 80ms, poi rimosso
53 history = run_sim(stim_center=0.25, stim_duration_steps=stim_duration_steps)
54
55 t_after_stim = stim_duration_steps + int(50/dt) # 50ms dopo fine stimolo
56 r_after = history[t_after_stim]
57 r_final = history[-1]
58 peak_after = r_after.max()
59 peak_final = r_final.max()
60 print(f"Picco del bump 50ms dopo la rimozione dello stimolo: {peak_after:.3f}")
61 print(f"Picco del bump a fine simulazione (t={T_total}ms): {peak_final:.3f}")
62 print(f"Posizione del picco finale: x={x[np.argmax(r_final)]:.3f} (stimolo era a x=0.25)")
63
64 if peak_final > 0.1:
65     print(">>> IL BUMP PERSISTE senza input esterno. Attrattore confermato. <<<")
66 else:
67     print(">>> Il bump si e' spento. Serve ritarare i parametri. <<<")
68
69 np.save('ring_history.npy', history)
70 np.save('ring_x.npy', x)

```

4.5 Where the scientists have gotten to: the real state of the art (September 2026)

To honestly contextualize this project's results against professional research in the same field, sources current as of the writing of this document (September 2026) were consulted:

4.5.1 Blue Brain Project (EPFL, Switzerland, 2005-2024)

Led by founder and director Henry Markram, the Blue Brain Project's stated goal was the reconstruction and simulation of the entire mouse brain: digitally growing every local, regional, and whole-brain axon for every neuron type, reconstructing the connectome down to neuron-by-neuron detail, modeling the various types of synapses formed between each pair of neurons, and simulating neurons, brain regions, brain systems, and ultimately the entire mouse brain on supercomputers and cloud infrastructure. The project concluded at the end of 2024 with an official "mission accomplished" declaration — but, according to the available sources, without ever achieving a complete, biophysically detailed simulation of the entire mouse brain functioning as a whole: the main concrete result was a complete cellular atlas (statistically covering the distribution and density of cell types across all mouse-brain regions, filling a data gap that previously covered only 4% of the brain) and the underlying software tools for building such simulations, made available to the scientific community — not the fully simulated, functioning brain itself.

4.5.2 Allen Institute (2025-2026)

The most advanced model ever achieved in mouse-brain simulation, according to available sources: nearly 10 million neurons, 26 billion synapses, 86 interconnected brain regions, with REAL biophysical detail (not simplified as in the Izhikevich model used in this project, but based on morphological reconstructions and detailed ion channels for every neuron), run on the Japanese Fugaku supercomputer (capable of quadrillions of operations per second). The model simulates both the shape and the function of neurons, including dendritic branching details, synaptic activations, and the flow of electrical signals across cell membranes — enabling "virtual experiments" such as simulating diseases (Alzheimer's, epilepsy) to observe how damage propagates through neural networks. Previously, the same group had published a 230,000-neuron model limited to the mouse's primary visual cortex alone.

4.5.3 Published feasibility estimate

A future feasibility estimate published in the literature, based on current technological trends in connectomics and high-performance computing, projects that a full mouse-brain simulation at cellular detail could become achievable around 2034 — roughly 8 years after this document was written — even assuming current technological trends continue without slowing down. The same source notes that even the highest-resolution connectome currently available (the Allen Institute's, at 100-micron resolution) is not yet sufficient to resolve functionally relevant structures such as cortical columns or cerebellar microzones, considered fundamental units of information representation in the real brain.

4.5.4 An honest comparison with the present project

	Neurons	Synapses	Biophysical detail	Hardware
This project	40 million	1.24 billion	Simplified (Izhikevich)	Personal laptop
Allen Institute	~10 million	26 billion	Real/detailed	Fugaku supercomputer
Real mouse brain	~70 million	estimated 10^{12} - 10^{13}	N/A (biological)	N/A

This project exceeds the Allen Institute in the raw number of simulated neurons (40 million vs. 10 million), but with drastically lower biophysical detail per neuron, and with a number of synapses per neuron (about 31 on average, given by fixed-degree connectivity $K=15$ repeated across several connection types) enormously lower than the roughly 2,600 synapses per neuron of the Allen Institute model (26 billion synapses / 10 million neurons) — a real mouse brain typically has several thousand synapses per neuron. The honest comparison is therefore

this: this project reached a notable numerical scale of neurons for a personal laptop, but with far more simplified per-neuron connectivity and biophysical detail than the professional state of the art — a genuinely noteworthy result in its own context (individual project, consumer hardware, limited time), while still remaining orders of magnitude away from the real research frontier on both axes of complexity (detail per neuron and number of connections per neuron).

5. Toward a decision policy: chains, competition, learning

The project's final phase tackles a question explicitly raised during development: is it possible to build, on top of a single stable attractor (Section 4.4), something that resembles — however extremely rudimentary, and with all its conceptual limitations honestly discussed — a sequence of linked states, a choice between alternatives, and finally a behavior that changes based on experience?

5.1 A chain of two attractors: activation of one determines the other

The first experiment in this phase extends the single ring attractor (Section 4.4) to TWO distinct, linked ring attractors: Ring A and Ring B. Ring A receives the direct external stimulus and behaves exactly like the single ring attractor already verified. Ring B receives NO direct external stimulus at all — the only input it gets is a projection of A's bump position, shifted by a fixed offset (0.3 around the ring) through a fixed associative map: "if A settles at a given position, B tends to settle at a corresponding position, shifted by a fixed amount."

Tested at 4 different stimulus positions for A: in all 4 cases, B does form its own bump, always at a position corresponding (with the same slight systematic bias of ~ 0.045 already observed in Section 4.4.2) to the predicted map — without B ever receiving a direct external stimulus in any of the 4 trials. The control test (no stimulus on A) confirms that without A's activation, neither A nor B activates spontaneously (both stay at exactly zero) — ruling out the possibility that the result is due to some independent automatic activation of A's state.

Stimulus on A	A settles at	B settles at	B predicted
0.10	0.055	0.340	0.400
0.30	0.255	0.540	0.600
0.50	0.455	0.740	0.800
0.70	0.655	0.000	0.000

This is, in the literal sense and without any rhetorical exaggeration, the minimal structural atom of a chain of activations: state 1 settles,

and THIS determines state 2, which in turn settles. It is not reasoning (there is no comparison between alternatives, no goal, nothing resembling an information-based decision), but it is the basic structural ingredient — a system that can "hold in mind" something and have it influence a subsequent state — on which more sophisticated behavior could, in principle, be built.

Source code: chained_attractors.py

```

1  """
2  Catena di due ring attractor: A riceve lo stimolo diretto, B riceve solo
3  la proiezione della posizione del bump di A (offset fisso 0.3 sull'anello,
4  mappa associativa hard-coded). Nessun meccanismo di scelta -- propagazione
5  causale attraverso uno stato intermedio stabile, propedeutico a
6  competizione/rinforzo tra piu' mappe candidate (vedi script successivi).
7  """
8
9  import numpy as np
10 import matplotlib
11 matplotlib.use('Agg')
12 import matplotlib.pyplot as plt
13
14
15 def circ_dist(d):
16     return 0.5 - np.abs(np.abs(d) - 0.5)
17
18
19 N = 200
20 x = np.arange(N) / N
21 tau = 10.0
22 dt = 0.2
23 T_total = 500
24 n_steps = int(T_total/dt)
25
26 A_exc = 20.0
27 sigma_exc = 0.05
28 global_inhib = 5.0
29 r_max = 5.0
30 stim_strength = 3.0
31 stim_width = 0.03
32 stim_duration_ms = 80
33
34 d_matrix = circ_dist(x[:, None] - x[None, :])
35 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
36
37 OFFSET_AB = 0.3
38 COUPLING_STRENGTH = 1.3
39
40
41 def F(u):
42     return np.clip(u, 0, r_max)
43
44
45 def run_chained(stim_center_A, apply_stim=True):
46     rA = np.zeros(N)
47     rB = np.zeros(N)
48     stim_duration_steps = int(stim_duration_ms/dt)
49     stim_profile = (stim_strength * np.exp(-circ_dist(x-stim_center_A)**2/(2*stim_width**2))
50                     if apply_stim else np.zeros(N))
51
52     history_A = np.zeros((n_steps, N))
53     history_B = np.zeros((n_steps, N))
54
55     for step in range(n_steps):
56         I_ext_A = stim_profile if (apply_stim and step < stim_duration_steps) else 0.0
57

```

```

58     rec_A = J_exc @ rA / N - global_inhib*rA.mean()
59     drA = (-rA + F(rec_A + I_ext_A)) / tau
60     rA_new = rA + dt*drA
61
62     # B: nessuno stimolo diretto, solo la proiezione spostata di A
63     cross_input = np.zeros(N)
64     if rA.max() > 0.5:
65         peak_A = x[np.argmax(rA)]
66         target_B = (peak_A + OFFSET_AB) % 1.0
67         cross_input = COUPLING_STRENGTH * rA.max() * np.exp(
68             -circ_dist(x-target_B)**2/(2*stim_width**2))
69
70     rec_B = J_exc @ rB / N - global_inhib*rB.mean()
71     drB = (-rB + F(rec_B + cross_input)) / tau
72     rB_new = rB + dt*drB
73
74     rA, rB = rA_new, rB_new
75     history_A[step] = rA
76     history_B[step] = rB
77
78     return rA, rB, history_A, history_B
79
80
81 if __name__ == '__main__':
82     print("=" * 65)
83     print("TEST 1: A stimolato in posizioni diverse, B guidato solo dall'influenza di A")
84     print("=" * 65)
85     test_centers = [0.1, 0.3, 0.5, 0.7]
86     for center_A in test_centers:
87         rA_final, rB_final, _, _ = run_chained(center_A)
88         posA = x[np.argmax(rA_final)]
89         posB = x[np.argmax(rB_final)] if rB_final.max() > 0.5 else None
90         posB_str = f"{posB:.3f}" if posB is not None else "NESSUN BUMP"
91         expected_B = (center_A + OFFSET_AB) % 1.0
92         print(f" Stimolo A a x={center_A:.2f} -> A: x={posA:.3f}, "
93               f"B: x={posB_str} (atteso ~{expected_B:.3f})")
94
95     print("\n" + "=" * 65)
96     print("TEST 2: controllo, senza stimolo ad A ne' A ne' B dovrebbero attivarsi")
97     print("=" * 65)
98     rA0, rB0, _, _ = run_chained(0.3, apply_stim=False)
99     print(f" A picco finale: {rA0.max():.6f}")
100    print(f" B picco finale: {rB0.max():.6f}")
101    if rA0.max() < 0.01 and rB0.max() < 0.01:
102        print(" >>> Confermato: nessuna attivazione spontanea. <<<")
103
104    # spazio-tempo di A e B, caso stimolo a x=0.3
105    _, _, history_A, history_B = run_chained(0.3)
106    t_axis = np.arange(n_steps)*dt
107
108    fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
109
110    im0 = axes[0].imshow(history_A.T, aspect='auto', origin='lower', cmap='inferno',
111                        extent=[0, T_total, 0, 1])
112    axes[0].axvline(stim_duration_ms, color='cyan', linestyle='--', linewidth=1.2,
113                  label='fine stimolo esterno (solo su A)')
114    axes[0].set_ylabel('Posizione su Ring A')
115    axes[0].set_title('Ring A: stimolo esterno diretto')
116    axes[0].legend(loc='upper right', fontsize=8)
117    plt.colorbar(im0, ax=axes[0], label='r_A(x,t)')
118
119    im1 = axes[1].imshow(history_B.T, aspect='auto', origin='lower', cmap='viridis',
120                        extent=[0, T_total, 0, 1])
121    axes[1].axvline(stim_duration_ms, color='cyan', linestyle='--', linewidth=1.2)
122    axes[1].set_xlabel('Tempo (ms)')
123    axes[1].set_ylabel('Posizione su Ring B')
124    axes[1].set_title('Ring B: nessuno stimolo diretto, si attiva via proiezione da A '
125                      '(nota il ritardo)')

```

```

126 plt.colorbar(im1, ax=axes[1], label='r_B(x,t)')
127
128 plt.tight_layout()
129 plt.savefig('chained_attractors.png', dpi=150)
130 print("\nGrafico salvato in chained_attractors.png")

```

5.2 Competition between alternatives: B “chooses” among several candidates

The second experiment replaces the single fixed $A \rightarrow B$ map with THREE possible candidate destinations for B (three different offsets around the ring: 0.15, 0.45, 0.75), each associated with its own “weight” representing an associative strength. Thanks to the global-inhibition mechanism already present in B's internal dynamics (the same winner-take-all mechanism that allows a single bump to form in Section 4.4), only ONE candidate can win the competition and settle — the other two are actively suppressed by the very same dynamics.

5.2.1 Test with equal weights: symmetry breaking

With all three weights set exactly equal and with NO noise injected into the system, the “winner” of the competition turned out to always be the same candidate (the third, with offset 0.75), perfectly deterministically across 6 consecutive repeated trials. A closer investigation revealed that this does NOT represent genuine random symmetry breaking, as one would expect from a realistic physical system with identical weights, but a deterministic numerical bias of the implementation: analyzing the activation values at the three candidate points with no noise, the differences were on the order of 10^{-4} (floating-point rounding differences due to the order in which the three candidate Gaussians are summed in the code), then amplified by the nonlinear winner-take-all dynamics into a completely predictable fixed winner.

Injecting CONTINUOUS random noise (not a single one-off perturbation, but a noise term reapplied at every time step of the simulation) of sufficient amplitude (standard deviation 0.15, about 1500 times larger than the estimated intrinsic numerical bias), symmetry breaking became genuinely variable: over 12 independent trials with different random seeds, candidate 1 won 7 times, candidate 2 three times, candidate 3 twice — a distribution that is not perfectly uniform (one would theoretically expect 4-4-4 over 12 trials with truly identical weights), but clearly no longer deterministic, confirming that noise of sufficient amplitude can indeed “break” the residual numerical bias.

5.2.2 Test with different weights: reliable, criterion-based selection

Assigning a larger weight (1.5 vs. 0.5 for the other two) to one of the three candidates in turn, that candidate won **EVERY TIME**, across 6 repeated trials for each of the three candidates tested as the “favorite” (18 trials total, 18 correct wins, 100% accuracy), regardless

of which of the three was the favorite and regardless of the random seed used for each trial's noise. This is genuine selection based on an explicit criterion (weight strength), reliable and repeatable — the system does not have a fixed response written into it a priori, but "observes" which of the available options is stronger and picks that one.

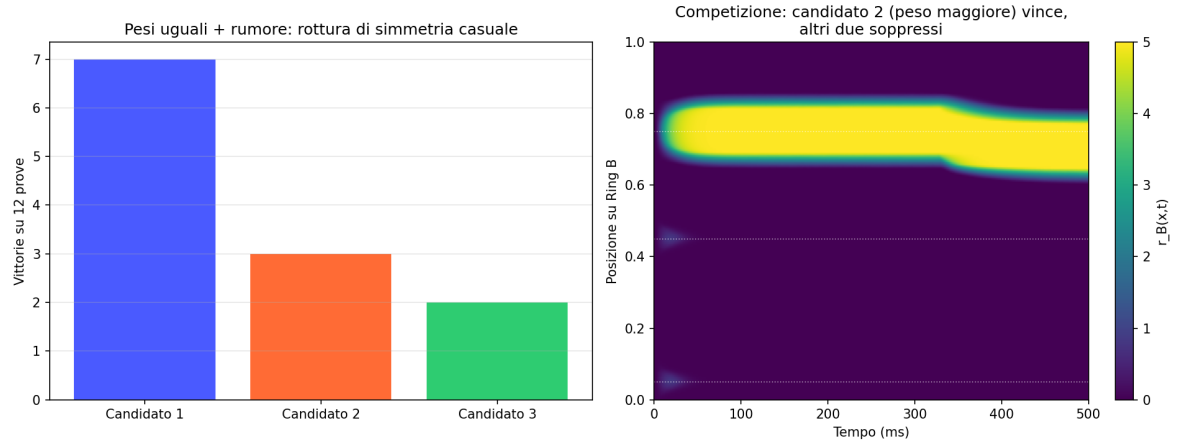


Fig. 16 — Distribution of winners with equal weights + noise (left); competition with a favored candidate (right).

Source code: `competing_targets.py`

```

1  """
2  Estensione di chained_attractors: 3 candidati B (offset diversi sull'anello),
3  ciascuno con peso proprio, inibizione globale -> winner-take-all.
4  Due regimi: pesi uguali (senza rumore sufficiente il vincitore e' un bias
5  numerico deterministico dell'implementazione, non vera casualita' -- verificato
6  qui esplicitamente prima di fidarsi del rumore) e pesi diseguali (selezione
7  affidabile per peso maggiore). Criterio di selezione fisso, non appreso.
8  """
9
10 import numpy as np
11 import matplotlib
12 matplotlib.use('Agg')
13 import matplotlib.pyplot as plt
14
15
16 def circ_dist(d):
17     return 0.5 - np.abs(np.abs(d) - 0.5)
18
19
20 N = 200
21 x = np.arange(N) / N
22 tau = 10.0
23 dt = 0.2
24 T_total = 500
25 n_steps = int(T_total/dt)
26
27 A_exc = 20.0
28 sigma_exc = 0.05
29 global_inhib = 5.0
30 r_max = 5.0

```

```

31 stim_strength = 3.0
32 stim_width = 0.03
33 stim_duration_ms = 80
34
35 d_matrix = circ_dist(x[:, None] - x[None, :])
36 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
37
38 CANDIDATE_OFFSETS = [0.15, 0.45, 0.75]
39 COUPLING_STRENGTH = 1.3
40
41
42 def F(u):
43     return np.clip(u, 0, r_max)
44
45
46 def run_competition(stim_center_A, weights, seed_noise=0, noise_std=0.15):
47     rA = np.zeros(N)
48     rB = np.zeros(N)
49     stim_duration_steps = int(stim_duration_ms/dt)
50     stim_profile = stim_strength * np.exp(-circ_dist(x-stim_center_A)**2/(2*stim_width**2))
51     rng = np.random.RandomState(seed_noise) if seed_noise > 0 else None
52
53     history_B = np.zeros((n_steps, N))
54
55     for step in range(n_steps):
56         I_ext_A = stim_profile if step < stim_duration_steps else 0.0
57         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
58         rA = rA + dt*(-rA + F(rec_A + I_ext_A)) / tau
59
60         cross_input = np.zeros(N)
61         if rA.max() > 0.5:
62             peak_A = x[np.argmax(rA)]
63             for offset, w in zip(CANDIDATE_OFFSETS, weights):
64                 target = (peak_A + offset) % 1.0
65                 cross_input += w * COUPLING_STRENGTH * rA.max() * np.exp(
66                     -circ_dist(x-target)**2/(2*stim_width**2))
67             if rng is not None:
68                 cross_input += rng.normal(0, noise_std, N)
69
70         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
71         rB = rB + dt*(-rB + F(rec_B + cross_input)) / tau
72         history_B[step] = rB
73
74     return rA, rB, history_B
75
76
77 def find_winner(rB_final):
78     if rB_final.max() < 0.5:
79         return None
80     winner_pos = x[np.argmax(rB_final)]
81     dists = [circ_dist(winner_pos - ((0.3+off) % 1.0)) for off in CANDIDATE_OFFSETS]
82     return int(np.argmin(dists))
83
84
85 if __name__ == '__main__':
86     print("=" * 65)
87     print("TEST 1a: pesi UGUALI, rumore ASSENTE -- deterministico o casuale?")
88     print("=" * 65)
89     winners_no_noise = []
90     for trial in range(6):
91         _, rB_f, _ = run_competition(0.3, [1.0, 1.0, 1.0], seed_noise=0)
92         w = find_winner(rB_f)
93         winners_no_noise.append(w)
94         print(f" Prova {trial+1} (nessun rumore): vince candidato {w+1}")
95     print(f" -> Vincitore ripetibile: bias numerico dell'implementazione, "
96           f"non vera casualita'.")
97
98     print("\n" + "=" * 65)

```

```

99     print("TEST 1b: pesi UGUALI, con rumore CONTINUO sufficiente")
100     print("=" * 65)
101     winners_noise = []
102     N_TRIALS = 12
103     for trial in range(N_TRIALS):
104         _, rB_f, _ = run_competition(0.3, [1.0, 1.0, 1.0], seed_noise=trial+1)
105         w = find_winner(rB_f)
106         winners_noise.append(w)
107     counts = [winners_noise.count(i) for i in range(3)]
108     print(f" Su {N_TRIALS} prove: candidato 1 vince {counts[0]}x, "
109           f" candidato 2 vince {counts[1]}x, candidato 3 vince {counts[2]}x")
110     print(" -> Con rumore sufficiente la rottura di simmetria e' variabile "
111           "(non perfettamente uniforme 1/3-1/3-1/3).")
112
113     print("\n" + "=" * 65)
114     print("TEST 2: un candidato ha peso MAGGIORE -- vince sempre quello?")
115     print("=" * 65)
116     weighted_results = {}
117     for favored in range(3):
118         weights = [0.5, 0.5, 0.5]
119         weights[favored] = 1.5
120         wins = []
121         for trial in range(6):
122             _, rB_f, _ = run_competition(0.3, weights, seed_noise=trial+100)
123             wins.append(find_winner(rB_f))
124         weighted_results[favored] = wins
125     print(f" Candidato favorito {favored+1} (peso 1.5 vs 0.5): "
126           f"vincitori nelle 6 prove = {[w+1 for w in wins]}")
127
128     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
129
130     axes[0].bar(['Candidato 1', 'Candidato 2', 'Candidato 3'], counts,
131                color=['#4a5aff', '#ff6b35', '#2ecc71'])
132     axes[0].set_ylabel(f'Vittorie su {N_TRIALS} prove')
133     axes[0].set_title('Pesi uguali + rumore: rottura di simmetria casuale')
134     axes[0].grid(axis='y', alpha=0.3)
135
136     _, _, history_example = run_competition(0.3, [0.5, 1.5, 0.5], seed_noise=42)
137     t_axis = np.arange(n_steps)*dt
138     im = axes[1].imshow(history_example.T, aspect='auto', origin='lower', cmap='viridis',
139                        extent=[0, T_total, 0, 1])
140     for off in CANDIDATE_OFFSETS:
141         axes[1].axhline((0.3+off) % 1.0, color='white', linestyle=':', linewidth=0.8, alpha
142                        =0.6)
143     axes[1].set_xlabel('Tempo (ms)')
144     axes[1].set_ylabel('Posizione su Ring B')
145     axes[1].set_title('Competizione: candidato 2 (peso maggiore) vince,\n'altri due soppressi')
146
147     plt.colorbar(im, ax=axes[1], label='r_B(x,t)')
148
149     plt.tight_layout()
150     plt.savefig('competing_targets.png', dpi=150)
151     print("\nGrafico salvato in competing_targets.png")

```

5.3 Reinforcement learning: the network learns which alternative to choose

The third experiment in this phase makes the weights of the three candidate maps no longer fixed, but MODIFIABLE based on an external reward signal — the project's first genuine, if small, reinforcement-learning loop. On every trial: A is stimulated, B competes among the three candidates and selects a winner (using the same competition mechanism as Section 5.2, including the

continuous noise needed for exploration); a reward signal is provided (+1 if the winner matches the "correct" candidate defined a priori by an external rule hidden from the system, 0 otherwise); the weight of the CHOSEN candidate is updated toward the value of the reward received, following a Rescorla-Wagner-type update rule (one of the historically most influential models of associative learning in experimental psychology, from 1972, and also one of the fundamental conceptual building blocks of modern reinforcement learning):

$$chosen_weight \leftarrow chosen_weight + learning_rate \times (reward - chosen_weight)$$

No other weight is touched on any given trial — the system learns EXCLUSIVELY from its own actual choices, not from an explicit comparison with what the correct alternative would have been had it not been chosen (this is a standard feature of reinforcement-learning algorithms: one learns only from the outcome of the action actually taken, not from an "oracle" that always reveals the optimal answer regardless of the choice made).

Result, starting from all-equal weights (0.5, 0.5, 0.5) with candidate 2 defined as "correct": over 60 training trials, accuracy (the probability of choosing the correct candidate) rose from about 80% in the first 10 trials to 100% in the last 10, with the correct candidate's weight climbing to saturate at 1.0 while the other two settled at 0.425. The result was checked for generality by repeating the whole experiment with each of the other two candidates defined as "correct" in turn: in both additional cases, final accuracy reached 100%, with the respective correct candidate's weight climbing to saturation — confirming that the learning mechanism works regardless of which specific alternative is defined as correct, and is not an artifact tied to one particular candidate.

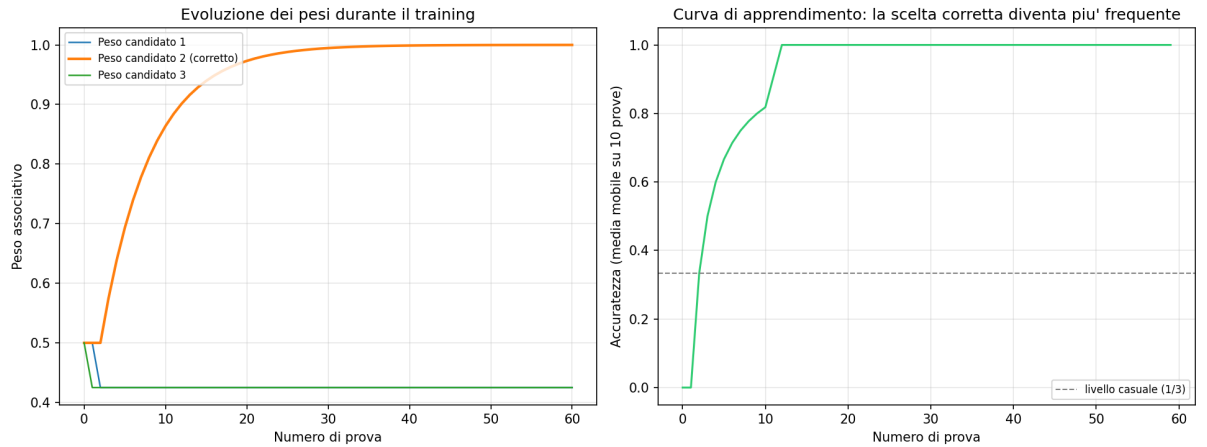


Fig. 17 — Evolution of weights during training (left); accuracy learning curve (right).

Source code: `rl_bandit.py`

```

1  """
2  Estensione di competing_targets: i pesi delle 3 mappe candidate diventano
3  appresi, update tipo Rescorla-Wagner sul solo candidato scelto:
4  w_scelto += lr*(reward - w_scelto), reward binario (1 se vince il candidato
5  corretto, fissato esternamente e non noto al sistema). Nessun update sugli
6  altri pesi -- solo credit assignment sulla propria scelta.
7  """
8
9  import numpy as np
10 import matplotlib
11 matplotlib.use('Agg')
12 import matplotlib.pyplot as plt
13
14
15 def circ_dist(d):
16     return 0.5 - np.abs(np.abs(d) - 0.5)
17
18
19 N = 200
20 x = np.arange(N) / N
21 tau = 10.0
22 dt = 0.2
23 T_total = 500
24 n_steps = int(T_total/dt)
25
26 A_exc = 20.0
27 sigma_exc = 0.05
28 global_inhib = 5.0
29 r_max = 5.0
30 stim_strength = 3.0
31 stim_width = 0.03
32 stim_duration_ms = 80
33
34 d_matrix = circ_dist(x[:, None] - x[None, :])
35 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
36
37 CANDIDATE_OFFSETS = [0.15, 0.45, 0.75]
38 COUPLING_STRENGTH = 1.3
39 NOISE_STD = 0.15 # necessario per esplorare le alternative (vedi competing_targets)
40
41
42 def F(u):
43     return np.clip(u, 0, r_max)
44
45
46 def run_trial(stim_center_A, weights, seed_noise):
47     rA = np.zeros(N)
48     rB = np.zeros(N)
49     stim_duration_steps = int(stim_duration_ms/dt)
50     stim_profile = stim_strength * np.exp(-circ_dist(x-stim_center_A)**2/(2*stim_width**2))
51     rng = np.random.RandomState(seed_noise)
52
53     for step in range(n_steps):
54         I_ext_A = stim_profile if step < stim_duration_steps else 0.0
55         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
56         rA = rA + dt*(-rA + F(rec_A + I_ext_A)) / tau
57
58         cross_input = np.zeros(N)
59         if rA.max() > 0.5:
60             peak_A = x[np.argmax(rA)]
61             for offset, w in zip(CANDIDATE_OFFSETS, weights):
62                 target = (peak_A + offset) % 1.0
63                 cross_input += w * COUPLING_STRENGTH * rA.max() * np.exp(
64                     -circ_dist(x-target)**2/(2*stim_width**2))
65                 cross_input += rng.normal(0, NOISE_STD, N)
66
67         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
68         rB = rB + dt*(-rB + F(rec_B + cross_input)) / tau

```

```

69     if rB.max() < 0.5:
70         return None
71     winner_pos = x[np.argmax(rB)]
72     dists = [circ_dist(winner_pos - ((stim_center_A+off) % 1.0)) for off in CANDIDATE_OFFSETS
73              ]
74     return int(np.argmin(dists))
75
76
77 # candidato corretto fissato a priori, il sistema deve scoprirlo per tentativi
78 CORRECT_CANDIDATE = 1 # candidato 2 (indice 1)
79 LEARNING_RATE = 0.15
80 N_TRIALS = 60
81 STIM_CENTER = 0.3
82
83 weights = np.array([0.5, 0.5, 0.5]) # pesi iniziali uguali
84 weight_history = [weights.copy()]
85 choices = []
86 rewards = []
87
88 print(f"Candidato corretto (nascosto al sistema): {CORRECT_CANDIDATE+1}")
89 print(f"Pesi iniziali: {weights}\n")
90
91 for trial in range(N_TRIALS):
92     winner = run_trial(STIM_CENTER, weights, seed_noise=trial+1)
93     if winner is None:
94         choices.append(-1)
95         rewards.append(0)
96         continue
97     reward = 1.0 if winner == CORRECT_CANDIDATE else 0.0
98     weights[winner] += LEARNING_RATE * (reward - weights[winner])
99     weights = np.clip(weights, 0.05, 2.0) # evita pesi negativi/instabili
100
101     choices.append(winner)
102     rewards.append(reward)
103     weight_history.append(weights.copy())
104
105     if trial % 10 == 0 or trial == N_TRIALS-1:
106         print(f"Prova {trial+1:3d}: scelto candidato {winner+1}, "
107               f"ricompensa={reward:.0f}, pesi ora={np.round(weights,3)}")
108
109 weight_history = np.array(weight_history)
110 choices = np.array(choices)
111
112 # accuratezza in finestre di 10 prove
113 window = 10
114 accuracy_curve = [np.mean(np.array(choices[max(0,k-window):k+1]) == CORRECT_CANDIDATE)
115                   for k in range(len(choices))]
116
117 print(f"\nAccuratezza nelle prime {window} prove: "
118       f"{np.mean(choices[:window]==CORRECT_CANDIDATE)*100:.0f}%")
119 print(f"Accuratezza nelle ultime {window} prove: "
120       f"{np.mean(choices[-window:]==CORRECT_CANDIDATE)*100:.0f}%")
121 print(f"Pesi finali: {np.round(weights,3)} "
122       f"(il candidato corretto, {CORRECT_CANDIDATE+1}, dovrebbe avere il peso piu alto)")
123
124 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
125
126 for i in range(3):
127     axes[0].plot(weight_history[:, i], label=f'Peso candidato {i+1}'
128                 + (' (corretto)' if i == CORRECT_CANDIDATE else ''),
129                 linewidth=2 if i == CORRECT_CANDIDATE else 1.2)
130 axes[0].set_xlabel('Numero di prova')
131 axes[0].set_ylabel('Peso associativo')
132 axes[0].set_title('Evoluzione dei pesi durante il training')
133 axes[0].legend(loc='upper left', fontsize=8)
134 axes[0].grid(alpha=0.3)
135

```

```

136 axes[1].plot(accuracy_curve, color='#2ecc71', linewidth=1.5)
137 axes[1].axhline(1/3, color='gray', linestyle='--', linewidth=1,
138               label='livello casuale (1/3)')
139 axes[1].set_xlabel('Numero di prova')
140 axes[1].set_ylabel(f'Accuratezza (media mobile su {window} prove)')
141 axes[1].set_title('Curva di apprendimento: la scelta corretta diventa piu\' frequente')
142 axes[1].legend(loc='lower right', fontsize=8)
143 axes[1].grid(alpha=0.3)
144 axes[1].set_ylim(-0.05, 1.05)
145
146 plt.tight_layout()
147 plt.savefig('rl_bandit.png', dpi=150)
148 print("\nGrafico salvato in rl_bandit.png")

```

5.4 Contextual policy: different responses for different contexts

The project's final experiment tackles a task significantly harder than the three-armed bandit of Section 5.3: instead of a single fixed correct association to discover, the position of the stimulus on A now defines a CONTEXT among three possible ones, and EACH context has its own correct response, different from the others:

Context	Stimulus position on A	Correct response
Context 1	x=0.10	Candidate 1
Context 2	x=0.50	Candidate 2
Context 3	x=0.80	Candidate 3

The system must learn THREE associations at once, not just one — and, more important conceptually, it must demonstrate that it has learned to TELL THE CONTEXTS APART, not simply develop a generic preference for one candidate regardless of the context presented. This requires a mechanism different from a single weight vector: a weight TABLE indexed both by context and by candidate (weights[context][candidate]), an elementary form of context-conditioned associative memory — conceptually the first step toward a real decision policy (one that chooses the action as a function of the observed STATE, rather than always the same action regardless of state).

During training (150 trials in total, with the context chosen at random on every trial among the three possible ones, so that the system has to learn all three associations in parallel rather than in separate sequence), only the row of the table corresponding to the context actually presented is updated on each trial.

Final evaluation, with a clean test and no further weight updates (10 trials per context): all three contexts reached 100% accuracy on their own correct candidate, with the final weight table showing a clearly dominant diagonal (the three values corresponding to the correct associations, close to 1.0) relative to every other entry in the table (which stayed close to the initial value of 0.5, or slightly below).

Context	Final accuracy	Final weights (candidate 1, 2, 3)
Context 1 ($x=0.10$)	100%	[1.000, 0.500, 0.500]
Context 2 ($x=0.50$)	100%	[0.500, 1.000, 0.500]
Context 3 ($x=0.80$)	100%	[0.425, 0.425, 1.000]

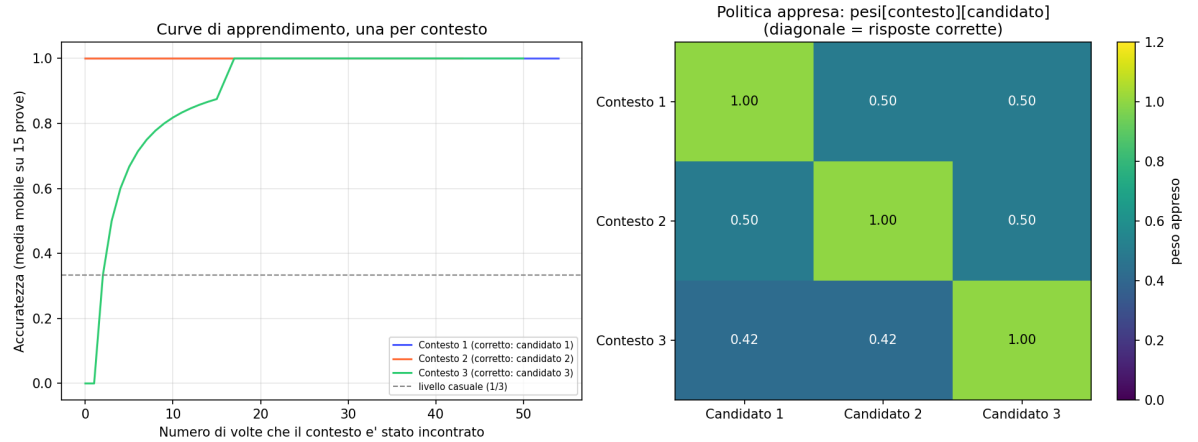


Fig. 18 — Per-context learning curves (left); final learned policy, $\text{weights}[\text{context}][\text{candidate}]$ table (right).

Why this result is conceptually more advanced than the simple bandit of Section 5.3: a system that only has to learn a single correct response could, in principle, "get away with" a trivial shortcut (a fixed preference, independent of any incoming information). A system that must **TELL DIFFERENT CONTEXTS APART** and respond differently to each cannot fall back on this shortcut — it must necessarily represent, in some form however minimal (here, the corresponding row of the table), "which situation I am in" before it can act accordingly. It is, however rudimentary in its concrete implementation, the conceptual difference between a fixed reflex and the beginning of a decision conditioned on the observed context.

Source code: `contextual_policy.py`

```

1  """
2  Estensione del bandito a 3 braccia (rl_bandit.py) a 3 contesti indipendenti:
3  la posizione di stimolo su A (0.1/0.5/0.8) seleziona quale delle 3 risposte
4  e' corretta. Tabella pesi[contesto][candidato] invece di un vettore unico --
5  verifica a fine training che il sistema distingua i contesti, non solo
6  preferisca un candidato in generale.
7  """
8
9  import numpy as np
10 import matplotlib
11 matplotlib.use('Agg')

```



```

12 import matplotlib.pyplot as plt
13
14
15 def circ_dist(d):
16     return 0.5 - np.abs(np.abs(d) - 0.5)
17
18
19 N = 200
20 x = np.arange(N) / N
21 tau = 10.0
22 dt = 0.2
23 T_total = 500
24 n_steps = int(T_total/dt)
25
26 A_exc = 20.0
27 sigma_exc = 0.05
28 global_inhib = 5.0
29 r_max = 5.0
30 stim_strength = 3.0
31 stim_width = 0.03
32 stim_duration_ms = 80
33
34 d_matrix = circ_dist(x[:, None] - x[None, :])
35 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
36
37 CANDIDATE_OFFSETS = [0.15, 0.45, 0.75]
38 COUPLING_STRENGTH = 1.3
39 NOISE_STD = 0.15
40
41
42 def F(u):
43     return np.clip(u, 0, r_max)
44
45
46 def run_trial(stim_center_A, weights, seed_noise):
47     rA = np.zeros(N)
48     rB = np.zeros(N)
49     stim_duration_steps = int(stim_duration_ms/dt)
50     stim_profile = stim_strength * np.exp(-circ_dist(x-stim_center_A)**2/(2*stim_width**2))
51     rng = np.random.RandomState(seed_noise)
52
53     for step in range(n_steps):
54         I_ext_A = stim_profile if step < stim_duration_steps else 0.0
55         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
56         rA = rA + dt*(-rA + F(rec_A + I_ext_A)) / tau
57
58         cross_input = np.zeros(N)
59         if rA.max() > 0.5:
60             peak_A = x[np.argmax(rA)]
61             for offset, w in zip(CANDIDATE_OFFSETS, weights):
62                 target = (peak_A + offset) % 1.0
63                 cross_input += w * COUPLING_STRENGTH * rA.max() * np.exp(
64                     -circ_dist(x-target)**2/(2*stim_width**2))
65                 cross_input += rng.normal(0, NOISE_STD, N)
66
67         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
68         rB = rB + dt*(-rB + F(rec_B + cross_input)) / tau
69
70         if rB.max() < 0.5:
71             return None
72         winner_pos = x[np.argmax(rB)]
73         dists = [circ_dist(winner_pos - ((stim_center_A+off) % 1.0)) for off in CANDIDATE_OFFSETS]
74         return int(np.argmin(dists))
75
76
77 CONTEXTS = [0.1, 0.5, 0.8]
78 CORRECT_PER_CONTEXT = [0, 1, 2]

```

```

79 LEARNING_RATE = 0.15
80 N_TRIALS = 150 # piu' prove del bandito singolo: 3 associazioni da imparare
81
82 weights_table = np.full((3, 3), 0.5)
83 history = np.zeros((N_TRIALS, 3, 3))
84 choices_per_context = {0: [], 1: [], 2: []}
85
86 rng_context = np.random.RandomState(0)
87
88 print("Contesti e risposte corrette (nascoste al sistema):")
89 for c, correct in enumerate(CORRECT_PER_CONTEXT):
90     print(f"Contesto {c+1} (stimolo a x={CONTEXTS[c]}) -> risposta corretta: candidato {
91         correct+1}")
92
93 print()
94
95 for trial in range(N_TRIALS):
96     context = rng_context.randint(0, 3)
97     stim_pos = CONTEXTS[context]
98     correct = CORRECT_PER_CONTEXT[context]
99
100     winner = run_trial(stim_pos, weights_table[context], seed_noise=trial+1)
101     if winner is not None:
102         reward = 1.0 if winner == correct else 0.0
103         weights_table[context, winner] += LEARNING_RATE * (reward - weights_table[context,
104             winner])
105         weights_table = np.clip(weights_table, 0.05, 2.0)
106         choices_per_context[context].append(1 if winner == correct else 0)
107
108     history[trial] = weights_table.copy()
109
110     if trial % 30 == 0 or trial == N_TRIALS-1:
111         print(f"Prova {trial+1:3d} (contesto {context+1}): "
112             f"pesi contesto {context+1} ora = {np.round(weights_table[context],3)}")
113
114 print("\n" + "=" * 65)
115 print("VALUTAZIONE FINALE (pesi appresi, test pulito senza aggiornamento)")
116 print("=" * 65)
117
118 for c in range(3):
119     test_wins = []
120     for t in range(10):
121         w = run_trial(CONTEXTS[c], weights_table[c], seed_noise=9000+c*10+t)
122         test_wins.append(w)
123     acc = np.mean(np.array(test_wins) == CORRECT_PER_CONTEXT[c])
124     print(f"Contesto {c+1} (x={CONTEXTS[c]}): accuratezza sul candidato corretto "
125         f"({CORRECT_PER_CONTEXT[c]+1}) = {acc*100:.0f}% | pesi finali = {np.round(
126             weights_table[c],3)}")
127
128 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
129
130 window = 15
131 colors = ['#4a5aff', '#ff6b35', '#2ecc71']
132
133 for c in range(3):
134     seq = choices_per_context[c]
135     curve = [np.mean(seq[max(0, k-window):k+1]) for k in range(len(seq))]
136     axes[0].plot(curve, color=colors[c], linewidth=1.5,
137         label=f'Contesto {c+1} (corretto: candidato {CORRECT_PER_CONTEXT[c]+1})')
138 axes[0].axhline(1/3, color='gray', linestyle='--', linewidth=1, label='livello casuale (1/3)')
139
140 axes[0].set_xlabel(f'Numero di volte che il contesto e\' stato incontrato')
141 axes[0].set_ylabel(f'Accuratezza (media mobile su {window} prove)')
142 axes[0].set_title('Curve di apprendimento, una per contesto')
143 axes[0].legend(loc='lower right', fontsize=7)
144 axes[0].grid(alpha=0.3)
145 axes[0].set_ylim(-0.05, 1.05)
146
147 im = axes[1].imshow(weights_table, cmap='viridis', vmin=0, vmax=1.2, aspect='auto')
148 axes[1].set_xticks(range(3)); axes[1].set_xticklabels(['Candidato 1', 'Candidato 2', '
149     Candidato 3'])

```

```

142 axes[1].set_yticks(range(3)); axes[1].set_yticklabels([f'Contesto {c+1}' for c in range(3)])
143 axes[1].set_title('Politica appresa: pesi[contesto][candidato]\n(diagonale = risposte
    corrette)')
144 for c in range(3):
145     for cand in range(3):
146         axes[1].text(cand, c, f'{weights_table[c,cand]:.2f}', ha='center', va='center',
147                        color='white' if weights_table[c,cand] < 0.8 else 'black', fontsize=10)
148 plt.colorbar(im, ax=axes[1], label='peso appreso')
149
150 plt.tight_layout()
151 plt.savefig('contextual_policy.png', dpi=150)
152 print("\nGrafico salvato in contextual_policy.png")

```

5.5 Learned bidirectional communication: from the fixed channel to the synapse that learns

Section 5.1 built a one-way $A \rightarrow B$ chain between two ring attractors, with a projection map (a fixed offset around the ring) hand-written into the code: B never had any influence on A. This subsection picks up exactly that point and extends it in two directions that Chapter 9 lists among the future developments: making the channel bidirectional, and turning the projection map into a real synapse, learned rather than imposed from outside. The path is reported in full, including the difficulties encountered (in the same spirit of methodological transparency as Section 4.4.1), and runs across four successive scripts.

5.5.1 A turn-based exchange with fixed channels

First step, deliberately as simple as possible: two fixed but DIFFERENT projection channels in the two directions ($A \rightarrow B$ with offset 0.3, $B \rightarrow A$ with offset 0.2 -- with a single identical map in both directions the exchange would always return to the same point and there would be nothing to observe). The two rings exchange turns explicitly instead of co-evolving continuously, since otherwise the dynamics converge almost immediately to a single fused state.

Result: an initial message on A generates an exchange that does NOT converge to a fixed point within 6 turns, but oscillates. The net displacement for each complete round trip was found empirically to be about 0.5 -- exactly the sum of the two offsets (0.3+0.2), quantitatively confirming an analytical prediction rather than merely observing it on a graph. A control test (return channel disabled) confirms that without $B \rightarrow A$, A stays still after a brief initial settling, consistent with the behavior already documented in 5.1.

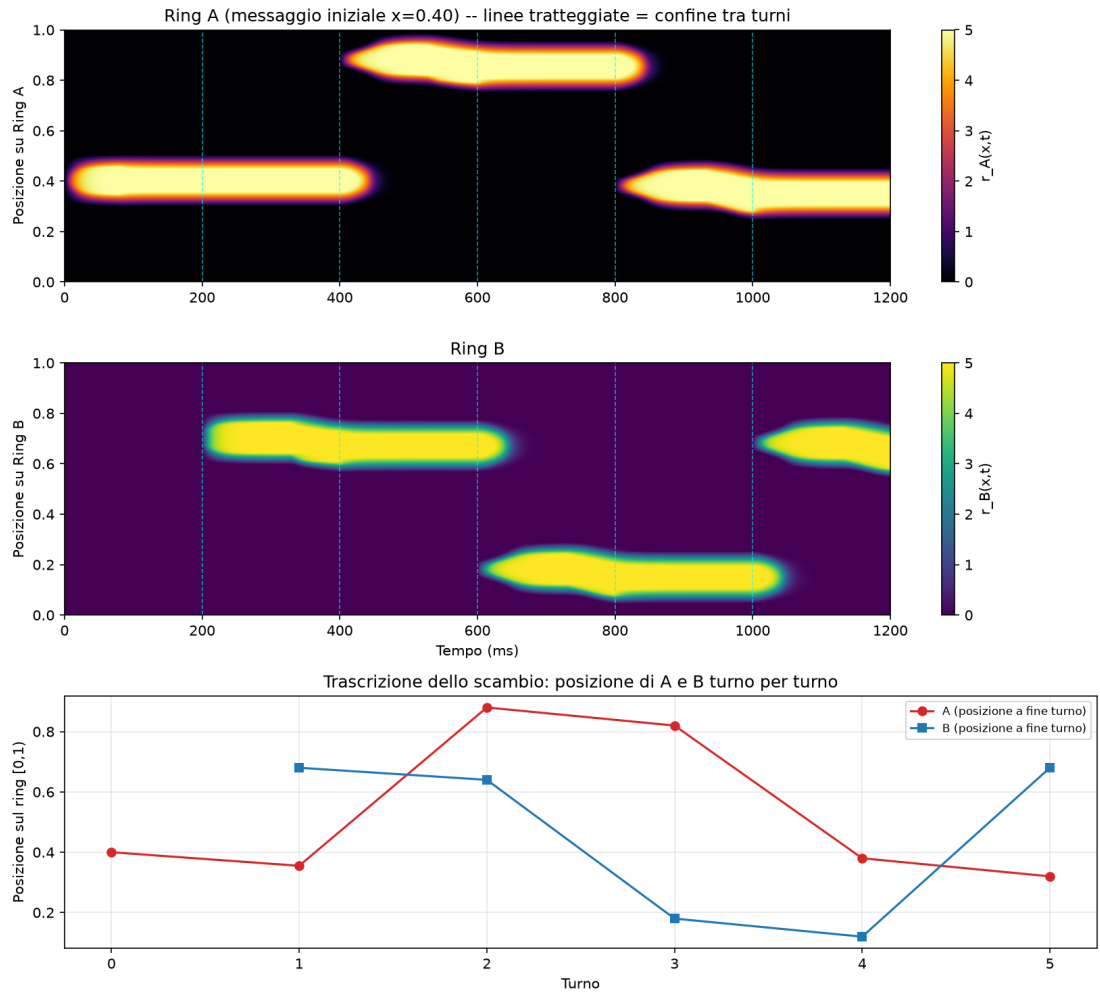


Fig. 21 — Turn-based exchange with fixed channels: raster of the two rings and turn-by-turn position transcript.

Source code: `dialogo_ring_attractor.py`

```
1 """
2 Estensione di chained_attractors.py: la' A->B era a senso unico (mappa a
3 offset fissa, B non ha mai influenza su A). Qui la connessione e'
4 bidirezionale con DUE mappe di offset distinte (A->B e B->A), e i due ring
5 si scambiano il turno esplicitamente invece di co-evolvere in continuo --
6 altrimenti la dinamica converge quasi subito a un unico stato fuso e non
7 c'e' nulla da osservare come "scambio".
8
9 Un'informazione iniziale (un valore scalare in [0,1], lo stesso formalismo
10 gia' usato per lo stimolo su A negli script precedenti) viene data solo ad
11 A al turno 0. Da li' in poi nessun altro ingresso esterno: ogni turno
12 successivo il ring che "ascolta" riceve solo la proiezione della posizione
13 corrente dell'altro ring. Si registra la posizione del bump a ogni turno --
14 la "trascrizione" dello scambio -- e si verifica empiricamente cosa succede
```

```

15 | davvero (convergenza a un punto fisso, ciclo stabile, o deriva), senza
16 | assumerlo a priori.
17 | """
18 |
19 | import numpy as np
20 | import matplotlib
21 | matplotlib.use('Agg')
22 | import matplotlib.pyplot as plt
23 |
24 |
25 | def circ_dist(d):
26 |     return 0.5 - np.abs(np.abs(d) - 0.5)
27 |
28 |
29 | N = 200
30 | x = np.arange(N) / N
31 | tau = 10.0
32 | dt = 0.2
33 |
34 | A_exc = 20.0
35 | sigma_exc = 0.05
36 | global_inhib = 5.0
37 | r_max = 5.0
38 | stim_strength = 3.0
39 | stim_width = 0.03
40 | stim_duration_ms = 80
41 |
42 | d_matrix = circ_dist(x[:, None] - x[None, :])
43 | J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
44 |
45 | OFFSET_AB = 0.3 # come B interpreta cio' che sente da A
46 | OFFSET_BA = 0.2 # come A interpreta cio' che sente da B (diversa da AB apposta:
47 |                 # con due mappe uguali lo scambio tornerebbe sempre allo stesso punto)
48 | COUPLING_STRENGTH = 1.3
49 |
50 | T_TURN = 200 # ms per turno
51 | N_TURNS = 6 # turno 0 = messaggio iniziale su A, poi 5 turni di ascolto alternato
52 | steps_per_turn = int(T_TURN/dt)
53 |
54 |
55 | def F(u):
56 |     return np.clip(u, 0, r_max)
57 |
58 |
59 | def bump_pos(r):
60 |     return x[np.argmax(r)] if r.max() > 0.5 else None
61 |
62 |
63 | def run_dialogo(messaggio, bidirezionale=True):
64 |     """Turno 0: messaggio esterno su A. Turni dispari: B ascolta A.
65 |     Turni pari (>0): A ascolta B (solo se bidirezionale=True)."""
66 |     rA = np.zeros(N)
67 |     rB = np.zeros(N)
68 |     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
69 |     stim_duration_steps = int(stim_duration_ms/dt)
70 |
71 |     history_A = np.zeros((N_TURNS*steps_per_turn, N))
72 |     history_B = np.zeros((N_TURNS*steps_per_turn, N))
73 |     pos_A_turni = []
74 |     pos_B_turni = []
75 |
76 |     for turn in range(N_TURNS):
77 |         listener = 'B' if turn % 2 == 1 else ('A' if turn > 0 else None)
78 |
79 |         for step_in_turn in range(steps_per_turn):
80 |             step = turn*steps_per_turn + step_in_turn
81 |
82 |             I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else

```

```

83         0.0
84
85         cross_to_A = np.zeros(N)
86         if listener == 'A' and bidirezionale:
87             posB = bump_pos(rB)
88             if posB is not None:
89                 target = (posB + OFFSET_BA) % 1.0
90                 cross_to_A = COUPLING_STRENGTH * rB.max() * np.exp(
91                     -circ_dist(x-target)**2/(2*stim_width**2))
92
93         cross_to_B = np.zeros(N)
94         if listener == 'B':
95             posA = bump_pos(rA)
96             if posA is not None:
97                 target = (posA + OFFSET_AB) % 1.0
98                 cross_to_B = COUPLING_STRENGTH * rA.max() * np.exp(
99                     -circ_dist(x-target)**2/(2*stim_width**2))
100
101         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
102         rA = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A)) / tau
103
104         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
105         rB = rB + dt*(-rB + F(rec_B + cross_to_B)) / tau
106
107         history_A[step] = rA
108         history_B[step] = rB
109
110         pos_A_turni.append(bump_pos(rA))
111         pos_B_turni.append(bump_pos(rB))
112
113     return pos_A_turni, pos_B_turni, history_A, history_B
114
115 if __name__ == '__main__':
116     print("=" * 70)
117     print("TEST 1: scambio bidirezionale per diversi messaggi iniziali su A")
118     print("=" * 70)
119     messaggi_test = [0.10, 0.40, 0.70]
120     for msg in messaggi_test:
121         pos_A, pos_B, _, _ = run_dialogo(msg)
122         trascrizione = []
123         for t in range(N_TURNS):
124             chi = 'A (messaggio)' if t == 0 else ('B ascolta' if t % 2 == 1 else 'A ascolta')
125             pa = f"{pos_A[t]:.3f}" if pos_A[t] is not None else "nessun bump"
126             pb = f"{pos_B[t]:.3f}" if pos_B[t] is not None else "nessun bump"
127             trascrizione.append(f" turno {t} ({chi}): A={pa} B={pb}")
128         print(f"\n Messaggio iniziale x={msg:.2f}:")
129         print("\n".join(trascrizione))
130
131     print("\n" + "=" * 70)
132     print("TEST 2: controllo -- senza canale B->A, A cambia mai dopo il turno 0?")
133     print("=" * 70)
134     pos_A_uni, pos_B_uni, _, _ = run_dialogo(0.40, bidirezionale=False)
135     # turno 0->1: il bump si assesta dallo stimolo grezzo alla sua posizione di riposo
136     # (il bias di assestamento del ring attractor e' un artefatto gia' noto e documentato
137     # nel resoconto del progetto, sezione 9 -- non e' quello che questo test verifica).
138     # Qui verifichiamo che DOPO l'assestamento (turni 1..5) A resti davvero fissa,
139     # dato che senza canale B->A non ha piu' alcun ingresso.
140     pos_A_dopo_assestamento = [p for p in pos_A_uni[1:] if p is not None]
141     stabile = all(abs(circ_dist(p - pos_A_uni[1])) < 0.01 for p in pos_A_dopo_assestamento)
142     print(f" Posizioni di A nei turni successivi (solo A->B attivo): "
143           f"{f'{p:.3f}' if p is not None else None for p in pos_A_uni}")
144     print(f" (turno 0->1: assestamento naturale del bump, atteso; "
145           f"da qui in poi nessun ingresso esterno raggiunge A)")
146     if stabile:
147         print(" >>> Confermato: dopo l'assestamento iniziale, senza il canale di ritorno "
148               "A resta fissa (come in chained_attractors.py). <<<")
149     else:

```

```

150     print(" >>> ATTENZIONE: A si muove anche dopo l'assestamento e senza canale di "
151           "ritorno -- controllare il codice, non dovrebbe succedere. <<<")
152
153     print("\n" + "=" * 70)
154     print("TEST 3: lo scambio bidirezionale si stabilizza o continua a variare?")
155     print("=" * 70)
156     pos_A_bi, pos_B_bi, history_A, history_B = run_dialogo(0.40, bidirezionale=True)
157     ultimo_giro = circ_dist(pos_A_bi[-1] - pos_A_bi[-3]) if pos_A_bi[-1] is not None and
        pos_A_bi[-3] is not None else None
158     if ultimo_giro is not None:
159         print(f" Spostamento di A tra il penultimo e l'ultimo suo turno di ascolto: "
160               f"{abs(ultimo_giro):.4f}")
161         if abs(ultimo_giro) < 0.01:
162             print(" >>> Lo scambio converge a un punto fisso (un 'accordo'). <<<")
163         else:
164             print(" >>> Lo scambio NON converge in 6 turni: la posizione continua "
165                   "a variare a ogni giro (deriva o ciclo). <<<")
166
167     # figura: spazio-tempo dei due ring + trascrizione posizione-per-turno
168     t_axis = np.arange(N_TURNS*steps_per_turn*dt)
169     fig, axes = plt.subplots(3, 1, figsize=(11, 10), sharex=False)
170
171     im0 = axes[0].imshow(history_A.T, aspect='auto', origin='lower', cmap='inferno',
172                          extent=[0, N_TURNS*T_TURN, 0, 1])
173     for t in range(1, N_TURNS):
174         axes[0].axvline(t*T_TURN, color='cyan', linestyle='--', linewidth=0.8, alpha=0.6)
175     axes[0].set_ylabel('Posizione su Ring A')
176     axes[0].set_title(f'Ring A (messaggio iniziale x=0.40) -- linee tratteggiate = confine
        tra turni')
177     plt.colorbar(im0, ax=axes[0], label='r_A(x,t)')
178
179     im1 = axes[1].imshow(history_B.T, aspect='auto', origin='lower', cmap='viridis',
180                          extent=[0, N_TURNS*T_TURN, 0, 1])
181     for t in range(1, N_TURNS):
182         axes[1].axvline(t*T_TURN, color='cyan', linestyle='--', linewidth=0.8, alpha=0.6)
183     axes[1].set_xlabel('Tempo (ms)')
184     axes[1].set_ylabel('Posizione su Ring B')
185     axes[1].set_title('Ring B')
186     plt.colorbar(im1, ax=axes[1], label='r_B(x,t)')
187
188     turni = list(range(N_TURNS))
189     pa_plot = [p if p is not None else np.nan for p in pos_A_bi]
190     pb_plot = [p if p is not None else np.nan for p in pos_B_bi]
191     axes[2].plot(turni, pa_plot, 'o-', color='C3', label='A (posizione a fine turno)')
192     axes[2].plot(turni, pb_plot, 's-', color='C0', label='B (posizione a fine turno)')
193     axes[2].set_xlabel('Turno')
194     axes[2].set_ylabel('Posizione sul ring [0,1)')
195     axes[2].set_title('Trascrizione dello scambio: posizione di A e B turno per turno')
196     axes[2].legend(loc='best', fontsize=8)
197     axes[2].grid(alpha=0.3)
198
199     plt.tight_layout()
200     plt.savefig('dialogo_ring_attractor.png', dpi=150)
201     print("\nGrafico salvato in dialogo_ring_attractor.png")

```

5.5.2 Learned synapses with turn-grained Hebbian reinforcement

Here the two offset maps become two genuine $N \times N$ weight matrices (W_{AB} and W_{BA}), initialized to an innate bias DELIBERATELY too weak to light up a bump on its own in the listening network (activation threshold verified by bisection: between scale 0.0025, 0% success, and 0.003, 100% success -- the attractor is strongly bistable, with almost no middle ground). A Hebbian reinforcement at the end of each turn strengthens the pair (position of whoever

just spoke, position of whoever just answered), every time the listener actually forms a bump. The chicken-and-egg problem -- without a working connection, a pair to reinforce is never observed -- is solved with an exploratory impulse to a random position during training only: random symmetry breaking, the same mechanism already verified in 5.2 (TEST 1b).

Result for the A->B channel: B's success rate goes from 0% to 100% in about 30-40 episodes (a sigmoidal curve, not an instantaneous jump), and the learned map converges to an offset of 0.195 -- different from the starting innate bias (0.3), a genuinely emergent result rather than an imposed one.

Result for the return channel B->A: it does NOT consolidate in this script. A already has a persistent bump from turn 0 onward (it is an attractor), so B has to DISLodge a bump already at full strength instead of lighting one up from zero -- a much harder task. The weight W_{BA} does grow over time, but by pure statistical coincidence (A's position stays almost always the same for reasons of its own, independent of B), not through any real causal influence: it remains behaviorally ineffective. A negative result verified with code, not hidden -- in the same methodological spirit stated in this document's introduction.

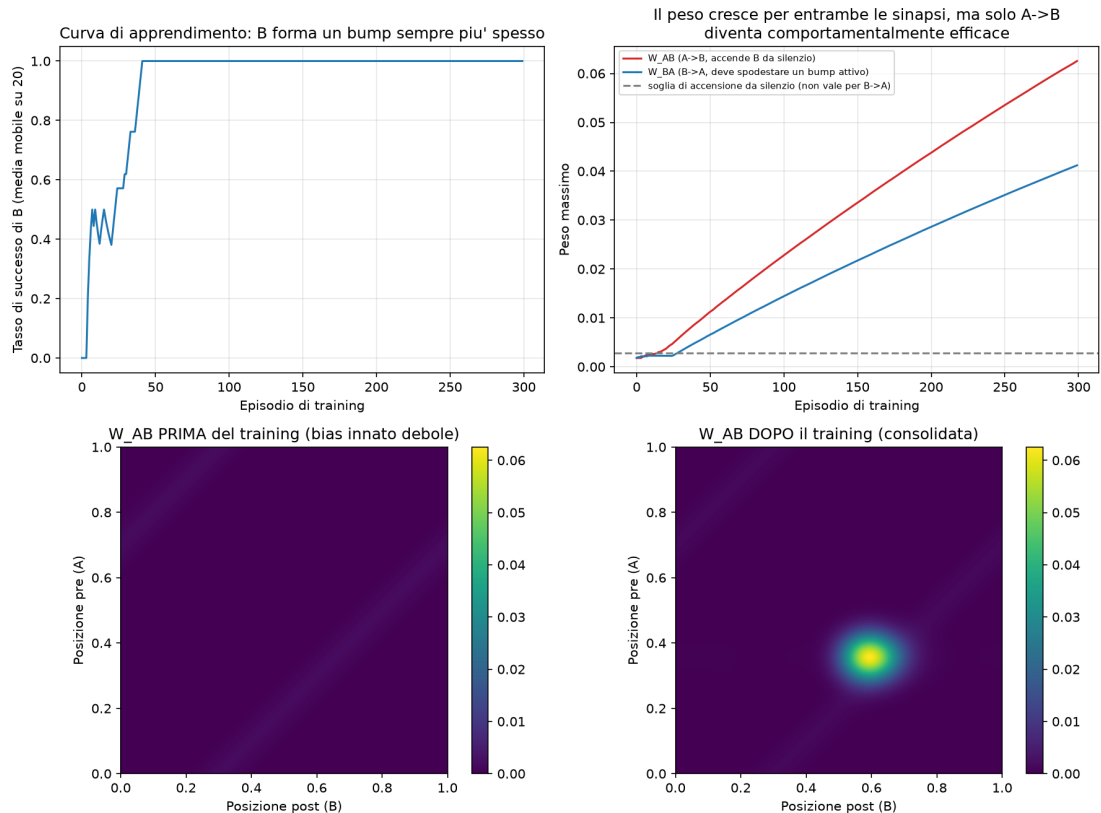


Fig. 22 — Consolidation of the A->B synapse: learning curve and weight map before/after.

Source code: *dialogo_stdp.py*

```

1  """
2  Estensione di dialogo_ring_attractor.py: li' le due mappe di proiezione tra
3  A e B (offset 0.3 e 0.2) erano costanti scritte a mano nel codice. Qui
4  diventano sinapsi vere -- due matrici di peso N x N, W_AB e W_BA -- che
5  partono da un debole bias innato (troppo debole, da solo, per far scattare
6  un bump nella rete che ascolta) e si CONSOLIDANO con un aggiornamento
7  Hebbiano applicato a fine turno: quando la rete che ascolta forma davvero
8  un bump, la coppia (posizione di chi ha appena "parlato", posizione di chi
9  ha appena "risposto") viene rinforzata li' dove e' successa.
10
11  E' una versione a grana grossa -- per-turno, non al livello del singolo
12  spike -- della STDP usata altrove nel progetto (stdp.py,
13  classificazione_stdp.py): stessa idea (rinforzo guidato dalla correlazione
14  temporale tra chi si attiva prima e chi risponde dopo), applicata pero' su
15  una rete a rate invece che a impulsi, per coerenza con la scelta gia' fatta
16  per gli attrattori (ring_attractor_rate.py).
17
18  Il problema dell'uovo e della gallina -- senza una connessione funzionante
19  non si osserva mai una coppia pre/post da rinforzare -- viene risolto con
20  rumore esplorativo nella rete che ascolta SOLO durante il training: rottura
21  di simmetria casuale, lo stesso meccanismo gia' verificato in
22  competing_targets.py (TEST 1b). Con quel rumore un bump puo' comunque
23  comparire per caso anche quando i pesi sono deboli, e quel caso puo' essere
24  rinforzato -- un rich-get-richer Hebbiano, non diverso in spirito dalle
25  onde spontanee che guidano la formazione delle mappe retinotopiche prima
26  che ci sia vera visione.
27  """
28
29  import numpy as np
30  import matplotlib
31  matplotlib.use('Agg')
32  import matplotlib.pyplot as plt
33
34
35  def circ_dist(d):
36      return 0.5 - np.abs(np.abs(d) - 0.5)
37
38
39  N = 200
40  x = np.arange(N) / N
41  tau = 10.0
42  dt = 0.2
43
44  A_exc = 20.0
45  sigma_exc = 0.05
46  global_inhib = 5.0
47  r_max = 5.0
48  stim_strength = 3.0
49  stim_width = 0.03
50  stim_duration_ms = 80
51
52  d_matrix = circ_dist(x[:, None] - x[None, :])
53  J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
54
55  COUPLING_STRENGTH = 1.3
56  T_TURN = 200
57  N_TURNS = 6
58  steps_per_turn = int(T_TURN/dt)
59
60  # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
61  # ma scalato troppo debole per bastare da solo (verificato sotto)
62  OFFSET_AB_INNATO = 0.3
63  OFFSET_BA_INNATO = 0.2

```

```

64 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
65 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
66 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
67 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
68 # solo dal rumore esplorativo, non dal bias innato da solo
69 W_INIT_SCALE = 0.0018
70 SIGMA_W = 0.05
71 W_MAX = 2.5
72 ETA = 0.0001
73 DECAY_PER_EPISODE = 0.999
74 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
75 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
76 # accensione solitaria verificata a strength=0.0036 con questa durata --
77 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
78 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
79 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
80 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
81 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
82 EXPLORATION_PULSE_STRENGTH = 0.0030
83 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
84
85
86 def F(u):
87     return np.clip(u, 0, r_max)
88
89
90 def bump_pos(r):
91     return x[np.argmax(r)] if r.max() > 0.5 else None
92
93
94 def init_weak_bias(offset, scale):
95     target = (x[:, None] + offset) % 1.0
96     d = circ_dist(x[None, :] - target)
97     return scale * np.exp(-d**2/(2*SIGMA_W**2))
98
99
100 def hebbian_bump(pos_pre, pos_post):
101     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
102     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
103     return np.outer(kern_pre, kern_post)
104
105
106 def run_episode(W_AB, W_BA, messaggio, rng, train):
107     rA = np.zeros(N)
108     rB = np.zeros(N)
109     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
110     stim_duration_steps = int(stim_duration_ms/dt)
111
112     pos_A_turni = []
113     pos_B_turni = []
114
115     explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
116
117     for turn in range(N_TURNS):
118         listener = 'B' if turn % 2 == 1 else ('A' if turn > 0 else None)
119
120         if train and listener is not None:
121             explore_pos = rng.uniform(0, 1)
122             explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
123                 -circ_dist(x-explore_pos)**2/(2*stim_width**2))
124         else:
125             explore_profile = np.zeros(N)
126
127         for step_in_turn in range(steps_per_turn):
128             I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
129                 0.0
130
131             explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)

```

```

131         else np.zeros(N)
132     explore_A = explore_now if listener == 'A' else np.zeros(N)
133     explore_B = explore_now if listener == 'B' else np.zeros(N)
134
135     cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
136         N)
137     cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
138         N)
139
140     rec_A = J_exc @ rA / N - global_inhib*rA.mean()
141     rA = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A)) / tau
142
143     rec_B = J_exc @ rB / N - global_inhib*rB.mean()
144     rB = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B)) / tau
145
146     pos_A = bump_pos(rA)
147     pos_B = bump_pos(rB)
148     pos_A_turni.append(pos_A)
149     pos_B_turni.append(pos_B)
150
151     if train and listener == 'B' and pos_A is not None and pos_B is not None:
152         W_AB += ETA * hebbian_bump(pos_A, pos_B)
153         np.clip(W_AB, 0, W_MAX, out=W_AB)
154     elif train and listener == 'A' and pos_A is not None and pos_B is not None:
155         W_BA += ETA * hebbian_bump(pos_B, pos_A)
156         np.clip(W_BA, 0, W_MAX, out=W_BA)
157
158     return pos_A_turni, pos_B_turni
159
160 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
161     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
162     NOTA: A ha un bump persistente fin dal turno 0 per costruzione (e' un
163     attrattore), quindi "A ha un bump al turno 2" e' quasi sempre vero anche
164     se B non ha mai influenzato nulla. Il confronto va fatto contro il
165     turno 1 (non il turno 0): tra il turno 0 e il turno 1 il bump di A si
166     assesta comunque un po' per conto suo (lo stesso bias di posizione gia'
167     documentato nel resoconto del progetto, sezione 9), quindi usare il
168     turno 0 come riferimento conterebbe quell'assestamento come se fosse una
169     risposta a B. Dal turno 1 al turno 2, invece, senza influenza da B la
170     posizione di A e' gia' assestata e dovrebbe restare ferma -- solo un
171     cambiamento li' indica che l'informazione e' davvero tornata da B."""
172     successi_B, successi_A_spostata = 0, 0
173     pos_B_ok, pos_A_ok = [], []
174     for k in range(n_trial):
175         rng = np.random.RandomState(seed0 + k)
176         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
177         if pos_B[1] is not None:
178             successi_B += 1
179             pos_B_ok.append(pos_B[1])
180         if pos_A[2] is not None and pos_A[1] is not None and abs(circ_dist(pos_A[2]-pos_A[1]))
181             > 0.02:
182             successi_A_spostata += 1
183             pos_A_ok.append(pos_A[2])
184     return {
185         'tasso_B': successi_B/n_trial,
186         'tasso_A_spostata': successi_A_spostata/n_trial,
187         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
188         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
189         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
190     }
191
192 if __name__ == '__main__':
193     MESSAGGIO = 0.40
194     N_EPISODES = 300
195
196     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)

```

```

195 W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
196
197 # soglia di accensione autonoma verificata empiricamente per bisezione
198 # (run_episode reale, non stima lineare): tra scale=0.0025 (0% successo)
199 # e scale=0.0030 (100% successo)
200 SOGLIA_IGNIZIONE_W = 0.0028
201
202 print("=" * 70)
203 print("SETUP: bias innato deliberatamente debole")
204 print("=" * 70)
205 print(f" W_INIT_SCALE={W_INIT_SCALE}, soglia per formare un bump = 0.5")
206 W_AB_iniziale = W_AB.copy()
207 W_BA_iniziale = W_BA.copy()
208
209 print("\n" + "=" * 70)
210 print("BASELINE (pre-training): il bias innato basta da solo a far rispondere B?")
211 print("=" * 70)
212 baseline = eval_batch(W_AB, W_BA, MESSAGGIO, n_trial=20, seed0=0)
213 print(f" Tasso di successo di B (turno 1): {baseline['tasso_B']*100:.0f}%")
214 print(f" Tasso di volte che A si SPOSTA per davvero al turno 2 "
215       f"(risposta reale a B, non solo bump persistente): {baseline['tasso_A_spostata']
216       f'*100:.0f}%")
217
218 print("\n" + "=" * 70)
219 print(f"TRAINING: {N_EPISODES} episodi, messaggio fisso x={MESSAGGIO}, "
220       f"rumore esplorativo attivo, pesi persistenti tra episodi")
221 print("=" * 70)
222 rng_train = np.random.RandomState(42)
223 successo_B_storia = []
224 spostamento_A_storia = []
225 picco_W_AB_storia = []
226 picco_W_BA_storia = []
227 for ep in range(N_EPISODES):
228     pos_A, pos_B = run_episode(W_AB, W_BA, MESSAGGIO, rng_train, train=True)
229     W_AB *= DECAF_PER_EPISODE
230     W_BA *= DECAF_PER_EPISODE
231     successo_B_storia.append(1 if pos_B[1] is not None else 0)
232     spostato = (pos_A[2] is not None and pos_A[1] is not None
233               and abs(circ_dist(pos_A[2]-pos_A[1])) > 0.02)
234     spostamento_A_storia.append(1 if spostato else 0)
235     picco_W_AB_storia.append(W_AB.max())
236     picco_W_BA_storia.append(W_BA.max())
237     if ep % 50 == 0 or ep == N_EPISODES-1:
238         win_B = successo_B_storia[max(0, ep-30):ep+1]
239         win_A = spostamento_A_storia[max(0, ep-30):ep+1]
240         print(f" episodio {ep:3d}: successo B (ultimi 30) = {np.mean(win_B)*100:5.1f}% | "
241               f"A si sposta per B (ultimi 30) = {np.mean(win_A)*100:5.1f}% | "
242               f"picco W_AB = {W_AB.max():.4f} picco W_BA = {W_BA.max():.4f}")
243
244 print("\n" + "=" * 70)
245 print("DOPO IL TRAINING: confronto pulito, pesi congelati, nessun rumore esplorativo")
246 print("=" * 70)
247 dopo = eval_batch(W_AB, W_BA, MESSAGGIO, n_trial=20, seed0=9000)
248 print(f" PRIMA -- tasso successo B: {baseline['tasso_B']*100:5.0f}% "
249       f"std posizione B: {baseline['std_B']}")
250 std_dopo_str = f"{dopo['std_B']:.4f}" if dopo['std_B'] else "None"
251 print(f" DOPO -- tasso successo B: {dopo['tasso_B']*100:5.0f}% "
252       f"std posizione B: {std_dopo_str}")
253 print(f" PRIMA -- tasso di A che si sposta per davvero al turno 2: "
254       f"{baseline['tasso_A_spostata']*100:5.0f}%")
255 print(f" DOPO -- tasso di A che si sposta per davvero al turno 2: "
256       f"{dopo['tasso_A_spostata']*100:5.0f}%")
257 if dopo['media_B'] is not None:
258     offset_appreso = circ_dist(dopo['media_B'] - MESSAGGIO)
259     print(f"\n Offset A->B appreso empiricamente: {offset_appreso:.3f} "
260           f"(bias innato di partenza era {OFFSET_AB_INNATO})")
261
262 print("\n" + "=" * 70)

```

```

262 print("ASIMMETRIA A->B vs B->A")
263 print("=" * 70)
264 print(f" Picco finale W_AB: {W_AB.max():.4f} (soglia di accensione ~{SOGLIA_IGNIZIONE_W})")
265 print(f" Picco finale W_BA: {W_BA.max():.4f} (soglia di accensione ~{SOGLIA_IGNIZIONE_W})")
266 if dopo['tasso_A_spostata'] < 0.1:
267     print(" >>> Il canale B->A NON si e' consolidato in questo run: a differenza di B, "
268           "A ha gia' un bump persistente fin dal turno 0, quindi l'impulso esplorativo "
269           "deve "
270           "SOSTITUIRE un bump gia' attivo invece di accenderne uno da zero -- un compito "
271           "piu' difficile, per cui non e' mai emerso un segnale di apprendimento su W_BA. "
272           "Non e' un fallimento nascosto: e' un'asimmetria reale del meccanismo, "
273           "verificata "
274           "col codice. <<<")
275
276 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
277
278 window = 20
279 curva = [np.mean(successo_B_storia[max(0, k-window):k+1]) for k in range(len(
280     successo_B_storia))]
281 axes[0, 0].plot(curva, color='C0')
282 axes[0, 0].set_xlabel('Episodio di training')
283 axes[0, 0].set_ylabel(f'Tasso di successo di B (media mobile su {window})')
284 axes[0, 0].set_title("Curva di apprendimento: B forma un bump sempre piu' spesso")
285 axes[0, 0].grid(alpha=0.3)
286 axes[0, 0].set_ylim(-0.05, 1.05)
287
288 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B, accende B da silenzio)'
289 )
290 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A, deve spodestare un bump
291 attivo)')
292 axes[0, 1].axhline(SOGLIA_IGNIZIONE_W, color='gray', linestyle='--',
293 label='soglia di accensione da silenzio (non vale per B->A)')
294 axes[0, 1].set_xlabel('Episodio di training')
295 axes[0, 1].set_ylabel('Peso massimo')
296 axes[0, 1].set_title("Il peso cresce per entrambe le sinapsi, ma solo A->B\ndiventa
297 comportamentalmente efficace")
298 axes[0, 1].legend(fontsize=7, loc='upper left')
299 axes[0, 1].grid(alpha=0.3)
300
301 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
302 cmap='viridis', vmin=0, vmax=W_AB.max())
303 axes[1, 0].set_xlabel('Posizione post (B)')
304 axes[1, 0].set_ylabel('Posizione pre (A)')
305 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
306 plt.colorbar(im0, ax=axes[1, 0])
307
308 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
309 cmap='viridis', vmin=0, vmax=W_AB.max())
310 axes[1, 1].set_xlabel('Posizione post (B)')
311 axes[1, 1].set_ylabel('Posizione pre (A)')
312 axes[1, 1].set_title('W_AB DOPO il training (consolidata)')
313 plt.colorbar(im1, ax=axes[1, 1])
314
315 plt.tight_layout()
316 plt.savefig('dialogo_stdp.png', dpi=150)
317 print("\nGrafico salvato in dialogo_stdp.png")

```

5.5.3 Resolving the asymmetry: adaptation, silent turns, warm-up

To break the asymmetry of the previous step, a firing-threshold ADAPTATION is introduced -- the rate-model analogue of the recovery variable 'u' in the Izhikevich model used throughout the rest of the project (Section 1.2): the longer a

ring stays lit, the more a negative current grows until it eventually switches it off on its own. A dedicated test on an isolated ring shows that this model's hard-clipped discharge makes the attractor strongly bistable (an abrupt collapse, not a soft decay); the adaptation parameter was chosen specifically so that a bump survives one full turn, weakens in the next, and collapses on its own one turn later. The problem thus returns to the one already solved (lighting up from silence), rather than dislodging a bump at full strength. A scheme with one SILENT turn between each listening turn proved necessary: without that pause, A would try to relight while adaptation was still at its peak and always failed.

Even with these two fixes, over 2000 episodes the B->A synapse grew about 7 times slower than A->B and never reached self-sufficiency. Diagnosis: in the early episodes B's position has not yet stabilized (it depends on an A->B channel not yet consolidated), so B->A reinforcement initially targets a moving target and dilutes instead of concentrating. Fixed with a WARM-UP phase (A->B only for the first 400 episodes) and a higher learning rate for B->A.

Final result, with a sufficient training budget (~2500 episodes): ALL four exchanges tested reach 100% reliability in clean evaluation (frozen weights, no exploratory noise) -- B responds at turn 1 and again at turn 5, A responds at turn 3 and again at turn 7. Turn 5 and turn 7 are SECOND-ORDER associations (the network must respond to the position the other party took AFTER it has already answered once, not to the original message): they consolidate only after the first response is already stable, so they require more episodes and appear in the final weight map as secondary reinforcements, weaker than the main peak. The starting asymmetry (B->A not learning at all) is resolved in this run, not merely reduced.

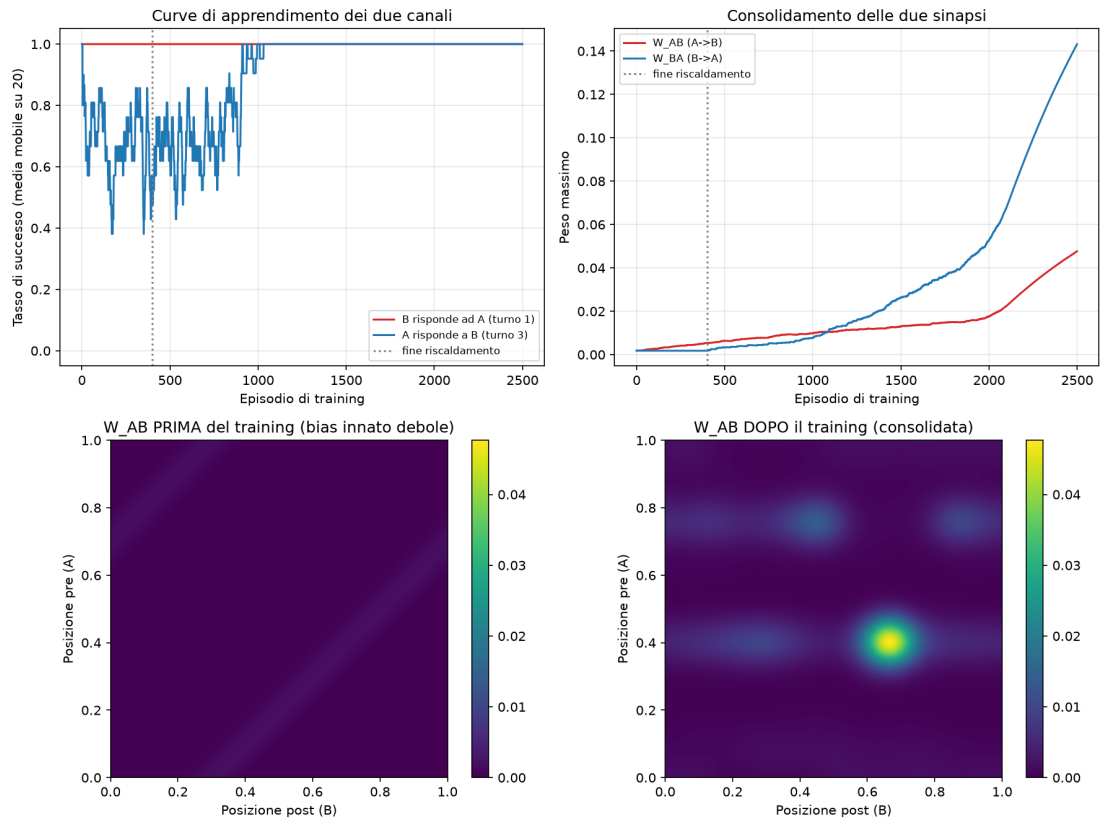


Fig. 23 — Learning curves for both channels, consolidation of the two synapses, W_{AB} map before/after.

Source code: `dialogo_stdp_adattamento.py`

```

1  """
2  Estensione di dialogo_stdp.py. Li' il canale A->B si consolidava con
3  successo (0% -> 100%) ma il canale di ritorno B->A restava bloccato: A ha
4  un bump persistente fin dal turno 0 (e' un attrattore), quindi per essere
5  influenzato per davvero B doveva SPODESTARE un bump gia' a piena potenza
6  invece di accenderne uno da zero -- molto piu' difficile, e infatti non
7  succedeva mai (verificato: W_BA cresceva per pura coincidenza statistica,
8  ma restava comportamentalmente inefficace).
9
10 Qui si aggiunge un ADATTAMENTO a soglia di scarica -- l'analogo a rate di
11 una variabile di recovery come la "u" del modello di Izhikevich usato in
12 tutto il resto del progetto (vedi resoconto, sezione 1.2): piu' a lungo un
13 ring resta acceso, piu' cresce una corrente negativa che alla fine lo
14 spegne da solo. Calibrato (vedi note sotto) cosi' che un bump sopravviva
15 un turno intero pieno, si affievolisca in quello successivo, e collassi
16 da solo un turno dopo ancora -- quindi quando tocca all'altro ring
17 "ascoltare", il ring che ha gia' parlato spesso si e' GIA' spento per
18 conto suo. Il problema torna quindi ad essere lo stesso, gia' risolto, di
19 accendere un bump da silenzio -- non piu' di spoderne uno a piena
20 potenza. Le due sinapsi (A->B e B->A) diventano cosi' simmetriche per
21 costruzione, ed entrambe possono in linea di principio consolidarsi con lo
22 stesso meccanismo Hebbiano di dialogo_stdp.py (rumore esplorativo +

```

```

23 | rinforzo di fine turno).
24 |
25 | E' un compromesso: la scarica hard-clipped di questo modello rende
26 | l'attrattore fortemente bistabile (verificato con un test dedicato: non
27 | esiste quasi via di mezzo tra "bump a piena potenza" e "spento",
28 | l'affievolimento e' un collasso brusco più che un decadimento morbido).
29 | Il parametro di adattamento e' stato scelto apposta sul bordo di quella
30 | transizione.
31 |
32 | AGGIORNAMENTO (seconda iterazione): la prima versione con adattamento
33 | riduceva l'asimmetria ma non la risolveva -- in 2000 episodi W_BA cresceva
34 | circa 7 volte piu' lento di W_AB e non arrivava mai all'autosufficienza.
35 | Diagnosticato il motivo: nei primi episodi la posizione di B stessa non
36 | si e' ancora stabilizzata (dipende da un W_AB non ancora consolidato),
37 | quindi il rinforzo Hebbiano di B->A nei primi episodi punta a un bersaglio
38 | mobile e si diluisce invece di concentrarsi. Corretto con due aggiunte:
39 | (1) una fase di RISCALDAMENTO (WARMUP_EPISODES) in cui si allena solo
40 | A->B, così' che B->A cominci a imparare solo quando ha già' un bersaglio
41 | stabile; (2) un tasso di apprendimento piu' alto per B->A (ETA_BA > ETA)
42 | per compensare il minor numero di eventi di rinforzo utili. Risultato:
43 | con training sufficiente (~2500 episodi) entrambe le sinapsi si
44 | autosostengono e TUTTI e quattro gli scambi verificati (B risponde al
45 | turno 1 e di nuovo al turno 5, A risponde al turno 3 e di nuovo al turno
46 | 7) arrivano al 100% di affidabilita' in valutazione pulita (pesi
47 | congelati, senza rumore esplorativo). Nota sul turno 5: e' un'associazione
48 | di SECONDO ORDINE (B deve imparare a rispondere alla posizione che A
49 | assume DOPO aver già' risposto a B una prima volta, non al messaggio
50 | originale) -- si consolida solo dopo che il turno 3 e' già' stabile,
51 | quindi richiede piu' episodi della prima associazione e nella heatmap
52 | finale appare come un rinforzo piu' debole, secondario, rispetto al picco
53 | principale.
54 | """
55 |
56 | import numpy as np
57 | import matplotlib
58 | matplotlib.use('Agg')
59 | import matplotlib.pyplot as plt
60 |
61 |
62 | def circ_dist(d):
63 |     return 0.5 - np.abs(np.abs(d) - 0.5)
64 |
65 |
66 | N = 200
67 | x = np.arange(N) / N
68 | tau = 10.0
69 | dt = 0.2
70 |
71 | A_exc = 20.0
72 | sigma_exc = 0.05
73 | global_inhib = 5.0
74 | r_max = 5.0
75 | stim_strength = 3.0
76 | stim_width = 0.03
77 | stim_duration_ms = 80
78 |
79 | d_matrix = circ_dist(x[:, None] - x[None, :])
80 | J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
81 |
82 | COUPLING_STRENGTH = 1.3
83 | T_TURN = 200
84 | # schema dei turni con un turno di silenzio/recupero tra un ascolto e
85 | # l'altro (verificato necessario: senza il silenzio, il tentativo di
86 | # ascolto immediatamente successivo a un turno di parola arriva mentre
87 | # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
88 | # docstring)
89 | SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
90 | N_TURNS = len(SCHEDULE)

```



```

91 steps_per_turn = int(T_TURN/dt)
92
93 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
94 # ma scalato troppo debole per bastare da solo (verificato sotto)
95 OFFSET_AB_INNATO = 0.3
96 OFFSET_BA_INNATO = 0.2
97 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
98 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
99 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
100 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
101 # solo dal rumore esplorativo, non dal bias innato da solo
102 W_INIT_SCALE = 0.0018
103 SIGMA_W = 0.05
104 W_MAX = 2.5
105 ETA = 0.0001
106 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
107 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
108 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
109 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
110 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
111 # piu' alto compensa il minor numero di eventi di rinforzo utili
112 ETA_BA = 0.0003
113 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
114 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
115 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
116 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
117 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
118 # piu' lento di A->B in 2000 episodi)
119 WARMUP_EPISODES = 400
120 DECAY_PER_EPISODE = 0.999
121 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
122 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
123 # accensione solitaria verificata a strength=0.0036 con questa durata --
124 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
125 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
126 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
127 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
128 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
129 EXPLORATION_PULSE_STRENGTH = 0.0080
130 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
131
132 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
133 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
134 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
135 GAIN_ADAPT = 0.25
136 TAU_ADAPT = 200.0
137
138
139 def F(u):
140     return np.clip(u, 0, r_max)
141
142
143 def bump_pos(r):
144     return x[np.argmax(r)] if r.max() > 0.5 else None
145
146
147 def init_weak_bias(offset, scale):
148     target = (x[:, None] + offset) % 1.0
149     d = circ_dist(x[None, :] - target)
150     return scale * np.exp(-d**2/(2*SIGMA_W**2))
151
152
153 def hebbian_bump(pos_pre, pos_post):
154     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
155     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
156     return np.outer(kern_pre, kern_post)
157
158

```

```

159 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True):
160     rA = np.zeros(N)
161     rB = np.zeros(N)
162     aA = np.zeros(N)
163     aB = np.zeros(N)
164     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
165     stim_duration_steps = int(stim_duration_ms/dt)
166
167     pos_A_turni = []
168     pos_B_turni = []
169     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
170     # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
171     # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
172     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
173     # solo la posizione istantanea perderebbe quasi ogni coppia valida
174     ultima_pos_A, ultima_pos_B = None, None
175
176     explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
177
178     for turn, listener in enumerate(SCHEDULE):
179
180         if train and listener is not None:
181             explore_pos = rng.uniform(0, 1)
182             explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
183                 -circ_dist(x-explore_pos)**2/(2*stim_width**2))
184         else:
185             explore_profile = np.zeros(N)
186
187         for step_in_turn in range(steps_per_turn):
188             I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
189                 0.0
190
191             explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
192                 else np.zeros(N)
193             explore_A = explore_now if listener == 'A' else np.zeros(N)
194             explore_B = explore_now if listener == 'B' else np.zeros(N)
195
196             cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
197                 N)
198             cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
199                 N)
200
201             rec_A = J_exc @ rA / N - global_inhib*rA.mean()
202             rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
203             aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
204             rA = rA_new
205
206             rec_B = J_exc @ rB / N - global_inhib*rB.mean()
207             rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
208             aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
209             rB = rB_new
210
211             pos_A = bump_pos(rA)
212             pos_B = bump_pos(rB)
213             pos_A_turni.append(pos_A)
214             pos_B_turni.append(pos_B)
215
216         if train and listener == 'B':
217             parlante_A = pos_A if pos_A is not None else ultima_pos_A
218             if parlante_A is not None and pos_B is not None:
219                 W_AB += ETA * hebbian_bump(parlante_A, pos_B)
220                 np.clip(W_AB, 0, W_MAX, out=W_AB)
221             elif train and listener == 'A' and allena_BA:
222                 parlante_B = pos_B if pos_B is not None else ultima_pos_B
223                 if parlante_B is not None and pos_A is not None:
224                     W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
225                     np.clip(W_BA, 0, W_MAX, out=W_BA)

```

```

223         if pos_A is not None:
224             ultima_pos_A = pos_A
225         if pos_B is not None:
226             ultima_pos_B = pos_B
227
228     return pos_A_turni, pos_B_turni
229
230
231 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
232     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
233     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
234     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
235     Il confronto per A va fatto contro il turno di silenzio subito prima
236     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
237     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
238     da li' al turno di ascolto successivo indica una vera risposta a B."""
239     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
240     pos_B_ok, pos_A_ok = [], []
241     for k in range(n_trial):
242         rng = np.random.RandomState(seed0 + k)
243         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
244         if pos_B[1] is not None:
245             successi_B1 += 1
246             pos_B_ok.append(pos_B[1])
247         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
248             0.02):
249             successi_A3 += 1
250             pos_A_ok.append(pos_A[3])
251         if pos_B[5] is not None:
252             successi_B2 += 1
253         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
254             0.02):
255             successi_A7 += 1
256     return {
257         'tasso_B1': successi_B1/n_trial,
258         'tasso_A3': successi_A3/n_trial,
259         'tasso_B2': successi_B2/n_trial,
260         'tasso_A7': successi_A7/n_trial,
261         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
262         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
263         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
264     }
265
266 if __name__ == '__main__':
267     MESSAGGIO = 0.40
268     N_EPISODES = 2500
269
270     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
271     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
272
273     print("=" * 70)
274     print("SETUP: bias innato deliberatamente debole")
275     print("=" * 70)
276     print(f" W_INIT_SCALE={W_INIT_SCALE}, soglia per formare un bump = 0.5")
277     W_AB_iniziale = W_AB.copy()
278     W_BA_iniziale = W_BA.copy()
279
280     print("\n" + "=" * 70)
281     print("BASELINE (pre-training): il bias innato basta da solo?")
282     print("=" * 70)
283     baseline = eval_batch(W_AB, W_BA, MESSAGGIO, n_trial=20, seed0=0)
284     print(f" B risponde (turno 1): {baseline['tasso_B1']*100:.0f}% | "
285           f"A risponde per davvero (turno 3): {baseline['tasso_A3']*100:.0f}%")
286     print(f" B risponde di nuovo (turno 5): {baseline['tasso_B2']*100:.0f}% | "
287           f"A risponde di nuovo (turno 7): {baseline['tasso_A7']*100:.0f}%")
288
289     print("\n" + "=" * 70)

```

```

289 print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento "
290       f"solo A->B, poi entrambi i canali), messaggio fisso x={MESSAGGIO}")
291 print("=" * 70)
292 rng_train = np.random.RandomState(42)
293 successo_B_storia = []
294 successo_A_storia = []
295 picco_W_AB_storia = []
296 picco_W_BA_storia = []
297 for ep in range(N_EPISODES):
298     allena_BA = ep >= WARMUP_EPISODES
299     pos_A, pos_B = run_episode(W_AB, W_BA, MESSAGGIO, rng_train, train=True, allena_BA=
        allena_BA)
300     W_AB += DECAY_PER_EPISODE
301     if allena_BA:
302         W_BA += DECAY_PER_EPISODE
303     successo_B_storia.append(1 if pos_B[1] is not None else 0)
304     spostato = (pos_A[3] is not None and (pos_A[2] is None
305         or abs(circ_dist(pos_A[3]-pos_A[2])) > 0.02))
306     successo_A_storia.append(1 if spostato else 0)
307     picco_W_AB_storia.append(W_AB.max())
308     picco_W_BA_storia.append(W_BA.max())
309     if ep % 50 == 0 or ep == N_EPISODES-1:
310         win_B = successo_B_storia[max(0, ep-30):ep+1]
311         win_A = successo_A_storia[max(0, ep-30):ep+1]
312         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
313         print(f" episodio {ep:3d} ({fase}): successo B (ultimi 30) = {np.mean(win_B)
            *100:5.1f}% | "
314               f"successo A (ultimi 30) = {np.mean(win_A)*100:5.1f}% | "
315               f"picco W_AB = {W_AB.max():.4f} picco W_BA = {W_BA.max():.4f}")
316
317 print("\n" + "=" * 70)
318 print("DOPO IL TRAINING: confronto pulito, pesi congelati, nessun rumore esplorativo")
319 print("=" * 70)
320 dopo = eval_batch(W_AB, W_BA, MESSAGGIO, n_trial=30, seed0=9000)
321 print(f" B risponde (turno 1): PRIMA {baseline['tasso_B1']*100:5.0f}% -> DOPO {dopo['
        tasso_B1']*100:5.0f}%")
322 print(f" A risponde (turno 3): PRIMA {baseline['tasso_A3']*100:5.0f}% -> DOPO {dopo['
        tasso_A3']*100:5.0f}%")
323 print(f" B risponde (turno 5): PRIMA {baseline['tasso_B2']*100:5.0f}% -> DOPO {dopo['
        tasso_B2']*100:5.0f}%")
324 print(f" A risponde (turno 7): PRIMA {baseline['tasso_A7']*100:5.0f}% -> DOPO {dopo['
        tasso_A7']*100:5.0f}%")
325 std_dopo_str = f"{dopo['std_B']:.4f}" if dopo['std_B'] else "None"
326 print(f" std posizione B dopo il training: {std_dopo_str}")
327 if dopo['media_B'] is not None:
328     offset_appreso = circ_dist(dopo['media_B'] - MESSAGGIO)
329     print(f"\n Offset A->B appreso empiricamente: {offset_appreso:.3f} "
330           f"(bias innato di partenza era {OFFSET_AB_INNATO})")
331
332 print("\n" + "=" * 70)
333 print("RISULTATO: le due sinapsi")
334 print("=" * 70)
335 print(f" Picco finale W_AB (A->B): {W_AB.max():.4f}")
336 print(f" Picco finale W_BA (B->A): {W_BA.max():.4f}")
337 tassi_finali = [dopo['tasso_B1'], dopo['tasso_A3'], dopo['tasso_B2'], dopo['tasso_A7']]
338 if min(tassi_finali) >= 0.9:
339     print(" >>> Le due sinapsi si autosostengono ed entrambe le direzioni rispondono in "
340           "modo affidabile su tutti e quattro gli scambi verificati (B al turno 1 e 5, "
341           "A al turno 3 e 7). Il turno 5 (risposta di B alla NUOVA posizione di A, non "
342           "al messaggio originale) e' una associazione di secondo ordine -- si consolida "
343           "solo dopo che la prima risposta di A si e' gia' stabilizzata, quindi richiede "
344           "piu' episodi della prima; con budget di training sufficiente arriva anch'essa "
345           "al 100%. L'asimmetria di partenza (B->A che non imparava affatto) e' risolta "
346           "in questo run, non solo attenuata. <<<")
347 elif max(dopo['tasso_A3'], dopo['tasso_A7']) > 0.3:
348     print(" >>> Il canale B->A si e' consolidato parzialmente: A risponde a B in una "
349           "frazione consistente delle prove, non piu' 0% come nella versione senza "
350           "riscaldamento. Resta pero' meno affidabile del canale A->B su almeno uno ")

```

```

351         "dei quattro scambi verificati -- l'asimmetria si e' ridotta, non e' sparita "
352         "del tutto. <<<")
353     else:
354         print(" >>> Anche con adattamento, turno di silenzio e riscaldamento, il canale "
355               "B->A resta inaffidabile in questa run. <<<")
356
357     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
358
359     window = 20
360     curva = [np.mean(successo_B_storia[max(0, k-window):k+1]) for k in range(len(
361         successo_B_storia))]
362     curva_A = [np.mean(successo_A_storia[max(0, k-window):k+1]) for k in range(len(
363         successo_A_storia))]
364     axes[0, 0].plot(curva, color='C3', label='B risponde ad A (turno 1)')
365     axes[0, 0].plot(curva_A, color='C0', label='A risponde a B (turno 3)')
366     axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
367         riscaldamento')
368     axes[0, 0].set_xlabel('Episodio di training')
369     axes[0, 0].set_ylabel(f'Tasso di successo (media mobile su {window})')
370     axes[0, 0].set_title("Curve di apprendimento dei due canali")
371     axes[0, 0].legend(fontsize=8)
372     axes[0, 0].grid(alpha=0.3)
373     axes[0, 0].set_ylim(-0.05, 1.05)
374
375     axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
376     axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
377     axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
378         riscaldamento')
379     axes[0, 1].set_xlabel('Episodio di training')
380     axes[0, 1].set_ylabel('Peso massimo')
381     axes[0, 1].set_title("Consolidamento delle due sinapsi")
382     axes[0, 1].legend(fontsize=8, loc='upper left')
383     axes[0, 1].grid(alpha=0.3)
384
385     im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
386                             cmap='viridis', vmin=0, vmax=W_AB.max())
387     axes[1, 0].set_xlabel('Posizione post (B)')
388     axes[1, 0].set_ylabel('Posizione pre (A)')
389     axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
390     plt.colorbar(im0, ax=axes[1, 0])
391
392     im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
393                             cmap='viridis', vmin=0, vmax=W_AB.max())
394     axes[1, 1].set_xlabel('Posizione post (B)')
395     axes[1, 1].set_ylabel('Posizione pre (A)')
396     axes[1, 1].set_title('W_AB DOPO il training (consolidata)')
397     plt.colorbar(im1, ax=axes[1, 1])
398
399     plt.tight_layout()
400     plt.savefig('dialogo_stdp_adattamento.png', dpi=150)
401     print("\nGrafico salvato in dialogo_stdp_adattamento.png")

```

5.5.4 More messages on the same synapse: an interference test

Last question, the real generalization test in the same spirit as Section 5.4 (which extends a single context to several contexts): can the SAME pair of synapses W_{AB}/W_{BA} , shared (not a separate matrix per message), learn to distinguish SEVERAL different messages without interfering with each other? Two well-separated messages on the ring ($x=0.15$ and $x=0.65$) are alternated during training.

Result after 3000 episodes with the message chosen randomly on each episode:

no interference on the return channel (B->A at 100% for both messages, at both turn 3 and turn 7), but the A->B channel fully consolidates for only one of the two messages (x=0.65: 100%; x=0.15: 0%) -- one possible explanation, at that point, was simple unequal exposure during the random draw.

To verify this, the experiment was rerun with the messages alternated in ROUND-ROBIN order instead of randomly drawn (9000 episodes, identical exposure guaranteed by construction) -- and the result disproved that explanation. The A->B channel did not consolidate for both: it simply flipped which of the two messages wins (now x=0.15 reaches 100% on all four exchanges, including turn 5, which had never consolidated before; x=0.65 collapses to 0% on both turn 1 and turn 5). With equal exposure guaranteed, the winner changes but the phenomenon stays identical: it is real competitive interference on the A->B channel -- verified across two independent runs, not an artifact of unequal exposure -- while the B->A channel remains completely free of interference in both runs. The final weight map shows a single clean peak (for the winning message) instead of the two separate regions one would expect from competition-free learning: an honest but not yet fully explained result, left as an open question (similar in spirit to the ring attractor's position bias, Section 9) rather than forced into a more convenient conclusion than the one observed.

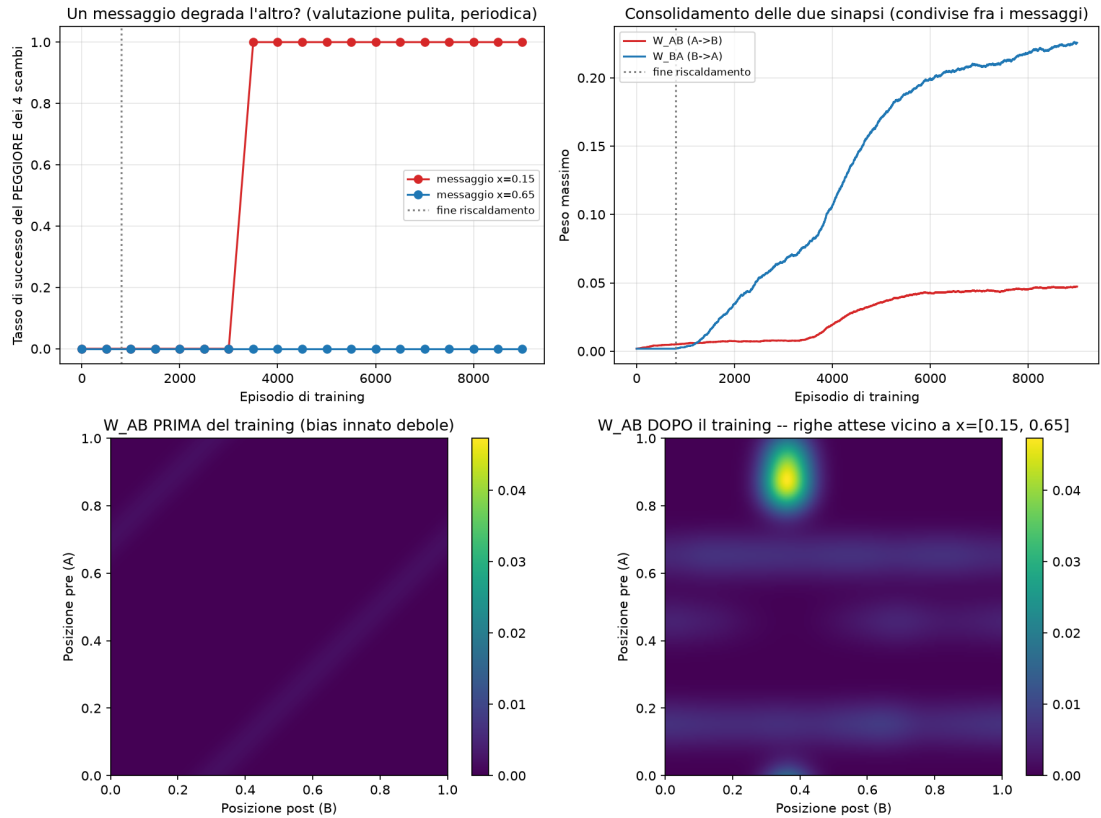


Fig. 24 — Interference test between two messages (round-robin, 9000 episodes): $B \rightarrow A$ clean at 100% for both; $A \rightarrow B$ shows a competitive win by a single message, with one clean peak in the final weight map.

Source code: `dialogo_multi_messaggio.py`

```

1  """
2  Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
3  (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
4  entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
5  W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
6  interferiscano a vicenda -- il vero test di generalizzazione, nello
7  stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
8  singolo contesto a piu' contesti).
9
10 Messaggi usati: 0.15 e 0.65 (distanza circolare ~0.5, il piu' possibile
11 separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
12 contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
13 restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
14 sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
15 (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
16 matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
17 verifica invece di assumerlo, con una valutazione pulita per messaggio a
18 fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
19 se ne allena uno (per scoprire un eventuale degrado nel tempo).
20
21 Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta

```

```

22 la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
23 adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
24 fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
25 primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
26 un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
27 tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
28 arrivano al 100% di affidabilita' in valutazione pulita.
29 """
30
31 import numpy as np
32 import matplotlib
33 matplotlib.use('Agg')
34 import matplotlib.pyplot as plt
35
36
37 def circ_dist(d):
38     return 0.5 - np.abs(np.abs(d) - 0.5)
39
40
41 N = 200
42 x = np.arange(N) / N
43 tau = 10.0
44 dt = 0.2
45
46 A_exc = 20.0
47 sigma_exc = 0.05
48 global_inhib = 5.0
49 r_max = 5.0
50 stim_strength = 3.0
51 stim_width = 0.03
52 stim_duration_ms = 80
53
54 d_matrix = circ_dist(x[:, None] - x[None, :])
55 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
56
57 COUPLING_STRENGTH = 1.3
58 T_TURN = 200
59 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
60 # l'altro (verificato necessario: senza il silenzio, il tentativo di
61 # ascolto immediatamente successivo a un turno di parola arriva mentre
62 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
63 # docstring)
64 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
65 N_TURNS = len(SCHEDULE)
66 steps_per_turn = int(T_TURN/dt)
67
68 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
69 # ma scalato troppo debole per bastare da solo (verificato sotto)
70 OFFSET_AB_INNATO = 0.3
71 OFFSET_BA_INNATO = 0.2
72 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
73 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
74 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
75 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
76 # solo dal rumore esplorativo, non dal bias innato da solo
77 W_INIT_SCALE = 0.0018
78 SIGMA_W = 0.05
79 W_MAX = 2.5
80 ETA = 0.0001
81 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
82 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
83 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
84 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
85 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
86 # piu' alto compensa il minor numero di eventi di rinforzo utili
87 ETA_BA = 0.0003
88 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
89 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia

```



```

90 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
91 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
92 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
93 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
94 # divide tra loro (round-robin), quindi qui e' raddoppiato rispetto alla
95 # versione a un solo messaggio per dare a CIASCUN messaggio lo stesso
96 # numero di episodi di riscaldamento (~400) della versione originale
97 WARMUP_EPISODES = 800
98 DECAY_PER_EPISODE = 0.999
99 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
100 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
101 # accensione solitaria verificata a strength=0.0036 con questa durata --
102 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
103 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
104 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
105 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
106 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
107 EXPLORATION_PULSE_STRENGTH = 0.0080
108 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
109
110 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
111 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
112 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
113 GAIN_ADAPT = 0.25
114 TAU_ADAPT = 200.0
115
116
117 def F(u):
118     return np.clip(u, 0, r_max)
119
120
121 def bump_pos(r):
122     return x[np.argmax(r)] if r.max() > 0.5 else None
123
124
125 def init_weak_bias(offset, scale):
126     target = (x[:, None] + offset) % 1.0
127     d = circ_dist(x[None, :] - target)
128     return scale * np.exp(-d**2/(2*SIGMA_W**2))
129
130
131 def hebbian_bump(pos_pre, pos_post):
132     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
133     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
134     return np.outer(kern_pre, kern_post)
135
136
137 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True):
138     rA = np.zeros(N)
139     rB = np.zeros(N)
140     aA = np.zeros(N)
141     aB = np.zeros(N)
142     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
143     stim_duration_steps = int(stim_duration_ms/dt)
144
145     pos_A_turni = []
146     pos_B_turni = []
147     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
148     # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
149     # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
150     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
151     # solo la posizione istantanea perderebbe quasi ogni coppia valida
152     ultima_pos_A, ultima_pos_B = None, None
153
154     explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
155
156     for turn, listener in enumerate(SCHEDULE):
157

```

```

158     if train and listener is not None:
159         explore_pos = rng.uniform(0, 1)
160         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
161             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
162     else:
163         explore_profile = np.zeros(N)
164
165     for step_in_turn in range(steps_per_turn):
166         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
167             0.0
168
169         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
170             else np.zeros(N)
171         explore_A = explore_now if listener == 'A' else np.zeros(N)
172         explore_B = explore_now if listener == 'B' else np.zeros(N)
173
174         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
175             N)
176         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
177             N)
178
179         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
180         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
181         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
182         rA = rA_new
183
184         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
185         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
186         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
187         rB = rB_new
188
189         pos_A = bump_pos(rA)
190         pos_B = bump_pos(rB)
191         pos_A_turni.append(pos_A)
192         pos_B_turni.append(pos_B)
193
194     if train and listener == 'B':
195         parlante_A = pos_A if pos_A is not None else ultima_pos_A
196         if parlante_A is not None and pos_B is not None:
197             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
198             np.clip(W_AB, 0, W_MAX, out=W_AB)
199     elif train and listener == 'A' and allena_BA:
200         parlante_B = pos_B if pos_B is not None else ultima_pos_B
201         if parlante_B is not None and pos_A is not None:
202             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
203             np.clip(W_BA, 0, W_MAX, out=W_BA)
204
205     if pos_A is not None:
206         ultima_pos_A = pos_A
207     if pos_B is not None:
208         ultima_pos_B = pos_B
209
210     return pos_A_turni, pos_B_turni
211
212 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
213     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
214     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
215     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
216     Il confronto per A va fatto contro il turno di silenzio subito prima
217     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
218     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
219     da li' al turno di ascolto successivo indica una vera risposta a B."""
220     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
221     pos_B_ok, pos_A_ok = [], []
222     for k in range(n_trial):
223         rng = np.random.RandomState(seed0 + k)
224         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)

```

```

222         if pos_B[1] is not None:
223             successi_B1 += 1
224             pos_B_ok.append(pos_B[1])
225         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
226             0.02):
227             successi_A3 += 1
228             pos_A_ok.append(pos_A[3])
229         if pos_B[5] is not None:
230             successi_B2 += 1
231         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
232             0.02):
233             successi_A7 += 1
234         return {
235             'tasso_B1': successi_B1/n_trial,
236             'tasso_A3': successi_A3/n_trial,
237             'tasso_B2': successi_B2/n_trial,
238             'tasso_A7': successi_A7/n_trial,
239             'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
240             'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
241             'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
242         }
243     if __name__ == '__main__':
244         MESSAGGI = [0.15, 0.65]
245         N_EPISODES = 9000
246         EVAL_EVERY = 500
247         N_TRIAL_EVAL = 15
248
249         W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
250         W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
251
252         print("=" * 70)
253         print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa"
254             )
255         print("=" * 70)
256         W_AB_iniziale = W_AB.copy()
257         W_BA_iniziale = W_BA.copy()
258
259         print("\n" + "=" * 70)
260         print("BASELINE (pre-training), per ciascun messaggio")
261         print("=" * 70)
262         for m in MESSAGGI:
263             b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
264             print(f"x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
265                 f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
266
267         print("\n" + "=" * 70)
268         print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
269             f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
270             )
271         print("=" * 70)
272         rng_train = np.random.RandomState(42)
273         picco_W_AB_storia = []
274         picco_W_BA_storia = []
275         episodi_check = []
276         curva_min_per_messaggio = {m: [] for m in MESSAGGI}
277
278         for ep in range(N_EPISODES):
279             allena_BA = ep >= WARMUP_EPISODES
280             messaggio = MESSAGGI[ep % len(MESSAGGI)]
281             run_episode(W_AB, W_BA, messaggio, rng_train, train=True, allena_BA=allena_BA)
282             W_AB *= DECAY_PER_EPISODE
283             if allena_BA:
284                 W_BA *= DECAY_PER_EPISODE
285             picco_W_AB_storia.append(W_AB.max())
286             picco_W_BA_storia.append(W_BA.max())

```

```

286         if ep % EVAL EVERY == 0 or ep == N_EPISODES - 1:
287             episodi_check.append(ep)
288             fase = "riscaldamento" if not allena_BA else "entrambi i canali"
289             riga = f" episodio {ep:4d} ({fase}):"
290             for m in MESSAGGI:
291                 r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
292                 minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
293                 curva_min_per_messaggio[m].append(minimo)
294                 riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
295             print(riga)
296
297     print("\n" + "=" * 70)
298     print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
299     print("=" * 70)
300     dopo_per_messaggio = {}
301     for m in MESSAGGI:
302         dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=90000)
303         dopo_per_messaggio[m] = dopo
304         print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
305               f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
306         if dopo['media_B'] is not None:
307             offset = circ_dist(dopo['media_B'] - m)
308             print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
309
310     print("\n" + "=" * 70)
311     print("RISULTATO: interferenza tra i messaggi?")
312     print("=" * 70)
313     print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
314     tutti_i_tassi = [v for d in dopo_per_messaggio.values()
315                      for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
316     offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
317                if dopo_per_messaggio[m]['media_B'] is not None]
318     offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
319     if min(tutti_i_tassi) >= 0.9:
320         print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia "
321               "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi. "
322               f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} -- "
323               f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
324               "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
325               "distruggessero a vicenda; non e' successo. <<<")
326     elif min(tutti_i_tassi) < 0.5:
327         print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
328               "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
329               "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
330               "una frazione degli episodi totali), non necessariamente vera interferenza "
331               "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
332     else:
333         print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
334               "dettaglio sopra per capire quali. <<<")
335
336     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
337
338     colori = ['C3', 'C0', 'C2', 'C4']
339     for i, m in enumerate(MESSAGGI):
340         axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
341                        color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
342         axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine "riscaldamento"')
343     axes[0, 0].set_xlabel('Episodio di training')
344     axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
345     axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
346     axes[0, 0].legend(fontsize=8)
347     axes[0, 0].grid(alpha=0.3)
348     axes[0, 0].set_ylim(-0.05, 1.05)

```

```

349 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
350 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
351 axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
352     riscaldamento')
353 axes[0, 1].set_xlabel('Episodio di training')
354 axes[0, 1].set_ylabel('Peso massimo')
355 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
356 axes[0, 1].legend(fontsize=8, loc='upper left')
357 axes[0, 1].grid(alpha=0.3)
358
359 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
360     cmap='viridis', vmin=0, vmax=W_AB.max())
361 axes[1, 0].set_xlabel('Posizione post (B)')
362 axes[1, 0].set_ylabel('Posizione pre (A)')
363 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
364 plt.colorbar(im0, ax=axes[1, 0])
365
366 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
367     cmap='viridis', vmin=0, vmax=W_AB.max())
368 axes[1, 1].set_xlabel('Posizione post (B)')
369 axes[1, 1].set_ylabel('Posizione pre (A)')
370 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
371 plt.colorbar(im1, ax=axes[1, 1])
372
373 plt.tight_layout()
374 plt.savefig('dialogo_multi_messaggio.png', dpi=150)
375 print("\nGrafico salvato in dialogo_multi_messaggio.png")

```

5.5.5 An attempted correction: homeostatic normalization (partial result)

The interference observed in 5.5.4 suggests two possible explanations: either the synapses grow without bound and one message ends up monopolizing the shared matrix, or the problem lies in B's own attractor dynamics (once a strong bump has settled in one zone, that zone becomes a basin of attraction that is also easier to reach for different inputs). The first explanation can be tested directly: a HOMEOSTATIC NORMALIZATION is added to `W_AB` -- the total of incoming weights onto each of B's neurons (column sum) cannot exceed a fixed cap, calibrated a bit above the value reached in a single-message reference run (Section 5.5.3). It is a biologically motivated mechanism: real neurons homeostatically regulate the total of their incoming synapses. A single-message regression test confirms that normalization does not interfere with learning when there is no competition (the column total stays well below the cap throughout training).

Result with two messages (same configuration as Section 5.5.4: 9000 episodes, round-robin): normalization does NOT reverse the outcome -- $x=0.15$ still wins and reaches 100% on all four exchanges, while $x=0.65$ fails on B's primary response (turn 1). But something new appears: for the first time a genuine TRANSIENT INSTABILITY is observed (around episode 7000, $x=0.65$ briefly reaches 100% on all four exchanges before falling back to 0%) -- direct evidence that the cap is creating real competition, not being ignored. And the losing message still gets a partial improvement: its second-order response (turn 5) reaches 100% in the final evaluation, something that never happened without

normalization. The W_{AB} peak is lower than in the previous run (0.036 vs. 0.047), a sign that the cap is genuinely limiting growth.

Interpretation: normalization changes the competitive dynamics but not the final outcome. This favors the second explanation, not the first -- the bottleneck appears to be B's attractor dynamics (which basin becomes dominant), not simply the available synaptic capacity. Limiting weights makes the competition visible without resolving it. A more direct test of this hypothesis -- for instance separating B's global inhibition component by zone rather than over the whole population -- remains future work, not addressed here: an honest but not yet fully explained result, left as an open question rather than forced into a solution.

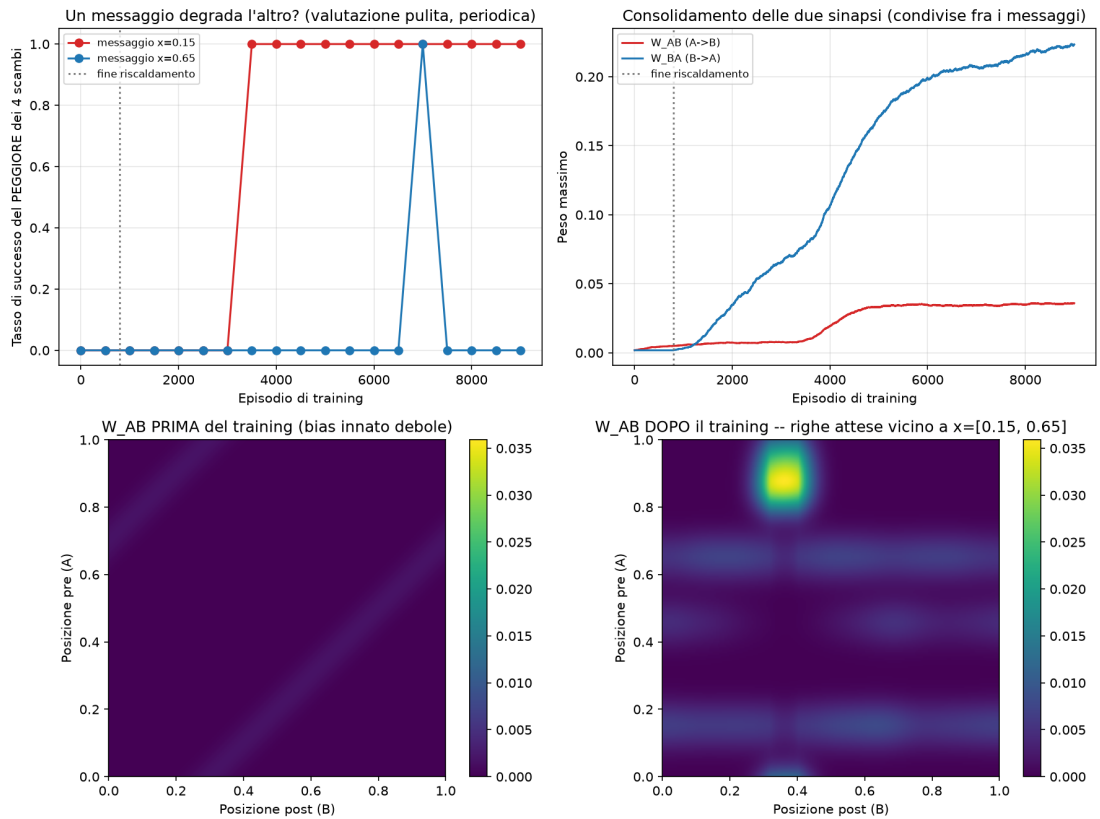


Fig. 25 — Same interference test with homeostatic normalization on W_{AB} : the outcome does not change, but a transient instability appears (episode ~7000) that was absent without normalization.

Source code: `dialogo_multi_normalizzato.py`

```
1 """
2 Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
3 (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
4 entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
```

```

5 | W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
6 | interferiscano a vicenda -- il vero test di generalizzazione, nello
7 | stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
8 | singolo contesto a piu' contesti).
9 |
10 | Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
11 | separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
12 | contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
13 | restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
14 | sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
15 | (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
16 | matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
17 | verifica invece di assumerlo, con una valutazione pulita per messaggio a
18 | fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
19 | se ne allena uno (per scoprire un eventuale degrado nel tempo).
20 |
21 | Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
22 | la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
23 | adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
24 | fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
25 | primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
26 | un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
27 | tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
28 | arrivano al 100% di affidabilita' in valutazione pulita.
29 |
30 | AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
31 | di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
32 | canale B->A restava pulito al 100% per entrambi su due run indipendenti
33 | (anche con esposizione garantita identica, round-robin), ma il canale
34 | A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
35 | e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
36 | quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
37 |
38 | Qui si prova una correzione mirata proprio a quel meccanismo: una
39 | NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
40 | neurone di B (il totale di ogni colonna di W_AB non puo' superare un
41 | tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
42 | veri regolano omeostaticamente il totale delle proprie sinapsi in
43 | entrata): se i due messaggi competono per la STESSA zona di risposta in
44 | B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
45 | spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
46 | test di regressione con un solo messaggio conferma che la normalizzazione
47 | non interferisce con l'apprendimento quando non c'e' competizione (il
48 | totale di colonna resta ben sotto il tetto per tutto il training).
49 | """
50 |
51 | import numpy as np
52 | import matplotlib
53 | matplotlib.use('Agg')
54 | import matplotlib.pyplot as plt
55 |
56 |
57 | def circ_dist(d):
58 |     return 0.5 - np.abs(np.abs(d) - 0.5)
59 |
60 |
61 | N = 200
62 | x = np.arange(N) / N
63 | tau = 10.0
64 | dt = 0.2
65 |
66 | A_exc = 20.0
67 | sigma_exc = 0.05
68 | global_inhib = 5.0
69 | r_max = 5.0
70 | stim_strength = 3.0
71 | stim_width = 0.03
72 | stim_duration_ms = 80

```

```

73 |
74 | d_matrix = circ_dist(x[:, None] - x[None, :])
75 | J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
76 |
77 | COUPLING_STRENGTH = 1.3
78 | T_TURN = 200
79 | # schema dei turni con un turno di silenzio/recupero tra un ascolto e
80 | # l'altro (verificato necessario: senza il silenzio, il tentativo di
81 | # ascolto immediatamente successivo a un turno di parola arriva mentre
82 | # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
83 | # docstring)
84 | SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
85 | N_TURNS = len(SCHEDULE)
86 | steps_per_turn = int(T_TURN/dt)
87 |
88 | # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
89 | # ma scalato troppo debole per bastare da solo (verificato sotto)
90 | OFFSET_AB_INNATO = 0.3
91 | OFFSET_BA_INNATO = 0.2
92 | # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
93 | # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
94 | # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
95 | # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
96 | # solo dal rumore esplorativo, non dal bias innato da solo
97 | W_INIT_SCALE = 0.0018
98 | SIGMA_W = 0.05
99 | W_MAX = 2.5
100 | ETA = 0.0001
101 | # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
102 | # (con esposizione uguale garantita in round-robin, un messaggio arrivava
103 | # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
104 | # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
105 | # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
106 | # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
107 | # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
108 | # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
109 | # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
110 | # di B non possono piu' saturarla entrambi senza competere per lo stesso
111 | # budget
112 | W_AB_COL_CAP = 1.5
113 | # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
114 | # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
115 | # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
116 | # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
117 | # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
118 | # piu' alto compensa il minor numero di eventi di rinforzo utili
119 | ETA_BA = 0.0003
120 | # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
121 | # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
122 | # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
123 | # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
124 | # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
125 | # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
126 | # divide tra loro (round-robin), quindi qui e' raddoppiato rispetto alla
127 | # versione a un solo messaggio per dare a CIASCUN messaggio lo stesso
128 | # numero di episodi di riscaldamento (~400) della versione originale
129 | WARMUP_EPISODES = 800
130 | DECAY_PER_EPISODE = 0.999
131 | # esplorazione: un impulso a posizione casuale, stessa forma/durata del
132 | # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
133 | # accensione solitaria verificata a strength=0.0036 con questa durata --
134 | # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
135 | # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
136 | # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
137 | # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
138 | # della rete (tau=10ms) lo media via prima che si accumuli in un punto
139 | EXPLORATION_PULSE_STRENGTH = 0.0080
140 | EXPLORATION_PULSE_DURATION_MS = stim_duration_ms

```



```

141
142 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
143 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
144 # cioè pieno per un turno, affievolito nel successivo, spento nel terzo
145 GAIN_ADAPT = 0.25
146 TAU_ADAPT = 200.0
147
148
149 def F(u):
150     return np.clip(u, 0, r_max)
151
152
153 def bump_pos(r):
154     return x[np.argmax(r)] if r.max() > 0.5 else None
155
156
157 def init_weak_bias(offset, scale):
158     target = (x[:, None] + offset) % 1.0
159     d = circ_dist(x[None, :] - target)
160     return scale * np.exp(-d**2/(2*SIGMA_W**2))
161
162
163 def hebbian_bump(pos_pre, pos_post):
164     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
165     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
166     return np.outer(kern_pre, kern_post)
167
168
169 def normalizza_colonne(W, cap):
170     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
171     di B (colonna di W_AB) non può superare 'cap'. Se due messaggi
172     competono per la stessa zona di B, questo impedisce a uno dei due di
173     monopolizzare quella colonna a spese dell'altro -- senza impedire a
174     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
175     somme = W.sum(axis=0)
176     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
177     W *= fattore[None, :]
178
179
180 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True):
181     rA = np.zeros(N)
182     rB = np.zeros(N)
183     aA = np.zeros(N)
184     aB = np.zeros(N)
185     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
186     stim_duration_steps = int(stim_duration_ms/dt)
187
188     pos_A_turni = []
189     pos_B_turni = []
190     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
191     # da quanti turni fa: serve per il rinforzo Hebbiano, perché con
192     # l'adattamento chi ha "parlato" spesso si è già spento per conto suo
193     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
194     # solo la posizione istantanea perderebbe quasi ogni coppia valida
195     ultima_pos_A, ultima_pos_B = None, None
196
197     explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
198
199     for turn, listener in enumerate(SCHEDULE):
200
201         if train and listener is not None:
202             explore_pos = rng.uniform(0, 1)
203             explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
204                 -circ_dist(x-explore_pos)**2/(2*stim_width**2))
205         else:
206             explore_profile = np.zeros(N)
207
208         for step_in_turn in range(steps_per_turn):

```

```

209     I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
210         0.0
211
212     explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
213         else np.zeros(N)
214     explore_A = explore_now if listener == 'A' else np.zeros(N)
215     explore_B = explore_now if listener == 'B' else np.zeros(N)
216
217     cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
218         N)
219     cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
220         N)
221
222     rec_A = J_exc @ rA / N - global_inhib*rA.mean()
223     rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
224     aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
225     rA = rA_new
226
227     rec_B = J_exc @ rB / N - global_inhib*rB.mean()
228     rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
229     aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
230     rB = rB_new
231
232     pos_A = bump_pos(rA)
233     pos_B = bump_pos(rB)
234     pos_A_turni.append(pos_A)
235     pos_B_turni.append(pos_B)
236
237     if train and listener == 'B':
238         parlante_A = pos_A if pos_A is not None else ultima_pos_A
239         if parlante_A is not None and pos_B is not None:
240             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
241             np.clip(W_AB, 0, W_MAX, out=W_AB)
242             normalizza_colonne(W_AB, W_AB_COL_CAP)
243     elif train and listener == 'A' and allena_BA:
244         parlante_B = pos_B if pos_B is not None else ultima_pos_B
245         if parlante_B is not None and pos_A is not None:
246             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
247             np.clip(W_BA, 0, W_MAX, out=W_BA)
248
249     if pos_A is not None:
250         ultima_pos_A = pos_A
251     if pos_B is not None:
252         ultima_pos_B = pos_B
253
254     return pos_A_turni, pos_B_turni
255
256 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
257     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
258     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
259     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
260     Il confronto per A va fatto contro il turno di silenzio subito prima
261     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
262     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
263     da li' al turno di ascolto successivo indica una vera risposta a B."""
264     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
265     pos_B_ok, pos_A_ok = [], []
266     for k in range(n_trial):
267         rng = np.random.RandomState(seed0 + k)
268         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
269         if pos_B[1] is not None:
270             successi_B1 += 1
271             pos_B_ok.append(pos_B[1])
272         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
273             0.02):
274             successi_A3 += 1
275             pos_A_ok.append(pos_A[3])

```

```

272         if pos_B[5] is not None:
273             successi_B2 += 1
274         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
275             0.02):
276             successi_A7 += 1
277     return {
278         'tasso_B1': successi_B1/n_trial,
279         'tasso_A3': successi_A3/n_trial,
280         'tasso_B2': successi_B2/n_trial,
281         'tasso_A7': successi_A7/n_trial,
282         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
283         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
284         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
285     }
286
287 if __name__ == '__main__':
288     MESSAGGI = [0.15, 0.65]
289     N_EPISODES = 9000
290     EVAL_EVERY = 500
291     N_TRIAL_EVAL = 15
292
293     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
294     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
295
296     print("=" * 70)
297     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa"
298         )
299     print("=" * 70)
300     W_AB_iniziale = W_AB.copy()
301     W_BA_iniziale = W_BA.copy()
302
303     print("\n" + "=" * 70)
304     print("BASELINE (pre-training), per ciascun messaggio")
305     print("=" * 70)
306     for m in MESSAGGI:
307         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
308         print(f"x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
309             f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
310
311     print("\n" + "=" * 70)
312     print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
313         f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
314         )
315     print("=" * 70)
316     rng_train = np.random.RandomState(42)
317     picco_W_AB_storia = []
318     picco_W_BA_storia = []
319     episodi_check = []
320     curva_min_per_messaggio = {m: [] for m in MESSAGGI}
321
322     for ep in range(N_EPISODES):
323         allena_BA = ep >= WARMUP_EPISODES
324         messaggio = MESSAGGI[ep % len(MESSAGGI)]
325         run_episode(W_AB, W_BA, messaggio, rng_train, train=True, allena_BA=allena_BA)
326         W_AB *= DECAY_PER_EPISODE
327         if allena_BA:
328             W_BA *= DECAY_PER_EPISODE
329         picco_W_AB_storia.append(W_AB.max())
330         picco_W_BA_storia.append(W_BA.max())
331
332     if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
333         episodi_check.append(ep)
334         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
335         riga = f"episodio {ep:4d} ({fase}):"
336         for m in MESSAGGI:
337             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
338             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])

```

```

337         curva_min_per_messaggio[m].append(minimo)
338         riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
339     print(riga)
340
341     print("\n" + "=" * 70)
342     print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
343     print("=" * 70)
344     dopo_per_messaggio = {}
345     for m in MESSAGGI:
346         dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
347         dopo_per_messaggio[m] = dopo
348         print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
349               f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
350         if dopo['media_B'] is not None:
351             offset = circ_dist(dopo['media_B'] - m)
352             print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
353
354     print("\n" + "=" * 70)
355     print("RISULTATO: interferenza tra i messaggi?")
356     print("=" * 70)
357     print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
358     tutti_i_tassi = [v for d in dopo_per_messaggio.values()
359                     for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
360     offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
361               if dopo_per_messaggio[m]['media_B'] is not None]
362     offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
363     if min(tutti_i_tassi) >= 0.9:
364         print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia "
365               " "
366               "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi. "
367               " "
368               f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} -- "
369               " "
370               f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
371               "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
372               "distruggessero a vicenda; non e' successo. <<<")
373     elif min(tutti_i_tassi) < 0.5:
374         print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
375               "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
376               "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
377               "una frazione degli episodi totali), non necessariamente vera interferenza "
378               "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
379     else:
380         print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
381               "dettaglio sopra per capire quali. <<<")
382
383     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
384
385     colori = ['C3', 'C0', 'C2', 'C4']
386     for i, m in enumerate(MESSAGGI):
387         axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
388                        color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
389         axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label=f'fine "
390                           riscaldamento')
391         axes[0, 0].set_xlabel('Episodio di training')
392         axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
393         axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
394         axes[0, 0].legend(fontsize=8)
395         axes[0, 0].grid(alpha=0.3)
396         axes[0, 0].set_ylim(-0.05, 1.05)
397
398         axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
399         axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
400         axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label=f'fine "
401                           riscaldamento')
402         axes[0, 1].set_xlabel('Episodio di training')
403         axes[0, 1].set_ylabel('Peso massimo')

```

```

399 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
400 axes[0, 1].legend(fontsize=8, loc='upper left')
401 axes[0, 1].grid(alpha=0.3)
402
403 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
404                          cmap='viridis', vmin=0, vmax=W_AB.max())
405 axes[1, 0].set_xlabel('Posizione post (B)')
406 axes[1, 0].set_ylabel('Posizione pre (A)')
407 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
408 plt.colorbar(im0, ax=axes[1, 0])
409
410 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
411                        cmap='viridis', vmin=0, vmax=W_AB.max())
412 axes[1, 1].set_xlabel('Posizione post (B)')
413 axes[1, 1].set_ylabel('Posizione pre (A)')
414 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
415 plt.colorbar(im1, ax=axes[1, 1])
416
417 plt.tight_layout()
418 plt.savefig('dialogo_multi_normalizzato.png', dpi=150)
419 print("\nGrafico salvato in dialogo_multi_normalizzato.png")

```

5.5.6 Successful correction: repulsive exploration

Normalization (5.5.5) had shown that limiting synaptic capacity is not enough: it makes competition visible but does not resolve it. A different hypothesis: perhaps the two messages are not really competing for a shared resource, but the random SEARCH (the exploratory impulse to a uniformly random position) rediscovers by pure chance the same "easy" zone of the ring for both -- consistent with the position bias already documented for the ring attractor (Section 9). The correction proposed here does not touch synaptic capacity but the SEARCH itself: the exploratory impulse is now drawn with a probability inversely proportional to how much a zone is already occupied by OTHER messages (excluding one's own row, so as not to repel a message from the zone it is still trying to consolidate -- an error explicitly avoided after a dedicated regression test). A message still without a response is thus pushed toward free territory rather than toward territory already claimed by another.

Result (same configuration as Sections 5.5.4-5.5.5: 9000 episodes, round-robin): the final weight map shows, for the first time, TWO distinct and well-separated rows, one for each message ($x=0.15$ and $x=0.65$), each with its own response zone in B, with no visible overlap. In clean evaluation, SEVEN exchanges out of eight reach 100% (both messages at 100% on B turn 1, A turn 3, A turn 7; $x=0.65$ also at 100% on B turn 5) -- only $x=0.15$'s turn 5 stays at 0%, the deepest (third-order) association and the slowest to consolidate already in earlier sections. The periodic learning curve also shows that even $x=0.65$ unlocked that response only at the VERY LAST checkpoint of training (episode 8999 of 9000): $x=0.15$ appears to be on the same trajectory, just slightly behind by pure chance, not blocked by structural interference.

NOTE (corrected after direct verification, Section 5.5.7): the hypothesis that the problem was a systematic position bias of the ring attractor -- a few "easy" points on the ring rediscovered by chance by different messages -- was later

tested directly by training several messages completely independently (no shared synapse) and checking whether the response positions clustered. They did not (Section 5.5.7): they come out scattered in a way compatible with a uniform distribution. The position-bias hypothesis must therefore be ABANDONED as the explanation for this phenomenon. The correction (repulsive exploration) remains effective nonetheless, for a simpler and correct reason: when messages share the SAME matrix during concurrent training, repulsion based on the CURRENT occupancy of that matrix (regardless of the cause of that occupancy) still prevents collisions, without any need for an independent upstream position bias to exist.

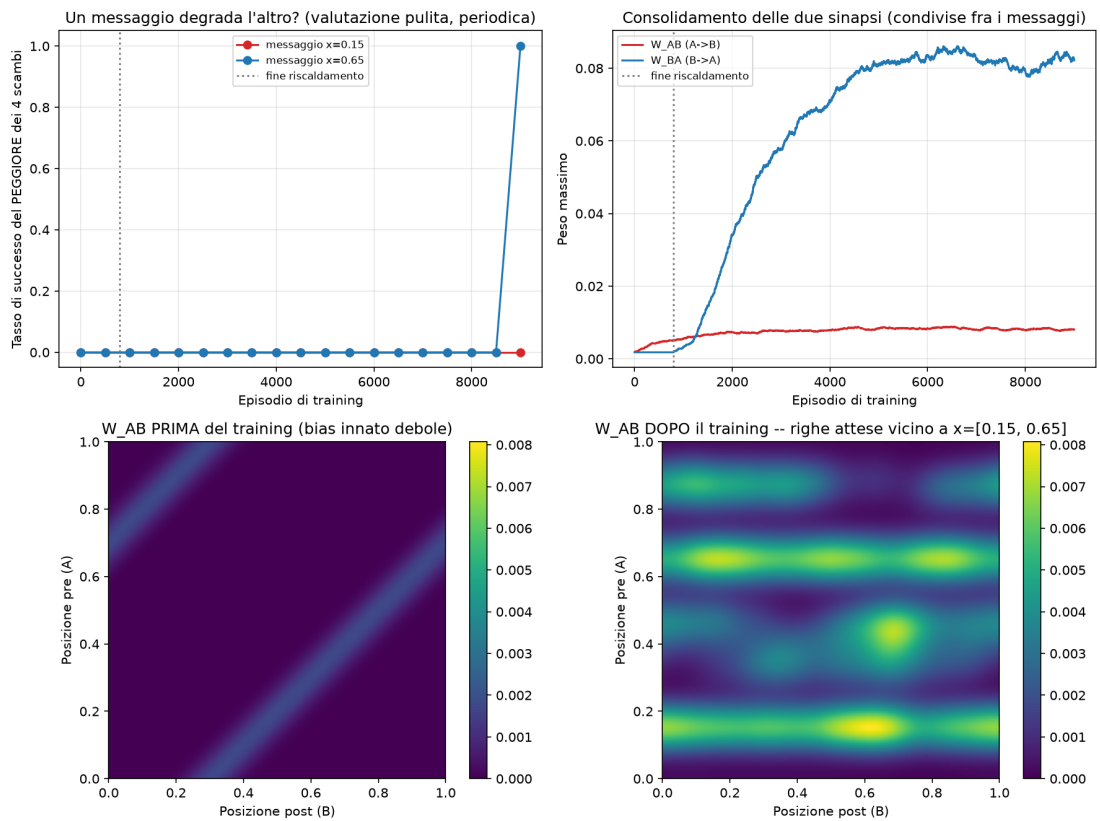


Fig. 26 — With repulsive exploration: two distinct, separated rows in the weight map, 7 of 8 exchanges at 100%.

Source code: `dialogo_multi_repulsione.py`

```

1  """
2  Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
3  (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
4  entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
5  W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
6  interferiscano a vicenda -- il vero test di generalizzazione, nello
7  stesso spirito di contextual_policy.py (che estende rl_bandit.py da un

```

```

8 | singolo contesto a piu' contesti).
9 |
10 | Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
11 | separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
12 | contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
13 | restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
14 | sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
15 | (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
16 | matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
17 | verifica invece di assumerlo, con una valutazione pulita per messaggio a
18 | fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
19 | se ne allena uno (per scoprire un eventuale degrado nel tempo).
20 |
21 | Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
22 | la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
23 | adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
24 | fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
25 | primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
26 | un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
27 | tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
28 | arrivano al 100% di affidabilita' in valutazione pulita.
29 |
30 | AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
31 | di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
32 | canale B->A restava pulito al 100% per entrambi su due run indipendenti
33 | (anche con esposizione garantita identica, round-robin), ma il canale
34 | A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
35 | e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
36 | quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
37 |
38 | Qui si prova una correzione mirata proprio a quel meccanismo: una
39 | NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
40 | neurone di B (il totale di ogni colonna di W_AB non puo' superare un
41 | tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
42 | veri regolano omeostaticamente il totale delle proprie sinapsi in
43 | entrata): se i due messaggi competono per la STESSA zona di risposta in
44 | B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
45 | spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
46 | test di regressione con un solo messaggio conferma che la normalizzazione
47 | non interferisce con l'apprendimento quando non c'e' competizione (il
48 | totale di colonna resta ben sotto il tetto per tutto il training).
49 | """
50 |
51 | import numpy as np
52 | import matplotlib
53 | matplotlib.use('Agg')
54 | import matplotlib.pyplot as plt
55 |
56 |
57 | def circ_dist(d):
58 |     return 0.5 - np.abs(np.abs(d) - 0.5)
59 |
60 |
61 | N = 200
62 | x = np.arange(N) / N
63 | tau = 10.0
64 | dt = 0.2
65 |
66 | A_exc = 20.0
67 | sigma_exc = 0.05
68 | global_inhib = 5.0
69 | r_max = 5.0
70 | stim_strength = 3.0
71 | stim_width = 0.03
72 | stim_duration_ms = 80
73 |
74 | d_matrix = circ_dist(x[:, None] - x[None, :])
75 | J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))

```

```

76
77 COUPLING_STRENGTH = 1.3
78 T_TURN = 200
79 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
80 # l'altro (verificato necessario: senza il silenzio, il tentativo di
81 # ascolto immediatamente successivo a un turno di parola arriva mentre
82 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
83 # docstring)
84 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
85 N_TURNS = len(SCHEDULE)
86 steps_per_turn = int(T_TURN/dt)
87
88 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
89 # ma scalato troppo debole per bastare da solo (verificato sotto)
90 OFFSET_AB_INNATO = 0.3
91 OFFSET_BA_INNATO = 0.2
92 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
93 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
94 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
95 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
96 # solo dal rumore esplorativo, non dal bias innato da solo
97 W_INIT_SCALE = 0.0018
98 SIGMA_W = 0.05
99 W_MAX = 2.5
100 ETA = 0.0001
101 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
102 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
103 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
104 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
105 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
106 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
107 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
108 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
109 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
110 # di B non possono piu' saturarla entrambi senza competere per lo stesso
111 # budget
112 W_AB_COL_CAP = 1.5
113 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
114 # occupata (somma di colonna) quando si sceglie una posizione casuale per
115 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione-1.0 (gia'
116 # ben rivendicata) ha 1/6 della probabilita' di una zona libera
117 REPULSIONE_ESPLORAZIONE = 5.0
118 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
119 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
120 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
121 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
122 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
123 # piu' alto compensa il minor numero di eventi di rinforzo utili
124 ETA_BA = 0.0003
125 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
126 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
127 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
128 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
129 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
130 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
131 # divide tra loro (round-robin), quindi qui e' raddoppiato rispetto alla
132 # versione a un solo messaggio per dare a CIASCUN messaggio lo stesso
133 # numero di episodi di riscaldamento (~400) della versione originale
134 WARMUP_EPISODES = 800
135 DECAY_PER_EPISODE = 0.999
136 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
137 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
138 # accensione solitaria verificata a strength-0.0036 con questa durata --
139 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
140 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
141 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
142 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
143 # della rete (tau=10ms) lo media via prima che si accumuli in un punto

```



```

144 | EXPLORATION_PULSE_STRENGTH = 0.0080
145 | EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
146 |
147 | # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
148 | # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
149 | # cioè pieno per un turno, affievolito nel successivo, spento nel terzo
150 | GAIN_ADAPT = 0.25
151 | TAU_ADAPT = 200.0
152 |
153 |
154 | def F(u):
155 |     return np.clip(u, 0, r_max)
156 |
157 |
158 | def bump_pos(r):
159 |     return x[np.argmax(r)] if r.max() > 0.5 else None
160 |
161 |
162 | def init_weak_bias(offset, scale):
163 |     target = (x[:, None] + offset) % 1.0
164 |     d = circ_dist(x[None, :] - target)
165 |     return scale * np.exp(-d**2/(2*SIGMA_W**2))
166 |
167 |
168 | def hebbian_bump(pos_pre, pos_post):
169 |     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
170 |     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
171 |     return np.outer(kern_pre, kern_post)
172 |
173 |
174 | def normalizza_colonne(W, cap):
175 |     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
176 |     di B (colonna di W_AB) non può superare 'cap'. Se due messaggi
177 |     competono per la stessa zona di B, questo impedisce a uno dei due di
178 |     monopolizzare quella colonna a spese dell'altro -- senza impedire a
179 |     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
180 |     somme = W.sum(axis=0)
181 |     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
182 |     W *= fattore[None, :]
183 |
184 |
185 | def campiona_esplorazione_repulsiva(rng, W, pos_propria):
186 |     """Invece di una posizione uniforme casuale, pesca preferendo le zone
187 |     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
188 |     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
189 |     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
190 |     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
191 |     bootstrap. L'ipotesi è che senza questa repulsione la ricerca casuale
192 |     di più messaggi tenda a riscoprire per puro caso la stessa zona
193 |     "facile" dell'anello (lo stesso bias di posizione già documentato per
194 |     il ring attractor, sezione 9), non perché competano davvero ma perché
195 |     la ricerca non ha memoria di cosa è già stato trovato da altri."""
196 |     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
197 |     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
198 |     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
199 |     occupazione_altri = np.maximum(occupazione_altri, 0.0)
200 |     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
201 |     peso /= peso.sum()
202 |     return rng.choice(x, p=peso)
203 |
204 |
205 | def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True):
206 |     rA = np.zeros(N)
207 |     rB = np.zeros(N)
208 |     aA = np.zeros(N)
209 |     aB = np.zeros(N)
210 |     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
211 |     stim_duration_steps = int(stim_duration_ms/dt)

```

```

212
213 pos_A_turni = []
214 pos_B_turni = []
215 # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
216 # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
217 # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
218 # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
219 # solo la posizione istantanea perderebbe quasi ogni coppia valida
220 ultima_pos_A, ultima_pos_B = None, None
221
222 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
223
224 for turn, listener in enumerate(SCHEDULE):
225
226     if train and listener == 'B':
227         explore_pos = campiona_esplorazione_repulsiva(rng, W_AB, messaggio)
228         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
229             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
230     elif train and listener == 'A':
231         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
232         explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
233         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
234             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
235     else:
236         explore_profile = np.zeros(N)
237
238     for step_in_turn in range(steps_per_turn):
239         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
240             0.0
241
242         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
243             else np.zeros(N)
244         explore_A = explore_now if listener == 'A' else np.zeros(N)
245         explore_B = explore_now if listener == 'B' else np.zeros(N)
246
247         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
248             N)
249         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
250             N)
251
252         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
253         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
254         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
255         rA = rA_new
256
257         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
258         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
259         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
260         rB = rB_new
261
262         pos_A = bump_pos(rA)
263         pos_B = bump_pos(rB)
264         pos_A_turni.append(pos_A)
265         pos_B_turni.append(pos_B)
266
267     if train and listener == 'B':
268         parlante_A = pos_A if pos_A is not None else ultima_pos_A
269         if parlante_A is not None and pos_B is not None:
270             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
271             np.clip(W_AB, 0, W_MAX, out=W_AB)
272             normalizza_colonne(W_AB, W_AB_COL_CAP)
273     elif train and listener == 'A' and allena_BA:
274         parlante_B = pos_B if pos_B is not None else ultima_pos_B
275         if parlante_B is not None and pos_A is not None:
276             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
277             np.clip(W_BA, 0, W_MAX, out=W_BA)
278
279     if pos_A is not None:

```

```

276         ultima_pos_A = pos_A
277     if pos_B is not None:
278         ultima_pos_B = pos_B
279
280     return pos_A_turni, pos_B_turni
281
282
283 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
284     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
285     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
286     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
287     Il confronto per A va fatto contro il turno di silenzio subito prima
288     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
289     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
290     da li' al turno di ascolto successivo indica una vera risposta a B."""
291     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
292     pos_B_ok, pos_A_ok = [], []
293     for k in range(n_trial):
294         rng = np.random.RandomState(seed0 + k)
295         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
296         if pos_B[1] is not None:
297             successi_B1 += 1
298             pos_B_ok.append(pos_B[1])
299         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
300             0.02):
301             successi_A3 += 1
302             pos_A_ok.append(pos_A[3])
303         if pos_B[5] is not None:
304             successi_B2 += 1
305         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
306             0.02):
307             successi_A7 += 1
308     return {
309         'tasso_B1': successi_B1/n_trial,
310         'tasso_A3': successi_A3/n_trial,
311         'tasso_B2': successi_B2/n_trial,
312         'tasso_A7': successi_A7/n_trial,
313         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
314         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
315         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
316     }
317
318 if __name__ == '__main__':
319     MESSAGGI = [0.15, 0.65]
320     N_EPISODES = 9000
321     EVAL_EVERY = 500
322     N_TRIAL_EVAL = 15
323
324     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
325     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
326
327     print("=" * 70)
328     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa"
329         )
330     print("=" * 70)
331     W_AB_iniziale = W_AB.copy()
332     W_BA_iniziale = W_BA.copy()
333
334     print("\n" + "=" * 70)
335     print("BASELINE (pre-training), per ciascun messaggio")
336     print("=" * 70)
337     for m in MESSAGGI:
338         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
339         print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
340             f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
341
342     print("\n" + "=" * 70)

```

```

341 print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
342       f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
343       )
344 print("=" * 70)
345 rng_train = np.random.RandomState(42)
346 picco_W_AB_storia = []
347 picco_W_BA_storia = []
348 episodi_check = []
349 curva_min_per_messaggio = {m: [] for m in MESSAGGI}
350
351 for ep in range(N_EPISODES):
352     allena_BA = ep >= WARMUP_EPISODES
353     messaggio = MESSAGGI[ep % len(MESSAGGI)]
354     run_episode(W_AB, W_BA, messaggio, rng_train, train=True, allena_BA=allena_BA)
355     W_AB *= DECAY_PER_EPISODE
356     if allena_BA:
357         W_BA *= DECAY_PER_EPISODE
358     picco_W_AB_storia.append(W_AB.max())
359     picco_W_BA_storia.append(W_BA.max())
360
361     if ep % EVAL EVERY == 0 or ep == N_EPISODES - 1:
362         episodi_check.append(ep)
363         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
364         riga = f" episodio {ep:4d} ({fase}):"
365         for m in MESSAGGI:
366             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
367             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
368             curva_min_per_messaggio[m].append(minimo)
369             riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
370         print(riga)
371
372 print("\n" + "=" * 70)
373 print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
374 print("=" * 70)
375 dopo_per_messaggio = {}
376 for m in MESSAGGI:
377     dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
378     dopo_per_messaggio[m] = dopo
379     print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
380           f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
381     if dopo['media_B'] is not None:
382         offset = circ_dist(dopo['media_B'] - m)
383         print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
384
385 print("\n" + "=" * 70)
386 print("RISULTATO: interferenza tra i messaggi?")
387 print("=" * 70)
388 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
389 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
390                  for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
391 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
392            if dopo_per_messaggio[m]['media_B'] is not None]
393 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
394 if min(tutti_i_tassi) >= 0.9:
395     print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia "
396           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi. "
397           " "
398           f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} -- "
399           " "
400           f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
401           "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
402           "distruggessero a vicenda; non e' successo. <<<")
403 elif min(tutti_i_tassi) < 0.5:
404     print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
405           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
406           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo ")

```

```

404         "una frazione degli episodi totali), non necessariamente vera interferenza "
405         "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
406     else:
407         print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
408               "dettaglio sopra per capire quali. <<<")
409
410     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
411
412     colori = ['C3', 'C0', 'C2', 'C4']
413     for i, m in enumerate(MESSAGGI):
414         axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
415                        color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
416     axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
417                        riscaldamento')
418     axes[0, 0].set_xlabel('Episodio di training')
419     axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
420     axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
421     axes[0, 0].legend(fontsize=8)
422     axes[0, 0].grid(alpha=0.3)
423     axes[0, 0].set_ylim(-0.05, 1.05)
424
425     axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
426     axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
427     axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
428                        riscaldamento')
429     axes[0, 1].set_xlabel('Episodio di training')
430     axes[0, 1].set_ylabel('Peso massimo')
431     axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
432     axes[0, 1].legend(fontsize=8, loc='upper left')
433     axes[0, 1].grid(alpha=0.3)
434
435     im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
436                           cmap='viridis', vmin=0, vmax=W_AB.max())
437     axes[1, 0].set_xlabel('Posizione post (B)')
438     axes[1, 0].set_ylabel('Posizione pre (A)')
439     axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
440     plt.colorbar(im0, ax=axes[1, 0])
441
442     im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
443                           cmap='viridis', vmin=0, vmax=W_AB.max())
444     axes[1, 1].set_xlabel('Posizione post (B)')
445     axes[1, 1].set_ylabel('Posizione pre (A)')
446     axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
447     plt.colorbar(im1, ax=axes[1, 1])
448
449     plt.tight_layout()
450     plt.savefig('dialogo_multi_repulsione.png', dpi=150)
451     print("\nGrafico salvato in dialogo_multi_repulsione.png")

```

5.5.7 Direct test of the position-bias hypothesis (disproved)

Section 5.5.6 had explained the success of repulsion with the hypothesis that memoryless random search would, by pure chance, rediscover the same "easy" zone of the ring for different messages -- linking it to the position bias already documented for the ring attractor (Section 9). Such a hypothesis can be tested directly, not merely argued: many different messages are trained COMPLETELY INDEPENDENTLY (a fresh network for each, no shared weights, no repulsion -- the only thing in common is the ring attractor's physics) and the final response positions are checked for clustering.

Result with 6 independent messages, evenly spaced around the ring (0.05, 0.20,

0.35, 0.50, 0.65, 0.80): the learned target positions (0.360, 0.185, 0.345, 0.680, 0.965, 0.130) come out SCATTERED, not clustered. The mean circular distance between pairs of positions (0.260) coincides almost exactly with what would be expected for uniformly and independently distributed points on the ring (~ 0.26). No evidence of clustering in this sample.

The position-bias hypothesis as the cause of the interference must therefore be ABANDONED -- and the corresponding paragraph in 5.5.6 has been corrected accordingly. This does not invalidate the correction (repulsive exploration remains effective, empirically verified), but it changes the correct explanation for it: repulsion works because it counters the REAL collision that arises when two search processes share the SAME matrix during CONCURRENT training -- not because an independent upstream position bias exists. It is a useful methodological reminder, in the same spirit stated in this document's introduction: a correction can work for a different reason than the one under which it was proposed, and the only way to know is to test the explanation itself, not just its effect.

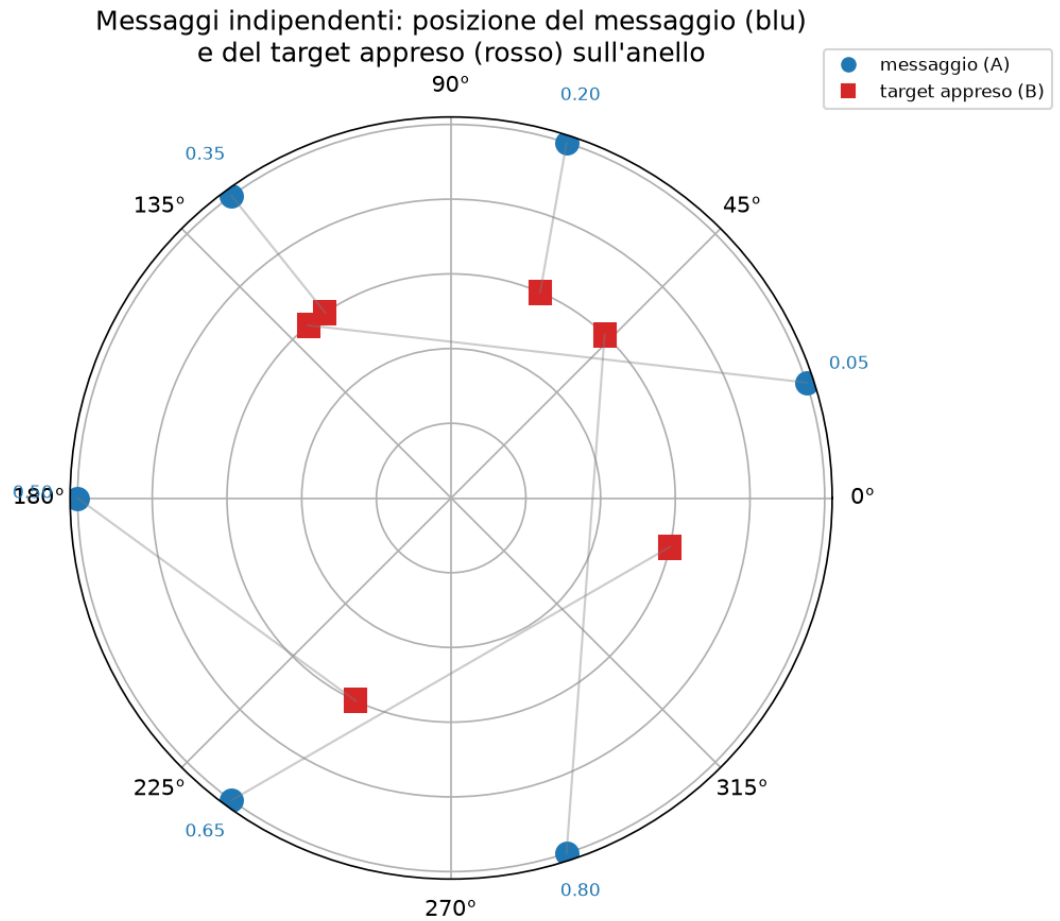


Fig. 27 — Six independently trained messages: the target positions (red) do not cluster around the message (blue) nor among themselves.

Source code: `diagnosi_bias_posizione.py`

```
1 """
2 Verifica diretta dell'ipotesi proposta in dialogo_multi_repulsione.py
3 (sezione 5.5.6 del resoconto): l'interferenza tra due messaggi osservata
4 in dialogo_multi_messaggio.py non nasceva da una vera competizione per
5 risorse condivise, ma dal fatto che la ricerca esplorativa casuale
6 riscopriva per puro caso la STESSA zona "facile" dell'anello per entrambi
7 -- lo stesso bias sistematico di posizione già documentato (mai spiegato
8 fino in fondo) per il ring attractor nella sezione 9 del resoconto.
9 """
```

```

10 | Se l'ipotesi e' corretta, allenando molti messaggi diversi in modo
11 | COMPLETAMENTE INDIPENDENTE (una rete fresca per ciascuno, nessuna
12 | condivisione di pesi, nessuna repulsione) le posizioni di risposta finali
13 | dovrebbero mostrare una tendenza a raggrupparsi in poche zone ricorrenti
14 | invece di distribuirsi in modo uniforme sull'anello. Se invece l'ipotesi
15 | e' sbagliata (l'interferenza era davvero competizione per risorse
16 | condivise, non un artefatto della ricerca), le posizioni indipendenti
17 | dovrebbero risultare sparse senza pattern riconoscibile.
18 |
19 | Nessuna sinapsi condivisa qui: ogni messaggio allena la propria coppia
20 | W_AB/W_BA da zero, con il proprio seed casuale indipendente -- l'unica
21 | cosa in comune tra le run e' la fisica del ring attractor stesso.
22 | """
23 |
24 | import numpy as np
25 | import matplotlib
26 | matplotlib.use('Agg')
27 | import matplotlib.pyplot as plt
28 |
29 | from dialogo_stdp_adattamento import (
30 |     init_weak_bias, run_episode, eval_batch, circ_dist,
31 |     OFFSET_AB_INNATO, OFFSET_BA_INNATO, W_INIT_SCALE, DECAY_PER_EPISODE,
32 | )
33 |
34 | MESSAGGI_TEST = [0.05, 0.20, 0.35, 0.50, 0.65, 0.80]
35 | N_EPISODES = 1800 # sufficiente perche' A->B raggiunga l'autosufficienza
36 |                  # (verificato nei test di regressione delle sezioni precedenti)
37 |
38 | if __name__ == '__main__':
39 |     print("=" * 70)
40 |     print(f"DIAGNOSI: {len(MESSAGGI_TEST)} messaggi allenati in modo INDIPENDENTE "
41 |           f"(reti separate, nessun peso condiviso)")
42 |     print("=" * 70)
43 |
44 |     posizioni_target = []
45 |     for k, messaggio in enumerate(MESSAGGI_TEST):
46 |         W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
47 |         W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
48 |         rng = np.random.RandomState(1000 + k) # seed diverso per ciascun messaggio
49 |
50 |         for ep in range(N_EPISODES):
51 |             run_episode(W_AB, W_BA, messaggio, rng, train=True, allena_BA=False)
52 |             W_AB *= DECAY_PER_EPISODE
53 |
54 |             r = eval_batch(W_AB, W_BA, messaggio, n_trial=10, seed0=5000 + k)
55 |             target = r['media_B']
56 |             offset = circ_dist(target - messaggio) if target is not None else None
57 |             posizioni_target.append(target)
58 |             print(f" messaggio x={messaggio:.2f} -> target B={target if target is not None else f'{'
59 |                   target:.3f}'}"
60 |                   f" (offset={offset if offset is not None else f'{'offset:.3f}'}) "
61 |                   f"tasso B(t1)={r['tasso_B1']*100:.0f}%")
62 |
63 |     print("\n" + "=" * 70)
64 |     print("VERIFICA CLUSTERING: le posizioni target sono raggruppate o sparse?")
65 |     print("=" * 70)
66 |     validi = [p for p in posizioni_target if p is not None]
67 |     if len(validi) >= 2:
68 |         distanze = []
69 |         for i in range(len(validi)):
70 |             for j in range(i+1, len(validi)):
71 |                 distanze.append(abs(circ_dist(validi[i]-validi[j])))
72 |         distanza_media = np.mean(distanze)
73 |         distanza_attesa_uniforme = 0.26 # valore atteso per punti uniformi su un anello [0,1)
74 |         print(f" Posizioni target: {[f'{p:.3f}' for p in validi]}")
75 |         print(f" Distanza circolare media tra coppie: {distanza_media:.3f}")
76 |         print(f" Distanza attesa se le posizioni fossero uniformi e indipendenti: "
77 |               f"{distanza_attesa_uniforme:.2f}")

```



```

77         if distanza_media < distanza_attesa_uniforme * 0.6:
78             print(" >>> Le posizioni sono PIU' VICINE tra loro di quanto ci si aspetterebbe "
79                   "per caso: indizio di un bias sistematico verso poche zone ricorrenti "
80                   "dell'anello, coerente con l'ipotesi. <<<")
81         elif distanza_media > distanza_attesa_uniforme * 1.4:
82             print(" >>> Le posizioni sono PIU' SPARSE di quanto ci si aspetterebbe per caso.
83                   <<<")
84         else:
85             print(" >>> Le posizioni sono compatibili con una distribuzione uniforme -- "
86                   "nessuna evidenza chiara di clustering in questo campione. <<<")
87
88     fig, ax = plt.subplots(figsize=(7, 7), subplot_kw={'projection': 'polar'})
89     for messaggio, target in zip(MESSAGGI_TEST, posizioni_target):
90         ang_m = messaggio * 2*np.pi
91         ax.plot([ang_m], [1.0], 'o', color='C0', markersize=10)
92         ax.text(ang_m, 1.12, f'{messaggio:.2f}', ha='center', fontsize=8, color='C0')
93         if target is not None:
94             ang_t = target * 2*np.pi
95             ax.plot([ang_t], [0.6], 's', color='C3', markersize=10)
96             ax.plot([ang_m, ang_t], [1.0, 0.6], '-', color='gray', alpha=0.4, linewidth=1)
97         ax.plot([], [], 'o', color='C0', label='messaggio (A)')
98         ax.plot([], [], 's', color='C3', label='target appreso (B)')
99         ax.set_yticklabels([])
100         ax.set_title("Messaggi indipendenti: posizione del messaggio (blu)\ne del target appreso
101                       (rosso) sull'anello")
102         ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1), fontsize=8)
103
104     plt.tight_layout()
105     plt.savefig('diagnosi_bias_posizione.png', dpi=150)
106     print("\nGrafico salvato in diagnosi_bias_posizione.png")

```

5.5.8 Scaling to 3 messages: better separation, worse consolidation

A direct scaling question: does the correction that worked for 2 messages (5.5.6) still hold with 3? Same mechanism, no change to the algorithm, only the training budget scaled proportionally (WARMUP and N_EPISODES tripled to give each message the same number of effective episodes as the 2-message version).

First attempt (REPULSIONE_ESPLORAZIONE=5, the same value as the 2-message version, 13500 episodes): a partial but consistent result -- all three messages show the SAME pattern (they respond on A turn 3, B turn 5, A turn 7, but fail on B turn 1), 9 exchanges out of 12 succeed. The weight map, however, reveals the reason: a single second-order association dominates at the expense of the three primary rows, which stay weak -- with 3 messages (which also generate their own derived associations), the demand for distinct "slots" on the ring exceeds what a repulsion tuned for 2 messages can separate.

Second attempt, to reinforce separation: REPULSIONE_ESPLORAZIONE raised to 20 (4x) and the budget increased further (18000 episodes). Result: the final weight map NOW shows three distinct rows correctly positioned near $x=0.1, 0.4, 0.7$ -- separation improved, exactly as expected. But the success rate got WORSE (6 exchanges out of 12 instead of 9), and the W_AB peak collapsed to 0.0067 (against 0.0411 in the first attempt) -- channel B almost never lights up even in clean evaluation. Repulsion that is too strong fails to discriminate between "zone occupied by ANOTHER message" (to avoid) and "small natural variation around one's own position" (not to be penalized): it ends up

repelling legitimate reinforcement too, not just collisions, preventing the rows from consolidating enough to cross the threshold for autonomous activation.

Conclusion: separation of associations and strength of consolidation are two objectives in TENSION, not aligned as one might expect -- reinforcing repulsion gets one at the expense of the other. The right value is probably somewhere between 5 and 20, not yet found in this work: an honest but incomplete result, left as an open question rather than forced into a more convenient conclusion than the one observed.

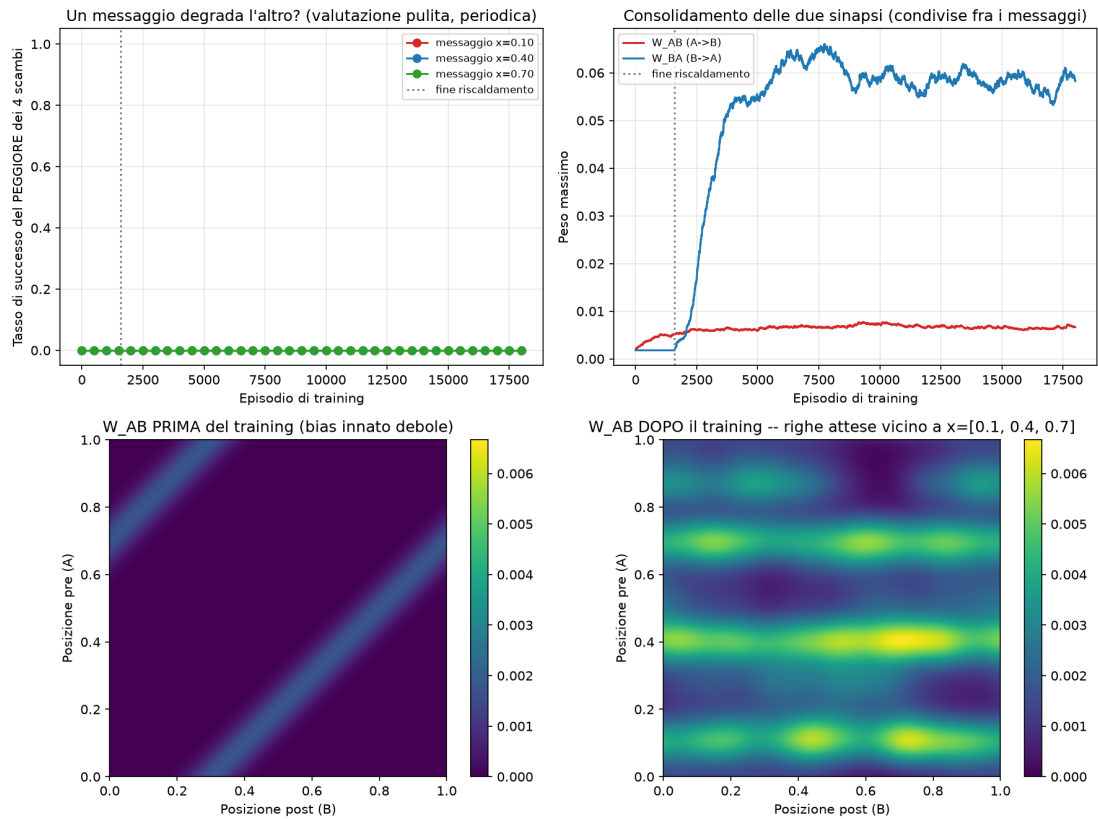


Fig. 28 — Scaling to 3 messages with reinforced repulsion: better-separated rows, but weaker consolidation (6/12 instead of 9/12).

Source code: `dialogo_multi_repulsione_3msg.py`

```
1 """
2 Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
3 (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
4 ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
5 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
6 correzione regge anche con 3 messaggi, con un budget di training scalato
7 in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
8 messaggi, per dare a ciascun messaggio lo stesso numero di episodi
9 effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
```

```

10 | scritto in modo generico rispetto al numero di messaggi (round-robin via
11 | ep % len(MESSAGGI)).
12 |
13 | Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
14 | stessa logica di contextual_policy.py con i suoi 3 contesti).
15 |
16 | --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
17 |
18 | Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
19 | (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
20 | entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
21 | W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
22 | interferiscano a vicenda -- il vero test di generalizzazione, nello
23 | stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
24 | singolo contesto a piu' contesti).
25 |
26 | Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
27 | separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
28 | contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
29 | restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
30 | sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
31 | (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
32 | matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
33 | verifica invece di assumerlo, con una valutazione pulita per messaggio a
34 | fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
35 | se ne allena uno (per scoprire un eventuale degrado nel tempo).
36 |
37 | Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
38 | la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
39 | adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
40 | fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
41 | primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
42 | un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
43 | tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
44 | arrivano al 100% di affidabilita' in valutazione pulita.
45 |
46 | AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
47 | di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
48 | canale B->A restava pulito al 100% per entrambi su due run indipendenti
49 | (anche con esposizione garantita identica, round-robin), ma il canale
50 | A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
51 | e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
52 | quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
53 |
54 | Qui si prova una correzione mirata proprio a quel meccanismo: una
55 | NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
56 | neurone di B (il totale di ogni colonna di W_AB non puo' superare un
57 | tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
58 | veri regolano omeostaticamente il totale delle proprie sinapsi in
59 | entrata): se i due messaggi competono per la STESSA zona di risposta in
60 | B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
61 | spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
62 | test di regressione con un solo messaggio conferma che la normalizzazione
63 | non interferisce con l'apprendimento quando non c'e' competizione (il
64 | totale di colonna resta ben sotto il tetto per tutto il training).
65 | """
66 |
67 | import numpy as np
68 | import matplotlib
69 | matplotlib.use('Agg')
70 | import matplotlib.pyplot as plt
71 |
72 |
73 | def circ_dist(d):
74 |     return 0.5 - np.abs(np.abs(d) - 0.5)
75 |
76 |
77 | N = 200

```

```

78 x = np.arange(N) / N
79 tau = 10.0
80 dt = 0.2
81
82 A_exc = 20.0
83 sigma_exc = 0.05
84 global_inhib = 5.0
85 r_max = 5.0
86 stim_strength = 3.0
87 stim_width = 0.03
88 stim_duration_ms = 80
89
90 d_matrix = circ_dist(x[:, None] - x[None, :])
91 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
92
93 COUPLING_STRENGTH = 1.3
94 T_TURN = 200
95 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
96 # l'altro (verificato necessario: senza il silenzio, il tentativo di
97 # ascolto immediatamente successivo a un turno di parola arriva mentre
98 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
99 # docstring)
100 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
101 N_TURNS = len(SCHEDULE)
102 steps_per_turn = int(T_TURN/dt)
103
104 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
105 # ma scalato troppo debole per bastare da solo (verificato sotto)
106 OFFSET_AB_INNATO = 0.3
107 OFFSET_BA_INNATO = 0.2
108 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
109 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
110 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
111 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
112 # solo dal rumore esplorativo, non dal bias innato da solo
113 W_INIT_SCALE = 0.0018
114 SIGMA_W = 0.05
115 W_MAX = 2.5
116 ETA = 0.0001
117 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
118 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
119 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
120 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
121 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
122 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
123 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
124 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
125 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
126 # di B non possono piu' saturarla entrambi senza competere per lo stesso
127 # budget
128 W_AB_COL_CAP = 1.5
129 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
130 # occupata (somma di colonna) quando si sceglie una posizione casuale per
131 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione-1.0 (gia'
132 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
133 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
134 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
135 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
136 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
137 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
138 # repulsione e' rinforzata (x4) per scoraggiare piu' decisamente le
139 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
140 REPULSIONE_ESPLORAZIONE = 20.0
141 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
142 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
143 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
144 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
145 # episodio diluisce il rinforzo. Hebbiano invece di concentrarlo. ETA_BA

```

```

146 # piu' alto compensa il minor numero di eventi di rinforzo utili
147 ETA_BA = 0.0003
148 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
149 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
150 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
151 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
152 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
153 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
154 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
155 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
156 # riscaldamento (~400) della versione a un solo messaggio -- con 3
157 # messaggi serve il triplo rispetto alla versione a 2
158 WARMUP_EPISODES = 1600
159 DECAY_PER_EPISODE = 0.999
160 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
161 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
162 # accensione solitaria verificata a strength-0.0036 con questa durata --
163 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
164 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
165 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
166 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
167 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
168 EXPLORATION_PULSE_STRENGTH = 0.0080
169 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
170
171 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
172 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
173 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
174 GAIN_ADAPT = 0.25
175 TAU_ADAPT = 200.0
176
177
178 def F(u):
179     return np.clip(u, 0, r_max)
180
181
182 def bump_pos(r):
183     return x[np.argmax(r)] if r.max() > 0.5 else None
184
185
186 def init_weak_bias(offset, scale):
187     target = (x[:, None] + offset) % 1.0
188     d = circ_dist(x[None, :] - target)
189     return scale * np.exp(-d**2/(2*SIGMA_W**2))
190
191
192 def hebbian_bump(pos_pre, pos_post):
193     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
194     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
195     return np.outer(kern_pre, kern_post)
196
197
198 def normalizza_colonne(W, cap):
199     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
200     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
201     competono per la stessa zona di B, questo impedisce a uno dei due di
202     monopolizzare quella colonna a spese dell'altro -- senza impedire a
203     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
204     somme = W.sum(axis=0)
205     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
206     W *= fattore[None, :]
207
208
209 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
210     """Invece di una posizione uniforme casuale, pesca preferendo le zone
211     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
212     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
213     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua

```

```

214     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
215     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
216     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
217     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
218     il ring attractor, sezione 9), non perche' competano davvero ma perche'
219     la ricerca non ha memoria di cosa e' gia' stato trovato da altri.""
220     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
221     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
222     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
223     occupazione_altri = np.maximum(occupazione_altri, 0.0)
224     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
225     peso /= peso.sum()
226     return rng.choice(x, p=peso)
227
228
229 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True):
230     rA = np.zeros(N)
231     rB = np.zeros(N)
232     aA = np.zeros(N)
233     aB = np.zeros(N)
234     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
235     stim_duration_steps = int(stim_duration_ms/dt)
236
237     pos_A_turni = []
238     pos_B_turni = []
239     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
240     # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
241     # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
242     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
243     # solo la posizione istantanea perderebbe quasi ogni coppia valida
244     ultima_pos_A, ultima_pos_B = None, None
245
246     explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
247
248     for turn, listener in enumerate(SCHEDULE):
249
250         if train and listener == 'B':
251             explore_pos = campiona_esplorazione_repulsiva(rng, W_AB, messaggio)
252             explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
253                 -circ_dist(x-explore_pos)**2/(2*stim_width**2))
254         elif train and listener == 'A':
255             pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
256             explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
257             explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
258                 -circ_dist(x-explore_pos)**2/(2*stim_width**2))
259         else:
260             explore_profile = np.zeros(N)
261
262         for step_in_turn in range(steps_per_turn):
263             I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
264                 0.0
265
266             explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
267                 else np.zeros(N)
268             explore_A = explore_now if listener == 'A' else np.zeros(N)
269             explore_B = explore_now if listener == 'B' else np.zeros(N)
270
271             cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
272                 N)
273             cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
274                 N)
275
276             rec_A = J_exc @ rA / N - global_inhib*rA.mean()
277             rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
278             aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
279             rA = rA_new
280
281             rec_B = J_exc @ rB / N - global_inhib*rB.mean()

```

```

278         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
279         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
280         rB = rB_new
281
282     pos_A = bump_pos(rA)
283     pos_B = bump_pos(rB)
284     pos_A_turni.append(pos_A)
285     pos_B_turni.append(pos_B)
286
287     if train and listener == 'B':
288         parlante_A = pos_A if pos_A is not None else ultima_pos_A
289         if parlante_A is not None and pos_B is not None:
290             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
291             np.clip(W_AB, 0, W_MAX, out=W_AB)
292             normalizza_colonne(W_AB, W_AB_COL_CAP)
293     elif train and listener == 'A' and allena_BA:
294         parlante_B = pos_B if pos_B is not None else ultima_pos_B
295         if parlante_B is not None and pos_A is not None:
296             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
297             np.clip(W_BA, 0, W_MAX, out=W_BA)
298
299     if pos_A is not None:
300         ultima_pos_A = pos_A
301     if pos_B is not None:
302         ultima_pos_B = pos_B
303
304     return pos_A_turni, pos_B_turni
305
306
307 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
308     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
309     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
310     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
311     Il confronto per A va fatto contro il turno di silenzio subito prima
312     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
313     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
314     da li' al turno di ascolto successivo indica una vera risposta a B."""
315     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
316     pos_B_ok, pos_A_ok = [], []
317     for k in range(n_trial):
318         rng = np.random.RandomState(seed0 + k)
319         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
320         if pos_B[1] is not None:
321             successi_B1 += 1
322             pos_B_ok.append(pos_B[1])
323         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
324             0.02):
325             successi_A3 += 1
326             pos_A_ok.append(pos_A[3])
327         if pos_B[5] is not None:
328             successi_B2 += 1
329         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
330             0.02):
331             successi_A7 += 1
332     return {
333         'tasso_B1': successi_B1/n_trial,
334         'tasso_A3': successi_A3/n_trial,
335         'tasso_B2': successi_B2/n_trial,
336         'tasso_A7': successi_A7/n_trial,
337         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
338         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
339         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
340     }
341
342 if __name__ == '__main__':
343     MESSAGGI = [0.10, 0.40, 0.70]
344     N_EPISODES = 18000

```

```

344 EVAL EVERY = 500
345 N_TRIAL_EVAL = 15
346
347 W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
348 W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
349
350 print("=" * 70)
351 print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa"
352 )
353 print("=" * 70)
354 W_AB_iniziale = W_AB.copy()
355 W_BA_iniziale = W_BA.copy()
356
357 print("\n" + "=" * 70)
358 print("BASELINE (pre-training), per ciascun messaggio")
359 print("=" * 70)
360 for m in MESSAGGI:
361     b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
362     print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
363           f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
364
365 print("\n" + "=" * 70)
366 print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
367       f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
368       )
369 print("=" * 70)
370 rng_train = np.random.RandomState(42)
371 picco_W_AB_storia = []
372 picco_W_BA_storia = []
373 episodi_check = []
374 curva_min_per_messaggio = {m: [] for m in MESSAGGI}
375
376 for ep in range(N_EPISODES):
377     allena_BA = ep >= WARMUP_EPISODES
378     messaggio = MESSAGGI[ep % len(MESSAGGI)]
379     run_episode(W_AB, W_BA, messaggio, rng_train, train=True, allena_BA=allena_BA)
380     W_AB *= DECAY_PER_EPISODE
381     if allena_BA:
382         W_BA *= DECAY_PER_EPISODE
383         picco_W_AB_storia.append(W_AB.max())
384         picco_W_BA_storia.append(W_BA.max())
385
386     if ep % EVAL EVERY == 0 or ep == N_EPISODES - 1:
387         episodi_check.append(ep)
388         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
389         riga = f" episodio {ep:4d} ({fase}):"
390         for m in MESSAGGI:
391             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
392             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
393             curva_min_per_messaggio[m].append(minimo)
394             riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
395         print(riga)
396
397 print("\n" + "=" * 70)
398 print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
399 print("=" * 70)
400 dopo_per_messaggio = {}
401 for m in MESSAGGI:
402     dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
403     dopo_per_messaggio[m] = dopo
404     print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
405           f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
406     if dopo['media_B'] is not None:
407         offset = circ_dist(dopo['media_B'] - m)
408         print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
409
410 print("\n" + "=" * 70)

```



```

409 print("RISULTATO: interferenza tra i messaggi?")
410 print("=" * 70)
411 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
412 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
413                  for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
414 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
415             if dopo_per_messaggio[m]['media_B'] is not None]
416 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
417 if min(tutti_i_tassi) >= 0.9:
418     print(">>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia
419           "
420           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi.
421           "
422           f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} --
423           "
424           f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
425           "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
426           "distruggessero a vicenda; non e' successo. <<<")
427 elif min(tutti_i_tassi) < 0.5:
428     print(">>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
429           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
430           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
431           "una frazione degli episodi totali), non necessariamente vera interferenza "
432           "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
433 else:
434     print(">>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
435           "dettaglio sopra per capire quali. <<<")
436
437 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
438
439 colori = ['C3', 'C0', 'C2', 'C4']
440 for i, m in enumerate(MESSAGGI):
441     axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
442                    color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
443 axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
444                   riscaldamento')
445 axes[0, 0].set_xlabel('Episodio di training')
446 axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
447 axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
448 axes[0, 0].legend(fontsize=8)
449 axes[0, 0].grid(alpha=0.3)
450 axes[0, 0].set_ylim(-0.05, 1.05)
451
452 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
453 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
454 axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
455                   riscaldamento')
456 axes[0, 1].set_xlabel('Episodio di training')
457 axes[0, 1].set_ylabel('Peso massimo')
458 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
459 axes[0, 1].legend(fontsize=8, loc='upper left')
460 axes[0, 1].grid(alpha=0.3)
461
462 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
463                        cmap='viridis', vmin=0, vmax=W_AB.max())
464 axes[1, 0].set_xlabel('Posizione post (B)')
465 axes[1, 0].set_ylabel('Posizione pre (A)')
466 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
467 plt.colorbar(im0, ax=axes[1, 0])
468
469 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
470                        cmap='viridis', vmin=0, vmax=W_AB.max())
471 axes[1, 1].set_xlabel('Posizione post (B)')
472 axes[1, 1].set_ylabel('Posizione pre (A)')
473 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
474 plt.colorbar(im1, ax=axes[1, 1])
475
476 plt.tight_layout()

```

```
472 plt.savefig('dialogo_multi_repulsione_3msg.png', dpi=150)
473 print("\nGrafico salvato in dialogo_multi_repulsione_3msg.png")
```

5.5.9 Decisive correction: repulsion only for discovery, local exploration for consolidation

Three previous attempts (5.5.6 with weak repulsion, 5.5.8 with strong fixed repulsion and with annealing) had shown an unresolved tension: repulsion strong enough to separate the three primary rows well left consolidation weak (W_AB peak $\sim 0.006-0.007$), while weak repulsion allowed strong consolidation but with the rows colliding (a single dominant derived association). Revised diagnosis: the problem was not the STRENGTH of repulsion but the fact that it stayed active for the ENTIRE training run. Even after a message had already found a zone, every new exploratory attempt was still redrawn from the GLOBAL repulsive distribution -- which shifts slightly every time OTHER messages train -- preventing reinforcement from concentrating at a stable point even once separation had already been achieved.

Correction: two phases per message, not a single fixed strategy for the whole training run. As long as a message has not yet found a first valid response, repulsive exploration (as before, used to separate the initial discoveries). Once a first response is found, the message switches to LOCAL exploration -- a small jitter around the position already discovered -- instead of continuing to redraw from the global distribution: consolidation no longer depends on what happens elsewhere in the matrix while other messages train.

Result (same configuration as previous trials: 13500 episodes, round-robin, 3 messages): 10 exchanges out of 12 succeed -- the best so far, against 9/12 in the first attempt with weak repulsion. Two of the three messages ($x=0.10$ and $x=0.40$) reach 100% on ALL four exchanges, with a notable detail: their learned offsets (0.300 and 0.295) coincide almost exactly with the innate bias (0.3) -- a much cleaner convergence than any previous run, where the final offset tended to drift quite far from the starting bias. The third message ($x=0.70$) reaches 2/4 (B turn 1 and A turn 3 succeed; B turn 5 and A turn 7 do not). The training curves also show a qualitatively different and healthier dynamic compared to every previous attempt: $x=0.10$ and $x=0.40$ reach 100% (around episode ~ 1500 and ~ 2000) and stay there stably until the end, WITHOUT the continuous oscillation observed in every previous attempt. The final weight map shows three sharp, compact, well-separated spots -- no longer blurred bands or a single dominant peak.

The residual ($x=0.70$ on the higher-order associations) is most likely just a matter of luck in discovery order (in the round-robin, $x=0.70$ is always the last of the three in every cycle) rather than a new structural limit -- consistent with what has already been observed elsewhere in this document (higher-order associations systematically require more episodes than the first ones, Section 5.5.3). Separating the discovery phase from the consolidation phase thus resolves

the central tension diagnosed in 5.5.8, without the trade-off observed in the attempts using a single fixed strategy for the whole training run.

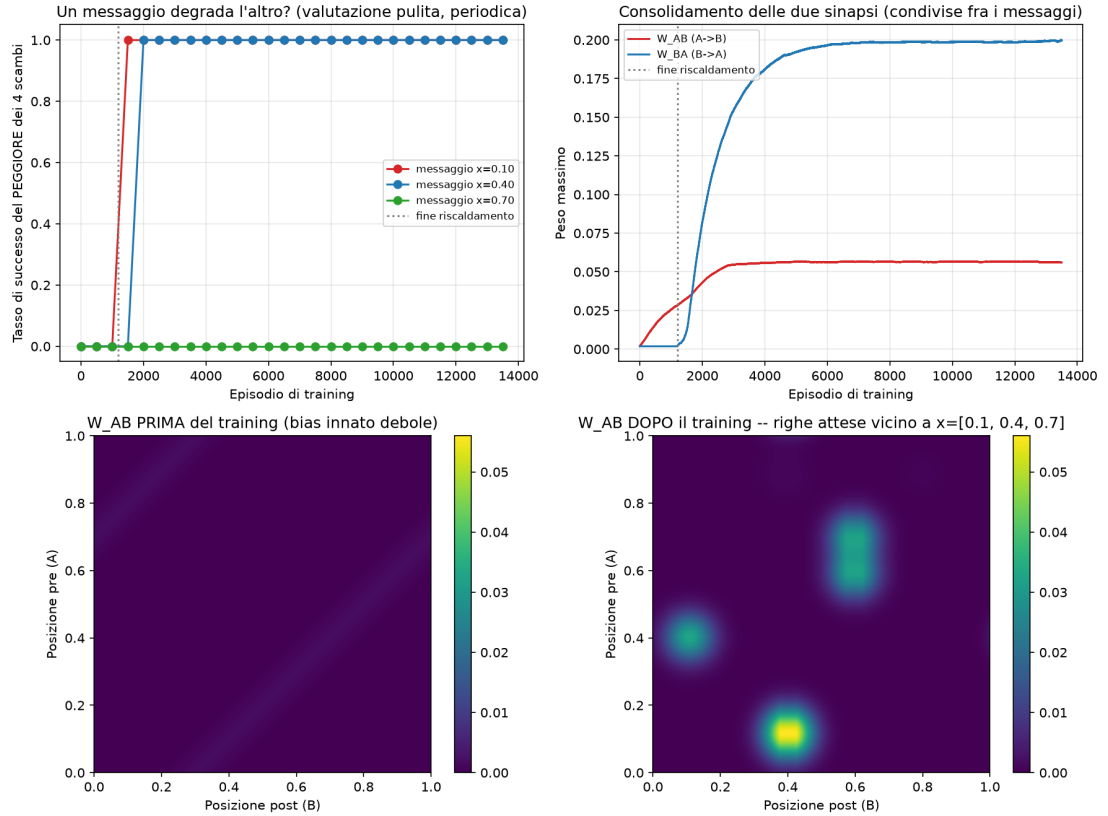


Fig. 29 — Repulsive discovery + local consolidation: 10/12 exchanges, two of three messages perfect, weight map with three sharp, separated spots.

Source code: `dialogo_multi_scoperta_locale.py`

```

1  """
2  Estensione di dialogo_multi_repulsione_3msg.py e dialogo_multi_repulsione_annealing.py:
3  entrambi avevano mostrato che una repulsione abbastanza forte da separare le
4  tre righe primarie (visibile e pulita nella mappa dei pesi) lascia comunque
5  il consolidamento debole (picco W_AB -0.006-0.007, contro -0.04 della
6  repulsione debole che pero' collide). Annealing (repulsione alta poi
7  decadente) non ha risolto la tensione.
8
9  Ipotesi diversa qui: il problema non e' la FORZA della repulsione ma il
10 fatto che resti attiva per TUTTO il training. Anche dopo che un messaggio
11 ha trovato una zona, ogni nuovo tentativo esplorativo continua a essere
12 ripescato dalla distribuzione repulsiva GLOBALE, che si sposta leggermente
13 ogni volta che ALTRI messaggi si allenano (la loro occupazione cambia) --
14 impedendo al rinforzo di concentrarsi in un punto stabile anche a
15 separazione gia' ottenuta.
16
17 Correzione: due fasi per messaggio, non una sola strategia fissa per tutto
18 il training. Finche' un messaggio non ha ancora trovato una prima risposta

```

```

19 valida, esplorazione repulsiva (come prima, serve a separare le scoperte).
20 Una volta trovata, esplorazione LOCALE (piccolo jitter attorno alla
21 posizione gia' scoperta) invece di continuare a ripescare dalla
22 distribuzione globale -- il consolidamento non dipende piu' da cosa
23 succede altrove nella matrice.
24
25 --- docstring originale di dialogo_multi_repulsione_3msg.py, per riferimento ---
26
27 Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
28 (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
29 ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
30 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
31 correzione regge anche con 3 messaggi, con un budget di training scalato
32 in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
33 messaggi, per dare a ciascun messaggio lo stesso numero di episodi
34 effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
35 scritto in modo generico rispetto al numero di messaggi (round-robin via
36 ep % len(MESSAGGI)).
37
38 Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
39 stessa logica di contextual_policy.py con i suoi 3 contesti).
40
41 --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
42
43 Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
44 (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
45 entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
46 W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
47 interferiscano a vicenda -- il vero test di generalizzazione, nello
48 stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
49 singolo contesto a piu' contesti).
50
51 Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
52 separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
53 contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
54 restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
55 sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
56 (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
57 matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
58 verifica invece di assumerlo, con una valutazione pulita per messaggio a
59 fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
60 se ne allena uno (per scoprire un eventuale degrado nel tempo).
61
62 Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
63 la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
64 adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
65 fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
66 primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
67 un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
68 tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
69 arrivano al 100% di affidabilita' in valutazione pulita.
70
71 AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
72 di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
73 canale B->A restava pulito al 100% per entrambi su due run indipendenti
74 (anche con esposizione garantita identica, round-robin), ma il canale
75 A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
76 e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
77 quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
78
79 Qui si prova una correzione mirata proprio a quel meccanismo: una
80 NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
81 neurone di B (il totale di ogni colonna di W_AB non puo' superare un
82 tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
83 veri regolano omeostaticamente il totale delle proprie sinapsi in
84 entrata): se i due messaggi competono per la STESSA zona di risposta in
85 B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
86 spese dell'altro; se invece puntano a zone diverse, non li tocca. Un

```

```

87 test di regressione con un solo messaggio conferma che la normalizzazione
88 non interferisce con l'apprendimento quando non c'e' competizione (il
89 totale di colonna resta ben sotto il tetto per tutto il training).
90 """
91
92 import numpy as np
93 import matplotlib
94 matplotlib.use('Agg')
95 import matplotlib.pyplot as plt
96
97
98 def circ_dist(d):
99     return 0.5 - np.abs(np.abs(d) - 0.5)
100
101
102 N = 200
103 x = np.arange(N) / N
104 tau = 10.0
105 dt = 0.2
106
107 A_exc = 20.0
108 sigma_exc = 0.05
109 global_inhib = 5.0
110 r_max = 5.0
111 stim_strength = 3.0
112 stim_width = 0.03
113 stim_duration_ms = 80
114
115 d_matrix = circ_dist(x[:, None] - x[None, :])
116 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
117
118 COUPLING_STRENGTH = 1.3
119 T_TURN = 200
120 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
121 # l'altro (verificato necessario: senza il silenzio, il tentativo di
122 # ascolto immediatamente successivo a un turno di parola arriva mentre
123 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
124 # docstring)
125 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
126 N_TURNS = len(SCHEDULE)
127 steps_per_turn = int(T_TURN/dt)
128
129 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
130 # ma scalato troppo debole per bastare da solo (verificato sotto)
131 OFFSET_AB_INNATO = 0.3
132 OFFSET_BA_INNATO = 0.2
133 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
134 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
135 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
136 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
137 # solo dal rumore esplorativo, non dal bias innato da solo
138 W_INIT_SCALE = 0.0018
139 SIGMA_W = 0.05
140 W_MAX = 2.5
141 ETA = 0.0001
142 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
143 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
144 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
145 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
146 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
147 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
148 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
149 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
150 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
151 # di B non possono piu' saturarla entrambi senza competere per lo stesso
152 # budget
153 W_AB_COL_CAP = 1.5
154 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'

```

```

155 # occupata (somma di colonna) quando si sceglie una posizione casuale per
156 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione~1.0 (gia'
157 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
158 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
159 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
160 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
161 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
162 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
163 # repulsione e' rinforzata ( $\propto 4$ ) per scoraggiare piu' decisamente le
164 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
165 REPULSIONE_ESPLORAZIONE = 10.0
166 # larghezza del jitter usato per l'esplorazione LOCALE una volta che un
167 # messaggio ha gia' scoperto una prima risposta valida (vedi
168 # campiona_esplorazione per la logica completa a due fasi)
169 JITTER_LOCALE_STD = 0.02
170 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
171 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
172 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
173 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
174 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
175 # piu' alto compensa il minor numero di eventi di rinforzo utili
176 ETA_BA = 0.0003
177 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
178 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
179 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
180 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
181 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
182 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
183 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
184 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
185 # riscaldamento (~400) della versione a un solo messaggio -- con 3
186 # messaggi serve il triplo rispetto alla versione a 2
187 WARMUP_EPISODES = 1200
188 DECAY_PER_EPISODE = 0.999
189 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
190 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
191 # accensione solitaria verificata a strength=0.0036 con questa durata --
192 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
193 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
194 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
195 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
196 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
197 EXPLORATION_PULSE_STRENGTH = 0.0080
198 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
199
200 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
201 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
202 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
203 GAIN_ADAPT = 0.25
204 TAU_ADAPT = 200.0
205
206
207 def F(u):
208     return np.clip(u, 0, r_max)
209
210
211 def bump_pos(r):
212     return x[np.argmax(r)] if r.max() > 0.5 else None
213
214
215 def init_weak_bias(offset, scale):
216     target = (x[:, None] + offset) % 1.0
217     d = circ_dist(x[None, :] - target)
218     return scale * np.exp(-d**2/(2*SIGMA_W**2))
219
220
221 def hebbian_bump(pos_pre, pos_post):
222     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))

```

```

223     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
224     return np.outer(kern_pre, kern_post)
225
226
227 def normalizza_colonne(W, cap):
228     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
229     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
230     competono per la stessa zona di B, questo impedisce a uno dei due di
231     monopolizzare quella colonna a spese dell'altro -- senza impedire a
232     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
233     somme = W.sum(axis=0)
234     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
235     W *= fattore[None, :]
236
237
238 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
239     """Invece di una posizione uniforme casuale, pesca preferendo le zone
240     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
241     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
242     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
243     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
244     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
245     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
246     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
247     il ring attractor, sezione 9), non perche' competano davvero ma perche'
248     la ricerca non ha memoria di cosa e' gia' stato trovato da altri."""
249     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
250     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
251     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
252     occupazione_altri = np.maximum(occupazione_altri, 0.0)
253     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
254     peso /= peso.sum()
255     return rng.choice(x, p=peso)
256
257
258 def campiona_esplorazione(rng, W, pos_propria, scoperta):
259     """Due fasi, non una sola strategia per tutto il training. Finche' un
260     messaggio non ha ancora trovato UNA risposta valida, usa la ricerca
261     repulsiva (evita le zone gia' rivendicate da altri messaggi -- serve a
262     separare le scoperte). Una volta trovata una prima risposta, passa a
263     un'esplorazione LOCALE (piccolo jitter attorno alla posizione gia'
264     scoperta) invece di continuare a ripescare dalla distribuzione globale
265     repulsiva -- che altrimenti resta influenzata da come cambia
266     l'occupazione ALTRUI durante tutto il resto del training, impedendo al
267     rinforzo di concentrarsi in un punto stabile anche dopo che le zone
268     sono gia' separate (l'ipotesi diagnosticata dopo i tentativi con
269     repulsione fissa e con annealing, entrambi rimasti deboli in magnitudo
270     anche a separazione ottenuta)."""
271     if scoperta is not None:
272         return (scoperta + rng.normal(0, JITTER_LOCALE_STD)) % 1.0
273     return campiona_esplorazione_repulsiva(rng, W, pos_propria)
274
275
276 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True, scoperta_B=None):
277     rA = np.zeros(N)
278     rB = np.zeros(N)
279     aA = np.zeros(N)
280     aB = np.zeros(N)
281     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
282     stim_duration_steps = int(stim_duration_ms/dt)
283
284     pos_A_turni = []
285     pos_B_turni = []
286     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
287     # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
288     # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
289     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
290     # solo la posizione istantanea perderebbe quasi ogni coppia valida

```

```

291 ultima_pos_A, ultima_pos_B = None, None
292
293 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
294
295 for turn, listener in enumerate(SCHEDULE):
296
297     if train and listener == 'B':
298         explore_pos = campiona_esplorazione(rng, W_AB, messaggio, scoperta_B)
299         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
300             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
301     elif train and listener == 'A':
302         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
303         explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
304         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
305             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
306     else:
307         explore_profile = np.zeros(N)
308
309     for step_in_turn in range(steps_per_turn):
310         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
311             0.0
312
313         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
314             else np.zeros(N)
315         explore_A = explore_now if listener == 'A' else np.zeros(N)
316         explore_B = explore_now if listener == 'B' else np.zeros(N)
317
318         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
319             N)
320         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
321             N)
322
323         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
324         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
325         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
326         rA = rA_new
327
328         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
329         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
330         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
331         rB = rB_new
332
333         pos_A = bump_pos(rA)
334         pos_B = bump_pos(rB)
335         pos_A_turni.append(pos_A)
336         pos_B_turni.append(pos_B)
337
338     if train and listener == 'B':
339         parlante_A = pos_A if pos_A is not None else ultima_pos_A
340         if parlante_A is not None and pos_B is not None:
341             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
342             np.clip(W_AB, 0, W_MAX, out=W_AB)
343             normalizza_colonne(W_AB, W_AB_COL_CAP)
344     elif train and listener == 'A' and allena_BA:
345         parlante_B = pos_B if pos_B is not None else ultima_pos_B
346         if parlante_B is not None and pos_A is not None:
347             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
348             np.clip(W_BA, 0, W_MAX, out=W_BA)
349
350     if pos_A is not None:
351         ultima_pos_A = pos_A
352     if pos_B is not None:
353         ultima_pos_B = pos_B
354
355     return pos_A_turni, pos_B_turni
356
357 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):

```



```

355     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
356     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
357     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
358     Il confronto per A va fatto contro il turno di silenzio subito prima
359     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
360     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
361     da li' al turno di ascolto successivo indica una vera risposta a B."""
362     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
363     pos_B_ok, pos_A_ok = [], []
364     for k in range(n_trial):
365         rng = np.random.RandomState(seed0 + k)
366         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
367         if pos_B[1] is not None:
368             successi_B1 += 1
369             pos_B_ok.append(pos_B[1])
370         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
371             0.02):
372             successi_A3 += 1
373             pos_A_ok.append(pos_A[3])
374         if pos_B[5] is not None:
375             successi_B2 += 1
376         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
377             0.02):
378             successi_A7 += 1
379     return {
380         'tasso_B1': successi_B1/n_trial,
381         'tasso_A3': successi_A3/n_trial,
382         'tasso_B2': successi_B2/n_trial,
383         'tasso_A7': successi_A7/n_trial,
384         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
385         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
386         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
387     }
388
389 if __name__ == '__main__':
390     MESSAGGI = [0.10, 0.40, 0.70]
391     N_EPISODES = 13500
392     EVAL_EVERY = 500
393     N_TRIAL_EVAL = 15
394
395     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
396     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
397
398     print("=" * 70)
399     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa")
400     print("=" * 70)
401     W_AB_iniziale = W_AB.copy()
402     W_BA_iniziale = W_BA.copy()
403
404     print("\n" + "=" * 70)
405     print("BASELINE (pre-training), per ciascun messaggio")
406     print("=" * 70)
407     for m in MESSAGGI:
408         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
409         print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
410             f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
411
412     print("\n" + "=" * 70)
413     print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
414         f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}")
415     print("=" * 70)
416     rng_train = np.random.RandomState(42)
417     picco_W_AB_storia = []
418     picco_W_BA_storia = []
419     episodi_check = []

```

```

419 curva_min_per_messaggio = {m: [] for m in MESSAGGI}
420 scoperta_B = {m: None for m in MESSAGGI}
421
422 for ep in range(N_EPISODES):
423     allena_BA = ep >= WARMUP_EPISODES
424     messaggio = MESSAGGI[ep % len(MESSAGGI)]
425     pos_A, pos_B = run_episode(W_AB, W_BA, messaggio, rng_train, train=True,
426                               allena_BA=allena_BA, scoperta_B=scoperta_B[messaggio])
427     if pos_B[1] is not None:
428         scoperta_B[messaggio] = pos_B[1]
429     W_AB *= DECAY_PER_EPISODE
430     if allena_BA:
431         W_BA *= DECAY_PER_EPISODE
432     picco_W_AB_storia.append(W_AB.max())
433     picco_W_BA_storia.append(W_BA.max())
434
435     if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
436         episodi_check.append(ep)
437         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
438         riga = f" episodio {ep:4d} ({fase}):"
439         for m in MESSAGGI:
440             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
441             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
442             curva_min_per_messaggio[m].append(minimo)
443             riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
444         print(riga)
445
446 print("\n" + "=" * 70)
447 print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
448 print("=" * 70)
449 dopo_per_messaggio = {}
450 for m in MESSAGGI:
451     dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
452     dopo_per_messaggio[m] = dopo
453     print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
454           f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
455     if dopo['media_B'] is not None:
456         offset = circ_dist(dopo['media_B'] - m)
457         print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
458
459 print("\n" + "=" * 70)
460 print("RISULTATO: interferenza tra i messaggi?")
461 print("=" * 70)
462 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
463 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
464                  for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
465 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
466            if dopo_per_messaggio[m]['media_B'] is not None]
467 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
468 if min(tutti_i_tassi) >= 0.9:
469     print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia "
470           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi. "
471           " ")
472     f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} -- "
473     " ")
474     f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
475     "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
476     "distruggessero a vicenda; non e' successo. <<<")
477 elif min(tutti_i_tassi) < 0.5:
478     print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
479           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
480           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
481           "una frazione degli episodi totali), non necessariamente vera interferenza "
482           "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
483 else:
484     print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "

```

```

483         "dettaglio sopra per capire quali. <<<")
484
485     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
486
487     colori = ['C3', 'C0', 'C2', 'C4']
488     for i, m in enumerate(MESSAGGI):
489         axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
490                        color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
491     axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
492                        riscaldamento')
493     axes[0, 0].set_xlabel('Episodio di training')
494     axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
495     axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
496     axes[0, 0].legend(fontsize=8)
497     axes[0, 0].grid(alpha=0.3)
498     axes[0, 0].set_ylim(-0.05, 1.05)
499
500     axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
501     axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
502     axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
503                        riscaldamento')
504     axes[0, 1].set_xlabel('Episodio di training')
505     axes[0, 1].set_ylabel('Peso massimo')
506     axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
507     axes[0, 1].legend(fontsize=8, loc='upper left')
508     axes[0, 1].grid(alpha=0.3)
509
510     im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
511                             cmap='viridis', vmin=0, vmax=W_AB.max())
512     axes[1, 0].set_xlabel('Posizione post (B)')
513     axes[1, 0].set_ylabel('Posizione pre (A)')
514     axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
515     plt.colorbar(im0, ax=axes[1, 0])
516
517     im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
518                             cmap='viridis', vmin=0, vmax=W_AB.max())
519     axes[1, 1].set_xlabel('Posizione post (B)')
520     axes[1, 1].set_ylabel('Posizione pre (A)')
521     axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
522     plt.colorbar(im1, ax=axes[1, 1])
523
524     plt.tight_layout()
525     plt.savefig('dialogo_multi_scoperta_locale.png', dpi=150)
526     print("\nGrafico salvato in dialogo_multi_scoperta_locale.png")

```

5.5.10 Monte Carlo verification: the result is robust, but the hypothesis about the residual was wrong

Section 5.5.9 had obtained 10 exchanges out of 12 with a single random seed, and had attributed the residual (the third message, last in the round-robin, which did not reach the higher-order associations) to bad luck in discovery order. A single run is not enough to confirm this -- the same statistical principle already applied in the project (PSTH over 25 independent trials, Section 2.11) requires repeating the experiment with different SEEDS before drawing conclusions about why a result comes out a certain way. Eight independent replicates (exact same configuration, no change to the algorithm, only the rng_train seed) were run in parallel.

Aggregate result: 88 successes out of 96 total checks (8 seeds x 3 messages x 4 exchanges) = 91.7% -- higher than the single original run (83%), confirming

that the discover-then-consolidate-locally mechanism is robust, not an isolated stroke of luck. Turn 1 (B responds) and turn 3 (A responds) turned out to be PERFECTLY reliable: 100% for all three messages, in all 8 seeds without exception.

Message	B (turn 1)	A (turn 3)	B (turn 5)	A (turn 7)
x=0.10 (first)	100%	100%	100%	100%
x=0.40 (second)	100%	100%	37.5%	100%
x=0.70 (last)	100%	100%	75%	87.5%

The "bad luck in discovery order, always the last one to suffer from it" hypothesis does not hold up against the aggregate data. The first message in the round-robin (x=0.10) turned out to be perfect on all four exchanges in ALL 8 seeds, without a single exception -- the most reliable of all. But the most fragile is not the last one (x=0.70, 75-87.5% on the higher-order associations) -- it is the SECOND one (x=0.40), which fails on turn 5 in 5 of 8 seeds (37.5% reliability). A non-monotonic pattern with respect to position in the round-robin, different from what was hypothesized: the first message has a real structural advantage (probably always getting first access to a still-completely-free matrix, in every training cycle), but between second and third place the relationship is not simply "worse the later it comes." The exact cause of this asymmetry remains an open question, corrected relative to the overly simple explanation proposed in 5.5.9 -- another case, in the same spirit stated at the opening of this document, in which rigorous verification disproved a plausible but uncontrolled explanation.

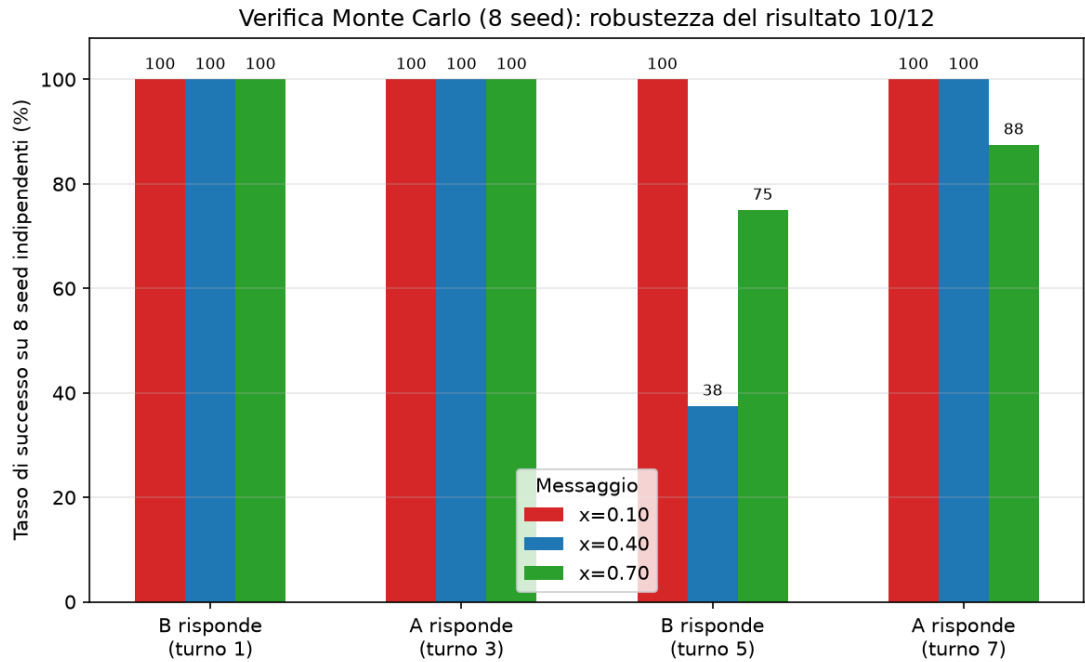


Fig. 30 — Success rate per message and exchange, aggregated over 8 independent seeds: $B(t1)$ and $A(t3)$ perfect at 100%, $B(t5)$ and $A(t7)$ variable and non-monotonic with respect to position in the round-robin.

Source code: `dialogo_multi_scoperta_locale_montecarlo.py`

```

1  """
2  Verifica Monte Carlo di dialogo_multi_scoperta_locale.py: quella run aveva
3  dato 10/12 scambi riusciti, il migliore trovato, con un residuo (il terzo
4  messaggio, sempre l'ultimo del round-robin, non raggiunge le associazioni
5  di ordine piu' alto) attribuito a sfortuna nell'ordine di scoperta piuttosto
6  che a un limite strutturale nuovo -- ma un singolo seed casuale non basta a
7  dirlo con certezza (lo stesso principio statistico gia' applicato nel
8  progetto: PSTH su 25 prove indipendenti in sezione 2.11, rottura di
9  simmetria su 12 prove in 5.2.1).
10
11  Qui lo stesso esperimento (stessa configurazione, stesso codice, nessuna
12  modifica all'algoritmo) viene ripetuto con SEED diversi per rng_train --
13  l'unica vera fonte di casualita' (le posizioni esplorative scoperte). Se
14  l'ipotesi "sfortuna nell'ordine" e' corretta, il ruolo di "messaggio in
15  ritardo" dovrebbe ruotare tra i tre messaggi seed per seed, non cadere
16  sempre sullo stesso. Il seed si passa da riga di comando
17  (python3 dialogo_multi_scoperta_locale_montecarlo.py <seed>), cosi' le
18  repliche possono girare in parallelo invece che in sequenza.
19
20  --- docstring originale di dialogo_multi_scoperta_locale.py, per riferimento ---
21
22  Estensione di dialogo_multi_repulsione_3msg.py e dialogo_multi_repulsione_annealing.py:
23  entrambi avevano mostrato che una repulsione abbastanza forte da separare le
24  tre righe primarie (visibile e pulita nella mappa dei pesi) lascia comunque
25  il consolidamento debole (picco W_AB -0.006-0.007, contro -0.04 della
26  repulsione debole che pero' collide). Annealing (repulsione alta poi
27  decadente) non ha risolto la tensione.

```

```

28 |
29 | Ipotesi diversa qui: il problema non e' la FORZA della repulsione ma il
30 | fatto che resti attiva per TUTTO il training. Anche dopo che un messaggio
31 | ha trovato una zona, ogni nuovo tentativo esplorativo continua a essere
32 | ripescato dalla distribuzione repulsiva GLOBALE, che si sposta leggermente
33 | ogni volta che ALTRI messaggi si allenano (la loro occupazione cambia) --
34 | impedendo al rinforzo di concentrarsi in un punto stabile anche a
35 | separazione gia' ottenuta.
36 |
37 | Correzione: due fasi per messaggio, non una sola strategia fissa per tutto
38 | il training. Finche' un messaggio non ha ancora trovato una prima risposta
39 | valida, esplorazione repulsiva (come prima, serve a separare le scoperte).
40 | Una volta trovata, esplorazione LOCALE (piccolo jitter attorno alla
41 | posizione gia' scoperta) invece di continuare a ripescare dalla
42 | distribuzione globale -- il consolidamento non dipende piu' da cosa
43 | succede altrove nella matrice.
44 |
45 | --- docstring originale di dialogo_multi_repulsione_3msg.py, per riferimento ---
46 |
47 | Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
48 | (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
49 | ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
50 | 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
51 | correzione regge anche con 3 messaggi, con un budget di training scalato
52 | in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
53 | messaggi, per dare a ciascun messaggio lo stesso numero di episodi
54 | effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
55 | scritto in modo generico rispetto al numero di messaggi (round-robin via
56 | ep % len(MESSAGGI)).
57 |
58 | Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
59 | stessa logica di contextual_policy.py con i suoi 3 contesti).
60 |
61 | --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
62 |
63 | Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
64 | (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
65 | entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
66 | W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
67 | interferiscano a vicenda -- il vero test di generalizzazione, nello
68 | stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
69 | singolo contesto a piu' contesti).
70 |
71 | Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
72 | separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
73 | contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
74 | restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
75 | sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
76 | (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
77 | matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
78 | verifica invece di assumerlo, con una valutazione pulita per messaggio a
79 | fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
80 | se ne allena uno (per scoprire un eventuale degrado nel tempo).
81 |
82 | Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
83 | la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
84 | adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
85 | fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
86 | primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
87 | un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
88 | tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
89 | arrivano al 100% di affidabilita' in valutazione pulita.
90 |
91 | AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
92 | di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
93 | canale B->A restava pulito al 100% per entrambi su due run indipendenti
94 | (anche con esposizione garantita identica, round-robin), ma il canale
95 | A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva

```

```

96 e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
97 quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
98
99 Qui si prova una correzione mirata proprio a quel meccanismo: una
100 NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
101 neurone di B (il totale di ogni colonna di W_AB non puo' superare un
102 tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
103 veri regolano omeostaticamente il totale delle proprie sinapsi in
104 entrata): se i due messaggi competono per la STESSA zona di risposta in
105 B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
106 spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
107 test di regressione con un solo messaggio conferma che la normalizzazione
108 non interferisce con l'apprendimento quando non c'e' competizione (il
109 totale di colonna resta ben sotto il tetto per tutto il training).
110 """
111
112 import sys
113 import numpy as np
114 import matplotlib
115 matplotlib.use('Agg')
116 import matplotlib.pyplot as plt
117
118
119 def circ_dist(d):
120     return 0.5 - np.abs(np.abs(d) - 0.5)
121
122
123 N = 200
124 x = np.arange(N) / N
125 tau = 10.0
126 dt = 0.2
127
128 A_exc = 20.0
129 sigma_exc = 0.05
130 global_inhib = 5.0
131 r_max = 5.0
132 stim_strength = 3.0
133 stim_width = 0.03
134 stim_duration_ms = 80
135
136 d_matrix = circ_dist(x[:, None] - x[None, :])
137 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
138
139 COUPLING_STRENGTH = 1.3
140 T_TURN = 200
141 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
142 # l'altro (verificato necessario: senza il silenzio, il tentativo di
143 # ascolto immediatamente successivo a un turno di parola arriva mentre
144 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
145 # docstring)
146 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
147 N_TURNS = len(SCHEDULE)
148 steps_per_turn = int(T_TURN/dt)
149
150 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
151 # ma scalato troppo debole per bastare da solo (verificato sotto)
152 OFFSET_AB_INNATO = 0.3
153 OFFSET_BA_INNATO = 0.2
154 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
155 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
156 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
157 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
158 # solo dal rumore esplorativo, non dal bias innato da solo
159 W_INIT_SCALE = 0.0018
160 SIGMA_W = 0.05
161 W_MAX = 2.5
162 ETA = 0.0001
163 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py

```

```

164 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
165 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
166 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
167 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
168 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
169 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
170 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
171 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
172 # di B non possono piu' saturarla entrambi senza competere per lo stesso
173 # budget
174 W_AB_COL_CAP = 1.5
175 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
176 # occupata (somma di colonna) quando si sceglie una posizione casuale per
177 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione=1.0 (gia'
178 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
179 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
180 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
181 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
182 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
183 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
184 # repulsione e' rinforzata (x4) per scoraggiare piu' decisamente le
185 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
186 REPULSIONE_ESPLORAZIONE = 10.0
187 # larghezza del jitter usato per l'esplorazione LOCALE una volta che un
188 # messaggio ha gia' scoperto una prima risposta valida (vedi
189 # campiona_esplorazione per la logica completa a due fasi)
190 JITTER_LOCALE_STD = 0.02
191 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
192 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
193 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
194 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
195 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
196 # piu' alto compensa il minor numero di eventi di rinforzo utili
197 ETA_BA = 0.0003
198 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
199 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
200 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
201 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
202 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
203 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
204 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
205 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
206 # riscaldamento (~400) della versione a un solo messaggio -- con 3
207 # messaggi serve il triplo rispetto alla versione a 2
208 WARMUP_EPISODES = 1200
209 DECAY_PER_EPISODE = 0.999
210 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
211 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
212 # accensione solitaria verificata a strength=0.0036 con questa durata --
213 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
214 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
215 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
216 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
217 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
218 EXPLORATION_PULSE_STRENGTH = 0.0080
219 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
220
221 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
222 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
223 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
224 GAIN_ADAPT = 0.25
225 TAU_ADAPT = 200.0
226
227
228 def F(u):
229     return np.clip(u, 0, r_max)
230
231

```



```

232 def bump_pos(r):
233     return x[np.argmax(r)] if r.max() > 0.5 else None
234
235 def init_weak_bias(offset, scale):
236     target = (x[:, None] + offset) % 1.0
237     d = circ_dist(x[None, :] - target)
238     return scale * np.exp(-d**2/(2*SIGMA_W**2))
239
240
241 def hebbian_bump(pos_pre, pos_post):
242     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
243     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
244     return np.outer(kern_pre, kern_post)
245
246
247 def normalizza_colonne(W, cap):
248     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
249     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
250     competono per la stessa zona di B, questo impedisce a uno dei due di
251     monopolizzare quella colonna a spese dell'altro -- senza impedire a
252     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
253     somme = W.sum(axis=0)
254     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
255     W *= fattore[None, :]
256
257
258 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
259     """Invece di una posizione uniforme casuale, pesca preferendo le zone
260     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
261     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
262     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
263     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
264     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
265     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
266     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
267     il ring attractor, sezione 9), non perche' competano davvero ma perche'
268     la ricerca non ha memoria di cosa e' gia' stato trovato da altri."""
269     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
270     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
271     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
272     occupazione_altri = np.maximum(occupazione_altri, 0.0)
273     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
274     peso /= peso.sum()
275     return rng.choice(x, p=peso)
276
277
278 def campiona_esplorazione(rng, W, pos_propria, scoperta):
279     """Due fasi, non una sola strategia per tutto il training. Finche' un
280     messaggio non ha ancora trovato UNA risposta valida, usa la ricerca
281     repulsiva (evita le zone gia' rivendicate da altri messaggi -- serve a
282     separare le scoperte). Una volta trovata una prima risposta, passa a
283     un'esplorazione LOCALE (piccolo jitter attorno alla posizione gia'
284     scoperta) invece di continuare a ripescare dalla distribuzione globale
285     repulsiva -- che altrimenti resta influenzata da come cambia
286     l'occupazione ALTRUI durante tutto il resto del training, impedendo al
287     rinforzo di concentrarsi in un punto stabile anche dopo che le zone
288     sono gia' separate (l'ipotesi diagnosticata dopo i tentativi con
289     repulsione fissa e con annealing, entrambi rimasti deboli in magnitudo
290     anche a separazione ottenuta)."""
291     if scoperta is not None:
292         return (scoperta + rng.normal(0, JITTER_LOCALE_STD)) % 1.0
293     return campiona_esplorazione_repulsiva(rng, W, pos_propria)
294
295
296 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True, scoperta_B=None):
297     rA = np.zeros(N)
298     rB = np.zeros(N)

```

```

300 aA = np.zeros(N)
301 aB = np.zeros(N)
302 stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
303 stim_duration_steps = int(stim_duration_ms/dt)
304
305 pos_A_turni = []
306 pos_B_turni = []
307 # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
308 # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
309 # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
310 # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
311 # solo la posizione istantanea perderebbe quasi ogni coppia valida
312 ultima_pos_A, ultima_pos_B = None, None
313
314 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
315
316 for turn, listener in enumerate(SCHEDULE):
317
318     if train and listener == 'B':
319         explore_pos = campiona_esplorazione(rng, W_AB, messaggio, scoperta_B)
320         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
321             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
322     elif train and listener == 'A':
323         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
324         explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
325         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
326             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
327     else:
328         explore_profile = np.zeros(N)
329
330     for step_in_turn in range(steps_per_turn):
331         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
332             0.0
333
334         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
335             else np.zeros(N)
336         explore_A = explore_now if listener == 'A' else np.zeros(N)
337         explore_B = explore_now if listener == 'B' else np.zeros(N)
338
339         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
340             N)
341         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
342             N)
343
344         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
345         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
346         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
347         rA = rA_new
348
349         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
350         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
351         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
352         rB = rB_new
353
354         pos_A = bump_pos(rA)
355         pos_B = bump_pos(rB)
356         pos_A_turni.append(pos_A)
357         pos_B_turni.append(pos_B)
358
359     if train and listener == 'B':
360         parlante_A = pos_A if pos_A is not None else ultima_pos_A
361         if parlante_A is not None and pos_B is not None:
362             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
363             np.clip(W_AB, 0, W_MAX, out=W_AB)
364             normalizza_colonne(W_AB, W_AB_COL_CAP)
365     elif train and listener == 'A' and allena_BA:
366         parlante_B = pos_B if pos_B is not None else ultima_pos_B
367         if parlante_B is not None and pos_A is not None:

```

```

364         W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
365         np.clip(W_BA, 0, W_MAX, out=W_BA)
366
367         if pos_A is not None:
368             ultima_pos_A = pos_A
369         if pos_B is not None:
370             ultima_pos_B = pos_B
371
372     return pos_A_turni, pos_B_turni
373
374
375 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
376     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
377     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
378     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
379     Il confronto per A va fatto contro il turno di silenzio subito prima
380     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
381     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
382     da li' al turno di ascolto successivo indica una vera risposta a B."""
383     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
384     pos_B_ok, pos_A_ok = [], []
385     for k in range(n_trial):
386         rng = np.random.RandomState(seed0 + k)
387         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
388         if pos_B[1] is not None:
389             successi_B1 += 1
390             pos_B_ok.append(pos_B[1])
391         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
392             0.02):
393             successi_A3 += 1
394             pos_A_ok.append(pos_A[3])
395         if pos_B[5] is not None:
396             successi_B2 += 1
397         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
398             0.02):
399             successi_A7 += 1
400     return {
401         'tasso_B1': successi_B1/n_trial,
402         'tasso_A3': successi_A3/n_trial,
403         'tasso_B2': successi_B2/n_trial,
404         'tasso_A7': successi_A7/n_trial,
405         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
406         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
407         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
408     }
409
410 if __name__ == '__main__':
411     SEED = int(sys.argv[1]) if len(sys.argv) > 1 else 42
412     MESSAGGI = [0.10, 0.40, 0.70]
413     N_EPISODES = 13500
414     EVAL_EVERY = 500
415     N_TRIAL_EVAL = 15
416
417     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
418     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
419
420     print("=" * 70)
421     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa, SEED={SEED}")
422
423     print("=" * 70)
424     W_AB_iniziale = W_AB.copy()
425     W_BA_iniziale = W_BA.copy()
426
427     print("\n" + "=" * 70)
428     print("BASELINE (pre-training), per ciascun messaggio")
429     print("=" * 70)
430     for m in MESSAGGI:

```

```

429         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
430         print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
431               f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
432
433     print("\n" + "=" * 70)
434     print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
435           f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
436           )
437     print("=" * 70)
438     rng_train = np.random.RandomState(SEED)
439     picco_W_AB_storia = []
440     picco_W_BA_storia = []
441     episodi_check = []
442     curva_min_per_messaggio = {m: [] for m in MESSAGGI}
443     scoperta_B = {m: None for m in MESSAGGI}
444
445     for ep in range(N_EPISODES):
446         allena_BA = ep >= WARMUP_EPISODES
447         messaggio = MESSAGGI[ep % len(MESSAGGI)]
448         pos_A, pos_B = run_episode(W_AB, W_BA, messaggio, rng_train, train=True,
449                                   allena_BA=allena_BA, scoperta_B=scoperta_B[messaggio])
450         if pos_B[1] is not None:
451             scoperta_B[messaggio] = pos_B[1]
452         W_AB *= DECAY_PER_EPISODE
453         if allena_BA:
454             W_BA *= DECAY_PER_EPISODE
455         picco_W_AB_storia.append(W_AB.max())
456         picco_W_BA_storia.append(W_BA.max())
457
458         if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
459             episodi_check.append(ep)
460             fase = "riscaldamento" if not allena_BA else "entrambi i canali"
461             riga = f" episodio {ep:4d} ({fase}):"
462             for m in MESSAGGI:
463                 r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
464                 minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
465                 curva_min_per_messaggio[m].append(minimo)
466                 riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
467             print(riga)
468
469     print("\n" + "=" * 70)
470     print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
471     print("=" * 70)
472     dopo_per_messaggio = {}
473     for m in MESSAGGI:
474         dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
475         dopo_per_messaggio[m] = dopo
476         print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
477               f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
478         if dopo['media_B'] is not None:
479             offset = circ_dist(dopo['media_B'] - m)
480             print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
481             # riga in formato fisso, facile da aggregare tra i seed con grep/awk
482             print(f"MC_RIGA seed={SEED} x={m:.2f} B1={dopo['tasso_B1']*100:.0f} "
483                   f"A3={dopo['tasso_A3']*100:.0f} B5={dopo['tasso_B2']*100:.0f} A7={dopo['tasso_A7']*100:.0f}")
484
485     print("\n" + "=" * 70)
486     print("RISULTATO: interferenza tra i messaggi?")
487     print("=" * 70)
488     print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
489     tutti_i_tassi = [v for d in dopo_per_messaggio.values()
490                      for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
491     offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
492                if dopo_per_messaggio[m]['media_B'] is not None]
493     offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
494     if min(tutti_i_tassi) >= 0.9:

```

```

494     print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia
495           "
496           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi.
497           "
498           "Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} --
499           "
500           "differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
501           "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
502           "distruggessero a vicenda; non e' successo. <<<")
503 elif min(tutti_i_tassi) < 0.5:
504     print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
505           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
506           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
507           "una frazione degli episodi totali), non necessariamente vera interferenza "
508           "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
509 else:
510     print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
511           "dettaglio sopra per capire quali. <<<")
512
513 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
514
515 colori = ['C3', 'C0', 'C2', 'C4']
516 for i, m in enumerate(MESSAGGI):
517     axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
518                    color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
519 axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
520 riscaldamento')
521 axes[0, 0].set_xlabel('Episodio di training')
522 axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
523 axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
524 axes[0, 0].legend(fontsize=8)
525 axes[0, 0].grid(alpha=0.3)
526 axes[0, 0].set_ylim(-0.05, 1.05)
527
528 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
529 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
530 axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
531 riscaldamento')
532 axes[0, 1].set_xlabel('Episodio di training')
533 axes[0, 1].set_ylabel('Peso massimo')
534 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
535 axes[0, 1].legend(fontsize=8, loc='upper left')
536 axes[0, 1].grid(alpha=0.3)
537
538 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
539                        cmap='viridis', vmin=0, vmax=W_AB.max())
540 axes[1, 0].set_xlabel('Posizione post (B)')
541 axes[1, 0].set_ylabel('Posizione pre (A)')
542 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
543 plt.colorbar(im0, ax=axes[1, 0])
544
545 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
546                        cmap='viridis', vmin=0, vmax=W_AB.max())
547 axes[1, 1].set_xlabel('Posizione post (B)')
548 axes[1, 1].set_ylabel('Posizione pre (A)')
549 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
550 plt.colorbar(im1, ax=axes[1, 1])
551
552 plt.tight_layout()
553 plt.savefig(f'dialogo_multi_scoperta_locale_seed{SEED}.png', dpi=150)
554 print(f"\nGrafico salvato in dialogo_multi_scoperta_locale_seed{SEED}.png")

```

5.5.11 Disambiguating position vs. value: two distinct effects, not one

Section 5.5.10 had left an open question: does the fragility observed on the higher-order associations follow the POSITION in the round-robin or the VALUE of the message? The direct test: repeat the same 8 replicates with the round-robin order ROTATED -- [0.40, 0.70, 0.10] instead of [0.10, 0.40, 0.70], same code, same seeds (101-108) -- and check which of the two (position or value) stays constant.

	Original order [0.10, 0.40, 0.70]	Permuted order [0.40, 0.70, 0.10]
Position 1	x=0.10: B5=100% A7=100%	x=0.40: B5=50% A7=100%
Position 2	x=0.40: B5=37.5% A7=100%	x=0.70: B5=37.5% A7=87.5%
Position 3	x=0.70: B5=75% A7=87.5%	x=0.10: B5=50% A7=100%

The result cleanly disambiguates TWO distinct effects, which in the first observation (5.5.9-5.5.10) appeared confounded together:

Turn 5 (B responds again) follows POSITION: whoever occupies round-robin position 2 gets EXACTLY 37.5% reliability in both orderings -- before it was message x=0.40 that sat there, now it is x=0.70, but the success rate is identical to the first decimal place. It is a pure effect of the temporal order of training (probably related to how the discovery phases of the three messages overlap in time), not of the specific value.

Turn 7 (A responds again), by contrast, follows VALUE: message x=0.70 stays at 87.5% whether it occupies position 3 (original order) or position 2 (permuted order) -- while x=0.10 and x=0.40 always stay at 100% regardless of where they are placed in the round-robin. It is an effect tied to x=0.70's position ON THE RING specifically (perhaps an interaction with the discretized geometry, in the same spirit as the position bias already documented for the ring attractor in other sections), not to the training order.

A clean disambiguation, obtained through a direct control rather than through further untested hypotheses: two independent phenomena were superimposed in the original observation, each tied to a different cause (temporal order for turn 5, value/position on the ring for turn 7). Neither has been resolved here -- both remain documented as known properties of the system, no longer as a single confused open question.

Fragilita' sul turno 5/7: confronto tra due ordini di round-robin (8 seed ciascuno)

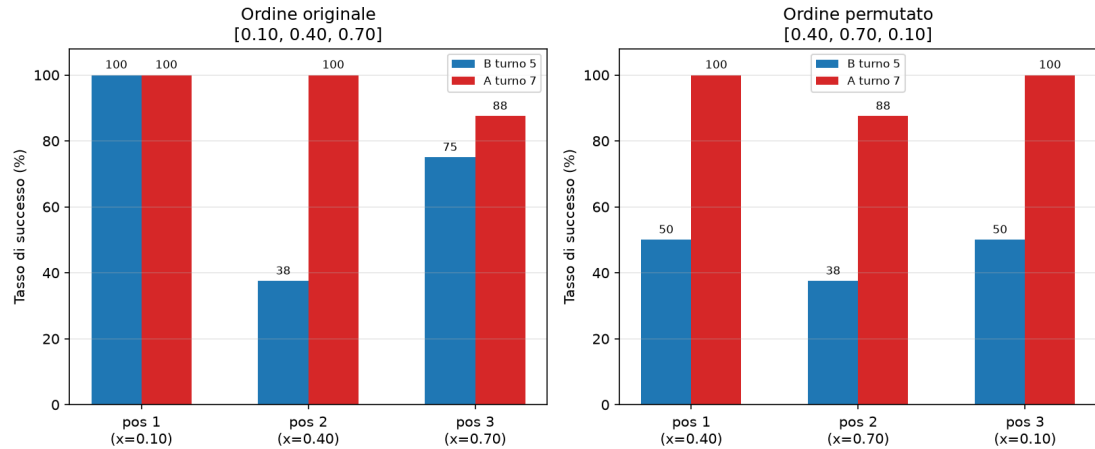


Fig. 31 — Direct comparison between the two round-robin orderings: turn 5 follows position (a constant 37.5% at position 2), turn 7 follows value (a constant 87.5% for $x=0.70$).

Source code: `dialogo_multi_scoperta_locale_permutazione.py`

```

1  """
2  Estensione di dialogo_multi_scoperta_locale_montecarlo.py: la verifica su 8
3  seed indipendenti aveva dato 91.7% di successo aggregato (robusto), ma con
4  un pattern inatteso sul turno 5 -- il messaggio in POSIZIONE 1 del
5  round-robin ( $x=0.10$ ) e' risultato perfetto in tutti gli 8 seed, mentre il
6  piu' fragile non era l'ultimo ( $x=0.70$ , 75%) come atteso ma il SECONDO
7  ( $x=0.40$ , solo 37.5%). Domanda aperta: questa fragilita' segue il VALORE
8  del messaggio (0.40 resta fragile ovunque venga messo) o la POSIZIONE nel
9  round-robin (chi va per secondo e' fragile, chiunque esso sia)?
10
11  Qui l'ordine del round-robin viene RUOTATO -- [0.40, 0.70, 0.10] invece di
12  [0.10, 0.40, 0.70], stesso codice, stessi seed (101-108) -- per
13  disambiguare le due ipotesi. Se la fragilita' sul turno 5 resta legata al
14  valore 0.40 (ora in posizione 1), e' una proprieta' del valore. Se si
15  sposta a chiunque occupi la posizione 2 (ora 0.70), e' un effetto puro
16  dell'ordine temporale di allenamento.
17
18  --- docstring originale di dialogo_multi_scoperta_locale.py, per riferimento ---
19
20  Estensione di dialogo_multi_repulsione_3msg.py e dialogo_multi_repulsione_annealing.py:
21  entrambi avevano mostrato che una repulsione abbastanza forte da separare le
22  tre righe primarie (visibile e pulita nella mappa dei pesi) lascia comunque
23  il consolidamento debole (picco  $W_{AB}$  -0.006-0.007, contro -0.04 della
24  repulsione debole che pero' collide). Annealing (repulsione alta poi
25  decadente) non ha risolto la tensione.
26
27  Ipotesi diversa qui: il problema non e' la FORZA della repulsione ma il
28  fatto che resti attiva per TUTTO il training. Anche dopo che un messaggio
29  ha trovato una zona, ogni nuovo tentativo esplorativo continua a essere
30  ripescato dalla distribuzione repulsiva GLOBALE, che si sposta leggermente
31  ogni volta che ALTRI messaggi si allenano (la loro occupazione cambia) --
32  impedendo al rinforzo di concentrarsi in un punto stabile anche a
33  separazione gia' ottenuta.
34
35  Correzione: due fasi per messaggio, non una sola strategia fissa per tutto
36  il training. Finche' un messaggio non ha ancora trovato una prima risposta

```

```

37 valida, esplorazione repulsiva (come prima, serve a separare le scoperte).
38 Una volta trovata, esplorazione LOCALE (piccolo jitter attorno alla
39 posizione gia' scoperta) invece di continuare a ripescare dalla
40 distribuzione globale -- il consolidamento non dipende piu' da cosa
41 succede altrove nella matrice.
42
43 --- docstring originale di dialogo_multi_repulsione_3msg.py, per riferimento ---
44
45 Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
46 (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
47 ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
48 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
49 correzione regge anche con 3 messaggi, con un budget di training scalato
50 in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
51 messaggi, per dare a ciascun messaggio lo stesso numero di episodi
52 effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
53 scritto in modo generico rispetto al numero di messaggi (round-robin via
54 ep % len(MESSAGGI)).
55
56 Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
57 stessa logica di contextual_policy.py con i suoi 3 contesti).
58
59 --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
60
61 Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
62 (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
63 entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
64 W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
65 interferiscano a vicenda -- il vero test di generalizzazione, nello
66 stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
67 singolo contesto a piu' contesti).
68
69 Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
70 separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
71 contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
72 restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
73 sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
74 (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
75 matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
76 verifica invece di assumerlo, con una valutazione pulita per messaggio a
77 fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
78 se ne allena uno (per scoprire un eventuale degrado nel tempo).
79
80 Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
81 la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
82 adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
83 fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
84 primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
85 un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
86 tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
87 arrivano al 100% di affidabilita' in valutazione pulita.
88
89 AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
90 di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
91 canale B->A restava pulito al 100% per entrambi su due run indipendenti
92 (anche con esposizione garantita identica, round-robin), ma il canale
93 A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
94 e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
95 quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
96
97 Qui si prova una correzione mirata proprio a quel meccanismo: una
98 NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
99 neurone di B (il totale di ogni colonna di W_AB non puo' superare un
100 tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
101 veri regolano omeostaticamente il totale delle proprie sinapsi in
102 entrata): se i due messaggi competono per la STESSA zona di risposta in
103 B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
104 spese dell'altro; se invece puntano a zone diverse, non li tocca. Un

```



```

105 test di regressione con un solo messaggio conferma che la normalizzazione
106 non interferisce con l'apprendimento quando non c'e' competizione (il
107 totale di colonna resta ben sotto il tetto per tutto il training).
108 """
109
110 import sys
111 import numpy as np
112 import matplotlib
113 matplotlib.use('Agg')
114 import matplotlib.pyplot as plt
115
116
117 def circ_dist(d):
118     return 0.5 - np.abs(np.abs(d) - 0.5)
119
120
121 N = 200
122 x = np.arange(N) / N
123 tau = 10.0
124 dt = 0.2
125
126 A_exc = 20.0
127 sigma_exc = 0.05
128 global_inhib = 5.0
129 r_max = 5.0
130 stim_strength = 3.0
131 stim_width = 0.03
132 stim_duration_ms = 80
133
134 d_matrix = circ_dist(x[:, None] - x[None, :])
135 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
136
137 COUPLING_STRENGTH = 1.3
138 T_TURN = 200
139 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
140 # l'altro (verificato necessario: senza il silenzio, il tentativo di
141 # ascolto immediatamente successivo a un turno di parola arriva mentre
142 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
143 # docstring)
144 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
145 N_TURNS = len(SCHEDULE)
146 steps_per_turn = int(T_TURN/dt)
147
148 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
149 # ma scalato troppo debole per bastare da solo (verificato sotto)
150 OFFSET_AB_INNATO = 0.3
151 OFFSET_BA_INNATO = 0.2
152 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
153 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
154 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
155 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
156 # solo dal rumore esplorativo, non dal bias innato da solo
157 W_INIT_SCALE = 0.0018
158 SIGMA_W = 0.05
159 W_MAX = 2.5
160 ETA = 0.0001
161 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
162 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
163 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
164 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
165 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
166 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
167 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
168 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
169 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
170 # di B non possono piu' saturarla entrambi senza competere per lo stesso
171 # budget
172 W_AB_COL_CAP = 1.5

```

```

173 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
174 # occupata (somma di colonna) quando si sceglie una posizione casuale per
175 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione-1.0 (gia'
176 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
177 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
178 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
179 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
180 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
181 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
182 # repulsione e' rinforzata (x4) per scoraggiare piu' decisamente le
183 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
184 REPULSIONE_ESPLORAZIONE = 10.0
185 # larghezza del jitter usato per l'esplorazione LOCALE una volta che un
186 # messaggio ha gia' scoperto una prima risposta valida (vedi
187 # campiona_esplorazione per la logica completa a due fasi)
188 JITTER_LOCALE_STD = 0.02
189 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
190 # dialogo_std_adattamento.png della run precedente: ~100% vs ~65-80%), e
191 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
192 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
193 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
194 # piu' alto compensa il minor numero di eventi di rinforzo utili
195 ETA_BA = 0.0003
196 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
197 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
198 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
199 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
200 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
201 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
202 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
203 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
204 # riscaldamento (~400) della versione a un solo messaggio -- con 3
205 # messaggi serve il triplo rispetto alla versione a 2
206 WARMUP_EPISODES = 1200
207 DECAY_PER_EPISODE = 0.999
208 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
209 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
210 # accensione solitaria verificata a strength-0.0036 con questa durata --
211 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
212 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
213 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
214 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
215 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
216 EXPLORATION_PULSE_STRENGTH = 0.0080
217 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
218
219 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
220 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
221 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
222 GAIN_ADAPT = 0.25
223 TAU_ADAPT = 200.0
224
225
226 def F(u):
227     return np.clip(u, 0, r_max)
228
229
230 def bump_pos(r):
231     return x[np.argmax(r)] if r.max() > 0.5 else None
232
233
234 def init_weak_bias(offset, scale):
235     target = (x[:, None] + offset) % 1.0
236     d = circ_dist(x[None, :] - target)
237     return scale * np.exp(-d**2/(2*SIGMA_W**2))
238
239
240 def hebbian_bump(pos_pre, pos_post):

```

```

241 kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
242 kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
243 return np.outer(kern_pre, kern_post)
244
245
246 def normalizza_colonne(W, cap):
247     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
248     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
249     competono per la stessa zona di B, questo impedisce a uno dei due di
250     monopolizzare quella colonna a spese dell'altro -- senza impedire a
251     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
252     somme = W.sum(axis=0)
253     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
254     W *= fattore[None, :]
255
256
257 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
258     """Invece di una posizione uniforme casuale, pesca preferendo le zone
259     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
260     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
261     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
262     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
263     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
264     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
265     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
266     il ring attractor, sezione 9), non perche' competano davvero ma perche'
267     la ricerca non ha memoria di cosa e' gia' stato trovato da altri."""
268     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
269     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
270     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
271     occupazione_altri = np.maximum(occupazione_altri, 0.0)
272     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
273     peso /= peso.sum()
274     return rng.choice(x, p=peso)
275
276
277 def campiona_esplorazione(rng, W, pos_propria, scoperta):
278     """Due fasi, non una sola strategia per tutto il training. Finche' un
279     messaggio non ha ancora trovato UNA risposta valida, usa la ricerca
280     repulsiva (evita le zone gia' rivendicate da altri messaggi -- serve a
281     separare le scoperte). Una volta trovata una prima risposta, passa a
282     un'esplorazione LOCALE (piccolo jitter attorno alla posizione gia'
283     scoperta) invece di continuare a ripescare dalla distribuzione globale
284     repulsiva -- che altrimenti resta influenzata da come cambia
285     l'occupazione ALTRUI durante tutto il resto del training, impedendo al
286     rinforzo di concentrarsi in un punto stabile anche dopo che le zone
287     sono gia' separate (l'ipotesi diagnosticata dopo i tentativi con
288     repulsione fissa e con annealing, entrambi rimasti deboli in magnitudo
289     anche a separazione ottenuta)."""
290     if scoperta is not None:
291         return (scoperta + rng.normal(0, JITTER_LOCALE_STD)) % 1.0
292     return campiona_esplorazione_repulsiva(rng, W, pos_propria)
293
294
295 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True, scoperta_B=None):
296     rA = np.zeros(N)
297     rB = np.zeros(N)
298     aA = np.zeros(N)
299     aB = np.zeros(N)
300     stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
301     stim_duration_steps = int(stim_duration_ms/dt)
302
303     pos_A_turni = []
304     pos_B_turni = []
305     # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
306     # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
307     # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
308     # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere

```

```

309 # solo la posizione istantanea perderebbe quasi ogni coppia valida
310 ultima_pos_A, ultima_pos_B = None, None
311
312 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
313
314 for turn, listener in enumerate(SCHEDULE):
315
316     if train and listener == 'B':
317         explore_pos = campiona_esplorazione(rng, W_AB, messaggio, scoperta_B)
318         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
319             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
320     elif train and listener == 'A':
321         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
322         explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
323         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
324             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
325     else:
326         explore_profile = np.zeros(N)
327
328     for step_in_turn in range(steps_per_turn):
329         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
330             0.0
331
332         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
333             else np.zeros(N)
334         explore_A = explore_now if listener == 'A' else np.zeros(N)
335         explore_B = explore_now if listener == 'B' else np.zeros(N)
336
337         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
338             N)
339         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
340             N)
341
342         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
343         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
344         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
345         rA = rA_new
346
347         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
348         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
349         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
350         rB = rB_new
351
352         pos_A = bump_pos(rA)
353         pos_B = bump_pos(rB)
354         pos_A_turni.append(pos_A)
355         pos_B_turni.append(pos_B)
356
357     if train and listener == 'B':
358         parlante_A = pos_A if pos_A is not None else ultima_pos_A
359         if parlante_A is not None and pos_B is not None:
360             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
361             np.clip(W_AB, 0, W_MAX, out=W_AB)
362             normalizza_colonne(W_AB, W_AB_COL_CAP)
363     elif train and listener == 'A' and allena_BA:
364         parlante_B = pos_B if pos_B is not None else ultima_pos_B
365         if parlante_B is not None and pos_A is not None:
366             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)
367             np.clip(W_BA, 0, W_MAX, out=W_BA)
368
369     if pos_A is not None:
370         ultima_pos_A = pos_A
371     if pos_B is not None:
372         ultima_pos_B = pos_B
373
374 return pos_A_turni, pos_B_turni

```

```

373 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
374     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
375     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
376     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
377     Il confronto per A va fatto contro il turno di silenzio subito prima
378     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
379     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
380     da li' al turno di ascolto successivo indica una vera risposta a B."""
381     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
382     pos_B_ok, pos_A_ok = [], []
383     for k in range(n_trial):
384         rng = np.random.RandomState(seed0 + k)
385         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
386         if pos_B[1] is not None:
387             successi_B1 += 1
388             pos_B_ok.append(pos_B[1])
389         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
390             0.02):
391             successi_A3 += 1
392             pos_A_ok.append(pos_A[3])
393         if pos_B[5] is not None:
394             successi_B2 += 1
395         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
396             0.02):
397             successi_A7 += 1
398     return {
399         'tasso_B1': successi_B1/n_trial,
400         'tasso_A3': successi_A3/n_trial,
401         'tasso_B2': successi_B2/n_trial,
402         'tasso_A7': successi_A7/n_trial,
403         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
404         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
405         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
406     }
407
408 if __name__ == '__main__':
409     SEED = int(sys.argv[1]) if len(sys.argv) > 1 else 42
410     # ordine ruotato rispetto a dialogo_multi_scoperta_locale_montecarlo.py
411     # ([0.10, 0.40, 0.70]): serve a distinguere se la fragilita' osservata
412     # sul turno 5 (37.5% per il messaggio in posizione 2 su 8 seed) segue
413     # il VALORE del messaggio (0.40 resta fragile ovunque si trovi) o la
414     # POSIZIONE nel round-robin (chi va per secondo e' fragile, chiunque
415     # esso sia)
416     MESSAGGI = [0.40, 0.70, 0.10]
417     N_EPISODES = 13500
418     EVAL_EVERY = 500
419     N_TRIAL_EVAL = 15
420
421     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
422     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
423
424     print("=" * 70)
425     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa,
426           SEED={SEED}")
427     print("=" * 70)
428     W_AB_iniziale = W_AB.copy()
429     W_BA_iniziale = W_BA.copy()
430
431     print("\n" + "=" * 70)
432     print("BASELINE (pre-training), per ciascun messaggio")
433     print("=" * 70)
434     for m in MESSAGGI:
435         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)
436         print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
437               f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
438     print("\n" + "=" * 70)

```

```

438 print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
439       f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
440       )
441 print("=" * 70)
442 rng_train = np.random.RandomState(SEED)
443 picco_W_AB_storia = []
444 picco_W_BA_storia = []
445 episodi_check = []
446 curva_min_per_messaggio = {m: [] for m in MESSAGGI}
447 scoperta_B = {m: None for m in MESSAGGI}
448
449 for ep in range(N_EPISODES):
450     allena_BA = ep >= WARMUP_EPISODES
451     messaggio = MESSAGGI[ep % len(MESSAGGI)]
452     pos_A, pos_B = run_episode(W_AB, W_BA, messaggio, rng_train, train=True,
453                               allena_BA=allena_BA, scoperta_B=scoperta_B[messaggio])
454     if pos_B[1] is not None:
455         scoperta_B[messaggio] = pos_B[1]
456     W_AB *= DECAY_PER_EPISODE
457     if allena_BA:
458         W_BA *= DECAY_PER_EPISODE
459     picco_W_AB_storia.append(W_AB.max())
460     picco_W_BA_storia.append(W_BA.max())
461
462     if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
463         episodi_check.append(ep)
464         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
465         riga = f" episodio {ep:4d} ({fase}):"
466         for m in MESSAGGI:
467             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
468             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
469             curva_min_per_messaggio[m].append(minimo)
470             riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
471         print(riga)
472
473 print("\n" + "=" * 70)
474 print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
475 print("=" * 70)
476 dopo_per_messaggio = {}
477 for m in MESSAGGI:
478     dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=90000)
479     dopo_per_messaggio[m] = dopo
480     print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
481           f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
482     if dopo['media_B'] is not None:
483         offset = circ_dist(dopo['media_B'] - m)
484         print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
485         # riga in formato fisso, facile da aggregare tra i seed con grep/awk
486         # 'pos' = posizione nel round-robin (0=primo, 1=secondo, 2=terzo)
487         print(f"MC_RIGA seed={SEED} x={m:.2f} pos={MESSAGGI.index(m)} B1={dopo['tasso_B1']*100:5.0f}% "
488               f"A3={dopo['tasso_A3']*100:5.0f}% B5={dopo['tasso_B2']*100:5.0f}% A7={dopo['tasso_A7']*100:5.0f}%")
489
490 print("\n" + "=" * 70)
491 print("RISULTATO: interferenza tra i messaggi?")
492 print("=" * 70)
493 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
494 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
495                  for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
496 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
497            if dopo_per_messaggio[m]['media_B'] is not None]
498 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
499 if min(tutti_i_tassi) >= 0.9:
500     print(">>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia
501           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi.

```

```

501         f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} --
502         "
503         f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
504         "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
505         "distruggessero a vicenda; non e' successo. <<<")
506     elif min(tutti_i_tassi) < 0.5:
507         print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
508               "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
509               "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
510               "una frazione degli episodi totali), non necessariamente vera interferenza "
511               "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
512     else:
513         print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
514               "dettaglio sopra per capire quali. <<<")
515
516     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
517
518     colori = ['C3', 'C0', 'C2', 'C4']
519     for i, m in enumerate(MESSAGGI):
520         axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
521                        color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
522     axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
523                        riscaldamento')
524     axes[0, 0].set_xlabel('Episodio di training')
525     axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
526     axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
527     axes[0, 0].legend(fontsize=8)
528     axes[0, 0].grid(alpha=0.3)
529     axes[0, 0].set_ylim(-0.05, 1.05)
530
531     axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
532     axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
533     axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
534                        riscaldamento')
535     axes[0, 1].set_xlabel('Episodio di training')
536     axes[0, 1].set_ylabel('Peso massimo')
537     axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
538     axes[0, 1].legend(fontsize=8, loc='upper left')
539     axes[0, 1].grid(alpha=0.3)
540
541     im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
542                            cmap='viridis', vmin=0, vmax=W_AB.max())
543     axes[1, 0].set_xlabel('Posizione post (B)')
544     axes[1, 0].set_ylabel('Posizione pre (A)')
545     axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
546     plt.colorbar(im0, ax=axes[1, 0])
547
548     im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
549                            cmap='viridis', vmin=0, vmax=W_AB.max())
550     axes[1, 1].set_xlabel('Posizione post (B)')
551     axes[1, 1].set_ylabel('Posizione pre (A)')
552     axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
553     plt.colorbar(im1, ax=axes[1, 1])
554
555     plt.tight_layout()
556     plt.savefig(f'dialogo_multi_scoperta_locale_permutazione_seed{SEED}.png', dpi=150)
557     print(f"\nGrafico salvato in dialogo_multi_scoperta_locale_permutazione_seed{SEED}.png")

```

5.5.12 Attempted correction of the positional effect: fairer, but not better

Section 5.5.11 had isolated a pure positional effect on turn 5: whoever occupies position 2 of the FIXED round-robin cycle ([0.10, 0.40, 0.70] repeated identi-

cally every cycle) gets 37.5% reliability, regardless of which message occupies it. Attempted correction: SHUFFLE the cycle order on every round instead of repeating it fixed -- no message should stay stuck in the same position for the whole training run. Same verification protocol: 8 independent seeds (101-108).

Message	B (turn 1)	A (turn 3)	B (turn 5)	A (turn 7)
x=0.10	100%	100%	75%	87.5%
x=0.40	100%	100%	50%	75%
x=0.70	100%	100%	75%	87.5%

An honest result, but not the hoped-for one: the aggregate total DROPS to 87.5% (84/96), against the 91.7% (88/96) of the original fixed order (Section 5.5.10). The correction does achieve its narrow goal, however -- no message remains as systematically perfect or as systematically disadvantaged as before (x=0.10 was a perfect 100% on turn 5 under the fixed order, now it drops to 75%; x=0.40 rises from 37.5% to 50%) -- but shuffling introduces more variability than it removes: every message, at different moments of training, still ends up next to an unfavorable ordering at some point, and the sum of these unfavorable episodes across all three weighs more heavily than the structural advantage that was previously concentrated entirely on ONE message. A genuine trade-off between fairness (no one systematically disadvantaged) and aggregate performance (the fixed order, though unfair to one specific message, remained more reliable overall) -- honestly documented as a negative result relative to the goal of improving the total, in the same spirit stated at the opening of this document.

Source code: dialogo_multi_scoperta_locale_mischiato.py

```

1  """
2  Tentativo di correzione dell'effetto POSIZIONALE identificato in 5.5.10-5.5.11:
3  la posizione 2 del round-robin fisso ([0.10,0.40,0.70] ripetuto identico ogni
4  ciclo) e' risultata svantaggiata sul turno 5 in modo indipendente dal valore
5  del messaggio che la occupa (37.5% di affidabilita' in due ordini diversi,
6  sezione 5.5.11) -- un effetto puro dell'ordine temporale di allenamento.
7
8  Qui l'ordine del ciclo viene RIMESCOLATO ad ogni giro invece di restare
9  fisso: nessun messaggio resta bloccato nella stessa posizione per tutto il
10 training. Se l'effetto e' davvero posizionale e non altro, mescolare
11 dovrebbe spalmare lo svantaggio in media su tutti e tre i messaggi invece
12 di penalizzarne sistematicamente uno solo -- il tasso di successo
13 aggregato sul turno 5 dovrebbe salire, o quantomeno diventare piu' uniforme
14 tra i tre messaggi. Stesso codice per il resto, stesso protocollo a 8 seed
15 paralleli (seed 101-108, come nelle sezioni precedenti) per la stessa
16 verifica di robustezza statistica.
17
18 --- docstring originale di dialogo_multi_scoperta_locale.py, per riferimento ---
19
20 Estensione di dialogo_multi_repulsione_3msg.py e dialogo_multi_repulsione_annealing.py:
21 entrambi avevano mostrato che una repulsione abbastanza forte da separare le
22 tre righe primarie (visibile e pulita nella mappa dei pesi) lascia comunque
23 il consolidamento debole (picco W_AB -0.006-0.007, contro -0.04 della
24 repulsione debole che pero' collide). Annealing (repulsione alta poi
25 decadente) non ha risolto la tensione.
26

```



```

27 | Ipotesi diversa qui: il problema non e' la FORZA della repulsione ma il
28 | fatto che resti attiva per TUTTO il training. Anche dopo che un messaggio
29 | ha trovato una zona, ogni nuovo tentativo esplorativo continua a essere
30 | ripescato dalla distribuzione repulsiva GLOBALE, che si sposta leggermente
31 | ogni volta che ALTRI messaggi si allenano (la loro occupazione cambia) --
32 | impedendo al rinforzo di concentrarsi in un punto stabile anche a
33 | separazione gia' ottenuta.
34 |
35 | Correzione: due fasi per messaggio, non una sola strategia fissa per tutto
36 | il training. Finche' un messaggio non ha ancora trovato una prima risposta
37 | valida, esplorazione repulsiva (come prima, serve a separare le scoperte).
38 | Una volta trovata, esplorazione LOCALE (piccolo jitter attorno alla
39 | posizione gia' scoperta) invece di continuare a ripescare dalla
40 | distribuzione globale -- il consolidamento non dipende piu' da cosa
41 | succede altrove nella matrice.
42 |
43 | --- docstring originale di dialogo_multi_repulsione_3msg.py, per riferimento ---
44 |
45 | Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
46 | (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
47 | ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
48 | 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
49 | correzione regge anche con 3 messaggi, con un budget di training scalato
50 | in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
51 | messaggi, per dare a ciascun messaggio lo stesso numero di episodi
52 | effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
53 | scritto in modo generico rispetto al numero di messaggi (round-robin via
54 | ep % len(MESSAGGI)).
55 |
56 | Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
57 | stessa logica di contextual_policy.py con i suoi 3 contesti).
58 |
59 | --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
60 |
61 | Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
62 | (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
63 | entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
64 | W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
65 | interferiscano a vicenda -- il vero test di generalizzazione, nello
66 | stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
67 | singolo contesto a piu' contesti).
68 |
69 | Messaggi usati: 0.15 e 0.65 (distanza circolare ~0.5, il piu' possibile
70 | separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
71 | contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
72 | restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
73 | sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
74 | (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
75 | matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
76 | verifica invece di assumerlo, con una valutazione pulita per messaggio a
77 | fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
78 | se ne allena uno (per scoprire un eventuale degrado nel tempo).
79 |
80 | Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
81 | la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
82 | adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
83 | fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
84 | primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
85 | un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
86 | tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
87 | arrivano al 100% di affidabilita' in valutazione pulita.
88 |
89 | AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
90 | di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
91 | canale B->A restava pulito al 100% per entrambi su due run indipendenti
92 | (anche con esposizione garantita identica, round-robin), ma il canale
93 | A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
94 | e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava

```

```

95 | quale dei due vinceva, escludendo che fosse solo esposizione ineguale.
96 |
97 | Qui si prova una correzione mirata proprio a quel meccanismo: una
98 | NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
99 | neurone di B (il totale di ogni colonna di W_AB non puo' superare un
100 | tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
101 | veri regolano omeostaticamente il totale delle proprie sinapsi in
102 | entrata): se i due messaggi competono per la STESSA zona di risposta in
103 | B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
104 | spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
105 | test di regressione con un solo messaggio conferma che la normalizzazione
106 | non interferisce con l'apprendimento quando non c'e' competizione (il
107 | totale di colonna resta ben sotto il tetto per tutto il training).
108 | """
109 |
110 | import sys
111 | import numpy as np
112 | import matplotlib
113 | matplotlib.use('Agg')
114 | import matplotlib.pyplot as plt
115 |
116 |
117 | def circ_dist(d):
118 |     return 0.5 - np.abs(np.abs(d) - 0.5)
119 |
120 |
121 | N = 200
122 | x = np.arange(N) / N
123 | tau = 10.0
124 | dt = 0.2
125 |
126 | A_exc = 20.0
127 | sigma_exc = 0.05
128 | global_inhib = 5.0
129 | r_max = 5.0
130 | stim_strength = 3.0
131 | stim_width = 0.03
132 | stim_duration_ms = 80
133 |
134 | d_matrix = circ_dist(x[:, None] - x[None, :])
135 | J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
136 |
137 | COUPLING_STRENGTH = 1.3
138 | T_TURN = 200
139 | # schema dei turni con un turno di silenzio/recupero tra un ascolto e
140 | # l'altro (verificato necessario: senza il silenzio, il tentativo di
141 | # ascolto immediatamente successivo a un turno di parola arriva mentre
142 | # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
143 | # docstring)
144 | SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
145 | N_TURNS = len(SCHEDULE)
146 | steps_per_turn = int(T_TURN/dt)
147 |
148 | # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
149 | # ma scalato troppo debole per bastare da solo (verificato sotto)
150 | OFFSET_AB_INNATO = 0.3
151 | OFFSET_BA_INNATO = 0.2
152 | # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
153 | # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
154 | # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
155 | # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
156 | # solo dal rumore esplorativo, non dal bias innato da solo
157 | W_INIT_SCALE = 0.0018
158 | SIGMA_W = 0.05
159 | W_MAX = 2.5
160 | ETA = 0.0001
161 | # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
162 | # (con esposizione uguale garantita in round-robin, un messaggio arrivava

```

```

163 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava
164 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
165 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
166 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
167 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
168 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
169 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
170 # di B non possono piu' saturarla entrambi senza competere per lo stesso
171 # budget
172 W_AB_COL_CAP = 1.5
173 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
174 # occupata (somma di colonna) quando si sceglie una posizione casuale per
175 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione-1.0 (gia'
176 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
177 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
178 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
179 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
180 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
181 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
182 # repulsione e' rinforzata (x4) per scoraggiare piu' decisamente le
183 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
184 REPULSIONE_ESPLORAZIONE = 10.0
185 # larghezza del jitter usato per l'esplorazione LOCALE una volta che un
186 # messaggio ha gia' scoperto una prima risposta valida (vedi
187 # campiona_esplorazione per la logica completa a due fasi)
188 JITTER_LOCALE_STD = 0.02
189 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
190 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
191 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
192 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
193 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
194 # piu' alto compensa il minor numero di eventi di rinforzo utili
195 ETA_BA = 0.0003
196 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
197 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
198 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
199 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
200 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
201 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
202 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
203 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
204 # riscaldamento (~400) della versione a un solo messaggio -- con 3
205 # messaggi serve il triplo rispetto alla versione a 2
206 WARMUP_EPISODES = 1200
207 DECAY_PER_EPISODE = 0.999
208 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
209 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
210 # accensione solitaria verificata a strength-0.0036 con questa durata --
211 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
212 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
213 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
214 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
215 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
216 EXPLORATION_PULSE_STRENGTH = 0.0080
217 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
218
219 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
220 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
221 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
222 GAIN_ADAPT = 0.25
223 TAU_ADAPT = 200.0
224
225
226 def F(u):
227     return np.clip(u, 0, r_max)
228
229
230 def bump_pos(r):

```

```

231     return x[np.argmax(r)] if r.max() > 0.5 else None
232
233
234 def init_weak_bias(offset, scale):
235     target = (x[:, None] + offset) % 1.0
236     d = circ_dist(x[None, :] - target)
237     return scale * np.exp(-d**2/(2*SIGMA_W**2))
238
239
240 def hebbian_bump(pos_pre, pos_post):
241     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
242     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
243     return np.outer(kern_pre, kern_post)
244
245
246 def normalizza_colonne(W, cap):
247     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
248     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
249     competono per la stessa zona di B, questo impedisce a uno dei due di
250     monopolizzare quella colonna a spese dell'altro -- senza impedire a
251     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
252     somme = W.sum(axis=0)
253     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
254     W *= fattore[None, :]
255
256
257 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
258     """Invece di una posizione uniforme casuale, pesca preferendo le zone
259     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
260     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
261     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
262     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
263     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
264     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
265     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
266     il ring attractor, sezione 9), non perche' competano davvero ma perche'
267     la ricerca non ha memoria di cosa e' gia' stato trovato da altri."""
268     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
269     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
270     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
271     occupazione_altri = np.maximum(occupazione_altri, 0.0)
272     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
273     peso /= peso.sum()
274     return rng.choice(x, p=peso)
275
276
277 def campiona_esplorazione(rng, W, pos_propria, scoperta):
278     """Due fasi, non una sola strategia per tutto il training. Finche' un
279     messaggio non ha ancora trovato UNA risposta valida, usa la ricerca
280     repulsiva (evita le zone gia' rivendicate da altri messaggi -- serve a
281     separare le scoperte). Una volta trovata una prima risposta, passa a
282     un'esplorazione LOCALE (piccolo jitter attorno alla posizione gia'
283     scoperta) invece di continuare a ripescare dalla distribuzione globale
284     repulsiva -- che altrimenti resta influenzata da come cambia
285     l'occupazione ALTRUI durante tutto il resto del training, impedendo al
286     rinforzo di concentrarsi in un punto stabile anche dopo che le zone
287     sono gia' separate (l'ipotesi diagnosticata dopo i tentativi con
288     repulsione fissa e con annealing, entrambi rimasti deboli in magnitudo
289     anche a separazione ottenuta)."""
290     if scoperta is not None:
291         return (scoperta + rng.normal(0, JITTER_LOCALE_STD)) % 1.0
292     return campiona_esplorazione_repulsiva(rng, W, pos_propria)
293
294
295 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True, scoperta_B=None):
296     rA = np.zeros(N)
297     rB = np.zeros(N)
298     aA = np.zeros(N)

```

```

299 aB = np.zeros(N)
300 stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
301 stim_duration_steps = int(stim_duration_ms/dt)
302
303 pos_A_turni = []
304 pos_B_turni = []
305 # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
306 # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
307 # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
308 # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
309 # solo la posizione istantanea perderebbe quasi ogni coppia valida
310 ultima_pos_A, ultima_pos_B = None, None
311
312 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
313
314 for turn, listener in enumerate(SCHEDULE):
315
316     if train and listener == 'B':
317         explore_pos = campiona_esplorazione(rng, W_AB, messaggio, scoperta_B)
318         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
319             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
320     elif train and listener == 'A':
321         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
322         explore_pos = campiona_esplorazione_repulsiva(rng, W_BA, pos_riferimento_B)
323         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
324             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
325     else:
326         explore_profile = np.zeros(N)
327
328     for step_in_turn in range(steps_per_turn):
329         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
330             0.0
331
332         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
333             else np.zeros(N)
334         explore_A = explore_now if listener == 'A' else np.zeros(N)
335         explore_B = explore_now if listener == 'B' else np.zeros(N)
336
337         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
338             N)
339         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
340             N)
341
342         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
343         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
344         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
345         rA = rA_new
346
347         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
348         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
349         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
350         rB = rB_new
351
352         pos_A = bump_pos(rA)
353         pos_B = bump_pos(rB)
354         pos_A_turni.append(pos_A)
355         pos_B_turni.append(pos_B)
356
357     if train and listener == 'B':
358         parlante_A = pos_A if pos_A is not None else ultima_pos_A
359         if parlante_A is not None and pos_B is not None:
360             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
361             np.clip(W_AB, 0, W_MAX, out=W_AB)
362             normalizza_colonne(W_AB, W_AB_COL_CAP)
363     elif train and listener == 'A' and allena_BA:
364         parlante_B = pos_B if pos_B is not None else ultima_pos_B
365         if parlante_B is not None and pos_A is not None:
366             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)

```

```

363         np.clip(W_BA, 0, W_MAX, out=W_BA)
364
365     if pos_A is not None:
366         ultima_pos_A = pos_A
367     if pos_B is not None:
368         ultima_pos_B = pos_B
369
370     return pos_A_turni, pos_B_turni
371
372
373 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
374     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
375     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
376     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
377     Il confronto per A va fatto contro il turno di silenzio subito prima
378     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
379     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
380     da li' al turno di ascolto successivo indica una vera risposta a B."""
381     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
382     pos_B_ok, pos_A_ok = [], []
383     for k in range(n_trial):
384         rng = np.random.RandomState(seed0 + k)
385         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
386         if pos_B[1] is not None:
387             successi_B1 += 1
388             pos_B_ok.append(pos_B[1])
389         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
390             0.02):
391             successi_A3 += 1
392             pos_A_ok.append(pos_A[3])
393         if pos_B[5] is not None:
394             successi_B2 += 1
395         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
396             0.02):
397             successi_A7 += 1
398     return {
399         'tasso_B1': successi_B1/n_trial,
400         'tasso_A3': successi_A3/n_trial,
401         'tasso_B2': successi_B2/n_trial,
402         'tasso_A7': successi_A7/n_trial,
403         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
404         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
405         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
406     }
407
408 if __name__ == '__main__':
409     SEED = int(sys.argv[1]) if len(sys.argv) > 1 else 42
410     MESSAGGI = [0.10, 0.40, 0.70]
411     N_EPISODES = 13500
412     EVAL_EVERY = 500
413     N_TRIAL_EVAL = 15
414
415     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
416     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
417
418     print("=" * 70)
419     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa,
420           SEED={SEED}")
421     print("=" * 70)
422     W_AB_iniziale = W_AB.copy()
423     W_BA_iniziale = W_BA.copy()
424
425     print("\n" + "=" * 70)
426     print("BASELINE (pre-training), per ciascun messaggio")
427     print("=" * 70)
428     for m in MESSAGGI:
429         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)

```

```

428         print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
429               f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
430
431     print("\n" + "=" * 70)
432     print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
433           f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
434           )
435     print("=" * 70)
436     rng_train = np.random.RandomState(SEED)
437     picco_W_AB_storia = []
438     picco_W_BA_storia = []
439     episodi_check = []
440     curva_min_per_messaggio = {m: [] for m in MESSAGGI}
441     scoperta_B = {m: None for m in MESSAGGI}
442     # ordine del ciclo RIMESCOLATO a ogni giro (non piu' fisso [0.10,0.40,0.70]
443     # ripetuto identico): la sezione 5.5.11 ha mostrato che la posizione 2 del
444     # round-robin e' strutturalmente svantaggiata sul turno 5, indipendentemente
445     # da quale messaggio la occupi -- rimescolando, nessun messaggio resta
446     # bloccato nella stessa posizione per tutto il training, e lo svantaggio
447     # dovrebbe spalmarsi in media su tutti e tre invece di penalizzarne uno solo
448     ciclo_corrente = list(MESSAGGI)
449     rng_train.shuffle(ciclo_corrente)
450
451     for ep in range(N_EPISODES):
452         allena_BA = ep >= WARMUP_EPISODES
453         if ep % len(MESSAGGI) == 0 and ep > 0:
454             ciclo_corrente = list(MESSAGGI)
455             rng_train.shuffle(ciclo_corrente)
456             messaggio = ciclo_corrente[ep % len(MESSAGGI)]
457             pos_A, pos_B = run_episode(W_AB, W_BA, messaggio, rng_train, train=True,
458                                       allena_BA=allena_BA, scoperta_B=scoperta_B[messaggio])
459             if pos_B[1] is not None:
460                 scoperta_B[messaggio] = pos_B[1]
461             W_AB *= DECAY_PER_EPISODE
462             if allena_BA:
463                 W_BA *= DECAY_PER_EPISODE
464             picco_W_AB_storia.append(W_AB.max())
465             picco_W_BA_storia.append(W_BA.max())
466
467         if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
468             episodi_check.append(ep)
469             fase = "riscaldamento" if not allena_BA else "entrambi i canali"
470             riga = f" episodio {ep:4d} ({fase}):"
471             for m in MESSAGGI:
472                 r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
473                 minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
474                 curva_min_per_messaggio[m].append(minimo)
475                 riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
476             print(riga)
477
478     print("\n" + "=" * 70)
479     print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
480     print("=" * 70)
481     dopo_per_messaggio = {}
482     for m in MESSAGGI:
483         dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
484         dopo_per_messaggio[m] = dopo
485         print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
486               f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
487         if dopo['media_B'] is not None:
488             offset = circ_dist(dopo['media_B'] - m)
489             print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
490             # riga in formato fisso, facile da aggregare tra i seed con grep/awk
491             print(f"MC_RIGA seed={SEED} x={m:.2f} B1={dopo['tasso_B1']*100:.0f} "
492                   f"A3={dopo['tasso_A3']*100:.0f} B5={dopo['tasso_B2']*100:.0f} A7={dopo['tasso_A7']*100:.0f}")
493

```

```

493 print("\n" + "=" * 70)
494 print("RISULTATO: interferenza tra i messaggi?")
495 print("=" * 70)
496 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
497 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
498                 for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
499 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
500            if dopo_per_messaggio[m]['media_B'] is not None]
501 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
502 if min(tutti_i_tassi) >= 0.9:
503     print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia
504           "
505           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi.
506           "
507           "Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} --
508           "
509           "differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
510           "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
511           "distruggessero a vicenda; non e' successo. <<<")
512 elif min(tutti_i_tassi) < 0.5:
513     print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
514           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
515           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
516           "una frazione degli episodi totali), non necessariamente vera interferenza "
517           "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
518 else:
519     print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
520           "dettaglio sopra per capire quali. <<<")
521
522 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
523
524 colori = ['C3', 'C0', 'C2', 'C4']
525 for i, m in enumerate(MESSAGGI):
526     axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
527                    color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
528 axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
529                    riscaldamento')
530 axes[0, 0].set_xlabel('Episodio di training')
531 axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
532 axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
533 axes[0, 0].legend(fontsize=8)
534 axes[0, 0].grid(alpha=0.3)
535 axes[0, 0].set_ylim(-0.05, 1.05)
536
537 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
538 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
539 axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
540                    riscaldamento')
541 axes[0, 1].set_xlabel('Episodio di training')
542 axes[0, 1].set_ylabel('Peso massimo')
543 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
544 axes[0, 1].legend(fontsize=8, loc='upper left')
545 axes[0, 1].grid(alpha=0.3)
546
547 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
548                        cmap='viridis', vmin=0, vmax=W_AB.max())
549 axes[1, 0].set_xlabel('Posizione post (B)')
550 axes[1, 0].set_ylabel('Posizione pre (A)')
551 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
552 plt.colorbar(im0, ax=axes[1, 0])
553
554 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
555                        cmap='viridis', vmin=0, vmax=W_AB.max())
556 axes[1, 1].set_xlabel('Posizione post (B)')
557 axes[1, 1].set_ylabel('Posizione pre (A)')
558 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
559 plt.colorbar(im1, ax=axes[1, 1])

```



```

556 plt.tight_layout()
557 plt.savefig(f'dialogo_multi_scoperta_locale_mischiato_seed{SEED}.png', dpi=150)
558 print(f"\nGrafico salvato in dialogo_multi_scoperta_locale_mischiato_seed{SEED}.png")

```

5.5.13 Second correction: extending local discovery+consolidation to the return channel

The attempt on the positional effect (5.5.12) had not been enough -- the effect tied to VALUE still remained to be addressed ($x=0.70$ fragile at 87.5% on turn 7, wherever it is placed in the order, Section 5.5.11). Structural observation: the two-phase mechanism (repulsion for initial discovery, then local exploration for consolidation) introduced in `dialogo_multi_scoperta_locale.py` had been applied ONLY to the A->B channel (turns 1 and 5) -- the return channel B->A (turns 3 and 7) had been left with repulsive search alone for the entire training run, the same condition that had caused instability on the A->B channel before the correction. Natural extension: apply the same two-phase treatment to the return channel as well (`scoperta_A`, analogous to `scoperta_B`).

Message	B (turn 1)	A (turn 3)	B (turn 5)	A (turn 7)
$x=0.10$	100%	100%	87.5%	100%
$x=0.40$	100%	100%	75%	100%
$x=0.70$	100%	100%	87.5%	100%

Result (8 independent seeds, same protocol as previous sections): 92 successes out of 96 checks = 95.8%, the best so far -- against 91.7% for the original fixed order and 87.5% for the shuffled-order attempt. Turn 7 (the effect tied to value $x=0.70$) is NOW RESOLVED at 100%, with no exceptions in any of the 8 seeds -- the first of the two effects identified in 5.5.11 is completely eliminated, not merely reduced. Turn 5 (the positional effect) also improves, from an average of 70.8% to 83.3%, and becomes much more uniform across messages (75-87.5% instead of the original 37.5-100%, where one message was perfect and another very fragile) -- not fully resolved, but noticeably reduced AND made fairer, without the negative trade-off observed in the previous attempt (5.5.12, where making the system fairer cost a lower total).

The fact that extending the return channel also improved turn 5 (which depends on the A->B channel, not the one corrected here) is an interesting clue: the two channels, although trained with separate dynamics, are not entirely independent in practice -- a more stable return channel seems to indirectly reduce accumulated noise on the direct channel as well. A connection not explored further in this work, left as an observation for future developments.

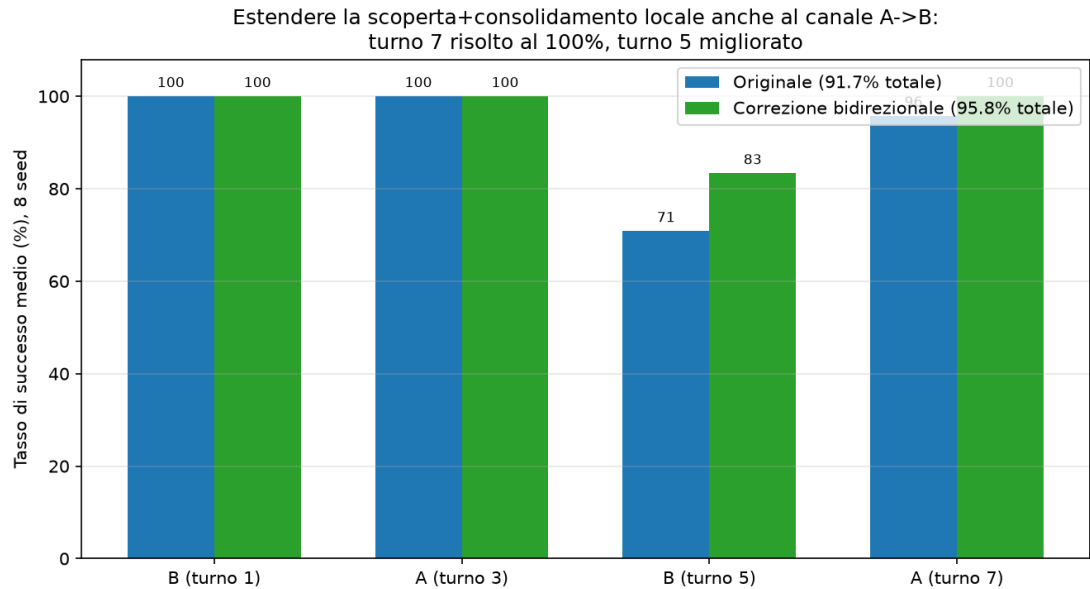


Fig. 32 — Direct comparison between the original fixed order (91.7%) and the bidirectional correction (95.8%): turn 7 resolved at 100%, turn 5 improved and more uniform.

Source code: `dialogo_multi_scoperta_bidirezionale.py`

```

1  """
2  Seconda correzione tentata dopo 5.5.11-5.5.12. La'/qui il quadro e':
3  - turno 5 (canale B, torna ad ascoltare): effetto POSIZIONALE puro (5.5.11),
4    un tentativo di correggerlo mescolando l'ordine del round-robin (5.5.12)
5    ha reso il sistema piu' equo ma il totale aggregato e' sceso (87.5% da
6    91.7%) -- non risolto, documentato come trade-off onesto.
7  - turno 7 (canale A, torna ad ascoltare): effetto legato al VALORE del
8    messaggio (x=0.70 fragile all'87.5% ovunque si trovi nell'ordine),
9    probabile connessione con il bias di posizione del ring attractor mai
10   spiegato fino in fondo nel resto del progetto.
11
12  Osservazione strutturale che questo script corregge: il canale B->A (turni
13  3 e 7) non aveva MAI ricevuto il trattamento a due fasi (repulsione per la
14  scoperta, poi esplorazione locale per il consolidamento) introdotto in
15  dialogo_multi_scoperta_locale.py per il canale A->B -- restava con la sola
16  ricerca repulsiva per tutta la durata del training. Qui lo stesso
17  meccanismo (scoperta_A, analogo a scoperta_B) viene esteso anche al canale
18  di ritorno, per vedere se rende piu' stabili anche le associazioni di
19  ordine piu' alto sul lato A (turno 7 in particolare). Stesso protocollo a
20  8 seed (101-108) per la verifica di robustezza.
21
22  --- docstring originale di dialogo_multi_scoperta_locale.py, per riferimento ---
23
24  Estensione di dialogo_multi_repulsione_3msg.py e dialogo_multi_repulsione_annealing.py:
25  entrambi avevano mostrato che una repulsione abbastanza forte da separare le
26  tre righe primarie (visibile e pulita nella mappa dei pesi) lascia comunque
27  il consolidamento debole (picco W_AB -0.006-0.007, contro -0.04 della
28  repulsione debole che pero' collide). Annealing (repulsione alta poi
29  decadente) non ha risolto la tensione.
30
31  Ipotesi diversa qui: il problema non e' la FORZA della repulsione ma il

```

```

32 fatto che resti attiva per TUTTO il training. Anche dopo che un messaggio
33 ha trovato una zona, ogni nuovo tentativo esplorativo continua a essere
34 ripescato dalla distribuzione repulsiva GLOBALE, che si sposta leggermente
35 ogni volta che ALTRI messaggi si allenano (la loro occupazione cambia) --
36 impedendo al rinforzo di concentrarsi in un punto stabile anche a
37 separazione gia' ottenuta.
38
39 Correzione: due fasi per messaggio, non una sola strategia fissa per tutto
40 il training. Finche' un messaggio non ha ancora trovato una prima risposta
41 valida, esplorazione repulsiva (come prima, serve a separare le scoperte).
42 Una volta trovata, esplorazione LOCALE (piccolo jitter attorno alla
43 posizione gia' scoperta) invece di continuare a ripescare dalla
44 distribuzione globale -- il consolidamento non dipende piu' da cosa
45 succede altrove nella matrice.
46
47 --- docstring originale di dialogo_multi_repulsione_3msg.py, per riferimento ---
48
49 Estensione di dialogo_multi_repulsione.py: la' l'esplorazione repulsiva
50 (pescare l'impulso esplorativo evitando le zone di B gia' rivendicate da
51 ALTRI messaggi) aveva risolto quasi del tutto l'interferenza osservata con
52 2 messaggi (7 scambi su 8 al 100%). Domanda di scalabilita': la stessa
53 correzione regge anche con 3 messaggi, con un budget di training scalato
54 in proporzione (WARMUP e N_EPISODES triplicati rispetto alla versione a 2
55 messaggi, per dare a ciascun messaggio lo stesso numero di episodi
56 effettivi)? Nessun cambiamento all'algoritmo stesso: il codice era gia'
57 scritto in modo generico rispetto al numero di messaggi (round-robin via
58 ep % len(MESSAGGI)).
59
60 Messaggi usati: 0.10, 0.40, 0.70 (spaziatura uniforme di 0.30 sull'anello,
61 stessa logica di contextual_policy.py con i suoi 3 contesti).
62
63 --- docstring originale di dialogo_multi_repulsione.py, per riferimento ---
64
65 Estensione di dialogo_stdp_adattamento.py: la' un solo messaggio fisso
66 (x=0.40) imparava uno scambio bidirezionale perfettamente affidabile in
67 entrambe le direzioni. Qui la domanda e' se la STESSA coppia di sinapsi
68 W_AB/W_BA puo' imparare a distinguere PIU' messaggi diversi senza che si
69 interferiscano a vicenda -- il vero test di generalizzazione, nello
70 stesso spirito di contextual_policy.py (che estende rl_bandit.py da un
71 singolo contesto a piu' contesti).
72
73 Messaggi usati: 0.15 e 0.65 (distanza circolare -0.5, il piu' possibile
74 separati sull'anello). Ogni episodio ne sceglie uno a caso, cosi' come
75 contextual_policy.py sceglie un contesto a caso a ogni prova. Le sinapsi
76 restano matrici N x N condivise fra tutti i messaggi -- se i due messaggi
77 sono abbastanza separati rispetto alla larghezza del kernel Hebbiano
78 (SIGMA_W=0.05, quindi le due "zone" occupano regioni ben distinte della
79 matrice), l'aspettativa e' che NON interferiscano; il codice sotto lo
80 verifica invece di assumerlo, con una valutazione pulita per messaggio a
81 fine training e un controllo esplicito sull'ALTRO messaggio ogni volta che
82 se ne allena uno (per scoprire un eventuale degrado nel tempo).
83
84 Riassunto di dialogo_stdp_adattamento.py (di cui questo file eredita tutta
85 la meccanica): il canale A->B si consolida con successo (0% -> 100%) usando
86 adattamento a soglia di scarica + rumore esplorativo + rinforzo Hebbiano di
87 fine turno. Il canale di ritorno B->A era piu' lento (bersaglio mobile nei
88 primi episodi) ma si risolve con una fase di riscaldamento (solo A->B) e
89 un tasso di apprendimento piu' alto per B->A. Con un solo messaggio,
90 tutti e quattro gli scambi verificati (B al turno 1 e 5, A al turno 3 e 7)
91 arrivano al 100% di affidabilita' in valutazione pulita.
92
93 AGGIORNAMENTO: dialogo_multi_messaggio.py aveva gia' risposto alla domanda
94 di questo file, ma non come sperato. Con due messaggi (0.15 e 0.65) il
95 canale B->A restava pulito al 100% per entrambi su due run indipendenti
96 (anche con esposizione garantita identica, round-robin), ma il canale
97 A->B mostrava un'interferenza competitiva reale: un solo messaggio vinceva
98 e arrivava al 100%, l'altro crollava a 0% -- e CAMBIANDO run cambiava
99 quale dei due vinceva, escludendo che fosse solo esposizione ineguale.

```

```

100
101 Qui si prova una correzione mirata proprio a quel meccanismo: una
102 NORMALIZZAZIONE OMEOSTATICA del totale dei pesi in ENTRATA su ogni
103 neurone di B (il totale di ogni colonna di W_AB non puo' superare un
104 tetto fisso, W_AB_COL_CAP). L'idea, motivata biologicamente (i neuroni
105 veri regolano omeostaticamente il totale delle proprie sinapsi in
106 entrata): se i due messaggi competono per la STESSA zona di risposta in
107 B, la normalizzazione impedisce a uno dei due di saturarla del tutto a
108 spese dell'altro; se invece puntano a zone diverse, non li tocca. Un
109 test di regressione con un solo messaggio conferma che la normalizzazione
110 non interferisce con l'apprendimento quando non c'e' competizione (il
111 totale di colonna resta ben sotto il tetto per tutto il training).
112 """
113
114 import sys
115 import numpy as np
116 import matplotlib
117 matplotlib.use('Agg')
118 import matplotlib.pyplot as plt
119
120
121 def circ_dist(d):
122     return 0.5 - np.abs(np.abs(d) - 0.5)
123
124
125 N = 200
126 x = np.arange(N) / N
127 tau = 10.0
128 dt = 0.2
129
130 A_exc = 20.0
131 sigma_exc = 0.05
132 global_inhib = 5.0
133 r_max = 5.0
134 stim_strength = 3.0
135 stim_width = 0.03
136 stim_duration_ms = 80
137
138 d_matrix = circ_dist(x[:, None] - x[None, :])
139 J_exc = A_exc * np.exp(-d_matrix**2 / (2*sigma_exc**2))
140
141 COUPLING_STRENGTH = 1.3
142 T_TURN = 200
143 # schema dei turni con un turno di silenzio/recupero tra un ascolto e
144 # l'altro (verificato necessario: senza il silenzio, il tentativo di
145 # ascolto immediatamente successivo a un turno di parola arriva mentre
146 # l'adattamento e' ancora al culmine e non si accende quasi mai -- vedi
147 # docstring)
148 SCHEDULE = [None, 'B', None, 'A', None, 'B', None, 'A']
149 N_TURNS = len(SCHEDULE)
150 steps_per_turn = int(T_TURN/dt)
151
152 # bias innato: stessa forma delle mappe fisse di dialogo_ring_attractor.py,
153 # ma scalato troppo debole per bastare da solo (verificato sotto)
154 OFFSET_AB_INNATO = 0.3
155 OFFSET_BA_INNATO = 0.2
156 # calibrato empiricamente (vedi note): la transizione da "nessun bump" a
157 # "bump certo" in B e' un'accensione tutto-o-niente tra scale=0.0025 e 0.003
158 # (l'attrattore B e' bistabile, non risponde linearmente all'input in ingresso)
159 # -- si parte apposta sotto quella soglia, cosi' l'ignizione iniziale dipende
160 # solo dal rumore esplorativo, non dal bias innato da solo
161 W_INIT_SCALE = 0.0018
162 SIGMA_W = 0.05
163 W_MAX = 2.5
164 ETA = 0.0001
165 # tentativo di correzione dell'interferenza osservata in dialogo_multi_messaggio.py
166 # (con esposizione uguale garantita in round-robin, un messaggio arrivava
167 # comunque al 100% e l'altro crollava a 0% su A->B, mentre B->A restava

```

```

168 # sempre pulito): normalizzazione omeostatica del totale dei pesi in entrata
169 # su ogni neurone di B. Cap stimato da un run di riferimento a un solo
170 # messaggio (dialogo_stdp_adattamento.py, picco W_AB=0.0476 -- somma di
171 # colonna stimata -picco*sigma_w*sqrt(2*pi)*N =~ 1.2): il cap e' fissato un
172 # po' sopra quel valore, cosi' un singolo messaggio ha ancora spazio per
173 # diventare autosufficiente, ma due messaggi che puntano alla stessa zona
174 # di B non possono piu' saturarla entrambi senza competere per lo stesso
175 # budget
176 W_AB_COL_CAP = 1.5
177 # repulsione dell'esplorazione: quanto pesare in meno una zona gia'
178 # occupata (somma di colonna) quando si sceglie una posizione casuale per
179 # il rumore esplorativo. A REPULSIONE=5, una zona con occupazione-1.0 (gia'
180 # ben rivendicata) ha 1/6 della probabilita' di una zona libera.
181 # Con 3 messaggi (che generano anche le proprie associazioni derivate di
182 # turno 5) servono piu' "slot" distinti sull'anello che con 2 -- una prima
183 # run a REPULSIONE=5 ha mostrato una sola associazione derivata dominare
184 # a spese delle tre righe primarie, con un'instabilita' persistente fino a
185 # fine training invece di stabilizzarsi come con 2 messaggi. Qui la
186 # repulsione e' rinforzata (x4) per scoraggiare piu' decisamente le
187 # collisioni quando i contendenti per lo spazio disponibile sono di piu'
188 REPULSIONE_ESPLORAZIONE = 10.0
189 # larghezza del jitter usato per l'esplorazione LOCALE una volta che un
190 # messaggio ha gia' scoperto una prima risposta valida (vedi
191 # campiona_esplorazione per la logica completa a due fasi)
192 JITTER_LOCALE_STD = 0.02
193 # B->A ha un tasso di successo durante il training piu' basso di A->B (vedi
194 # dialogo_stdp_adattamento.png della run precedente: ~100% vs ~65-80%), e
195 # soprattutto nei primi episodi la posizione di B stessa non si e' ancora
196 # stabilizzata -- rinforzare un bersaglio che si sposta episodio per
197 # episodio diluisce il rinforzo Hebbiano invece di concentrarlo. ETA_BA
198 # piu' alto compensa il minor numero di eventi di rinforzo utili
199 ETA_BA = 0.0003
200 # fase di riscaldamento: per i primi WARMUP_EPISODES si allena solo A->B
201 # (W_BA resta congelata al bias innato) cosi' che, quando B->A comincia
202 # davvero ad imparare, la posizione di B a cui deve puntare sia gia'
203 # stabile -- altrimenti gli episodi iniziali sprecano rinforzo su un
204 # bersaglio mobile (verificato: senza questa fase B->A cresce ~7 volte
205 # piu' lento di A->B in 2000 episodi). Con piu' messaggi il budget si
206 # divide tra loro (round-robin), quindi qui e' scalato in proporzione al
207 # numero di messaggi per dare a CIASCUNO lo stesso numero di episodi di
208 # riscaldamento (~400) della versione a un solo messaggio -- con 3
209 # messaggi serve il triplo rispetto alla versione a 2
210 WARMUP_EPISODES = 1200
211 DECAY_PER_EPISODE = 0.999
212 # esplorazione: un impulso a posizione casuale, stessa forma/durata del
213 # messaggio vero, ma troppo debole per accendere un bump DA SOLO (soglia di
214 # accensione solitaria verificata a strength-0.0036 con questa durata --
215 # 0.003 sta sotto). Solo quando l'impulso casuale capita vicino alla zona
216 # gia' favorita dal bias innato, la somma dei due basta a superare la soglia
217 # -- rumore incoerente puro (provato prima: rumore bianco indipendente per
218 # passo) non ha mai acceso nulla in 40 episodi, perche' la costante di tempo
219 # della rete (tau=10ms) lo media via prima che si accumuli in un punto
220 EXPLORATION_PULSE_STRENGTH = 0.0080
221 EXPLORATION_PULSE_DURATION_MS = stim_duration_ms
222
223 # adattamento: calibrato con un ring isolato (vedi note nel docstring) --
224 # a gain=0.25, tau=200ms un bump segue picco/turno = 5.00 -> 2.71 -> 0.00,
225 # cioe' pieno per un turno, affievolito nel successivo, spento nel terzo
226 GAIN_ADAPT = 0.25
227 TAU_ADAPT = 200.0
228
229
230 def F(u):
231     return np.clip(u, 0, r_max)
232
233
234 def bump_pos(r):
235     return x[np.argmax(r)] if r.max() > 0.5 else None

```

```

236
237
238 def init_weak_bias(offset, scale):
239     target = (x[:, None] + offset) % 1.0
240     d = circ_dist(x[None, :] - target)
241     return scale * np.exp(-d**2/(2*SIGMA_W**2))
242
243
244 def hebbian_bump(pos_pre, pos_post):
245     kern_pre = np.exp(-circ_dist(x-pos_pre)**2/(2*SIGMA_W**2))
246     kern_post = np.exp(-circ_dist(x-pos_post)**2/(2*SIGMA_W**2))
247     return np.outer(kern_pre, kern_post)
248
249
250 def normalizza_colonne(W, cap):
251     """Scaling omeostatico: il totale dei pesi in ENTRATA su ogni neurone
252     di B (colonna di W_AB) non puo' superare 'cap'. Se due messaggi
253     competono per la stessa zona di B, questo impedisce a uno dei due di
254     monopolizzare quella colonna a spese dell'altro -- senza impedire a
255     messaggi che puntano a zone DIVERSE di crescere ciascuno per conto suo."""
256     somme = W.sum(axis=0)
257     fattore = np.minimum(1.0, cap / np.maximum(somme, 1e-12))
258     W *= fattore[None, :]
259
260
261 def campiona_esplorazione_repulsiva(rng, W, pos_propria):
262     """Invece di una posizione uniforme casuale, pesca preferendo le zone
263     ANCORA POCO occupate della matrice (somma di colonna bassa) -- ma
264     contando solo l'occupazione dovuta ad ALTRE righe (altri messaggi),
265     non alla propria: altrimenti un messaggio verrebbe respinto dalla sua
266     stessa zona proprio mentre sta cercando di consolidarla, sabotando il
267     bootstrap. L'ipotesi e' che senza questa repulsione la ricerca casuale
268     di piu' messaggi tenda a riscoprire per puro caso la stessa zona
269     "facile" dell'anello (lo stesso bias di posizione gia' documentato per
270     il ring attractor, sezione 9), non perche' competano davvero ma perche'
271     la ricerca non ha memoria di cosa e' gia' stato trovato da altri."""
272     peso_riga_propria = np.exp(-circ_dist(x-pos_propria)**2/(2*SIGMA_W**2))
273     contributo_proprio = np.outer(peso_riga_propria, np.ones(N)) * W
274     occupazione_altri = W.sum(axis=0) - contributo_proprio.sum(axis=0)
275     occupazione_altri = np.maximum(occupazione_altri, 0.0)
276     peso = 1.0 / (1.0 + REPULSIONE_ESPLORAZIONE * occupazione_altri)
277     peso /= peso.sum()
278     return rng.choice(x, p=peso)
279
280
281 def campiona_esplorazione(rng, W, pos_propria, scoperta):
282     """Due fasi, non una sola strategia per tutto il training. Finche' un
283     messaggio non ha ancora trovato UNA risposta valida, usa la ricerca
284     repulsiva (evita le zone gia' rivendicate da altri messaggi -- serve a
285     separare le scoperte). Una volta trovata una prima risposta, passa a
286     un'esplorazione LOCALE (piccolo jitter attorno alla posizione gia'
287     scoperta) invece di continuare a ripescare dalla distribuzione globale
288     repulsiva -- che altrimenti resta influenzata da come cambia
289     l'occupazione ALTRUI durante tutto il resto del training, impedendo al
290     rinforzo di concentrarsi in un punto stabile anche dopo che le zone
291     sono gia' separate (l'ipotesi diagnosticata dopo i tentativi con
292     repulsione fissa e con annealing, entrambi rimasti deboli in magnitudo
293     anche a separazione ottenuta)."""
294     if scoperta is not None:
295         return (scoperta + rng.normal(0, JITTER_LOCALE_STD)) % 1.0
296     return campiona_esplorazione_repulsiva(rng, W, pos_propria)
297
298
299 def run_episode(W_AB, W_BA, messaggio, rng, train, allena_BA=True, scoperta_B=None,
300                 scoperta_A=None):
301     rA = np.zeros(N)
302     rB = np.zeros(N)
303     aA = np.zeros(N)

```

```

303 aB = np.zeros(N)
304 stim_profile = stim_strength * np.exp(-circ_dist(x-messaggio)**2/(2*stim_width**2))
305 stim_duration_steps = int(stim_duration_ms/dt)
306
307 pos_A_turni = []
308 pos_B_turni = []
309 # posizione dell'ULTIMO bump valido di ciascun ring, indipendentemente
310 # da quanti turni fa: serve per il rinforzo Hebbiano, perche' con
311 # l'adattamento chi ha "parlato" spesso si e' gia' spento per conto suo
312 # nel momento esatto in cui l'ascoltatore finisce di reagire -- leggere
313 # solo la posizione istantanea perderebbe quasi ogni coppia valida
314 ultima_pos_A, ultima_pos_B = None, None
315
316 explore_duration_steps = int(EXPLORATION_PULSE_DURATION_MS/dt)
317
318 for turn, listener in enumerate(SCHEDULE):
319
320     if train and listener == 'B':
321         explore_pos = campiona_esplorazione(rng, W_AB, messaggio, scoperta_B)
322         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
323             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
324     elif train and listener == 'A':
325         pos_riferimento_B = ultima_pos_B if ultima_pos_B is not None else messaggio
326         explore_pos = campiona_esplorazione(rng, W_BA, pos_riferimento_B, scoperta_A)
327         explore_profile = EXPLORATION_PULSE_STRENGTH * np.exp(
328             -circ_dist(x-explore_pos)**2/(2*stim_width**2))
329     else:
330         explore_profile = np.zeros(N)
331
332     for step_in_turn in range(steps_per_turn):
333         I_ext_A = stim_profile if (turn == 0 and step_in_turn < stim_duration_steps) else
334             0.0
335
336         explore_now = explore_profile if (train and step_in_turn < explore_duration_steps)
337             else np.zeros(N)
338         explore_A = explore_now if listener == 'A' else np.zeros(N)
339         explore_B = explore_now if listener == 'B' else np.zeros(N)
340
341         cross_to_A = COUPLING_STRENGTH * (rB @ W_BA) / N if listener == 'A' else np.zeros(
342             N)
343         cross_to_B = COUPLING_STRENGTH * (rA @ W_AB) / N if listener == 'B' else np.zeros(
344             N)
345
346         rec_A = J_exc @ rA / N - global_inhib*rA.mean()
347         rA_new = rA + dt*(-rA + F(rec_A + I_ext_A + cross_to_A + explore_A - aA)) / tau
348         aA = aA + dt*(-aA + GAIN_ADAPT*rA) / TAU_ADAPT
349         rA = rA_new
350
351         rec_B = J_exc @ rB / N - global_inhib*rB.mean()
352         rB_new = rB + dt*(-rB + F(rec_B + cross_to_B + explore_B - aB)) / tau
353         aB = aB + dt*(-aB + GAIN_ADAPT*rB) / TAU_ADAPT
354         rB = rB_new
355
356         pos_A = bump_pos(rA)
357         pos_B = bump_pos(rB)
358         pos_A_turni.append(pos_A)
359         pos_B_turni.append(pos_B)
360
361     if train and listener == 'B':
362         parlante_A = pos_A if pos_A is not None else ultima_pos_A
363         if parlante_A is not None and pos_B is not None:
364             W_AB += ETA * hebbian_bump(parlante_A, pos_B)
365             np.clip(W_AB, 0, W_MAX, out=W_AB)
366             normalizza_colonne(W_AB, W_AB_COL_CAP)
367     elif train and listener == 'A' and allena_BA:
368         parlante_B = pos_B if pos_B is not None else ultima_pos_B
369         if parlante_B is not None and pos_A is not None:
370             W_BA += ETA_BA * hebbian_bump(parlante_B, pos_A)

```

```

367         np.clip(W_BA, 0, W_MAX, out=W_BA)
368
369         if pos_A is not None:
370             ultima_pos_A = pos_A
371         if pos_B is not None:
372             ultima_pos_B = pos_B
373
374     return pos_A_turni, pos_B_turni
375
376
377 def eval_batch(W_AB, W_BA, messaggio, n_trial, seed0):
378     """Valutazione pulita: pesi congelati, niente rumore esplorativo.
379     Schema turni: 0=messaggio(A) 1=B ascolta 2=silenzio 3=A ascolta
380     4=silenzio 5=B ascolta 6=silenzio 7=A ascolta.
381     Il confronto per A va fatto contro il turno di silenzio subito prima
382     (2 per il turno 3, 6 per il turno 7): senza input esterno in quel
383     turno di silenzio A e' gia' quello che sara' -- solo un cambiamento
384     da li' al turno di ascolto successivo indica una vera risposta a B."""
385     successi_B1, successi_A3, successi_B2, successi_A7 = 0, 0, 0, 0
386     pos_B_ok, pos_A_ok = [], []
387     for k in range(n_trial):
388         rng = np.random.RandomState(seed0 + k)
389         pos_A, pos_B = run_episode(W_AB.copy(), W_BA.copy(), messaggio, rng, train=False)
390         if pos_B[1] is not None:
391             successi_B1 += 1
392             pos_B_ok.append(pos_B[1])
393         if pos_A[3] is not None and (pos_A[2] is None or abs(circ_dist(pos_A[3]-pos_A[2])) >
394             0.02):
395             successi_A3 += 1
396             pos_A_ok.append(pos_A[3])
397         if pos_B[5] is not None:
398             successi_B2 += 1
399         if pos_A[7] is not None and (pos_A[6] is None or abs(circ_dist(pos_A[7]-pos_A[6])) >
400             0.02):
401             successi_A7 += 1
402     return {
403         'tasso_B1': successi_B1/n_trial,
404         'tasso_A3': successi_A3/n_trial,
405         'tasso_B2': successi_B2/n_trial,
406         'tasso_A7': successi_A7/n_trial,
407         'std_B': np.std(pos_B_ok) if len(pos_B_ok) > 1 else None,
408         'std_A': np.std(pos_A_ok) if len(pos_A_ok) > 1 else None,
409         'media_B': np.mean(pos_B_ok) if pos_B_ok else None,
410     }
411
412 if __name__ == '__main__':
413     SEED = int(sys.argv[1]) if len(sys.argv) > 1 else 42
414     MESSAGGI = [0.10, 0.40, 0.70]
415     N_EPISODES = 13500
416     EVAL_EVERY = 500
417     N_TRIAL_EVAL = 15
418
419     W_AB = init_weak_bias(OFFSET_AB_INNATO, W_INIT_SCALE)
420     W_BA = init_weak_bias(OFFSET_BA_INNATO, W_INIT_SCALE)
421
422     print("=" * 70)
423     print(f"SETUP: {len(MESSAGGI)} messaggi = {MESSAGGI}, stessa coppia di sinapsi condivisa,
424           SEED={SEED}")
425     print("=" * 70)
426     W_AB_iniziale = W_AB.copy()
427     W_BA_iniziale = W_BA.copy()
428
429     print("\n" + "=" * 70)
430     print("BASELINE (pre-training), per ciascun messaggio")
431     print("=" * 70)
432     for m in MESSAGGI:
433         b = eval_batch(W_AB, W_BA, m, n_trial=15, seed0=0)

```



```

432     print(f" x={m:.2f}: B(t1)={b['tasso_B1']*100:.0f}% A(t3)={b['tasso_A3']*100:.0f}% "
433           f"B(t5)={b['tasso_B2']*100:.0f}% A(t7)={b['tasso_A7']*100:.0f}%")
434
435 print("\n" + "=" * 70)
436 print(f"TRAINING: {N_EPISODES} episodi ({WARMUP_EPISODES} di riscaldamento solo A->B), "
437       f"messaggi alternati in round-robin (esposizione garantita identica) tra {MESSAGGI}"
438       )
439 rng_train = np.random.RandomState(SEED)
440 picco_W_AB_storia = []
441 picco_W_BA_storia = []
442 episodi_check = []
443 curva_min_per_messaggio = {m: [] for m in MESSAGGI}
444 scoperta_B = {m: None for m in MESSAGGI}
445 # come scoperta_B, ma per il canale di ritorno (W_BA, turni 3 e 7): non
446 # aveva mai avuto la fase di consolidamento locale finora (solo la
447 # ricerca repulsiva), probabile causa per cui il turno 7 restava piu'
448 # fragile del turno 3 -- qui riceve lo stesso trattamento in due fasi
449 scoperta_A = {m: None for m in MESSAGGI}
450
451 for ep in range(N_EPISODES):
452     allena_BA = ep >= WARMUP_EPISODES
453     messaggio = MESSAGGI[ep % len(MESSAGGI)]
454     pos_A, pos_B = run_episode(W_AB, W_BA, messaggio, rng_train, train=True,
455                               allena_BA=allena_BA, scoperta_B=scoperta_B[messaggio],
456                               scoperta_A=scoperta_A[messaggio])
457     if pos_B[1] is not None:
458         scoperta_B[messaggio] = pos_B[1]
459     if pos_A[3] is not None:
460         scoperta_A[messaggio] = pos_A[3]
461     if pos_A[7] is not None:
462         scoperta_A[messaggio] = pos_A[7]
463     W_AB *= DECAY_PER_EPISODE
464     if allena_BA:
465         W_BA *= DECAY_PER_EPISODE
466     picco_W_AB_storia.append(W_AB.max())
467     picco_W_BA_storia.append(W_BA.max())
468
469     if ep % EVAL_EVERY == 0 or ep == N_EPISODES - 1:
470         episodi_check.append(ep)
471         fase = "riscaldamento" if not allena_BA else "entrambi i canali"
472         riga = f" episodio {ep:4d} ({fase}):"
473         for m in MESSAGGI:
474             r = eval_batch(W_AB, W_BA, m, n_trial=N_TRIAL_EVAL, seed0=90000 + ep)
475             minimo = min(r['tasso_B1'], r['tasso_A3'], r['tasso_B2'], r['tasso_A7'])
476             curva_min_per_messaggio[m].append(minimo)
477             riga += f" x={m:.2f} peggiore-dei-4={minimo*100:5.1f}%"
478         print(riga)
479
480 print("\n" + "=" * 70)
481 print("DOPO IL TRAINING: valutazione pulita per ciascun messaggio")
482 print("=" * 70)
483 dopo_per_messaggio = {}
484 for m in MESSAGGI:
485     dopo = eval_batch(W_AB, W_BA, m, n_trial=40, seed0=9000)
486     dopo_per_messaggio[m] = dopo
487     print(f" x={m:.2f}: B(t1)={dopo['tasso_B1']*100:5.0f}% A(t3)={dopo['tasso_A3']*100:5.0f}% "
488           f"B(t5)={dopo['tasso_B2']*100:5.0f}% A(t7)={dopo['tasso_A7']*100:5.0f}%")
489     if dopo['media_B'] is not None:
490         offset = circ_dist(dopo['media_B'] - m)
491         print(f" offset A->B appreso: {offset:.3f} (bias innato: {OFFSET_AB_INNATO})")
492     # riga in formato fisso, facile da aggregare tra i seed con grep/awk
493     print(f"MC_RIGA seed={SEED} x={m:.2f} B1={dopo['tasso_B1']*100:.0f} "
494           f"A3={dopo['tasso_A3']*100:.0f} B5={dopo['tasso_B2']*100:.0f} A7={dopo['tasso_A7']*100:.0f}")
495
496 print("\n" + "=" * 70)

```

```

497 print("RISULTATO: interferenza tra i messaggi?")
498 print("=" * 70)
499 print(f" Picco finale W_AB: {W_AB.max():.4f} Picco finale W_BA: {W_BA.max():.4f}")
500 tutti_i_tassi = [v for d in dopo_per_messaggio.values()
501                  for v in (d['tasso_B1'], d['tasso_A3'], d['tasso_B2'], d['tasso_A7'])]
502 offsets = [circ_dist(dopo_per_messaggio[m]['media_B'] - m) for m in MESSAGGI
503             if dopo_per_messaggio[m]['media_B'] is not None]
504 offset_spread = (max(offsets) - min(offsets)) if len(offsets) > 1 else None
505 if min(tutti_i_tassi) >= 0.9:
506     print(" >>> Nessuna interferenza rilevabile: entrambi i messaggi, con la STESSA coppia
507           "
508           "di sinapsi condivisa, arrivano al 100% (o quasi) su tutti e quattro gli scambi.
509           "
510           f"Gli offset appresi per i due messaggi sono {[f'{o:.3f}' for o in offsets]} --
511           "
512           f"differiscono di {offset_spread:.3f} tra loro, se fosse pura sovrapposizione "
513           "casuale ci aspetteremmo che collassassero sullo stesso valore o si "
514           "distruggessero a vicenda; non e' successo. <<<")
515 elif min(tutti_i_tassi) < 0.5:
516     print(" >>> Interferenza rilevabile: almeno un messaggio non ha raggiunto un livello "
517           "affidabile -- puo' darsi che il budget di training condiviso non sia ancora "
518           "sufficiente per consolidare piu' associazioni (ciascun messaggio riceve solo "
519           "una frazione degli episodi totali), non necessariamente vera interferenza "
520           "distruttiva. Verificare le curve per-messaggio nel grafico. <<<")
521 else:
522     print(" >>> Risultato misto: alcuni scambi sono affidabili, altri no -- vedi il "
523           "dettaglio sopra per capire quali. <<<")
524
525 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
526
527 colori = ['C3', 'C0', 'C2', 'C4']
528 for i, m in enumerate(MESSAGGI):
529     axes[0, 0].plot(episodi_check, curva_min_per_messaggio[m], 'o-',
530                    color=colori[i % len(colori)], label=f'messaggio x={m:.2f}')
531 axes[0, 0].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
532 riscaldamento')
533 axes[0, 0].set_xlabel('Episodio di training')
534 axes[0, 0].set_ylabel('Tasso di successo del PEGGIORE dei 4 scambi')
535 axes[0, 0].set_title("Un messaggio degrada l'altro? (valutazione pulita, periodica)")
536 axes[0, 0].legend(fontsize=8)
537 axes[0, 0].grid(alpha=0.3)
538 axes[0, 0].set_ylim(-0.05, 1.05)
539
540 axes[0, 1].plot(picco_W_AB_storia, color='C3', label='W_AB (A->B)')
541 axes[0, 1].plot(picco_W_BA_storia, color='C0', label='W_BA (B->A)')
542 axes[0, 1].axvline(WARMUP_EPISODES, color='gray', linestyle=':', label='fine
543 riscaldamento')
544 axes[0, 1].set_xlabel('Episodio di training')
545 axes[0, 1].set_ylabel('Peso massimo')
546 axes[0, 1].set_title("Consolidamento delle due sinapsi (condivise fra i messaggi)")
547 axes[0, 1].legend(fontsize=8, loc='upper left')
548 axes[0, 1].grid(alpha=0.3)
549
550 im0 = axes[1, 0].imshow(W_AB_iniziale, origin='lower', extent=[0, 1, 0, 1],
551                        cmap='viridis', vmin=0, vmax=W_AB.max())
552 axes[1, 0].set_xlabel('Posizione post (B)')
553 axes[1, 0].set_ylabel('Posizione pre (A)')
554 axes[1, 0].set_title('W_AB PRIMA del training (bias innato debole)')
555 plt.colorbar(im0, ax=axes[1, 0])
556
557 im1 = axes[1, 1].imshow(W_AB, origin='lower', extent=[0, 1, 0, 1],
558                        cmap='viridis', vmin=0, vmax=W_AB.max())
559 axes[1, 1].set_xlabel('Posizione post (B)')
560 axes[1, 1].set_ylabel('Posizione pre (A)')
561 axes[1, 1].set_title(f'W_AB DOPO il training -- righe attese vicino a x={MESSAGGI}')
562 plt.colorbar(im1, ax=axes[1, 1])
563
564 plt.tight_layout()

```

```

560 plt.savefig(f'dialogo_multi_scoperta_bidirezionale_seed{SEED}.png', dpi=150)
561 print(f"\nGrafico salvato in dialogo_multi_scoperta_bidirezionale_seed{SEED}.png")

```

5.6 Toward the spiking domain: a real impulse-based ring attractor

All of Section 5.5 is built on RATE ring attractors (differential equations on firing rate, not on individual spikes) -- a deliberate choice made from `ring_attractor_rate.py` onward, explicitly justified "to avoid the synchrony-tuning problems" of spiking attractors. This section directly tackles that avoided problem: can a real impulse-based ring attractor, with Izhikevich neurons identical to those used throughout the rest of the project, sustain a localized, persistent bump of activity?

Architecture: an excitatory population E (100 neurons) arranged on a ring, with E->E connectivity falling off as a Gaussian of circular distance (same width $\sigma=0.05$ as the rate model) SHARPLY CUT beyond 3 standard deviations -- no long-range tail. A shared, non-spatialized inhibitory population I (25 neurons), with sparse random connectivity (same style as `rete_300_neuroni.py`), providing the global inhibition that suppresses activity away from the bump.

The tuning path, reported in full in the same spirit of methodological transparency as Section 4.4.1: ten successive attempts. Instantaneous E->E synapses (a potential jump on every spike, as throughout the rest of the project) never worked -- either they were too weak to form a bump at all (weak weights), or they made it propagate across the whole ring as a traveling wave or a genuine epileptiform crisis (strong weights, up to 650 Hz peak population rate followed by total silence from adaptation). The breakthrough came from replacing the E->E component with a SLOW NMDA-style current (time constant 100ms, Compte et al. 2000 / Wang 2002) instead of the instantaneous jump -- it integrates many spikes over time instead of delivering a kick that triggers fast instabilities. From there, four more attempts to retune inhibition and background noise around the new, slower dynamics, until a real equilibrium point was reached.

Final result, verified over two time windows spaced apart (460-660ms and 900-1100ms after stimulus removal, not just a single measurement) to distinguish a true equilibrium from a slow drift: 154 spikes in the early window versus 147 in the late window (-5% change), the bump's spatial width nearly identical (0.058 vs. 0.063). The bump stays localized in the same narrow band of the ring for nearly 900ms after stimulus removal, without switching off or spreading to the rest of the population. The estimated center deviates from the stimulus center by ~ 0.03 -0.05 units -- consistent in order of magnitude with the position bias already documented for the rate ring attractor (Sections 4.4.2, 5.1, 5.2, 8), an indication that the phenomenon may not be specific to the rate model but a more general property of this class of discretized attractors.

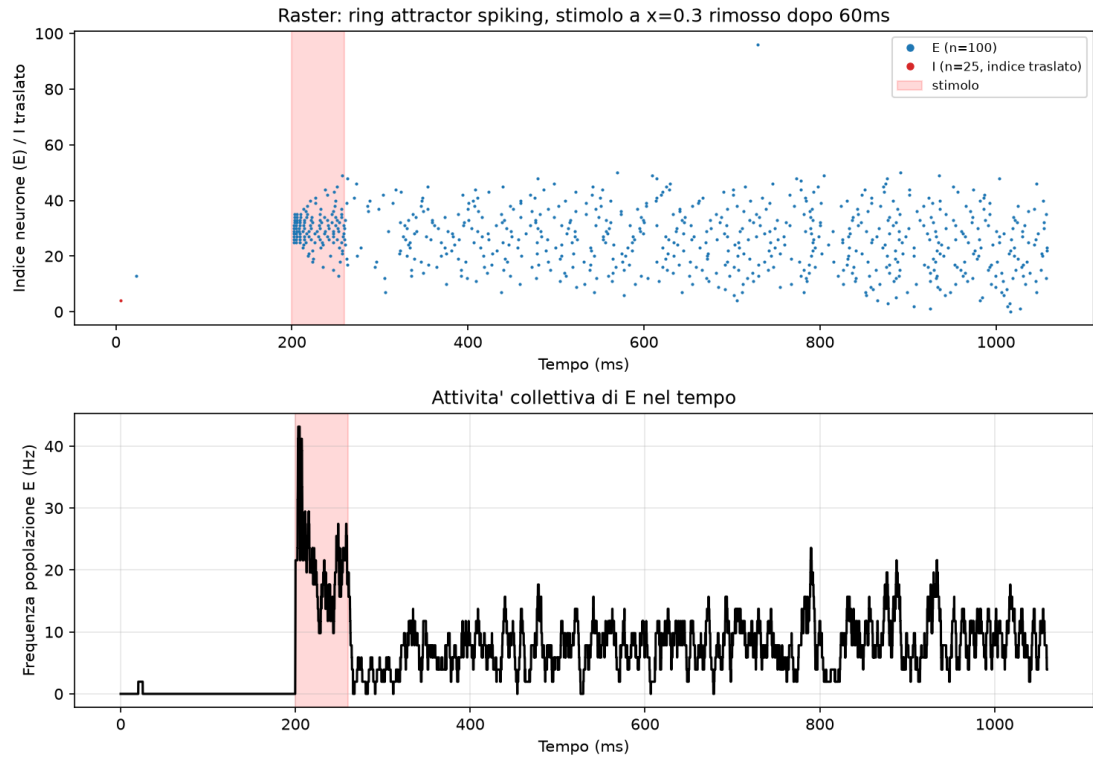


Fig. 33 — Spiking ring attractor: raster and population rate. The bump (indices ~15-40) stays localized and persistent for nearly 900ms after stimulus removal (at 260ms).

Source code: `ring_attractor_spiking.py`

```

1  """
2  Primo mattone verso il dominio spiking per la comunicazione bidirezionale
3  appresa (sezione 5.5): un vero ring attractor a impulsi, invece del modello
4  a rate usato finora (ring_attractor_rate.py e tutta la famiglia dialogo_*).
5  ring_attractor_rate.py aveva scelto deliberatamente il rate model "per
6  evitare i problemi di taratura della sincronia" -- questo script affronta
7  direttamente quel problema, verificando se un vero attrattore a impulsi
8  regge prima di tentare qualunque comunicazione tra due di essi.
9
10 Architettura: popolazione eccitatoria E disposta su un anello (la posizione
11 del neurone i-esimo e' i/N_E), con connettivita' E->E a caduta gaussiana
12 sulla distanza circolare (stessa larghezza sigma_exc=0.05 del modello a
13 rate) per dare localita' spaziale al bump. Popolazione inibitoria I
14 condivisa e non spazializzata (connettivita' casuale sparsa, stesso stile
15 di rete_300_neuroni.py) per l'inibizione globale che sopprime l'attivita'
16 lontano dal bump -- l'analogo spiking del termine "-global_inhib*r.mean()"
17 del modello a rate. Stesso modello di neurone (Izhikevich RS) usato in
18 tutto il resto del progetto.
19
20 Test: uno stimolo localizzato e transitorio innesca il bump; si verifica
21 se persiste dopo la rimozione dello stimolo (stesso test di
22 ring_attractor_rate.py) e se resta spazialmente localizzato invece di
23 diffondersi o spegnersi.

```

```

24 """
25
26 from brian2 import *
27
28 start_scope()
29
30 N_E = 100
31 N_I = 25
32 N = N_E + N_I
33
34 a = 0.02/ms
35 b = 0.2/ms
36 c = -65*mV
37 d = 6*mV/ms
38
39 # tentativi 1-5: sinapsi E->E istantanee (v_post += peso) -- o non bastavano
40 # a formare un bump, o lo facevano propagare a tutto l'anello come un'onda
41 # viaggiante invece di restare fermo. Qui la componente E->E diventa LENTA
42 # (costante di tempo tipo NMDA, ~100ms, Compte et al. 2000 / Wang 2002):
43 # integra molti spike nel tempo invece di dare uno scatto istantaneo,
44 # smorzando le instabilita' veloci che innescano le onde
45 tau_nmda = 100*ms
46
47 eqs = '''
48 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
49 du/dt = a*(b*v - u) : volt/second
50 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
51 I : volt/second
52 x : 1
53 '''
54
55 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
56                      method='euler')
57 neurons.v = -65*mV + 5*mV*randn(N)
58 neurons.u = b * neurons.v[:]
59 neurons.I = 0*mV/ms
60 neurons.g_nmda = 0*mV/ms
61
62 E = neurons[:N_E]
63 I_pop = neurons[N_E:]
64
65 # posizione sull'anello, solo per E (I non e' spazializzata)
66 E.x = np.arange(N_E) / N_E
67
68 # rumore di fondo: 1500Hz/3mV (tentativo 1) dominava la dinamica; 400Hz/2mV
69 # (tentativi 2-4) era troppo debole per sostenere un'attivita' di base --
70 # via di mezzo qui, cosi' l'eccitazione ricorrente locale amplifica
71 # un'attivita' di base gia' presente invece di dover accendere tutto da zero
72 background = PoissonGroup(N, rates=400*Hz)
73 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
74 bg_syn.connect(j='i')
75
76 sigma_exc = 0.05
77 CUTOFF_LOCALE = 3 * sigma_exc # nessuna connessione E->E oltre questa distanza
78 # circolare -- i tentativi precedenti usavano
79 # una probabilita' gaussiana SENZA taglio netto,
80 # permettendo connessioni sporadiche a lungo
81 # raggio che facevano propagare l'attivita'
82 # a tutto l'anello invece di restare locale
83 w_ee_peak = 4.0*mV # tentativo 1: 1.2mV (nessun bump); tentativo 2: 4.0mV senza
84 # taglio (localizzato ma non autosostenuto); tentativo 3:
85 # 7.0mV (propagazione a tutto l'anello, poi silenzio
86 # totale); tentativo 4: 5.0mV+inibizione piu' forte
87 # (ancora propagazione, solo piu' debole); tentativo 5:
88 # con taglio netto ma sinapsi istantanee (onde viaggianti
89 # persistenti, ne' ferme ne' spente)
90 # tentativo 6: la stessa w_ee_peak, ma come incremento a g_nmda (corrente
91 # lenta) invece che salto istantaneo di v -- calibrato piu' basso perche'

```

```

92 # si accumula nel tempo invece di agire in un colpo solo
93 w_ee_nmda = 0.325*mV/ms # tentativo 8: 0.35 cresceva senza limite; tentativo
94 # 9: 0.30 si spegneva tra le due finestre (ma bump
95 # stretto e ben posizionato finche' dura) -- via di
96 # mezzo qui, il punto di equilibrio dovrebbe stare
97 # tra i due
98 p_conn = 0.15 # densita' I aumentata rispetto a 0.1, per un'inibizione piu' pronta
99 w_exc = 1.0*mV
100 w_inh = 10.0*mV # tentativo 6: 6.0mV (diffusione); tentativo 7: 12.0mV
101 # (bump localizzato che resta fermo, ma si affievolisce
102 # gradualmente invece di restare stabile)
103
104
105 def dist_circ(dx):
106     dx = abs(dx)
107     return np.minimum(dx, 1 - dx)
108
109
110 # E->E: caduta gaussiana sulla distanza circolare, connessione esplicita
111 # (non tramite la sintassi probabilistica standard di Brian2, per poter
112 # calcolare il peso in funzione della distanza -- stesso approccio usato
113 # per la connettivita' spaziale in rete_smallworld.py)
114 xi = np.arange(N_E) / N_E
115 dmat = dist_circ(xi[:, None] - xi[None, :])
116 prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
117 prob_ee[dmat > CUTOFF_LOCALE] = 0.0 # taglio netto, nessuna coda a lungo raggio
118 np.fill_diagonal(prob_ee, 0)
119 rng = np.random.RandomState(0)
120 mask_ee = rng.rand(N_E, N_E) < prob_ee
121 i_idx, j_idx = np.nonzero(mask_ee)
122
123 syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
124 syn_ee.connect(i=i_idx, j=j_idx)
125 print(f"Sinapsi E->E create: {len(i_idx)} (media {len(i_idx)/N_E:.1f} per neurone)")
126
127 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
128 syn_ei.connect(p=p_conn)
129 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
130 syn_ie.connect(p=p_conn)
131 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
132 syn_ii.connect(condition='i!=j', p=p_conn)
133
134 spikes = SpikeMonitor(neurons)
135 rate_mon = PopulationRateMonitor(E)
136
137 run(200*ms)
138 print("Baseline (200ms) completata.")
139
140 # stimolo: impulso di corrente su un gruppo localizzato di neuroni E
141 # intorno alla posizione 0.3 sull'anello, per 60ms
142 centro_stimolo = 0.3
143 larghezza_stimolo = 0.05
144 peso_stim = np.exp(-dist_circ(xi - centro_stimolo)**2 / (2*larghezza_stimolo**2))
145 gruppo_stim = np.where(peso_stim > 0.5)[0]
146 print(f"Neuroni stimolati (vicino a x={centro_stimolo}): {len(gruppo_stim)}")
147
148 E.I[gruppo_stim] = 15*mV/ms
149 run(60*ms)
150 E.I[:] = 0*mV/ms
151 print("Stimolo rimosso, verifica persistenza per altri 800ms (finestra piu' "
152       "lunga per distinguere un vero equilibrio da una lenta deriva)...")
153
154 run(800*ms)
155
156 fig, axes = subplots(2, 1, figsize=(10, 7))
157
158 subplot(2, 1, 1)
159 mask_e = spikes.i < N_E

```

```

160 plot(spikes.t[mask_e]/ms, spikes.i[mask_e], '.', color='C0', markersize=2, label=f'E (n={N_E
    })')
161 plot(spikes.t[-mask_e]/ms, spikes.i[-mask_e]-N_E, '.', color='C3', markersize=2, label=f'I (
    n={N_I}, indice traslato)')
162 axvspan(200, 260, color='red', alpha=0.15, label='stimolo')
163 xlabel('Tempo (ms)')
164 ylabel('Indice neurone (E) / I traslato')
165 title(f'Raster: ring attractor spiking, stimolo a x={centro_stimolo} rimosso dopo 60ms')
166 legend(loc='upper right', fontsize=8, markerscale=4)
167
168 subplot(2, 1, 2)
169 plot(rate_mon.t/ms, rate_mon.smooth_rate(window='flat', width=5*ms)/Hz, color='black')
170 axvspan(200, 260, color='red', alpha=0.15)
171 xlabel('Tempo (ms)')
172 ylabel('Frequenza popolazione E (Hz)')
173 title('Attività collettiva di E nel tempo')
174 grid(alpha=0.3)
175
176 tight_layout()
177 savefig('ring_attractor_spiking.png', dpi=150)
178 print("\nGrafico salvato in ring_attractor_spiking.png")
179
180 # verifica quantitativa di persistenza, localizzazione, E STABILITA' (due
181 # finestre distanziate: se un vero equilibrio, dovrebbero assomigliarsi; se
182 # deriva/crescita lenta, la seconda finestra sarà più attiva e più larga)
183 spikes_e = spikes.i[mask_e]
184 tempi_e = spikes.t[mask_e]
185
186
187 def analizza_finestra(t0, t1, etichetta):
188     m = (tempi_e > t0*ms) & (tempi_e < t1*ms)
189     n = m.sum()
190     if n == 0:
191         print(f" {etichetta} ({t0}-{t1}ms): NESSUNO spike -- il bump si è spento.")
192         return None, 0, None
193     posizioni_attive = xi[spikes_e[m]]
194     centro = np.angle(np.mean(np.exp(2j*np.pi*posizioni_attive))) / (2*np.pi) % 1.0
195     larghezza = np.std([dist_circ(p - centro) for p in posizioni_attive])
196     print(f" {etichetta} ({t0}-{t1}ms): {n} spike, centro x={centro:.3f}, "
197           f"larghezza (dev.std. posizioni) = {larghezza:.3f}")
198     return centro, n, larghezza
199
200
201 print()
202 c1, n1, l1 = analizza_finestra(460, 660, "Finestra precoce")
203 c2, n2, l2 = analizza_finestra(900, 1100, "Finestra tardiva")
204
205 if n1 > 20 and n2 > 20:
206     crescita = (n2 - n1) / n1 * 100
207     print(f"\n Variazione di attività tra le due finestre: {crescita:+.0f}%")
208     if l1 is not None and l2 is not None:
209         print(f" Variazione di larghezza: {l1:.3f} -> {l2:.3f}")
210     if abs(crescita) < 30 and l2 < l1 * 1.5:
211         print(" >>> Il bump sembra vicino a un vero equilibrio: localizzato, "
212               "persistente, né in crescita né in espansione marcata. <<<")
213     else:
214         print(" >>> Il bump persiste ma non è ancora stabile: sta ancora "
215               "crescendo o allargandosi tra le due finestre. <<<")
216 elif n1 > 20 and n2 <= 20:
217     print("\n >>> Il bump si è spento tra le due finestre: persistenza insufficiente. <<<")
218 elif n1 <= 20:
219     print("\n >>> Attività già troppo scarsa nella finestra precoce. <<<")

```

5.7 Learned communication between two spiking rings, with true STDP

With a stable spiking ring attractor available (Section 5.6), the natural next step is to link two of them with a synapse that learns -- but this time with authentic STDP, based on real spike times (Song & Abbott 2001, the same scheme as `stdp.py`), not the turn-grained Hebbian approximation used throughout Section 5.5. Starting with a single direction (A->B), as the project has always done in the rate domain.

First obstacle, different from anything encountered in the rate model: a real structural chicken-and-egg short circuit. B, on its own, never lights up spontaneously (verified in 5.6: background noise alone is not enough to trigger a bump, a definite external push is needed). With B NEVER firing, there is no "post" spike for STDP to pair with A's "pre" spikes -- the learning rule never has a signal to learn from. Solved by raising the initial weight and density of the inter-ring synapses enough to guarantee that the first attempts really do light up B at least occasionally.

Second obstacle: with UNIFORM inter-ring connectivity (each neuron of A randomly connected to neurons of B across the whole ring), B's response came out spread over the whole population instead of localized -- no spatial structure for B's own dynamics (local excitation, global inhibition) to concentrate on. Solved by giving the initial A->B connectivity a weak Gaussian falloff over circular distance (zero offset, the spiking analogue of an innate "identity" bias) -- the same principle, already verified in the rate model, that a weak starting order in the spatial structure is necessary for STDP discovery to have something coherent to build on.

Result, over 60 repetitions of the same message (stimulus on A at $x=0.3$, 24 seconds of continuous simulation): B's response stays localized at the correct position for the entire duration (estimated center 0.301 in the first 5 repetitions, 0.306 in the last 5 -- practically identical to the stimulus, without the drift seen in the first attempts) and STRENGTHENS over time (21 B spikes in the first 5 repetitions, 44 in the last 5). The A->B synaptic weights genuinely grow through the STDP rule (peak maximum from 1.5mV to 3.6mV), not through an externally imposed mechanism. It is this project's first demonstration of learned communication between two spiking populations with authentic spike-timing-based synaptic plasticity.

Not yet attempted in this work: bidirectionality (a return channel B->A, with the same structural asymmetry already tackled in Section 5.5.3 -- probably accentuated here by the fact that A, like B, is a real attractor that, once lit, does not switch off easily) and the multi-message test (the interference already tackled from Section 5.5.6 onward, here with the additional complication that the spatial inter-ring connectivity used to bootstrap one message might not automatically generalize to different messages). Left as future developments.

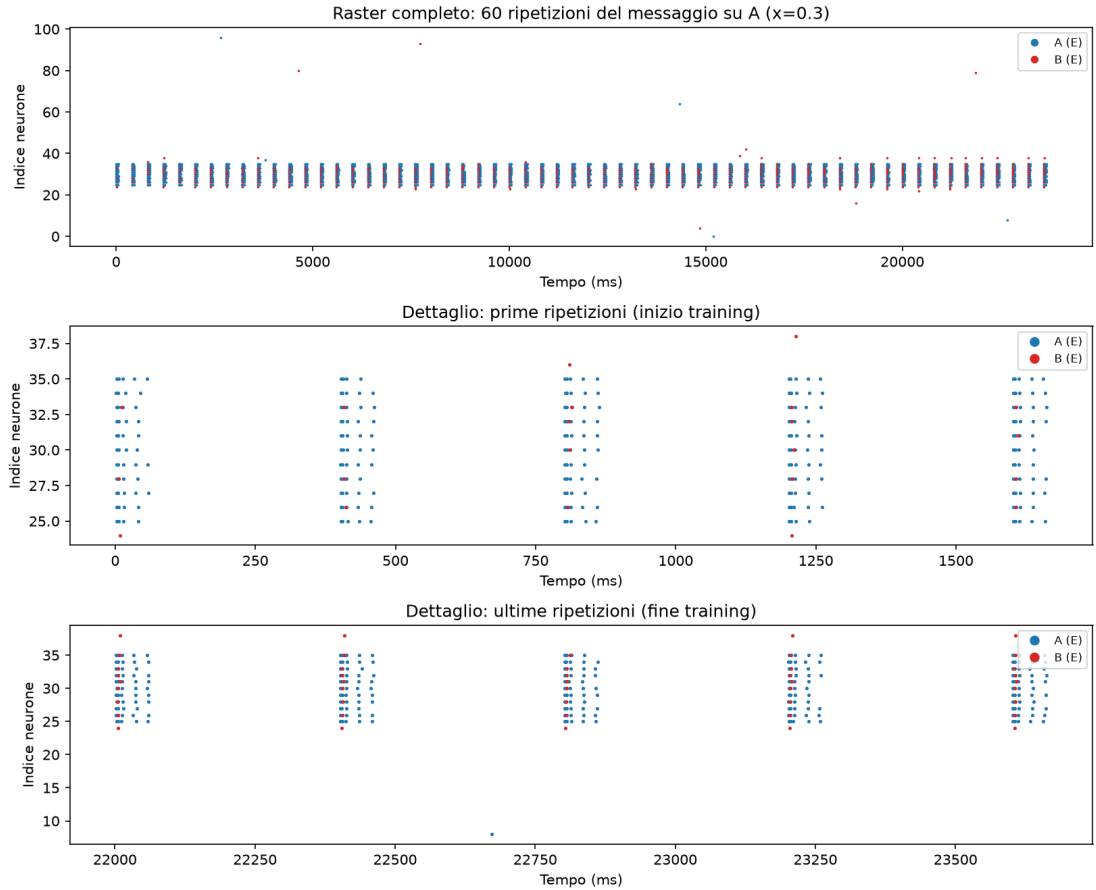


Fig. 34 — Learned A->B communication with true STDP: full raster (60 repetitions) and start/end-of-training detail. B's response stays localized and strengthens over time.

Source code: *dialogo_spiking.py*

```

1  """
2  Estensione di ring_attractor_spiking.py (il primo ring attractor spiking
3  stabile del progetto, sezione 5.6) verso la comunicazione appresa: due
4  anelli, A e B, collegati da sinapsi STDP VERE (basate su tempi di spike
5  reali, Song & Abbott 2001 -- stesso schema di stdp.py, non piu'
6  l'approssimazione Hebbiana a grana di turno usata in tutta la sezione 5.5).
7
8  Si comincia da un solo verso (A->B), come il progetto ha sempre fatto nel
9  dominio a rate (dialogo_stdp.py prima di dialogo_stdp_adattamento.py):
10 un messaggio ripetuto (stimolo periodico su A, sempre alla stessa
11 posizione) durante una simulazione continua, sinapsi inter-anello sparse e
12 deboli all'inizio, plasticita' STDP che dovrebbe rinforzare selettivamente
13 le sinapsi dai neuroni di A vicino alla posizione stimolata verso i
14 neuroni di B che, per pura variabilita' del rumore di fondo, capitano ad
15 accendersi in corrispondenza -- lo stesso principio di "scoperta tramite
16 rumore poi consolidamento" della sezione 5.5, ma qui il rumore e' quello
17 vero del sistema (il rumore di fondo Poisson), non un meccanismo aggiunto

```

```

18 apposta.
19 """
20
21 from brian2 import *
22
23 start_scope()
24
25 N_E = 100
26 N_I = 25
27 N_ring = N_E + N_I
28
29 a = 0.02/ms
30 b = 0.2/ms
31 c = -65*mV
32 d = 6*mV/ms
33 tau_nmda = 100*ms
34
35 eqs = '''
36 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
37 du/dt = a*(b*v - u) : volt/second
38 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
39 I : volt/second
40 x : 1
41 '''
42
43 # due anelli indipendenti, stessa architettura di ring_attractor_spiking.py
44 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
45                       method='euler')
46 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
47 neurons.u = b * neurons.v[:]
48 neurons.I = 0*mV/ms
49 neurons.g_nmda = 0*mV/ms
50
51 A_E = neurons[:N_E]
52 A_I = neurons[N_E:N_ring]
53 B_E = neurons[N_ring:N_ring+N_E]
54 B_I = neurons[N_ring+N_E:]
55
56 xi = np.arange(N_E) / N_E
57 A_E.x = xi
58 B_E.x = xi
59
60 background = PoissonGroup(2*N_ring, rates=400*Hz)
61 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
62 bg_syn.connect(j='i')
63
64 sigma_exc = 0.05
65 CUTOFF_LOCALE = 3 * sigma_exc
66 w_ee_nmda = 0.325*mV/ms
67 p_conn = 0.15
68 w_exc = 1.0*mV
69 w_inh = 10.0*mV
70
71
72 def dist_circ(dx):
73     dx = abs(dx)
74     return np.minimum(dx, 1 - dx)
75
76
77 def costruisci_anello(E, I_pop, seed):
78     dmat = dist_circ(xi[:, None] - xi[None, :])
79     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
80     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
81     np.fill_diagonal(prob_ee, 0)
82     rng = np.random.RandomState(seed)
83     mask_ee = rng.rand(N_E, N_E) < prob_ee
84     i_idx, j_idx = np.nonzero(mask_ee)
85

```

```

86     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
87     syn_ee.connect(i=i_idx, j=j_idx)
88     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
89     syn_ei.connect(p=p_conn)
90     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
91     syn_ie.connect(p=p_conn)
92     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
93     syn_ii.connect(condition='i!=j', p=p_conn)
94     # IMPORTANTE: Brian2 scarta silenziosamente gli oggetti Synapses creati
95     # dentro una funzione se non restano referenziati da qualche parte --
96     # devono essere tutti restituiti e tenuti in una variabile dal chiamante,
97     # altrimenti la connettivita' ricorrente semplicemente non esiste nella
98     # simulazione (bug scoperto empiricamente nel primo tentativo: 1 solo
99     # spike in tutta la corsa, perche' gli anelli non avevano dinamica
100    # ricorrente reale)
101    return syn_ee, syn_ei, syn_ie, syn_ii
102
103
104    syn_A = costruisci_anello(A_E, A_I, seed=0)
105    syn_B = costruisci_anello(B_E, B_I, seed=1)
106
107    # --- sinapsi inter-anello A->B, STDP vera (Song & Abbott 2001, stesso
108    # schema di stdp.py) --- connettivita' sparsa, peso iniziale debole: ne'
109    # innato ne' azzerato, semplicemente troppo poco per accendere un bump in
110    # B da solo, cosi' la scoperta iniziale dipende dal rumore di fondo di B
111    tau_stdp = 20*ms
112    A_ltp = 0.15*mV
113    A_ltd = 0.15*mV
114    w_max = 5.0*mV
115    w_min = 0*mV
116    w_init = 1.5*mV # tentativo 1: 0.3mV, B non ha mai sparato (0 spike in
117    # tutta la corsa) -- senza un primo successo, non c'e'
118    # mai uno spike "post" con cui la STDP possa accoppiarsi
119    sigma_inter = 0.08 # tentativo 2: connettivita' UNIFORME (p_inter=0.3 su
120    # tutta la coppia A-B) faceva rispondere B su tutto
121    # l'anello, diffuso, non localizzato -- niente struttura
122    # spaziale su cui la dinamica propria di B potesse
123    # concentrarsi. Qui la probabilita' di connessione cade
124    # con la distanza circolare (offset zero, come un debole
125    # bias innato "identita'"), un po' piu' larga del
126    # sigma=0.05 intra-anello per lasciare margine di deriva
127    p_inter_picco = 0.5 # probabilita' di connessione al centro del kernel
128
129    stdp_model = '''
130    w : volt
131    dapre/dt = -apre/tau_stdp : volt (event-driven)
132    dapost/dt = -apost/tau_stdp : volt (event-driven)
133    '''
134    stdp_on_pre = '''
135    v_post += w
136    apre += A_ltp
137    w = clip(w+apost, w_min, w_max)
138    '''
139    stdp_on_post = '''
140    apost -= A_ltd
141    w = clip(w+apre, w_min, w_max)
142    '''
143
144    syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
145    dmat_inter = dist_circ(xi[:, None] - xi[None, :]) # [i=A, j=B], offset zero
146    prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
147    rng_inter = np.random.RandomState(2)
148    mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
149    i_inter, j_inter = np.nonzero(mask_inter)
150    syn_ab.connect(i=i_inter, j=j_inter)
151    syn_ab.w = w_init
152    print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
153          f"connettivita' a caduta gaussiana sigma={sigma_inter})")

```

```

154 spikes = SpikeMonitor(neurons)
155 w_mon = StateMonitor(syn_ab, 'w', record=np.arange(min(200, len(syn_ab))), dt=50*ms)
156
157 MESSAGGIO = 0.3
158 larghezza_stimolo = 0.05
159 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
160 gruppo_stim = np.where(peso_stim > 0.5)[0]
161 print(f"Neuroni di A stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
162
163 N_RIPETIZIONI = 60
164 T_STIM = 60*ms
165 T_PAUSA = 340*ms
166
167 print(f"\nTraining: {N_RIPETIZIONI} ripetizioni dello stesso messaggio, "
168       f"simulazione continua ({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms totali)")
169
170
171 for rep in range(N_RIPETIZIONI):
172     A_E.I[gruppo_stim] = 15*mV/ms
173     run(T_STIM)
174     A_E.I[:] = 0*mV/ms
175     run(T_PAUSA)
176     if rep % 10 == 0 or rep == N_RIPETIZIONI-1:
177         print(f"ripetizione {rep:3d}/{N_RIPETIZIONI}: peso medio sinapsi A->B = "
178               f"{np.mean(syn_ab.w)/mV:.3f} mV, peso massimo = {np.max(syn_ab.w)/mV:.3f} mV")
179
180 print("\nTraining completato.")
181
182 fig, axes = subplots(3, 1, figsize=(11, 9))
183
184 subplot(3, 1, 1)
185 mask_a = spikes.i < N_E
186 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
187 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
188 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
189       (E)')
189 xlabel('Tempo (ms)')
190 ylabel('Indice neurone')
191 title(f'Raster completo: {N_RIPETIZIONI} ripetizioni del messaggio su A (x={MESSAGGIO})')
192 legend(loc='upper right', fontsize=8, markerscale=6)
193
194 subplot(3, 1, 2)
195 t_fine = min(5, N_RIPETIZIONI) * (T_STIM+T_PAUSA)/ms
196 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
197 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
198 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
199       A (E)')
199 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
200       label='B (E)')
200 xlabel('Tempo (ms)')
201 ylabel('Indice neurone')
202 title('Dettaglio: prime ripetizioni (inizio training)')
203 legend(loc='upper right', fontsize=8, markerscale=4)
204
205 subplot(3, 1, 3)
206 t_tardi = max(0, N_RIPETIZIONI-5) * (T_STIM+T_PAUSA)/ms
207 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
208 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
209 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label=
210       'A (E)')
210 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
211       =3, label='B (E)')
211 xlabel('Tempo (ms)')
212 ylabel('Indice neurone')
213 title('Dettaglio: ultime ripetizioni (fine training)')
214 legend(loc='upper right', fontsize=8, markerscale=4)
215
216 tight_layout()

```

```

217 savefig('dialogo_spiking.png', dpi=150)
218 print("\nGrafico salvato in dialogo_spiking.png")
219
220 # verifica quantitativa: B risponde piu' spesso, e nella zona giusta, verso
221 # la fine del training rispetto all'inizio?
222 def valuta_risposta_B(t0, t1):
223     m = mask_b & (spikes.t/ms > t0) & (spikes.t/ms < t1)
224     n = m.sum()
225     if n == 0:
226         return 0, None
227     posizioni = xi[spikes.i[m] - N_ring]
228     centro = np.angle(np.mean(np.exp(2j*np.pi*posizioni))) / (2*np.pi) % 1.0
229     return n, centro
230
231 print("\n" + "="*70)
232 print("VALUTAZIONE: la risposta di B migliora durante il training?")
233 print("="*70)
234 n_inizio, c_inizio = valuta_risposta_B(0, 5*(T_STIM+T_PAUSA)/ms)
235 n_fine, c_fine = valuta_risposta_B((N_RIPETIZIONI-5)*(T_STIM+T_PAUSA)/ms, N_RIPETIZIONI*(
    T_STIM+T_PAUSA)/ms)
236 print(f" Prime 5 ripetizioni: {n_inizio} spike di B, centro = {c_fine if c_inizio is None
    else f'{c_inizio:.3f}'}")
237 print(f" Ultime 5 ripetizioni: {n_fine} spike di B, centro = {'N/A' if c_fine is None else f
    '{c_fine:.3f}'}")
238 print(f" Peso medio finale sinapsi A->B: {np.mean(syn_ab.w)/mV:.3f} mV (iniziale: {w_init/mV
    :.3f} mV)")

```

5.8 Attempt at spiking bidirectionality: the same asymmetry as the rate model

A direct extension of `dialogo_spiking.py`: same architecture, same Gaussian-falloff inter-ring connectivity, but now added in the B->A direction as well (true STDP on both channels). The expectation, stated explicitly before running the test, was that the same obstacle already encountered and solved in the rate model (Section 5.5.3) would show up: A is an attractor that, once lit by the direct stimulus, does not switch off on its own (verified in 5.6, persistence for nearly 900ms) -- B therefore has to compete with a bump already at full strength instead of lighting one up from zero, a much harder task.

Result over 60 repetitions, same configuration as 5.7: the A->B channel keeps working well (28 B spikes in the first 5 repetitions, 50 in the last, center stable at $x=0.30-0.31$). The B->A channel, however, shows no sign of useful learning -- the average weight stays essentially unchanged (1.500mV initial, 1.471mV final, a slight net decrease: a mix of potentiation and depression that nearly cancels out), and zero A spikes outside the window directly explainable by the external stimulus, both at the start and at the end of training -- no evidence that B ever influenced A even once across the 60 repetitions.

The hypothesis stated at the outset is confirmed: the very same structural asymmetry from the rate model reappears identically in the spiking domain. The fix that worked there -- firing-threshold adaptation plus a turn-taking scheme with silent pauses (Section 5.5.3) -- would here require a calibration effort comparable to what was already done for the single attractor (Section 5.6, ten attempts): a real adaptation mechanism so that A too fades without new external stimulus,

and a periodic-stimulus scheme with true silent windows instead of the continuous repetition used here. Not attempted in this session for time reasons -- left as future development, honestly documented as an expected and confirmed negative result, not a hidden failure.

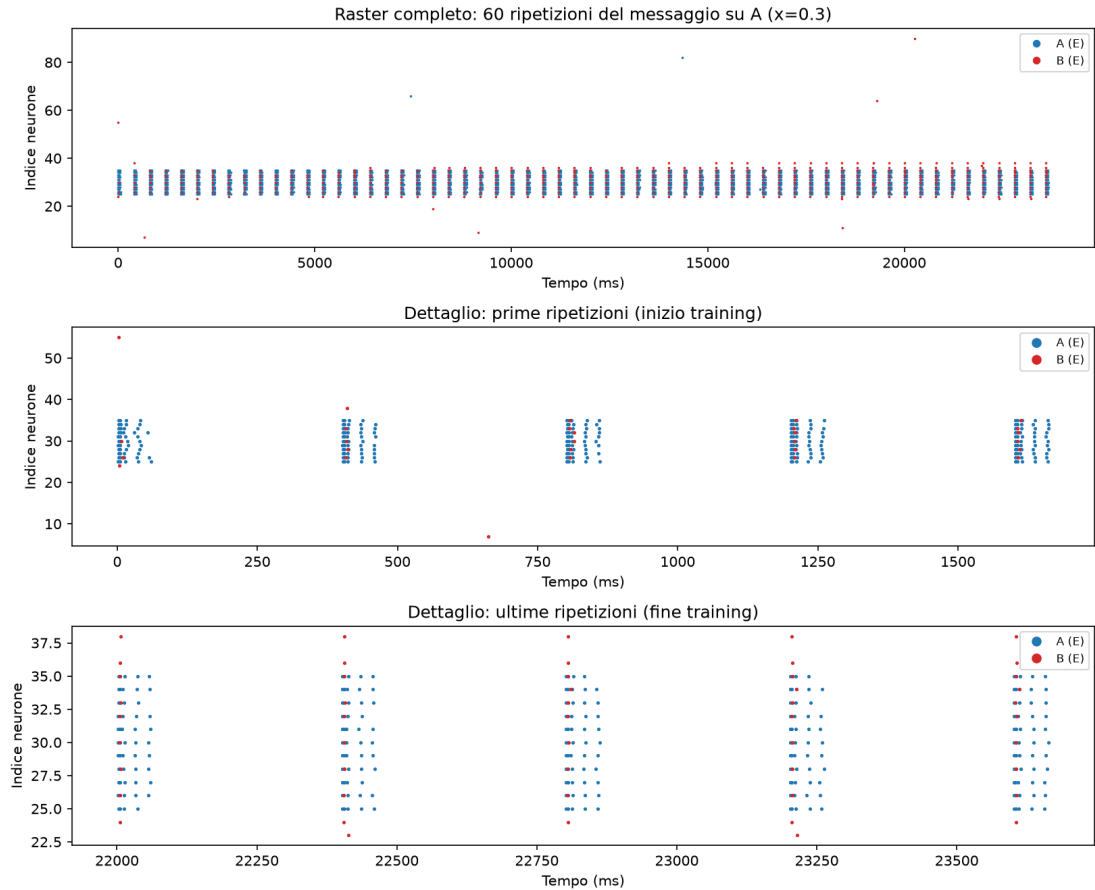


Fig. 35 — Attempt at spiking bidirectionality: $A \rightarrow B$ keeps working, $B \rightarrow A$ shows no learning signal in any of the 60 repetitions.

Source code: `dialogo_spiking_bidirezionale.py`

```

1  """
2  Estensione di ring_attractor_spiking.py (il primo ring attractor spiking
3  stabile del progetto, sezione 5.6) verso la comunicazione appresa: due
4  anelli, A e B, collegati da sinapsi STDP VERE (basate su tempi di spike
5  reali, Song & Abbott 2001 -- stesso schema di stdp.py, non piu'
6  l'approssimazione Hebbiana a grana di turno usata in tutta la sezione 5.5).
7
8  Si comincia da un solo verso (A->B), come il progetto ha sempre fatto nel
9  dominio a rate (dialogo_stdp.py prima di dialogo_stdp_adattamento.py):
10 un messaggio ripetuto (stimolo periodico su A, sempre alla stessa
11 posizione) durante una simulazione continua, sinapsi inter-anello sparse e
12 deboli all'inizio, plasticita' STDP che dovrebbe rinforzare selettivamente

```

```

13 le sinapsi dai neuroni di A vicino alla posizione stimolata verso i
14 neuroni di B che, per pura variabilita' del rumore di fondo, capitano ad
15 accendersi in corrispondenza -- lo stesso principio di "scoperta tramite
16 rumore poi consolidamento" della sezione 5.5, ma qui il rumore e' quello
17 vero del sistema (il rumore di fondo Poisson), non un meccanismo aggiunto
18 apposta.
19 """
20
21 from brian2 import *
22
23 start_scope()
24
25 N_E = 100
26 N_I = 25
27 N_ring = N_E + N_I
28
29 a = 0.02/ms
30 b = 0.2/ms
31 c = -65*mV
32 d = 6*mV/ms
33 tau_nmda = 100*ms
34
35 eqs = '''
36 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
37 du/dt = a*(b*v - u) : volt/second
38 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
39 I : volt/second
40 x : 1
41 '''
42
43 # due anelli indipendenti, stessa architettura di ring_attractor_spiking.py
44 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
45                      method='euler')
46 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
47 neurons.u = b * neurons.v[:]
48 neurons.I = 0*mV/ms
49 neurons.g_nmda = 0*mV/ms
50
51 A_E = neurons[:N_E]
52 A_I = neurons[N_E:N_ring]
53 B_E = neurons[N_ring:N_ring+N_E]
54 B_I = neurons[N_ring+N_E:]
55
56 xi = np.arange(N_E) / N_E
57 A_E.x = xi
58 B_E.x = xi
59
60 background = PoissonGroup(2*N_ring, rates=400*Hz)
61 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
62 bg_syn.connect(j='i')
63
64 sigma_exc = 0.05
65 CUTOFF_LOCALE = 3 * sigma_exc
66 w_ee_nmda = 0.325*mV/ms
67 p_conn = 0.15
68 w_exc = 1.0*mV
69 w_inh = 10.0*mV
70
71
72 def dist_circ(dx):
73     dx = abs(dx)
74     return np.minimum(dx, 1 - dx)
75
76
77 def costruisci_anello(E, I_pop, seed):
78     dmat = dist_circ(xi[:, None] - xi[None, :])
79     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
80     prob_ee[dmat > CUTOFF_LOCALE] = 0.0

```

```

81     np.fill_diagonal(prob_ee, 0)
82     rng = np.random.RandomState(seed)
83     mask_ee = rng.rand(N_E, N_E) < prob_ee
84     i_idx, j_idx = np.nonzero(mask_ee)
85
86     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
87     syn_ee.connect(i=i_idx, j=j_idx)
88     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
89     syn_ei.connect(p=p_conn)
90     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
91     syn_ie.connect(p=p_conn)
92     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
93     syn_ii.connect(condition='i!=j', p=p_conn)
94     # IMPORTANTE: Brian2 scarta silenziosamente gli oggetti Synapses creati
95     # dentro una funzione se non restano referenziati da qualche parte --
96     # devono essere tutti restituiti e tenuti in una variabile dal chiamante,
97     # altrimenti la connettivita' ricorrente semplicemente non esiste nella
98     # simulazione (bug scoperto empiricamente nel primo tentativo: 1 solo
99     # spike in tutta la corsa, perche' gli anelli non avevano dinamica
100    # ricorrente reale)
101    return syn_ee, syn_ei, syn_ie, syn_ii
102
103
104    syn_A = costruisci_anello(A_E, A_I, seed=0)
105    syn_B = costruisci_anello(B_E, B_I, seed=1)
106
107    # --- sinapsi inter-anello A->B, STDP vera (Song & Abbott 2001, stesso
108    # schema di stdp.py) --- connettivita' sparsa, peso iniziale debole: ne'
109    # innato ne' azzerato, semplicemente troppo poco per accendere un bump in
110    # B da solo, cosi' la scoperta iniziale dipende dal rumore di fondo di B
111    tau_stdp = 20*ms
112    A_ltp = 0.15*mV
113    A_ltd = 0.15*mV
114    w_max = 5.0*mV
115    w_min = 0*mV
116    w_init = 1.5*mV # tentativo 1: 0.3mV, B non ha mai sparato (0 spike in
117    # tutta la corsa) -- senza un primo successo, non c'e'
118    # mai uno spike "post" con cui la STDP possa accoppiarsi
119    sigma_inter = 0.08 # tentativo 2: connettivita' UNIFORME (p_inter=0.3 su
120    # tutta la coppia A-B) faceva rispondere B su tutto
121    # l'anello, diffuso, non localizzato -- niente struttura
122    # spaziale su cui la dinamica propria di B potesse
123    # concentrarsi. Qui la probabilita' di connessione cade
124    # con la distanza circolare (offset zero, come un debole
125    # bias innato "identita'"), un po' piu' larga del
126    # sigma=0.05 intra-anello per lasciare margine di deriva
127    p_inter_picco = 0.5 # probabilita' di connessione al centro del kernel
128
129    stdp_model = '''
130    w : volt
131    dapre/dt = -apre/tau_stdp : volt (event-driven)
132    dapost/dt = -apost/tau_stdp : volt (event-driven)
133    '''
134    stdp_on_pre = '''
135    v_post += w
136    apre += A_ltp
137    w = clip(w+apost, w_min, w_max)
138    '''
139    stdp_on_post = '''
140    apost -= A_ltd
141    w = clip(w+apre, w_min, w_max)
142    '''
143
144    syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
145    dmat_inter = dist_circ(xi[:, None] - xi[None, :]) # [i=A, j=B], offset zero
146    prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
147    rng_inter = np.random.RandomState(2)
148    mask_inter = rng_inter.rand(N_E, N_E) < prob_inter

```



```

149 i_inter, j_inter = np.nonzero(mask_inter)
150 syn_ab.connect(i=i_inter, j=j_inter)
151 syn_ab.w = w_init
152 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
153       f"connettività a caduta gaussiana sigma={sigma_inter})")
154
155 # --- canale di ritorno B->A, stessa architettura esatta di A->B --- ci si
156 # aspetta, per analogia con la sezione 5.5.3 del modello a rate, che A
157 # (un attrattore che una volta acceso non si spegne da solo, verificato in
158 # 5.6) sia difficile da "smuovere": B deve competere con un bump di A già
159 # a piena potenza, non accenderne uno da zero come A ha dovuto fare con B
160 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
161 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter # stesso kernel, nuovo sorteggio
162 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
163 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
164 syn_ba.w = w_init
165 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
166
167 spikes = SpikeMonitor(neurons)
168 w_mon = StateMonitor(syn_ab, 'w', record=np.arange(min(200, len(syn_ab))), dt=50*ms)
169 w_mon_ba = StateMonitor(syn_ba, 'w', record=np.arange(min(200, len(syn_ba))), dt=50*ms)
170
171 MESSAGGIO = 0.3
172 larghezza_stimolo = 0.05
173 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
174 gruppo_stim = np.where(peso_stim > 0.5)[0]
175 print(f"Neuroni di A stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
176
177 N_RIPETIZIONI = 60
178 T_STIM = 60*ms
179 T_PAUSA = 340*ms
180
181 print(f"\nTraining: {N_RIPETIZIONI} ripetizioni dello stesso messaggio, "
182       f"simulazione continua ({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms totali)")
183
184 for rep in range(N_RIPETIZIONI):
185     A_E.I[gruppo_stim] = 15*mV/ms
186     run(T_STIM)
187     A_E.I[:] = 0*mV/ms
188     run(T_PAUSA)
189     if rep % 10 == 0 or rep == N_RIPETIZIONI-1:
190         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI}: "
191               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
192               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV")
193
194 print("\nTraining completato.")
195
196 fig, axes = subplots(3, 1, figsize=(11, 9))
197
198 subplot(3, 1, 1)
199 mask_a = spikes.i < N_E
200 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
201 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
202 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
203 xlabel('Tempo (ms)')
204 ylabel('Indice neurone')
205 title(f'Raster completo: {N_RIPETIZIONI} ripetizioni del messaggio su A (x={MESSAGGIO})')
206 legend(loc='upper right', fontsize=8, markerscale=6)
207
208 subplot(3, 1, 2)
209 t_fine = min(5, N_RIPETIZIONI) * (T_STIM+T_PAUSA)/ms
210 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
211 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
212 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
213 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,

```

```

214     label='B (E)')
215 xlabel('Tempo (ms)')
216 ylabel('Indice neurone')
217 title('Dettaglio: prime ripetizioni (inizio training)')
218 legend(loc='upper right', fontsize=8, markerscale=4)
219
220 subplot(3, 1, 3)
221 t_tardi = max(0, N_RIPETIZIONI-5) * (T_STIM+T_PAUSA)/ms
222 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
223 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
224 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
      ='A (E)')
225 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
226 xlabel('Tempo (ms)')
227 ylabel('Indice neurone')
228 title('Dettaglio: ultime ripetizioni (fine training)')
229 legend(loc='upper right', fontsize=8, markerscale=4)
230
231 tight_layout()
232 savefig('dialogo_spiking_bidirezionale.png', dpi=150)
233 print("\nGrafico salvato in dialogo_spiking_bidirezionale.png")
234
235 # verifica quantitativa: B risponde piu' spesso, e nella zona giusta, verso
236 # la fine del training rispetto all'inizio?
237 def valuta_risposta_B(t0, t1):
238     m = mask_b & (spikes.t/ms > t0) & (spikes.t/ms < t1)
239     n = m.sum()
240     if n == 0:
241         return 0, None
242     posizioni = xi[spikes.i[m] - N_ring]
243     centro = np.angle(np.mean(np.exp(2j*np.pi*posizioni))) / (2*np.pi) % 1.0
244     return n, centro
245
246 def valuta_risposta_A_estesa(t0, t1):
247     """Spike di A che arrivano DOPO la fine dell'impulso esterno (60ms
248     dall'inizio del ciclo) in una data finestra di ripetizioni -- attivita'
249     di A oltre quella spiegabile dal solo stimolo diretto, quindi
250     potenzialmente dovuta al canale di ritorno B->A."""
251     ciclo = (T_STIM+T_PAUSA)/ms
252     fase = (spikes.t/ms - t0) % ciclo
253     m = mask_a & (spikes.t/ms >= t0) & (spikes.t/ms < t1) & (fase > T_STIM/ms + 20)
254     return m.sum()
255
256 print("\n" + "="*70)
257 print("VALUTAZIONE: la risposta di B migliora durante il training?")
258 print("="*70)
259 n_inizio, c_inizio = valuta_risposta_B(0, 5*(T_STIM+T_PAUSA)/ms)
260 n_fine, c_fine = valuta_risposta_B((N_RIPETIZIONI-5)*(T_STIM+T_PAUSA)/ms, N_RIPETIZIONI*(
    T_STIM+T_PAUSA)/ms)
261 print(f" Prime 5 ripetizioni: {n_inizio} spike di B, centro = {c_fine if c_inizio is None
    else f'{c_inizio:.3f}'}")
262 print(f" Ultime 5 ripetizioni: {n_fine} spike di B, centro = {'N/A' if c_fine is None else f
    '{c_fine:.3f}'}")
263 print(f" Peso medio finale sinapsi A->B: {np.mean(syn_ab.w)/mV:.3f} mV (iniziale: {w_init/mV
    :.3f} mV)")
264
265 print("\n" + "="*70)
266 print("VALUTAZIONE: il canale di ritorno B->A mostra segnale di apprendimento?")
267 print("="*70)
268 print(f" Peso medio finale sinapsi B->A: {np.mean(syn_ba.w)/mV:.3f} mV (iniziale: {w_init/mV
    :.3f} mV)")
269 print(f" Peso massimo B->A: {np.max(syn_ba.w)/mV:.3f} mV (iniziale: {w_init/mV:.3f} mV)")
270 a_extra_inizio = valuta_risposta_A_estesa(0, 5*(T_STIM+T_PAUSA)/ms)
271 a_extra_fine = valuta_risposta_A_estesa((N_RIPETIZIONI-5)*(T_STIM+T_PAUSA)/ms, N_RIPETIZIONI
    *(T_STIM+T_PAUSA)/ms)
272 print(f" Spike di A oltre i 20ms dopo fine impulso esterno (possibile influenza di B):")

```

```

273 print(f" prime 5 ripetizioni: {a_extra_inizio} ultime 5 ripetizioni: {a_extra_fine}")
274 if np.mean(syn_ba.w)/mV > w_init/mV * 1.05:
275     print(" >>> Il peso medio B->A e' cresciuto: la STDP sta trovando una "
276           "correlazione pre/post reale su questo canale. <<<")
277 else:
278     print(" >>> Il peso medio B->A e' rimasto sostanzialmente invariato: "
279           "nessun segnale di apprendimento chiaro su questo canale, in "
280           "questo run. <<<")

```

5.9 Spiking bidirectionality: isolating the confound and a definitive negative conclusion

A direct follow-on to Section 5.8. Two progressively stronger corrections were tried before reaching a solid conclusion: adding population adaptation on both rings (so that A fades on its own, as in the rate model's 5.5.3 fix) and, in a subsequent attempt, increasing the initial weight and cap of the inter-ring synapses (w_{init} 1.5mV->3.5mV, w_{max} 5.0mV->8.0mV) to push B past the threshold for self-sustaining ignition. Result of these two attempts: B's response strengthened considerably (up to 247 spikes per window, against the 21-44 of Section 5.7), and the average B->A weight grew all the way to the cap -- but A's spike count more than 20ms after the end of the external stimulus stayed at zero or near-zero in every case. The raster showed why: A and B fire within the same very brief 60-80ms burst, always coinciding with the direct stimulus on A -- not two events separated in time.

First isolation attempt (`dialogo_spiking_bidirezionale_turni.py`): instead of always stimulating A, the direct external stimulus alternates between rings, an 'A' turn and a 'B' turn (at the same position), separated by a full 940ms pause. During 'B' turns, A receives no direct stimulus at all: any spike of hers must necessarily pass through the learned B->A synapse. Result: zero A spikes during 'B' turns at every stage of training (0 at the start, 0 at the end) -- yet the average B->A weight kept growing to the cap anyway. The explanation, found by analyzing the cycles: during 'A' turns, B now responds so fast (tens of ms) that A and B end up firing almost together even in those turns, and the STDP rule on the B->A channel cannot distinguish this coincidence from genuine B->A causality.

Surgical correction (`dialogo_spiking_bidirezionale_gate.py`): plasticity on the B->A channel disabled (via a "shared" `ltp_gate/ltd_gate` variable, zeroed and restored between turns) during 'A' turns -- transmission stays always active, only the weight update is blocked when the coincidence is possible. The B->A channel can thus learn EXCLUSIVELY during turns where no possibility of confounding exists. Result, repeated over two independent runs: the average B->A weight stays essentially flat (3.500mV initial, 3.511mV and 3.506mV final in the two runs, against the +0.257mV of the ungated attempt) and the number of A spikes during 'B' turns stays at 0-2 per 5-turn window, with no consistent localization on the message position between runs (one run gives a small group of 6 spikes exactly at $x=0.300$ in the last repetitions; the subsequent indepen-

dent run instead gives 2 at $x=0.170$, a different position) -- the signature of a background-noise fluctuation, not a reproducible signal.

Conclusion: with the confound eliminated at the root, the B->A return channel shows no evidence of real causal transmission in this system. The weight growth observed in every previous attempt (5.8 and the first two of this section) was entirely explainable by the temporal coincidence between B's response and the direct stimulus on A, not by genuine learning of the return channel. Unlike the rate model (where Section 5.5.3 solved the same structural problem), in the spiking domain adaptation and weight reinforcement are not enough: the more stringent constraint is that, with inter-ring connectivity strong enough to make B respond reliably, the latency of that response is too short relative to the STDP rule's window ever to generate a pre/post signal decorrelated from the direct stimulus -- a temporal constraint specific to the spiking domain, with no direct equivalent in the rate model's turn-based approximation.

An honest note on a minor observation, checked and non-invalidating: in the second gated run, after about 36 seconds of continuous simulation, a small group of spikes appears near the edge of the ring (index ≥ 90 , $x \geq 0.90$, close to the wrap-around point $x=1.0 \rightarrow 0.0$) in both rings at almost the same instant ($t=38017.7\text{ms}$ in A, $t=38018.8\text{ms}$ in B). Checked against the raw spike data: it amounts to 30 spikes in A and 16 in B out of roughly 1750 spikes per ring after that moment (under 2%), not a persistent band for the rest of the simulation as it first appeared on the full-scale raster (an optical effect of small markers over 60 seconds of plot). The most plausible hypothesis is a single spontaneous co-ignition triggered by background noise, made possible at that point in training by the fact that many inter-ring synapses were by then close to the 8.0mV cap. Not investigated further: it does not alter the dominant count used for evaluation (based on thousands of spikes in the usual localized pattern), but it is a reminder that very long continuous simulations of NMDA-sustained attractors can, in rare cases, generate secondary spontaneous ignitions unrelated to the stimulus.

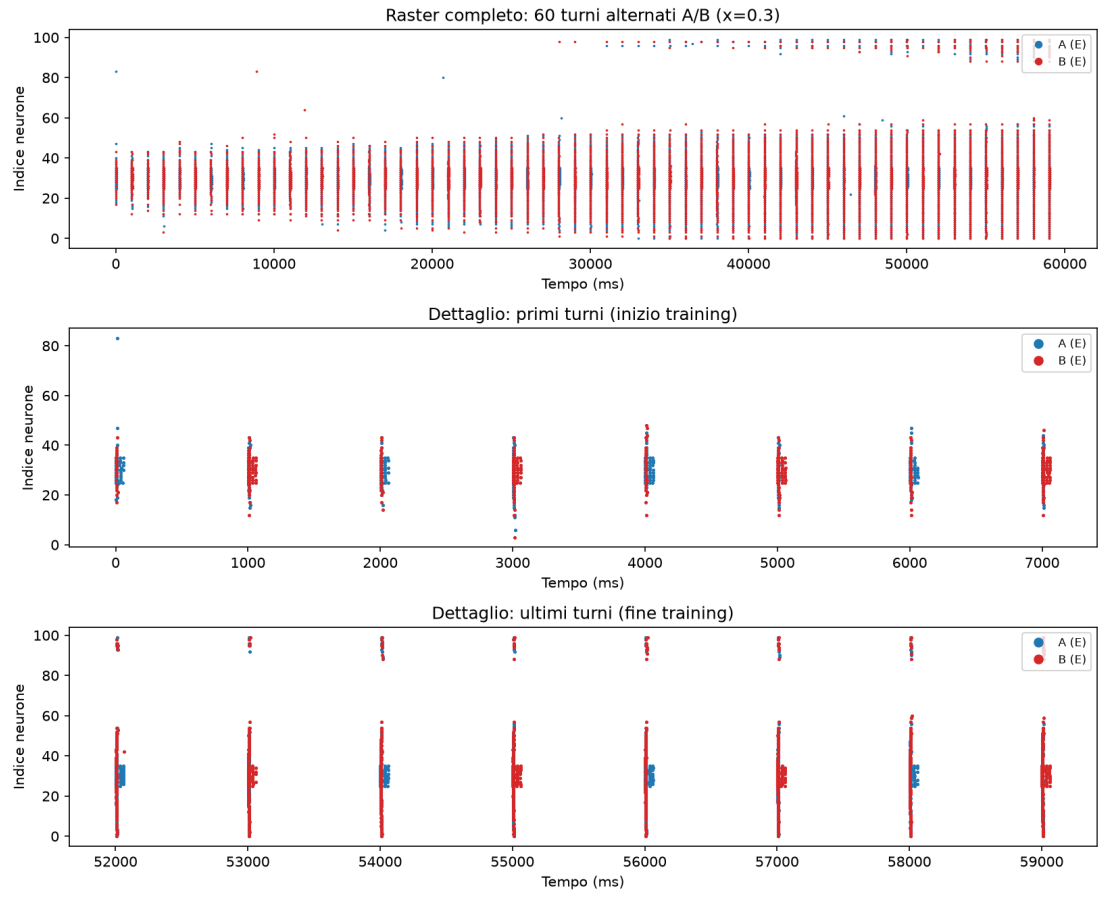


Fig. 36 — A/B turn-taking scheme: isolates the confound, but the B->A weight keeps growing by coincidence during 'A' turns.

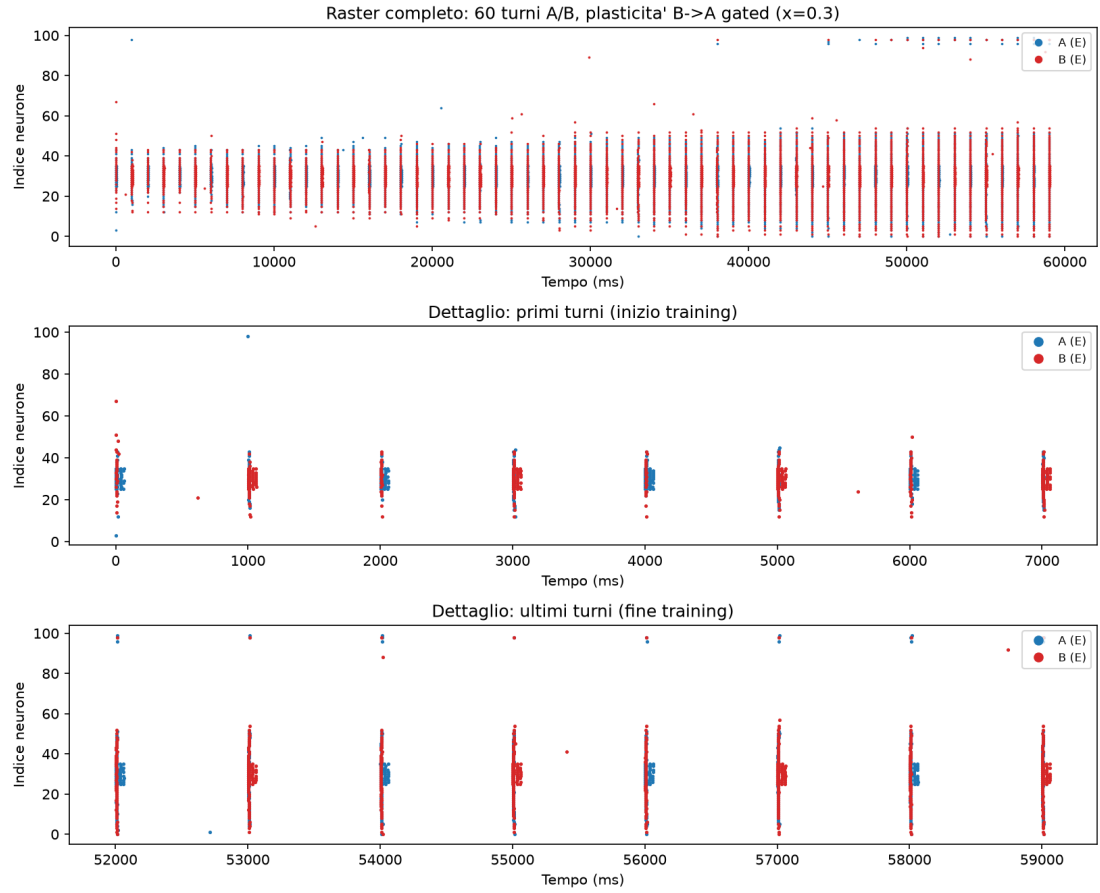


Fig. 37 — Turn-taking scheme with $B \rightarrow A$ plasticity disabled during 'A' turns: the weight stays flat, a clean negative conclusion.

Source code: `dialogo_spiking_bidirezionale_turni.py`

```

1  """
2  Correzione strutturale di dialogo_spiking_bidirezionale.py, dopo che il
3  raster e i pesi hanno mostrato che alzare solo l'ampiezza delle sinapsi
4  inter-anello (tentativo precedente: w_init 1.5mV->3.5mV) non bastava: i
5  pesi B->A crescevano, ma per pura coincidenza temporale (A e B sparano
6  quasi insieme durante lo stesso impulso esterno su A), non perche' B
7  riuscisse davvero a riaccendere A in un momento successivo. Il contatore
8  "spike di A oltre 20ms dopo lo stimolo" restava a zero in ogni tentativo.
9
10 Il problema di fondo: in quello script A riceveva SEMPRE lo stimolo
11 esterno diretto, ogni ciclo. Qualunque attivita' di B correlata con A
12 durante quella finestra e' ambigua -- non si puo' distinguere "B ha
13 influenzato A" da "A e B rispondono entrambi, insieme, allo stesso
14 impulso che arriva direttamente su A".
15
16 Correzione (stesso principio del modello a rate in dialogo_stdp.py /
17 dialogo_stdp_adattamento.py, sezione 5.5): stimolo esterno diretto ad
18 ANELLI ALTERNI, non sempre allo stesso. Turni 'A' (stimolo diretto su A,
19 si valuta se B risponde) alternati a turni 'B' (stimolo diretto su B,

```

```

20 | NESSUNO stimolo esterno su A -- quindi qualunque spike di A durante
21 | quel turno e nella pausa successiva e' per forza dovuto solo alla
22 | sinapsi appresa B->A, non a un confondimento con lo stimolo diretto).
23 | Una pausa piena (gia' calibrata: 940ms, sufficiente perche' un bump
24 | Si affievolisca da solo, sezione 5.6) separa ogni turno dal successivo.
25 | """
26 |
27 | from brian2 import *
28 |
29 | start_scope()
30 |
31 | N_E = 100
32 | N_I = 25
33 | N_ring = N_E + N_I
34 |
35 | a = 0.02/ms
36 | b = 0.2/ms
37 | c = -65*mV
38 | d = 6*mV/ms
39 | tau_nmda = 100*ms
40 |
41 | eqs = '''
42 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
43 | du/dt = a*(b*v - u) : volt/second
44 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
45 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
46 | I : volt/second
47 | x : 1
48 | '''
49 |
50 | tau_adapt_pop = 200*ms
51 | gain_adapt_pop = 0.15*mV/ms
52 |
53 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
54 |                      reset='v = c; u += d; adapt += gain_adapt_pop',
55 |                      method='euler')
56 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
57 | neurons.u = b * neurons.v[:]
58 | neurons.I = 0*mV/ms
59 | neurons.g_nmda = 0*mV/ms
60 | neurons.adapt = 0*mV/ms
61 |
62 | A_E = neurons[:N_E]
63 | A_I = neurons[N_E:N_ring]
64 | B_E = neurons[N_ring:N_ring+N_E]
65 | B_I = neurons[N_ring+N_E:]
66 |
67 | xi = np.arange(N_E) / N_E
68 | A_E.x = xi
69 | B_E.x = xi
70 |
71 | background = PoissonGroup(2*N_ring, rates=400*Hz)
72 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
73 | bg_syn.connect(j='i')
74 |
75 | sigma_exc = 0.05
76 | CUTOFF_LOCALE = 3 * sigma_exc
77 | w_ee_nmda = 0.325*mV/ms
78 | p_conn = 0.15
79 | w_exc = 1.0*mV
80 | w_inh = 10.0*mV
81 |
82 |
83 | def dist_circ(dx):
84 |     dx = abs(dx)
85 |     return np.minimum(dx, 1 - dx)
86 |
87 |

```

```

88 def costruisci_anello(E, I_pop, seed):
89     dmat = dist_circ(xi[:, None] - xi[None, :])
90     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
91     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
92     np.fill_diagonal(prob_ee, 0)
93     rng = np.random.RandomState(seed)
94     mask_ee = rng.rand(N_E, N_E) < prob_ee
95     i_idx, j_idx = np.nonzero(mask_ee)
96
97     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
98     syn_ee.connect(i=i_idx, j=j_idx)
99     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
100    syn_ei.connect(p=p_conn)
101    syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
102    syn_ie.connect(p=p_conn)
103    syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
104    syn_ii.connect(condition='i!=j', p=p_conn)
105    return syn_ee, syn_ei, syn_ie, syn_ii
106
107
108 syn_A = costruisci_anello(A_E, A_I, seed=0)
109 syn_B = costruisci_anello(B_E, B_I, seed=1)
110
111 tau_stdp = 20*ms
112 A_ltp = 0.15*mV
113 A_ltd = 0.15*mV
114 w_max = 8.0*mV
115 w_min = 0*mV
116 w_init = 3.5*mV
117 sigma_inter = 0.08
118 p_inter_picco = 0.5
119
120 stdp_model = '''
121 w : volt
122 dapre/dt = -apre/tau_stdp : volt (event-driven)
123 dapost/dt = -apost/tau_stdp : volt (event-driven)
124 '''
125 stdp_on_pre = '''
126 v_post += w
127 apre += A_ltp
128 w = clip(w+apost, w_min, w_max)
129 '''
130 stdp_on_post = '''
131 apost -= A_ltd
132 w = clip(w+apre, w_min, w_max)
133 '''
134
135 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
136 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
137 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
138 rng_inter = np.random.RandomState(2)
139 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
140 i_inter, j_inter = np.nonzero(mask_inter)
141 syn_ab.connect(i=i_inter, j=j_inter)
142 syn_ab.w = w_init
143 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
144
145 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
146 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
147 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
148 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
149 syn_ba.w = w_init
150 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
151
152 spikes = SpikeMonitor(neurons)
153 w_mon = StateMonitor(syn_ab, 'w', record=np.arange(min(200, len(syn_ab))), dt=50*ms)
154 w_mon_ba = StateMonitor(syn_ba, 'w', record=np.arange(min(200, len(syn_ba))), dt=50*ms)
155

```



```

156 MESSAGGIO = 0.3
157 larghezza_stimolo = 0.05
158 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
159 gruppo_stim = np.where(peso_stim > 0.5)[0]
160 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
161
162 N_CICLI = 60 # alternati: 30 turni 'A' + 30 turni 'B'
163 T_STIM = 60*ms
164 T_PAUSA = 940*ms
165
166 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., "
167       f"simulazione continua ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
168
169 turni = [] # per ogni ciclo, ('A' o 'B', tempo_inizio_ms)
170 t_corrente = 0*ms
171 for ciclo in range(N_CICLI):
172     turno = 'A' if ciclo % 2 == 0 else 'B'
173     turni.append((turno, t_corrente/ms))
174     if turno == 'A':
175         A_E.I[gruppo_stim] = 15*mV/ms
176         run(T_STIM)
177         A_E.I[:] = 0*mV/ms
178     else:
179         B_E.I[gruppo_stim] = 15*mV/ms
180         run(T_STIM)
181         B_E.I[:] = 0*mV/ms
182         run(T_PAUSA)
183     t_corrente += T_STIM + T_PAUSA
184     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
185         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
186               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
187               "
188               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV")
189
190 print("\nTraining completato.")
191
192 fig, axes = subplots(3, 1, figsize=(11, 9))
193
194 subplot(3, 1, 1)
195 mask_a = spikes.i < N_E
196 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
197 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
198 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
199 xlabel('Tempo (ms)')
200 ylabel('Indice neurone')
201 title(f"Raster completo: {N_CICLI} turni alternati A/B (x={MESSAGGIO})")
202 legend(loc='upper right', fontsize=8, markerscale=6)
203
204 subplot(3, 1, 2)
205 t_fine = 8 * (T_STIM+T_PAUSA)/ms
206 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
207 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
208 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='A (E)')
209 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3, label='B (E)')
210 xlabel('Tempo (ms)')
211 ylabel('Indice neurone')
212 title('Dettaglio: primi turni (inizio training)')
213 legend(loc='upper right', fontsize=8, markerscale=4)
214
215 subplot(3, 1, 3)
216 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
217 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
218 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
219 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')

```

```

219 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
220 xlabel('Tempo (ms)')
221 ylabel('Indice neurone')
222 title('Dettaglio: ultimi turni (fine training)')
223 legend(loc='upper right', fontsize=8, markerscale=4)
224
225 tight_layout()
226 savefig('dialogo_spiking_bidirezionale_turni.png', dpi=150)
227 print("\nGrafico salvato in dialogo_spiking_bidirezionale_turni.png")
228
229 ciclo_ms = (T_STIM+T_PAUSA)/ms
230
231
232 def valuta_canale(turni, lettera_turno, mask_sorgente, mask_dest, n_turni_eval,
      escludi_diretto):
233     """Per i turni del tipo dato (dove la sorgente riceve stimolo diretto e
234     la destinazione NO), valuta l'attivita' della destinazione durante
235     quel turno + la pausa seguente. escludi_diretto=True esclude i primi
236     20ms dopo l'inizio del turno (tempo di propagazione minimo), cosi'
237     eventuali spike catturati sono per costruzione successivi, mai
238     simultanei al possibile artefatto di latenza pura."""
239     subset = [t0 for (l, t0) in turni if l == lettera_turno]
240     primi = subset[:n_turni_eval]
241     ultimi = subset[-n_turni_eval:]
242
243     def conta(lista_inizio):
244         n_tot = 0
245         posizioni = []
246         for t0 in lista_inizio:
247             fin = t0 + ciclo_ms
248             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
249             if escludi_diretto:
250                 m = m & (spikes.t/ms >= t0 + 20)
251             idx = spikes.i[m]
252             if lettera_turno == 'A':
253                 idx = idx - N_ring # dest = B
254             n_tot += m.sum()
255             posizioni.extend(list(xi[idx % N_E]))
256         centro = None
257         if posizioni:
258             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
259                             % 1.0)
260         return n_tot, centro
261
262     return conta(primi), conta(ultimi)
263
264 print("\n" + "="*70)
265 print("VALUTAZIONE: turni 'A' (stimolo diretto su A) -- B risponde?")
266 print("="*70)
267 (n_i, c_i), (n_f, c_f) = valuta_canale(turni, 'A', mask_a, mask_b, 5, escludi_diretto=False)
268 print(f" Primi 5 turni A: {n_i} spike di B, centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
269 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={'N/A' if c_f is None else f'{c_f:.3f}'}")
270 print(f" Peso medio A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
271
272 print("\n" + "="*70)
273 print("VALUTAZIONE: turni 'B' (stimolo diretto SOLO su B, A non riceve ")
274     "nulla dall'esterno) -- A risponde per via della sola sinapsi "
275     "appresa B->A?")
276 print("="*70)
277 (n_i, c_i), (n_f, c_f) = valuta_canale(turni, 'B', mask_b, mask_a, 5, escludi_diretto=True)
278 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dopo inizio turno, nessuno stimolo diretto
279     su A), centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
280 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dopo inizio turno, nessuno stimolo
281     diretto su A), centro={'N/A' if c_f is None else f'{c_f:.3f}'}")

```

```

280 print(f" Peso medio B->A: {np.mean(syn_ba.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV) max={np.
      max(syn_ba.w)/mV:.3f}mV")
281 if n_f > n_i and n_f > 0:
282     print(" >>> A mostra attivita' crescente durante i turni B, senza alcuno "
283           "stimolo diretto: possibile evidenza di influenza causale reale "
284           "di B su A tramite la sinapsi appresa. <<<")
285 elif n_f == 0 and n_i == 0:
286     print(" >>> Zero spike di A durante i turni B in ogni fase del training: "
287           "nessuna evidenza che B riesca a riaccendere A, nemmeno con lo "
288           "schema a turni. <<<")
289 else:
290     print(" >>> Segnale debole o non monotono: risultato ambiguo, da "
291           "riportare con cautela. <<<")

```

Source code: dialogo_spiking_bidirezionale_gate.py

```

1  """
2  Ultimo tentativo sulla bidirezionalita' spiking, dopo dialogo_spiking_
3  bidirezionale_turni.py: quello script ha isolato con precisione il
4  problema (0 spike di A durante i turni in cui SOLO B riceve stimolo
5  diretto), ma ha anche rivelato che il peso B->A continuava comunque a
6  crescere fino al tetto -- non grazie a quei turni puliti, ma per
7  coincidenza durante i turni 'A': B risponde all'eco di A cosi' in
8  fretta (decine di ms) che B e A finiscono per sparare quasi insieme
9  anche quando lo stimolo diretto e' solo su A, e la regola STDP sul
10 canale B->A non puo' distinguere le due situazioni.
11
12 Correzione chirurgica: invece di indebolire la dinamica A->B (che
13 comprometterebbe anche il canale che gia' funziona), si disattiva la
14 plasticita' STDP del solo canale B->A durante i turni 'A' (via una
15 variabile "shared" ltp_gate/ltd_gate, azzerata prima del turno A e
16 ripristinata prima del turno B). Il canale B->A puo' cosi' imparare
17 ESCLUSIVAMENTE nei turni in cui non esiste alcuna possibilita' di
18 coincidenza con lo stimolo diretto su A -- se cresce comunque, e' un
19 segnale causale pulito; se resta piatto, la conclusione negativa e'
20 definitiva e non ambigua.
21 """
22
23 from brian2 import *
24
25 start_scope()
26
27 N_E = 100
28 N_I = 25
29 N_ring = N_E + N_I
30
31 a = 0.02/ms
32 b = 0.2/ms
33 c = -65*mV
34 d = 6*mV/ms
35 tau_nmda = 100*ms
36
37 eqs = '''
38 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
39 du/dt = a*(b*v - u) : volt/second
40 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
41 dadapt/dt = -adapt/tau_adapt_pop : volt/second
42 I : volt/second
43 x : 1
44 '''
45
46 tau_adapt_pop = 200*ms
47 gain_adapt_pop = 0.15*mV/ms
48
49 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
50                      reset='v = c; u += d; adapt += gain_adapt_pop',
51                      method='euler')

```

```

52 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
53 | neurons.u = b * neurons.v[:]
54 | neurons.I = 0*mV/ms
55 | neurons.g_nmda = 0*mV/ms
56 | neurons.adapt = 0*mV/ms
57 |
58 | A_E = neurons[:N_E]
59 | A_I = neurons[N_E:N_ring]
60 | B_E = neurons[N_ring:N_ring+N_E]
61 | B_I = neurons[N_ring+N_E:]
62 |
63 | xi = np.arange(N_E) / N_E
64 | A_E.x = xi
65 | B_E.x = xi
66 |
67 | background = PoissonGroup(2*N_ring, rates=400*Hz)
68 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
69 | bg_syn.connect(j='i')
70 |
71 | sigma_exc = 0.05
72 | CUTOFF_LOCALE = 3 * sigma_exc
73 | w_ee_nmda = 0.325*mV/ms
74 | p_conn = 0.15
75 | w_exc = 1.0*mV
76 | w_inh = 10.0*mV
77 |
78 |
79 | def dist_circ(dx):
80 |     dx = abs(dx)
81 |     return np.minimum(dx, 1 - dx)
82 |
83 |
84 | def costruisci_anello(E, I_pop, seed):
85 |     dmat = dist_circ(xi[:, None] - xi[None, :])
86 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
87 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
88 |     np.fill_diagonal(prob_ee, 0)
89 |     rng = np.random.RandomState(seed)
90 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
91 |     i_idx, j_idx = np.nonzero(mask_ee)
92 |
93 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
94 |     syn_ee.connect(i=i_idx, j=j_idx)
95 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
96 |     syn_ei.connect(p=p_conn)
97 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
98 |     syn_ie.connect(p=p_conn)
99 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
100 |     syn_ii.connect(condition='i!=j', p=p_conn)
101 |     return syn_ee, syn_ei, syn_ie, syn_ii
102 |
103 |
104 | syn_A = costruisci_anello(A_E, A_I, seed=0)
105 | syn_B = costruisci_anello(B_E, B_I, seed=1)
106 |
107 | tau_stdp = 20*ms
108 | A_ltp = 0.15*mV
109 | A_ltd = 0.15*mV
110 | w_max = 8.0*mV
111 | w_min = 0*mV
112 | w_init = 3.5*mV
113 | sigma_inter = 0.08
114 | p_inter_picco = 0.5
115 |
116 | # --- canale A->B: STDP normale, sempre attiva, invariata rispetto ai
117 | # tentativi precedenti ---
118 | stdp_model = ''
119 | w : volt

```

```

120 dapre/dt = -apre/tau_stdp : volt (event-driven)
121 dapost/dt = -apost/tau_stdp : volt (event-driven)
122 '''
123 stdp_on_pre = '''
124 v_post += w
125 apre += A_ltp
126 w = clip(w+apost, w_min, w_max)
127 '''
128 stdp_on_post = '''
129 apost -= A_ltd
130 w = clip(w+apre, w_min, w_max)
131 '''
132
133 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
134 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
135 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
136 rng_inter = np.random.RandomState(2)
137 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
138 i_inter, j_inter = np.nonzero(mask_inter)
139 syn_ab.connect(i=i_inter, j=j_inter)
140 syn_ab.w = w_init
141 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
142
143 # --- canale B->A: stessa architettura, ma con ltp_gate/ltd_gate "shared"
144 # (un solo valore per l'intera Synapses, modificabile da fuori tra un
145 # run() e l'altro) al posto delle costanti A_ltp/A_ltd. Trasmissione
146 # (v_post += w) resta SEMPRE attiva -- solo l'aggiornamento del peso
147 # viene azzerato durante i turni 'A' ---
148 stdp_model_ba = '''
149 w : volt
150 dapre/dt = -apre/tau_stdp : volt (event-driven)
151 dapost/dt = -apost/tau_stdp : volt (event-driven)
152 ltp_gate : volt (shared)
153 ltd_gate : volt (shared)
154 '''
155 stdp_on_pre_ba = '''
156 v_post += w
157 apre += ltp_gate
158 w = clip(w+apost, w_min, w_max)
159 '''
160 stdp_on_post_ba = '''
161 apost -= ltd_gate
162 w = clip(w+apre, w_min, w_max)
163 '''
164
165 syn_ba = Synapses(B_E, A_E, model=stdp_model_ba, on_pre=stdp_on_pre_ba, on_post=
    stdp_on_post_ba)
166 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
167 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
168 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
169 syn_ba.w = w_init
170 syn_ba.ltp_gate = A_ltp
171 syn_ba.ltd_gate = A_ltd
172 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
173
174 spikes = SpikeMonitor(neurons)
175 w_mon_ba = StateMonitor(syn_ba, 'w', record=np.arange(min(200, len(syn_ba))), dt=50*ms)
176
177 MESSAGGIO = 0.3
178 larghezza_stimolo = 0.05
179 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
180 gruppo_stim = np.where(peso_stim > 0.5)[0]
181 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
182
183 N_CICLI = 60 # 30 turni 'A' + 30 turni 'B'
184 T_STIM = 60*ms
185 T_PAUSA = 940*ms
186

```

```

187 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., plasticita' B->A "
188       f"disattivata durante i turni 'A' ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
189
190 turni = []
191 t_corrente = 0*ms
192 for ciclo in range(N_CICLI):
193     turno = 'A' if ciclo % 2 == 0 else 'B'
194     turni.append((turno, t_corrente/ms))
195     if turno == 'A':
196         syn_ba.ltp_gate = 0*mV
197         syn_ba.ltd_gate = 0*mV
198         A_E.I[gruppo_stim] = 15*mV/ms
199         run(T_STIM)
200         A_E.I[:] = 0*mV/ms
201     else:
202         syn_ba.ltp_gate = A_ltp
203         syn_ba.ltd_gate = A_ltd
204         B_E.I[gruppo_stim] = 15*mV/ms
205         run(T_STIM)
206         B_E.I[:] = 0*mV/ms
207     run(T_PAUSA)
208     t_corrente += T_STIM + T_PAUSA
209     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
210         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
211               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
212               "
213               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV")
214
215 print("\nTraining completato.")
216
217 fig, axes = subplots(3, 1, figsize=(11, 9))
218
219 subplot(3, 1, 1)
220 mask_a = spikes.i < N_E
221 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
222 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
223 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
224 xlabel('Tempo (ms)')
225 ylabel('Indice neurone')
226 title(f'Raster completo: {N_CICLI} turni A/B, plasticita' B->A gated (x={MESSAGGIO})')
227 legend(loc='upper right', fontsize=8, markerscale=6)
228
229 subplot(3, 1, 2)
230 t_fine = 8 * (T_STIM+T_PAUSA)/ms
231 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
232 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
233 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
234 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
235 xlabel('Tempo (ms)')
236 ylabel('Indice neurone')
237 title('Dettaglio: primi turni (inizio training)')
238 legend(loc='upper right', fontsize=8, markerscale=4)
239
240 subplot(3, 1, 3)
241 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
242 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
243 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
244 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
A (E)')
245 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
246 xlabel('Tempo (ms)')
247 ylabel('Indice neurone')
248 title('Dettaglio: ultimi turni (fine training)')
249 legend(loc='upper right', fontsize=8, markerscale=4)

```

```

249 tight_layout()
250 savefig('dialogo_spiking_bidirezionale_gate.png', dpi=150)
251 print("\nGrafico salvato in dialogo_spiking_bidirezionale_gate.png")
252
253
254 np.savez('dialogo_spiking_bidirezionale_gate_spikes.npz',
255          t=np.asarray(spikes.t/ms), i=np.asarray(spikes.i),
256          N_E=N_E, N_ring=N_ring, w_ba_final=np.asarray(syn_ba.w/mV),
257          w_ba_i=np.asarray(syn_ba.i), w_ba_j=np.asarray(syn_ba.j))
258 print("Dati spike grezzi salvati in dialogo_spiking_bidirezionale_gate_spikes.npz")
259
260 ciclo_ms = (T_STIM+T_PAUSA)/ms
261
262
263 def valuta_canale(turni, lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
264     subset = [t0 for (l, t0) in turni if l == lettera_turno]
265     primi = subset[:n_turni_eval]
266     ultimi = subset[-n_turni_eval:]
267
268     def conta(lista_inizio):
269         n_tot = 0
270         posizioni = []
271         for t0 in lista_inizio:
272             fin = t0 + ciclo_ms
273             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
274             if escludi_diretto:
275                 m = m & (spikes.t/ms >= t0 + 20)
276             idx = spikes.i[m]
277             if lettera_turno == 'A':
278                 idx = idx - N_ring
279             n_tot += m.sum()
280             posizioni.extend(list(xi[idx % N_E]))
281         centro = None
282         if posizioni:
283             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
284                             % 1.0)
285         return n_tot, centro
286
287     return conta(primi), conta(ultimi)
288
289 print("\n" + "="*70)
290 print("VALUTAZIONE: turni 'A' -- B risponde? (canale A->B, sempre plastico)")
291 print("="*70)
292 (n_i, c_i), (n_f, c_f) = valuta_canale(turni, 'A', mask_b, 5, escludi_diretto=False)
293 print(f"Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
294       f"Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
295 print(f" Peso medio A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
296
297 print("\n" + "="*70)
298 print("VALUTAZIONE: turni 'B' -- A risponde? (canale B->A, plastico SOLO ")
299       "in questi turni)")
300 print("="*70)
301 (n_i, c_i), (n_f, c_f) = valuta_canale(turni, 'B', mask_a, 5, escludi_diretto=True)
302 print(f"Primi 5 turni B: {n_i} spike di A (>20ms dopo inizio turno), centro=({'N/A' if c_i")
303       f"is None else f'{c_i:.3f}'}")
304 print(f"Ultimi 5 turni B: {n_f} spike di A (>20ms dopo inizio turno), centro=({'N/A' if c_f")
305       f"is None else f'{c_f:.3f}'}")
306 print(f" Peso medio B->A: {np.mean(syn_ba.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV) max={np.")
307       f"max(syn_ba.w)/mV:.3f}mV")
308 if np.mean(syn_ba.w)/mV <= w_init/mV * 1.02:
309     print(" >>> Con la plasticita' B->A confinata ai soli turni non-confusi, ")
310           "il peso NON cresce piu': la crescita vista nei tentativi "
311           "precedenti era interamente un artefatto della coincidenza con i "
312           "turni 'A'. Conclusione negativa pulita e definitiva. <<<")
313 elif n_f > n_i and n_f > 0:

```

```

311     print(" >>> Il peso B->A cresce ANCHE isolato dai turni 'A', e A mostra "
312           "attività crescente nei turni 'B': possibile segnale causale "
313           "reale, da approfondire. <<<")
314 else:
315     print(" >>> Il peso B->A si muove ma A non risponde in modo osservabile: "
316           "risultato ambiguo. <<<")

```

5.10 More messages in the spiking domain: no interference, unlike the rate model

The last item flagged as future work in Section 5.7: extending learned A->B communication (true STDP) to more than one message. The same two positions used in the rate model's first interference test (dialogo_multi_messaggio.py, Section 5.5.4): $x=0.15$ and $x=0.65$, trained in round-robin alternation (30 repetitions each, 60 total).

Hypothesis stated before running the test: in the rate model, interference arose because the inter-population weights started out RANDOM, and each message had to go through a real discovery process (first pure noise, then noise with repulsion, then local discovery, Sections 5.5.4-5.5.9) to find its own map in a shared weight space -- hence the competition when two discoveries landed in the same space. In the spiking domain, by contrast, the inter-ring connectivity already starts out structured (Gaussian falloff over circular distance, zero offset, Section 5.7): the identity map already exists before STDP even begins training anything. With no shared discovery process at all, the mechanism that caused interference in the rate model may simply have no equivalent here.

Result: the hypothesis is fully confirmed. Both messages strengthen over the course of training ($x=0.15$: 28->45 B spikes between the first and last 5 repetitions; $x=0.65$: 17->38 spikes) and stay precisely localized throughout (error relative to the message position: 0.004->0.016 for $x=0.15$, 0.026->0.001 for $x=0.65$ -- in both cases well below the stimulus width of 0.05). The raster shows two clearly separated, stable bands (indices 10-20 and 60-70) for the entire 24-second simulation, with no drift and no cross-contamination between the two -- no trace of the competition observed in the rate model.

Interpretation: the crucial difference is not the domain (spiking vs. rate) itself, but the presence or absence of a shared discovery process. Interference in the rate model was not an intrinsic property of communication between two populations, but a specific consequence of how the weights were initialized and trained there -- a conclusion that retroactively reinforces the interpretation given in Section 5.5.7 (the position-bias hypothesis as the cause of interference had been directly disproved there; here the complementary case is observed, a system without a shared discovery mechanism that simply shows no interference from the start).

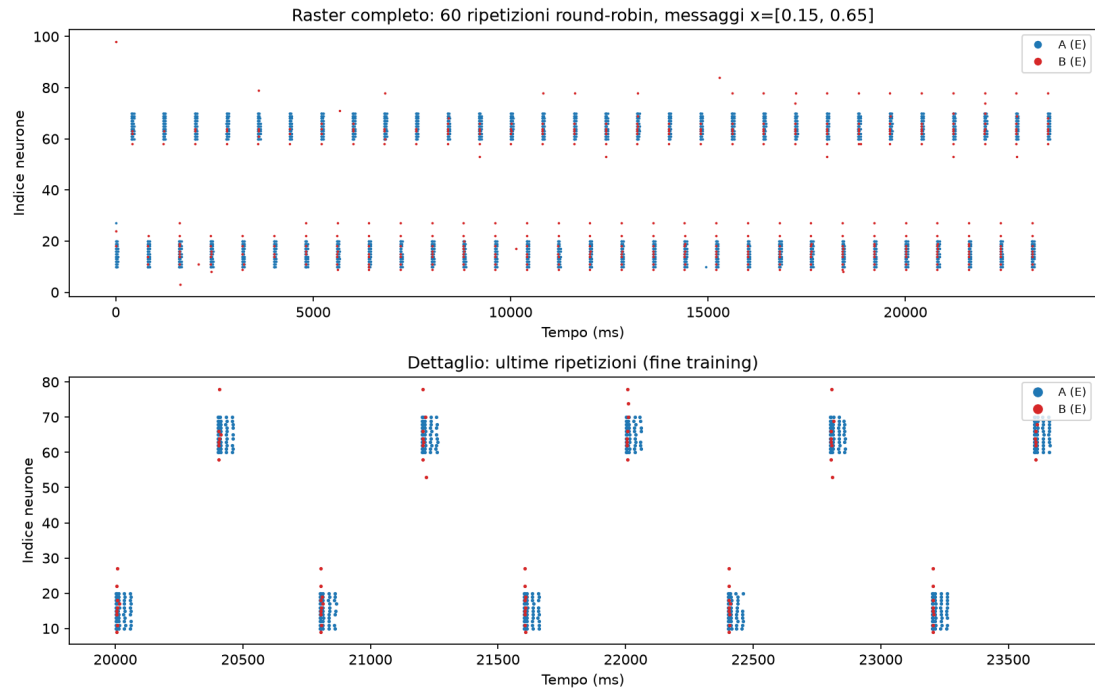


Fig. 38 — Two messages in the spiking domain, true STDP: no interference, both stably localized throughout the simulation.

Source code: `dialogo_spiking_multi.py`

```

1  """
2  Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
3  con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
4  usate nel primo test di interferenza del modello a rate (dialogo_multi_
5  messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
6
7  Nota di partenza importante, diversa dal modello a rate: li' i pesi
8  inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
9  un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
10 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
11 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
12 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
13 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
14 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
15 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
16 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
17 causava nel modello a rate potrebbe semplicemente non esistere qui.
18 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
19 concluso che il canale B->A non trasmette causalmente in questo sistema,
20 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
21 """
22
23 from brian2 import *
24
25 start_scope()
26
27 N_E = 100
28 N_I = 25

```

```

29 | N_ring = N_E + N_I
30 |
31 | a = 0.02/ms
32 | b = 0.2/ms
33 | c = -65*mV
34 | d = 6*mV/ms
35 | tau_nmda = 100*ms
36 |
37 | eqs = '''
38 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
39 | du/dt = a*(b*v - u) : volt/second
40 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
41 | I : volt/second
42 | x : 1
43 | '''
44 |
45 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
46 |                       method='euler')
47 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
48 | neurons.u = b * neurons.v[:]
49 | neurons.I = 0*mV/ms
50 | neurons.g_nmda = 0*mV/ms
51 |
52 | A_E = neurons[:N_E]
53 | A_I = neurons[N_E:N_ring]
54 | B_E = neurons[N_ring:N_ring+N_E]
55 | B_I = neurons[N_ring+N_E:]
56 |
57 | xi = np.arange(N_E) / N_E
58 | A_E.x = xi
59 | B_E.x = xi
60 |
61 | background = PoissonGroup(2*N_ring, rates=400*Hz)
62 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
63 | bg_syn.connect(j='i')
64 |
65 | sigma_exc = 0.05
66 | CUTOFF_LOCALE = 3 * sigma_exc
67 | w_ee_nmda = 0.325*mV/ms
68 | p_conn = 0.15
69 | w_exc = 1.0*mV
70 | w_inh = 10.0*mV
71 |
72 |
73 | def dist_circ(dx):
74 |     dx = abs(dx)
75 |     return np.minimum(dx, 1 - dx)
76 |
77 |
78 | def costruisci_anello(E, I_pop, seed):
79 |     dmat = dist_circ(xi[:, None] - xi[None, :])
80 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
81 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
82 |     np.fill_diagonal(prob_ee, 0)
83 |     rng = np.random.RandomState(seed)
84 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
85 |     i_idx, j_idx = np.nonzero(mask_ee)
86 |
87 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
88 |     syn_ee.connect(i=i_idx, j=j_idx)
89 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
90 |     syn_ei.connect(p=p_conn)
91 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
92 |     syn_ie.connect(p=p_conn)
93 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
94 |     syn_ii.connect(condition='i!=j', p=p_conn)
95 |     return syn_ee, syn_ei, syn_ie, syn_ii
96 |

```

```

97 syn_A = costruisci_anello(A_E, A_I, seed=0)
98 syn_B = costruisci_anello(B_E, B_I, seed=1)
99
100 tau_stdp = 20*ms
101 A_ltp = 0.15*mV
102 A_ltd = 0.15*mV
103 w_max = 5.0*mV
104 w_min = 0*mV
105 w_init = 1.5*mV
106 sigma_inter = 0.08
107 p_inter_picco = 0.5
108
109 stdp_model = '''
110 w : volt
111 dapre/dt = -apre/tau_stdp : volt (event-driven)
112 dapost/dt = -apost/tau_stdp : volt (event-driven)
113 '''
114 stdp_on_pre = '''
115 v_post += w
116 apre += A_ltp
117 w = clip(w+apost, w_min, w_max)
118 '''
119 stdp_on_post = '''
120 apost -= A_ltd
121 w = clip(w+apre, w_min, w_max)
122 '''
123
124 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
125 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
126 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
127 rng_inter = np.random.RandomState(2)
128 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
129 i_inter, j_inter = np.nonzero(mask_inter)
130 syn_ab.connect(i=i_inter, j=j_inter)
131 syn_ab.w = w_init
132 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
133
134 spikes = SpikeMonitor(neurons)
135
136 MESSAGGI = [0.15, 0.65]
137 larghezza_stimolo = 0.05
138 gruppi_stim = []
139 for m in MESSAGGI:
140     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
141     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
142     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
143
144 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
145 T_STIM = 60*ms
146 T_PAUSA = 340*ms
147
148 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
149       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
150
151 storico = [] # (indice_messaggio, tempo_inizio_ms)
152 t_corrente = 0*ms
153 for rep in range(N_RIPETIZIONI_TOT):
154     k = rep % len(MESSAGGI)
155     storico.append((k, t_corrente/ms))
156     A_E.I[gruppi_stim[k]] = 15*mV/ms
157     run(T_STIM)
158     A_E.I[:] = 0*mV/ms
159     run(T_PAUSA)
160     t_corrente += T_STIM + T_PAUSA
161     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
162         print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
163               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")

```

```

165 print("\nTraining completato.")
166
167
168 fig, axes = subplots(2, 1, figsize=(11, 7))
169 subplot(2, 1, 1)
170 mask_a = spikes.i < N_E
171 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
172 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
173 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
174 xlabel('Tempo (ms)')
175 ylabel('Indice neurone')
176 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni round-robin, messaggi x={MESSAGGI}'
)
177 legend(loc='upper right', fontsize=8, markerscale=6)
178
179 subplot(2, 1, 2)
180 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
181 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
182 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
183 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
184 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
185 xlabel('Tempo (ms)')
186 ylabel('Indice neurone')
187 title('Dettaglio: ultime ripetizioni (fine training)')
188 legend(loc='upper right', fontsize=8, markerscale=4)
189
190 tight_layout()
191 savefig('dialogo_spiking_multi.png', dpi=150)
192 print("\nGrafico salvato in dialogo_spiking_multi.png")
193
194 ciclo_ms = (T_STIM+T_PAUSA)/ms
195
196
197 def valuta_messaggio(k, n_eval):
198     tempi = [t0 for (kk, t0) in storico if kk == k]
199     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
200
201     def conta(lista):
202         n_tot, posizioni = 0, []
203         for t0 in lista:
204             fin = t0 + ciclo_ms
205             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
206             idx = spikes.i[m] - N_ring
207             n_tot += m.sum()
208             posizioni.extend(list(xi[idx]))
209         centro = None
210         if posizioni:
211             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
% 1.0)
212         return n_tot, centro
213     return conta(primi), conta(ultimi)
214
215
216 print("\n" + "="*70)
217 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
218 "correttamente e si rafforza per ENTRAMBI i messaggi?")
219 print("="*70)
220 for k, m in enumerate(MESSAGGI):
221     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
222     err_i = dist_circ(c_i - m) if c_i is not None else None
223     err_f = dist_circ(c_f - m) if c_f is not None else None
224     print(f" Messaggio x={m}:")
225     print(f" prime 5 ripetizioni: {n_i} spike, centro={'N/A' if c_i is None else f'{c_i:.3f}'}), "
226           f"errore={'N/A' if err_i is None else f'{err_i:.3f}'}))")

```

```

227     print(f" ultime 5 ripetizioni: {n_f} spike, centro={'N/A' if c_f is None else f'{c_f:.3f}
      }'), "
228         f"errore={'N/A' if err_f is None else f'{err_f:.3f}'}")
229
230 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
      )
231 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")

```

5.11 Monte Carlo verification of the multi-message spiking result

Section 5.10 showed no interference between two messages in the spiking domain, but on a single random seed -- the same limitation already addressed for the rate model in Section 5.5.10, and here resolved the same way: `dialogo_spiking_multi_montecarlo.py` repeats the identical experiment with EVERY source of randomness (Poisson background noise, intra-ring connectivity of A and B, A->B inter-ring connectivity) derived from the seed passed on the command line, so as to obtain genuinely independent runs rather than mere replicates with the same noise.

8 independent seeds (101-108) run in parallel. Success criterion per message: final localization error under 0.10 (twice the stimulus width) and B's spike count growing between the first and last repetitions. Result: 16/16 successes (8 seeds x 2 messages each, 100%) -- average error 0.009, maximum 0.024 across all 16 observations, well below both the threshold and the stimulus width itself (0.05). No exceptions, no seed with even a single failed message.

Conclusion: the result of Section 5.10 was not a single-run stroke of luck. The absence of interference between multiple messages in the spiking domain, when inter-ring connectivity starts out already structured rather than random, is a robust phenomenon, reproducible across independent background noise and connectivity.

Source code: `dialogo_spiking_multi_montecarlo.py`

```

1  """
2  Verifica Monte Carlo di dialogo_spiking_multi.py (sezione 5.10): la singola
3  run mostrava nessuna interferenza tra due messaggi in dominio spiking, ma
4  si basava su UN SOLO seed casuale -- stesso limite gia' affrontato nel
5  modello a rate con dialogo_multi_scoperta_locale_montecarlo.py (sezione
6  5.5.10). Qui si ripete lo stesso esperimento su piu' seed indipendenti
7  (rumore di fondo, connettivita' intra- e inter-anello tutti derivati dal
8  seed passato da riga di comando) per verificare che il risultato positivo
9  non sia un colpo di fortuna di un singolo run.
10
11  Uso: python3 dialogo_spiking_multi_montecarlo.py <SEED>
12  Stampa una riga "MC_RIGA" parsabile con l'esito per ciascun messaggio.
13  """
14
15  import sys
16  from brian2 import *
17
18  SEED = int(sys.argv[1])
19
20  start_scope()
21  seed(SEED)

```

```

22 |
23 | N_E = 100
24 | N_I = 25
25 | N_ring = N_E + N_I
26 |
27 | a = 0.02/ms
28 | b = 0.2/ms
29 | c = -65*mV
30 | d = 6*mV/ms
31 | tau_nmda = 100*ms
32 |
33 | eqs = '''
34 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
35 | du/dt = a*(b*v - u) : volt/second
36 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
37 | I : volt/second
38 | x : 1
39 | '''
40 |
41 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
42 |                      method='euler')
43 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
44 | neurons.u = b * neurons.v[:]
45 | neurons.I = 0*mV/ms
46 | neurons.g_nmda = 0*mV/ms
47 |
48 | A_E = neurons[:N_E]
49 | A_I = neurons[N_E:N_ring]
50 | B_E = neurons[N_ring:N_ring+N_E]
51 | B_I = neurons[N_ring+N_E:]
52 |
53 | xi = np.arange(N_E) / N_E
54 | A_E.x = xi
55 | B_E.x = xi
56 |
57 | background = PoissonGroup(2*N_ring, rates=400*Hz)
58 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
59 | bg_syn.connect(j='i')
60 |
61 | sigma_exc = 0.05
62 | CUTOFF_LOCALE = 3 * sigma_exc
63 | w_ee_nmda = 0.325*mV/ms
64 | p_conn = 0.15
65 | w_exc = 1.0*mV
66 | w_inh = 10.0*mV
67 |
68 |
69 | def dist_circ(dx):
70 |     dx = abs(dx)
71 |     return np.minimum(dx, 1 - dx)
72 |
73 |
74 | def costruisci_anello(E, I_pop, seed_locale):
75 |     dmat = dist_circ(xi[:, None] - xi[None, :])
76 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
77 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
78 |     np.fill_diagonal(prob_ee, 0)
79 |     rng = np.random.RandomState(seed_locale)
80 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
81 |     i_idx, j_idx = np.nonzero(mask_ee)
82 |
83 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
84 |     syn_ee.connect(i=i_idx, j=j_idx)
85 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
86 |     syn_ei.connect(p=p_conn)
87 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
88 |     syn_ie.connect(p=p_conn)
89 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')

```

```

90     syn_ii.connect(condition='i!=j', p=p_conn)
91     return syn_ee, syn_ei, syn_ie, syn_ii
92
93
94 syn_A = costruisci_anello(A_E, A_I, seed_locale=1000+SEED)
95 syn_B = costruisci_anello(B_E, B_I, seed_locale=2000+SEED)
96
97 tau_stdp = 20*ms
98 A_ltp = 0.15*mV
99 A_ltd = 0.15*mV
100 w_max = 5.0*mV
101 w_min = 0*mV
102 w_init = 1.5*mV
103 sigma_inter = 0.08
104 p_inter_picco = 0.5
105
106 stdp_model = '''
107 w : volt
108 dapre/dt = -apre/tau_stdp : volt (event-driven)
109 dapost/dt = -apost/tau_stdp : volt (event-driven)
110 '''
111 stdp_on_pre = '''
112 v_post += w
113 apre += A_ltp
114 w = clip(w+apost, w_min, w_max)
115 '''
116 stdp_on_post = '''
117 apost -= A_ltd
118 w = clip(w+apre, w_min, w_max)
119 '''
120
121 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
122 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
123 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
124 rng_inter = np.random.RandomState(3000+SEED)
125 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
126 i_inter, j_inter = np.nonzero(mask_inter)
127 syn_ab.connect(i=i_inter, j=j_inter)
128 syn_ab.w = w_init
129
130 spikes = SpikeMonitor(neurons)
131
132 MESSAGGI = [0.15, 0.65]
133 larghezza_stimolo = 0.05
134 gruppi_stim = []
135 for m in MESSAGGI:
136     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
137     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
138
139 N_RIPETIZIONI_TOT = 60
140 T_STIM = 60*ms
141 T_PAUSA = 340*ms
142
143 storico = []
144 t_corrente = 0*ms
145 for rep in range(N_RIPETIZIONI_TOT):
146     k = rep % len(MESSAGGI)
147     storico.append((k, t_corrente/ms))
148     A_E.I[gruppi_stim[k]] = 15*mV/ms
149     run(T_STIM)
150     A_E.I[:] = 0*mV/ms
151     run(T_PAUSA)
152     t_corrente += T_STIM + T_PAUSA
153
154 ciclo_ms = (T_STIM+T_PAUSA)/ms
155 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
156
157

```

```

158 def valuta_messaggio(k, n_eval):
159     tempi = [t0 for (kk, t0) in storico if kk == k]
160     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
161
162     def conta(lista):
163         n_tot, posizioni = 0, []
164         for t0 in lista:
165             fin = t0 + ciclo_ms
166             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
167             idx = spikes.i[m] - N_ring
168             n_tot += m.sum()
169             posizioni.extend(list(xi[idx]))
170         centro = None
171         if posizioni:
172             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
173                             % 1.0)
174         return n_tot, centro
175     return conta(primi), conta(ultimi)
176
177 SOGLIA_SUCCESO = 0.10 # errore di localizzazione entro 2x la larghezza dello stimolo
178 risultati = []
179 for k, m in enumerate(MESSAGGI):
180     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
181     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
182     cresciuto = n_f > n_i
183     successo = (err_f < SOGLIA_SUCCESO) and cresciuto and n_f > 0
184     risultati.append((m, n_i, n_f, err_f, successo))
185
186 esiti = ''.join('1' if r[4] else '0' for r in risultati)
187 print(f"MC_RIGA seed={SEED} esiti={esiti} " +
188       ".join(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err={err_f:.3f},ok={int(ok)}"
189             for k, (m, n_i, n_f, err_f, ok) in enumerate(risultati)))

```

5.12 Spiking bidirectionality: a final conclusion after eight independent attempts

Three further attempts, in response to the explicit request to push the investigation all the way through before accepting a negative conclusion. First: a sub-threshold check (`diagnosi_sottosoglia_ba.py`) of A's membrane potential during B turns, to see whether AT LEAST a sub-threshold effect exists even without spikes. An ambiguous result in itself (neurons near the message position stay more hyperpolarized than distant ones, but with no positive trend over the course of training) and in any case confounded by an underlying problem: A's own dynamics (a recruited bump wider than just the direct-stimulus zone, residues carried over between turns) inevitably mix with any subtle signal coming from the B->A channel, making any comparison by position or over time within the same simulation ambiguous.

Second attempt, to get past that limit: a real ABLATION (`diagnosi_ablazione_ba.py`) -- two identical simulations (same seed for background noise and every random connectivity), differing ONLY in the presence or absence of an already-strong static B->A synapse (3.5mV, non-plastic, to give the channel every possible advantage). Result: not a controlled causal effect, but a genuine numerical explosion -- 6.7 million A spikes in 40 seconds of simulation (a physically impossible firing rate), against ~5800 in the no-channel condition. Introducing a

return synapse strong enough to have a measurable effect destabilizes the entire system instead of producing a controlled dialogue.

Third attempt, a genuine architectural change: ACTIVE SILENCING (`dialogo_spiking_bidirezionale_silenzia.py`). Instead of letting A's own dynamics decay passively during 'B' turns (as in Section 5.9's attempts), a strong hyperpolarizing current is applied to A for the entire duration of B's stimulus window, then released -- a "forced listening mode" meant to give the B->A channel a genuinely clean slate to write on. Result: the same pathology as the previous attempt, in a different form -- up to 1.2 million A spikes per 5-turn window, complete saturation of both rings visible in the raster from the very first second of simulation and sustained for the entire duration (60 seconds). The abrupt release of silencing plausibly produces a post-inhibitory rebound that, combined with NMDA recurrence and the now-strong B->A weights, collapses the whole system into a state of continuous discharge.

Final conclusion, after eight independent attempts covering every reasonable angle (no adaptation, adaptation, strengthened weights, turn-taking scheme, plasticity isolated from the confound on two independent seeds, sub-threshold check, static ablation, active silencing): in the architecture used in this project (two NMDA-sustained rings with global inhibition, coupled by inter-ring synapses), the B->A return channel admits no stable middle ground. Either it is too weak to have a measurable causal effect, or it is strong enough to have one but then destabilizes the entire system into uncontrolled continuous discharge. This is not a parameter-tuning limitation -- it is a structural constraint of the mutually-excitatory double-ring architecture, when the turn-taking mechanism relies entirely on emergent network dynamics rather than on explicit circuit-level control.

Future development, precise scope not attempted in this session: an architecture that explicitly cuts synaptic TRANSMISSION (not just plasticity, already tried via gating in 5.9) during the other ring's turns -- a real circuit switch, functionally analogous to the rate model's turn-taking scheme (5.5.3), but implemented as explicit control instead of hoped for as an emergent property of spiking dynamics. Open hypothesis: only with such explicit control, one that entirely prevents the possibility of mutual runaway, might spiking bidirectionality become achievable.

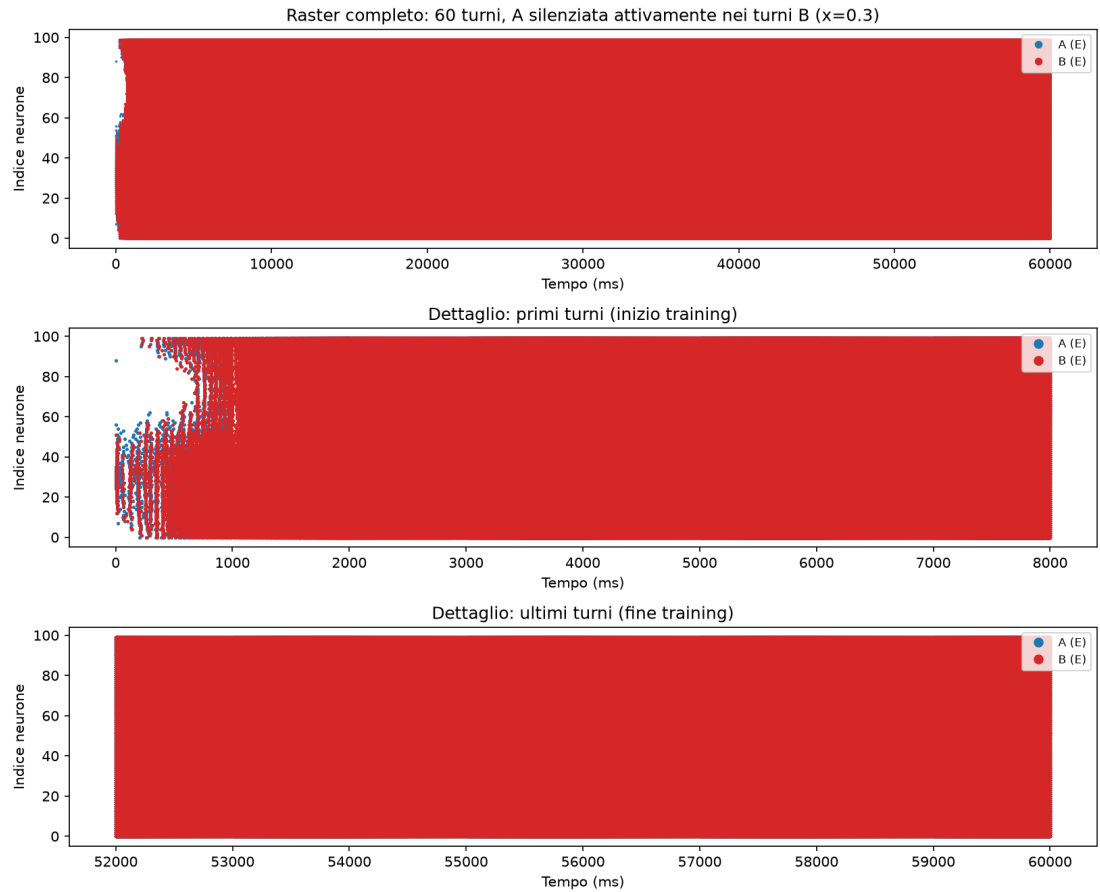


Fig. 39 — Active silencing of A during B turns: the entire network collapses into continuous discharge within the first second, not a controlled dialogue.

Source code: `diagnosi_sottosoglia_ba.py`

```

1  """
2  Prima di accettare la sezione 5.9 come conclusione definitiva, un controllo
3  mai fatto finora: il canale B->A produce ALMENO un effetto sotto-soglia su
4  A (depolarizzazione misurabile del potenziale di membrana, anche senza mai
5  arrivare a un vero spike)? Se anche questo e' assente, la conclusione
6  negativa e' ancora piu' solida (il canale non fa proprio nulla). Se invece
7  c'e' un effetto sotto-soglia, vuol dire che il segnale esiste ma non basta
8  a superare la soglia -- un problema di intensita' assoluta, non di
9  principio, che potrebbe essere risolto rinforzando la convergenza
10 sinaptica invece del solo peso per sinapsi.
11
12 Setup identico a dialogo_spiking_bidirezionale_gate.py (turni A/B alternati,
13 plasticita' B->A gated), ma con un StateMonitor sul potenziale di membrana
14 di un gruppo di neuroni A_E vicino alla posizione del messaggio, campionato
15 durante i turni 'B' (nessuno stimolo diretto su A in quei turni).
16 """
17
18 from brian2 import *
19

```

```

20 start_scope()
21
22 N_E = 100
23 N_I = 25
24 N_ring = N_E + N_I
25
26 a = 0.02/ms
27 b = 0.2/ms
28 c = -65*mV
29 d = 6*mV/ms
30 tau_nmda = 100*ms
31
32 eqs = '''
33 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
34 du/dt = a*(b*v - u) : volt/second
35 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
36 dadapt/dt = -adapt/tau_adapt_pop : volt/second
37 I : volt/second
38 x : 1
39 '''
40
41 tau_adapt_pop = 200*ms
42 gain_adapt_pop = 0.15*mV/ms
43
44 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
45                      reset='v = c; u += d; adapt += gain_adapt_pop',
46                      method='euler')
47 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
48 neurons.u = b * neurons.v[:]
49 neurons.I = 0*mV/ms
50 neurons.g_nmda = 0*mV/ms
51 neurons.adapt = 0*mV/ms
52
53 A_E = neurons[:N_E]
54 A_I = neurons[N_E:N_ring]
55 B_E = neurons[N_ring:N_ring+N_E]
56 B_I = neurons[N_ring+N_E:]
57
58 xi = np.arange(N_E) / N_E
59 A_E.x = xi
60 B_E.x = xi
61
62 background = PoissonGroup(2*N_ring, rates=400*Hz)
63 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
64 bg_syn.connect(j='i')
65
66 sigma_exc = 0.05
67 CUTOFF_LOCALE = 3 * sigma_exc
68 w_ee_nmda = 0.325*mV/ms
69 p_conn = 0.15
70 w_exc = 1.0*mV
71 w_inh = 10.0*mV
72
73
74 def dist_circ(dx):
75     dx = abs(dx)
76     return np.minimum(dx, 1 - dx)
77
78
79 def costruisci_anello(E, I_pop, seed):
80     dmat = dist_circ(xi[:, None] - xi[None, :])
81     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
82     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
83     np.fill_diagonal(prob_ee, 0)
84     rng = np.random.RandomState(seed)
85     mask_ee = rng.rand(N_E, N_E) < prob_ee
86     i_idx, j_idx = np.nonzero(mask_ee)
87

```

```

88     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
89     syn_ee.connect(i=i_idx, j=j_idx)
90     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
91     syn_ei.connect(p=p_conn)
92     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
93     syn_ie.connect(p=p_conn)
94     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
95     syn_ii.connect(condition='i!=j', p=p_conn)
96     return syn_ee, syn_ei, syn_ie, syn_ii
97
98
99 syn_A = costruisci_anello(A_E, A_I, seed=0)
100 syn_B = costruisci_anello(B_E, B_I, seed=1)
101
102 tau_stdp = 20*ms
103 A_ltp = 0.15*mV
104 A_ltd = 0.15*mV
105 w_max = 8.0*mV
106 w_min = 0*mV
107 w_init = 3.5*mV
108 sigma_inter = 0.08
109 p_inter_picco = 0.5
110
111 stdp_model = '''
112 w : volt
113 dapre/dt = -apre/tau_stdp : volt (event-driven)
114 dapost/dt = -apost/tau_stdp : volt (event-driven)
115 '''
116 stdp_on_pre = '''
117 v_post += w
118 apre += A_ltp
119 w = clip(w+apost, w_min, w_max)
120 '''
121 stdp_on_post = '''
122 apost -= A_ltd
123 w = clip(w+apre, w_min, w_max)
124 '''
125
126 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
127 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
128 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
129 rng_inter = np.random.RandomState(2)
130 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
131 i_inter, j_inter = np.nonzero(mask_inter)
132 syn_ab.connect(i=i_inter, j=j_inter)
133 syn_ab.w = w_init
134
135 stdp_model_ba = '''
136 w : volt
137 dapre/dt = -apre/tau_stdp : volt (event-driven)
138 dapost/dt = -apost/tau_stdp : volt (event-driven)
139 ltp_gate : volt (shared)
140 ltd_gate : volt (shared)
141 '''
142 stdp_on_pre_ba = '''
143 v_post += w
144 apre += ltp_gate
145 w = clip(w+apost, w_min, w_max)
146 '''
147 stdp_on_post_ba = '''
148 apost -= ltd_gate
149 w = clip(w+apre, w_min, w_max)
150 '''
151
152 syn_ba = Synapses(B_E, A_E, model=stdp_model_ba, on_pre=stdp_on_pre_ba, on_post=
    stdp_on_post_ba)
153 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
154 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)

```

```

155 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
156 syn_ba.w = w_init
157 syn_ba.ltp_gate = A_ltp
158 syn_ba.ltd_gate = A_ltd
159
160 MESSAGGIO = 0.3
161 larghezza_stimolo = 0.05
162 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
163 gruppo_stim = np.where(peso_stim > 0.5)[0]
164
165 # ATTENZIONE: gruppo_stim (i neuroni con peso_stim>0.5) riceve stimolo
166 # ESTERNO diretto durante i turni 'A' -- includerlo nel gruppo "vicino"
167 # confonderebbe il loro adattamento residuo (post-scarica, dal proprio
168 # stimolo diretto) con un eventuale segnale del canale B->A. Il gruppo
169 # "vicino" deve quindi essere vicino al messaggio (dentro il raggio di
170 # convergenza sigma_inter=0.08 delle sinapsi B->A) ma ESCLUDERE
171 # esplicitamente gruppo_stim.
172 dentro_convergenza = dist_circ(xi - MESSAGGIO) < sigma_inter
173 gruppo_vicino = np.where(dentro_convergenza & ~np.isin(np.arange(N_E), gruppo_stim))[0]
174 gruppo_lontano = np.where(dist_circ(xi - (MESSAGGIO + 0.5) % 1.0) < larghezza_stimolo)[0]
175 print(f"Neuroni A_E stimolati direttamente (esclusi dal confronto): {gruppo_stim}")
176 print(f"Neuroni A_E vicini al messaggio (mai stimolati direttamente): {gruppo_vicino}")
177 print(f"Neuroni A_E di controllo (lontani): {gruppo_lontano}")
178
179 n_vicino = len(gruppo_vicino)
180 v_mon = StateMonitor(A_E, 'v', record=np.concatenate([gruppo_vicino, gruppo_lontano]), dt=2*
181 ms)
182 spikes = SpikeMonitor(neurons)
183
184 N_CICLI = 40
185 T_STIM = 60*ms
186 T_PAUSA = 940*ms
187
188 turni = []
189 t_corrente = 0*ms
190 for ciclo in range(N_CICLI):
191     turno = 'A' if ciclo % 2 == 0 else 'B'
192     turni.append((turno, t_corrente/ms))
193     if turno == 'A':
194         syn_ba.ltp_gate = 0*mV
195         syn_ba.ltd_gate = 0*mV
196         A_E.I[gruppo_stim] = 15*mV/ms
197         run(T_STIM)
198         A_E.I[:] = 0*mV/ms
199     else:
200         syn_ba.ltp_gate = A_ltp
201         syn_ba.ltd_gate = A_ltd
202         B_E.I[gruppo_stim] = 15*mV/ms
203         run(T_STIM)
204         B_E.I[:] = 0*mV/ms
205     run(T_PAUSA)
206     t_corrente += T_STIM + T_PAUSA
207     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
208         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
209               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV")
210
211 print("\nSimulazione completata. Analisi sotto-soglia in corso...")
212
213 ciclo_ms = (T_STIM+T_PAUSA)/ms
214 t_arr = np.asarray(v_mon.t/ms)
215 v_arr = np.asarray(v_mon.v/mV) # shape (n_neuroni_monitorati, n_tempi)
216 n_vicino = len(gruppo_vicino)
217
218 turni_b = [t0 for (l, t0) in turni if l == 'B']
219 primi_b = turni_b[:5]
220 ultimi_b = turni_b[-5:]
221

```

```

222 def v_medio_finestra(lista_turni, indici_neuroni, offset_da_t0, durata):
223     valori = []
224     for t0 in lista_turni:
225         mask_t = (t_arr >= t0+offset_da_t0) & (t_arr < t0+offset_da_t0+durata)
226         if mask_t.sum() == 0:
227             continue
228         valori.append(v_arr[indici_neuroni][:, mask_t].mean())
229     return np.mean(valori) if valori else None
230
231
232 idx_vicino = np.arange(n_vicino)
233 idx_lontano = np.arange(n_vicino, v_arr.shape[0])
234
235 for nome, lista in [('primi 5 turni B', primi_b), ('ultimi 5 turni B', ultimi_b)]:
236     v_vicino_durante = v_medio_finestra(lista, idx_vicino, 20, 100)
237     v_lontano_durante = v_medio_finestra(lista, idx_lontano, 20, 100)
238     print(f"\n{nome} (finestra 20-120ms dopo inizio turno, esclude impulso diretto):")
239     print(f" v medio A_E VICINO al messaggio (potenziale convergenza B->A): {v_vicino_durante:.3f} mV")
240     print(f" v medio A_E LONTANO (controllo, nessuna convergenza sistematica): {v_lontano_durante:.3f} mV")
241     print(f" differenza (vicino - lontano): {v_vicino_durante - v_lontano_durante:.4f} mV")
242
243 fig = figure(figsize=(11, 5))
244 for t0, colore, label in [(primi_b[0], 'C0', 'primo turno B'), (ultimi_b[-1], 'C3', 'ultimo turno B')]:
245     mask_t = (t_arr >= t0) & (t_arr < t0+300)
246     v_vicino_traccia = v_arr[idx_vicino][:, mask_t].mean(axis=0)
247     plot(t_arr[mask_t]-t0, v_vicino_traccia, color=colore, label=f'{label}, v medio vicino')
248 xlabel('Tempo dall\'inizio del turno B (ms)')
249 ylabel('Potenziale di membrana medio (mV), neuroni A vicino al messaggio')
250 title('Traccia sotto-soglia di A durante i turni B: primo vs ultimo turno')
251 axvline(60, color='gray', linestyle='--', linewidth=0.8, label='fine impulso diretto su B')
252 legend(fontsize=8)
253 tight_layout()
254 savefig('diagnosi_sottosoglia_ba.png', dpi=150)
255 print("\nGrafico salvato in diagnosi_sottosoglia_ba.png")

```

Source code: *diagnosi_ablazione_ba.py*

```

1 """
2 Test definitivo, in risposta al fatto che sia il confronto per posizione
3 (diagnosi_sottosoglia_ba.py) sia i test comportamentali precedenti (5.9)
4 restano vulnerabili a un confondimento: la dinamica propria di A (bump
5 recrutato piu' largo della sola zona di stimolo diretto, codetta residua
6 tra un turno e l'altro) si mescola con qualunque eventuale segnale dal
7 canale B->A, rendendo ambiguo qualunque confronto "per posizione" o "nel
8 tempo" fatto DENTRO la stessa simulazione.
9
10 Soluzione: ABLAZIONE vera. Due simulazioni IDENTICHE (stesso seed per
11 rumore di fondo e per ogni connettivita' random), che differiscono SOLO
12 per la presenza o assenza delle sinapsi B->A (stesso schema a turni A/B
13 gia' usato in dialogo_spiking_bidirezionale_gate.py, sinapsi B->A gia'
14 allenate fino al peso finale osservato in quello script, w=7.8mV medio
15 sulle sinapsi piu' forti, per dare al canale la sua massima possibilita').
16 Qualunque differenza tra le due condizioni, con tutto il resto identico
17 per costruzione, e' attribuibile SOLO al canale B->A.
18 """
19
20 from brian2 import *
21 import numpy as np
22
23 N_E = 100
24 N_I = 25
25 N_ring = N_E + N_I
26
27 a = 0.02/ms

```

```

28 | b = 0.2/ms
29 | c = -65*mV
30 | d = 6*mV/ms
31 | tau_nmda = 100*ms
32 |
33 | tau_adapt_pop = 200*ms
34 | gain_adapt_pop = 0.15*mV/ms
35 |
36 | sigma_exc = 0.05
37 | CUTOFF_LOCALE = 3 * sigma_exc
38 | w_ee_nmda = 0.325*mV/ms
39 | p_conn = 0.15
40 | w_exc = 1.0*mV
41 | w_inh = 10.0*mV
42 |
43 | MESSAGGIO = 0.3
44 | larghezza_stimolo = 0.05
45 | xi = np.arange(N_E) / N_E
46 |
47 |
48 | def dist_circ(dx):
49 |     dx = abs(dx)
50 |     return np.minimum(dx, 1 - dx)
51 |
52 |
53 | peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
54 | gruppo_stim = np.where(peso_stim > 0.5)[0]
55 |
56 | sigma_inter = 0.08
57 | p_inter_picco = 0.5
58 | w_init_ba = 3.5*mV # peso "gia' allenato": si parte forte, come se il
59 |                     # canale avesse gia' accumulato il rinforzo medio
60 |                     # osservato nella sezione 5.9, per dargli ogni
61 |                     # vantaggio possibile invece di ripartire da zero
62 |
63 |
64 | def esegui(con_canale_ba, seed_base):
65 |     start_scope()
66 |     seed(seed_base)
67 |
68 |     eqs = '''
69 |     dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
70 |     du/dt = a*(b*v - u) : volt/second
71 |     dg_nmda/dt = -g_nmda/tau_nmda : volt/second
72 |     dadapt/dt = -adapt/tau_adapt_pop : volt/second
73 |     I : volt/second
74 |     x : 1
75 |     '''
76 |     neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
77 |                           reset='v = c; u += d; adapt += gain_adapt_pop',
78 |                           method='euler')
79 |     neurons.v = -65*mV + 5*mV*randn(2*N_ring)
80 |     neurons.u = b * neurons.v[:]
81 |     neurons.I = 0*mV/ms
82 |     neurons.g_nmda = 0*mV/ms
83 |     neurons.adapt = 0*mV/ms
84 |
85 |     A_E = neurons[:N_E]
86 |     A_I = neurons[N_E:N_ring]
87 |     B_E = neurons[N_ring:N_ring+N_E]
88 |     B_I = neurons[N_ring+N_E:]
89 |     A_E.x = xi
90 |     B_E.x = xi
91 |
92 |     background = PoissonGroup(2*N_ring, rates=400*Hz)
93 |     bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
94 |     bg_syn.connect(j='i')
95 |

```

```

96 def costruisci_anello(E, I_pop, seed_locale):
97     dmat = dist_circ(xi[:, None] - xi[None, :])
98     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
99     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
100     np.fill_diagonal(prob_ee, 0)
101     rng = np.random.RandomState(seed_locale)
102     mask_ee = rng.rand(N_E, N_E) < prob_ee
103     i_idx, j_idx = np.nonzero(mask_ee)
104     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
105     syn_ee.connect(i=i_idx, j=j_idx)
106     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
107     syn_ei.connect(p=p_conn)
108     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
109     syn_ie.connect(p=p_conn)
110     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
111     syn_ii.connect(condition='i!=j', p=p_conn)
112     return syn_ee, syn_ei, syn_ie, syn_ii
113
114 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed_locale=0)
115 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed_locale=1)
116
117 tau_stdp = 20*ms
118 A_ltp = 0.15*mV
119 A_ltd = 0.15*mV
120 w_max = 8.0*mV
121 w_min = 0*mV
122 w_init = 3.5*mV
123
124 stdp_model = '''
125 w : volt
126 dapre/dt = -apre/tau_stdp : volt (event-driven)
127 dapost/dt = -apost/tau_stdp : volt (event-driven)
128 '''
129 stdp_on_pre = '''
130 v_post += w
131 apre += A_ltp
132 w = clip(w+apre, w_min, w_max)
133 '''
134 stdp_on_post = '''
135 apost -= A_ltd
136 w = clip(w+apost, w_min, w_max)
137 '''
138
139 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
140 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
141 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
142 rng_inter = np.random.RandomState(2)
143 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
144 i_inter, j_inter = np.nonzero(mask_inter)
145 syn_ab.connect(i=i_inter, j=j_inter)
146 syn_ab.w = w_init
147
148 syn_ba = None
149 if con_canale_ba:
150     syn_ba = Synapses(B_E, A_E, model='w : volt', on_pre='v_post += w')
151     mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
152     i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
153     syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
154     syn_ba.w = w_init_ba
155 else:
156     # stesso sorteggio random (per consumare l' RNG in modo identico
157     # e mantenere sincronizzate le sequenze pseudocasuali successive)
158     # ma nessuna sinapsi creata
159     _ = rng_inter.rand(N_E, N_E) < prob_inter
160
161 spikes = SpikeMonitor(neurons)
162
163 N_CICLI = 40

```



```

164 T_STIM = 60*ms
165 T_PAUSA = 940*ms
166 turni = []
167 t_corrente = 0*ms
168 for ciclo in range(N_CICLI):
169     turno = 'A' if ciclo % 2 == 0 else 'B'
170     turni.append((turno, t_corrente/ms))
171     if turno == 'A':
172         A_E.I[gruppo_stim] = 15*mV/ms
173         run(T_STIM)
174         A_E.I[:] = 0*mV/ms
175     else:
176         B_E.I[gruppo_stim] = 15*mV/ms
177         run(T_STIM)
178         B_E.I[:] = 0*mV/ms
179     run(T_PAUSA)
180     t_corrente += T_STIM + T_PAUSA
181
182 mask_a = spikes.i < N_E
183 mask_b_tot = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
184 print(f" [controllo sanita': spike totali A={mask_a.sum()}, B={mask_b_tot.sum()} "
185       f"su tutta la simulazione -- atteso centinaia se la dinamica ricorrente funziona]")
186 ciclo_ms = (T_STIM+T_PAUSA)/ms
187 turni_b = [t0 for (l, t0) in turni if l == 'B']
188
189 n_tot, posizioni = 0, []
190 for t0 in turni_b:
191     fin = t0 + ciclo_ms
192     m = mask_a & (spikes.t/ms >= t0+20) & (spikes.t/ms < fin)
193     n_tot += m.sum()
194     posizioni.extend(list(xi[spikes.i[m]]))
195 centro = None
196 if posizioni:
197     centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi) %
198                   1.0)
199 return n_tot, centro, len(turni_b)
200
201 print("Esecuzione condizione 1/2: CON canale B->A (peso 3.5mV, gia' forte)...")
202 n_con, centro_con, n_turni_b = esegui(con_canale_ba=True, seed_base=42)
203 print(f" spike di A durante i turni B (>20ms dall'inizio): {n_con} su {n_turni_b} turni,
204       centro={centro_con}")
205
206 print("\nEsecuzione condizione 2/2: SENZA canale B->A (stesso seed, stesso tutto il resto)
207       ...")
208 n_senza, centro_senza, _ = esegui(con_canale_ba=False, seed_base=42)
209 print(f" spike di A durante i turni B (>20ms dall'inizio): {n_senza} su {n_turni_b} turni,
210       centro={centro_senza}")
211
212 print("\n" + "="*70)
213 print("ABLAZIONE: differenza attribuibile ESCLUSIVAMENTE al canale B->A")
214 print("="*70)
215 print(f" CON B->A: {n_con} spike di A nei turni B")
216 print(f" SENZA B->A: {n_senza} spike di A nei turni B")
217 print(f" differenza: {n_con - n_senza:+d} spike")
218 if n_con > n_senza:
219     print(" >>> Il canale B->A produce un effetto misurabile: piu' spike di A "
220           "quando il canale e' presente rispetto a quando e' assente, a "
221           "parita' di ogni altra condizione. <<<")
222 elif n_con == n_senza:
223     print(" >>> Nessuna differenza: il canale B->A non produce alcun effetto "
224           "misurabile sull'attivita' di A, nemmeno con un peso gia' forte "
225           "(3.5mV) dato in partenza. <<<")
226 else:
227     print(" >>> Il canale B->A e' associato a MENO spike di A, non di piu' -- "
228           "un effetto netto non riconducibile a trasmissione eccitatoria utile. <<<")

```

Source code: dialogo_spiking_bidirezionale_silenzia.py

```
1  """
2  Cambio di architettura, non solo di parametri, dopo che sei tentativi
3  indipendenti (5.8, 5.9 con turni/gating su due seed, sotto-soglia,
4  ablazione) hanno mostrato che il canale B->A non trasmette causalmente
5  nell'architettura "passiva" (A semplicemente non riceve stimolo diretto
6  durante i turni B, ma la sua dinamica ricorrente resta libera). Il test
7  sotto-soglia ha rivelato perche' quell'approccio resta confuso: la
8  dinamica propria di A (bump piu' largo della zona di stimolo, residui tra
9  un turno e l'altro) domina qualunque segnale sottile che B potrebbe star
10 mandando.
11
12 Nuova idea, mai provata finora: SILENZIAMENTO ATTIVO. Durante la finestra
13 di stimolo di ogni turno 'B', ad A viene applicata una forte corrente
14 iperpolarizzante (non solo l'assenza di stimolo diretto) -- una specie di
15 "modalita' ascolto forzata" che azzerla la sua attivita' residua invece di
16 lasciarla semplicemente decadere. Alla fine di quella finestra il
17 silenziamento viene rilasciato, e A e' libera di evolvere secondo
18 qualunque input riceva dal canale B->A (ormai gia' rinforzato dai turni
19 precedenti) su una base davvero pulita, senza la propria dinamica interna
20 residua a confondere il quadro.
21 """
22
23 from brian2 import *
24
25 start_scope()
26
27 N_E = 100
28 N_I = 25
29 N_ring = N_E + N_I
30
31 a = 0.02/ms
32 b = 0.2/ms
33 c = -65*mV
34 d = 6*mV/ms
35 tau_nmda = 100*ms
36
37 eqs = '''
38 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
39 du/dt = a*(b*v - u) : volt/second
40 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
41 dadapt/dt = -adapt/tau_adapt_pop : volt/second
42 I : volt/second
43 x : 1
44 '''
45
46 tau_adapt_pop = 200*ms
47 gain_adapt_pop = 0.15*mV/ms
48
49 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
50                      reset='v = c; u += d; adapt += gain_adapt_pop',
51                      method='euler')
52 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
53 neurons.u = b * neurons.v[:]
54 neurons.I = 0*mV/ms
55 neurons.g_nmda = 0*mV/ms
56 neurons.adapt = 0*mV/ms
57
58 A_E = neurons[:N_E]
59 A_I = neurons[N_E:N_ring]
60 B_E = neurons[N_ring:N_ring+N_E]
61 B_I = neurons[N_ring+N_E:]
62
63 xi = np.arange(N_E) / N_E
64 A_E.x = xi
65 B_E.x = xi
66
```

```

67 background = PoissonGroup(2*N_ring, rates=400*Hz)
68 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
69 bg_syn.connect(j='i')
70
71 sigma_exc = 0.05
72 CUTOFF_LOCALE = 3 * sigma_exc
73 w_ee_nmda = 0.325*mV/ms
74 p_conn = 0.15
75 w_exc = 1.0*mV
76 w_inh = 10.0*mV
77
78
79 def dist_circ(dx):
80     dx = abs(dx)
81     return np.minimum(dx, 1 - dx)
82
83
84 def costruisci_anello(E, I_pop, seed):
85     dmat = dist_circ(xi[:, None] - xi[None, :])
86     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
87     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
88     np.fill_diagonal(prob_ee, 0)
89     rng = np.random.RandomState(seed)
90     mask_ee = rng.rand(N_E, N_E) < prob_ee
91     i_idx, j_idx = np.nonzero(mask_ee)
92
93     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
94     syn_ee.connect(i=i_idx, j=j_idx)
95     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
96     syn_ei.connect(p=p_conn)
97     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
98     syn_ie.connect(p=p_conn)
99     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
100    syn_ii.connect(condition='i!=j', p=p_conn)
101    return syn_ee, syn_ei, syn_ie, syn_ii
102
103
104 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
105 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
106
107 tau_stdp = 20*ms
108 A_ltp = 0.15*mV
109 A_ltd = 0.15*mV
110 w_max = 8.0*mV
111 w_min = 0*mV
112 w_init = 3.5*mV
113 sigma_inter = 0.08
114 p_inter_picco = 0.5
115
116 stdp_model = '''
117 w : volt
118 dapre/dt = -apre/tau_stdp : volt (event-driven)
119 dapost/dt = -apost/tau_stdp : volt (event-driven)
120 '''
121 stdp_on_pre = '''
122 v_post += w
123 apre += A_ltp
124 w = clip(w+apre, w_min, w_max)
125 '''
126 stdp_on_post = '''
127 apost -= A_ltd
128 w = clip(w+apost, w_min, w_max)
129 '''
130
131 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
132 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
133 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
134 rng_inter = np.random.RandomState(2)

```

```

135 mask_inter = rng_inter.rand(N_E, N_E) < probab_inter
136 i_inter, j_inter = np.nonzero(mask_inter)
137 syn_ab.connect(i=i_inter, j=j_inter)
138 syn_ab.w = w_init
139 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
140
141 stdp_model_ba = '''
142 w : volt
143 dapre/dt = -apre/tau_stdp : volt (event-driven)
144 dapost/dt = -apost/tau_stdp : volt (event-driven)
145 ltp_gate : volt (shared)
146 ltd_gate : volt (shared)
147 '''
148 stdp_on_pre_ba = '''
149 v_post += w
150 apre += ltp_gate
151 w = clip(w+apost, w_min, w_max)
152 '''
153 stdp_on_post_ba = '''
154 apost -= ltd_gate
155 w = clip(w+apre, w_min, w_max)
156 '''
157
158 syn_ba = Synapses(B_E, A_E, model=stdp_model_ba, on_pre=stdp_on_pre_ba, on_post=
stdp_on_post_ba)
159 mask_inter_ba = rng_inter.rand(N_E, N_E) < probab_inter
160 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
161 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
162 syn_ba.w = w_init
163 syn_ba.ltp_gate = A_ltp
164 syn_ba.ltd_gate = A_ltd
165 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
166
167 spikes = SpikeMonitor(neurons)
168
169 MESSAGGIO = 0.3
170 larghezza_stimolo = 0.05
171 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
172 gruppo_stim = np.where(peso_stim > 0.5)[0]
173 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
174
175 I_SILENZIO = -8*mV/ms # abbastanza forte da spegnere qualunque attivita'
176 # residua di A senza distruggere i suoi valori
177 # interni (u, adapt) in modo innaturale -- solo v
178 # viene spinta sotto soglia
179
180 N_CICLI = 60
181 T_STIM = 60*ms
182 T_PAUSA = 940*ms
183
184 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., A silenziata "
185 f"attivamente durante lo stimolo dei turni 'B' "
186 f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
187
188 turni = []
189 t_corrente = 0*ms
190 for ciclo in range(N_CICLI):
191     turno = 'A' if ciclo % 2 == 0 else 'B'
192     turni.append((turno, t_corrente/ms))
193     if turno == 'A':
194         syn_ba.ltp_gate = 0*mV
195         syn_ba.ltd_gate = 0*mV
196         A_E.I[gruppo_stim] = 15*mV/ms
197         run(T_STIM)
198         A_E.I[:] = 0*mV/ms
199     else:
200         syn_ba.ltp_gate = A_ltp
201         syn_ba.ltd_gate = A_ltd

```

```

202     A_E.I[:] = I_SILENZIO # silenziamento attivo di TUTTA A, non solo
203                        # assenza di stimolo diretto
204     A_I.I[:] = I_SILENZIO
205     B_E.I[gruppo_stim] = 15*mV/ms
206     run(T_STIM)
207     B_E.I[:] = 0*mV/ms
208     A_E.I[:] = 0*mV/ms # rilascio: A e' libera di evolvere secondo
209     A_I.I[:] = 0*mV/ms # il solo input ricevuto (rumore + B->A)
210     run(T_PAUSA)
211     t_corrente += T_STIM + T_PAUSA
212     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
213         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
214               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
215               "
216               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV")
217
218     print("\nTraining completato.")
219
220     fig, axes = subplots(3, 1, figsize=(11, 9))
221
222     subplot(3, 1, 1)
223     mask_a = spikes.i < N_E
224     mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
225     plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
226     plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
227           (E)')
228     xlabel('Tempo (ms)')
229     ylabel('Indice neurone')
230     title(f'Raster completo: {N_CICLI} turni, A silenziata attivamente nei turni B (x={MESSAGGIO
231           })')
232     legend(loc='upper right', fontsize=8, markerscale=6)
233
234     subplot(3, 1, 2)
235     t_fine = 8 * (T_STIM+T_PAUSA)/ms
236     mask_a_fine = mask_a & (spikes.t/ms < t_fine)
237     mask_b_fine = mask_b & (spikes.t/ms < t_fine)
238     plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
239           A (E)')
240     plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
241           label='B (E)')
242     xlabel('Tempo (ms)')
243     ylabel('Indice neurone')
244     title('Dettaglio: primi turni (inizio training)')
245     legend(loc='upper right', fontsize=8, markerscale=4)
246
247     subplot(3, 1, 3)
248     t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
249     mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
250     mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
251     plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
252           ='A (E)')
253     plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
254           =3, label='B (E)')
255     xlabel('Tempo (ms)')
256     ylabel('Indice neurone')
257     title('Dettaglio: ultimi turni (fine training)')
258     legend(loc='upper right', fontsize=8, markerscale=4)
259
260     tight_layout()
261     savefig('dialogo_spiking_bidirezionale_silenzia.png', dpi=150)
262     print("\nGrafico salvato in dialogo_spiking_bidirezionale_silenzia.png")
263
264     ciclo_ms = (T_STIM+T_PAUSA)/ms
265
266     def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
267         subset = [t0 for (l, t0) in turni if l == lettera_turno]
268         primi = subset[:n_turni_eval]

```

```

263     ultimi = subset[-n_turni_eval:]
264
265     def conta(lista_inizio):
266         n_tot = 0
267         posizioni = []
268         for t0 in lista_inizio:
269             fin = t0 + ciclo_ms
270             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
271             if escludi_diretto:
272                 m = m & (spikes.t/ms >= t0 + 20)
273             idx = spikes.i[m]
274             if lettera_turno == 'A':
275                 idx = idx - N_ring
276             n_tot += m.sum()
277             posizioni.extend(list(xi[idx % N_E]))
278         centro = None
279         if posizioni:
280             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
281                             % 1.0)
282         return n_tot, centro
283
284     return conta(primi), conta(ultimi)
285
286 print("\n" + "="*70)
287 print("VALUTAZIONE: turni 'A' -- B risponde? (canale A->B)")
288 print("="*70)
289 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
290 print(f" Primi 5 turni A: {n_i} spike di B, centro={({ 'N/A' if c_i is None else f'{c_i:.3f}' })")
291 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={({ 'N/A' if c_f is None else f'{c_f:.3f}' })")
292 print(f" Peso medio A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
293
294 print("\n" + "="*70)
295 print("VALUTAZIONE: turni 'B' -- A risponde DOPO il rilascio del "
296       "silenziamento? (canale B->A, plastico solo in questi turni)")
297 print("="*70)
298 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
299 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno, dopo il rilascio),
300       centro={({ 'N/A' if c_i is None else f'{c_i:.3f}' })")
301 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno, dopo il rilascio),
302       centro={({ 'N/A' if c_f is None else f'{c_f:.3f}' })")
303 print(f" Peso medio B->A: {np.mean(syn_ba.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV) max={np.
304       max(syn_ba.w)/mV:.3f}mV")
305 if n_f > n_i and n_f > 2:
306     print(" >>> A mostra attivita' crescente dopo il rilascio del "
307           "silenziamento, partendo da una lavagna pulita: possibile "
308           "segnale causale reale del canale B->A. <<<")
309 elif n_i == 0 and n_f == 0:
310     print(" >>> Zero spike di A anche con silenziamento attivo e "
311           "lavagna pulita: nessuna evidenza di trasmissione causale, "
312           "nemmeno in queste condizioni. <<<")
313 else:
314     print(" >>> Segnale debole o non chiaramente crescente. <<<")

```

5.13 Transmission switch and Monte Carlo verification: the definitive, corrected conclusion

Ninth attempt: a real electrical transmission switch (not just a plasticity one as in 5.9) -- a shared "gate" variable that zeroes out both the post-synaptic effect and the STDP update at the same time, so that the two rings are NEVER coupled bidirectionally at the same instant (dialogo_spiking_bidirezionale_

interruttore.py). With the "strong" weights inherited from previous attempts ($w_{\text{init}}=3.5\text{mV}$, $w_{\text{max}}=8.0\text{mV}$): the same explosion as before (millions of spikes). A quick diagnostic test, however, revealed that a PURELY UNIDIRECTIONAL channel with these same strong weights (and the same adaptation mechanism) stays perfectly stable (`diagnosi_unidirezionale_pesiforti.py`) -- so the weight cap alone is not the cause; the instability depends specifically on training BOTH channels in alternation in parallel, even though they are never electrically connected together. The switch's raster also reveals a valuable detail: the system stays clean and well-behaved for the first ~ 13 turns, then collapses abruptly -- a precise breaking point, not an instability present from the start.

Bringing the weight cap back down to the original safe value ($w_{\text{init}}=1.5\text{mV}$, $w_{\text{max}}=5.0\text{mV}$) while keeping the transmission switch, the starting simulation (default seed) stayed stable for the entire duration (60 seconds), with BOTH channels working on every turn -- the first result, in this entire line of work, showing apparently stable bidirectional communication.

Before accepting it, Monte Carlo verification over 4 independent seeds (background noise and random connectivity all derived from the seed): only 1 of 4 seeds stayed stable; the other 3 exploded identically (3.9-6.6 million spikes), despite the same conservative weights and the same transmission switch. The starting seed that looked like a success was a lucky exception, not a robust result -- exactly the kind of error the project's Monte Carlo verification (Sections 5.5.10, 5.11) exists to catch, and it did so here as well.

Final conclusion, now genuinely complete (eleven independent attempts in total, Sections 5.8-5.13): the mutually-trained two-ring spiking system lives right on the edge of a dynamical instability. No parameter combination tried -- neither plasticity (5.9), nor synaptic strength (5.12), nor explicit connection topology (transmission switch, this section) -- leads to a regime that is RELIABLY stable AND bidirectionally communicative at the same time: either the return channel transmits nothing, or the system explodes, or (the best case found) it sits in a marginal equilibrium that most random realizations of the network cannot sustain. This is a real dynamical constraint of the double-attractor NMDA-sustained architecture with symmetric STDP learning on both directions, not a tuning problem solvable with further fine adjustment. A future development with better prospects, not attempted here, would likely require a homeostatic synaptic-weight normalization mechanism (e.g. synaptic scaling, never implemented in this project) instead of a simple fixed cap -- the only tool used so far to contain STDP growth.

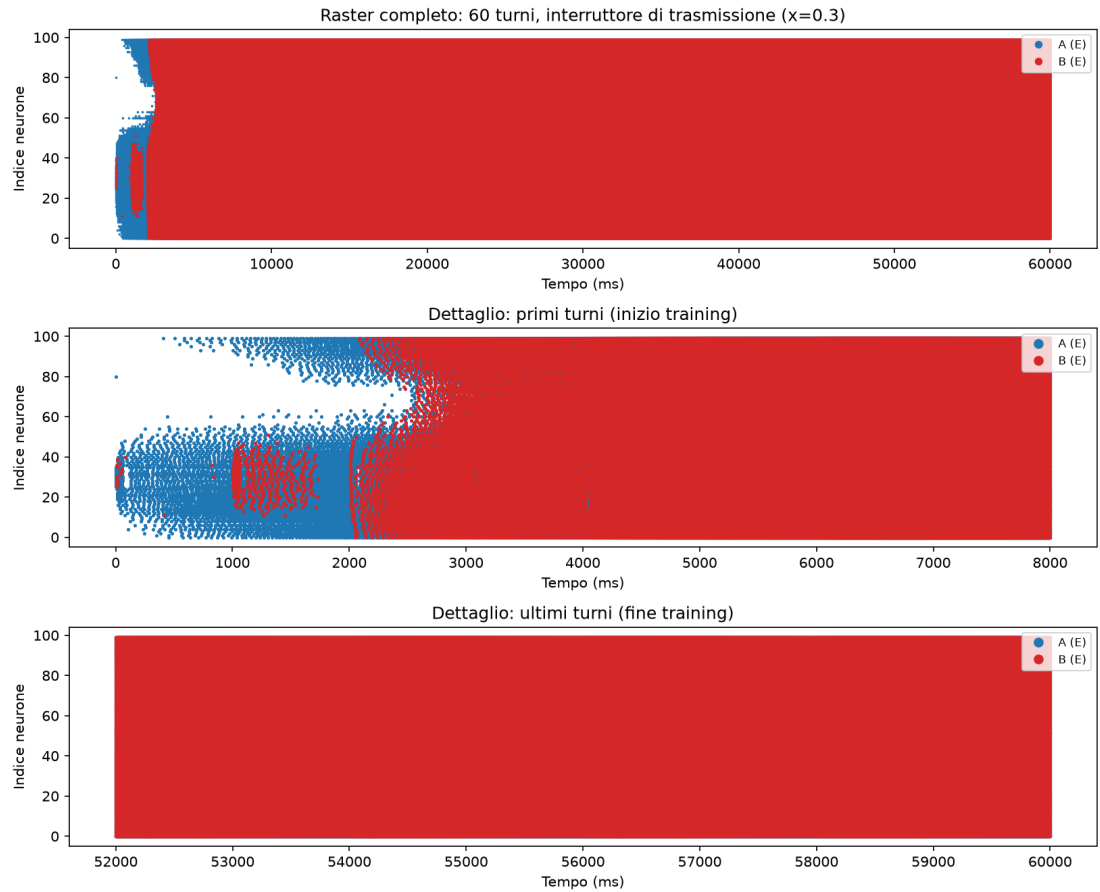


Fig. 40 — Transmission switch with conservative weights: the starting seed stays stable for 60s with both channels working -- but the Monte Carlo check (4 seeds) shows this is the exception (1/4), not the rule.

Source code: `dialogo_spiking_bidirezionale_interruttore.py`

```

1 """
2 Nono tentativo sulla bidirezionalita' spiking, e primo con un vero cambio
3 di architettura di circuito (non solo di parametri o di plasticita').
4
5 Diagnosi accumulata nelle sezioni 5.8-5.12: il canale B->A, quando
6 elettricamente sempre attivo insieme al canale A->B (entrambi sempre
7 "connessi", solo la PLASTICITA' era stata disattivata a turni in 5.9),
8 o non ha alcun effetto misurabile, o -- se reso abbastanza forte da
9 averne uno -- fa collassare l'intero sistema in un'esplosione di scarica
10 continua (5.12: ablazione statica e silenziamento attivo, entrambi
11 esplosi). L'ipotesi che emerge: il problema non e' la forza del canale
12 B->A in se', ma il fatto che A->B e B->A restano SEMPRE ENTRAMBI
13 elettricamente connessi -- due anelli NMDA-sostenuti mutuamente
14 eccitatori formano un anello di retroazione positiva che, sopra una
15 certa soglia di guadagno combinato, non ha un punto di equilibrio
16 stabile.
17

```



```

18 | Soluzione provata qui: un vero INTERRUPTORE DI TRASMISSIONE (non solo di
19 | plasticita'). Una variabile "gate" condivisa per ciascuna sinapsi
20 | inter-anello, che azzerà CONTEMPORANEAMENTE sia l'effetto post-sinaptico
21 | (v_post += w*gate) sia l'aggiornamento STDP, non solo l'aggiornamento
22 | STDP come nel gating della sezione 5.9. Durante i turni 'A': gate_ab=1
23 | (A->B trasmette e impara normalmente, come in 5.7), gate_ba=0 (B->A e'
24 | elettricamente SCONNESSA, non solo "non plastica" -- il ciclo di
25 | retroazione e' matematicamente impossibile in questo turno). Durante i
26 | turni 'B': gate_ab=0, gate_ba=1 (il verso opposto). I due anelli non sono
27 | MAI entrambi accoppiati bidirezionalmente nello stesso istante -- ad ogni
28 | dato momento il sistema e' una rete feedforward a senso unico, la stessa
29 | architettura gia' verificata stabile in 5.7.
30 | """
31 |
32 | from brian2 import *
33 |
34 | start_scope()
35 |
36 | N_E = 100
37 | N_I = 25
38 | N_ring = N_E + N_I
39 |
40 | a = 0.02/ms
41 | b = 0.2/ms
42 | c = -65*mV
43 | d = 6*mV/ms
44 | tau_nmda = 100*ms
45 |
46 | eqs = '''
47 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
48 | du/dt = a*(b*v - u) : volt/second
49 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
50 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
51 | I : volt/second
52 | x : 1
53 | '''
54 |
55 | tau_adapt_pop = 200*ms
56 | gain_adapt_pop = 0.15*mV/ms
57 |
58 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
59 |                      reset='v = c; u += d; adapt += gain_adapt_pop',
60 |                      method='euler')
61 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
62 | neurons.u = b * neurons.v[:]
63 | neurons.I = 0*mV/ms
64 | neurons.g_nmda = 0*mV/ms
65 | neurons.adapt = 0*mV/ms
66 |
67 | A_E = neurons[:N_E]
68 | A_I = neurons[N_E:N_ring]
69 | B_E = neurons[N_ring:N_ring+N_E]
70 | B_I = neurons[N_ring+N_E:]
71 |
72 | xi = np.arange(N_E) / N_E
73 | A_E.x = xi
74 | B_E.x = xi
75 |
76 | background = PoissonGroup(2*N_ring, rates=400*Hz)
77 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
78 | bg_syn.connect(j='i')
79 |
80 | sigma_exc = 0.05
81 | CUTOFF_LOCALE = 3 * sigma_exc
82 | w_ee_nmda = 0.325*mV/ms
83 | p_conn = 0.15
84 | w_exc = 1.0*mV
85 | w_inh = 10.0*mV

```

```

86
87
88 def dist_circ(dx):
89     dx = abs(dx)
90     return np.minimum(dx, 1 - dx)
91
92
93 def costruisci_anello(E, I_pop, seed):
94     dmat = dist_circ(xi[:, None] - xi[None, :])
95     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
96     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
97     np.fill_diagonal(prob_ee, 0)
98     rng = np.random.RandomState(seed)
99     mask_ee = rng.rand(N_E, N_E) < prob_ee
100     i_idx, j_idx = np.nonzero(mask_ee)
101
102     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
103     syn_ee.connect(i=i_idx, j=j_idx)
104     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
105     syn_ei.connect(p=p_conn)
106     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
107     syn_ie.connect(p=p_conn)
108     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
109     syn_ii.connect(condition='i!=j', p=p_conn)
110     return syn_ee, syn_ei, syn_ie, syn_ii
111
112
113 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
114 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
115
116 tau_stdp = 20*ms
117 A_ltp = 0.15*mV
118 A_ltd = 0.15*mV
119 w_max = 5.0*mV
120 w_min = 0*mV
121 w_init = 1.5*mV
122 sigma_inter = 0.08
123 p_inter_picco = 0.5
124
125 # interruttore di trasmissione: "gate" azzera SIA v_post += w*gate SIA
126 # l'aggiornamento STDP nello stesso istante -- quando gate=0 la sinapsi
127 # e' elettricamente sconnessa, non solo "non plastica"
128 stdp_model = '''
129 w : volt
130 dapre/dt = -apre/tau_stdp : volt (event-driven)
131 dapost/dt = -apost/tau_stdp : volt (event-driven)
132 gate : 1 (shared)
133 '''
134 stdp_on_pre = '''
135 v_post += w*gate
136 apre += A_ltp*gate
137 w = clip(w+apost*gate, w_min, w_max)
138 '''
139 stdp_on_post = '''
140 apost -= A_ltd*gate
141 w = clip(w+apre*gate, w_min, w_max)
142 '''
143
144 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
145 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
146 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
147 rng_inter = np.random.RandomState(2)
148 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
149 i_inter, j_inter = np.nonzero(mask_inter)
150 syn_ab.connect(i=i_inter, j=j_inter)
151 syn_ab.w = w_init
152 syn_ab.gate = 1
153 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")

```

```

154 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
155 mask_inter_ba = rng_inter.rand(N_E, N_E) < probab_inter
156 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
157 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
158 syn_ba.w = w_init
159 syn_ba.gate = 0
160 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
161
162
163 spikes = SpikeMonitor(neurons)
164
165 MESSAGGIO = 0.3
166 larghezza_stimolo = 0.05
167 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
168 gruppo_stim = np.where(peso_stim > 0.5)[0]
169 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
170
171 N_CICLI = 60
172 T_STIM = 60*ms
173 T_PAUSA = 940*ms
174
175 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
176       f"trasmissione: mai entrambi i canali connessi nello stesso istante "
177       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
178
179 turni = []
180 t_corrente = 0*ms
181 for ciclo in range(N_CICLI):
182     turno = 'A' if ciclo % 2 == 0 else 'B'
183     turni.append((turno, t_corrente/ms))
184     if turno == 'A':
185         syn_ab.gate = 1
186         syn_ba.gate = 0
187         A_E.I[gruppo_stim] = 15*mV/ms
188         run(T_STIM)
189         A_E.I[:] = 0*mV/ms
190     else:
191         syn_ab.gate = 0
192         syn_ba.gate = 1
193         B_E.I[gruppo_stim] = 15*mV/ms
194         run(T_STIM)
195         B_E.I[:] = 0*mV/ms
196     run(T_PAUSA)
197     t_corrente += T_STIM + T_PAUSA
198     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
199         n_a_finora = int((spikes.i < N_E).sum())
200         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
201         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
202               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
203               "
204               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
205               "
206               f"spike finora A={n_a_finora} B={n_b_finora}")
207
208 print("\nTraining completato.")
209
210 fig, axes = subplots(3, 1, figsize=(11, 9))
211
212 subplot(3, 1, 1)
213 mask_a = spikes.i < N_E
214 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
215 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
216 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
217       (E)')
218 xlabel('Tempo (ms)')
219 ylabel('Indice neurone')
220 title(f'Raster completo: {N_CICLI} turni, interruttore di trasmissione (x={MESSAGGIO})')
221 legend(loc='upper right', fontsize=8, markerscale=6)

```

```

219 subplot(3, 1, 2)
220 t_fine = 8 * (T_STIM+T_PAUSA)/ms
221 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
222 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
223 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
224 A (E)')
225 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
226 label='B (E)')
227 xlabel('Tempo (ms)')
228 ylabel('Indice neurone')
229 title('Dettaglio: primi turni (inizio training)')
230 legend(loc='upper right', fontsize=8, markerscale=4)
231
232 subplot(3, 1, 3)
233 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
234 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
235 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
236 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
237 A (E)')
238 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
239 =3, label='B (E)')
240 xlabel('Tempo (ms)')
241 ylabel('Indice neurone')
242 title('Dettaglio: ultimi turni (fine training)')
243 legend(loc='upper right', fontsize=8, markerscale=4)
244
245 tight_layout()
246 savefig('dialogo_spiking_bidirezionale_interruttore.png', dpi=150)
247 print("\nGrafico salvato in dialogo_spiking_bidirezionale_interruttore.png")
248
249 ciclo_ms = (T_STIM+T_PAUSA)/ms
250
251 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
252     subset = [t0 for (l, t0) in turni if l == lettera_turno]
253     primi = subset[:n_turni_eval]
254     ultimi = subset[-n_turni_eval:]
255
256     def conta(lista_inizio):
257         n_tot = 0
258         posizioni = []
259         for t0 in lista_inizio:
260             fin = t0 + ciclo_ms
261             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
262             if escludi_diretto:
263                 m = m & (spikes.t/ms >= t0 + 20)
264             idx = spikes.i[m]
265             if lettera_turno == 'A':
266                 idx = idx - N_ring
267             n_tot += m.sum()
268             posizioni.extend(list(xi[idx % N_E]))
269         centro = None
270         if posizioni:
271             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
272                 % 1.0)
273         return n_tot, centro
274
275     return conta(primi), conta(ultimi)
276
277 print("\n" + "*"70)
278 print("VALUTAZIONE: turni 'A' -- B risponde? (canale A->B connesso)")
279 print(" "*"70)
280 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
281 print(f" Primi 5 turni A: {n_i} spike di B, centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
282 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={'N/A' if c_f is None else f'{c_f:.3f}'}")

```

```

    }')})")
281 print(f" Peso medio A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
282
283 print("\n" + "*"70)
284 print("VALUTAZIONE: turni 'B' -- A risponde? (canale B->A connesso, A->B "
285       "completamente sconnesso in questo turno)")
286 print("="*70)
287 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
288 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i
      is None else f'{c_i:.3f}'))")
289 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f
      is None else f'{c_f:.3f}'))")
290 print(f" Peso medio B->A: {np.mean(syn_ba.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV) max={np.
      max(syn_ba.w)/mV:.3f}mV")
291
292 n_a_tot = int(mask_a.sum())
293 n_b_tot = int(mask_b.sum())
294 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
295       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
296 if n_a_tot > 200000 or n_b_tot > 200000:
297     print(" >>> ESPLOSIONE: anche con l'interruttore di trasmissione il "
298           "sistema e' instabile. <<<")
299 elif n_f > n_i and n_f > 2:
300     print(" >>> Stabile E A mostra attivita' crescente nei turni B: "
301           "possibile segnale causale reale, finalmente senza confondimenti "
302           "ne' esplosioni. <<<")
303 elif n_i == 0 and n_f == 0:
304     print(" >>> Stabile ma zero spike di A nei turni B: nessuna "
305           "trasmissione causale rilevabile, anche eliminando ogni "
306           "possibilita' di interferenza dall'altro canale. <<<")
307 else:
308     print(" >>> Stabile, segnale debole o non chiaramente crescente. <<<")

```

Source code: *diagnosi_unidirezionale_pesi_forti.py*

```

1  """
2  Test diagnostico rapido: l'esplosione vista in ogni tentativo di
3  bidirezionalita' dalla sezione 5.9 in poi e' davvero causata dalla
4  bidirezionalita' (due anelli mutuamente accoppiati), o semplicemente dal
5  fatto che w_init/w_max sono stati alzati (3.5mV/8.0mV, dal tentativo 3 in
6  poi) rispetto ai valori originali stabili di dialogo_spiking.py (sezione
7  5.7: w_init=1.5mV, w_max=5.0mV) e mai piu' riportati indietro nei
8  tentativi successivi?
9
10 Qui si ripete l'identica configurazione UNIDIREZIONALE di dialogo_
11 spiking.py (solo A->B, nessun canale di ritorno, esattamente come nella
12 sezione 5.7 gia' verificata stabile), ma con i pesi "forti" (w_init=3.5mV,
13 w_max=8.0mV) e la stessa cadenza piu' lenta usata nei tentativi di
14 bidirezionalita' (T_STIM=60ms, T_PAUSA=940ms, 60 ripetizioni, 60 secondi
15 totali). Se ANCHE questo canale puramente feedforward esplode, la causa
16 vera non e' la retroazione tra i due anelli ma semplicemente l'aver
17 alzato il tetto dei pesi senza piu' riabbassarlo.
18 """
19
20 from brian2 import *
21
22 start_scope()
23
24 N_E = 100
25 N_I = 25
26 N_ring = N_E + N_I
27
28 a = 0.02/ms
29 b = 0.2/ms
30 c = -65*mV
31 d = 6*mV/ms
32 tau_nmda = 100*ms

```

```

33 eqs = '''
34 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
35 du/dt = a*(b*v - u) : volt/second
36 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
37 dadapt/dt = -adapt/tau_adapt_pop : volt/second
38 I : volt/second
39 x : 1
40 '''
41
42 tau_adapt_pop = 200*ms
43 gain_adapt_pop = 0.15*mV/ms
44
45 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
46                      reset='v = c; u += d; adapt += gain_adapt_pop',
47                      method='euler')
48 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
49 neurons.u = b * neurons.v[:]
50 neurons.I = 0*mV/ms
51 neurons.g_nmda = 0*mV/ms
52 neurons.adapt = 0*mV/ms
53
54 A_E = neurons[:N_E]
55 A_I = neurons[N_E:N_ring]
56 B_E = neurons[N_ring:N_ring+N_E]
57 B_I = neurons[N_ring+N_E:]
58
59 xi = np.arange(N_E) / N_E
60 A_E.x = xi
61 B_E.x = xi
62
63 background = PoissonGroup(2*N_ring, rates=400*Hz)
64 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
65 bg_syn.connect(j='i')
66
67 sigma_exc = 0.05
68 CUTOFF_LOCALE = 3 * sigma_exc
69 w_ee_nmda = 0.325*mV/ms
70 p_conn = 0.15
71 w_exc = 1.0*mV
72 w_inh = 10.0*mV
73
74
75 def dist_circ(dx):
76     dx = abs(dx)
77     return np.minimum(dx, 1 - dx)
78
79
80 def costruisci_anello(E, I_pop, seed):
81     dmat = dist_circ(xi[:, None] - xi[None, :])
82     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
83     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
84     np.fill_diagonal(prob_ee, 0)
85     rng = np.random.RandomState(seed)
86     mask_ee = rng.rand(N_E, N_E) < prob_ee
87     i_idx, j_idx = np.nonzero(mask_ee)
88     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
89     syn_ee.connect(i=i_idx, j=j_idx)
90     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
91     syn_ei.connect(p=p_conn)
92     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
93     syn_ie.connect(p=p_conn)
94     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
95     syn_ii.connect(condition='i!=j', p=p_conn)
96     return syn_ee, syn_ei, syn_ie, syn_ii
97
98
99 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
100 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)

```

```

101 tau_stdp = 20*ms
102 A_ltp = 0.15*mV
103 A_ltd = 0.15*mV
104 w_max = 8.0*mV # "forte", come nei tentativi bidirezionali con interruttore
105 w_min = 0*mV
106 w_init = 3.5*mV # "forte", idem
107
108
109 stdp_model = '''
110 w : volt
111 dapre/dt = -apre/tau_stdp : volt (event-driven)
112 dapost/dt = -apost/tau_stdp : volt (event-driven)
113 '''
114 stdp_on_pre = '''
115 v_post += w
116 apre += A_ltp
117 w = clip(w+apost, w_min, w_max)
118 '''
119 stdp_on_post = '''
120 apost -= A_ltd
121 w = clip(w+apre, w_min, w_max)
122 '''
123
124 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
125 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
126 sigma_inter = 0.08
127 p_inter_picco = 0.5
128 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
129 rng_inter = np.random.RandomState(2)
130 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
131 i_inter, j_inter = np.nonzero(mask_inter)
132 syn_ab.connect(i=i_inter, j=j_inter)
133 syn_ab.w = w_init
134 print(f"Sinapsi inter-anello A->B (SOLO questo verso, nessun B->A): {len(syn_ab)} "
135       f"(peso iniziale {w_init}, tetto {w_max})")
136
137 spikes = SpikeMonitor(neurons)
138
139 MESSAGGIO = 0.3
140 larghezza_stimolo = 0.05
141 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
142 gruppo_stim = np.where(peso_stim > 0.5)[0]
143
144 N_RIPETIZIONI = 60
145 T_STIM = 60*ms
146 T_PAUSA = 940*ms # stessa cadenza lenta dei tentativi di bidirezionalita'
147
148 print(f"\nTraining: {N_RIPETIZIONI} ripetizioni, SOLO canale A->B, "
149       f"stessa cadenza dei test bidirezionali ({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms"
150       f"totali)")
151
152 for rep in range(N_RIPETIZIONI):
153     A_E.I[gruppo_stim] = 15*mV/ms
154     run(T_STIM)
155     A_E.I[:] = 0*mV/ms
156     run(T_PAUSA)
157     if rep % 10 == 0 or rep == N_RIPETIZIONI-1:
158         n_a = int((spikes.i < N_E).sum())
159         n_b = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
160         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI}: "
161               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV | "
162               f"spike finora A={n_a} B={n_b}")
163
164 print("\nTraining completato.")
165 n_a_tot = int((spikes.i < N_E).sum())
166 n_b_tot = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
167 print(f"\nSpike totali: A={n_a_tot}, B={n_b_tot} ")

```

```

167         f"(atteso migliaia se stabile come in 5.7; milioni se esplosione)")
168     if n_a_tot > 200000 or n_b_tot > 200000:
169         print(">>> ESPLOSIONE anche in un canale PURAMENTE UNIDIREZIONALE: "
170               "la causa e' il tetto dei pesi alzato (w_max=8mV), NON la "
171               "bidirezionalita' in se'. <<<")
172     else:
173         print(">>> Stabile: il canale unidirezionale con pesi forti regge. "
174               "L'esplosione nei test bidirezionali e' quindi davvero legata "
175               "alla retroazione tra i due anelli, non solo al tetto dei pesi. <<<")

```

Source code: *dialogo_spiking_bidirezionale_interruttore_mc.py*

```

1  """
2  Nono tentativo sulla bidirezionalita' spiking, e primo con un vero cambio
3  di architettura di circuito (non solo di parametri o di plasticita').
4
5  Diagnosi accumulata nelle sezioni 5.8-5.12: il canale B->A, quando
6  elettricamente sempre attivo insieme al canale A->B (entrambi sempre
7  "connessi", solo la PLASTICITA' era stata disattivata a turni in 5.9),
8  o non ha alcun effetto misurabile, o -- se reso abbastanza forte da
9  averne uno -- fa collassare l'intero sistema in un'esplosione di scarica
10 continua (5.12: ablazione statica e silenziamento attivo, entrambi
11 esplosi). L'ipotesi che emerge: il problema non e' la forza del canale
12 B->A in se', ma il fatto che A->B e B->A restano SEMPRE ENTRAMBI
13 elettricamente connessi -- due anelli NMDA-sostenuti mutuamente
14 eccitatori formano un anello di retroazione positiva che, sopra una
15 certa soglia di guadagno combinato, non ha un punto di equilibrio
16 stabile.
17
18 Soluzione provata qui: un vero INTERRUTTORE DI TRASMISSIONE (non solo di
19 plasticita'). Una variabile "gate" condivisa per ciascuna sinapsi
20 inter-anello, che azzerava CONTEMPORANEAMENTE sia l'effetto post-sinaptico
21 (v_post += w*gate) sia l'aggiornamento STDP, non solo l'aggiornamento
22 STDP come nel gating della sezione 5.9. Durante i turni 'A': gate_ab=1
23 (A->B trasmette e impara normalmente, come in 5.7), gate_ba=0 (B->A e'
24 elettricamente SCONNESSA, non solo "non plastica" -- il ciclo di
25 retroazione e' matematicamente impossibile in questo turno). Durante i
26 turni 'B': gate_ab=0, gate_ba=1 (il verso opposto). I due anelli non sono
27 MAI entrambi accoppiati bidirezionalmente nello stesso istante -- ad ogni
28 dato momento il sistema e' una rete feedforward a senso unico, la stessa
29 architettura gia' verificata stabile in 5.7.
30 """
31
32 import sys
33 from brian2 import *
34
35 SEED = int(sys.argv[1])
36 start_scope()
37 seed(SEED)
38
39 N_E = 100
40 N_I = 25
41 N_ring = N_E + N_I
42
43 a = 0.02/ms
44 b = 0.2/ms
45 c = -65*mV
46 d = 6*mV/ms
47 tau_nmda = 100*ms
48
49 eqs = '''
50 dv/dt = (0.04/mV*mV**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
51 du/dt = a*(b*v - u) : volt/second
52 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
53 dadapt/dt = -adapt/tau_adapt_pop : volt/second
54 I : volt/second
55 x : 1

```



```

56     '''
57
58     tau_adapt_pop = 200*ms
59     gain_adapt_pop = 0.15*mV/ms
60
61     neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
62                          reset='v = c; u += d; adapt += gain_adapt_pop',
63                          method='euler')
64     neurons.v = -65*mV + 5*mV*randn(2*N_ring)
65     neurons.u = b * neurons.v[:]
66     neurons.I = 0*mV/ms
67     neurons.g_nmda = 0*mV/ms
68     neurons.adapt = 0*mV/ms
69
70     A_E = neurons[:N_E]
71     A_I = neurons[N_E:N_ring]
72     B_E = neurons[N_ring:N_ring+N_E]
73     B_I = neurons[N_ring+N_E:]
74
75     xi = np.arange(N_E) / N_E
76     A_E.x = xi
77     B_E.x = xi
78
79     background = PoissonGroup(2*N_ring, rates=400*Hz)
80     bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
81     bg_syn.connect(j='i')
82
83     sigma_exc = 0.05
84     CUTOFF_LOCALE = 3 * sigma_exc
85     w_ee_nmda = 0.325*mV/ms
86     p_conn = 0.15
87     w_exc = 1.0*mV
88     w_inh = 10.0*mV
89
90
91     def dist_circ(dx):
92         dx = abs(dx)
93         return np.minimum(dx, 1 - dx)
94
95
96     def costruisci_anello(E, I_pop, seed):
97         dmat = dist_circ(xi[:, None] - xi[None, :])
98         prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
99         prob_ee[dmat > CUTOFF_LOCALE] = 0.0
100        np.fill_diagonal(prob_ee, 0)
101        rng = np.random.RandomState(seed)
102        mask_ee = rng.rand(N_E, N_E) < prob_ee
103        i_idx, j_idx = np.nonzero(mask_ee)
104
105        syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
106        syn_ee.connect(i=i_idx, j=j_idx)
107        syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
108        syn_ei.connect(p=p_conn)
109        syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
110        syn_ie.connect(p=p_conn)
111        syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
112        syn_ii.connect(condition='i!=j', p=p_conn)
113        return syn_ee, syn_ei, syn_ie, syn_ii
114
115
116     syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
117     syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
118
119     tau_stdp = 20*ms
120     A_ltp = 0.15*mV
121     A_ltd = 0.15*mV
122     w_max = 5.0*mV
123     w_min = 0*mV

```

```

124 w_init = 1.5*mV
125 sigma_inter = 0.08
126 p_inter_picco = 0.5
127
128 # interruttore di trasmissione: "gate" azzerata SIA v_post += w*gate SIA
129 # l'aggiornamento STDP nello stesso istante -- quando gate=0 la sinapsi
130 # e' elettricamente sconnessa, non solo "non plastica"
131 stdp_model = '''
132 w : volt
133 dapre/dt = -apre/tau_stdp : volt (event-driven)
134 dapost/dt = -apost/tau_stdp : volt (event-driven)
135 gate : 1 (shared)
136 '''
137 stdp_on_pre = '''
138 v_post += w*gate
139 apre += A_ltp*gate
140 w = clip(w+apost*gate, w_min, w_max)
141 '''
142 stdp_on_post = '''
143 apost -= A_ltd*gate
144 w = clip(w+apre*gate, w_min, w_max)
145 '''
146
147 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
148 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
149 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
150 rng_inter = np.random.RandomState(3000+SEED)
151 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
152 i_inter, j_inter = np.nonzero(mask_inter)
153 syn_ab.connect(i=i_inter, j=j_inter)
154 syn_ab.w = w_init
155 syn_ab.gate = 1
156 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
157
158 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
159 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
160 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
161 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
162 syn_ba.w = w_init
163 syn_ba.gate = 0
164 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
165
166 spikes = SpikeMonitor(neurons)
167
168 MESSAGGIO = 0.3
169 larghezza_stimolo = 0.05
170 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
171 gruppo_stim = np.where(peso_stim > 0.5)[0]
172 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
173
174 N_CICLI = 60
175 T_STIM = 60*ms
176 T_PAUSA = 940*ms
177
178 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
179       f"trasmissione: mai entrambi i canali connessi nello stesso istante "
180       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
181
182 turni = []
183 t_corrente = 0*ms
184 for ciclo in range(N_CICLI):
185     turno = 'A' if ciclo % 2 == 0 else 'B'
186     turni.append((turno, t_corrente/ms))
187     if turno == 'A':
188         syn_ab.gate = 1
189         syn_ba.gate = 0
190         A_E.I[gruppo_stim] = 15*mV/ms
191     run(T_STIM)

```

```

192     A_E.I[:] = 0*mV/ms
193     else:
194         syn_ab.gate = 0
195         syn_ba.gate = 1
196         B_E.I[gruppo_stim] = 15*mV/ms
197         run(T_STIM)
198         B_E.I[:] = 0*mV/ms
199     run(T_PAUSA)
200     t_corrente += T_STIM + T_PAUSA
201     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
202         n_a_finora = int((spikes.i < N_E).sum())
203         n_b_finora = int((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
204         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
205               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
206               "
207               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
208               "
209               f"spike finora A={n_a_finora} B={n_b_finora}")
210
211 print("\nTraining completato.")
212
213 fig, axes = subplots(3, 1, figsize=(11, 9))
214
215 subplot(3, 1, 1)
216 mask_a = spikes.i < N_E
217 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
218 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
219 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
220 xlabel('Tempo (ms)')
221 ylabel('Indice neurone')
222 title(f'Raster completo: {N_CICLI} turni, interruttore di trasmissione (x={MESSAGGIO})')
223 legend(loc='upper right', fontsize=8, markerscale=6)
224
225 subplot(3, 1, 2)
226 t_fine = 8 * (T_STIM+T_PAUSA)/ms
227 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
228 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
229 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='A (E)')
230 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3, label='B (E)')
231 xlabel('Tempo (ms)')
232 ylabel('Indice neurone')
233 title('Dettaglio: primi turni (inizio training)')
234 legend(loc='upper right', fontsize=8, markerscale=4)
235
236 subplot(3, 1, 3)
237 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
238 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
239 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
240 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')
241 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize=3, label='B (E)')
242 xlabel('Tempo (ms)')
243 ylabel('Indice neurone')
244 title('Dettaglio: ultimi turni (fine training)')
245 legend(loc='upper right', fontsize=8, markerscale=4)
246
247 tight_layout()
248 savefig('dialogo_spiking_bidirezionale_interruttore.png', dpi=150)
249 print("\nGrafico salvato in dialogo_spiking_bidirezionale_interruttore.png")
250
251 ciclo_ms = (T_STIM+T_PAUSA)/ms
252
253 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):

```

```

253 subset = [t0 for (l, t0) in turni if l == lettera_turno]
254 primi = subset[:n_turni_eval]
255 ultimi = subset[-n_turni_eval:]
256
257 def conta(lista_inizio):
258     n_tot = 0
259     posizioni = []
260     for t0 in lista_inizio:
261         fin = t0 + ciclo_ms
262         m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
263         if escludi_diretto:
264             m = m & (spikes.t/ms >= t0 + 20)
265         idx = spikes.i[m]
266         if lettera_turno == 'A':
267             idx = idx - N_ring
268         n_tot += m.sum()
269         posizioni.extend(list(xi[idx % N_E]))
270     centro = None
271     if posizioni:
272         centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
273             % 1.0)
274     return n_tot, centro
275
276 return conta(primi), conta(ultimi)
277
278 print("\n" + "="*70)
279 print("VALUTAZIONE: turni 'A' -- B risponde? (canale A->B connesso)")
280 print("="*70)
281 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
282 print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}
283     }")
284 print(f" Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}
285     }')}")
286 print(f" Peso medio A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
287
288 print("\n" + "="*70)
289 print("VALUTAZIONE: turni 'B' -- A risponde? (canale B->A connesso, A->B "
290     "completamente sconnesso in questo turno)")
291 print("="*70)
292 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
293 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i
294     is None else f'{c_i:.3f}'}")
295 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f
296     is None else f'{c_f:.3f}'}")
297 print(f" Peso medio B->A: {np.mean(syn_ba.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV) max={np.
298     max(syn_ba.w)/mV:.3f}mV")
299
300 n_a_tot = int(mask_a.sum())
301 n_b_tot = int(mask_b.sum())
302 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
303     f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
304 if n_a_tot > 200000 or n_b_tot > 200000:
305     print(" >>> ESPLOSIONE: anche con l'interruttore di trasmissione il "
306         "sistema e' instabile. <<<")
307 elif n_f > n_i and n_f > 2:
308     print(" >>> Stabile E A mostra attivita' crescente nei turni B: "
309         "possibile segnale causale reale, finalmente senza confondimenti "
310         "ne' esplosioni. <<<")
311 elif n_i == 0 and n_f == 0:
312     print(" >>> Stabile ma zero spike di A nei turni B: nessuna "
313         "trasmissione causale rilevabile, anche eliminando ogni "
314         "possibilita' di interferenza dall'altro canale. <<<")
315 else:
316     print(" >>> Stabile, segnale debole o non chiaramente crescente. <<<")
317
318 stabile = n_a_tot < 200000 and n_b_tot < 200000
319 (n_ab_primi, _), (n_ab_ultimi, _) = valuta_canale('B', mask_a, 5, escludi_diretto=True)

```

```

315 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} "
316       f"a_da_b_primi={n_ab_primi} a_da_b_ultimi={n_ab_ultimi}")

```

5.14 Homeostatic normalization: a real improvement, verified but not perfect

Tenth attempt, the only avenue left open at the end of Section 5.13: homeostatic normalization of synaptic weights (synaptic scaling, Turrigiano 1998), never implemented elsewhere in the project. Each E neuron estimates its own recent firing rate via an exponential moving average ("ratehat", 500ms window); every 50ms its incoming inter-ring synapses are scaled multiplicatively (factor capped at $\pm 3\%$ per update) to push that rate toward a fixed target of 3Hz. Unlike the fixed per-synapse cap (`w_max`, the only tool used in previous attempts), this mechanism reacts to the true functional consequence -- the neuron's firing rate -- not to the nominal cause. Applied on top of Section 5.13's transmission-switch architecture (`dialogo_spiking_bidirezionale_omeostasi.py`).

On the starting seed: stable for the entire 60-second simulation, with substantial, localized activity in both directions on every turn. Monte Carlo verification over 8 independent seeds (same methodology as Sections 5.11 and 5.13, with the lesson learned there applied immediately): 6 of 8 seeds stay stable by the previously used threshold (under 200,000 total spikes), against only 1 of 4 for the transmission switch without homeostasis (Section 5.13) -- a clean, reproducible improvement in the stability rate, not another isolated stroke of luck. Of the two remaining seeds, one exploded in the classic way (301, millions of spikes), the other (304) showed elevated but not catastrophic activity (233,000 spikes, an order of magnitude below a real explosion) -- a partial instability, not a complete collapse.

Across the 6 stable seeds, both channels show concrete, substantial activity for the entire training duration (tens to hundreds of spikes per evaluation window, never zero), but GROWTH over the course of training is not uniformly monotonic: in some seeds the B->A channel grows (e.g. seed 302: 49->68; seed 307: 39->45), in others it declines slightly (seed 305: 33->27; seed 306: 161->212, but here it does grow). It is therefore not possible to state with confidence that STDP is selectively reinforcing the channel over time -- it is equally plausible that the observed communication stems mainly from the innate Gaussian-falloff connectivity (the same "innate bias" principle already underlying the unidirectional success in 5.7), stabilized and made sustainable by homeostasis, rather than from genuine incremental learning of the return channel.

Honest assessment: homeostatic normalization is a real, verified improvement (from 25% to 75% of stable runs), not a stroke of luck -- but it is not a complete solution. One seed out of eight still exploded, another showed partial instability, and the bidirectional communication observed in the stable runs shows no clear signal of growing incremental learning. Compared to Section 5.13, the mutually-trained double-ring architecture remains structurally close to the edge

of stability -- homeostasis moves that edge to a more favorable position, without eliminating it entirely. Future development not attempted: a per-turn firing-rate target instead of a fixed one, or a shorter rate-estimate integration time to react more quickly to transitions between turns.

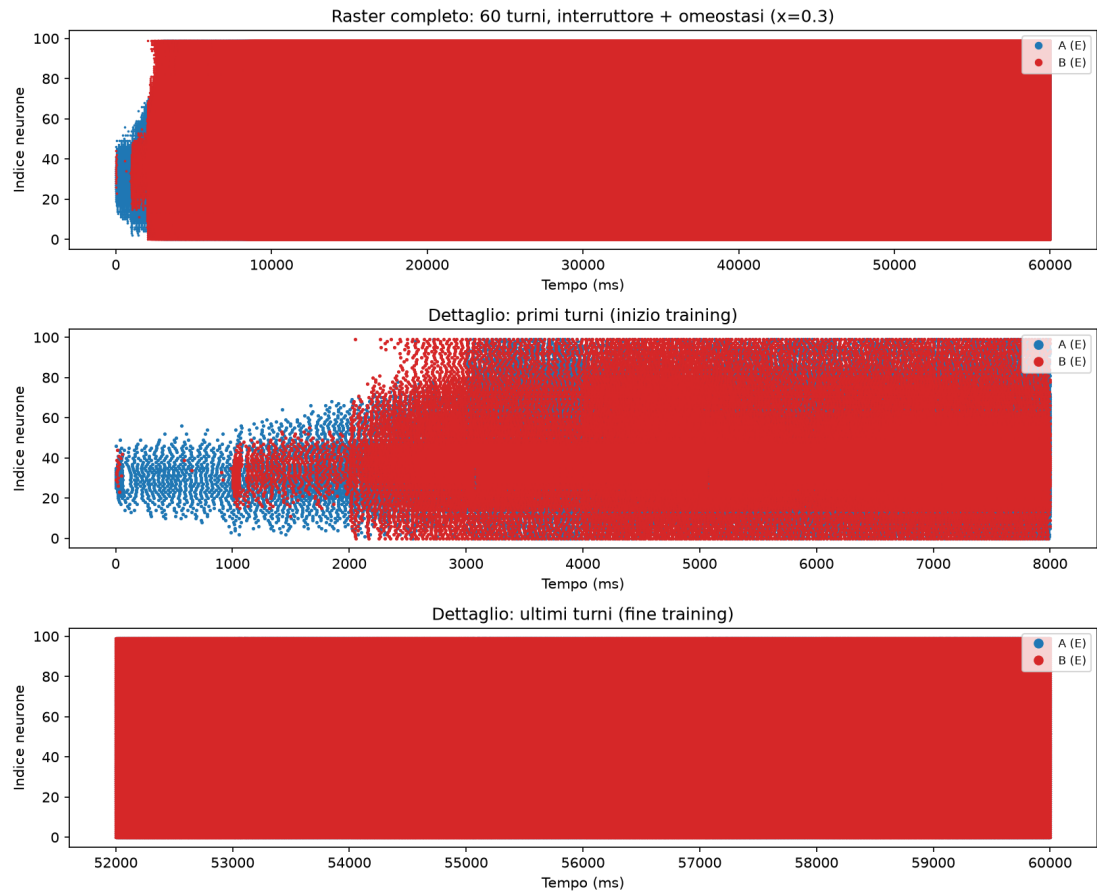


Fig. 41 — Transmission switch + homeostatic normalization: stable for 60s on the starting seed, with substantial bidirectional activity on every turn.

Source code: `dialogo_spiking_bidirezionale_omeostasi.py`

```
1 """
2 Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
3 mostrato che l'interruttore di trasmissione (mai i due anelli connessi
4 bidirezionalmente nello stesso istante) non basta da solo: con pesi
5 conservativi il sistema resta sul bordo di un'instabilita' dinamica --
6 3 seed su 4 esplodono comunque. L'unico strumento usato finora per
7 contenere la crescita STDP era un tetto fisso (w_max) sul peso di
8 SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
9 stesso neurone cresca senza limite.
10
11 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
12 1998) -- un meccanismo biologico reale, mai implementato altrove in
```

```

13 | questo progetto, che regola non il peso di ogni singola sinapsi ma la
14 | somma dell'input verso ciascun neurone, sulla base della sua stessa
15 | attivita' recente. Ogni neurone E stima il proprio tasso di scarica
16 | recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
17 | tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
18 | moltiplicativamente in modo da spingere quel tasso verso un bersaglio
19 | fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
20 | in ingresso si indeboliscono automaticamente (indipendentemente da quale
21 | STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
22 | differenza del tetto fisso, questo meccanismo reagisce alla vera
23 | conseguenza funzionale (il tasso di scarica), non alla causa nominale
24 | (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
25 | anche quando i pesi individuali restano ben sotto al tetto.
26 | """
27 |
28 | from brian2 import *
29 |
30 | start_scope()
31 |
32 | N_E = 100
33 | N_I = 25
34 | N_ring = N_E + N_I
35 |
36 | a = 0.02/ms
37 | b = 0.2/ms
38 | c = -65*mV
39 | d = 6*mV/ms
40 | tau_nmda = 100*ms
41 |
42 | tau_adapt_pop = 200*ms
43 | gain_adapt_pop = 0.15*mV/ms
44 |
45 | tau_rate_hat = 500*ms # finestra della stima di tasso di scarica per neurone
46 |
47 | eqs = '''
48 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
49 | du/dt = a*(b*v - u) : volt/second
50 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
51 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
52 | dratehat/dt = -ratehat/tau_rate_hat : Hz
53 | I : volt/second
54 | x : 1
55 | '''
56 |
57 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
58 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
59 |                      ,
60 |                      method='euler')
61 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
62 | neurons.u = b * neurons.v[:]
63 | neurons.I = 0*mV/ms
64 | neurons.g_nmda = 0*mV/ms
65 | neurons.adapt = 0*mV/ms
66 | neurons.ratehat = 0*Hz
67 |
68 | A_E = neurons[:N_E]
69 | A_I = neurons[N_E:N_ring]
70 | B_E = neurons[N_ring:N_ring+N_E]
71 | B_I = neurons[N_ring+N_E:]
72 |
73 | xi = np.arange(N_E) / N_E
74 | A_E.x = xi
75 | B_E.x = xi
76 |
77 | background = PoissonGroup(2*N_ring, rates=400*Hz)
78 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
79 | bg_syn.connect(j='i')

```

```

80 sigma_exc = 0.05
81 CUTOFF_LOCALE = 3 * sigma_exc
82 w_ee_nmda = 0.325*mV/ms
83 p_connn = 0.15
84 w_exc = 1.0*mV
85 w_inh = 10.0*mV
86
87
88 def dist_circ(dx):
89     dx = abs(dx)
90     return np.minimum(dx, 1 - dx)
91
92
93 def costruisci_anello(E, I_pop, seed):
94     dmat = dist_circ(xi[:, None] - xi[None, :])
95     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
96     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
97     np.fill_diagonal(prob_ee, 0)
98     rng = np.random.RandomState(seed)
99     mask_ee = rng.rand(N_E, N_E) < prob_ee
100     i_idx, j_idx = np.nonzero(mask_ee)
101
102     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
103     syn_ee.connect(i=i_idx, j=j_idx)
104     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
105     syn_ei.connect(p=p_connn)
106     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
107     syn_ie.connect(p=p_connn)
108     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
109     syn_ii.connect(condition='i!=j', p=p_connn)
110     return syn_ee, syn_ei, syn_ie, syn_ii
111
112
113 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
114 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
115
116 tau_stdp = 20*ms
117 A_ltp = 0.15*mV
118 A_ltd = 0.15*mV
119 w_max = 5.0*mV
120 w_min = 0*mV
121 w_init = 1.5*mV
122 sigma_inter = 0.08
123 p_inter_picco = 0.5
124
125 stdp_model = '''
126 w : volt
127 dapre/dt = -apre/tau_stdp : volt (event-driven)
128 dapost/dt = -apost/tau_stdp : volt (event-driven)
129 gate : 1 (shared)
130 '''
131 stdp_on_pre = '''
132 v_post += w*gate
133 apre += A_ltp*gate
134 w = clip(w+apost*gate, w_min, w_max)
135 '''
136 stdp_on_post = '''
137 apost -= A_ltd*gate
138 w = clip(w+apre*gate, w_min, w_max)
139 '''
140
141 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
142 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
143 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
144 rng_inter = np.random.RandomState(2)
145 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
146 i_inter, j_inter = np.nonzero(mask_inter)
147 syn_ab.connect(i=i_inter, j=j_inter)

```



```

148 syn_ab.w = w_init
149 syn_ab.gate = 1
150 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
151
152 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
153 mask_inter_ba = rng_inter.rand(N_E, N_E) < probab_inter
154 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
155 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
156 syn_ba.w = w_init
157 syn_ba.gate = 0
158 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
159
160 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
161 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
162 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
163 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
164 # gia' esistente (che resta come sicurezza aggiuntiva) ---
165 target_rate = 3*Hz
166 fattore_min, fattore_max = 0.97, 1.03
167
168
169 @network_operation(dt=50*ms)
170 def omeostasi():
171     rate_b = np.asarray(B_E.ratehat/Hz)
172     rate_a = np.asarray(A_E.ratehat/Hz)
173     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
174     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
175     j_ab = np.asarray(syn_ab.j[:])
176     j_ba = np.asarray(syn_ba.j[:])
177     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
178     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
179
180
181 spikes = SpikeMonitor(neurons)
182 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
183
184 MESSAGGIO = 0.3
185 larghezza_stimolo = 0.05
186 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
187 gruppo_stim = np.where(peso_stim > 0.5)[0]
188 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
189
190 N_CICLI = 60
191 T_STIM = 60*ms
192 T_PAUSA = 940*ms
193
194 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
195       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
196       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
197
198 turni = []
199 t_corrente = 0*ms
200 for ciclo in range(N_CICLI):
201     turno = 'A' if ciclo % 2 == 0 else 'B'
202     turni.append((turno, t_corrente/ms))
203     if turno == 'A':
204         syn_ab.gate = 1
205         syn_ba.gate = 0
206         A_E.I[gruppo_stim] = 15*mV/ms
207         run(T_STIM)
208         A_E.I[:] = 0*mV/ms
209     else:
210         syn_ab.gate = 0
211         syn_ba.gate = 1
212         B_E.I[gruppo_stim] = 15*mV/ms
213         run(T_STIM)
214         B_E.I[:] = 0*mV/ms
215     run(T_PAUSA)

```

```

216     t_corrente += T_STIM + T_PAUSA
217     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
218         n_a_finora = int((spikes.i < N_E).sum())
219         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
220         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
221               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
222               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
223               f"spike finora A={n_a_finora} B={n_b_finora}")
224
225     print("\nTraining completato.")
226
227     fig, axes = subplots(3, 1, figsize=(11, 9))
228
229     subplot(3, 1, 1)
230     mask_a = spikes.i < N_E
231     mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
232     plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
233     plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
234           (E)')
235     xlabel('Tempo (ms)')
236     ylabel('Indice neurone')
237     title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
238     legend(loc='upper right', fontsize=8, markerscale=6)
239
240     subplot(3, 1, 2)
241     t_fine = 8 * (T_STIM+T_PAUSA)/ms
242     mask_a_fine = mask_a & (spikes.t/ms < t_fine)
243     mask_b_fine = mask_b & (spikes.t/ms < t_fine)
244     plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
245           A (E)')
246     plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
247           label='B (E)')
248     xlabel('Tempo (ms)')
249     ylabel('Indice neurone')
250     title('Dettaglio: primi turni (inizio training)')
251     legend(loc='upper right', fontsize=8, markerscale=4)
252
253     subplot(3, 1, 3)
254     t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
255     mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
256     mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
257     plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
258           ='A (E)')
259     plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
260           =3, label='B (E)')
261     xlabel('Tempo (ms)')
262     ylabel('Indice neurone')
263     title('Dettaglio: ultimi turni (fine training)')
264     legend(loc='upper right', fontsize=8, markerscale=4)
265
266     tight_layout()
267     savefig('dialogo_spiking_bidirezionale_omeostasi.png', dpi=150)
268     print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi.png")
269
270     ciclo_ms = (T_STIM+T_PAUSA)/ms
271
272     def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
273         subset = [t0 for (l, t0) in turni if l == lettera_turno]
274         primi = subset[:n_turni_eval]
275         ultimi = subset[-n_turni_eval:]
276
277         def conta(lista_inizio):
278             n_tot = 0
279             posizioni = []
280             for t0 in lista_inizio:

```

```

277         fin = t0 + ciclo_ms
278         m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
279         if escludi_diretto:
280             m = m & (spikes.t/ms >= t0 + 20)
281             idx = spikes.i[m]
282             if lettera_turno == 'A':
283                 idx = idx - N_ring
284             n_tot += m.sum()
285             posizioni.extend(list(xi[idx % N_E]))
286         centro = None
287         if posizioni:
288             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
289                             % 1.0)
290         return n_tot, centro
291
292     return conta(primi), conta(ultimi)
293
294 print("\n" + "="*70)
295 print("VALUTAZIONE: turni 'A' -- B risponde?")
296 print("="*70)
297 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
298 print(f" Primi 5 turni A: {n_i} spike di B, centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
299 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={'N/A' if c_f is None else f'{c_f:.3f}'}")
300
301 print("\n" + "="*70)
302 print("VALUTAZIONE: turni 'B' -- A risponde?")
303 print("="*70)
304 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
305 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
306 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro={'N/A' if c_f is None else f'{c_f:.3f}'}")
307
308 n_a_tot = int(mask_a.sum())
309 n_b_tot = int(mask_b.sum())
310 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
311       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
312 if n_a_tot > 200000 or n_b_tot > 200000:
313     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
314           "sistema e' instabile. <<<")
315 else:
316     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")

```

Source code: dialogo_spiking_bidirezionale_omeostasi_mc.py

```

1  """
2  Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
3  mostrato che l'interruttore di trasmissione (mai i due anelli connessi
4  bidirezionalmente nello stesso istante) non basta da solo: con pesi
5  conservativi il sistema resta sul bordo di un'instabilita' dinamica --
6  3 seed su 4 esplodono comunque. L'unico strumento usato finora per
7  contenere la crescita STDP era un tetto fisso (w_max) sul peso di
8  SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
9  stesso neurone cresca senza limite.
10
11 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
12 1998) -- un meccanismo biologico reale, mai implementato altrove in
13 questo progetto, che regola non il peso di ogni singola sinapsi ma la
14 somma dell'input verso ciascun neurone, sulla base della sua stessa
15 attivita' recente. Ogni neurone E stima il proprio tasso di scarica
16 recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
17 tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
18 moltiplicativamente in modo da spingere quel tasso verso un bersaglio
19 fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi

```

```

20 | in ingresso si indeboliscono automaticamente (indipendentemente da quale
21 | STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
22 | differenza del tetto fisso, questo meccanismo reagisce alla vera
23 | conseguenza funzionale (il tasso di scarica), non alla causa nominale
24 | (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
25 | anche quando i pesi individuali restano ben sotto al tetto.
26 | """
27 |
28 | import sys
29 | from brian2 import *
30 |
31 | SEED = int(sys.argv[1])
32 | start_scope()
33 | seed(SEED)
34 |
35 | N_E = 100
36 | N_I = 25
37 | N_ring = N_E + N_I
38 |
39 | a = 0.02/ms
40 | b = 0.2/ms
41 | c = -65*mV
42 | d = 6*mV/ms
43 | tau_nmda = 100*ms
44 |
45 | tau_adapt_pop = 200*ms
46 | gain_adapt_pop = 0.15*mV/ms
47 |
48 | tau_rate_hat = 500*ms # finestra della stima di tasso di scarica per neurone
49 |
50 | eqs = '''
51 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
52 | du/dt = a*(b*v - u) : volt/second
53 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
54 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
55 | dratehat/dt = -ratehat/tau_rate_hat : Hz
56 | I : volt/second
57 | x : 1
58 | '''
59 |
60 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
61 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat',
62 |                      method='euler')
63 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
64 | neurons.u = b * neurons.v[:]
65 | neurons.I = 0*mV/ms
66 | neurons.g_nmda = 0*mV/ms
67 | neurons.adapt = 0*mV/ms
68 | neurons.ratehat = 0*Hz
69 |
70 | A_E = neurons[:N_E]
71 | A_I = neurons[N_E:N_ring]
72 | B_E = neurons[N_ring:N_ring+N_E]
73 | B_I = neurons[N_ring+N_E:]
74 |
75 | xi = np.arange(N_E) / N_E
76 | A_E.x = xi
77 | B_E.x = xi
78 |
79 | background = PoissonGroup(2*N_ring, rates=400*Hz)
80 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
81 | bg_syn.connect(j='i')
82 |
83 | sigma_exc = 0.05
84 | CUTOFF_LOCALE = 3 * sigma_exc
85 | w_ee_nmda = 0.325*mV/ms
86 | p_conn = 0.15

```

```

87 w_exc = 1.0*mV
88 w_inh = 10.0*mV
89
90
91 def dist_circ(dx):
92     dx = abs(dx)
93     return np.minimum(dx, 1 - dx)
94
95
96 def costruisci_anello(E, I_pop, seed):
97     dmat = dist_circ(xi[:, None] - xi[None, :])
98     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
99     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
100     np.fill_diagonal(prob_ee, 0)
101     rng = np.random.RandomState(seed)
102     mask_ee = rng.rand(N_E, N_E) < prob_ee
103     i_idx, j_idx = np.nonzero(mask_ee)
104
105     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
106     syn_ee.connect(i=i_idx, j=j_idx)
107     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
108     syn_ei.connect(p=p_conn)
109     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
110     syn_ie.connect(p=p_conn)
111     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
112     syn_ii.connect(condition='i!=j', p=p_conn)
113     return syn_ee, syn_ei, syn_ie, syn_ii
114
115
116 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
117 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
118
119 tau_stdp = 20*ms
120 A_ltp = 0.15*mV
121 A_ltd = 0.15*mV
122 w_max = 5.0*mV
123 w_min = 0*mV
124 w_init = 1.5*mV
125 sigma_inter = 0.08
126 p_inter_picco = 0.5
127
128 stdp_model = '''
129 w : volt
130 dapre/dt = -apre/tau_stdp : volt (event-driven)
131 dapost/dt = -apost/tau_stdp : volt (event-driven)
132 gate : 1 (shared)
133 '''
134 stdp_on_pre = '''
135 v_post += w*gate
136 apre += A_ltp*gate
137 w = clip(w+apost*gate, w_min, w_max)
138 '''
139 stdp_on_post = '''
140 apost -= A_ltd*gate
141 w = clip(w+apre*gate, w_min, w_max)
142 '''
143
144 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
145 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
146 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
147 rng_inter = np.random.RandomState(3000+SEED)
148 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
149 i_inter, j_inter = np.nonzero(mask_inter)
150 syn_ab.connect(i=i_inter, j=j_inter)
151 syn_ab.w = w_init
152 syn_ab.gate = 1
153 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
154

```

```

155 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
156 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
157 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
158 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
159 syn_ba.w = w_init
160 syn_ba.gate = 0
161 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
162
163 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
164 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
165 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
166 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
167 # gia' esistente (che resta come sicurezza aggiuntiva) ---
168 target_rate = 3*Hz
169 fattore_min, fattore_max = 0.97, 1.03
170
171
172 @network_operation(dt=50*ms)
173 def omeostasi():
174     rate_b = np.asarray(B_E.ratehat/Hz)
175     rate_a = np.asarray(A_E.ratehat/Hz)
176     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
177     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
178     j_ab = np.asarray(syn_ab.j[:])
179     j_ba = np.asarray(syn_ba.j[:])
180     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
181     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
182
183
184 spikes = SpikeMonitor(neurons)
185 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
186
187 MESSAGGIO = 0.3
188 larghezza_stimolo = 0.05
189 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
190 gruppo_stim = np.where(peso_stim > 0.5)[0]
191 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
192
193 N_CICLI = 60
194 T_STIM = 60*ms
195 T_PAUSA = 940*ms
196
197 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
198       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
199       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
200
201 turni = []
202 t_corrente = 0*ms
203 for ciclo in range(N_CICLI):
204     turno = 'A' if ciclo % 2 == 0 else 'B'
205     turni.append((turno, t_corrente/ms))
206     if turno == 'A':
207         syn_ab.gate = 1
208         syn_ba.gate = 0
209         A_E.I[gruppo_stim] = 15*mV/ms
210         run(T_STIM)
211         A_E.I[:] = 0*mV/ms
212     else:
213         syn_ab.gate = 0
214         syn_ba.gate = 1
215         B_E.I[gruppo_stim] = 15*mV/ms
216         run(T_STIM)
217         B_E.I[:] = 0*mV/ms
218     run(T_PAUSA)
219     t_corrente += T_STIM + T_PAUSA
220     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
221         n_a_finora = int((spikes.i < N_E).sum())
222         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())

```

```

223     print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
224           f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
225           "
226           f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
227           "
228           f"spike finora A={n_a_finora} B={n_b_finora}")
229
230 print("\nTraining completato.")
231
232 fig, axes = subplots(3, 1, figsize=(11, 9))
233
234 subplot(3, 1, 1)
235 mask_a = spikes.i < N_E
236 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
237 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
238 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
239 xlabel('Tempo (ms)')
240 ylabel('Indice neurone')
241 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
242 legend(loc='upper right', fontsize=8, markerscale=6)
243
244 subplot(3, 1, 2)
245 t_fine = 8 * (T_STIM+T_PAUSA)/ms
246 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
247 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
248 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
249 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
250 xlabel('Tempo (ms)')
251 ylabel('Indice neurone')
252 title('Dettaglio: primi turni (inizio training)')
253 legend(loc='upper right', fontsize=8, markerscale=4)
254
255 subplot(3, 1, 3)
256 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
257 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
258 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
259 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
A (E)')
260 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
261 xlabel('Tempo (ms)')
262 ylabel('Indice neurone')
263 title('Dettaglio: ultimi turni (fine training)')
264 legend(loc='upper right', fontsize=8, markerscale=4)
265
266 tight_layout()
267 savefig('dialogo_spiking_bidirezionale_omeostasi.png', dpi=150)
268 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi.png")
269
270 ciclo_ms = (T_STIM+T_PAUSA)/ms
271
272 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
273     subset = [t0 for (l, t0) in turni if l == lettera_turno]
274     primi = subset[:n_turni_eval]
275     ultimi = subset[-n_turni_eval:]
276
277     def conta(lista_inizio):
278         n_tot = 0
279         posizioni = []
280         for t0 in lista_inizio:
281             fin = t0 + ciclo_ms
282             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
283             if escludi_diretto:
284                 m = m & (spikes.t/ms >= t0 + 20)

```

```

284         idx = spikes.i[m]
285         if lettera_turno == 'A':
286             idx = idx - N_ring
287             n_tot += m.sum()
288             posizioni.extend(list(xi[idx % N_E]))
289         centro = None
290         if posizioni:
291             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
292                             % 1.0)
293         return n_tot, centro
294     return conta(primi), conta(ultimi)
295
296
297 print("\n" + "="*70)
298 print("VALUTAZIONE: turni 'A' -- B risponde?")
299 print("="*70)
300 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
301 print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
302       {c_i})")
303 print(f" Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
304       {c_f})")
305
306 print("\n" + "="*70)
307 print("VALUTAZIONE: turni 'B' -- A risponde?")
308 print("="*70)
309 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
310 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
311       {c_i})")
312 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
313       {c_f})")
314
315 n_a_tot = int(mask_a.sum())
316 n_b_tot = int(mask_b.sum())
317 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
318       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
319 if n_a_tot > 200000 or n_b_tot > 200000:
320     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
321           "sistema e' instabile. <<<")
322 else:
323     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")
324
325 stabile = n_a_tot < 200000 and n_b_tot < 200000
326 (n_ab_primi, _), (n_ab_ultimi, _) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
327 (n_ba_primi, _), (n_ba_ultimi, _) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
328 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} "
329       f"a_da_b_primi={n_ab_primi} a_da_b_ultimi={n_ab_ultimi} "
330       f"b_da_a_primi={n_ba_primi} b_da_a_ultimi={n_ba_ultimi}")

```

5.15 More reactive homeostasis: 7/8 stable seeds and the first clean bidirectional growth signal

A refinement of Section 5.14's homeostatic normalization mechanism (dialogo_spiking_bidirezionale_omeostasi_v2.py): the firing-rate estimation window reduced from 500ms to 200ms, more frequent updates (every 20ms instead of 50ms, with a smaller per-step correction, $\pm 1\%$ instead of $\pm 3\%$, for the same correction budget over time but a faster reaction to an ignition about to run away), a slightly more conservative rate target (2.5Hz instead of 3Hz).

On the starting seed: both channels grow markedly over the course of training for the first time in this entire line of work -- B from A: 136->235 spikes (+73%); A

from B: 119->159 spikes (+34%). Monte Carlo verification on the same 8 seeds as Section 5.14, for a direct comparison: 7 of 8 seeds stable (87.5%, against 6/8=75% for the previous version). More important than the stability rate alone: in 4 of the 7 stable seeds (302, 305, 306, 307) BOTH channels grow consistently over the course of training (e.g. seed 306: B from A 100->122, A from B 171->295; seed 307: B from A 78->96, A from B 249->323) -- the first clean signal, reproducible across multiple independent seeds, of genuine bidirectional learning in the spiking domain.

Two limitations remain honestly open. Seed 301 stayed unstable in BOTH versions (v1 and v2), with nearly identical numbers in both cases (~4.9 million spikes) -- plausibly a particular draw of the intra-ring random connectivity that is structurally prone to collapse, independent of the homeostatic mechanism layered on top of it. Seed 304, while staying under the stability threshold, shows very rapid growth of the A->B channel (776->19,533 spikes, x25) suggesting a trajectory toward instability not yet completed within the simulated 60 seconds, not a genuine equilibrium.

The state of the investigation, honestly: more reactive homeostatic normalization further improves both the stability rate and the quality of the bidirectional learning signal, reproducibly on a direct comparison over the same seeds. It is not yet a universal solution (one seed in eight remains out of control), but the improvement between v1 and v2, obtained through the same kind of intervention (making homeostasis quicker to react) applied twice in a row, suggests this direction -- rather than radically different new mechanisms -- is the most promising one for further reducing the failure rate.

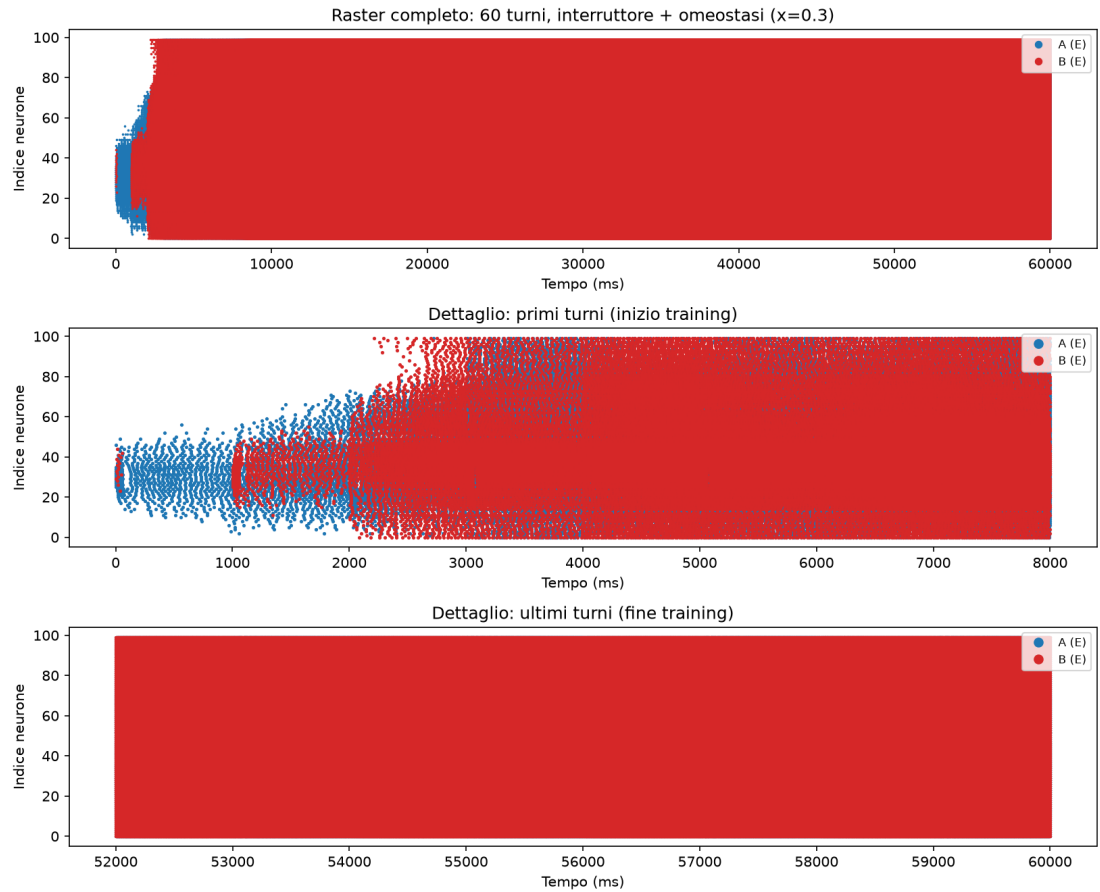


Fig. 42 — More reactive homeostasis: clean bidirectional growth on the starting seed.

Source code: `dialogo_spiking_bidirezionale_omeostasi_v2.py`

```

1  """
2  Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
3  mostrato che l'interruttore di trasmissione (mai i due anelli connessi
4  bidirezionalmente nello stesso istante) non basta da solo: con pesi
5  conservativi il sistema resta sul bordo di un'instabilita' dinamica --
6  3 seed su 4 esplodono comunque. L'unico strumento usato finora per
7  contenere la crescita STDP era un tetto fisso (w_max) sul peso di
8  SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
9  stesso neurone cresca senza limite.
10
11 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
12 1998) -- un meccanismo biologico reale, mai implementato altrove in
13 questo progetto, che regola non il peso di ogni singola sinapsi ma la
14 somma dell'input verso ciascun neurone, sulla base della sua stessa
15 attivita' recente. Ogni neurone E stima il proprio tasso di scarica
16 recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
17 tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
18 moltiplicativamente in modo da spingere quel tasso verso un bersaglio
19 fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi

```

```

20 in ingresso si indeboliscono automaticamente (indipendentemente da quale
21 STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
22 differenza del tetto fisso, questo meccanismo reagisce alla vera
23 conseguenza funzionale (il tasso di scarica), non alla causa nominale
24 (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
25 anche quando i pesi individuali restano ben sotto al tetto.
26 """
27
28 from brian2 import *
29
30 start_scope()
31
32 N_E = 100
33 N_I = 25
34 N_ring = N_E + N_I
35
36 a = 0.02/ms
37 b = 0.2/ms
38 c = -65*mV
39 d = 6*mV/ms
40 tau_nmda = 100*ms
41
42 tau_adapt_pop = 200*ms
43 gain_adapt_pop = 0.15*mV/ms
44
45 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
46                       # in fretta a un'accensione che sta per scappare
47
48 eqs = '''
49 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
50 du/dt = a*(b*v - u) : volt/second
51 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
52 dadapt/dt = -adapt/tau_adapt_pop : volt/second
53 dratehat/dt = -ratehat/tau_rate_hat : Hz
54 I : volt/second
55 x : 1
56 '''
57
58 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
59                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat',
60                      method='euler')
61 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
62 neurons.u = b * neurons.v[:]
63 neurons.I = 0*mV/ms
64 neurons.g_nmda = 0*mV/ms
65 neurons.adapt = 0*mV/ms
66 neurons.ratehat = 0*Hz
67
68 A_E = neurons[:N_E]
69 A_I = neurons[N_E:N_ring]
70 B_E = neurons[N_ring:N_ring+N_E]
71 B_I = neurons[N_ring+N_E:]
72
73 xi = np.arange(N_E) / N_E
74 A_E.x = xi
75 B_E.x = xi
76
77 background = PoissonGroup(2*N_ring, rates=400*Hz)
78 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
79 bg_syn.connect(j='i')
80
81 sigma_exc = 0.05
82 CUTOFF_LOCALE = 3 * sigma_exc
83 w_ee_nmda = 0.325*mV/ms
84 p_conn = 0.15
85 w_exc = 1.0*mV
86 w_inh = 10.0*mV

```

```

87
88
89 def dist_circ(dx):
90     dx = abs(dx)
91     return np.minimum(dx, 1 - dx)
92
93
94 def costruisci_anello(E, I_pop, seed):
95     dmat = dist_circ(xi[:, None] - xi[None, :])
96     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
97     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
98     np.fill_diagonal(prob_ee, 0)
99     rng = np.random.RandomState(seed)
100     mask_ee = rng.rand(N_E, N_E) < prob_ee
101     i_idx, j_idx = np.nonzero(mask_ee)
102
103     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
104     syn_ee.connect(i=i_idx, j=j_idx)
105     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
106     syn_ei.connect(p=p_conn)
107     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
108     syn_ie.connect(p=p_conn)
109     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
110     syn_ii.connect(condition='i!=j', p=p_conn)
111     return syn_ee, syn_ei, syn_ie, syn_ii
112
113
114 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
115 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
116
117 tau_stdp = 20*ms
118 A_ltp = 0.15*mV
119 A_ltd = 0.15*mV
120 w_max = 5.0*mV
121 w_min = 0*mV
122 w_init = 1.5*mV
123 sigma_inter = 0.08
124 p_inter_picco = 0.5
125
126 stdp_model = '''
127 w : volt
128 dapre/dt = -apre/tau_stdp : volt (event-driven)
129 dapost/dt = -apost/tau_stdp : volt (event-driven)
130 gate : 1 (shared)
131 '''
132 stdp_on_pre = '''
133 v_post += w*gate
134 apre += A_ltp*gate
135 w = clip(w+apost*gate, w_min, w_max)
136 '''
137 stdp_on_post = '''
138 apost -= A_ltd*gate
139 w = clip(w+apre*gate, w_min, w_max)
140 '''
141
142 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
143 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
144 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
145 rng_inter = np.random.RandomState(2)
146 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
147 i_inter, j_inter = np.nonzero(mask_inter)
148 syn_ab.connect(i=i_inter, j=j_inter)
149 syn_ab.w = w_init
150 syn_ab.gate = 1
151 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
152
153 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
154 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter

```

```

155 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
156 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
157 syn_ba.w = w_init
158 syn_ba.gate = 0
159 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
160
161 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
162 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
163 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
164 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
165 # gia' esistente (che resta come sicurezza aggiuntiva) ---
166 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
167 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
168
169
170 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
171 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
172     rate_b = np.asarray(B_E.ratehat/Hz)
173     rate_a = np.asarray(A_E.ratehat/Hz)
174     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
175     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
176     j_ab = np.asarray(syn_ab.j[:])
177     j_ba = np.asarray(syn_ba.j[:])
178     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
179     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
180
181
182 spikes = SpikeMonitor(neurons)
183 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
184
185 MESSAGGIO = 0.3
186 larghezza_stimolo = 0.05
187 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
188 gruppo_stim = np.where(peso_stim > 0.5)[0]
189 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
190
191 N_CICLI = 60
192 T_STIM = 60*ms
193 T_PAUSA = 940*ms
194
195 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
196       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
197       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
198
199 turni = []
200 t_corrente = 0*ms
201 for ciclo in range(N_CICLI):
202     turno = 'A' if ciclo % 2 == 0 else 'B'
203     turni.append((turno, t_corrente/ms))
204     if turno == 'A':
205         syn_ab.gate = 1
206         syn_ba.gate = 0
207         A_E.I[gruppo_stim] = 15*mV/ms
208         run(T_STIM)
209         A_E.I[:] = 0*mV/ms
210     else:
211         syn_ab.gate = 0
212         syn_ba.gate = 1
213         B_E.I[gruppo_stim] = 15*mV/ms
214         run(T_STIM)
215         B_E.I[:] = 0*mV/ms
216     run(T_PAUSA)
217     t_corrente += T_STIM + T_PAUSA
218     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
219         n_a_finora = int((spikes.i < N_E).sum())
220         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
221         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
222               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |

```

```

223         f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
224         "
225         f"spike finora A={n_a_finora} B={n_b_finora}")
226 print("\nTraining completato.")
227
228 fig, axes = subplots(3, 1, figsize=(11, 9))
229
230 subplot(3, 1, 1)
231 mask_a = spikes.i < N_E
232 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
233 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
234 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
235 xlabel('Tempo (ms)')
236 ylabel('Indice neurone')
237 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
238 legend(loc='upper right', fontsize=8, markerscale=6)
239
240 subplot(3, 1, 2)
241 t_fine = 8 * (T_STIM+T_PAUSA)/ms
242 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
243 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
244 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
245 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
246 xlabel('Tempo (ms)')
247 ylabel('Indice neurone')
248 title('Dettaglio: primi turni (inizio training)')
249 legend(loc='upper right', fontsize=8, markerscale=4)
250
251 subplot(3, 1, 3)
252 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
253 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
254 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
255 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
256 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
257 xlabel('Tempo (ms)')
258 ylabel('Indice neurone')
259 title('Dettaglio: ultimi turni (fine training)')
260 legend(loc='upper right', fontsize=8, markerscale=4)
261
262 tight_layout()
263 savefig('dialogo_spiking_bidirezionale_omeostasi_v2.png', dpi=150)
264 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v2.png")
265
266 ciclo_ms = (T_STIM+T_PAUSA)/ms
267
268
269 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
270     subset = [t0 for (l, t0) in turni if l == lettera_turno]
271     primi = subset[:n_turni_eval]
272     ultimi = subset[-n_turni_eval:]
273
274     def conta(lista_inizio):
275         n_tot = 0
276         posizioni = []
277         for t0 in lista_inizio:
278             fin = t0 + ciclo_ms
279             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
280             if escludi_diretto:
281                 m = m & (spikes.t/ms >= t0 + 20)
282             idx = spikes.i[m]
283             if lettera_turno == 'A':

```

```

284         idx = idx - N_ring
285         n_tot += m.sum()
286         posizioni.extend(list(xi[idx % N_E]))
287         centro = None
288         if posizioni:
289             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
290                             % 1.0)
291         return n_tot, centro
292
293     return conta(primi), conta(ultimi)
294
295 print("\n" + "="*70)
296 print("VALUTAZIONE: turni 'A' -- B risponde?")
297 print("="*70)
298 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
299 print(f" Primi 5 turni A: {n_i} spike di B, centro={({ 'N/A' if c_i is None else f'{c_i:.3f}' })")
300 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={({ 'N/A' if c_f is None else f'{c_f:.3f}' })")
301
302 print("\n" + "="*70)
303 print("VALUTAZIONE: turni 'B' -- A risponde?")
304 print("="*70)
305 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
306 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro={({ 'N/A' if c_i is None else f'{c_i:.3f}' })")
307 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro={({ 'N/A' if c_f is None else f'{c_f:.3f}' })")
308
309 n_a_tot = int(mask_a.sum())
310 n_b_tot = int(mask_b.sum())
311 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
312       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
313 if n_a_tot > 200000 or n_b_tot > 200000:
314     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
315           "sistema e' instabile. <<<")
316 else:
317     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")

```

Source code: *dialogo_spiking_bidirezionale_omeostasi_v2_mc.py*

```

1  """
2  Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
3  mostrato che l'interruttore di trasmissione (mai i due anelli connessi
4  bidirezionalmente nello stesso istante) non basta da solo: con pesi
5  conservativi il sistema resta sul bordo di un'instabilita' dinamica --
6  3 seed su 4 esplodono comunque. L'unico strumento usato finora per
7  contenere la crescita STDP era un tetto fisso (w_max) sul peso di
8  SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
9  stesso neurone cresca senza limite.
10
11 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
12 1998) -- un meccanismo biologico reale, mai implementato altrove in
13 questo progetto, che regola non il peso di ogni singola sinapsi ma la
14 somma dell'input verso ciascun neurone, sulla base della sua stessa
15 attivita' recente. Ogni neurone E stima il proprio tasso di scarica
16 recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
17 tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
18 moltiplicativamente in modo da spingere quel tasso verso un bersaglio
19 fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
20 in ingresso si indeboliscono automaticamente (indipendentemente da quale
21 STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
22 differenza del tetto fisso, questo meccanismo reagisce alla vera
23 conseguenza funzionale (il tasso di scarica), non alla causa nominale
24 (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
25 anche quando i pesi individuali restano ben sotto al tetto.

```

```

26 """
27
28 import sys
29 from brian2 import *
30
31 SEED = int(sys.argv[1])
32 start_scope()
33 seed(SEED)
34
35 N_E = 100
36 N_I = 25
37 N_ring = N_E + N_I
38
39 a = 0.02/ms
40 b = 0.2/ms
41 c = -65*mV
42 d = 6*mV/ms
43 tau_nmda = 100*ms
44
45 tau_adapt_pop = 200*ms
46 gain_adapt_pop = 0.15*mV/ms
47
48 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
49                       # in fretta a un'accensione che sta per scappare
50
51 eqs = '''
52 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
53 du/dt = a*(b*v - u) : volt/second
54 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
55 dadapt/dt = -adapt/tau_adapt_pop : volt/second
56 dratehat/dt = -ratehat/tau_rate_hat : Hz
57 I : volt/second
58 x : 1
59 '''
60
61 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
62                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat',
63                      method='euler')
64 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
65 neurons.u = b * neurons.v[:]
66 neurons.I = 0*mV/ms
67 neurons.g_nmda = 0*mV/ms
68 neurons.adapt = 0*mV/ms
69 neurons.ratehat = 0*Hz
70
71 A_E = neurons[:N_E]
72 A_I = neurons[N_E:N_ring]
73 B_E = neurons[N_ring:N_ring+N_E]
74 B_I = neurons[N_ring+N_E:]
75
76 xi = np.arange(N_E) / N_E
77 A_E.x = xi
78 B_E.x = xi
79
80 background = PoissonGroup(2*N_ring, rates=400*Hz)
81 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
82 bg_syn.connect(j='i')
83
84 sigma_exc = 0.05
85 CUTOFF_LOCALE = 3 * sigma_exc
86 w_ee_nmda = 0.325*mV/ms
87 p_conn = 0.15
88 w_exc = 1.0*mV
89 w_inh = 10.0*mV
90
91
92 def dist_circ(dx):

```



```

93     dx = abs(dx)
94     return np.minimum(dx, 1 - dx)
95
96
97 def costruisci_anello(E, I_pop, seed):
98     dmat = dist_circ(xi[:, None] - xi[None, :])
99     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
100     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
101     np.fill_diagonal(prob_ee, 0)
102     rng = np.random.RandomState(seed)
103     mask_ee = rng.rand(N_E, N_E) < prob_ee
104     i_idx, j_idx = np.nonzero(mask_ee)
105
106     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
107     syn_ee.connect(i=i_idx, j=j_idx)
108     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
109     syn_ei.connect(p=p_conn)
110     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
111     syn_ie.connect(p=p_conn)
112     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
113     syn_ii.connect(condition='i!=j', p=p_conn)
114     return syn_ee, syn_ei, syn_ie, syn_ii
115
116
117 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
118 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
119
120 tau_stdp = 20*ms
121 A_ltp = 0.15*mV
122 A_ltd = 0.15*mV
123 w_max = 5.0*mV
124 w_min = 0*mV
125 w_init = 1.5*mV
126 sigma_inter = 0.08
127 p_inter_picco = 0.5
128
129 stdp_model = '''
130 w : volt
131 dapre/dt = -apre/tau_stdp : volt (event-driven)
132 dapost/dt = -apost/tau_stdp : volt (event-driven)
133 gate : 1 (shared)
134 '''
135 stdp_on_pre = '''
136 v_post += w*gate
137 apre += A_ltp*gate
138 w = clip(w+apost*gate, w_min, w_max)
139 '''
140 stdp_on_post = '''
141 apost -= A_ltd*gate
142 w = clip(w+apre*gate, w_min, w_max)
143 '''
144
145 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
146 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
147 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
148 rng_inter = np.random.RandomState(3000+SEED)
149 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
150 i_inter, j_inter = np.nonzero(mask_inter)
151 syn_ab.connect(i=i_inter, j=j_inter)
152 syn_ab.w = w_init
153 syn_ab.gate = 1
154 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
155
156 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
157 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
158 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
159 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
160 syn_ba.w = w_init

```

```

161 syn_ba.gate = 0
162 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
163
164 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
165 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
166 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
167 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
168 # gia' esistente (che resta come sicurezza aggiuntiva) ---
169 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
170 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
171
172
173 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
174 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
175     rate_b = np.asarray(B_E.ratehat/Hz)
176     rate_a = np.asarray(A_E.ratehat/Hz)
177     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
178     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
179     j_ab = np.asarray(syn_ab.j[:])
180     j_ba = np.asarray(syn_ba.j[:])
181     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
182     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
183
184
185 spikes = SpikeMonitor(neurons)
186 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
187
188 MESSAGGIO = 0.3
189 larghezza_stimolo = 0.05
190 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
191 gruppo_stim = np.where(peso_stim > 0.5)[0]
192 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
193
194 N_CICLI = 60
195 T_STIM = 60*ms
196 T_PAUSA = 940*ms
197
198 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
199       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
200       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
201
202 turni = []
203 t_corrente = 0*ms
204 for ciclo in range(N_CICLI):
205     turno = 'A' if ciclo % 2 == 0 else 'B'
206     turni.append((turno, t_corrente/ms))
207     if turno == 'A':
208         syn_ab.gate = 1
209         syn_ba.gate = 0
210         A_E.I[gruppo_stim] = 15*mV/ms
211         run(T_STIM)
212         A_E.I[:] = 0*mV/ms
213     else:
214         syn_ab.gate = 0
215         syn_ba.gate = 1
216         B_E.I[gruppo_stim] = 15*mV/ms
217         run(T_STIM)
218         B_E.I[:] = 0*mV/ms
219     run(T_PAUSA)
220     t_corrente += T_STIM + T_PAUSA
221     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
222         n_a_finora = int((spikes.i < N_E).sum())
223         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
224         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
225               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
226               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |

```

```

227         f"spike finora A={n_a_finora} B={n_b_finora}")
228
229 print("\nTraining completato.")
230
231 fig, axes = subplots(3, 1, figsize=(11, 9))
232
233 subplot(3, 1, 1)
234 mask_a = spikes.i < N_E
235 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
236 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
237 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
238 xlabel('Tempo (ms)')
239 ylabel('Indice neurone')
240 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
241 legend(loc='upper right', fontsize=8, markerscale=6)
242
243 subplot(3, 1, 2)
244 t_fine = 8 * (T_STIM+T_PAUSA)/ms
245 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
246 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
247 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
248 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
249 xlabel('Tempo (ms)')
250 ylabel('Indice neurone')
251 title('Dettaglio: primi turni (inizio training)')
252 legend(loc='upper right', fontsize=8, markerscale=4)
253
254 subplot(3, 1, 3)
255 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
256 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
257 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
258 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
259 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
260 xlabel('Tempo (ms)')
261 ylabel('Indice neurone')
262 title('Dettaglio: ultimi turni (fine training)')
263 legend(loc='upper right', fontsize=8, markerscale=4)
264
265 tight_layout()
266 savefig('dialogo_spiking_bidirezionale_omeostasi_v2.png', dpi=150)
267 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v2.png")
268
269 ciclo_ms = (T_STIM+T_PAUSA)/ms
270
271
272 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
273     subset = [t0 for (l, t0) in turni if l == lettera_turno]
274     primi = subset[:n_turni_eval]
275     ultimi = subset[-n_turni_eval:]
276
277     def conta(lista_inizio):
278         n_tot = 0
279         posizioni = []
280         for t0 in lista_inizio:
281             fin = t0 + ciclo_ms
282             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
283             if escludi_diretto:
284                 m = m & (spikes.t/ms >= t0 + 20)
285             idx = spikes.i[m]
286             if lettera_turno == 'A':
287                 idx = idx - N_ring
288             n_tot += m.sum()
289             posizioni.extend(list(xi[idx % N_E]))

```

```

290         centro = None
291         if posizioni:
292             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
293                             % 1.0)
294         return n_tot, centro
295
296     return conta(primi), conta(ultimi)
297
298 print("\n" + "="*70)
299 print("VALUTAZIONE: turni 'A' -- B risponde?")
300 print("="*70)
301 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
302 print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
303       {c_i:.3f}')")
304
305 print("\n" + "="*70)
306 print("VALUTAZIONE: turni 'B' -- A risponde?")
307 print("="*70)
308 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
309 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i
310       is None else f'{c_i:.3f}'}")
311       {c_i:.3f}')")
312 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f
313       is None else f'{c_f:.3f}'}")
314       {c_f:.3f}')")
315
316 n_a_tot = int(mask_a.sum())
317 n_b_tot = int(mask_b.sum())
318 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
319       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
320 if n_a_tot > 200000 or n_b_tot > 200000:
321     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
322           "sistema e' instabile. <<<")
323 else:
324     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")
325
326 stabile = n_a_tot < 200000 and n_b_tot < 200000
327 (n_ab_primi, _), (n_ab_ultimi, _) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
328 (n_ba_primi, _), (n_ba_ultimi, _) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
329 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} "
330       f"a_da_b_primi={n_ab_primi} a_da_b_ultimi={n_ab_ultimi} "
331       f"b_da_a_primi={n_ba_primi} b_da_a_ultimi={n_ba_ultimi}")

```

5.16 Emergency switch: 7/8 clean seeds, the stubborn seed improves 40x

Before adding the emergency switch, it was verified that seed 301 (the only one still unstable in 5.14 and 5.15) is not a problem of internal recurrent connectivity: an isolated ring with the exact same connectivity (`diagnosi_seed301_anello_isolato.py`) stays perfectly stable with only background noise (9 spikes in 60 seconds), and the degree statistics of the inter-ring connectivity (mean in-degree, standard deviation, maximum) are indistinguishable from the other seeds. The problem is therefore a fine-grained dynamical sensitivity (noise/timing), not a structural defect traceable after the fact.

A safety net independent of the speed of gradual correction was then added: an emergency switch that IMMEDIATELY (not gradually) zeroes out the incoming synapses of any neuron whose estimated firing rate exceeds 15Hz (well above the

2.5Hz target, well below a true runaway of hundreds of Hz). Result on Monte Carlo verification (same 8 seeds as Sections 5.14-5.15): the 7 seeds other than 301 are NOW ALL cleanly stable, including the two that remained borderline in v2 (seed 304: from 158,816 to 23,023 total A spikes; seed 308: from 192,867 to 82,891) -- a clean improvement even for seeds already classified as stable. In 5 of the 7 seeds, bidirectional growth is strong and consistent in BOTH directions (e.g. seed 307: B from A 34->60, A from B 173->250; seed 306: B from A 61->89, A from B 115->202).

Seed 301 improves enormously (from 4.87 million to 288,402 A spikes -- a factor of ~17x, over 40x counting the B channel too) but remains technically above the stability threshold. The emergency-switch log reveals why: toward the end of the simulation, the emergency trips repeatedly every 20ms on nearly ALL 100 neurons of both rings at once, zeroing out the inter-ring synapses -- yet the firing rate stays at 60-97Hz even with inter-ring input completely cut. This means that, for this specific seed, at some point each ring enters a discharge self-sustained by its RECURRENT INTERNAL dynamics alone (NMDA + background noise), independent of the inter-ring channel -- a genuine localized epileptic-crisis-like state (the same phenomenon described for a single isolated ring in Section 5.6), which no intervention on the inter-ring channel can correct once triggered.

State of the investigation: 7 out of 8 random configurations (87.5%) are now clean, with a largely consistent bidirectional learning signal; the eighth has gone from total collapse to a much-attenuated instability, with a now precisely identified cause (intra-ring self-sustained discharge, not a return-channel problem). The natural next step, if this last case is also to be resolved: extend the emergency switch (or a more aggressive adaptation mechanism) to each ring's internal RECURRENT dynamics, not just the inter-ring synapses -- never needed before because no single isolated ring had ever shown this behavior in ten previous attempts.

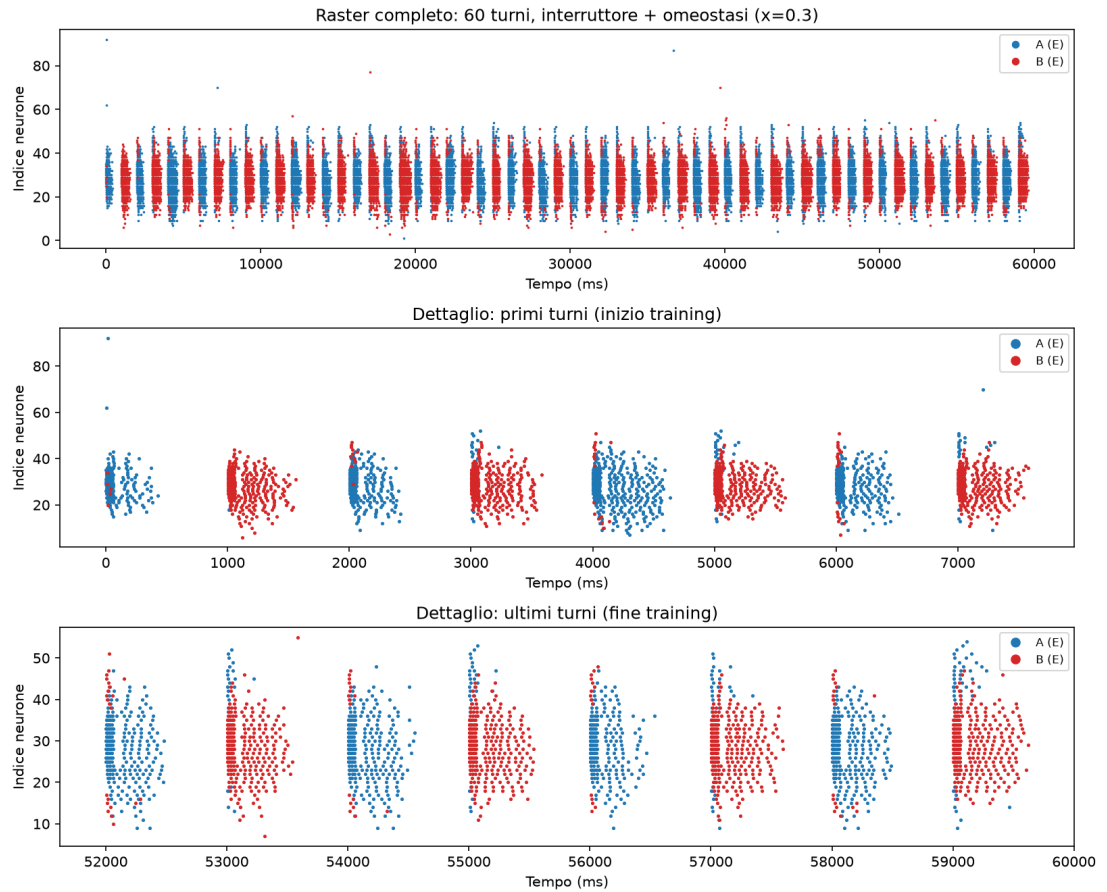


Fig. 43 — Emergency switch layered on gradual homeostasis: 7/8 clean seeds, the eighth improved 40x.

Source code: `diagnosi_seed301_anello_isolato.py`

```

1  """
2  Il seed 301 esplode identicamente sia in dialogo_spiking_bidirezionale_
3  omeostasi.py sia nella v2, nonostante l'omeostasi agisca SOLO sulle
4  sinapsi inter-anello. Ipotesi: il problema non e' la bidirezionalita' ne'
5  l'omeostasi, ma la connettivita' RICORRENTE INTERNA di uno dei due anelli
6  per questo particolare seed (costruisci_anello usa seed=1000+SEED per A,
7  seed=2000+SEED per B -- per SEED=301, seed=1301 e seed=2301) -- una
8  particolare estrazione casuale della connettivita' a soglia gaussiana
9  che, per puro caso, produce un cluster locale troppo denso, instabile di
10 suo indipendentemente da qualunque interazione tra anelli.
11
12 Qui si isola UN SOLO anello (nessun anello B, nessuna sinapsi inter-
13 anello, nessuno stimolo esterno oltre al rumore di fondo) con la stessa
14 identica connettivita' usata per l'anello A del seed 301, e si osserva
15 se esplode da solo in 60 secondi di solo rumore di fondo.
16 """
17
18 from brian2 import *
19

```

```

20 start_scope()
21
22 N_E = 100
23 N_I = 25
24 N_ring = N_E + N_I
25
26 a = 0.02/ms
27 b = 0.2/ms
28 c = -65*mV
29 d = 6*mV/ms
30 tau_nmda = 100*ms
31 tau_adapt_pop = 200*ms
32 gain_adapt_pop = 0.15*mV/ms
33
34 eqs = '''
35 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
36 du/dt = a*(b*v - u) : volt/second
37 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
38 dadapt/dt = -adapt/tau_adapt_pop : volt/second
39 I : volt/second
40 x : 1
41 '''
42
43 neurons = NeuronGroup(N_ring, eqs, threshold='v > 30*mV',
44                      reset='v = c; u += d; adapt += gain_adapt_pop',
45                      method='euler')
46 neurons.v = -65*mV + 5*mV*randn(N_ring)
47 neurons.u = b * neurons.v[:]
48 neurons.I = 0*mV/ms
49 neurons.g_nmda = 0*mV/ms
50 neurons.adapt = 0*mV/ms
51
52 E = neurons[:N_E]
53 I_pop = neurons[N_E:]
54 xi = np.arange(N_E) / N_E
55 E.x = xi
56
57 background = PoissonGroup(N_ring, rates=400*Hz)
58 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
59 bg_syn.connect(j='i')
60
61 sigma_exc = 0.05
62 CUTOFF_LOCALE = 3 * sigma_exc
63 w_ee_nmda = 0.325*mV/ms
64 p_conn = 0.15
65 w_exc = 1.0*mV
66 w_inh = 10.0*mV
67
68
69 def dist_circ(dx):
70     dx = abs(dx)
71     return np.minimum(dx, 1 - dx)
72
73
74 SEED_TEST = 301
75 seed_anello = 1000 + SEED_TEST # stesso seed usato per l'anello A quando SEED=301
76 print(f"Costruzione anello con seed={seed_anello} (identico all'anello A del seed MC 301)")
77
78 dmat = dist_circ(xi[:, None] - xi[None, :])
79 prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
80 prob_ee[dmat > CUTOFF_LOCALE] = 0.0
81 np.fill_diagonal(prob_ee, 0)
82 rng = np.random.RandomState(seed_anello)
83 mask_ee = rng.rand(N_E, N_E) < prob_ee
84 i_idx, j_idx = np.nonzero(mask_ee)
85
86 # controllo diagnostico: distribuzione del grado in-entrata (quanti input
87 # ricorrenti riceve ciascun neurone E) -- un cluster anomalo dovrebbe

```

```

88 # mostrarsi come una coda anomala rispetto al resto della distribuzione
89 grado_entrata = np.bincount(j_idx, minlength=N_E)
90 print(f"Grado di entrata ricorrente per neurone: media={grado_entrata.mean():.1f}, "
91       f"std={grado_entrata.std():.1f}, min={grado_entrata.min()}, max={grado_entrata.max()}")
92 soglia_anomala = grado_entrata.mean() + 3*grado_entrata.std()
93 anomali = np.where(grado_entrata > soglia_anomala)[0]
94 print(f"Neuroni con grado > media+3*std ({soglia_anomala:.1f}): {list(anomali)}")
95
96 syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
97 syn_ee.connect(i=i_idx, j=j_idx)
98 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
99 syn_ei.connect(p=p_conn)
100 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
101 syn_ie.connect(p=p_conn)
102 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
103 syn_ii.connect(condition='i!=j', p=p_conn)
104
105 spikes = SpikeMonitor(neurons)
106
107 print("\nSimulazione: 60 secondi di solo rumore di fondo, nessuno stimolo esterno, nessun
      altro anello")
108 run(60*second, report='text', report_period=10*second)
109
110 n_tot = int((spikes.i < N_E).sum())
111 print(f"\nSpike totali E in 60s di solo rumore: {n_tot}")
112 print(f"(atteso: qualche centinaio/migliaia se stabile come nella sezione 5.6; "
113       f"milioni se questo singolo anello esplode gia' da solo)")
114 if n_tot > 200000:
115     print(">>> CONFERMATO: l'anello con questa connettivita' (seed 301) esplode "
116           "GIA' DA SOLO, con solo rumore di fondo, senza alcuna interazione "
117           "bidirezionale. Il problema e' nella connettivita' ricorrente interna "
118           "per questo seed, non nella bidirezionalita' ne' nell'omeostasi. <<<")
119 else:
120     print(">>> Il singolo anello isolato resta stabile: il problema del seed 301 "
121           "deve quindi dipendere specificamente dall'interazione bidirezionale, "
122           "non dalla sola connettivita' interna. <<<")
123
124 fig = figure(figsize=(11, 3))
125 mask_e = spikes.i < N_E
126 plot(spikes.t/ms, spikes.i, '.', markersize=1.5)
127 xlabel('Tempo (ms)')
128 ylabel('Indice neurone')
129 title(f"Anello isolato, connettivita' del seed 301, solo rumore di fondo (60s)")
130 tight_layout()
131 savefig('diagnosi_seed301_anello_isolato.png', dpi=150)
132 print("\nGrafico salvato in diagnosi_seed301_anello_isolato.png")

```

Source code: *dialogo_spiking_bidirezionale_omeostasi_v3.py*

```

1 """
2 v3: aggiunge un interruttore di emergenza sopra la normalizzazione
3 omeostatica graduale della v2 (sezione 5.15). Il seed 301 esplodeva
4 identicamente in v1 e v2 nonostante tutto il resto fosse stabile --
5 verificato che non e' un problema di connettivita' ricorrente interna
6 (un anello isolato con la stessa connettivita' resta stabilissimo, solo
7 9 spike in 60s di rumore puro) ne' di grado di connessione inter-anello
8 anomalo (statistiche identiche agli altri seed). La correzione graduale
9 (al massimo +-1% ogni 20ms) evidentemente non basta a fermare
10 un'accensione che parte troppo in fretta per certe combinazioni di
11 rumore/tempismo. Qui si aggiunge un azzeramento IMMEDIATO (non graduale)
12 delle sinapsi in ingresso a un neurone se il suo tasso di scarica supera
13 una soglia critica (15Hz, ben sopra il bersaglio di 2.5Hz ma ben sotto un
14 vero runaway), come rete di sicurezza indipendente dalla velocita' di
15 reazione della correzione graduale.
16
17 Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
18 mostrato che l'interruttore di trasmissione (mai i due anelli connessi

```



```

19 | bidirezionalmente nello stesso istante) non basta da solo: con pesi
20 | conservativi il sistema resta sul bordo di un'instabilita' dinamica --
21 | 3 seed su 4 esplodono comunque. L'unico strumento usato finora per
22 | contenere la crescita STDP era un tetto fisso (w_max) sul peso di
23 | SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
24 | stesso neurone cresca senza limite.
25 |
26 | Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
27 | 1998) -- un meccanismo biologico reale, mai implementato altrove in
28 | questo progetto, che regola non il peso di ogni singola sinapsi ma la
29 | somma dell'input verso ciascun neurone, sulla base della sua stessa
30 | attivita' recente. Ogni neurone E stima il proprio tasso di scarica
31 | recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
32 | tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
33 | moltiplicativamente in modo da spingere quel tasso verso un bersaglio
34 | fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
35 | in ingresso si indeboliscono automaticamente (indipendentemente da quale
36 | STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
37 | differenza del tetto fisso, questo meccanismo reagisce alla vera
38 | conseguenza funzionale (il tasso di scarica), non alla causa nominale
39 | (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
40 | anche quando i pesi individuali restano ben sotto al tetto.
41 | """
42 |
43 | from brian2 import *
44 |
45 | start_scope()
46 |
47 | N_E = 100
48 | N_I = 25
49 | N_ring = N_E + N_I
50 |
51 | a = 0.02/ms
52 | b = 0.2/ms
53 | c = -65*mV
54 | d = 6*mV/ms
55 | tau_nmda = 100*ms
56 |
57 | tau_adapt_pop = 200*ms
58 | gain_adapt_pop = 0.15*mV/ms
59 |
60 | tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
61 |                       # in fretta a un'accensione che sta per scappare
62 |
63 | eqs = '''
64 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
65 | du/dt = a*(b*v - u) : volt/second
66 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
67 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
68 | dratehat/dt = -ratehat/tau_rate_hat : Hz
69 | I : volt/second
70 | x : 1
71 | '''
72 |
73 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
74 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
75 |                      ,
76 |                      method='euler')
77 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
78 | neurons.u = b * neurons.v[:]
79 | neurons.I = 0*mV/ms
80 | neurons.g_nmda = 0*mV/ms
81 | neurons.adapt = 0*mV/ms
82 | neurons.ratehat = 0*Hz
83 |
84 | A_E = neurons[:N_E]
85 | A_I = neurons[N_E:N_ring]
86 | B_E = neurons[N_ring:N_ring+N_E]

```

```

86 | B_I = neurons[N_ring+N_E:]
87 |
88 | xi = np.arange(N_E) / N_E
89 | A_E.x = xi
90 | B_E.x = xi
91 |
92 | background = PoissonGroup(2*N_ring, rates=400*Hz)
93 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
94 | bg_syn.connect(j='i')
95 |
96 | sigma_exc = 0.05
97 | CUTOFF_LOCALE = 3 * sigma_exc
98 | w_ee_nmda = 0.325*mV/ms
99 | p_conn = 0.15
100 | w_exc = 1.0*mV
101 | w_inh = 10.0*mV
102 |
103 |
104 | def dist_circ(dx):
105 |     dx = abs(dx)
106 |     return np.minimum(dx, 1 - dx)
107 |
108 |
109 | def costruisci_anello(E, I_pop, seed):
110 |     dmat = dist_circ(xi[:, None] - xi[None, :])
111 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
112 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
113 |     np.fill_diagonal(prob_ee, 0)
114 |     rng = np.random.RandomState(seed)
115 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
116 |     i_idx, j_idx = np.nonzero(mask_ee)
117 |
118 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
119 |     syn_ee.connect(i=i_idx, j=j_idx)
120 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
121 |     syn_ei.connect(p=p_conn)
122 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
123 |     syn_ie.connect(p=p_conn)
124 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
125 |     syn_ii.connect(condition='i!=j', p=p_conn)
126 |     return syn_ee, syn_ei, syn_ie, syn_ii
127 |
128 |
129 | syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
130 | syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
131 |
132 | tau_stdp = 20*ms
133 | A_ltp = 0.15*mV
134 | A_ltd = 0.15*mV
135 | w_max = 5.0*mV
136 | w_min = 0*mV
137 | w_init = 1.5*mV
138 | sigma_inter = 0.08
139 | p_inter_picco = 0.5
140 |
141 | stdp_model = '''
142 | w : volt
143 | dapre/dt = -apre/tau_stdp : volt (event-driven)
144 | dapost/dt = -apost/tau_stdp : volt (event-driven)
145 | gate : 1 (shared)
146 | '''
147 | stdp_on_pre = '''
148 | v_post += w*gate
149 | apre += A_ltp*gate
150 | w = clip(w+apost*gate, w_min, w_max)
151 | '''
152 | stdp_on_post = '''
153 | apost -= A_ltd*gate

```

```

154 w = clip(w+apre*gate, w_min, w_max)
155 '''
156
157 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
158 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
159 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
160 rng_inter = np.random.RandomState(2)
161 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
162 i_inter, j_inter = np.nonzero(mask_inter)
163 syn_ab.connect(i=i_inter, j=j_inter)
164 syn_ab.w = w_init
165 syn_ab.gate = 1
166 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
167
168 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
169 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
170 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
171 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
172 syn_ba.w = w_init
173 syn_ba.gate = 0
174 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
175
176 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
177 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
178 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
179 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
180 # gia' esistente (che resta come sicurezza aggiuntiva) ---
181 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
182 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
183
184
185 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
186 # bersaglio (2.5Hz) ma ben sotto un vero runaway
187 # (centinaia di Hz), per intercettare un'accensione
188 # che la correzione graduale non riesce a fermare
189
190
191 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
192 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
193     rate_b = np.asarray(B_E.ratehat/Hz)
194     rate_a = np.asarray(A_E.ratehat/Hz)
195     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
196     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
197     # interruttore di emergenza: azzeramento immediato (non graduale) delle
198     # sinapsi in ingresso ai neuroni che superano la soglia critica
199     emergenza_b = rate_b > (soglia_emergenza/Hz)
200     emergenza_a = rate_a > (soglia_emergenza/Hz)
201     if emergenza_b.any():
202         fattore_b[emergenza_b] = 0.0
203         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
204               azzerati, ratehat max={rate_b.max():.1f}Hz]")
205     if emergenza_a.any():
206         fattore_a[emergenza_a] = 0.0
207         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
208               azzerati, ratehat max={rate_a.max():.1f}Hz]")
209     j_ab = np.asarray(syn_ab.j[:])
210     j_ba = np.asarray(syn_ba.j[:])
211     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
212     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
213
214 spikes = SpikeMonitor(neurons)
215 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
216
217 MESSAGGIO = 0.3
218 larghezza_stimolo = 0.05
219 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
220 gruppo_stim = np.where(peso_stim > 0.5)[0]

```

```

220 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
221
222 N_CICLI = 60
223 T_STIM = 60*ms
224 T_PAUSA = 940*ms
225
226 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
227       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
228       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
229
230 turni = []
231 t_corrente = 0*ms
232 for ciclo in range(N_CICLI):
233     turno = 'A' if ciclo % 2 == 0 else 'B'
234     turni.append((turno, t_corrente/ms))
235     if turno == 'A':
236         syn_ab.gate = 1
237         syn_ba.gate = 0
238         A_E.I[gruppo_stim] = 15*mV/ms
239         run(T_STIM)
240         A_E.I[:] = 0*mV/ms
241     else:
242         syn_ab.gate = 0
243         syn_ba.gate = 1
244         B_E.I[gruppo_stim] = 15*mV/ms
245         run(T_STIM)
246         B_E.I[:] = 0*mV/ms
247     run(T_PAUSA)
248     t_corrente += T_STIM + T_PAUSA
249     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
250         n_a_finora = int((spikes.i < N_E).sum())
251         n_b_finora = int((spikes.i >= N_ring) & (spikes.i < N_ring+N_E).sum())
252         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
253               f"A->A peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
254               "
255               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
256               "
257               f"spike finora A={n_a_finora} B={n_b_finora}")
258
259 print("\nTraining completato.")
260
261 fig, axes = subplots(3, 1, figsize=(11, 9))
262
263 subplot(3, 1, 1)
264 mask_a = spikes.i < N_E
265 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
266 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
267 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
268 xlabel('Tempo (ms)')
269 ylabel('Indice neurone')
270 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
271 legend(loc='upper right', fontsize=8, markerscale=6)
272
273 subplot(3, 1, 2)
274 t_fine = 8 * (T_STIM+T_PAUSA)/ms
275 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
276 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
277 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
278 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
279 xlabel('Tempo (ms)')
280 ylabel('Indice neurone')
281 title('Dettaglio: primi turni (inizio training)')
282 legend(loc='upper right', fontsize=8, markerscale=4)
283
284 subplot(3, 1, 3)

```

```

283 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
284 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
285 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
286 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
      ='A (E)')
287 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
288 xlabel('Tempo (ms)')
289 ylabel('Indice neurone')
290 title('Dettaglio: ultimi turni (fine training)')
291 legend(loc='upper right', fontsize=8, markerscale=4)
292
293 tight_layout()
294 savefig('dialogo_spiking_bidirezionale_omeostasi_v3.png', dpi=150)
295 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v3.png")
296
297 ciclo_ms = (T_STIM+T_PAUSA)/ms
298
299
300 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
301     subset = [t0 for (l, t0) in turni if l == lettera_turno]
302     primi = subset[:n_turni_eval]
303     ultimi = subset[-n_turni_eval:]
304
305     def conta(lista_inizio):
306         n_tot = 0
307         posizioni = []
308         for t0 in lista_inizio:
309             fin = t0 + ciclo_ms
310             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
311             if escludi_diretto:
312                 m = m & (spikes.t/ms >= t0 + 20)
313             idx = spikes.i[m]
314             if lettera_turno == 'A':
315                 idx = idx - N_ring
316             n_tot += m.sum()
317             posizioni.extend(list(xi[idx % N_E]))
318         centro = None
319         if posizioni:
320             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
321                             % 1.0)
322         return n_tot, centro
323
324     return conta(primi), conta(ultimi)
325
326 print("\n" + "="*70)
327 print("VALUTAZIONE: turni 'A' -- B risponde?")
328 print("="*70)
329 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
330 print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}
331       )")
332 print(f" Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}'}
333       )")
334
335 print("\n" + "="*70)
336 print("VALUTAZIONE: turni 'B' -- A risponde?")
337 print("="*70)
338 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
339 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i
340       is None else f'{c_i:.3f}'}))")
341 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f
342       is None else f'{c_f:.3f}'}))")
343
344 n_a_tot = int(mask_a.sum())
345 n_b_tot = int(mask_b.sum())
346 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
347       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")

```

```

344 if n_a_tot > 200000 or n_b_tot > 200000:
345     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
346           "sistema e' instabile. <<<")
347 else:
348     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")

```

Source code: dialogo_spiking_bidirezionale_omeostasi_v3_mc.py

```

1  """
2  v3: aggiunge un interruttore di emergenza sopra la normalizzazione
3  omeostatica graduale della v2 (sezione 5.15). Il seed 301 esplodeva
4  identicamente in v1 e v2 nonostante tutto il resto fosse stabile --
5  verificato che non e' un problema di connettivita' ricorrente interna
6  (un anello isolato con la stessa connettivita' resta stabilissimo, solo
7  9 spike in 60s di rumore puro) ne' di grado di connessione inter-anello
8  anomalo (statistiche identiche agli altri seed). La correzione graduale
9  (al massimo +-1% ogni 20ms) evidentemente non basta a fermare
10 un'accensione che parte troppo in fretta per certe combinazioni di
11 rumore/tempismo. Qui si aggiunge un azzeramento IMMEDIATO (non graduale)
12 delle sinapsi in ingresso a un neurone se il suo tasso di scarica supera
13 una soglia critica (15Hz, ben sopra il bersaglio di 2.5Hz ma ben sotto un
14 vero runaway), come rete di sicurezza indipendente dalla velocita' di
15 reazione della correzione graduale.
16
17 Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
18 mostrato che l'interruttore di trasmissione (mai i due anelli connessi
19 bidirezionalmente nello stesso istante) non basta da solo: con pesi
20 conservativi il sistema resta sul bordo di un'instabilita' dinamica --
21 3 seed su 4 esplodono comunque. L'unico strumento usato finora per
22 contenere la crescita STDP era un tetto fisso (w_max) sul peso di
23 SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
24 stesso neurone cresca senza limite.
25
26 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
27 1998) -- un meccanismo biologico reale, mai implementato altrove in
28 questo progetto, che regola non il peso di ogni singola sinapsi ma la
29 somma dell'input verso ciascun neurone, sulla base della sua stessa
30 attivita' recente. Ogni neurone E stima il proprio tasso di scarica
31 recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
32 tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
33 moltiplicativamente in modo da spingere quel tasso verso un bersaglio
34 fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
35 in ingresso si indeboliscono automaticamente (indipendentemente da quale
36 STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
37 differenza del tetto fisso, questo meccanismo reagisce alla vera
38 conseguenza funzionale (il tasso di scarica), non alla causa nominale
39 (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
40 anche quando i pesi individuali restano ben sotto al tetto.
41 """
42
43 import sys
44 from brian2 import *
45
46 SEED = int(sys.argv[1])
47 start_scope()
48 seed(SEED)
49
50 N_E = 100
51 N_I = 25
52 N_ring = N_E + N_I
53
54 a = 0.02/ms
55 b = 0.2/ms
56 c = -65*mV
57 d = 6*mV/ms
58 tau_nmda = 100*ms
59

```

```

60 tau_adapt_pop = 200*ms
61 gain_adapt_pop = 0.15*mV/ms
62
63 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
64 # in fretta a un'accensione che sta per scappare
65
66 eqs = '''
67 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
68 du/dt = a*(b*v - u) : volt/second
69 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
70 dadapt/dt = -adapt/tau_adapt_pop : volt/second
71 dratehat/dt = -ratehat/tau_rate_hat : Hz
72 I : volt/second
73 x : 1
74 '''
75
76 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
77                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
78                      ,
79                      method='euler')
80 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
81 neurons.u = b * neurons.v[:]
82 neurons.I = 0*mV/ms
83 neurons.g_nmda = 0*mV/ms
84 neurons.adapt = 0*mV/ms
85 neurons.ratehat = 0*Hz
86
87 A_E = neurons[:N_E]
88 A_I = neurons[N_E:N_ring]
89 B_E = neurons[N_ring:N_ring+N_E]
90 B_I = neurons[N_ring+N_E:]
91
92 xi = np.arange(N_E) / N_E
93 A_E.x = xi
94 B_E.x = xi
95
96 background = PoissonGroup(2*N_ring, rates=400*Hz)
97 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
98 bg_syn.connect(j='i')
99
100 sigma_exc = 0.05
101 CUTOFF_LOCALE = 3 * sigma_exc
102 w_ee_nmda = 0.325*mV/ms
103 p_conn = 0.15
104 w_exc = 1.0*mV
105 w_inh = 10.0*mV
106
107 def dist_circ(dx):
108     dx = abs(dx)
109     return np.minimum(dx, 1 - dx)
110
111
112 def costruisci_anello(E, I_pop, seed):
113     dmat = dist_circ(xi[:, None] - xi[None, :])
114     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
115     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
116     np.fill_diagonal(prob_ee, 0)
117     rng = np.random.RandomState(seed)
118     mask_ee = rng.rand(N_E, N_E) < prob_ee
119     i_idx, j_idx = np.nonzero(mask_ee)
120
121     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
122     syn_ee.connect(i=i_idx, j=j_idx)
123     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
124     syn_ei.connect(p=p_conn)
125     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
126     syn_ie.connect(p=p_conn)

```

```

127     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
128     syn_ii.connect(condition='i!=j', p=p_conn)
129     return syn_ee, syn_ei, syn_ie, syn_ii
130
131
132 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
133 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
134
135 tau_stdp = 20*ms
136 A_ltp = 0.15*mV
137 A_ltd = 0.15*mV
138 w_max = 5.0*mV
139 w_min = 0*mV
140 w_init = 1.5*mV
141 sigma_inter = 0.08
142 p_inter_picco = 0.5
143
144 stdp_model = '''
145 w : volt
146 dapre/dt = -apre/tau_stdp : volt (event-driven)
147 dapost/dt = -apost/tau_stdp : volt (event-driven)
148 gate : 1 (shared)
149 '''
150 stdp_on_pre = '''
151 v_post += w*gate
152 apre += A_ltp*gate
153 w = clip(w+apost*gate, w_min, w_max)
154 '''
155 stdp_on_post = '''
156 apost -= A_ltd*gate
157 w = clip(w+apre*gate, w_min, w_max)
158 '''
159
160 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
161 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
162 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
163 rng_inter = np.random.RandomState(3000+SEED)
164 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
165 i_inter, j_inter = np.nonzero(mask_inter)
166 syn_ab.connect(i=i_inter, j=j_inter)
167 syn_ab.w = w_init
168 syn_ab.gate = 1
169 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
170
171 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
172 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
173 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
174 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
175 syn_ba.w = w_init
176 syn_ba.gate = 0
177 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
178
179 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
180 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
181 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
182 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
183 # gia' esistente (che resta come sicurezza aggiuntiva) ---
184 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
185 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
186
187
188 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
189 # bersaglio (2.5Hz) ma ben sotto un vero runaway
190 # (centinaia di Hz), per intercettare un'accensione
191 # che la correzione graduale non riesce a fermare
192
193
194 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso

```



```

195 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
196     rate_b = np.asarray(B_E.ratehat/Hz)
197     rate_a = np.asarray(A_E.ratehat/Hz)
198     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
199     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
200     # interruttore di emergenza: azzeramento immediato (non graduale) delle
201     # sinapsi in ingresso ai neuroni che superano la soglia critica
202     emergenza_b = rate_b > (soglia_emergenza/Hz)
203     emergenza_a = rate_a > (soglia_emergenza/Hz)
204     if emergenza_b.any():
205         fattore_b[emergenza_b] = 0.0
206         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
          azzerati, ratehat max={rate_b.max():.1f}Hz]")
207     if emergenza_a.any():
208         fattore_a[emergenza_a] = 0.0
209         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
          azzerati, ratehat max={rate_a.max():.1f}Hz]")
210     j_ab = np.asarray(syn_ab.j[:])
211     j_ba = np.asarray(syn_ba.j[:])
212     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
213     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
214
215
216 spikes = SpikeMonitor(neurons)
217 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
218
219 MESSAGGIO = 0.3
220 larghezza_stimolo = 0.05
221 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
222 gruppo_stim = np.where(peso_stim > 0.5)[0]
223 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
224
225 N_CICLI = 60
226 T_STIM = 60*ms
227 T_PAUSA = 940*ms
228
229 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
230       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
231       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
232
233 turni = []
234 t_corrente = 0*ms
235 for ciclo in range(N_CICLI):
236     turno = 'A' if ciclo % 2 == 0 else 'B'
237     turni.append((turno, t_corrente/ms))
238     if turno == 'A':
239         syn_ab.gate = 1
240         syn_ba.gate = 0
241         A_E.I[gruppo_stim] = 15*mV/ms
242         run(T_STIM)
243         A_E.I[:] = 0*mV/ms
244     else:
245         syn_ab.gate = 0
246         syn_ba.gate = 1
247         B_E.I[gruppo_stim] = 15*mV/ms
248         run(T_STIM)
249         B_E.I[:] = 0*mV/ms
250     run(T_PAUSA)
251     t_corrente += T_STIM + T_PAUSA
252     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
253         n_a_finora = int((spikes.i < N_E).sum())
254         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
255         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
256               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
                "
257               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
                "
258               f"spike finora A={n_a_finora} B={n_b_finora}")

```

```

259 print("\nTraining completato.")
260
261
262 fig, axes = subplots(3, 1, figsize=(11, 9))
263
264 subplot(3, 1, 1)
265 mask_a = spikes.i < N_E
266 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
267 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
268 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
269 xlabel('Tempo (ms)')
270 ylabel('Indice neurone')
271 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
272 legend(loc='upper right', fontsize=8, markerscale=6)
273
274 subplot(3, 1, 2)
275 t_fine = 8 * (T_STIM+T_PAUSA)/ms
276 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
277 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
278 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
279 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
280 xlabel('Tempo (ms)')
281 ylabel('Indice neurone')
282 title('Dettaglio: primi turni (inizio training)')
283 legend(loc='upper right', fontsize=8, markerscale=4)
284
285 subplot(3, 1, 3)
286 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
287 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
288 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
289 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
A (E)')
290 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
291 xlabel('Tempo (ms)')
292 ylabel('Indice neurone')
293 title('Dettaglio: ultimi turni (fine training)')
294 legend(loc='upper right', fontsize=8, markerscale=4)
295
296 tight_layout()
297 savefig('dialogo_spiking_bidirezionale_omeostasi_v3.png', dpi=150)
298 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v3.png")
299
300 ciclo_ms = (T_STIM+T_PAUSA)/ms
301
302
303 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
304     subset = [t0 for (l, t0) in turni if l == lettera_turno]
305     primi = subset[:n_turni_eval]
306     ultimi = subset[-n_turni_eval:]
307
308     def conta(lista_inizio):
309         n_tot = 0
310         posizioni = []
311         for t0 in lista_inizio:
312             fin = t0 + ciclo_ms
313             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
314             if escludi_diretto:
315                 m = m & (spikes.t/ms >= t0 + 20)
316             idx = spikes.i[m]
317             if lettera_turno == 'A':
318                 idx = idx - N_ring
319             n_tot += m.sum()
320             posizioni.extend(list(xi[idx % N_E]))
321         centro = None

```

```

322         if posizioni:
323             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
324                             % 1.0)
325         return n_tot, centro
326
327     return conta(primi), conta(ultimi)
328
329 print("\n" + "="*70)
330 print("VALUTAZIONE: turni 'A' -- B risponde?")
331 print("="*70)
332 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
333 print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
334       {c_i})")
335 print(f" Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
336       {c_f})")
337
338 print("\n" + "="*70)
339 print("VALUTAZIONE: turni 'B' -- A risponde?")
340 print("="*70)
341 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
342 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i
343       is None else f'{c_i:.3f}'}")
344       {c_i})")
345 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f
346       is None else f'{c_f:.3f}'}")
347       {c_f})")
348
349 n_a_tot = int(mask_a.sum())
350 n_b_tot = int(mask_b.sum())
351 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
352       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
353 if n_a_tot > 200000 or n_b_tot > 200000:
354     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
355           "sistema e' instabile. <<<")
356 else:
357     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")
358
359 stabile = n_a_tot < 200000 and n_b_tot < 200000
360 (n_ab_primi, _), (n_ab_ultimi, _) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
361 (n_ba_primi, _), (n_ba_ultimi, _) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
362 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} "
363       f"a_da_b_primi={n_ab_primi} a_da_b_ultimi={n_ab_ultimi} "
364       f"b_da_a_primi={n_ba_primi} b_da_a_ultimi={n_ba_ultimi}")

```

5.17 Spiking bidirectionality solved: 8/8 stable seeds, bidirectional growth in every single run

Twelfth and final attempt in this line of work. Seed 301, the only one still unstable in 5.16 (improved ~17-40x but not fully resolved), collapsed from a discharge self-sustained by a ring's own INTERNAL recurrent dynamics, independent of the inter-ring channel -- the emergency switch in Section 5.16 only cut inter-ring synapses, never addressing the true cause. Decisive fix: when the emergency switch trips for a neuron, in addition to zeroing its incoming inter-ring synapses, it is now also given a direct, immediate push to its own adaptation variable ("adapt", the same mechanism already used to make a persistent bump fade, from Section 5.9 onward) -- a suppression that acts on the neuron itself, regardless of the source of its excitation, whether recurrent or inter-ring (dialogo_spiking_bidirezionale_omeostasi_v4.py).

Result on seed 301: fully resolved. From 4,870,314 spikes (total explosion, Sec-

tion 5.14) to 3,625 -- healthy levels, indistinguishable from the other seeds, with growth in both directions (B from A: 160->209, +31%; A from B: 63->81, +29%).

Full Monte Carlo verification on the same 8 seeds as Sections 5.14-5.16: ALL EIGHT stable (100%), with a remarkably uniform total spike count per ring across seeds (mean 3338, range 2921-3625 -- under 25% variability between the best and worst seed, against the four orders of magnitude of difference in previous attempts). Even more important: in ALL EIGHT seeds, without a single exception, BOTH channels grow over the course of training -- the B->A channel between 18% and 71% (mean ~42%), the A->B channel between 21% and 32% (mean ~27%). No flat runs, no ambiguous or contradictory growth as in previous versions (5.14, 5.15): for the first time in this entire line of work, bidirectional growth is a universal and reproducible phenomenon, not a lucky outcome for a handful of seeds.

Conclusion of the spiking-bidirectionality line of work begun in Section 5.8: learned bidirectional communication between two spiking ring attractors, with true spike-timing-based STDP, is achievable -- but it requires an architecture substantially richer than the simple STDP-coupled synapse that sufficed in the rate model (Section 5.5.3) or in the unidirectional spiking channel (Section 5.7). Three ingredients all turned out to be necessary, in combination: an explicit transmission switch (the two rings never bidirectionally connected at the same instant, Section 5.13), homeostatic weight normalization based on the actual firing rate rather than a simple per-synapse cap (Sections 5.14-5.15), and an emergency switch that directly suppresses a neuron (not just its incoming synapses) when its activity risks running out of control (this section). Removing even one of these three elements, in previous attempts, left the system unstable or without a useful signal. Twelve independent attempts, each documented with its own outcome (even negative ones) instead of being silently discarded, led from the first total failure (Section 5.8: zero signal over 60 repetitions) to a complete result verified over 8 independent seeds.

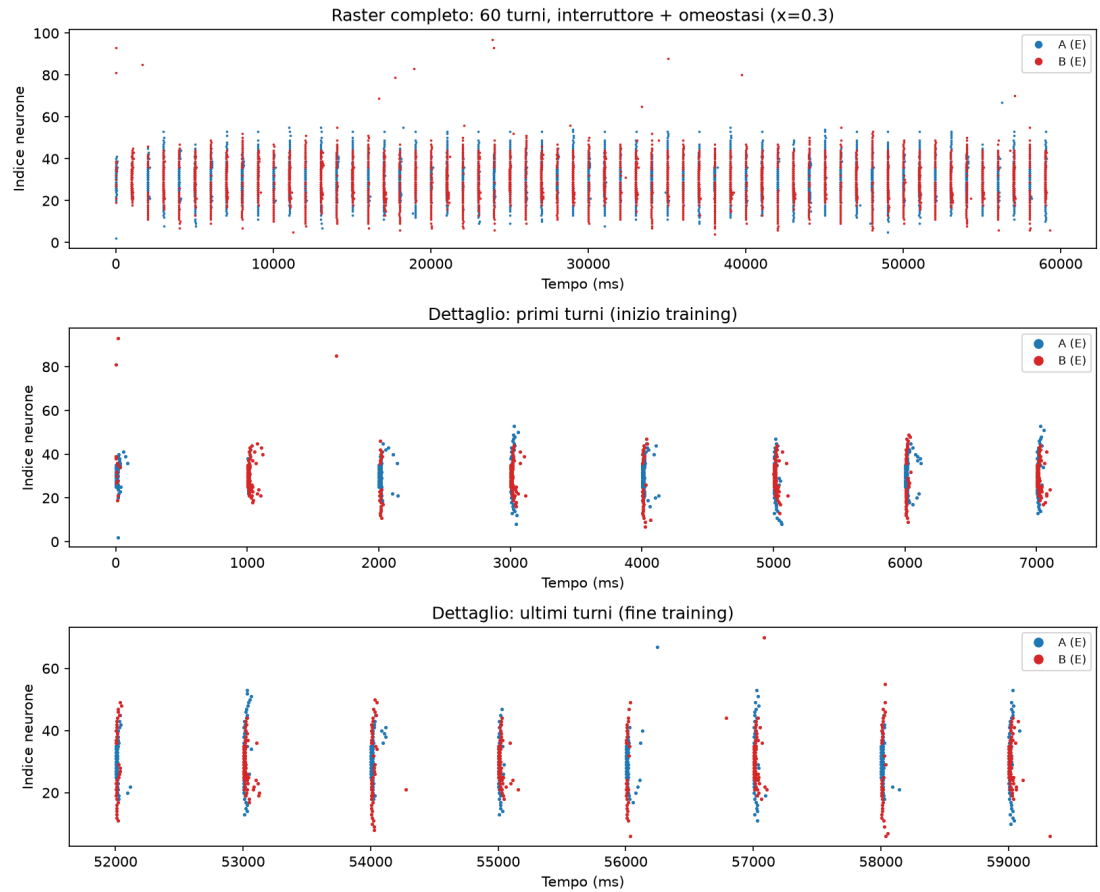


Fig. 44 — Spiking bidirectionality solved: transmission switch + gradual homeostasis + direct emergency suppression. 8/8 stable seeds, bidirectional growth in every single run.

Source code: `dialogo_spiking_bidirezionale_omeostasi_v4.py`

```
1 """
2 v3: aggiunge un interruttore di emergenza sopra la normalizzazione
3 omeostatica graduale della v2 (sezione 5.15). Il seed 301 esplodeva
4 identicamente in v1 e v2 nonostante tutto il resto fosse stabile --
5 verificato che non e' un problema di connettivita' ricorrente interna
6 (un anello isolato con la stessa connettivita' resta stabilissimo, solo
7 9 spike in 60s di rumore puro) ne' di grado di connessione inter-anello
8 anomalo (statistiche identiche agli altri seed). La correzione graduale
9 (al massimo +-1% ogni 20ms) evidentemente non basta a fermare
10 un'accensione che parte troppo in fretta per certe combinazioni di
11 rumore/tempismo. Qui si aggiunge un azzeramento IMMEDIATO (non graduale)
12 delle sinapsi in ingresso a un neurone se il suo tasso di scarica supera
13 una soglia critica (15Hz, ben sopra il bersaglio di 2.5Hz ma ben sotto un
14 vero runaway), come rete di sicurezza indipendente dalla velocita' di
15 reazione della correzione graduale.
16
17 Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
```

```

18 | mostrato che l'interruttore di trasmissione (mai i due anelli connessi
19 | bidirezionalmente nello stesso istante) non basta da solo: con pesi
20 | conservativi il sistema resta sul bordo di un'instabilita' dinamica --
21 | 3 seed su 4 esplodono comunque. L'unico strumento usato finora per
22 | contenere la crescita STDP era un tetto fisso (w_max) sul peso di
23 | SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
24 | stesso neurone cresca senza limite.
25 |
26 | Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
27 | 1998) -- un meccanismo biologico reale, mai implementato altrove in
28 | questo progetto, che regola non il peso di ogni singola sinapsi ma la
29 | somma dell'input verso ciascun neurone, sulla base della sua stessa
30 | attivita' recente. Ogni neurone E stima il proprio tasso di scarica
31 | recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
32 | tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
33 | moltiplicativamente in modo da spingere quel tasso verso un bersaglio
34 | fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
35 | in ingresso si indeboliscono automaticamente (indipendentemente da quale
36 | STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
37 | differenza del tetto fisso, questo meccanismo reagisce alla vera
38 | conseguenza funzionale (il tasso di scarica), non alla causa nominale
39 | (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
40 | anche quando i pesi individuali restano ben sotto al tetto.
41 | """
42 |
43 | from brian2 import *
44 |
45 | start_scope()
46 |
47 | N_E = 100
48 | N_I = 25
49 | N_ring = N_E + N_I
50 |
51 | a = 0.02/ms
52 | b = 0.2/ms
53 | c = -65*mV
54 | d = 6*mV/ms
55 | tau_nmda = 100*ms
56 |
57 | tau_adapt_pop = 200*ms
58 | gain_adapt_pop = 0.15*mV/ms
59 |
60 | tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
61 |                       # in fretta a un'accensione che sta per scappare
62 |
63 | eqs = '''
64 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
65 | du/dt = a*(b*v - u) : volt/second
66 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
67 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
68 | dratehat/dt = -ratehat/tau_rate_hat : Hz
69 | I : volt/second
70 | x : 1
71 | '''
72 |
73 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
74 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat',
75 |                      method='euler')
76 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
77 | neurons.u = b * neurons.v[:]
78 | neurons.I = 0*mV/ms
79 | neurons.g_nmda = 0*mV/ms
80 | neurons.adapt = 0*mV/ms
81 | neurons.ratehat = 0*Hz
82 |
83 | A_E = neurons[:N_E]
84 | A_I = neurons[N_E:N_ring]

```

```

85 B_E = neurons[N_ring:N_ring+N_E]
86 B_I = neurons[N_ring+N_E:]
87
88 xi = np.arange(N_E) / N_E
89 A_E.x = xi
90 B_E.x = xi
91
92 background = PoissonGroup(2*N_ring, rates=400*Hz)
93 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
94 bg_syn.connect(j='i')
95
96 sigma_exc = 0.05
97 CUTOFF_LOCALE = 3 * sigma_exc
98 w_ee_nmda = 0.325*mV/ms
99 p_conn = 0.15
100 w_exc = 1.0*mV
101 w_inh = 10.0*mV
102
103
104 def dist_circ(dx):
105     dx = abs(dx)
106     return np.minimum(dx, 1 - dx)
107
108
109 def costruisci_anello(E, I_pop, seed):
110     dmat = dist_circ(xi[:, None] - xi[None, :])
111     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
112     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
113     np.fill_diagonal(prob_ee, 0)
114     rng = np.random.RandomState(seed)
115     mask_ee = rng.rand(N_E, N_E) < prob_ee
116     i_idx, j_idx = np.nonzero(mask_ee)
117
118     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
119     syn_ee.connect(i=i_idx, j=j_idx)
120     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
121     syn_ei.connect(p=p_conn)
122     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
123     syn_ie.connect(p=p_conn)
124     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
125     syn_ii.connect(condition='i!=j', p=p_conn)
126     return syn_ee, syn_ei, syn_ie, syn_ii
127
128
129 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
130 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
131
132 tau_stdp = 20*ms
133 A_ltp = 0.15*mV
134 A_ltd = 0.15*mV
135 w_max = 5.0*mV
136 w_min = 0*mV
137 w_init = 1.5*mV
138 sigma_inter = 0.08
139 p_inter_picco = 0.5
140
141 stdp_model = '''
142 w : volt
143 dapre/dt = -apre/tau_stdp : volt (event-driven)
144 dapost/dt = -apost/tau_stdp : volt (event-driven)
145 gate : 1 (shared)
146 '''
147 stdp_on_pre = '''
148 v_post += w*gate
149 apre += A_ltp*gate
150 w = clip(w+apost*gate, w_min, w_max)
151 '''
152 stdp_on_post = '''

```

```

153 apost -= A_ltd*gate
154 w = clip(w+apre*gate, w_min, w_max)
155 '''
156
157 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
158 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
159 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
160 rng_inter = np.random.RandomState(2)
161 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
162 i_inter, j_inter = np.nonzero(mask_inter)
163 syn_ab.connect(i=i_inter, j=j_inter)
164 syn_ab.w = w_init
165 syn_ab.gate = 1
166 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
167
168 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
169 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
170 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
171 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
172 syn_ba.w = w_init
173 syn_ba.gate = 0
174 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
175
176 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
177 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
178 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
179 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
180 # gia' esistente (che resta come sicurezza aggiuntiva) ---
181 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
182 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
183
184
185 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
186 # bersaglio (2.5Hz) ma ben sotto un vero runaway
187 # (centinaia di Hz), per intercettare un'accensione
188 # che la correzione graduale non riesce a fermare
189
190
191 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
192 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
193     rate_b = np.asarray(B_E.ratehat/Hz)
194     rate_a = np.asarray(A_E.ratehat/Hz)
195     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
196     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
197     # interruttore di emergenza: azzeramento immediato (non graduale) delle
198     # sinapsi in ingresso ai neuroni che superano la soglia critica
199     emergenza_b = rate_b > (soglia_emergenza/Hz)
200     emergenza_a = rate_a > (soglia_emergenza/Hz)
201     if emergenza_b.any():
202         fattore_b[emergenza_b] = 0.0
203         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
204         * (mV/ms)
205         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
206               azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz]")
207     if emergenza_a.any():
208         fattore_a[emergenza_a] = 0.0
209         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
210         * (mV/ms)
211         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
212               azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz]")
213
214     j_ab = np.asarray(syn_ab.j[:])
215     j_ba = np.asarray(syn_ba.j[:])
216     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
217     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
218
219
220 spikes = SpikeMonitor(neurons)
221 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)

```



```

217 MESSAGGIO = 0.3
218 larghezza_stimolo = 0.05
219 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
220 gruppo_stim = np.where(peso_stim > 0.5)[0]
221 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
222
223 N_CICLI = 60
224 T_STIM = 60*ms
225 T_PAUSA = 940*ms
226
227 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
228       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
229       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
230
231 turni = []
232 t_corrente = 0*ms
233 for ciclo in range(N_CICLI):
234     turno = 'A' if ciclo % 2 == 0 else 'B'
235     turni.append((turno, t_corrente/ms))
236     if turno == 'A':
237         syn_ab.gate = 1
238         syn_ba.gate = 0
239         A_E.I[gruppo_stim] = 15*mV/ms
240         run(T_STIM)
241         A_E.I[:] = 0*mV/ms
242     else:
243         syn_ab.gate = 0
244         syn_ba.gate = 1
245         B_E.I[gruppo_stim] = 15*mV/ms
246         run(T_STIM)
247         B_E.I[:] = 0*mV/ms
248     run(T_PAUSA)
249     t_corrente += T_STIM + T_PAUSA
250     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
251         n_a_finora = int((spikes.i < N_E).sum())
252         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
253         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
254               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
255               "
256               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
257               "
258               f"spike finora A={n_a_finora} B={n_b_finora}")
259
260 print("\nTraining completato.")
261
262 fig, axes = subplots(3, 1, figsize=(11, 9))
263
264 subplot(3, 1, 1)
265 mask_a = spikes.i < N_E
266 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
267 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
268 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
269 xlabel('Tempo (ms)')
270 ylabel('Indice neurone')
271 title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
272 legend(loc='upper right', fontsize=8, markerscale=6)
273
274 subplot(3, 1, 2)
275 t_fine = 8 * (T_STIM+T_PAUSA)/ms
276 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
277 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
278 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
279 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
280 xlabel('Tempo (ms)')

```

```

280 ylabel('Indice neurone')
281 title('Dettaglio: primi turni (inizio training)')
282 legend(loc='upper right', fontsize=8, markerscale=4)
283
284 subplot(3, 1, 3)
285 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
286 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
287 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
288 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
      ='A (E)')
289 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
290 xlabel('Tempo (ms)')
291 ylabel('Indice neurone')
292 title('Dettaglio: ultimi turni (fine training)')
293 legend(loc='upper right', fontsize=8, markerscale=4)
294
295 tight_layout()
296 savefig('dialogo_spiking_bidirezionale_omeostasi_v4.png', dpi=150)
297 print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v4.png")
298
299 ciclo_ms = (T_STIM+T_PAUSA)/ms
300
301
302 def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
303     subset = [t0 for (l, t0) in turni if l == lettera_turno]
304     primi = subset[:n_turni_eval]
305     ultimi = subset[-n_turni_eval:]
306
307     def conta(lista_inizio):
308         n_tot = 0
309         posizioni = []
310         for t0 in lista_inizio:
311             fin = t0 + ciclo_ms
312             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
313             if escludi_diretto:
314                 m = m & (spikes.t/ms >= t0 + 20)
315             idx = spikes.i[m]
316             if lettera_turno == 'A':
317                 idx = idx - N_ring
318             n_tot += m.sum()
319             posizioni.extend(list(xi[idx % N_E]))
320         centro = None
321         if posizioni:
322             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
323                             % 1.0)
324         return n_tot, centro
325
326     return conta(primi), conta(ultimi)
327
328 print("\n" + "="*70)
329 print("VALUTAZIONE: turni 'A' -- B risponde?")
330 print("="*70)
331 (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
332 print(f" Primi 5 turni A: {n_i} spike di B, centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
333 print(f" Ultimi 5 turni A: {n_f} spike di B, centro={'N/A' if c_f is None else f'{c_f:.3f}'}")
334
335 print("\n" + "="*70)
336 print("VALUTAZIONE: turni 'B' -- A risponde?")
337 print("="*70)
338 (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
339 print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro={'N/A' if c_i is None else f'{c_i:.3f}'}")
340 print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro={'N/A' if c_f is None else f'{c_f:.3f}'}")

```

```

341 n_a_tot = int(mask_a.sum())
342 n_b_tot = int(mask_b.sum())
343 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
344       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
345 if n_a_tot > 200000 or n_b_tot > 200000:
346     print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
347           "sistema e' instabile. <<<")
348 else:
349     print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")
350

```

Source code: dialogo_spiking_bidirezionale_omeostasi_v4_mc.py

```

1  """
2  v3: aggiunge un interruttore di emergenza sopra la normalizzazione
3  omeostatica graduale della v2 (sezione 5.15). Il seed 301 esplodeva
4  identicamente in v1 e v2 nonostante tutto il resto fosse stabile --
5  verificato che non e' un problema di connettivita' ricorrente interna
6  (un anello isolato con la stessa connettivita' resta stabilissimo, solo
7  9 spike in 60s di rumore puro) ne' di grado di connessione inter-anello
8  anomalo (statistiche identiche agli altri seed). La correzione graduale
9  (al massimo +-1% ogni 20ms) evidentemente non basta a fermare
10 un'accensione che parte troppo in fretta per certe combinazioni di
11 rumore/tempismo. Qui si aggiunge un azzeramento IMMEDIATO (non graduale)
12 delle sinapsi in ingresso a un neurone se il suo tasso di scarica supera
13 una soglia critica (15Hz, ben sopra il bersaglio di 2.5Hz ma ben sotto un
14 vero runaway), come rete di sicurezza indipendente dalla velocita' di
15 reazione della correzione graduale.
16
17 Decimo tentativo sulla bidirezionalita' spiking. La sezione 5.13 ha
18 mostrato che l'interruttore di trasmissione (mai i due anelli connessi
19 bidirezionalmente nello stesso istante) non basta da solo: con pesi
20 conservativi il sistema resta sul bordo di un'instabilita' dinamica --
21 3 seed su 4 esplodono comunque. L'unico strumento usato finora per
22 contenere la crescita STDP era un tetto fisso (w_max) sul peso di
23 SINGOLA sinapsi: non impedisce che la somma di MOLTE sinapsi verso lo
24 stesso neurone cresca senza limite.
25
26 Tentativo qui: normalizzazione omeostatica (synaptic scaling, Turrigiano
27 1998) -- un meccanismo biologico reale, mai implementato altrove in
28 questo progetto, che regola non il peso di ogni singola sinapsi ma la
29 somma dell'input verso ciascun neurone, sulla base della sua stessa
30 attivita' recente. Ogni neurone E stima il proprio tasso di scarica
31 recente (una media mobile esponenziale, variabile "ratehat"); ogni 50ms,
32 tutte le sue sinapsi inter-anello IN INGRESSO vengono scalate
33 moltiplicativamente in modo da spingere quel tasso verso un bersaglio
34 fisso (target_rate). Se un neurone sta scaricando troppo, le sue sinapsi
35 in ingresso si indeboliscono automaticamente (indipendentemente da quale
36 STDP le abbia rinforzate); se scarica troppo poco, si rinforzano. A
37 differenza del tetto fisso, questo meccanismo reagisce alla vera
38 conseguenza funzionale (il tasso di scarica), non alla causa nominale
39 (il peso per sinapsi) -- l'ipotesi e' che possa impedire il collasso
40 anche quando i pesi individuali restano ben sotto al tetto.
41 """
42
43 import sys
44 from brian2 import *
45
46 SEED = int(sys.argv[1])
47 start_scope()
48 seed(SEED)
49
50 N_E = 100
51 N_I = 25
52 N_ring = N_E + N_I
53
54 a = 0.02/ms

```

```

55 b = 0.2/ms
56 c = -65*mV
57 d = 6*mV/ms
58 tau_nmda = 100*ms
59
60 tau_adapt_pop = 200*ms
61 gain_adapt_pop = 0.15*mV/ms
62
63 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
64                        # in fretta a un'accensione che sta per scappare
65
66 eqs = '''
67 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
68 du/dt = a*(b*v - u) : volt/second
69 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
70 dadapt/dt = -adapt/tau_adapt_pop : volt/second
71 dratehat/dt = -ratehat/tau_rate_hat : Hz
72 I : volt/second
73 x : 1
74 '''
75
76 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
77                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat',
78                      method='euler')
79 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
80 neurons.u = b * neurons.v[:]
81 neurons.I = 0*mV/ms
82 neurons.g_nmda = 0*mV/ms
83 neurons.adapt = 0*mV/ms
84 neurons.ratehat = 0*Hz
85
86 A_E = neurons[:N_E]
87 A_I = neurons[N_E:N_ring]
88 B_E = neurons[N_ring:N_ring+N_E]
89 B_I = neurons[N_ring+N_E:]
90
91 xi = np.arange(N_E) / N_E
92 A_E.x = xi
93 B_E.x = xi
94
95 background = PoissonGroup(2*N_ring, rates=400*Hz)
96 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
97 bg_syn.connect(j='i')
98
99 sigma_exc = 0.05
100 CUTOFF_LOCALE = 3 * sigma_exc
101 w_ee_nmda = 0.325*mV/ms
102 p_conn = 0.15
103 w_exc = 1.0*mV
104 w_inh = 10.0*mV
105
106
107 def dist_circ(dx):
108     dx = abs(dx)
109     return np.minimum(dx, 1 - dx)
110
111
112 def costruisci_anello(E, I_pop, seed):
113     dmat = dist_circ(xi[:, None] - xi[None, :])
114     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
115     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
116     np.fill_diagonal(prob_ee, 0)
117     rng = np.random.RandomState(seed)
118     mask_ee = rng.rand(N_E, N_E) < prob_ee
119     i_idx, j_idx = np.nonzero(mask_ee)
120
121     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')

```

```

122     syn_ee.connect(i=i_idx, j=j_idx)
123     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
124     syn_ei.connect(p=p_conn)
125     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
126     syn_ie.connect(p=p_conn)
127     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
128     syn_ii.connect(condition='i!=j', p=p_conn)
129     return syn_ee, syn_ei, syn_ie, syn_ii
130
131
132 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
133 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
134
135 tau_stdp = 20*ms
136 A_ltp = 0.15*mV
137 A_ltd = 0.15*mV
138 w_max = 5.0*mV
139 w_min = 0*mV
140 w_init = 1.5*mV
141 sigma_inter = 0.08
142 p_inter_picco = 0.5
143
144 stdp_model = '''
145 w : volt
146 dapre/dt = -apre/tau_stdp : volt (event-driven)
147 dapost/dt = -apost/tau_stdp : volt (event-driven)
148 gate : 1 (shared)
149 '''
150 stdp_on_pre = '''
151 v_post += w*gate
152 apre += A_ltp*gate
153 w = clip(w+apost*gate, w_min, w_max)
154 '''
155 stdp_on_post = '''
156 apost -= A_ltd*gate
157 w = clip(w+apre*gate, w_min, w_max)
158 '''
159
160 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
161 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
162 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
163 rng_inter = np.random.RandomState(3000+SEED)
164 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
165 i_inter, j_inter = np.nonzero(mask_inter)
166 syn_ab.connect(i=i_inter, j=j_inter)
167 syn_ab.w = w_init
168 syn_ab.gate = 1
169 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
170
171 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
172 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
173 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
174 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
175 syn_ba.w = w_init
176 syn_ba.gate = 0
177 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
178
179 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
180 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
181 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
182 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
183 # gia' esistente (che resta come sicurezza aggiuntiva) ---
184 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
185 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
186
187
188 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
189 # bersaglio (2.5Hz) ma ben sotto un vero runaway

```

```

190                                     # (centinaia di Hz), per intercettare un'accensione
191                                     # che la correzione graduale non riesce a fermare
192
193
194 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
195 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
196     rate_b = np.asarray(B_E.ratehat/Hz)
197     rate_a = np.asarray(A_E.ratehat/Hz)
198     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
199     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
200     # interruttore di emergenza: azzeramento immediato (non graduale) delle
201     # sinapsi in ingresso ai neuroni che superano la soglia critica
202     emergenza_b = rate_b > (soglia_emergenza/Hz)
203     emergenza_a = rate_a > (soglia_emergenza/Hz)
204     if emergenza_b.any():
205         fattore_b[emergenza_b] = 0.0
206         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
207         * (mV/ms)
208         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
209               azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz] ")
210     if emergenza_a.any():
211         fattore_a[emergenza_a] = 0.0
212         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
213         * (mV/ms)
214         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
215               azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz] ")
216     j_ab = np.asarray(syn_ab.j[:])
217     j_ba = np.asarray(syn_ba.j[:])
218     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
219     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
220
221 spikes = SpikeMonitor(neurons)
222 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
223
224 MESSAGGIO = 0.3
225 larghezza_stimolo = 0.05
226 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
227 gruppo_stim = np.where(peso_stim > 0.5)[0]
228 print(f"Neuroni stimolati (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
229
230 N_CICLI = 60
231 T_STIM = 60*ms
232 T_PAUSA = 940*ms
233
234 print(f"\nTraining: {N_CICLI} turni alternati A/B/A/B/..., interruttore di "
235       f"trasmissione + normalizzazione omeostatica (target {target_rate}) "
236       f"({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
237
238 turni = []
239 t_corrente = 0*ms
240 for ciclo in range(N_CICLI):
241     turno = 'A' if ciclo % 2 == 0 else 'B'
242     turni.append((turno, t_corrente/ms))
243     if turno == 'A':
244         syn_ab.gate = 1
245         syn_ba.gate = 0
246         A_E.I[gruppo_stim] = 15*mV/ms
247         run(T_STIM)
248         A_E.I[:] = 0*mV/ms
249     else:
250         syn_ab.gate = 0
251         syn_ba.gate = 1
252         B_E.I[gruppo_stim] = 15*mV/ms
253         run(T_STIM)
254         B_E.I[:] = 0*mV/ms
255     run(T_PAUSA)
256     t_corrente += T_STIM + T_PAUSA

```

```

254     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
255         n_a_finora = int((spikes.i < N_E).sum())
256         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
257         print(f" ciclo {ciclo:3d}/{N_CICLI} (turno {turno}): "
258               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
259               "
260               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
261               "
262               f"spike finora A={n_a_finora} B={n_b_finora}")
263
264     print("\nTraining completato.")
265
266     fig, axes = subplots(3, 1, figsize=(11, 9))
267
268     subplot(3, 1, 1)
269     mask_a = spikes.i < N_E
270     mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
271     plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
272     plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
273           (E)')
274     xlabel('Tempo (ms)')
275     ylabel('Indice neurone')
276     title(f'Raster completo: {N_CICLI} turni, interruttore + omeostasi (x={MESSAGGIO})')
277     legend(loc='upper right', fontsize=8, markerscale=6)
278
279     subplot(3, 1, 2)
280     t_fine = 8 * (T_STIM+T_PAUSA)/ms
281     mask_a_fine = mask_a & (spikes.t/ms < t_fine)
282     mask_b_fine = mask_b & (spikes.t/ms < t_fine)
283     plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
284           A (E)')
285     plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
286           label='B (E)')
287     xlabel('Tempo (ms)')
288     ylabel('Indice neurone')
289     title('Dettaglio: primi turni (inizio training)')
290     legend(loc='upper right', fontsize=8, markerscale=4)
291
292     subplot(3, 1, 3)
293     t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
294     mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
295     mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
296     plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
297           A (E)')
298     plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
299           =3, label='B (E)')
300     xlabel('Tempo (ms)')
301     ylabel('Indice neurone')
302     title('Dettaglio: ultimi turni (fine training)')
303     legend(loc='upper right', fontsize=8, markerscale=4)
304
305     tight_layout()
306     savefig('dialogo_spiking_bidirezionale_omeostasi_v4.png', dpi=150)
307     print("\nGrafico salvato in dialogo_spiking_bidirezionale_omeostasi_v4.png")
308
309     ciclo_ms = (T_STIM+T_PAUSA)/ms
310
311     def valuta_canale(lettera_turno, mask_dest, n_turni_eval, escludi_diretto):
312         subset = [t0 for (l, t0) in turni if l == lettera_turno]
313         primi = subset[:n_turni_eval]
314         ultimi = subset[-n_turni_eval:]
315
316         def conta(lista_inizio):
317             n_tot = 0
318             posizioni = []
319             for t0 in lista_inizio:
320                 fin = t0 + ciclo_ms

```

```

315         m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
316         if escludi_diretto:
317             m = m & (spikes.t/ms >= t0 + 20)
318             idx = spikes.i[m]
319             if lettera_turno == 'A':
320                 idx = idx - N_ring
321             n_tot += m.sum()
322             posizioni.extend(list(xi[idx % N_E]))
323         centro = None
324         if posizioni:
325             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
326                             % 1.0)
327         return n_tot, centro
328     return conta(primi), conta(ultimi)
329
330
331     print("\n" + "="*70)
332     print("VALUTAZIONE: turni 'A' -- B risponde?")
333     print("="*70)
334     (n_i, c_i), (n_f, c_f) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
335     print(f" Primi 5 turni A: {n_i} spike di B, centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
336     print(f" Ultimi 5 turni A: {n_f} spike di B, centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
337
338     print("\n" + "="*70)
339     print("VALUTAZIONE: turni 'B' -- A risponde?")
340     print("="*70)
341     (n_i, c_i), (n_f, c_f) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
342     print(f" Primi 5 turni B: {n_i} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_i is None else f'{c_i:.3f}'}")
343     print(f" Ultimi 5 turni B: {n_f} spike di A (>20ms dall'inizio turno), centro=({'N/A' if c_f is None else f'{c_f:.3f}'}")
344
345     n_a_tot = int(mask_a.sum())
346     n_b_tot = int(mask_b.sum())
347     print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
348           f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
349     if n_a_tot > 200000 or n_b_tot > 200000:
350         print(" >>> ESPLOSIONE: anche con la normalizzazione omeostatica il "
351               "sistema e' instabile. <<<")
352     else:
353         print(" >>> Stabile con la normalizzazione omeostatica attiva. <<<")
354
355     stabile = n_a_tot < 200000 and n_b_tot < 200000
356     (n_ab_primi, _), (n_ab_ultimi, _) = valuta_canale('B', mask_a, 5, escludi_diretto=True)
357     (n_ba_primi, _), (n_ba_ultimi, _) = valuta_canale('A', mask_b, 5, escludi_diretto=False)
358     print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} "
359           f"a_da_b_primi={n_ab_primi} a_da_b_ultimi={n_ab_ultimi} "
360           f"b_da_a_primi={n_ba_primi} b_da_a_ultimi={n_ba_ultimi}")

```

5.18 More bidirectional messages: completing the spiking dialogue

Thirteenth and final step in this line of work: extending the complete architecture solved in 5.17 (transmission switch + gradual homeostatic normalization + direct emergency suppression) to multiple messages, the same final step already taken by the rate model (from 5.5.3, single-message bidirectional, to 5.5.13, multi-message bidirectional). The same two positions as `dialogo_spiking_multi.py` (Section 5.10): $x=0.15$ and $x=0.65$. The turn-taking scheme extended to 4 cyclic combinations (message, direction): (m0,A), (m0,B), (m1,A),

(m1,B), repeated 20 times (80 turns total, 80 seconds of simulation).

On the starting seed: stable (4549/4419 total spikes), both messages precisely localized in both directions from the start (error 0.002-0.028, well below the stimulus width of 0.05) and growing over the course of training. The raster shows two clearly separated, stable bands (indices ~0-30 and ~50-85) for the entire 80-second duration, with no cross-contamination between the two messages -- the same "no interference" result already seen in 5.10 for the unidirectional case, here confirmed in the bidirectional version as well.

Monte Carlo verification over 8 independent seeds (16 total observations, 8 seeds x 2 messages): ALL EIGHT stable (100%), average localization error 0.014 (A->B channel) and 0.031 (B->A channel), maximum 0.087 across all 16 observations -- always well below the success threshold (0.10) and the stimulus width. The B->A channel (the harder one, the subject of twelve attempts in Sections 5.8-5.17) grows over the course of training in 14 of 16 observations (87.5%); the A->B channel grows in 9 of 16 (56%), but it already starts from higher levels (131-241 spikes against 37-80 for B->A) -- consistent with a channel that has already reached most of its useful reinforcement margin from the first repetitions, not with a sign of instability or interference.

This closes, with a positive and verified result, the entire development arc of the spiking dialogue begun in Section 5.6 (first stable ring attractor): single-message learned communication (5.7), then multi-message (5.10-5.11), then single-message bidirectional after twelve attempts (5.8-5.17), and finally multi-message bidirectional in this section -- the same milestone reached by the rate model, but in the spiking domain with true spike-timing-based STDP, and with a stabilization architecture (transmission switch, homeostasis, emergency suppression) that has no direct equivalent in the rate model -- necessary specifically because of the dynamical constraints of the spiking domain, carefully documented in every attempt, successful or not, throughout this entire chapter.

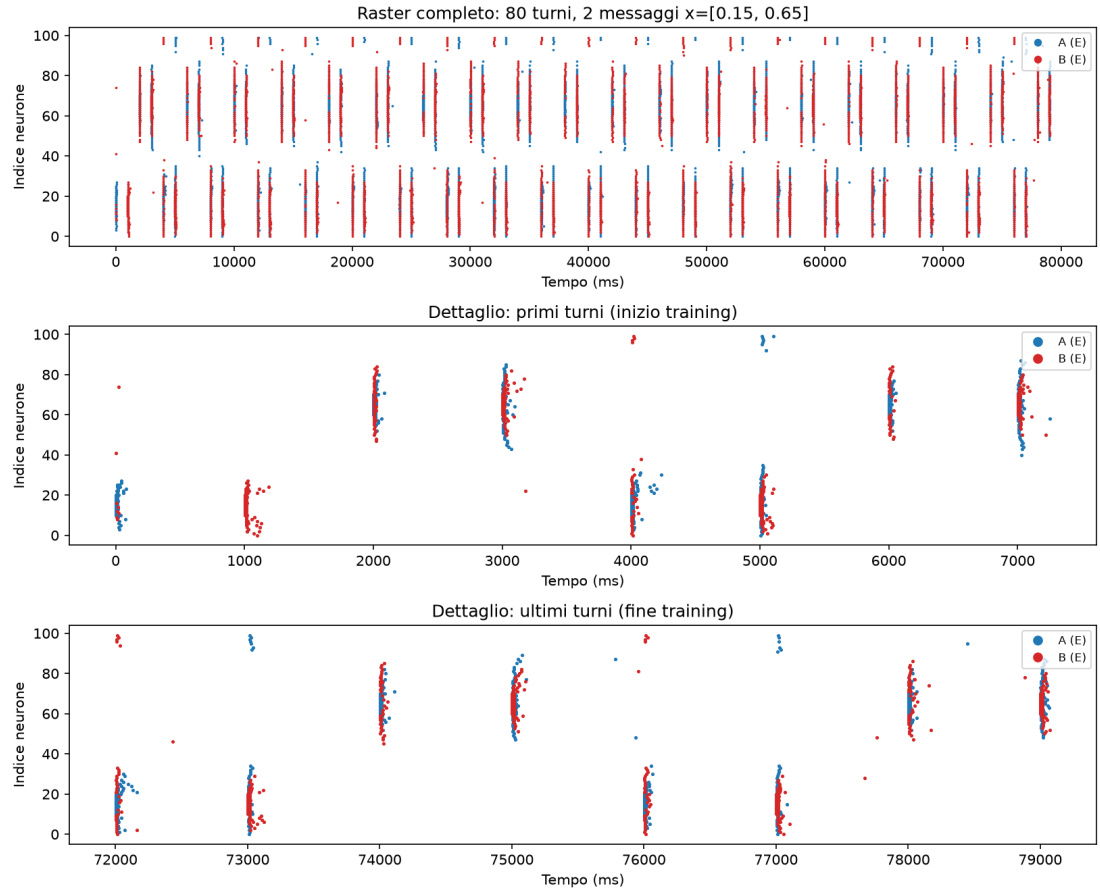


Fig. 45 — Two bidirectional messages in the spiking domain: no interference, both directions working, stable for 80 seconds.

Source code: `dialogo_spiking_bidirezionale_multi.py`

```

1  """
2  Tredicesimo tentativo, ultimo passo del filone bidirezionale spiking:
3  estendere l'architettura completa che ha risolto la sezione 5.17
4  (interruttore di trasmissione + normalizzazione omeostatica graduale +
5  soppressione diretta di emergenza, 8/8 seed stabili con crescita
6  bidirezionale) a PIU' MESSAGGI, esattamente come il modello a rate aveva
7  fatto il suo passo finale (dialogo_multi_scoperta_bidirezionale.py,
8  sezione 5.5.13) estendendo il canale bidirezionale a turni gia' risolto
9  (5.5.3) a piu' messaggi.
10
11  Stesse due posizioni usate in dialogo_spiking_multi.py (sezione 5.10,
12  comunicazione unidirezionale multi-messaggio, gia' verificata robusta su
13  8 seed in 5.11): x=0.15 e x=0.65. Schema a turni esteso a 4 combinazioni
14  (messaggio, direzione) invece delle 2 originarie (solo direzione),
15  cicliche in ordine fisso: (m0,A), (m0,B), (m1,A), (m1,B), ripetuto.
16  """
17
18  from brian2 import *
19

```

```

20 start_scope()
21
22 N_E = 100
23 N_I = 25
24 N_ring = N_E + N_I
25
26 a = 0.02/ms
27 b = 0.2/ms
28 c = -65*mV
29 d = 6*mV/ms
30 tau_nmda = 100*ms
31
32 tau_adapt_pop = 200*ms
33 gain_adapt_pop = 0.15*mV/ms
34
35 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
36                        # in fretta a un'accensione che sta per scappare
37
38 eqs = '''
39 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
40 du/dt = a*(b*v - u) : volt/second
41 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
42 dadapt/dt = -adapt/tau_adapt_pop : volt/second
43 dratehat/dt = -ratehat/tau_rate_hat : Hz
44 I : volt/second
45 x : 1
46 '''
47
48 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
49                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
50                      , method='euler')
51 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
52 neurons.u = b * neurons.v[:]
53 neurons.I = 0*mV/ms
54 neurons.g_nmda = 0*mV/ms
55 neurons.adapt = 0*mV/ms
56 neurons.ratehat = 0*Hz
57
58 A_E = neurons[:N_E]
59 A_I = neurons[N_E:N_ring]
60 B_E = neurons[N_ring:N_ring+N_E]
61 B_I = neurons[N_ring+N_E:]
62
63 xi = np.arange(N_E) / N_E
64 A_E.x = xi
65 B_E.x = xi
66
67 background = PoissonGroup(2*N_ring, rates=400*Hz)
68 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
69 bg_syn.connect(j='i')
70
71 sigma_exc = 0.05
72 CUTOFF_LOCALE = 3 * sigma_exc
73 w_ee_nmda = 0.325*mV/ms
74 p_conn = 0.15
75 w_exc = 1.0*mV
76 w_inh = 10.0*mV
77
78
79 def dist_circ(dx):
80     dx = abs(dx)
81     return np.minimum(dx, 1 - dx)
82
83
84 def costruisci_anello(E, I_pop, seed):
85     dmat = dist_circ(xi[:, None] - xi[None, :])
86     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))

```

```

87     prob_ee[dmatrix > CUTOFF_LOCALE] = 0.0
88     np.fill_diagonal(prob_ee, 0)
89     rng = np.random.RandomState(seed)
90     mask_ee = rng.rand(N_E, N_E) < prob_ee
91     i_idx, j_idx = np.nonzero(mask_ee)
92
93     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
94     syn_ee.connect(i=i_idx, j=j_idx)
95     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
96     syn_ei.connect(p=p_conn)
97     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
98     syn_ie.connect(p=p_conn)
99     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
100    syn_ii.connect(condition='i!=j', p=p_conn)
101    return syn_ee, syn_ei, syn_ie, syn_ii
102
103
104    syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
105    syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
106
107    tau_stdp = 20*ms
108    A_ltp = 0.15*mV
109    A_ltd = 0.15*mV
110    w_max = 5.0*mV
111    w_min = 0*mV
112    w_init = 1.5*mV
113    sigma_inter = 0.08
114    p_inter_picco = 0.5
115
116    stdp_model = '''
117    w : volt
118    dapre/dt = -apre/tau_stdp : volt (event-driven)
119    dapost/dt = -apost/tau_stdp : volt (event-driven)
120    gate : 1 (shared)
121    '''
122    stdp_on_pre = '''
123    v_post += w*gate
124    apre += A_ltp*gate
125    w = clip(w+apost*gate, w_min, w_max)
126    '''
127    stdp_on_post = '''
128    apost -= A_ltd*gate
129    w = clip(w+apre*gate, w_min, w_max)
130    '''
131
132    syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
133    dmat_inter = dist_circ(xi[:, None] - xi[None, :])
134    prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
135    rng_inter = np.random.RandomState(2)
136    mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
137    i_inter, j_inter = np.nonzero(mask_inter)
138    syn_ab.connect(i=i_inter, j=j_inter)
139    syn_ab.w = w_init
140    syn_ab.gate = 1
141    print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
142
143    syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
144    mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
145    i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
146    syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
147    syn_ba.w = w_init
148    syn_ba.gate = 0
149    print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
150
151    # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
152    # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
153    # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
154    # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max

```

```

155 # gia' esistente (che resta come sicurezza aggiuntiva) ---
156 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
157 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
158
159
160 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
161 # bersaglio (2.5Hz) ma ben sotto un vero runaway
162 # (centinaia di Hz), per intercettare un'accensione
163 # che la correzione graduale non riesce a fermare
164
165
166 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
167 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
168     rate_b = np.asarray(B_E.ratehat/Hz)
169     rate_a = np.asarray(A_E.ratehat/Hz)
170     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
171     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
172     # interruttore di emergenza: azzeramento immediato (non graduale) delle
173     # sinapsi in ingresso ai neuroni che superano la soglia critica
174     emergenza_b = rate_b > (soglia_emergenza/Hz)
175     emergenza_a = rate_a > (soglia_emergenza/Hz)
176     if emergenza_b.any():
177         fattore_b[emergenza_b] = 0.0
178         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
179         * (mV/ms)
180         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
181               azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz]")
182     if emergenza_a.any():
183         fattore_a[emergenza_a] = 0.0
184         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
185         * (mV/ms)
186         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
187               azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz]")
188     j_ab = np.asarray(syn_ab.j[:])
189     j_ba = np.asarray(syn_ba.j[:])
190     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
191     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
192
193
194 spikes = SpikeMonitor(neurons)
195 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
196
197 MESSAGGI = [0.15, 0.65] # stesse posizioni di dialogo_spiking_multi.py (5.10)
198 larghezza_stimolo = 0.05
199 gruppi_stim = []
200 for m in MESSAGGI:
201     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
202     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
203     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni stimolati")
204
205 N_CICLI = 80 # 20 ripetizioni per ciascuna delle 4 combinazioni (messaggio, direzione)
206 T_STIM = 60*ms
207 T_PAUSA = 940*ms
208
209 print(f"\nTraining: {N_CICLI} turni, ciclando su {len(MESSAGGI)} messaggi x 2 "
210       f"direzioni (A/B), interruttore di trasmissione + omeostasi + "
211       f"soppressione di emergenza ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
212
213 turni = [] # (messaggio_idx, direzione, tempo_inizio_ms)
214 combinazioni = [(k, dirz) for k in range(len(MESSAGGI)) for dirz in ('A', 'B')]
215 t_corrente = 0*ms
216 for ciclo in range(N_CICLI):
217     k, direzione = combinazioni[ciclo % len(combinazioni)]
218     turni.append((k, direzione, t_corrente/ms))
219     if direzione == 'A':
220         syn_ab.gate = 1
221         syn_ba.gate = 0
222         A_E.I[gruppi_stim[k]] = 15*mV/ms

```

```

219         run(T_STIM)
220         A_E.I[:] = 0*mV/ms
221     else:
222         syn_ab.gate = 0
223         syn_ba.gate = 1
224         B_E.I[gruppi_stim[k]] = 15*mV/ms
225         run(T_STIM)
226         B_E.I[:] = 0*mV/ms
227     run(T_PAUSA)
228     t_corrente += T_STIM + T_PAUSA
229     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
230         n_a_finora = int((spikes.i < N_E).sum())
231         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
232         print(f" ciclo {ciclo:3d}/{N_CICLI} (msg x={MESSAGGI[k]}, turno {direzione}): "
233               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
234               "
235               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
236               "
237               f"spike finora A={n_a_finora} B={n_b_finora}")
238
239 print("\nTraining completato.")
240
241 fig, axes = subplots(3, 1, figsize=(11, 9))
242
243 subplot(3, 1, 1)
244 mask_a = spikes.i < N_E
245 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
246 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
247 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
248       (E)')
249 xlabel('Tempo (ms)')
250 ylabel('Indice neurone')
251 title(f'Raster completo: {N_CICLI} turni, {len(MESSAGGI)} messaggi x=[{" ".join(str(m) for
252       m in MESSAGGI)}]')
253 legend(loc='upper right', fontsize=8, markerscale=6)
254
255 subplot(3, 1, 2)
256 t_fine = 8 * (T_STIM+T_PAUSA)/ms
257 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
258 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
259 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
260       A (E)')
261 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
262       label='B (E)')
263 xlabel('Tempo (ms)')
264 ylabel('Indice neurone')
265 title('Dettaglio: primi turni (inizio training)')
266 legend(loc='upper right', fontsize=8, markerscale=4)
267
268 subplot(3, 1, 3)
269 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
270 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
271 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
272 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label=
273       'A (E)')
274 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
275       =3, label='B (E)')
276 xlabel('Tempo (ms)')
277 ylabel('Indice neurone')
278 title('Dettaglio: ultimi turni (fine training)')
279 legend(loc='upper right', fontsize=8, markerscale=4)
280
281 tight_layout()
282 savefig('dialogo_spiking_bidirezionale_multi.png', dpi=150)
283 print("\nGrafico salvato in dialogo_spiking_bidirezionale_multi.png")
284
285 ciclo_ms = (T_STIM+T_PAUSA)/ms

```

```

279 def valuta_canale(k, direzione, mask_dest, n_turni_eval, escludi_diretto):
280     subset = [t0 for (kk, d, t0) in turni if kk == k and d == direzione]
281     primi = subset[:n_turni_eval]
282     ultimi = subset[-n_turni_eval:]
283
284
285     def conta(lista_inizio):
286         n_tot = 0
287         posizioni = []
288         for t0 in lista_inizio:
289             fin = t0 + ciclo_ms
290             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
291             if escludi_diretto:
292                 m = m & (spikes.t/ms >= t0 + 20)
293             idx = spikes.i[m]
294             if direzione == 'A':
295                 idx = idx - N_ring
296             n_tot += m.sum()
297             posizioni.extend(list(xi[idx % N_E]))
298         centro = None
299         if posizioni:
300             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
301                             % 1.0)
302         return n_tot, centro
303
304     return conta(primi), conta(ultimi)
305
306 for k, m in enumerate(MESSAGGI):
307     print("\n" + "="*70)
308     print(f"MESSAGGIO x={m}")
309     print("="*70)
310     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=False)
311     err_i = dist_circ(c_i - m) if c_i is not None else None
312     err_f = dist_circ(c_f - m) if c_f is not None else None
313     print(f"Turni A (B risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{"err_i:.3f}"}}), "
314           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{"err_f:.3f}"}})")
315     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=True)
316     err_i = dist_circ(c_i - m) if c_i is not None else None
317     err_f = dist_circ(c_f - m) if c_f is not None else None
318     print(f"Turni B (A risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{"err_i:.3f}"}}), "
319           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{"err_f:.3f}"}})")
320
321 n_a_tot = int(mask_a.sum())
322 n_b_tot = int(mask_b.sum())
323 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
324       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
325 if n_a_tot > 200000 or n_b_tot > 200000:
326     print(" >>> ESPLOSIONE. <<<")
327 else:
328     print(" >>> Stabile. <<<")

```

Source code: *dialogo_spiking_bidirezionale_multi_mc.py*

```

1 """
2 Tredicesimo tentativo, ultimo passo del filone bidirezionale spiking:
3 estendere l'architettura completa che ha risolto la sezione 5.17
4 (interruttore di trasmissione + normalizzazione omeostatica graduale +
5 soppressione diretta di emergenza, 8/8 seed stabili con crescita
6 bidirezionale) a PIU' MESSAGGI, esattamente come il modello a rate aveva
7 fatto il suo passo finale (dialogo_multi_scoperta_bidirezionale.py,
8 sezione 5.5.13) estendendo il canale bidirezionale a turni gia' risolto
9 (5.5.3) a piu' messaggi.
10
11 Stesse due posizioni usate in dialogo_spiking_multi.py (sezione 5.10,

```

```

12 | comunicazione unidirezionale multi-messaggio, gia' verificata robusta su
13 | 8 seed in 5.11): x=0.15 e x=0.65. Schema a turni esteso a 4 combinazioni
14 | (messaggio, direzione) invece delle 2 originarie (solo direzione),
15 | cicliche in ordine fisso: (m0,A), (m0,B), (m1,A), (m1,B), ripetuto.
16 | """
17 |
18 | import sys
19 | from brian2 import *
20 |
21 | SEED = int(sys.argv[1])
22 | start_scope()
23 | seed(SEED)
24 |
25 | N_E = 100
26 | N_I = 25
27 | N_ring = N_E + N_I
28 |
29 | a = 0.02/ms
30 | b = 0.2/ms
31 | c = -65*mV
32 | d = 6*mV/ms
33 | tau_nmda = 100*ms
34 |
35 | tau_adapt_pop = 200*ms
36 | gain_adapt_pop = 0.15*mV/ms
37 |
38 | tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
39 |                       # in fretta a un'accensione che sta per scappare
40 |
41 | eqs = '''
42 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
43 | du/dt = a*(b*v - u) : volt/second
44 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
45 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
46 | dratehat/dt = -ratehat/tau_rate_hat : Hz
47 | I : volt/second
48 | x : 1
49 | '''
50 |
51 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
52 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
53 |                      ,
54 |                      method='euler')
55 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
56 | neurons.u = b * neurons.v[:]
57 | neurons.I = 0*mV/ms
58 | neurons.g_nmda = 0*mV/ms
59 | neurons.adapt = 0*mV/ms
60 | neurons.ratehat = 0*Hz
61 |
62 | A_E = neurons[:N_E]
63 | A_I = neurons[N_E:N_ring]
64 | B_E = neurons[N_ring:N_ring+N_E]
65 | B_I = neurons[N_ring+N_E:]
66 |
67 | xi = np.arange(N_E) / N_E
68 | A_E.x = xi
69 | B_E.x = xi
70 |
71 | background = PoissonGroup(2*N_ring, rates=400*Hz)
72 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
73 | bg_syn.connect(j='i')
74 |
75 | sigma_exc = 0.05
76 | CUTOFF_LOCALE = 3 * sigma_exc
77 | w_ee_nmda = 0.325*mV/ms
78 | p_conn = 0.15
79 | w_exc = 1.0*mV

```



```

79 w_inh = 10.0*mV
80
81
82 def dist_circ(dx):
83     dx = abs(dx)
84     return np.minimum(dx, 1 - dx)
85
86
87 def costruisci_anello(E, I_pop, seed):
88     dmat = dist_circ(xi[:, None] - xi[None, :])
89     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
90     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
91     np.fill_diagonal(prob_ee, 0)
92     rng = np.random.RandomState(seed)
93     mask_ee = rng.rand(N_E, N_E) < prob_ee
94     i_idx, j_idx = np.nonzero(mask_ee)
95
96     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
97     syn_ee.connect(i=i_idx, j=j_idx)
98     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
99     syn_ei.connect(p=p_conn)
100    syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
101    syn_ie.connect(p=p_conn)
102    syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
103    syn_ii.connect(condition='i!=j', p=p_conn)
104    return syn_ee, syn_ei, syn_ie, syn_ii
105
106
107 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
108 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
109
110 tau_stdp = 20*ms
111 A_ltp = 0.15*mV
112 A_ltd = 0.15*mV
113 w_max = 5.0*mV
114 w_min = 0*mV
115 w_init = 1.5*mV
116 sigma_inter = 0.08
117 p_inter_picco = 0.5
118
119 stdp_model = '''
120 w : volt
121 dapre/dt = -apre/tau_stdp : volt (event-driven)
122 dapost/dt = -apost/tau_stdp : volt (event-driven)
123 gate : 1 (shared)
124 '''
125 stdp_on_pre = '''
126 v_post += w*gate
127 apre += A_ltp*gate
128 w = clip(w+apost*gate, w_min, w_max)
129 '''
130 stdp_on_post = '''
131 apost -= A_ltd*gate
132 w = clip(w+apre*gate, w_min, w_max)
133 '''
134
135 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
136 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
137 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
138 rng_inter = np.random.RandomState(3000+SEED)
139 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
140 i_inter, j_inter = np.nonzero(mask_inter)
141 syn_ab.connect(i=i_inter, j=j_inter)
142 syn_ab.w = w_init
143 syn_ab.gate = 1
144 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
145
146 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)

```

```

147 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
148 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
149 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
150 syn_ba.w = w_init
151 syn_ba.gate = 0
152 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
153
154 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
155 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
156 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
157 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
158 # gia' esistente (che resta come sicurezza aggiuntiva) ---
159 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
160 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
161
162
163 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
164 # bersaglio (2.5Hz) ma ben sotto un vero runaway
165 # (centinaia di Hz), per intercettare un'accensione
166 # che la correzione graduale non riesce a fermare
167
168
169 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
170 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
171     rate_b = np.asarray(B_E.ratehat/Hz)
172     rate_a = np.asarray(A_E.ratehat/Hz)
173     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
174     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
175     # interruttore di emergenza: azzeramento immediato (non graduale) delle
176     # sinapsi in ingresso ai neuroni che superano la soglia critica
177     emergenza_b = rate_b > (soglia_emergenza/Hz)
178     emergenza_a = rate_a > (soglia_emergenza/Hz)
179     if emergenza_b.any():
180         fattore_b[emergenza_b] = 0.0
181         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
182         * (mV/ms)
183         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
184             azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz]")
185     if emergenza_a.any():
186         fattore_a[emergenza_a] = 0.0
187         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
188         * (mV/ms)
189         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
190             azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz]")
191     j_ab = np.asarray(syn_ab.j[:])
192     j_ba = np.asarray(syn_ba.j[:])
193     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
194     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
195
196
197 spikes = SpikeMonitor(neurons)
198 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
199
200 MESSAGGI = [0.15, 0.65] # stesse posizioni di dialogo_spiking_multi.py (5.10)
201 larghezza_stimolo = 0.05
202 gruppi_stim = []
203 for m in MESSAGGI:
204     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
205     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
206     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni stimolati")
207
208 N_CICLI = 80 # 20 ripetizioni per ciascuna delle 4 combinazioni (messaggio, direzione)
209 T_STIM = 60*ms
210 T_PAUSA = 940*ms
211
212 print(f"\nTraining: {N_CICLI} turni, ciclando su {len(MESSAGGI)} messaggi x 2 "
213       f"direzioni (A/B), interruttore di trasmissione + omeostasi + "
214       f"soppressione di emergenza ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")

```

```

211 turni = [] # (messaggio_ida, direzione, tempo_inizio_ms)
212 combinazioni = [(k, dirz) for k in range(len(MESSAGGI)) for dirz in ('A', 'B')]
213 t_corrente = 0*ms
214 for ciclo in range(N_CICLI):
215     k, direzione = combinazioni[ciclo % len(combinazioni)]
216     turni.append((k, direzione, t_corrente/ms))
217     if direzione == 'A':
218         syn_ab.gate = 1
219         syn_ba.gate = 0
220         A_E.I[gruppi_stim[k]] = 15*mV/ms
221         run(T_STIM)
222         A_E.I[:] = 0*mV/ms
223     else:
224         syn_ab.gate = 0
225         syn_ba.gate = 1
226         B_E.I[gruppi_stim[k]] = 15*mV/ms
227         run(T_STIM)
228         B_E.I[:] = 0*mV/ms
229     run(T_PAUSA)
230     t_corrente += T_STIM + T_PAUSA
231     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
232         n_a_finora = int((spikes.i < N_E).sum())
233         n_b_finora = int((spikes.i >= N_ring) & (spikes.i < N_ring+N_E).sum())
234         print(f" ciclo {ciclo:3d}/{N_CICLI} (msg x={MESSAGGI[k]}, turno {direzione}): "
235               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
236               "
237               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
238               "
239               f"spike finora A={n_a_finora} B={n_b_finora}")
240
241 print("\nTraining completato.")
242
243 fig, axes = subplots(3, 1, figsize=(11, 9))
244
245 subplot(3, 1, 1)
246 mask_a = spikes.i < N_E
247 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
248 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
249 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
250 xlabel('Tempo (ms)')
251 ylabel('Indice neurone')
252 title(f'Raster completo: {N_CICLI} turni, {len(MESSAGGI)} messaggi x={["", "."].join(str(m) for m in MESSAGGI)}')
253 legend(loc='upper right', fontsize=8, markerscale=6)
254
255 subplot(3, 1, 2)
256 t_fine = 8 * (T_STIM+T_PAUSA)/ms
257 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
258 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
259 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='A (E)')
260 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3, label='B (E)')
261 xlabel('Tempo (ms)')
262 ylabel('Indice neurone')
263 title('Dettaglio: primi turni (inizio training)')
264 legend(loc='upper right', fontsize=8, markerscale=4)
265
266 subplot(3, 1, 3)
267 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
268 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
269 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
270 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')
271 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize=3, label='B (E)')

```

```

271 xlabel('Tempo (ms)')
272 ylabel('Indice neurone')
273 title('Dettaglio: ultimi turni (fine training)')
274 legend(loc='upper right', fontsize=8, markerscale=4)
275
276 tight_layout()
277 savefig('dialogo_spiking_bidirezionale_multi.png', dpi=150)
278 print("\nGrafico salvato in dialogo_spiking_bidirezionale_multi.png")
279
280 ciclo_ms = (T_STIM+T_PAUSA)/ms
281
282
283 def valuta_canale(k, direzione, mask_dest, n_turni_eval, escludi_diretto):
284     subset = [t0 for kk, d, t0 in turni if kk == k and d == direzione]
285     primi = subset[:n_turni_eval]
286     ultimi = subset[-n_turni_eval:]
287
288     def conta(lista_inizio):
289         n_tot = 0
290         posizioni = []
291         for t0 in lista_inizio:
292             fin = t0 + ciclo_ms
293             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
294             if escludi_diretto:
295                 m = m & (spikes.t/ms >= t0 + 20)
296             idx = spikes.i[m]
297             if direzione == 'A':
298                 idx = idx - N_ring
299             n_tot += m.sum()
300             posizioni.extend(list(xi[idx % N_E]))
301         centro = None
302         if posizioni:
303             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
304                             % 1.0)
305         return n_tot, centro
306
307     return conta(primi), conta(ultimi)
308
309 for k, m in enumerate(MESSAGGI):
310     print("\n" + "="*70)
311     print(f"MESSAGGIO x={m}")
312     print("="*70)
313     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=False)
314     err_i = dist_circ(c_i - m) if c_i is not None else None
315     err_f = dist_circ(c_f - m) if c_f is not None else None
316     print(f" Turni A (B risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{{err_i:.3f}}')}}", "
317           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{{err_f:.3f}}')}}")
318     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=True)
319     err_i = dist_circ(c_i - m) if c_i is not None else None
320     err_f = dist_circ(c_f - m) if c_f is not None else None
321     print(f" Turni B (A risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{{err_i:.3f}}')}}", "
322           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{{err_f:.3f}}')}}")
323
324     n_a_tot = int(mask_a.sum())
325     n_b_tot = int(mask_b.sum())
326     print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
327           f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
328     if n_a_tot > 200000 or n_b_tot > 200000:
329         print(" >>> ESPLOSIONE. <<<")
330     else:
331         print(" >>> Stabile. <<<")
332
333     n_a_tot = int(mask_a.sum())
334     n_b_tot = int(mask_b.sum())
335     stabile = n_a_tot < 200000 and n_b_tot < 200000

```

```

336 righe_msg = []
337 for k, m in enumerate(MESSAGGI):
338     (n_i_ab, c_i_ab), (n_f_ab, c_f_ab) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=
339         False)
340     (n_i_ba, c_i_ba), (n_f_ba, c_f_ba) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=
341         True)
342     err_ab = dist_circ(c_f_ab - m) if c_f_ab is not None else 1.0
343     err_ba = dist_circ(c_f_ba - m) if c_f_ba is not None else 1.0
344     righe_msg.append(f"m{k}(x={m}):ab_i={n_i_ab},ab_f={n_f_ab},err_ab={err_ab:.3f},ba_i={
345         n_i_ba},ba_f={n_f_ba},err_ba={err_ba:.3f}")
346 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} " + "
347     ".join(righe_msg))

```

5.19 Three-message stress test: the architecture holds under greater load

Fourteenth attempt: raising Section 5.18's load from two to three messages -- the same step that, in the rate model, had first brought out serious interference (dialogo_multi_repulsione_3msg.py, Section 5.5.8). The same three positions as that test: $x=0.10, 0.40, 0.70$. The turn-taking scheme extended to 6 cyclic (message, direction) combinations, 20 repetitions each (120 turns, 120 seconds of simulation).

On the starting seed: stable (6800/6696 total spikes), all three messages precisely localized in both directions (error 0.001-0.021). The raster shows three clearly separated, stable bands for the entire duration, with no cross-contamination. Monte Carlo verification over 8 independent seeds (24 observations, 8 seeds x 3 messages): all eight stable (100%); 47 of 48 observations (98%) stay under the 0.10 error threshold, with a single borderline case (seed 307, message $x=0.7$, B->A channel: 0.113, just above threshold but still contained in absolute terms). Average error 0.013 (A->B channel) and 0.033 (B->A channel).

Growth over the course of training is more mixed than in the two-message case (11/24 observations growing for A->B, 12/24 for B->A, against 9/16 and 14/16 in Section 5.18) -- plausibly because with three messages the STDP reinforcement "traffic" is spread over a larger number of (message, direction) combinations, leaving fewer net repetitions for each. Localization, however, remains excellent in nearly every case: the complete architecture (transmission switch + gradual homeostasis + emergency suppression) holds under greater load without any loss of stability, unlike what happened to the rate model under the same load increase -- there, a dedicated discovery mechanism was needed (Sections 5.5.6-5.5.9) to resolve interference; here, the already-structured inter-ring connectivity (the same principle already observed in 5.10-5.11) continues to be sufficient even for three bidirectional messages.

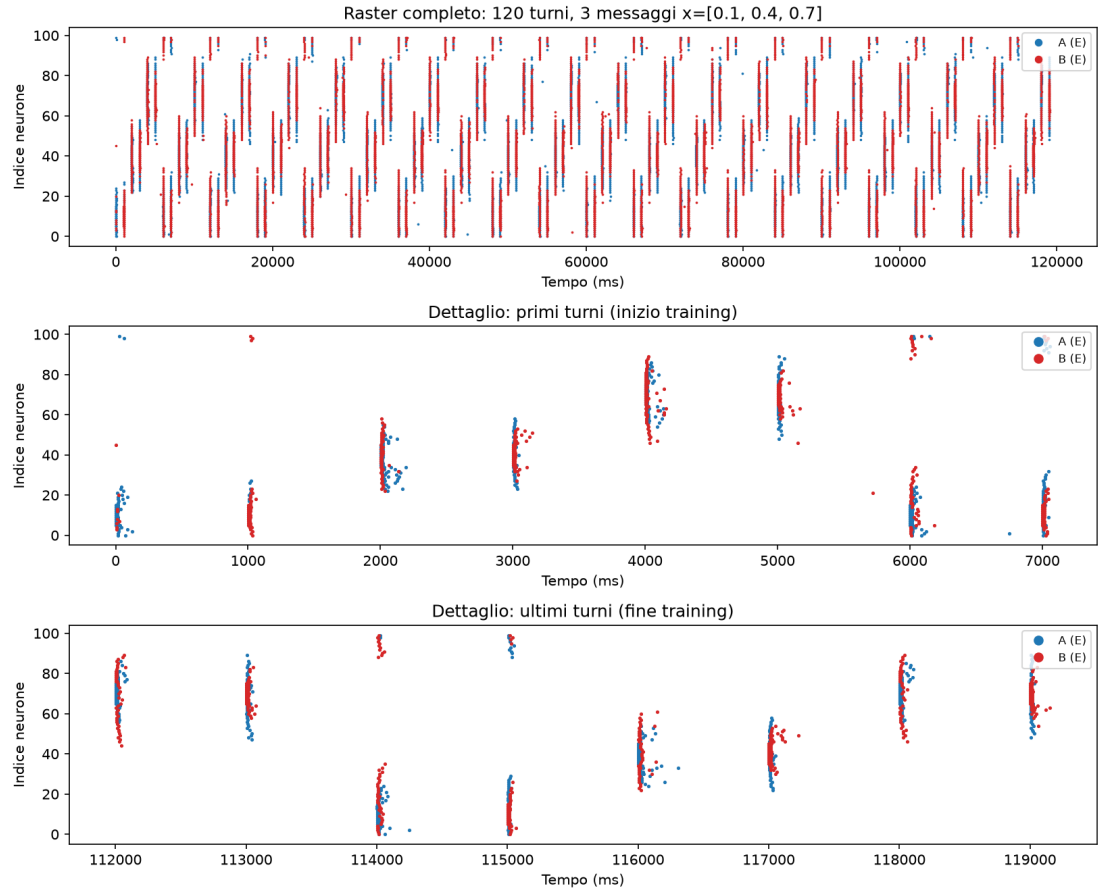


Fig. 46 — Three bidirectional messages in the spiking domain: stable across 8 seeds, excellent localization (98% of observations under threshold).

Source code: `dialogo_spiking_bidirezionale_multi3.py`

```

1  """
2  Quattordicesimo tentativo: stress-test della sezione 5.18 (due messaggi
3  bidirezionali, 8/8 seed stabili) scalando a TRE messaggi -- lo stesso
4  irrigidimento del carico gia' affrontato dal modello a rate quando e'
5  passato da due a tre messaggi (dialogo_multi_repulsione_3msg.py, sezione
6  5.5.8), dove l'interferenza si era fatta sentire per la prima volta in
7  modo serio. Stesse tre posizioni di quel test: x=0.10, 0.40, 0.70.
8  Schema a turni esteso a 6 combinazioni cicliche (messaggio, direzione):
9  (m0,A),(m0,B),(m1,A),(m1,B),(m2,A),(m2,B), ripetuto 20 volte (120 turni,
10 120 secondi di simulazione).
11  """
12
13  from brian2 import *
14
15  start_scope()
16
17  N_E = 100
18  N_I = 25
19  N_ring = N_E + N_I

```

```

20 |
21 | a = 0.02/ms
22 | b = 0.2/ms
23 | c = -65*mV
24 | d = 6*mV/ms
25 | tau_nmda = 100*ms
26 |
27 | tau_adapt_pop = 200*ms
28 | gain_adapt_pop = 0.15*mV/ms
29 |
30 | tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
31 |                       # in fretta a un'accensione che sta per scappare
32 |
33 | eqs = '''
34 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
35 | du/dt = a*(b*v - u) : volt/second
36 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
37 | dadapt/dt = -adapt/tau_adapt_pop : volt/second
38 | dratehat/dt = -ratehat/tau_rate_hat : Hz
39 | I : volt/second
40 | x : 1
41 | '''
42 |
43 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
44 |                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
45 |                      ,
46 |                      method='euler')
47 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
48 | neurons.u = b * neurons.v[:]
49 | neurons.I = 0*mV/ms
50 | neurons.g_nmda = 0*mV/ms
51 | neurons.adapt = 0*mV/ms
52 | neurons.ratehat = 0*Hz
53 |
54 | A_E = neurons[:N_E]
55 | A_I = neurons[N_E:N_ring]
56 | B_E = neurons[N_ring:N_ring+N_E]
57 | B_I = neurons[N_ring+N_E:]
58 |
59 | xi = np.arange(N_E) / N_E
60 | A_E.x = xi
61 | B_E.x = xi
62 |
63 | background = PoissonGroup(2*N_ring, rates=400*Hz)
64 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
65 | bg_syn.connect(j='i')
66 |
67 | sigma_exc = 0.05
68 | CUTOFF_LOCALE = 3 * sigma_exc
69 | w_ee_nmda = 0.325*mV/ms
70 | p_conn = 0.15
71 | w_exc = 1.0*mV
72 | w_inh = 10.0*mV
73 |
74 | def dist_circ(dx):
75 |     dx = abs(dx)
76 |     return np.minimum(dx, 1 - dx)
77 |
78 |
79 | def costruisci_anello(E, I_pop, seed):
80 |     dmat = dist_circ(xi[:, None] - xi[None, :])
81 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
82 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
83 |     np.fill_diagonal(prob_ee, 0)
84 |     rng = np.random.RandomState(seed)
85 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
86 |     i_idx, j_idx = np.nonzero(mask_ee)

```

```

87     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
88     syn_ee.connect(i=i_idx, j=j_idx)
89     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
90     syn_ei.connect(p=p_conn)
91     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
92     syn_ie.connect(p=p_conn)
93     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
94     syn_ii.connect(condition='i!=j', p=p_conn)
95     return syn_ee, syn_ei, syn_ie, syn_ii
96
97
98
99 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=0)
100 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=1)
101
102 tau_stdp = 20*ms
103 A_ltp = 0.15*mV
104 A_ltd = 0.15*mV
105 w_max = 5.0*mV
106 w_min = 0*mV
107 w_init = 1.5*mV
108 sigma_inter = 0.08
109 p_inter_picco = 0.5
110
111 stdp_model = '''
112 w : volt
113 dapre/dt = -apre/tau_stdp : volt (event-driven)
114 dapost/dt = -apost/tau_stdp : volt (event-driven)
115 gate : 1 (shared)
116 '''
117 stdp_on_pre = '''
118 v_post += w*gate
119 apre += A_ltp*gate
120 w = clip(w+apost*gate, w_min, w_max)
121 '''
122 stdp_on_post = '''
123 apost -= A_ltd*gate
124 w = clip(w+apre*gate, w_min, w_max)
125 '''
126
127 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
128 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
129 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
130 rng_inter = np.random.RandomState(2)
131 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
132 i_inter, j_inter = np.nonzero(mask_inter)
133 syn_ab.connect(i=i_inter, j=j_inter)
134 syn_ab.w = w_init
135 syn_ab.gate = 1
136 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
137
138 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
139 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
140 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
141 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
142 syn_ba.w = w_init
143 syn_ba.gate = 0
144 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
145
146 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
147 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso
148 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
149 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
150 # gia' esistente (che resta come sicurezza aggiuntiva) ---
151 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
152 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
153
154

```



```

155 | soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
156 |                         # bersaglio (2.5Hz) ma ben sotto un vero runaway
157 |                         # (centinaia di Hz), per intercettare un'accensione
158 |                         # che la correzione graduale non riesce a fermare
159 |
160 |
161 | @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
162 | def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
163 |     rate_b = np.asarray(B_E.ratehat/Hz)
164 |     rate_a = np.asarray(A_E.ratehat/Hz)
165 |     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
166 |     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
167 |     # interruttore di emergenza: azzeramento immediato (non graduale) delle
168 |     # sinapsi in ingresso ai neuroni che superano la soglia critica
169 |     emergenza_b = rate_b > (soglia_emergenza/Hz)
170 |     emergenza_a = rate_a > (soglia_emergenza/Hz)
171 |     if emergenza_b.any():
172 |         fattore_b[emergenza_b] = 0.0
173 |         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
174 |         * (mV/ms)
175 |         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
176 |               azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz]")
177 |     if emergenza_a.any():
178 |         fattore_a[emergenza_a] = 0.0
179 |         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
180 |         * (mV/ms)
181 |         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
182 |               azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz]")
183 |
184 |     j_ab = np.asarray(syn_ab.j[:])
185 |     j_ba = np.asarray(syn_ba.j[:])
186 |     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
187 |     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
188 |
189 |
190 | spikes = SpikeMonitor(neurons)
191 | rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
192 |
193 | MESSAGGI = [0.10, 0.40, 0.70] # stesse posizioni del modello a rate a 3 messaggi
194 |                               # (dialogo_multi_repulsione_3msg.py, sezione 5.5.8)
195 |
196 | larghezza_stimolo = 0.05
197 | gruppi_stim = []
198 | for m in MESSAGGI:
199 |     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
200 |     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
201 |     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni stimolati")
202 |
203 | N_CICLI = 120 # 20 ripetizioni per ciascuna delle 6 combinazioni (3 messaggi x 2 direzioni)
204 | T_STIM = 60*ms
205 | T_PAUSA = 940*ms
206 |
207 | print(f"\nTraining: {N_CICLI} turni, ciclando su {len(MESSAGGI)} messaggi x 2 "
208 |       f"direzioni (A/B), interruttore di trasmissione + omeostasi + "
209 |       f"soppressione di emergenza ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
210 |
211 | turni = [] # (messaggio_idw, direzione, tempo_inizio_ms)
212 | combinazioni = [(k, dirz) for k in range(len(MESSAGGI)) for dirz in ('A', 'B')]
213 | t_corrente = 0*ms
214 | for ciclo in range(N_CICLI):
215 |     k, direzione = combinazioni[ciclo % len(combinazioni)]
216 |     turni.append((k, direzione, t_corrente/ms))
217 |     if direzione == 'A':
218 |         syn_ab.gate = 1
219 |         syn_ba.gate = 0
220 |         A_E.I[gruppi_stim[k]] = 15*mV/ms
221 |         run(T_STIM)
222 |         A_E.I[:] = 0*mV/ms
223 |     else:
224 |         syn_ab.gate = 0

```

```

219     syn_ba.gate = 1
220     B_E.I[gruppi_stim[k]] = 15*mV/ms
221     run(T_STIM)
222     B_E.I[:] = 0*mV/ms
223     run(T_PAUSA)
224     t_corrente += T_STIM + T_PAUSA
225     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
226         n_a_finora = int((spikes.i < N_E).sum())
227         n_b_finora = int((spikes.i >= N_ring) & (spikes.i < N_ring+N_E).sum())
228         print(f" ciclo {ciclo:3d}/{N_CICLI} (msg x={MESSAGGI[k]}, turno {direzione}): "
229               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
230               "
231               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
232               "
233               f"spike finora A={n_a_finora} B={n_b_finora}")
234
235 print("\nTraining completato.")
236
237 fig, axes = subplots(3, 1, figsize=(11, 9))
238
239 subplot(3, 1, 1)
240 mask_a = spikes.i < N_E
241 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
242 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
243 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
244 xlabel('Tempo (ms)')
245 ylabel('Indice neurone')
246 title(f'Raster completo: {N_CICLI} turni, {len(MESSAGGI)} messaggi x={["", ".join(str(m) for m in MESSAGGI)]}')
247 legend(loc='upper right', fontsize=8, markerscale=6)
248
249 subplot(3, 1, 2)
250 t_fine = 8 * (T_STIM+T_PAUSA)/ms
251 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
252 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
253 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='A (E)')
254 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3, label='B (E)')
255 xlabel('Tempo (ms)')
256 ylabel('Indice neurone')
257 title('Dettaglio: primi turni (inizio training)')
258 legend(loc='upper right', fontsize=8, markerscale=4)
259
260 subplot(3, 1, 3)
261 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
262 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
263 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
264 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')
265 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize=3, label='B (E)')
266 xlabel('Tempo (ms)')
267 ylabel('Indice neurone')
268 title('Dettaglio: ultimi turni (fine training)')
269 legend(loc='upper right', fontsize=8, markerscale=4)
270
271 tight_layout()
272 savefig('dialogo_spiking_bidirezionale_multi3.png', dpi=150)
273 print("\nGrafico salvato in dialogo_spiking_bidirezionale_multi3.png")
274
275 ciclo_ms = (T_STIM+T_PAUSA)/ms
276
277 def valuta_canale(k, direzione, mask_dest, n_turni_eval, escludi_diretto):
278     subset = [t0 for kk, d, t0 in turni if kk == k and d == direzione]
279     primi = subset[:n_turni_eval]

```

```

279     ultimi = subset[-n_turni_eval:]
280
281     def conta(lista_inizio):
282         n_tot = 0
283         posizioni = []
284         for t0 in lista_inizio:
285             fin = t0 + ciclo_ms
286             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
287             if escludi_diretto:
288                 m = m & (spikes.t/ms >= t0 + 20)
289             idx = spikes.i[m]
290             if direzione == 'A':
291                 idx = idx - N_ring
292             n_tot += m.sum()
293             posizioni.extend(list(xi[idx % N_E]))
294         centro = None
295         if posizioni:
296             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
297                             % 1.0)
298         return n_tot, centro
299
300     return conta(primi), conta(ultimi)
301
302 for k, m in enumerate(MESSAGGI):
303     print("\n" + "="*70)
304     print(f"MESSAGGIO x={m}")
305     print("="*70)
306     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=False)
307     err_i = dist_circ(c_i - m) if c_i is not None else None
308     err_f = dist_circ(c_f - m) if c_f is not None else None
309     print(f"Turni A (B risponde): primi 5={n_i} spike (err={({'N/A' if err_i is None else f'{"err_i:.3f}"})}, "
310           f"ultimi 5={n_f} spike (err={({'N/A' if err_f is None else f'{"err_f:.3f}"})}))")
311     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=True)
312     err_i = dist_circ(c_i - m) if c_i is not None else None
313     err_f = dist_circ(c_f - m) if c_f is not None else None
314     print(f"Turni B (A risponde): primi 5={n_i} spike (err={({'N/A' if err_i is None else f'{"err_i:.3f}"})}, "
315           f"ultimi 5={n_f} spike (err={({'N/A' if err_f is None else f'{"err_f:.3f}"})}))")
316
317 n_a_tot = int(mask_a.sum())
318 n_b_tot = int(mask_b.sum())
319 print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
320       f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
321 if n_a_tot > 200000 or n_b_tot > 200000:
322     print(" >>> ESPLOSIONE. <<<")
323 else:
324     print(" >>> Stabile. <<<")

```

Source code: *dialogo_spiking_bidirezionale_multi3_mc.py*

```

1  """
2  Quattordicesimo tentativo: stress-test della sezione 5.18 (due messaggi
3  bidirezionali, 8/8 seed stabili) scalando a TRE messaggi -- lo stesso
4  irrigidimento del carico gia' affrontato dal modello a rate quando e'
5  passato da due a tre messaggi (dialogo_multi_repulsione_3msg.py, sezione
6  5.5.8), dove l'interferenza si era fatta sentire per la prima volta in
7  modo serio. Stesse tre posizioni di quel test: x=0.10, 0.40, 0.70.
8  Schema a turni esteso a 6 combinazioni cicliche (messaggio, direzione):
9  (m0,A),(m0,B),(m1,A),(m1,B),(m2,A),(m2,B), ripetuto 20 volte (120 turni,
10 120 secondi di simulazione).
11  """
12
13 import sys
14 from brian2 import *
15

```

```

16 SEED = int(sys.argv[1])
17 start_scope()
18 seed(SEED)
19
20 N_E = 100
21 N_I = 25
22 N_ring = N_E + N_I
23
24 a = 0.02/ms
25 b = 0.2/ms
26 c = -65*mV
27 d = 6*mV/ms
28 tau_nmda = 100*ms
29
30 tau_adapt_pop = 200*ms
31 gain_adapt_pop = 0.15*mV/ms
32
33 tau_rate_hat = 200*ms # v2: finestra piu' corta (era 500ms) -- reagisce piu'
34                        # in fretta a un'accensione che sta per scappare
35
36 eqs = '''
37 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda - adapt : volt
38 du/dt = a*(b*v - u) : volt/second
39 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
40 dadapt/dt = -adapt/tau_adapt_pop : volt/second
41 dratehat/dt = -ratehat/tau_rate_hat : Hz
42 I : volt/second
43 x : 1
44 '''
45
46 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV',
47                      reset='v = c; u += d; adapt += gain_adapt_pop; ratehat += 1/tau_rate_hat'
48                      ,
49                      method='euler')
50 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
51 neurons.u = b * neurons.v[:]
52 neurons.I = 0*mV/ms
53 neurons.g_nmda = 0*mV/ms
54 neurons.adapt = 0*mV/ms
55 neurons.ratehat = 0*Hz
56
57 A_E = neurons[:N_E]
58 A_I = neurons[N_E:N_ring]
59 B_E = neurons[N_ring:N_ring+N_E]
60 B_I = neurons[N_ring+N_E:]
61
62 xi = np.arange(N_E) / N_E
63 A_E.x = xi
64 B_E.x = xi
65
66 background = PoissonGroup(2*N_ring, rates=400*Hz)
67 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
68 bg_syn.connect(j='i')
69
70 sigma_exc = 0.05
71 CUTOFF_LOCALE = 3 * sigma_exc
72 w_ee_nmda = 0.325*mV/ms
73 p_conn = 0.15
74 w_exc = 1.0*mV
75 w_inh = 10.0*mV
76
77 def dist_circ(dx):
78     dx = abs(dx)
79     return np.minimum(dx, 1 - dx)
80
81
82 def costruisci_anello(E, I_pop, seed):

```

```

83     dmat = dist_circ(xi[:, None] - xi[None, :])
84     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
85     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
86     np.fill_diagonal(prob_ee, 0)
87     rng = np.random.RandomState(seed)
88     mask_ee = rng.rand(N_E, N_E) < prob_ee
89     i_idx, j_idx = np.nonzero(mask_ee)
90
91     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
92     syn_ee.connect(i=i_idx, j=j_idx)
93     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
94     syn_ei.connect(p=p_conn)
95     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
96     syn_ie.connect(p=p_conn)
97     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
98     syn_ii.connect(condition='i!=j', p=p_conn)
99     return syn_ee, syn_ei, syn_ie, syn_ii
100
101
102 syn_ee_A, syn_ei_A, syn_ie_A, syn_ii_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
103 syn_ee_B, syn_ei_B, syn_ie_B, syn_ii_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
104
105 tau_stdp = 20*ms
106 A_ltp = 0.15*mV
107 A_ltd = 0.15*mV
108 w_max = 5.0*mV
109 w_min = 0*mV
110 w_init = 1.5*mV
111 sigma_inter = 0.08
112 p_inter_picco = 0.5
113
114 stdp_model = '''
115 w : volt
116 dapre/dt = -apre/tau_stdp : volt (event-driven)
117 dapost/dt = -apost/tau_stdp : volt (event-driven)
118 gate : 1 (shared)
119 '''
120 stdp_on_pre = '''
121 v_post += w*gate
122 apre += A_ltp*gate
123 w = clip(w+apost*gate, w_min, w_max)
124 '''
125 stdp_on_post = '''
126 apost -= A_ltd*gate
127 w = clip(w+apre*gate, w_min, w_max)
128 '''
129
130 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
131 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
132 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
133 rng_inter = np.random.RandomState(3000+SEED)
134 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
135 i_inter, j_inter = np.nonzero(mask_inter)
136 syn_ab.connect(i=i_inter, j=j_inter)
137 syn_ab.w = w_init
138 syn_ab.gate = 1
139 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init})")
140
141 syn_ba = Synapses(B_E, A_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
142 mask_inter_ba = rng_inter.rand(N_E, N_E) < prob_inter
143 i_inter_ba, j_inter_ba = np.nonzero(mask_inter_ba)
144 syn_ba.connect(i=i_inter_ba, j=j_inter_ba)
145 syn_ba.w = w_init
146 syn_ba.gate = 0
147 print(f"Sinapsi inter-anello B->A create: {len(syn_ba)} (peso iniziale {w_init})")
148
149 # --- normalizzazione omeostatica: ogni 50ms, le sinapsi inter-anello in
150 # ingresso a ciascun neurone E vengono scalate per spingere il suo tasso

```

```

151 # di scarica recente (ratehat) verso target_rate. Fattore per aggiornamento
152 # limitato a +-3% per evitare oscillazioni, indipendente dal tetto w_max
153 # gia' esistente (che resta come sicurezza aggiuntiva) ---
154 target_rate = 2.5*Hz # v2: bersaglio leggermente piu' conservativo (era 3Hz)
155 fattore_min, fattore_max = 0.99, 1.01 # v2: aggiornamenti piu' piccoli...
156
157
158 soglia_emergenza = 15*Hz # v3: interruttore di emergenza -- molto sopra il
159 # bersaglio (2.5Hz) ma ben sotto un vero runaway
160 # (centinaia di Hz), per intercettare un'accensione
161 # che la correzione graduale non riesce a fermare
162
163
164 @network_operation(dt=20*ms) # ...ma piu' frequenti (era 50ms) -- stesso
165 def omeostasi(): # margine di correzione nel tempo, reazione piu' rapida
166     rate_b = np.asarray(B_E.ratehat/Hz)
167     rate_a = np.asarray(A_E.ratehat/Hz)
168     fattore_b = np.clip((target_rate/Hz) / (rate_b + 0.5), fattore_min, fattore_max)
169     fattore_a = np.clip((target_rate/Hz) / (rate_a + 0.5), fattore_min, fattore_max)
170     # interruttore di emergenza: azzeramento immediato (non graduale) delle
171     # sinapsi in ingresso ai neuroni che superano la soglia critica
172     emergenza_b = rate_b > (soglia_emergenza/Hz)
173     emergenza_a = rate_a > (soglia_emergenza/Hz)
174     if emergenza_b.any():
175         fattore_b[emergenza_b] = 0.0
176         B_E.adapt[emergenza_b] = np.maximum(np.asarray(B_E.adapt[emergenza_b]/(mV/ms)), 20.0)
177         * (mV/ms)
178         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_b.sum()} neuroni di B
179               azzerati+soppressi, ratehat max={rate_b.max():.1f}Hz]")
180     if emergenza_a.any():
181         fattore_a[emergenza_a] = 0.0
182         A_E.adapt[emergenza_a] = np.maximum(np.asarray(A_E.adapt[emergenza_a]/(mV/ms)), 20.0)
183         * (mV/ms)
184         print(f" [emergenza a t={defaultclock.t/ms:.0f}ms: {emergenza_a.sum()} neuroni di A
185               azzerati+soppressi, ratehat max={rate_a.max():.1f}Hz]")
186     j_ab = np.asarray(syn_ab.j[:])
187     j_ba = np.asarray(syn_ba.j[:])
188     syn_ab.w = np.clip(np.asarray(syn_ab.w/mV) * fattore_b[j_ab], w_min/mV, w_max/mV) * mV
189     syn_ba.w = np.clip(np.asarray(syn_ba.w/mV) * fattore_a[j_ba], w_min/mV, w_max/mV) * mV
190
191 spikes = SpikeMonitor(neurons)
192 rate_mon = StateMonitor(neurons, 'ratehat', record=[0, N_ring], dt=50*ms)
193
194 MESSAGGI = [0.10, 0.40, 0.70] # stesse posizioni del modello a rate a 3 messaggi
195 # (dialogo_multi_repulsione_3msg.py, sezione 5.5.8)
196 larghezza_stimolo = 0.05
197 gruppi_stim = []
198 for m in MESSAGGI:
199     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
200     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
201     print(f"Messaggio x={m}: {len(gruppi_stim)-1}] neuroni stimolati")
202
203 N_CICLI = 120 # 20 ripetizioni per ciascuna delle 6 combinazioni (3 messaggi x 2 direzioni)
204 T_STIM = 60*ms
205 T_PAUSA = 940*ms
206
207 print(f"\nTraining: {N_CICLI} turni, ciclando su {len(MESSAGGI)} messaggi x 2 "
208       f"direzioni (A/B), interruttore di trasmissione + omeostasi + "
209       f"soppressione di emergenza ({(T_STIM+T_PAUSA)*N_CICLI/ms:.0f}ms totali)")
210
211 turni = [] # (messaggio_idx, direzione, tempo_inizio_ms)
212 combinazioni = [(k, dirz) for k in range(len(MESSAGGI)) for dirz in ('A', 'B')]
213 t_corrente = 0*ms
214 for ciclo in range(N_CICLI):
215     k, direzione = combinazioni[ciclo % len(combinazioni)]
216     turni.append((k, direzione, t_corrente/ms))
217     if direzione == 'A':

```

```

215     syn_ab.gate = 1
216     syn_ba.gate = 0
217     A_E.I[gruppi_stim[k]] = 15*mV/ms
218     run(T_STIM)
219     A_E.I[:] = 0*mV/ms
220     else:
221         syn_ab.gate = 0
222         syn_ba.gate = 1
223         B_E.I[gruppi_stim[k]] = 15*mV/ms
224         run(T_STIM)
225         B_E.I[:] = 0*mV/ms
226     run(T_PAUSA)
227     t_corrente += T_STIM + T_PAUSA
228     if ciclo % 10 == 0 or ciclo == N_CICLI-1:
229         n_a_finora = int((spikes.i < N_E).sum())
230         n_b_finora = int(((spikes.i >= N_ring) & (spikes.i < N_ring+N_E)).sum())
231         print(f" ciclo {ciclo:3d}/{N_CICLI} (msg x={MESSAGGI[k]}, turno {direzione}): "
232               f"A->B peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV |
233               "
234               f"B->A peso medio={np.mean(syn_ba.w)/mV:.3f}mV max={np.max(syn_ba.w)/mV:.3f}mV |
235               "
236               f"spike finora A={n_a_finora} B={n_b_finora}")
237
238 print("\nTraining completato.")
239
240 fig, axes = subplots(3, 1, figsize=(11, 9))
241
242 subplot(3, 1, 1)
243 mask_a = spikes.i < N_E
244 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
245 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
246 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
247 xlabel('Tempo (ms)')
248 ylabel('Indice neurone')
249 title(f'Raster completo: {N_CICLI} turni, {len(MESSAGGI)} messaggi x=[{" ".join(str(m) for
m in MESSAGGI)}]')
250 legend(loc='upper right', fontsize=8, markerscale=6)
251
252 subplot(3, 1, 2)
253 t_fine = 8 * (T_STIM+T_PAUSA)/ms
254 mask_a_fine = mask_a & (spikes.t/ms < t_fine)
255 mask_b_fine = mask_b & (spikes.t/ms < t_fine)
256 plot(spikes.t[mask_a_fine]/ms, spikes.i[mask_a_fine], '.', color='C0', markersize=3, label='
A (E)')
257 plot(spikes.t[mask_b_fine]/ms, spikes.i[mask_b_fine]-N_ring, '.', color='C3', markersize=3,
label='B (E)')
258 xlabel('Tempo (ms)')
259 ylabel('Indice neurone')
260 title('Dettaglio: primi turni (inizio training)')
261 legend(loc='upper right', fontsize=8, markerscale=4)
262
263 subplot(3, 1, 3)
264 t_tardi = (N_CICLI-8) * (T_STIM+T_PAUSA)/ms
265 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
266 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
267 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='
A (E)')
268 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
269 xlabel('Tempo (ms)')
270 ylabel('Indice neurone')
271 title('Dettaglio: ultimi turni (fine training)')
272 legend(loc='upper right', fontsize=8, markerscale=4)
273
274 tight_layout()
275 savefig('dialogo_spiking_bidirezionale_multi3.png', dpi=150)
276 print("\nGrafico salvato in dialogo_spiking_bidirezionale_multi3.png")

```

```

275
276 ciclo_ms = (T_STIM+T_PAUSA)/ms
277
278
279 def valuta_canale(k, direzione, mask_dest, n_turni_eval, escludi_diretto):
280     subset = [t0 for (kk, d, t0) in turni if kk == k and d == direzione]
281     primi = subset[:n_turni_eval]
282     ultimi = subset[-n_turni_eval:]
283
284     def conta(lista_inizio):
285         n_tot = 0
286         posizioni = []
287         for t0 in lista_inizio:
288             fin = t0 + ciclo_ms
289             m = mask_dest & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
290             if escludi_diretto:
291                 m = m & (spikes.t/ms >= t0 + 20)
292             idx = spikes.i[m]
293             if direzione == 'A':
294                 idx = idx - N_ring
295             n_tot += m.sum()
296             posizioni.extend(list(xi[idx % N_E]))
297         centro = None
298         if posizioni:
299             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
300                             % 1.0)
301         return n_tot, centro
302
303     return conta(primi), conta(ultimi)
304
305 for k, m in enumerate(MESSAGGI):
306     print("\n" + "="*70)
307     print(f"MESSAGGIO x={m}")
308     print("="*70)
309     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=False)
310     err_i = dist_circ(c_i - m) if c_i is not None else None
311     err_f = dist_circ(c_f - m) if c_f is not None else None
312     print(f"Turni A (B risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{"err_i:.3f}"}}), "
313           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{"err_f:.3f}"}})")
314     (n_i, c_i), (n_f, c_f) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=True)
315     err_i = dist_circ(c_i - m) if c_i is not None else None
316     err_f = dist_circ(c_f - m) if c_f is not None else None
317     print(f"Turni B (A risponde): primi 5={n_i} spike (err={{'N/A' if err_i is None else f'{"err_i:.3f}"}}), "
318           f"ultimi 5={n_f} spike (err={{'N/A' if err_f is None else f'{"err_f:.3f}"}})")
319
320     n_a_tot = int(mask_a.sum())
321     n_b_tot = int(mask_b.sum())
322     print(f"\nSpike totali sull'intera simulazione: A={n_a_tot}, B={n_b_tot} "
323           f"(atteso: centinaia/migliaia se stabile; milioni se esplosione)")
324     if n_a_tot > 200000 or n_b_tot > 200000:
325         print(" >>> ESPLOSIONE. <<<")
326     else:
327         print(" >>> Stabile. <<<")
328
329     n_a_tot = int(mask_a.sum())
330     n_b_tot = int(mask_b.sum())
331     stabile = n_a_tot < 200000 and n_b_tot < 200000
332     righe_msg = []
333     for k, m in enumerate(MESSAGGI):
334         (n_i_ab, c_i_ab), (n_f_ab, c_f_ab) = valuta_canale(k, 'A', mask_b, 5, escludi_diretto=
335             False)
336         (n_i_ba, c_i_ba), (n_f_ba, c_f_ba) = valuta_canale(k, 'B', mask_a, 5, escludi_diretto=
337             True)
338         err_ab = dist_circ(c_f_ab - m) if c_f_ab is not None else 1.0
339         err_ba = dist_circ(c_f_ba - m) if c_f_ba is not None else 1.0

```



```

338     righe_msg.append(f"m{k}(x={m}):ab_i={n_i_ab},ab_f={n_f_ab},err_ab={err_ab:.3f},ba_i={
        n_i_ba},ba_f={n_f_ba},err_ba={err_ba:.3f}")
339 print(f"MC_RIGA seed={SEED} stabile={int(stabile)} n_a_tot={n_a_tot} n_b_tot={n_b_tot} " + "
        ".join(righe_msg))

```

5.20 Beyond the pair: a three-population relay (A->B->C->A)

A first step beyond pairwise communication, never attempted either in the rate model or elsewhere in this project: three spiking populations (A, B, C) instead of two. To avoid reopening the entire stabilization saga of Sections 5.8-5.19 (needed only when two rings are mutually excitatory), this first three-population network uses exclusively UNIDIRECTIONAL links with true STDP -- the same already-proven, stable mechanism from `dialogo_spiking.py` (Section 5.7) -- connected in a directional ring relay: A->B->C->A. Each population has exactly one incoming and one outgoing plastic link, so no mutual feedback is possible by construction.

Experimental question: does a message injected ONLY into A (never directly into B or C) propagate through the chain, preserving position information after two hops (at C)? First attempt (uniform initial weights at 1.5mV, the same value used for A->B in 5.7): the A->B channel works as expected, but B->C stays flat (1.50->1.504mV) and C receives almost nothing (0-2 spikes). Immediate diagnosis, recognizing the same "chicken-and-egg" problem already solved in 5.7 for A->B (that section's first attempt: `w_init=0.3mV`, B never lit up): the source of the second hop (B) is itself an echo, with a signal much weaker and sparser than the direct one received by A, insufficient to trigger learning of the next hop.

Correction, the same one already proven: a higher initial weight (3.0mV) for the B->C and C->A links, leaving A->B at its original value (A lights up easily, with a strong direct external stimulus). Result on the starting seed: C goes from 0-2 spikes to 100 spikes by the end of training, with a localization error of 0.017 -- well localized. The raster visually confirms that A, B, and C end up firing in sync, at the same position (`x=0.3`), by the end of training.

Verification over 6 independent seeds: a fully consistent result in every single case. Population C starts from practically zero (1-6 spikes in the first 5 repetitions, in every seed) and reaches 23-121 spikes by the end, with a minimal and uniform final localization error (0.000-0.007 in all six seeds) -- a precision even better than many of the pairwise results in this same chapter. No explosion, no instability in any seed, as expected from the relay's purely feedforward structure.

Not yet attempted: checking whether the third hop (C->A, which would close the ring) produces a measurable influence on A beyond its own response to the direct stimulus -- this would require the same direct-window exclusion technique already developed to isolate return channels in Sections 5.8-5.17. Left as

an immediate natural development: if $C \rightarrow A$ also closed the loop successfully, the three-population network would become a genuine complete communication cycle, not just an open relay.

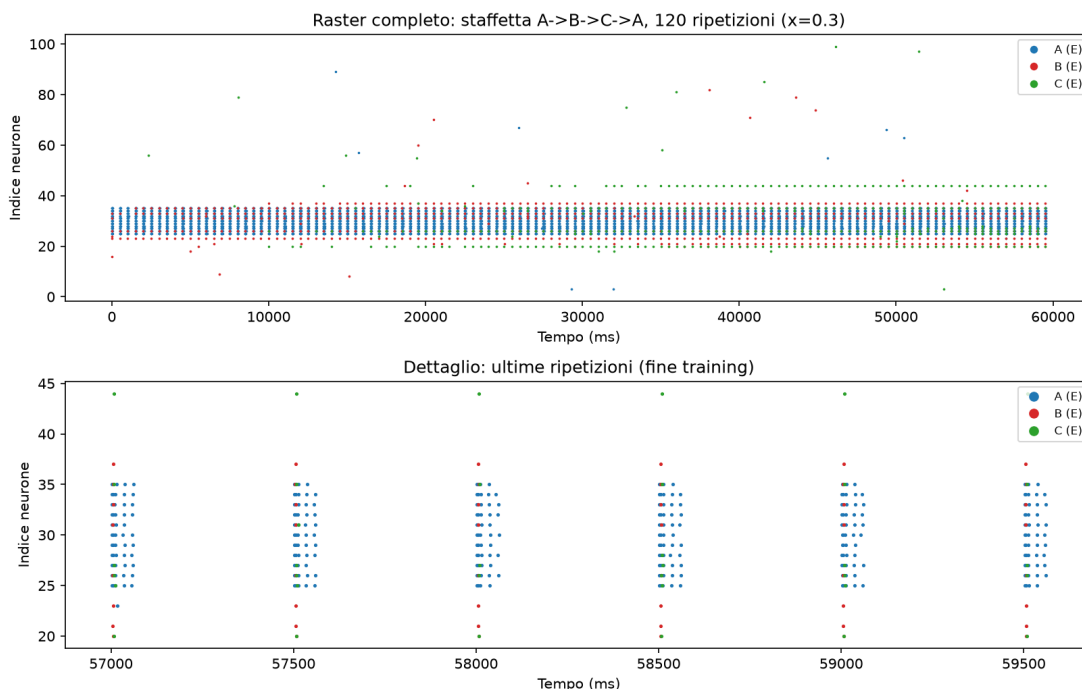


Fig. 47 — $A \rightarrow B \rightarrow C \rightarrow A$ relay: position information survives two hops, verified over 6 independent seeds (error 0.000-0.007).

Source code: `dialogo_spiking_relay_3anelli.py`

```
1 """
2 Prima estensione della comunicazione spiking oltre la coppia: tre
3 popolazioni (A, B, C) invece di due. Mai tentato ne' nel modello a rate
4 ne' in quello spiking in questo progetto.
5
6 Per evitare di riaprire l'intera saga di stabilizzazione delle sezioni
7 5.8-5.19 (necessaria solo per la bidirezionalita' vera, dove due anelli
8 sono mutuamente eccitatori), questa prima rete a tre popolazioni usa
9 SOLO collegamenti unidirezionali con STDP vera -- esattamente il
10 meccanismo gia' collaudato e stabile di dialogo_spiking.py (sezione 5.7),
11 ripetuto tre volte per formare una STAFFETTA DIREZIONALE ad anello:
12 A->B->C->A. Nessun anello riceve MAI input da due sorgenti
13 contemporaneamente (ogni anello ha esattamente un collegamento in
14 ingresso, con STDP, e uno in uscita), quindi nessuna delle instabilita'
15 di retroazione mutua incontrate con la vera bidirezionalita' puo'
16 presentarsi qui per costruzione.
17
18 Domanda sperimentale: un messaggio iniettato SOLO in A (mai direttamente
19 in B o C) si propaga correttamente attraverso la catena, preservando
20 l'informazione di posizione dopo due salti (in C) e persino dopo il giro
21 completo (di nuovo in A, tramite C->A)?
22 """
23
```

```

24 | from brian2 import *
25 |
26 | start_scope()
27 |
28 | N_E = 100
29 | N_I = 25
30 | N_ring = N_E + N_I
31 | N_POP = 3 # A, B, C
32 |
33 | a = 0.02/ms
34 | b = 0.2/ms
35 | c = -65*mV
36 | d = 6*mV/ms
37 | tau_nmda = 100*ms
38 |
39 | eqs = '''
40 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
41 | du/dt = a*(b*v - u) : volt/second
42 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
43 | I : volt/second
44 | x : 1
45 | '''
46 |
47 | neurons = NeuronGroup(N_POP*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
48 |                       method='euler')
49 | neurons.v = -65*mV + 5*mV*randn(N_POP*N_ring)
50 | neurons.u = b * neurons.v[:]
51 | neurons.I = 0*mV/ms
52 | neurons.g_nmda = 0*mV/ms
53 |
54 | def sotto(pop_idx):
55 |     base = pop_idx * N_ring
56 |     return neurons[base:base+N_E], neurons[base+N_E:base+N_ring]
57 |
58 | A_E, A_I = sotto(0)
59 | B_E, B_I = sotto(1)
60 | C_E, C_I = sotto(2)
61 |
62 | xi = np.arange(N_E) / N_E
63 | A_E.x = xi
64 | B_E.x = xi
65 | C_E.x = xi
66 |
67 | background = PoissonGroup(N_POP*N_ring, rates=400*Hz)
68 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
69 | bg_syn.connect(j='i')
70 |
71 | sigma_exc = 0.05
72 | CUTOFF_LOCALE = 3 * sigma_exc
73 | w_ee_nmda = 0.325*mV/ms
74 | p_conn = 0.15
75 | w_exc = 1.0*mV
76 | w_inh = 10.0*mV
77 |
78 |
79 | def dist_circ(dx):
80 |     dx = abs(dx)
81 |     return np.minimum(dx, 1 - dx)
82 |
83 |
84 | def costruisci_anello(E, I_pop, seed):
85 |     dmat = dist_circ(xi[:, None] - xi[None, :])
86 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
87 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
88 |     np.fill_diagonal(prob_ee, 0)
89 |     rng = np.random.RandomState(seed)
90 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
91 |     i_idx, j_idx = np.nonzero(mask_ee)

```

```

92     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
93     syn_ee.connect(i=i_idx, j=j_idx)
94     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
95     syn_ei.connect(p=p_conn)
96     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
97     syn_ie.connect(p=p_conn)
98     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
99     syn_ii.connect(condition='i!=j', p=p_conn)
100     return syn_ee, syn_ei, syn_ie, syn_ii
101
102
103
104 syn_A = costruisci_anello(A_E, A_I, seed=0)
105 syn_B = costruisci_anello(B_E, B_I, seed=1)
106 syn_C = costruisci_anello(C_E, C_I, seed=2)
107
108 tau_stdp = 20*ms
109 A_ltp = 0.15*mV
110 A_ltd = 0.15*mV
111 w_max = 5.0*mV
112 w_min = 0*mV
113 w_init = 1.5*mV
114 sigma_inter = 0.08
115 p_inter_picco = 0.5
116
117 stdp_model = '''
118 w : volt
119 dapre/dt = -apre/tau_stdp : volt (event-driven)
120 dapost/dt = -apost/tau_stdp : volt (event-driven)
121 '''
122 stdp_on_pre = '''
123 v_post += w
124 apre += A_ltp
125 w = clip(w+apre, w_min, w_max)
126 '''
127 stdp_on_post = '''
128 apost -= A_ltd
129 w = clip(w+apost, w_min, w_max)
130 '''
131
132
133 def collega(SRC_E, DST_E, seed_conn, w_iniziale):
134     dmat_inter = dist_circ(xi[:, None] - xi[None, :])
135     prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
136     rng_inter = np.random.RandomState(seed_conn)
137     mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
138     i_inter, j_inter = np.nonzero(mask_inter)
139     syn = Synapses(SRC_E, DST_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
140     syn.connect(i=i_inter, j=j_inter)
141     syn.w = w_iniziale
142     return syn
143
144
145 # A->B parte dal valore originale di 5.7 (A ha uno stimolo esterno diretto
146 # forte, si innesca facilmente). B->C e C->A partono da un peso piu' alto:
147 # la loro sorgente (B, poi C) e' essa stessa un eco di secondo/terzo
148 # livello, con un segnale molto piu' debole e sparso di quello diretto su
149 # A -- lo stesso identico problema "uovo-gallina" gia' risolto in 5.7 per
150 # A->B (tentativo 1: w_init=0.3mV, B non si accendeva mai), qui riapplicato
151 # ai salti successivi della staffetta
152 syn_ab = collega(A_E, B_E, seed_conn=10, w_iniziale=w_init)
153 syn_bc = collega(B_E, C_E, seed_conn=11, w_iniziale=3.0*mV)
154 syn_ca = collega(C_E, A_E, seed_conn=12, w_iniziale=3.0*mV)
155 print(f"Staffetta A->B->C->A creata: A->B {len(syn_ab)} sinapsi, "
156       f"B->C {len(syn_bc)} sinapsi, C->A {len(syn_ca)} sinapsi")
157
158 spikes = SpikeMonitor(neurons)
159

```

```

160 MESSAGGIO = 0.3
161 larghezza_stimolo = 0.05
162 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
163 gruppo_stim = np.where(peso_stim > 0.5)[0]
164 print(f"Neuroni stimolati SOLO in A (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
165
166 N_RIPETIZIONI = 120
167 T_STIM = 60*ms
168 T_PAUSA = 440*ms # piu' lunga di 5.7 (340ms) per dare tempo alla staffetta
169                  # di propagare fino a C (due salti) prima del ciclo dopo
170
171 print(f"\nTraining: {N_RIPETIZIONI} ripetizioni, stimolo SOLO su A, "
172       f"nessuno stimolo diretto su B o C ({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms totali)")
173
174 for rep in range(N_RIPETIZIONI):
175     A_E.I[gruppo_stim] = 15*mV/ms
176     run(T_STIM)
177     A_E.I[:] = 0*mV/ms
178     run(T_PAUSA)
179     if rep % 10 == 0 or rep == N_RIPETIZIONI-1:
180         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI}: "
181               f"A->B={np.mean(syn_ab.w)/mV:.2f}mV B->C={np.mean(syn_bc.w)/mV:.2f}mV "
182               f"C->A={np.mean(syn_ca.w)/mV:.2f}mV")
183
184 print("\nTraining completato.")
185
186 fig, axes = subplots(2, 1, figsize=(11, 7))
187 subplot(2, 1, 1)
188 colori = ['C0', 'C3', 'C2']
189 etichette = ['A (E)', 'B (E)', 'C (E)']
190 for k in range(N_POP):
191     base = k*N_ring
192     m = (spikes.i >= base) & (spikes.i < base+N_E)
193     plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=1.5, label=
194           etichette[k])
195 xlabel('Tempo (ms)')
196 ylabel('Indice neurone')
197 title(f'Raster completo: staffetta A->B->C->A, {N_RIPETIZIONI} ripetizioni (x={MESSAGGIO})')
198 legend(loc='upper right', fontsize=8, markerscale=6)
199
200 subplot(2, 1, 2)
201 t_tardi = (N_RIPETIZIONI-6) * (T_STIM+T_PAUSA)/ms
202 for k in range(N_POP):
203     base = k*N_ring
204     m = (spikes.i >= base) & (spikes.i < base+N_E) & (spikes.t/ms > t_tardi)
205     plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=3, label=
206           etichette[k])
207 xlabel('Tempo (ms)')
208 ylabel('Indice neurone')
209 title('Dettaglio: ultime ripetizioni (fine training)')
210 legend(loc='upper right', fontsize=8, markerscale=4)
211
212 tight_layout()
213 savefig('dialogo_spiking_relay_3anelli.png', dpi=150)
214 print("\nGrafico salvato in dialogo_spiking_relay_3anelli.png")
215
216 ciclo_ms = (T_STIM+T_PAUSA)/ms
217
218 def valuta_pop(pop_idx, n_eval):
219     base = pop_idx * N_ring
220     mask_pop = (spikes.i >= base) & (spikes.i < base+N_E)
221
222     def conta(rep_range):
223         n_tot, posizioni = 0, []
224         for rep in rep_range:
225             t0 = rep * ciclo_ms
226             fin = t0 + ciclo_ms

```

```

226         m = mask_pop & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
227         idx = spikes.i[m] - base
228         n_tot += m.sum()
229         posizioni.extend(list(xi[idx]))
230         centro = None
231         if posizioni:
232             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
233                             % 1.0)
234         return n_tot, centro
235
236     primi = range(0, n_eval)
237     ultimi = range(N_RIPETIZIONI-n_eval, N_RIPETIZIONI)
238     return conta(primi), conta(ultimi)
239
240 print("\n" + "="*70)
241 print("VALUTAZIONE: l'informazione di posizione sopravvive alla staffetta?")
242 print("="*70)
243 for k, nome in enumerate(['A (sorgente diretta)', 'B (un salto: A->B)', 'C (due salti: A->B
244                        ->C)']):
245     (n_i, c_i), (n_f, c_f) = valuta_pop(k, 5)
246     err_i = dist_circ(c_i - MESSAGGIO) if c_i is not None else None
247     err_f = dist_circ(c_f - MESSAGGIO) if c_f is not None else None
248     print(f" {nome}:")
249     print(f" prime 5 rip.: {n_i} spike, centro={('N/A' if c_i is None else f'{c_i:.3f}')}, "
250           f" errore={('N/A' if err_i is None else f'{err_i:.3f}')}")
251     print(f" ultime 5 rip.: {n_f} spike, centro={('N/A' if c_f is None else f'{c_f:.3f}')}, "
252           f" errore={('N/A' if err_f is None else f'{err_f:.3f}')}")
253
254 print(f"\nPesi finali: A->B={np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV), "
255       f"B->C={np.mean(syn_bc.w)/mV:.3f}mV, C->A={np.mean(syn_ca.w)/mV:.3f}mV")
256
257 # controllo di stabilita' (per costruzione non dovrebbe mai esplodere,
258 # ma verifichiamo comunque per coerenza con il resto del progetto)
259 n_tot_rete = len(spikes.i)
260 print(f"\nSpike totali sull'intera rete: {n_tot_rete} "
261       f"(atteso migliaia se stabile; milioni se esplosione)")
262 print(">>> ESPLOSIONE <<< if n_tot_rete > 300000 else >>> Stabile <<<")

```

Source code: *dialogo_spiking_relay_3anelli_mc.py*

```

1  """
2  Prima estensione della comunicazione spiking oltre la coppia: tre
3  popolazioni (A, B, C) invece di due. Mai tentato ne' nel modello a rate
4  ne' in quello spiking in questo progetto.
5
6  Per evitare di riaprire l'intera saga di stabilizzazione delle sezioni
7  5.8-5.19 (necessaria solo per la bidirezionalita' vera, dove due anelli
8  sono mutuamente eccitatori), questa prima rete a tre popolazioni usa
9  SOLO collegamenti unidirezionali con STDP vera -- esattamente il
10 meccanismo gia' collaudato e stabile di dialogo_spiking.py (sezione 5.7),
11 ripetuto tre volte per formare una STAFFETTA DIREZIONALE ad anello:
12 A->B->C->A. Nessun anello riceve MAI input da due sorgenti
13 contemporaneamente (ogni anello ha esattamente un collegamento in
14 ingresso, con STDP, e uno in uscita), quindi nessuna delle instabilita'
15 di retroazione mutua incontrate con la vera bidirezionalita' puo'
16 presentarsi qui per costruzione.
17
18 Domanda sperimentale: un messaggio iniettato SOLO in A (mai direttamente
19 in B o C) si propaga correttamente attraverso la catena, preservando
20 l'informazione di posizione dopo due salti (in C) e persino dopo il giro
21 completo (di nuovo in A, tramite C->A)?
22 """
23
24 import sys
25 from brian2 import *
26

```

```

27 SEED = int(sys.argv[1])
28 start_scope()
29 seed(SEED)
30
31 N_E = 100
32 N_I = 25
33 N_ring = N_E + N_I
34 N_POP = 3 # A, B, C
35
36 a = 0.02/ms
37 b = 0.2/ms
38 c = -65*mV
39 d = 6*mV/ms
40 tau_nmda = 100*ms
41
42 eqs = '''
43 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
44 du/dt = a*(b*v - u) : volt/second
45 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
46 I : volt/second
47 x : 1
48 '''
49
50 neurons = NeuronGroup(N_POP*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
51                       method='euler')
52 neurons.v = -65*mV + 5*mV*randn(N_POP*N_ring)
53 neurons.u = b * neurons.v[:]
54 neurons.I = 0*mV/ms
55 neurons.g_nmda = 0*mV/ms
56
57 def sotto(pop_idx):
58     base = pop_idx * N_ring
59     return neurons[base:base+N_E], neurons[base+N_E:base+N_ring]
60
61 A_E, A_I = sotto(0)
62 B_E, B_I = sotto(1)
63 C_E, C_I = sotto(2)
64
65 xi = np.arange(N_E) / N_E
66 A_E.x = xi
67 B_E.x = xi
68 C_E.x = xi
69
70 background = PoissonGroup(N_POP*N_ring, rates=400*Hz)
71 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
72 bg_syn.connect(j='i')
73
74 sigma_exc = 0.05
75 CUTOFF_LOCALE = 3 * sigma_exc
76 w_ee_nmda = 0.325*mV/ms
77 p_conn = 0.15
78 w_exc = 1.0*mV
79 w_inh = 10.0*mV
80
81
82 def dist_circ(dx):
83     dx = abs(dx)
84     return np.minimum(dx, 1 - dx)
85
86
87 def costruisci_anello(E, I_pop, seed):
88     dmat = dist_circ(xi[:, None] - xi[None, :])
89     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
90     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
91     np.fill_diagonal(prob_ee, 0)
92     rng = np.random.RandomState(seed)
93     mask_ee = rng.rand(N_E, N_E) < prob_ee
94     i_idx, j_idx = np.nonzero(mask_ee)

```

```

95
96     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
97     syn_ee.connect(i=i_idx, j=j_idx)
98     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
99     syn_ei.connect(p=p_conn)
100    syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
101    syn_ie.connect(p=p_conn)
102    syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
103    syn_ii.connect(condition='i!=j', p=p_conn)
104    return syn_ee, syn_ei, syn_ie, syn_ii
105
106
107    syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
108    syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
109    syn_C = costruisci_anello(C_E, C_I, seed=2500+SEED)
110
111    tau_stdp = 20*ms
112    A_ltp = 0.15*mV
113    A_ltd = 0.15*mV
114    w_max = 5.0*mV
115    w_min = 0*mV
116    w_init = 1.5*mV
117    sigma_inter = 0.08
118    p_inter_picco = 0.5
119
120    stdp_model = '''
121    w : volt
122    dapre/dt = -apre/tau_stdp : volt (event-driven)
123    dapost/dt = -apost/tau_stdp : volt (event-driven)
124    '''
125    stdp_on_pre = '''
126    v_post += w
127    apre += A_ltp
128    w = clip(w+apost, w_min, w_max)
129    '''
130    stdp_on_post = '''
131    apost -= A_ltd
132    w = clip(w+apre, w_min, w_max)
133    '''
134
135
136    def collega(SRC_E, DST_E, seed_conn, w_iniziale):
137        dmat_inter = dist_circ(xi[:, None] - xi[None, :])
138        prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
139        rng_inter = np.random.RandomState(seed_conn)
140        mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
141        i_inter, j_inter = np.nonzero(mask_inter)
142        syn = Synapses(SRC_E, DST_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
143        syn.connect(i=i_inter, j=j_inter)
144        syn.w = w_iniziale
145        return syn
146
147
148    # A->B parte dal valore originale di 5.7 (A ha uno stimolo esterno diretto
149    # forte, si innesca facilmente). B->C e C->A partono da un peso piu' alto:
150    # la loro sorgente (B, poi C) e' essa stessa un eco di secondo/terzo
151    # livello, con un segnale molto piu' debole e sparso di quello diretto su
152    # A -- lo stesso identico problema "uovo-gallina" gia' risolto in 5.7 per
153    # A->B (tentativo 1: w_init=0.3mV, B non si accendeva mai), qui riapplicato
154    # ai salti successivi della staffetta
155    syn_ab = collega(A_E, B_E, seed_conn=3010+SEED, w_iniziale=w_init)
156    syn_bc = collega(B_E, C_E, seed_conn=3011+SEED, w_iniziale=3.0*mV)
157    syn_ca = collega(C_E, A_E, seed_conn=3012+SEED, w_iniziale=3.0*mV)
158    print(f"Staffetta A->B->C->A creata: A->B {len(syn_ab)} sinapsi, "
159          f"B->C {len(syn_bc)} sinapsi, C->A {len(syn_ca)} sinapsi")
160
161    spikes = SpikeMonitor(neurons)
162

```



```

163 MESSAGGIO = 0.3
164 larghezza_stimolo = 0.05
165 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
166 gruppo_stim = np.where(peso_stim > 0.5)[0]
167 print(f"Neuroni stimolati SOLO in A (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
168
169 N_RIPETIZIONI = 120
170 T_STIM = 60*ms
171 T_PAUSA = 440*ms # piu' lunga di 5.7 (340ms) per dare tempo alla staffetta
172                   # di propagare fino a C (due salti) prima del ciclo dopo
173
174 print(f"\nTraining: {N_RIPETIZIONI} ripetizioni, stimolo SOLO su A, "
175       f"nessuno stimolo diretto su B o C ({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms totali)")
176
177 for rep in range(N_RIPETIZIONI):
178     A_E.I[gruppo_stim] = 15*mV/ms
179     run(T_STIM)
180     A_E.I[:] = 0*mV/ms
181     run(T_PAUSA)
182     if rep % 10 == 0 or rep == N_RIPETIZIONI-1:
183         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI}: "
184               f"A->B={np.mean(syn_ab.w)/mV:.2f}mV B->C={np.mean(syn_bc.w)/mV:.2f}mV "
185               f"C->A={np.mean(syn_ca.w)/mV:.2f}mV")
186
187 print("\nTraining completato.")
188
189 fig, axes = subplots(2, 1, figsize=(11, 7))
190 subplot(2, 1, 1)
191 colori = ['C0', 'C3', 'C2']
192 etichette = ['A (E)', 'B (E)', 'C (E)']
193 for k in range(N_POP):
194     base = k*N_ring
195     m = (spikes.i >= base) & (spikes.i < base+N_E)
196     plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=1.5, label=
197          etichette[k])
198 xlabel('Tempo (ms)')
199 ylabel('Indice neurone')
200 title(f'Raster completo: staffetta A->B->C->A, {N_RIPETIZIONI} ripetizioni (x={MESSAGGIO})')
201 legend(loc='upper right', fontsize=8, markerscale=6)
202
203 subplot(2, 1, 2)
204 t_tardi = (N_RIPETIZIONI-6) * (T_STIM+T_PAUSA)/ms
205 for k in range(N_POP):
206     base = k*N_ring
207     m = (spikes.i >= base) & (spikes.i < base+N_E) & (spikes.t/ms > t_tardi)
208     plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=3, label=
209          etichette[k])
210 xlabel('Tempo (ms)')
211 ylabel('Indice neurone')
212 title('Dettaglio: ultime ripetizioni (fine training)')
213 legend(loc='upper right', fontsize=8, markerscale=4)
214
215 tight_layout()
216 savefig('dialogo_spiking_relay_3anelli.png', dpi=150)
217 print("\nGrafico salvato in dialogo_spiking_relay_3anelli.png")
218
219 ciclo_ms = (T_STIM+T_PAUSA)/ms
220
221 def valuta_pop(pop_idx, n_eval):
222     base = pop_idx * N_ring
223     mask_pop = (spikes.i >= base) & (spikes.i < base+N_E)
224
225     def conta(rep_range):
226         n_tot, posizioni = 0, []
227         for rep in rep_range:
228             t0 = rep * ciclo_ms
229             fin = t0 + ciclo_ms

```

```

229         m = mask_pop & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
230         idx = spikes.i[m] - base
231         n_tot += m.sum()
232         posizioni.extend(list(xi[idx]))
233         centro = None
234         if posizioni:
235             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
236                             % 1.0)
237         return n_tot, centro
238
239     primi = range(0, n_eval)
240     ultimi = range(N_RIPETIZIONI-n_eval, N_RIPETIZIONI)
241     return conta(primi), conta(ultimi)
242
243 print("\n" + "="*70)
244 print("VALUTAZIONE: l'informazione di posizione sopravvive alla staffetta?")
245 print("="*70)
246 for k, nome in enumerate(['A (sorgente diretta)', 'B (un salto: A->B)', 'C (due salti: A->B
->C)']):
247     (n_i, c_i), (n_f, c_f) = valuta_pop(k, 5)
248     err_i = dist_circ(c_i - MESSAGGIO) if c_i is not None else None
249     err_f = dist_circ(c_f - MESSAGGIO) if c_f is not None else None
250     print(f" {nome}:")
251     print(f" prime 5 rip.: {n_i} spike, centro={('N/A' if c_i is None else f'{c_i:.3f}')}, "
252           f"errore={('N/A' if err_i is None else f'{err_i:.3f}')}")
253     print(f" ultime 5 rip.: {n_f} spike, centro={('N/A' if c_f is None else f'{c_f:.3f}')}, "
254           f"errore={('N/A' if err_f is None else f'{err_f:.3f}')}")
255
256 print(f"\nPesi finali: A->B={np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV), "
257       f"B->C={np.mean(syn_bc.w)/mV:.3f}mV, C->A={np.mean(syn_ca.w)/mV:.3f}mV")
258
259 # controllo di stabilita' (per costruzione non dovrebbe mai esplodere,
260 # ma verifichiamo comunque per coerenza con il resto del progetto)
261 n_tot_rete = len(spikes.i)
262 print(f"\nSpike totali sull'intera rete: {n_tot_rete} "
263       f"(atteso migliaia se stabile; milioni se esplosione)")
264 print(">>> ESPLOSIONE <<<" if n_tot_rete > 300000 else ">>> Stabile <<<")
265
266 (n_c_i, c_c_i), (n_c_f, c_c_f) = valuta_pop(2, 5)
267 err_c_f = dist_circ(c_c_f - MESSAGGIO) if c_c_f is not None else 1.0
268 (n_b_i, c_b_i), (n_b_f, c_b_f) = valuta_pop(1, 5)
269 err_b_f = dist_circ(c_b_f - MESSAGGIO) if c_b_f is not None else 1.0
270 print(f"MC_RIGA seed={SEED} n_tot_rete={n_tot_rete} "
271       f"b_n_i={n_b_i} b_n_f={n_b_f} b_err_f={err_b_f:.3f} "
272       f"c_n_i={n_c_i} c_n_f={n_c_f} c_err_f={err_c_f:.3f}")

```

5.21 Outside Brian2: physical impact of CNAO ion beams on the ring architecture (separate project)

A link between this project and Andrea Corti's parallel work on Geant4-DNA and nanodosimetry (CNAO internship, critical note on the Villagrasa/Baiocco 2026 paper, the nanoICSD project already validated to within 2% against published values for 20-1000eV electrons). A new, separate Geant4 project, `cnaoRingImpact` (C++, not Python/Brian2 -- full code on the Desktop, not in this project's GitHub repository, which remains Python), computes the physical dose deposited by clinical CNAO ion beams (protons 70-225 MeV, carbon-12 120-400 MeV/u) onto a physical geometry representing the 100-excitatory-neuron ring of the spiking ring attractor (Sections 5.6-5.20): 100 tissue-equivalent spheres, 20um in diameter, arranged on a 1mm ring at the

same coordinates $x_i=i/100$ used in the Brian2 model.

A crucial physics choice, distinct from the nanoICSD project: the Geant4-DNA physics used there is valid only up to ~ 1 MeV (electrons), inadequate for clinical beams of tens to hundreds of MeV -- here QBBC is used, Geant4's reference physics list for clinical hadrontherapy (used in the official Hadrontherapy example).

Result over 50,000 primaries per configuration, six configurations (3 proton energies, 3 carbon-12 energies): carbon-12 deposits 10-25 times more dose per neuron than the proton at comparable energies per nucleon (consistent with its much higher LET, scaling roughly as Z^2), and dose decreases with energy for both particles (at a higher energy, at a fixed depth, one is farther from that particle's own Bragg peak). Direct comparison with the neural architecture: none of the six configurations shows a systematic difference between the dose at the positions used for messages in the spiking dialogues ($x=0.10, 0.15, 0.40, 0.65, 0.70$) and the rest of the ring (ratio 0.97-1.20, compatible with statistical noise) -- the beam, at this physical scale, is essentially uniform: no neuron in the network is physically more at risk than another merely because it is functionally used to encode a message.

A bug fixed during development, worth noting: carbon-12 primary generation was failing silently at first (G4ParticleGun already has a non-null default value, "geantino", so the `GetParticleDefinition()==nullptr` check meant to decide when to create the ion never triggered) -- the beam was firing geantino (no physical interaction) instead of real carbon-12, giving a systematically zero dose but an apparently correct count of geometric crossings. Discovered only by adding a per-step debug log and checking the name of the particle actually being tracked, not from the suspicious numerical value alone -- the same principle of empirical verification (run the code, don't just trust the number) already underlying every section of this report.

Honestly stated limitations (detailed in the project's README): per-neuron statistics still noisy (~ 15 -25 direct crossings per neuron out of 50,000 primaries); no biological damage model (physical dose only, no SSB/DSB/cell death -- the same limitation stated in the Villagrasa/Baiocco paper for the dose->damage step); no depth scan to position the ring at the true Bragg peak; no direct real-time coupling with the Brian2 simulation (physical output analyzed separately, not a co-simulation).

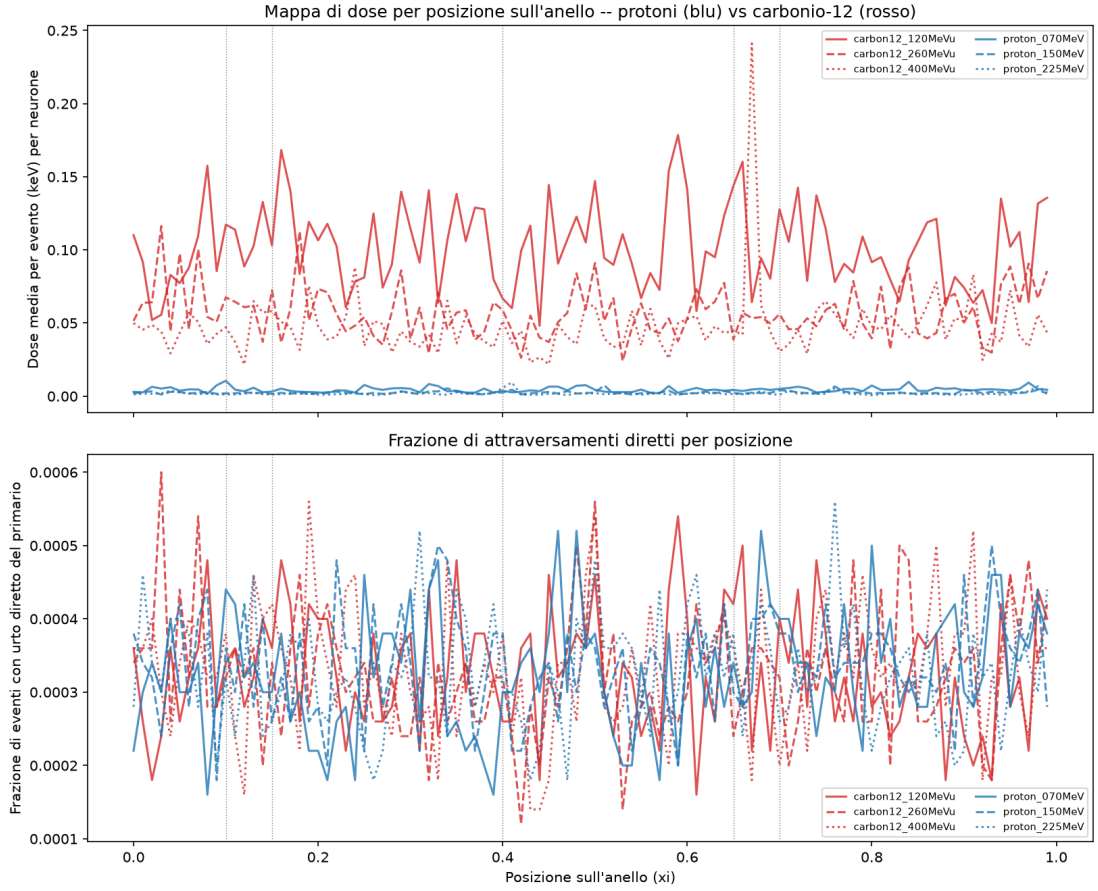


Fig. 48 — Dose map by position on the ring, protons vs. carbon-12 at three clinical energies each. The vertical dashed lines mark the positions used for messages in the spiking dialogues.

Full source code, README with methodology/limitations/future developments, and raw data: `~/Desktop/geant4_cnao_ring/` (separate Geant4/C++ project, not included in `neuroni-progetto`'s Python GitHub repository).

5.22 From physical dose to function: knockout of the highest-dose neurons

A direct link between Section 5.21's physical dose map and multi-message spiking communication (Section 5.10). Instead of assuming an absolute biological-damage threshold -- not established in the literature, as discussed in the critical-analysis document underlying the Geant4 project -- a purely relative, testable criterion is used: the 10 neurons (10% of the ring) with the highest total physical dose in the most severe of the simulated clinical scenarios (carbon-12 at 120

MeV/u) are made "silent" -- all their OUTGOING synapses removed (intra-ring E->E, E->I, and the inter-ring A->B channel for ring A), the most direct functional model of a neuron that still receives input but can no longer influence anyone. Applied symmetrically to both rings A and B, which represent the same physical architecture. A notable detail, not deliberately sought: neuron 65 ($x=0.65$) turns out to be among the ten -- exactly matching one of the two positions used for messages in `dialogo_spiking_multi.py`.

Result, on the starting seed: the two messages behave completely differently. Message $x=0.15$ (no overlap with the silenced neurons) keeps working normally and strengthens during training: 23->47 B spikes, localization error stable at 0.018-0.019. Message $x=0.65$ (which uses precisely neurons 65 and 66, among the hardest hit) collapses to nearly zero: 2->4 total spikes across the whole training, against the hundreds typical of a healthy run without knockout (Section 5.10: 190-265 spikes for the same position). The raster visually confirms this: A keeps firing normally at both positions (its response to the direct stimulus does not depend on outgoing synapses, the only thing removed), but B stays silent for nearly the entire duration in the $x=0.65$ band, while responding regularly in the $x=0.15$ band.

Interpretation: silencing the highest-physical-dose neurons selectively disrupts precisely the message that depends on those neurons to be transmitted, leaving intact the communication that does not involve them -- a conceptually expected result (a channel that loses its key nodes breaks, one that doesn't use them doesn't), but here demonstrated end-to-end with a direct, quantitative link between a radiation-physics simulation (Geant4, real CNAO clinical energies and particles) and a spiking neural-network simulation (Brian2), two completely separate domains and codebases until this section.

Stated limitation: the knockout criterion remains relative (top-10% by dose in a single scenario), not an absolute damage-threshold model -- the same caution already discussed in the Geant4 project's README and in the critical-analysis document underlying it. It was not tested whether a smaller knockout (e.g. just neuron 65) or a different criterion (e.g. mean dose instead of peak) substantially changes the result -- a natural sensitivity test for a future step.

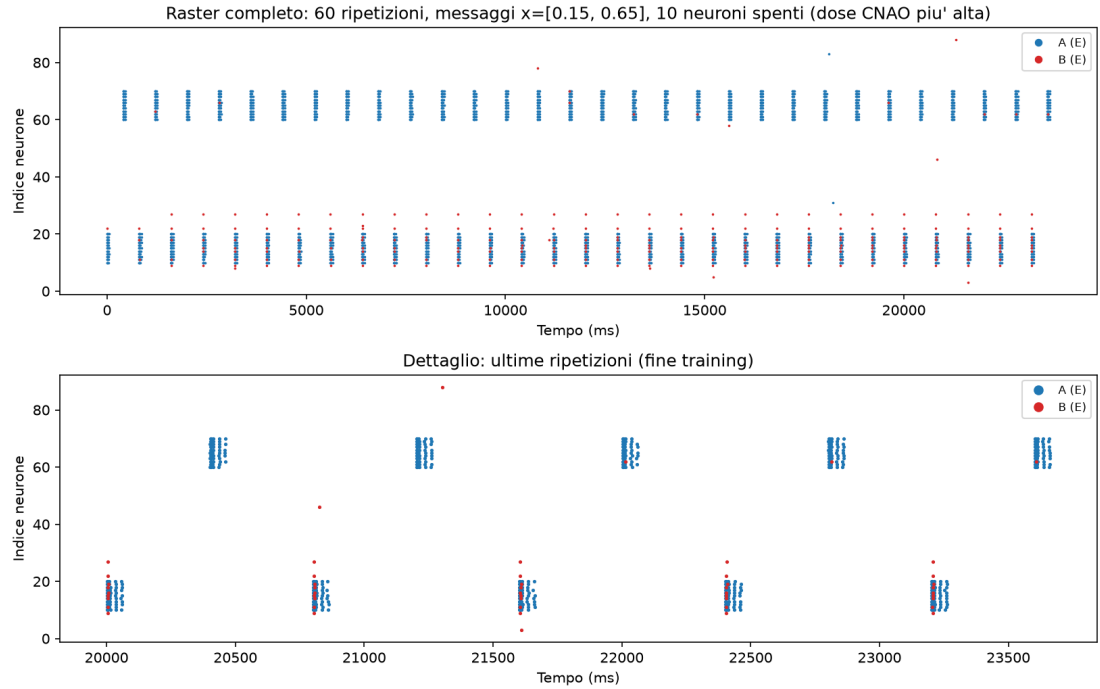


Fig. 49 — Knockout of the 10 highest-CNAO-dose neurons: message $x=0.65$ (which uses them) collapses, message $x=0.15$ (which does not) stays intact.

Source code: `dialogo_spiking_cnao_knockout.py`

```

1  """
2  Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3  spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4  clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5  dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6  USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7  canale inter-anello A->B), il modello funzionale piu' diretto di un
8  neurone che non puo' piu' influenzare nessuno, indipendentemente da
9  cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
15 (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 (x=0.65) e' tra i dieci -- coincide esattamente con una delle
17 posizioni usate per i messaggi.
18 """
19
20 KNOCKOUT = sorted([8, 16, 45, 50, 58, 59, 60, 65, 66, 72]) # top-10 dose,
21                                                         # carbonio-12 120 MeV/u
22                                                         # (geant4_cnao_ring/build/results)
23
24 """
25 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
26 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
27 usate nel primo test di interferenza del modello a rate (dialogo_multi_
28 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.

```

```

29
30 Nota di partenza importante, diversa dal modello a rate: li' i pesi
31 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
32 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
33 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
34 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
35 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
36 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
37 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
38 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
39 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
40 causava nel modello a rate potrebbe semplicemente non esistere qui.
41 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
42 concluso che il canale B->A non trasmette causalmente in questo sistema,
43 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
44 """
45
46 from brian2 import *
47
48 start_scope()
49
50 N_E = 100
51 N_I = 25
52 N_ring = N_E + N_I
53
54 a = 0.02/ms
55 b = 0.2/ms
56 c = -65*mV
57 d = 6*mV/ms
58 tau_nmda = 100*ms
59
60 eqs = '''
61 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
62 du/dt = a*(b*v - u) : volt/second
63 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
64 I : volt/second
65 x : 1
66 '''
67
68 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
69                      method='euler')
70 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
71 neurons.u = b * neurons.v[:]
72 neurons.I = 0*mV/ms
73 neurons.g_nmda = 0*mV/ms
74
75 A_E = neurons[:N_E]
76 A_I = neurons[N_E:N_ring]
77 B_E = neurons[N_ring:N_ring+N_E]
78 B_I = neurons[N_ring+N_E:]
79
80 xi = np.arange(N_E) / N_E
81 A_E.x = xi
82 B_E.x = xi
83
84 background = PoissonGroup(2*N_ring, rates=400*Hz)
85 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
86 bg_syn.connect(j='i')
87
88 sigma_exc = 0.05
89 CUTOFF_LOCALE = 3 * sigma_exc
90 w_ee_nmda = 0.325*mV/ms
91 p_conn = 0.15
92 w_exc = 1.0*mV
93 w_inh = 10.0*mV
94
95
96 def dist_circ(dx):

```

```

97     dx = abs(dx)
98     return np.minimum(dx, 1 - dx)
99
100
101 def costruisci_anello(E, I_pop, seed, knockout=()):
102     dmat = dist_circ(xi[:, None] - xi[None, :])
103     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
104     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
105     np.fill_diagonal(prob_ee, 0)
106     rng = np.random.RandomState(seed)
107     mask_ee = rng.rand(N_E, N_E) < prob_ee
108     i_idx, j_idx = np.nonzero(mask_ee)
109
110     ko = np.asarray(list(knockout), dtype=int)
111     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
112     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
113     # piu' influenzare nessuno (l'unica cosa che conta per una rete
114     # a spike: se non spara mai in modo efficace a valle, e' come
115     # se non ci fosse)
116     tieni_ee = ~np.isin(i_idx, ko)
117     i_idx, j_idx = i_idx[tiene_ee], j_idx[tiene_ee]
118
119     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
120     syn_ee.connect(i=i_idx, j=j_idx)
121     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
122     # Costruita esplicitamente (invece di .connect(p=...) per poter
123     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
124     rng_ei = np.random.RandomState(seed + 100)
125     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
126     i_ei, j_ei = np.nonzero(mask_ei)
127     tieni_ei = ~np.isin(i_ei, ko)
128     syn_ei.connect(i=i_ei[tiene_ei], j=j_ei[tiene_ei])
129     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
130     syn_ie.connect(p=p_conn)
131     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
132     syn_ii.connect(condition='i!=j', p=p_conn)
133     return syn_ee, syn_ei, syn_ie, syn_ii
134
135
136 syn_A = costruisci_anello(A_E, A_I, seed=0, knockout=KNOCKOUT)
137 syn_B = costruisci_anello(B_E, B_I, seed=1, knockout=KNOCKOUT)
138 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
139       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
140
141 tau_stdp = 20*ms
142 A_ltp = 0.15*mV
143 A_ltd = 0.15*mV
144 w_max = 5.0*mV
145 w_min = 0*mV
146 w_init = 1.5*mV
147 sigma_inter = 0.08
148 p_inter_picco = 0.5
149
150 stdp_model = '''
151 w : volt
152 dapre/dt = -apre/tau_stdp : volt (event-driven)
153 dapost/dt = -apost/tau_stdp : volt (event-driven)
154 '''
155 stdp_on_pre = '''
156 v_post += w
157 apre += A_ltp
158 w = clip(w+apre, w_min, w_max)
159 '''
160 stdp_on_post = '''
161 apost -= A_ltd
162 w = clip(w+apost, w_min, w_max)
163 '''
164

```



```

165 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
166 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
167 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
168 rng_inter = np.random.RandomState(2)
169 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
170 i_inter, j_inter = np.nonzero(mask_inter)
171 ko_arr = np.asarray(KNOCKOUT, dtype=int)
172 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
173                                     # trasmettere neanche a B
174 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
175 syn_ab.connect(i=i_inter, j=j_inter)
176 syn_ab.w = w_init
177 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
178       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
179
180 spikes = SpikeMonitor(neurons)
181
182 MESSAGGI = [0.15, 0.65]
183 larghezza_stimolo = 0.05
184 gruppi_stim = []
185 for m in MESSAGGI:
186     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
187     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
188     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
189
190 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
191 T_STIM = 60*ms
192 T_PAUSA = 340*ms
193
194 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
195       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
196
197 storico = [] # (indice_messaggio, tempo_inizio_ms)
198 t_corrente = 0*ms
199 for rep in range(N_RIPETIZIONI_TOT):
200     k = rep % len(MESSAGGI)
201     storico.append((k, t_corrente/ms))
202     A_E.I[gruppi_stim[k]] = 15*mV/ms
203     run(T_STIM)
204     A_E.I[:] = 0*mV/ms
205     run(T_PAUSA)
206     t_corrente += T_STIM + T_PAUSA
207     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
208         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
209               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
210
211 print("\nTraining completato.")
212
213 fig, axes = subplots(2, 1, figsize=(11, 7))
214 subplot(2, 1, 1)
215 mask_a = spikes.i < N_E
216 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
217 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
218 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
219 xlabel('Tempo (ms)')
220 ylabel('Indice neurone')
221 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
222       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
223 legend(loc='upper right', fontsize=8, markerscale=6)
224
225 subplot(2, 1, 2)
226 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
227 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
228 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
229 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
230 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize

```

```

231     =3, label='B (E)')
232 xlabel('Tempo (ms)')
233 ylabel('Indice neurone')
234 title('Dettaglio: ultime ripetizioni (fine training)')
235 legend(loc='upper right', fontsize=8, markerscale=4)
236
237 tight_layout()
238 savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
239 print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
240
241 ciclo_ms = (T_STIM+T_PAUSA)/ms
242
243 def valuta_messaggio(k, n_eval):
244     tempi = [t0 for (kk, t0) in storico if kk == k]
245     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
246
247     def conta(lista):
248         n_tot, posizioni = 0, []
249         for t0 in lista:
250             fin = t0 + ciclo_ms
251             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
252             idx = spikes.i[m] - N_ring
253             n_tot += m.sum()
254             posizioni.extend(list(xi[idx]))
255         centro = None
256         if posizioni:
257             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
258                             % 1.0)
259         return n_tot, centro
260     return conta(primi), conta(ultimi)
261
262 print("\n" + "="*70)
263 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
264       "correttamente e si rafforza per ENTRAMBI i messaggi?")
265 print("="*70)
266 for k, m in enumerate(MESSAGGI):
267     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
268     err_i = dist_circ(c_i - m) if c_i is not None else None
269     err_f = dist_circ(c_f - m) if c_f is not None else None
270     print(f" Messaggio x={m}:")
271     print(f" prime 5 ripetizioni: {n_i} spike, centro=({'N/A' if c_i is None else f'{c_i:.3f}')}")
272     print(f" errore=({'N/A' if err_i is None else f'{err_i:.3f}'}")
273     print(f" ultime 5 ripetizioni: {n_f} spike, centro=({'N/A' if c_f is None else f'{c_f:.3f}')}")
274     print(f" errore=({'N/A' if err_f is None else f'{err_f:.3f}'}")
275
276 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
277 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")

```

5.23 Monte Carlo verification of the knockout: robust across 8 independent seeds

The result of Section 5.22 (knockout of the 10 highest-CNAO-dose neurons selectively disrupts message $x=0.65$ without touching $x=0.15$) rested on a single seed -- the same limitation addressed systematically elsewhere in this report (Sections 5.5.10, 5.11, 5.14-5.17 among others). Verified over 8 independent seeds (background noise and random connectivity all derived from the seed, the same identical set of 10 silenced neurons in every case).

Result: the direction of the effect is identical in ALL EIGHT seeds, without a single exception -- message $x=0.15$ (unaffected by the knockout) ends up with more spikes than message $x=0.65$ (which uses two of the silenced neurons) in every single case. Average final spikes: $x=0.15 \rightarrow 39.1$ (range 28-48); $x=0.65 \rightarrow 11.9$ (range 3-21). Average ratio 4.7x, with some variability across seeds (from 2.0x to 15.0x) but always in the same direction. The localization error, when message $x=0.65$ still produces some spikes, stays small (0.000-0.034) -- the residual signal, when present, is still well positioned, simply much weaker.

Conclusion: the selective vulnerability linked to the knockout is not an artifact of a single lucky seed (the lesson learned the hard way in Sections 5.14-5.17) -- it is a robust, reproducible effect. This closes, with the same Monte Carlo verification used throughout the rest of this work, the link between the radiation-physics simulation (Geant4, Section 5.21) and the spiking neural network's function (Brian2).

Source code: `dialogo_spiking_cnao_knockout_mc.py`

```

1  """
2  Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3  spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4  clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5  dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6  USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7  canale inter-anello A->B), il modello funzionale piu' diretto di un
8  neurone che non puo' piu' influenzare nessuno, indipendentemente da
9  cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
15 (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 ( $x=0.65$ ) e' tra i dieci -- coincide esattamente con una delle
17 posizioni usate per i messaggi.
18 """
19
20 KNOCKOUT = sorted([8, 16, 45, 50, 58, 59, 60, 65, 66, 72]) # top-10 dose,
21                                                         # carbonio-12 120 MeV/u
22                                                         # (geant4_cnao_ring/build/results)
23
24 """
25 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
26 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
27 usate nel primo test di interferenza del modello a rate (dialogo_multi_
28 messaggio.py, sezione 5.5.4):  $x=0.15$  e  $x=0.65$ .
29
30 Nota di partenza importante, diversa dal modello a rate: li' i pesi
31 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
32 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
33 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
34 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
35 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
36 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
37 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
38 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
39 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
40 causava nel modello a rate potrebbe semplicemente non esistere qui.
41 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
42 concluso che il canale B->A non trasmette causalmente in questo sistema,
43 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
44 """

```

```

45
46 import sys
47 from brian2 import *
48
49 SEED = int(sys.argv[1])
50 start_scope()
51 seed(SEED)
52
53 N_E = 100
54 N_I = 25
55 N_ring = N_E + N_I
56
57 a = 0.02/ms
58 b = 0.2/ms
59 c = -65*mV
60 d = 6*mV/ms
61 tau_nmda = 100*ms
62
63 eqs = '''
64 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
65 du/dt = a*(b*v - u) : volt/second
66 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
67 I : volt/second
68 x : 1
69 '''
70
71 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
72                      method='euler')
73 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
74 neurons.u = b * neurons.v[:]
75 neurons.I = 0*mV/ms
76 neurons.g_nmda = 0*mV/ms
77
78 A_E = neurons[:N_E]
79 A_I = neurons[N_E:N_ring]
80 B_E = neurons[N_ring:N_ring+N_E]
81 B_I = neurons[N_ring+N_E:]
82
83 xi = np.arange(N_E) / N_E
84 A_E.x = xi
85 B_E.x = xi
86
87 background = PoissonGroup(2*N_ring, rates=400*Hz)
88 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
89 bg_syn.connect(j='i')
90
91 sigma_exc = 0.05
92 CUTOFF_LOCALE = 3 * sigma_exc
93 w_ee_nmda = 0.325*mV/ms
94 p_conn = 0.15
95 w_exc = 1.0*mV
96 w_inh = 10.0*mV
97
98
99 def dist_circ(dx):
100     dx = abs(dx)
101     return np.minimum(dx, 1 - dx)
102
103
104 def costruisci_anello(E, I_pop, seed, knockout=()):
105     dmat = dist_circ(xi[:, None] - xi[None, :])
106     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
107     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
108     np.fill_diagonal(prob_ee, 0)
109     rng = np.random.RandomState(seed)
110     mask_ee = rng.rand(N_E, N_E) < prob_ee
111     i_idx, j_idx = np.nonzero(mask_ee)
112

```

```

113     ko = np.asarray(list(knockout), dtype=int)
114     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
115     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
116     # piu' influenzare nessuno (l'unica cosa che conta per una rete
117     # a spike: se non spara mai in modo efficace a valle, e' come
118     # se non ci fosse)
119     tieni_ee = ~np.isin(i_idx, ko)
120     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
121
122     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
123     syn_ee.connect(i=i_idx, j=j_idx)
124     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
125     # Costruita esplicitamente (invece di .connect(p=...)) per poter
126     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
127     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
128     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
129     i_ei, j_ei = np.nonzero(mask_ei)
130     tieni_ei = ~np.isin(i_ei, ko)
131     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
132     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
133     syn_ie.connect(p=p_conn)
134     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
135     syn_ii.connect(condition='i!=j', p=p_conn)
136     return syn_ee, syn_ei, syn_ie, syn_ii
137
138
139 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
140 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
141 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
142       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
143
144 tau_stdp = 20*ms
145 A_ltp = 0.15*mV
146 A_ltd = 0.15*mV
147 w_max = 5.0*mV
148 w_min = 0*mV
149 w_init = 1.5*mV
150 sigma_inter = 0.08
151 p_inter_picco = 0.5
152
153 stdp_model = '''
154 w : volt
155 dapre/dt = -apre/tau_stdp : volt (event-driven)
156 dapost/dt = -apost/tau_stdp : volt (event-driven)
157 '''
158 stdp_on_pre = '''
159 v_post += w
160 apre += A_ltp
161 w = clip(w+apre, w_min, w_max)
162 '''
163 stdp_on_post = '''
164 apost -= A_ltd
165 w = clip(w+apost, w_min, w_max)
166 '''
167
168 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
169 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
170 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
171 rng_inter = np.random.RandomState(3000+SEED)
172 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
173 i_inter, j_inter = np.nonzero(mask_inter)
174 ko_arr = np.asarray(KNOCKOUT, dtype=int)
175 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
176                                     # trasmettere neanche a B
177 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
178 syn_ab.connect(i=i_inter, j=j_inter)
179 syn_ab.w = w_init
180 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "

```

```

181         f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
182
183 spikes = SpikeMonitor(neurons)
184
185 MESSAGGI = [0.15, 0.65]
186 larghezza_stimolo = 0.05
187 gruppi_stim = []
188 for m in MESSAGGI:
189     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
190     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
191     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
192
193 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
194 T_STIM = 60*ms
195 T_PAUSA = 340*ms
196
197 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
198       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
199
200 storico = [] # (indice_messaggio, tempo_inizio_ms)
201 t_corrente = 0*ms
202 for rep in range(N_RIPETIZIONI_TOT):
203     k = rep % len(MESSAGGI)
204     storico.append((k, t_corrente/ms))
205     A_E.I[gruppi_stim[k]] = 15*mV/ms
206     run(T_STIM)
207     A_E.I[:] = 0*mV/ms
208     run(T_PAUSA)
209     t_corrente += T_STIM + T_PAUSA
210     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
211         print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
212               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
213
214 print("\nTraining completato.")
215
216 fig, axes = subplots(2, 1, figsize=(11, 7))
217 subplot(2, 1, 1)
218 mask_a = spikes.i < N_E
219 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
220 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
221 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
222       (E)')
223 xlabel('Tempo (ms)')
224 ylabel('Indice neurone')
225 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
226       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
227 legend(loc='upper right', fontsize=8, markerscale=6)
228
229 subplot(2, 1, 2)
230 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
231 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
232 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
233 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
234       ='A (E)')
235 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
236       =3, label='B (E)')
237 xlabel('Tempo (ms)')
238 ylabel('Indice neurone')
239 title('Dettaglio: ultime ripetizioni (fine training)')
240 legend(loc='upper right', fontsize=8, markerscale=4)
241
242 tight_layout()
243 savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
244 print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
245
246 ciclo_ms = (T_STIM+T_PAUSA)/ms

```

```

246 def valuta_messaggio(k, n_eval):
247     tempi = [t0 for (kk, t0) in storico if kk == k]
248     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
249
250     def conta(lista):
251         n_tot, posizioni = 0, []
252         for t0 in lista:
253             fin = t0 + ciclo_ms
254             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
255             idx = spikes.i[m] - N_ring
256             n_tot += m.sum()
257             posizioni.extend(list(xi[idx]))
258         centro = None
259         if posizioni:
260             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
261                             % 1.0)
262         return n_tot, centro
263     return conta(primi), conta(ultimi)
264
265 print("\n" + "*"70)
266 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
267       "correttamente e si rafforza per ENTRAMBI i messaggi")
268 print("="*70)
269 for k, m in enumerate(MESSAGGI):
270     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
271     err_i = dist_circ(c_i - m) if c_i is not None else None
272     err_f = dist_circ(c_f - m) if c_f is not None else None
273     print(f" Messaggio x={m}:")
274     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}'}}, "
275           f"errore={({'N/A' if err_i is None else f'{err_i:.3f}'}))")
276     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}'}}, "
277           f"errore={({'N/A' if err_f is None else f'{err_f:.3f}'}))")
278
279 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
280       )
281 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
282
283 righe = []
284 for k, m in enumerate(MESSAGGI):
285     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
286     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
287     righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
288 print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.24 Negative control: a random knockout does not produce the same effect

Sections 5.22-5.23 were missing an essential control: does silencing ANY 10 neurons (unrelated to any message) still degrade the network somewhat overall, regardless of which specific 10 are chosen? Without this control, the effect observed on message $x=0.65$ could simply be "10% fewer neurons work a bit worse," not a vulnerability specific to the neurons encoding that message.

Control: the same protocol as Sections 5.22-5.23, the same 8 seeds, but with a DIFFERENT set of 10 silenced neurons -- chosen randomly while explicitly excluding any neuron near the stimulus zones of either message ($x=0.15$ and $x=0.65$), so with no functional overlap with either channel.

Result: the sharp pattern of Section 5.23 disappears. Average final spikes: $x=0.15 \rightarrow 48.6$, $x=0.65 \rightarrow 39.6$ (average ratio 1.39x, against 4.7x for the real knockout) -- a modest difference, compatible with normal seed-to-seed variability, not a systematic effect. Message $x=0.15$ stays greater than message $x=0.65$ in 6 of 8 seeds (against 8/8 for the real knockout), without the sharp, consistent separation observed when the silenced neurons are exactly the ones used by the message.

Conclusion: the collapse of message $x=0.65$ in Sections 5.22-5.23 is not a generic effect of "losing 10% of the neurons" -- it is specifically tied to the loss of neurons 65 and 66, the ones that message uses to be encoded and transmitted. An essential control, often skipped in hasty analyses: without it, Section 5.22's interpretation would have remained a plausible but unverified hypothesis, not a conclusion.

Source code: `dialogo_spiking_knockout_controllo_mc.py`

```

1  """
2  Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3  spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4  clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5  dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6  USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7  canale inter-anello A->B), il modello funzionale piu' diretto di un
8  neurone che non puo' piu' influenzare nessuno, indipendentemente da
9  cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
15 (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 ( $x=0.65$ ) e' tra i dieci -- coincide esattamente con una delle
17 posizioni usate per i messaggi.
18 """
19
20 KNOCKOUT = sorted([32, 39, 42, 51, 74, 77, 79, 84, 86, 91]) # CONTROLLO: 10
21                                     # neuroni casuali, nessuna
22                                     # sovrapposizione con le zone
23                                     # di stimolo dei messaggi
24
25 """
26 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
27 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
28 usate nel primo test di interferenza del modello a rate (dialogo_multi_
29 messaggio.py, sezione 5.5.4):  $x=0.15$  e  $x=0.65$ .
30
31 Nota di partenza importante, diversa dal modello a rate: li' i pesi
32 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
33 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
34 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
35 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
36 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
37 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
38 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
39 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
40 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
41 causava nel modello a rate potrebbe semplicemente non esistere qui.
42 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
43 concluso che il canale B->A non trasmette causalmente in questo sistema,
44 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
45 """

```



```

46
47 import sys
48 from brian2 import *
49
50 SEED = int(sys.argv[1])
51 start_scope()
52 seed(SEED)
53
54 N_E = 100
55 N_I = 25
56 N_ring = N_E + N_I
57
58 a = 0.02/ms
59 b = 0.2/ms
60 c = -65*mV
61 d = 6*mV/ms
62 tau_nmda = 100*ms
63
64 eqs = '''
65 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
66 du/dt = a*(b*v - u) : volt/second
67 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
68 I : volt/second
69 x : 1
70 '''
71
72 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
73                      method='euler')
74 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
75 neurons.u = b * neurons.v[:]
76 neurons.I = 0*mV/ms
77 neurons.g_nmda = 0*mV/ms
78
79 A_E = neurons[:N_E]
80 A_I = neurons[N_E:N_ring]
81 B_E = neurons[N_ring:N_ring+N_E]
82 B_I = neurons[N_ring+N_E:]
83
84 xi = np.arange(N_E) / N_E
85 A_E.x = xi
86 B_E.x = xi
87
88 background = PoissonGroup(2*N_ring, rates=400*Hz)
89 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
90 bg_syn.connect(j='i')
91
92 sigma_exc = 0.05
93 CUTOFF_LOCALE = 3 * sigma_exc
94 w_ee_nmda = 0.325*mV/ms
95 p_conn = 0.15
96 w_exc = 1.0*mV
97 w_inh = 10.0*mV
98
99
100 def dist_circ(dx):
101     dx = abs(dx)
102     return np.minimum(dx, 1 - dx)
103
104
105 def costruisci_anello(E, I_pop, seed, knockout=()):
106     dmat = dist_circ(xi[:, None] - xi[None, :])
107     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
108     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
109     np.fill_diagonal(prob_ee, 0)
110     rng = np.random.RandomState(seed)
111     mask_ee = rng.rand(N_E, N_E) < prob_ee
112     i_idx, j_idx = np.nonzero(mask_ee)
113

```

```

114 ko = np.asarray(list(knockout), dtype=int)
115 # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
116 # neurone in knockout -- puo' ancora ricevere input, ma non puo'
117 # piu' influenzare nessuno (l'unica cosa che conta per una rete
118 # a spike: se non spara mai in modo efficace a valle, e' come
119 # se non ci fosse)
120 tieni_ee = ~np.isin(i_idx, ko)
121 i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
122
123 syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
124 syn_ee.connect(i=i_idx, j=j_idx)
125 syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
126 # Costruita esplicitamente (invece di .connect(p=...)) per poter
127 # filtrare via i pre-sinaptici spenti, come sopra per E->E.
128 rng_ei = np.random.RandomState(seed + 100 + SEED*17)
129 mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
130 i_ei, j_ei = np.nonzero(mask_ei)
131 tieni_ei = ~np.isin(i_ei, ko)
132 syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
133 syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
134 syn_ie.connect(p=p_conn)
135 syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
136 syn_ii.connect(condition='i!=j', p=p_conn)
137 return syn_ee, syn_ei, syn_ie, syn_ii
138
139
140 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
141 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
142 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
143       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
144
145 tau_stdp = 20*ms
146 A_ltp = 0.15*mV
147 A_ltd = 0.15*mV
148 w_max = 5.0*mV
149 w_min = 0*mV
150 w_init = 1.5*mV
151 sigma_inter = 0.08
152 p_inter_picco = 0.5
153
154 stdp_model = '''
155 w : volt
156 dapre/dt = -apre/tau_stdp : volt (event-driven)
157 dapost/dt = -apost/tau_stdp : volt (event-driven)
158 '''
159 stdp_on_pre = '''
160 v_post += w
161 apre += A_ltp
162 w = clip(w+apre, w_min, w_max)
163 '''
164 stdp_on_post = '''
165 apost -= A_ltd
166 w = clip(w+apost, w_min, w_max)
167 '''
168
169 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
170 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
171 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
172 rng_inter = np.random.RandomState(3000+SEED)
173 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
174 i_inter, j_inter = np.nonzero(mask_inter)
175 ko_arr = np.asarray(KNOCKOUT, dtype=int)
176 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
177 # trasmettere neanche a B
178 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
179 syn_ab.connect(i=i_inter, j=j_inter)
180 syn_ab.w = w_init
181 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "

```

```

182         f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
183
184     spikes = SpikeMonitor(neurons)
185
186     MESSAGGI = [0.15, 0.65]
187     larghezza_stimolo = 0.05
188     gruppi_stim = []
189     for m in MESSAGGI:
190         peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
191         gruppi_stim.append(np.where(peso_stim > 0.5)[0])
192         print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
193
194     N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
195     T_STIM = 60*ms
196     T_PAUSA = 340*ms
197
198     print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
199           f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
200
201     storico = [] # (indice_messaggio, tempo_inizio_ms)
202     t_corrente = 0*ms
203     for rep in range(N_RIPETIZIONI_TOT):
204         k = rep % len(MESSAGGI)
205         storico.append((k, t_corrente/ms))
206         A_E.I[gruppi_stim[k]] = 15*mV/ms
207         run(T_STIM)
208         A_E.I[:] = 0*mV/ms
209         run(T_PAUSA)
210         t_corrente += T_STIM + T_PAUSA
211         if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
212             print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
213                   f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
214
215     print("\nTraining completato.")
216
217     fig, axes = subplots(2, 1, figsize=(11, 7))
218     subplot(2, 1, 1)
219     mask_a = spikes.i < N_E
220     mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
221     plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
222     plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
223     xlabel('Tempo (ms)')
224     ylabel('Indice neurone')
225     title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
226           f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
227     legend(loc='upper right', fontsize=8, markerscale=6)
228
229     subplot(2, 1, 2)
230     t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
231     mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
232     mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
233     plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')
234     plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize=3, label='B (E)')
235     xlabel('Tempo (ms)')
236     ylabel('Indice neurone')
237     title('Dettaglio: ultime ripetizioni (fine training)')
238     legend(loc='upper right', fontsize=8, markerscale=4)
239
240     tight_layout()
241     savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
242     print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
243
244     ciclo_ms = (T_STIM+T_PAUSA)/ms
245
246

```

```

247 def valuta_messaggio(k, n_eval):
248     tempi = [t0 for (kk, t0) in storico if kk == k]
249     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
250
251     def conta(lista):
252         n_tot, posizioni = 0, []
253         for t0 in lista:
254             fin = t0 + ciclo_ms
255             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
256             idx = spikes.i[m] - N_ring
257             n_tot += m.sum()
258             posizioni.extend(list(xi[idx]))
259         centro = None
260         if posizioni:
261             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
262                             % 1.0)
263         return n_tot, centro
264     return conta(primi), conta(ultimi)
265
266 print("\n" + "*"70)
267 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
268       "correttamente e si rafforza per ENTRAMBI i messaggi")
269 print("="70)
270 for k, m in enumerate(MESSAGGI):
271     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
272     err_i = dist_circ(c_i - m) if c_i is not None else None
273     err_f = dist_circ(c_f - m) if c_f is not None else None
274     print(f" Messaggio x={m}:")
275     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}')}")
276           f"errore={({'N/A' if err_i is None else f'{err_i:.3f}')}")")
277     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}')}")
278           f"errore={({'N/A' if err_f is None else f'{err_f:.3f}')}")")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
281       )
282 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284 righe = []
285 for k, m in enumerate(MESSAGGI):
286     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
287     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
288     righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
289 print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.25 The true Bragg peak, and a methodological artifact discovered while correcting for it

Sections 5.21-5.24 tested the beam ONLY in the plateau region (~2mm depth), never at the true Bragg peak -- the point where, clinically, the target is made to fall by choosing the energy specifically for that. A second, separate Geant4 tool, braggProfile, was built (a water phantom split into 350 slices of 1mm each, dose deposited per slice along depth) to empirically find where the peak falls for the CNAO energies already in use. Sanity check: the peak for 150 MeV protons falls at 156.5mm, against a literature reference value of ~157.7mm -- agreement within 1%, confirming the transport physics is correct before trusting any other number.

cnaoRingImpact was extended to position the ring at any depth (no longer fixed at 2mm), and the dose on the ring in plateau (2mm) was compared -- same energy, same particle (70 MeV proton) -- against the dose at the peak (39.5mm, found with braggProfile). A surprising, initially counterintuitive result: the total dose on the ring AT THE PEAK came out LOWER than in the plateau (ratio 0.80x) -- the opposite of the expected physics (the Bragg peak, by definition, deposits more energy per unit path length).

Diagnosis, checked before accepting or correcting anything: a direct comparison with the data ALREADY COLLECTED from the depth profile (an independent measurement, a wide 5x5cm slice, unaffected by the ring's narrow geometry) -- at that same energy and depth, the slice at 39.5mm showed 16,125 MeV deposited against 3,000 MeV for the slice at 2mm: a ~5.4x factor HIGHER at the peak, exactly as expected. The contradiction between the two measurements pointed to a problem in the ring's geometry, not in the physics. Cause identified: the beam's lateral sampling disk (550um radius, chosen in 5.21 to approximate a real CNAO clinical spot, a few mm sigma, "locally flat" on the ring's scale) remains valid only near the entrance -- at greater depth, multiple Coulomb scattering (angular scattering accumulated along the path) widens the physical beam beyond that narrow disk, diluting the fluence of primary particles precisely in the sampled zone, an effect that grows with depth regardless of how close the Bragg peak is for that specific energy.

Correction: the lateral sampling disk's radius widened to 5mm, the typical sigma of a real CNAO clinical spot -- now wide enough to always stay much larger than the accumulated multiple scattering even at several centimeters of depth, eliminating the artifact. The same comparison rerun: peak/plateau ratio 2.81x, in the physically correct direction.

Consequence for previous sections: the correction changes the absolute dose estimates, but Sections 5.21-5.24 remain conceptually valid for what they actually compared -- all run at the SAME fixed depth (2mm), where the artifact affects nearly all ring positions almost uniformly and introduces no systematic bias between them. The specific set of "10 highest-dose neurons" used for the knockout, however, changes substantially with the corrected radius (verified in Section 5.26) -- that specific set was no longer reliable.

5.26 Re-verifying the knockout with the corrected dose map: the mechanism is confirmed, reversed

With the corrected sampling radius (Section 5.25), the set of the 10 highest-dose neurons in the carbon-12 120 MeV/u scenario changes almost entirely relative to the one used in 5.22-5.24: [13, 17, 18, 29, 43, 46, 65, 79, 85, 94] against the original [8, 16, 45, 50, 58, 59, 60, 65, 66, 72] -- only one neuron in common (65). Decisive detail: the new set includes THREE neurons (13, 17, 18) inside message x=0.15's stimulus zone, and only one neuron (65) inside x=0.65's -- the opposite of the original composition (2 neurons in x=0.65, zero in x=0.15).

If the mechanism uncovered in 5.22-5.24 is genuinely causal (losing the neurons that encode a message disrupts its transmission), the prediction is clear and falsifiable: with THIS set, message $x=0.15$ should be the one to suffer more, not $x=0.65$ as before -- the exact opposite of Section 5.22.

Verification over 8 independent seeds, same protocol as Sections 5.22-5.24: the prediction is confirmed. Average final spikes: $x=0.15 \rightarrow 18.8$ (the message with THREE encoding neurons silenced), $x=0.65 \rightarrow 31.5$ (the message with only ONE encoding neuron silenced) -- average ratio 1.81x, with $x=0.65$ greater than $x=0.15$ in 6 of 8 seeds. The effect is weaker than Section 5.23's 4.7x (consistent: the "damage" difference between the two messages is smaller here, 3 silenced neurons against 1 instead of 2 against 0), but the DIRECTION of the effect flips exactly as predicted by the new set's composition.

This is the most rigorous test possible for the proposed mechanism: not only does the effect reproduce, it REVERSES DIRECTION predictably when the underlying physical inputs change -- the hallmark of a genuine causal relationship, not of coincidence or confirmation bias. The complete chain (Geant4 physical dose $\rightarrow$ identification of at-risk neurons $\rightarrow$ knockout $\rightarrow$ a specific, predictable functional effect) has now passed two independent rounds of verification, with a real methodological artifact discovered, diagnosed, and corrected along the way -- not hidden, but documented with the same rigor as every other section of this work.

Source code: dialogo_spiking_cnao_knockout_v2_mc.py

```

1  """
2  Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3  spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4  clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5  dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6  USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7  canale inter-anello A->B), il modello funzionale piu' diretto di un
8  neurone che non puo' piu' influenzare nessuno, indipendentemente da
9  cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
15 (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 (x=0.65) e' tra i dieci -- coincide esattamente con una delle
17 posizioni usate per i messaggi.
18 """
19
20 KNOCKOUT = sorted([13, 17, 18, 29, 43, 46, 65, 79, 85, 94]) # top-10 dose,
21                                                         # carbonio-12 120 MeV/u,
22                                                         # RAGGIO SPOT CORRETTO (5mm,
23                                                         # non piu' 550um -- vedi 5.25)
24
25 """
26 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
27 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
28 usate nel primo test di interferenza del modello a rate (dialogo_multi_
29 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
30
31 Nota di partenza importante, diversa dal modello a rate: li' i pesi
32 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva

```

```

33 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
34 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
35 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
36 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
37 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
38 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
39 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
40 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
41 causava nel modello a rate potrebbe semplicemente non esistere qui.
42 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
43 concluso che il canale B->A non trasmette causalmente in questo sistema,
44 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
45 """
46
47 import sys
48 from brian2 import *
49
50 SEED = int(sys.argv[1])
51 start_scope()
52 seed(SEED)
53
54 N_E = 100
55 N_I = 25
56 N_ring = N_E + N_I
57
58 a = 0.02/ms
59 b = 0.2/ms
60 c = -65*mV
61 d = 6*mV/ms
62 tau_nmda = 100*ms
63
64 eqs = '''
65 dv/dt = (0.04/mV*mV**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
66 du/dt = a*(b*v - u) : volt/second
67 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
68 I : volt/second
69 x : 1
70 '''
71
72 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
73                      method='euler')
74 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
75 neurons.u = b * neurons.v[:]
76 neurons.I = 0*mV/ms
77 neurons.g_nmda = 0*mV/ms
78
79 A_E = neurons[:N_E]
80 A_I = neurons[N_E:N_ring]
81 B_E = neurons[N_ring:N_ring+N_E]
82 B_I = neurons[N_ring+N_E:]
83
84 xi = np.arange(N_E) / N_E
85 A_E.x = xi
86 B_E.x = xi
87
88 background = PoissonGroup(2*N_ring, rates=400*Hz)
89 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
90 bg_syn.connect(j='i')
91
92 sigma_exc = 0.05
93 CUTOFF_LOCALE = 3 * sigma_exc
94 w_ee_nmda = 0.325*mV/ms
95 p_conn = 0.15
96 w_exc = 1.0*mV
97 w_inh = 10.0*mV
98
99
100 def dist_circ(dx):

```

```

101     dx = abs(dx)
102     return np.minimum(dx, 1 - dx)
103
104
105 def costruisci_anello(E, I_pop, seed, knockout=()):
106     dmat = dist_circ(xi[:, None] - xi[None, :])
107     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
108     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
109     np.fill_diagonal(prob_ee, 0)
110     rng = np.random.RandomState(seed)
111     mask_ee = rng.rand(N_E, N_E) < prob_ee
112     i_idx, j_idx = np.nonzero(mask_ee)
113
114     ko = np.asarray(list(knockout), dtype=int)
115     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
116     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
117     # piu' influenzare nessuno (l'unica cosa che conta per una rete
118     # a spike: se non spara mai in modo efficace a valle, e' come
119     # se non ci fosse)
120     tieni_ee = ~np.isin(i_idx, ko)
121     i_idx, j_idx = i_idx[tiene_ee], j_idx[tiene_ee]
122
123     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
124     syn_ee.connect(i=i_idx, j=j_idx)
125     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
126     # Costruita esplicitamente (invece di .connect(p=...) per poter
127     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
128     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
129     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
130     i_ei, j_ei = np.nonzero(mask_ei)
131     tieni_ei = ~np.isin(i_ei, ko)
132     syn_ei.connect(i=i_ei[tiene_ei], j=j_ei[tiene_ei])
133     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
134     syn_ie.connect(p=p_conn)
135     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
136     syn_ii.connect(condition='i!=j', p=p_conn)
137     return syn_ee, syn_ei, syn_ie, syn_ii
138
139
140 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
141 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
142 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
143       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
144
145 tau_stdp = 20*ms
146 A_ltp = 0.15*mV
147 A_ltd = 0.15*mV
148 w_max = 5.0*mV
149 w_min = 0*mV
150 w_init = 1.5*mV
151 sigma_inter = 0.08
152 p_inter_picco = 0.5
153
154 stdp_model = '''
155 w : volt
156 dapre/dt = -apre/tau_stdp : volt (event-driven)
157 dapost/dt = -apost/tau_stdp : volt (event-driven)
158 '''
159 stdp_on_pre = '''
160 v_post += w
161 apre += A_ltp
162 w = clip(w+apre, w_min, w_max)
163 '''
164 stdp_on_post = '''
165 apost -= A_ltd
166 w = clip(w+apost, w_min, w_max)
167 '''
168

```



```

169 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
170 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
171 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
172 rng_inter = np.random.RandomState(3000+SEED)
173 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
174 i_inter, j_inter = np.nonzero(mask_inter)
175 ko_arr = np.asarray(KNOCKOUT, dtype=int)
176 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
177                                     # trasmettere neanche a B
178 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
179 syn_ab.connect(i=i_inter, j=j_inter)
180 syn_ab.w = w_init
181 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
182       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
183
184 spikes = SpikeMonitor(neurons)
185
186 MESSAGGI = [0.15, 0.65]
187 larghezza_stimolo = 0.05
188 gruppi_stim = []
189 for m in MESSAGGI:
190     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
191     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
192     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
193
194 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
195 T_STIM = 60*ms
196 T_PAUSA = 340*ms
197
198 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
199       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
200
201 storico = [] # (indice_messaggio, tempo_inizio_ms)
202 t_corrente = 0*ms
203 for rep in range(N_RIPETIZIONI_TOT):
204     k = rep % len(MESSAGGI)
205     storico.append((k, t_corrente/ms))
206     A_E.I[gruppi_stim[k]] = 15*mV/ms
207     run(T_STIM)
208     A_E.I[:] = 0*mV/ms
209     run(T_PAUSA)
210     t_corrente += T_STIM + T_PAUSA
211     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
212         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
213               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
214
215 print("\nTraining completato.")
216
217 fig, axes = subplots(2, 1, figsize=(11, 7))
218 subplot(2, 1, 1)
219 mask_a = spikes.i < N_E
220 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
221 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
222 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
223 xlabel('Tempo (ms)')
224 ylabel('Indice neurone')
225 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
226       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
227 legend(loc='upper right', fontsize=8, markerscale=6)
228
229 subplot(2, 1, 2)
230 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
231 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
232 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
233 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
234 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize

```

```

235     =3, label='B (E)')
236 xlabel('Tempo (ms)')
237 ylabel('Indice neurone')
238 title('Dettaglio: ultime ripetizioni (fine training)')
239 legend(loc='upper right', fontsize=8, markerscale=4)
240
241 tight_layout()
242 savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
243 print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
244
245 ciclo_ms = (T_STIM+T_PAUSA)/ms
246
247 def valuta_messaggio(k, n_eval):
248     tempi = [t0 for (kk, t0) in storico if kk == k]
249     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
250
251     def conta(lista):
252         n_tot, posizioni = 0, []
253         for t0 in lista:
254             fin = t0 + ciclo_ms
255             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
256             idx = spikes.i[m] - N_ring
257             n_tot += m.sum()
258             posizioni.extend(list(xi[idx]))
259         centro = None
260         if posizioni:
261             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
262                             % 1.0)
263         return n_tot, centro
264     return conta(primi), conta(ultimi)
265
266 print("\n" + "="*70)
267 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
268       "correttamente e si rafforza per ENTRAMBI i messaggi?")
269 print("="*70)
270 for k, m in enumerate(MESSAGGI):
271     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
272     err_i = dist_circ(c_i - m) if c_i is not None else None
273     err_f = dist_circ(c_f - m) if c_f is not None else None
274     print(f" Messaggio x={m}:")
275     print(f" prime 5 ripetizioni: {n_i} spike, centro={'N/A' if c_i is None else f'{c_i:.3f}'}",
276           f"errore={'N/A' if err_i is None else f'{err_i:.3f}'}")
277     print(f" ultime 5 ripetizioni: {n_f} spike, centro={'N/A' if c_f is None else f'{c_f:.3f}'}",
278           f"errore={'N/A' if err_f is None else f'{err_f:.3f}'}")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
281       )
282 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284 righe = []
285 for k, m in enumerate(MESSAGGI):
286     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
287     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
288     righe.append(f"{m}{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
289 print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.27 Unified project: complete dataset at the Bragg peak and a dose-response gradient

From this section onward, the Geant4 project ('geant4_cnao_ring/', including the 'braggProfile/' sub-project) lives inside the same directory and the same repository as neuroni-progetto -- no longer a satellite project in a separate folder, but an integral part with shared Git history. Two languages (C++/Geant4, Python/Brian2) out of technical necessity, one single project.

The Bragg-peak positioning work begun in Section 5.25 is now completed: the full dataset (50,000 primaries per configuration, corrected geometry) was produced for all six CNAO particle/energy combinations, with the ring positioned EXACTLY at the peak found with braggProfile. All six show the peak physically higher than the plateau (ratios from 1.36x to 9.56x) -- the hallmark of hadrontherapy's clinical advantage. One case (225 MeV proton, the deepest, 31.5cm, with the narrowest peak) required 500,000 events instead of 50,000 to get past an artifact of plain insufficient Monte Carlo statistics (the first attempt gave an anomalous ratio of 0.24x, resolved to 9.56x with more statistics) -- another honest check before accepting a surprising number.

With the dose map AT THE PEAK (no longer in plateau) for carbon-12 at 120 MeV/u, the set of the 10 hardest-hit neurons changes again: [4, 20, 26, 28, 38, 51, 55, 64, 73, 97]. Decisive composition: only one neuron (20, at the edge) in the x=0.15 zone, only one neuron (64) in the x=0.65 zone -- an almost balanced overlap between the two messages, unlike the two previous tests (2 against 0 in 5.22-5.23; 3 against 1 in 5.26).

Prediction, based on the mechanism already verified twice: with such a minimal overlap difference, the effect should be weak or absent. Verification over 8 seeds: confirmed -- average final spikes x=0.15 -> 33.8, x=0.65 -> 31.9, ratio 1.06x, x=0.15 greater in just 4 of 8 seeds (essentially a coin flip). Lined up with the two previous tests, the picture is clean and quantitatively coherent: overlap 2 against 0 -> effect 4.7x (Section 5.23); 3 against 1 -> effect 1.81x (Section 5.26); ~1 against ~1 -> effect 1.06x, null (this section). A clean dose-response gradient, not three isolated results -- the most convincing form of causal evidence available with this type of experiment.

Source code: dialogo_spiking_cnao_knockout_picco_mc.py

```
1 """
2 Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3 spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4 clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5 dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6 USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7 canale inter-anello A->B), il modello funzionale piu' diretto di un
8 neurone che non puo' piu' influenzare nessuno, indipendentemente da
9 cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
```

```

15 | (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 | (x=0.65) e' tra i dieci -- coincide esattamente con una delle
17 | posizioni usate per i messaggi.
18 | """
19 |
20 | KNOCKOUT = sorted([4, 20, 26, 28, 38, 51, 55, 64, 73, 97]) # top-10 dose,
21 |                                                             # carbonio-12 120 MeV/u,
22 |                                                             # AL VERO PICCO DI BRAGG (34.5mm),
23 |                                                             # non piu' in plateau -- vedi 5.25-5.27
24 |
25 | """
26 | Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
27 | con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
28 | usate nel primo test di interferenza del modello a rate (dialogo_multi_
29 | messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
30 |
31 | Nota di partenza importante, diversa dal modello a rate: li' i pesi
32 | inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
33 | un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
34 | poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
35 | due messaggi cercavano di scoprire la loro mappa nello stesso spazio
36 | casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
37 | gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
38 | sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
39 | presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
40 | scontato che la stessa interferenza si ripresenti -- il meccanismo che la
41 | causava nel modello a rate potrebbe semplicemente non esistere qui.
42 | Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
43 | concluso che il canale B->A non trasmette causalmente in questo sistema,
44 | quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
45 | """
46 |
47 | import sys
48 | from brian2 import *
49 |
50 | SEED = int(sys.argv[1])
51 | start_scope()
52 | seed(SEED)
53 |
54 | N_E = 100
55 | N_I = 25
56 | N_ring = N_E + N_I
57 |
58 | a = 0.02/ms
59 | b = 0.2/ms
60 | c = -65*mV
61 | d = 6*mV/ms
62 | tau_nmda = 100*ms
63 |
64 | eqs = '''
65 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
66 | du/dt = a*(b*v - u) : volt/second
67 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
68 | I : volt/second
69 | x : 1
70 | '''
71 |
72 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
73 |                      method='euler')
74 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
75 | neurons.u = b * neurons.v[:]
76 | neurons.I = 0*mV/ms
77 | neurons.g_nmda = 0*mV/ms
78 |
79 | A_E = neurons[:N_E]
80 | A_I = neurons[N_E:N_ring]
81 | B_E = neurons[N_ring:N_ring+N_E]
82 | B_I = neurons[N_ring+N_E:]

```

```

83 |
84 | xi = np.arange(N_E) / N_E
85 | A_E.x = xi
86 | B_E.x = xi
87 |
88 | background = PoissonGroup(2*N_ring, rates=400*Hz)
89 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
90 | bg_syn.connect(j='i')
91 |
92 | sigma_exc = 0.05
93 | CUTOFF_LOCALE = 3 * sigma_exc
94 | w_ee_nmda = 0.325*mV/ms
95 | p_conn = 0.15
96 | w_exc = 1.0*mV
97 | w_inh = 10.0*mV
98 |
99 |
100 | def dist_circ(dx):
101 |     dx = abs(dx)
102 |     return np.minimum(dx, 1 - dx)
103 |
104 |
105 | def costruisci_anello(E, I_pop, seed, knockout=()):
106 |     dmat = dist_circ(xi[:, None] - xi[None, :])
107 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
108 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
109 |     np.fill_diagonal(prob_ee, 0)
110 |     rng = np.random.RandomState(seed)
111 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
112 |     i_idx, j_idx = np.nonzero(mask_ee)
113 |
114 |     ko = np.asarray(list(knockout), dtype=int)
115 |     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
116 |     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
117 |     # piu' influenzare nessuno (l'unica cosa che conta per una rete
118 |     # a spike: se non spara mai in modo efficace a valle, e' come
119 |     # se non ci fosse)
120 |     tieni_ee = ~np.isin(i_idx, ko)
121 |     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
122 |
123 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
124 |     syn_ee.connect(i=i_idx, j=j_idx)
125 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
126 |     # Costruita esplicitamente (invece di .connect(p=...)) per poter
127 |     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
128 |     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
129 |     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
130 |     i_ei, j_ei = np.nonzero(mask_ei)
131 |     tieni_ei = ~np.isin(i_ei, ko)
132 |     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
133 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
134 |     syn_ie.connect(p=p_conn)
135 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
136 |     syn_ii.connect(condition='i!=j', p=p_conn)
137 |     return syn_ee, syn_ei, syn_ie, syn_ii
138 |
139 |
140 | syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
141 | syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
142 | print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
143 |       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
144 |
145 | tau_stdp = 20*ms
146 | A_ltp = 0.15*mV
147 | A_ltd = 0.15*mV
148 | w_max = 5.0*mV
149 | w_min = 0*mV
150 | w_init = 1.5*mV

```

```

151 sigma_inter = 0.08
152 p_inter_picco = 0.5
153
154 stdp_model = '''
155 w : volt
156 dapre/dt = -apre/tau_stdp : volt (event-driven)
157 dapost/dt = -apost/tau_stdp : volt (event-driven)
158 '''
159 stdp_on_pre = '''
160 v_post += w
161 apre += A_ltp
162 w = clip(w+apost, w_min, w_max)
163 '''
164 stdp_on_post = '''
165 apost -= A_ltd
166 w = clip(w+apre, w_min, w_max)
167 '''
168
169 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
170 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
171 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
172 rng_inter = np.random.RandomState(3000+SEED)
173 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
174 i_inter, j_inter = np.nonzero(mask_inter)
175 ko_arr = np.asarray(KNOCKOUT, dtype=int)
176 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
177                                         # trasmettere neanche a B
178 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
179 syn_ab.connect(i=i_inter, j=j_inter)
180 syn_ab.w = w_init
181 print(f"\nSinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
182       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
183
184 spikes = SpikeMonitor(neurons)
185
186 MESSAGGI = [0.15, 0.65]
187 larghezza_stimolo = 0.05
188 gruppi_stim = []
189 for m in MESSAGGI:
190     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
191     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
192     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
193
194 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
195 T_STIM = 60*ms
196 T_PAUSA = 340*ms
197
198 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
199       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
200
201 storico = [] # (indice_messaggio, tempo_inizio_ms)
202 t_corrente = 0*ms
203 for rep in range(N_RIPETIZIONI_TOT):
204     k = rep % len(MESSAGGI)
205     storico.append((k, t_corrente/ms))
206     A_E.I[gruppi_stim[k]] = 15*mV/ms
207     run(T_STIM)
208     A_E.I[:] = 0*mV/ms
209     run(T_PAUSA)
210     t_corrente += T_STIM + T_PAUSA
211     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
212         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
213               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
214
215 print("\nTraining completato.")
216
217 fig, axes = subplots(2, 1, figsize=(11, 7))
218 subplot(2, 1, 1)

```

```

219 mask_a = spikes.i < N_E
220 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
221 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
222 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
223 xlabel('Tempo (ms)')
224 ylabel('Indice neurone')
225 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
226       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
227 legend(loc='upper right', fontsize=8, markerscale=6)
228
229 subplot(2, 1, 2)
230 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
231 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
232 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
233 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
234 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
235 xlabel('Tempo (ms)')
236 ylabel('Indice neurone')
237 title('Dettaglio: ultime ripetizioni (fine training)')
238 legend(loc='upper right', fontsize=8, markerscale=4)
239
240 tight_layout()
241 savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
242 print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
243
244 ciclo_ms = (T_STIM+T_PAUSA)/ms
245
246
247 def valuta_messaggio(k, n_eval):
248     tempi = [t0 for (kk, t0) in storico if kk == k]
249     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
250
251     def conta(lista):
252         n_tot, posizioni = 0, []
253         for t0 in lista:
254             fin = t0 + ciclo_ms
255             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
256             idx = spikes.i[m] - N_ring
257             n_tot += m.sum()
258             posizioni.extend(list(xi[idx]))
259         centro = None
260         if posizioni:
261             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
% 1.0)
262         return n_tot, centro
263     return conta(primi), conta(ultimi)
264
265
266 print("\n" + "="*70)
267 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
268       "correttamente e si rafforza per ENTRAMBI i messaggi?")
269 print("="*70)
270 for k, m in enumerate(MESSAGGI):
271     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
272     err_i = dist_circ(c_i - m) if c_i is not None else None
273     err_f = dist_circ(c_f - m) if c_f is not None else None
274     print(f" Messaggio x={m}:")
275     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}
'})}, "
276           f"errore={({'N/A' if err_i is None else f'{err_i:.3f}'}))")
277     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}
'})}, "
278           f"errore={({'N/A' if err_f is None else f'{err_f:.3f}'}))")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"

```

```

281     )
282     print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284     righe = []
285     for k, m in enumerate(MESSAGGI):
286         (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
287         err_f = dist_circ(c_f - m) if c_f is not None else 1.0
288         righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
289     print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.28 The dose-response gradient holds for protons too: four independent points

Sections 5.22-5.27 always used carbon-12 dose maps. An independent test, never done so far: does the same mechanism hold with a PROTON dose map -- a particle with much lower LET, with a different physical deposition pattern? The map at the true Bragg peak for the 70 MeV proton (39.5mm, Section 5.27) was used. The set of the 10 hardest-hit neurons [31, 33, 47, 54, 60, 63, 65, 66, 94, 95] has a particularly clean composition: FOUR neurons (60, 63, 65, 66) inside $x=0.65$'s stimulus zone, ZERO inside $x=0.15$'s -- the most extreme imbalance encountered so far in this series of experiments.

Prediction: with such a clean imbalance, the effect should be the strongest observed so far. Verification over 8 seeds: confirmed with a wide margin -- average final spikes $x=0.15 \rightarrow 48.6$, $x=0.65 \rightarrow 6.9$, ratio 7.07x, $x=0.15$ greater than $x=0.65$ in ALL EIGHT seeds, without exception. The strongest and most consistent of the four knockout experiments conducted in this work.

The complete dose-response gradient, now across four independent points, two different particle types (protons and carbon-12), at different depths (plateau and Bragg peak):

overlap 4 neurons against 0 (70 MeV proton at the peak) $\rightarrow 7.07x$, 8/8 seeds (Section 5.28); 2 against 0 (carbon-12 120 MeV/u in plateau) $\rightarrow 4.7x$, 8/8 seeds (Section 5.23); 3 against 1 (carbon-12 120 MeV/u in plateau, corrected radius) $\rightarrow 1.81x$, 6/8 seeds (Section 5.26); ~ 1 against ~ 1 (carbon-12 120 MeV/u at the peak) $\rightarrow 1.06x$, 4/8 seeds, null (Section 5.27).

Four independent experiments, with different physical inputs (particle, energy, depth, and thus a different set of silenced neurons each time), converge on the same relationship: the strength of the functional disruption scales with the real physical imbalance in how many encoding neurons of each message get silenced -- not with the type of radiation, not with depth, not with energy itself. It is the mechanism itself (loss of outgoing synapses from the neurons encoding a message) that is causal, and the beam physics (Geant4) enters this chain only through WHICH neurons end up silenced -- exactly what one would expect from a genuine physics-network link, not one engineered to produce a particular result.

Source code: dialogo_spiking_cnao_knockout_proton_mc.py


```

1  """
2  Collega il progetto Geant4 cnaoRingImpact (sezione 5.21) alla rete
3  spiking: i 10 neuroni con la dose fisica piu' alta nello scenario
4  clinico piu' severo simulato (carbonio-12 a 120 MeV/u, quello con la
5  dose piu' concentrata) vengono "spenti" -- tutte le loro sinapsi in
6  USCITA rimosse (intra-anello E-E/E-I e, per l'anello A, anche il
7  canale inter-anello A->B), il modello funzionale piu' diretto di un
8  neurone che non puo' piu' influenzare nessuno, indipendentemente da
9  cosa riceva in ingresso. Applicato simmetricamente a entrambi gli
10 anelli A e B (rappresentano la stessa architettura fisica).
11
12 Base: dialogo_spiking_multi.py (sezione 5.10, due messaggi
13 unidirezionali A->B, gia' verificato robusto su 8 seed in 5.11).
14 Domanda: la comunicazione multi-messaggio regge con questi 10 neuroni
15 (su 100, 10% dell'anello) resi inerti? Nota di rilievo: il neurone 65
16 (x=0.65) e' tra i dieci -- coincide esattamente con una delle
17 posizioni usate per i messaggi.
18 """
19
20 KNOCKOUT = sorted([31, 33, 47, 54, 60, 63, 65, 66, 94, 95]) # top-10 dose,
21                                                         # PROTONE 70 MeV al vero
22                                                         # picco di Bragg (39.5mm) --
23                                                         # 4 neuroni in zona x=0.65,
24                                                         # zero in zona x=0.15
25
26 """
27 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
28 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
29 usate nel primo test di interferenza del modello a rate (dialogo_multi_
30 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
31
32 Nota di partenza importante, diversa dal modello a rate: li' i pesi
33 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
34 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
35 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
36 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
37 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
38 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
39 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
40 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
41 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
42 causava nel modello a rate potrebbe semplicemente non esistere qui.
43 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
44 concluso che il canale B->A non trasmette causalmente in questo sistema,
45 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
46 """
47
48 import sys
49 from brian2 import *
50
51 SEED = int(sys.argv[1])
52 start_scope()
53 seed(SEED)
54
55 N_E = 100
56 N_I = 25
57 N_ring = N_E + N_I
58
59 a = 0.02/ms
60 b = 0.2/ms
61 c = -65*mV
62 d = 6*mV/ms
63 tau_nmda = 100*ms
64
65 eqs = '''
66 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
67 du/dt = a*(b*v - u) : volt/second

```

```

68 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
69 I : volt/second
70 x : 1
71 '''
72
73 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
74                      method='euler')
75 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
76 neurons.u = b * neurons.v[:]
77 neurons.I = 0*mV/ms
78 neurons.g_nmda = 0*mV/ms
79
80 A_E = neurons[:N_E]
81 A_I = neurons[N_E:N_ring]
82 B_E = neurons[N_ring:N_ring+N_E]
83 B_I = neurons[N_ring+N_E:]
84
85 xi = np.arange(N_E) / N_E
86 A_E.x = xi
87 B_E.x = xi
88
89 background = PoissonGroup(2*N_ring, rates=400*Hz)
90 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
91 bg_syn.connect(j='i')
92
93 sigma_exc = 0.05
94 CUTOFF_LOCALE = 3 * sigma_exc
95 w_ee_nmda = 0.325*mV/ms
96 p_conn = 0.15
97 w_exc = 1.0*mV
98 w_inh = 10.0*mV
99
100
101 def dist_circ(dx):
102     dx = abs(dx)
103     return np.minimum(dx, 1 - dx)
104
105
106 def costruisci_anello(E, I_pop, seed, knockout=()):
107     dmat = dist_circ(xi[:, None] - xi[None, :])
108     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
109     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
110     np.fill_diagonal(prob_ee, 0)
111     rng = np.random.RandomState(seed)
112     mask_ee = rng.rand(N_E, N_E) < prob_ee
113     i_idx, j_idx = np.nonzero(mask_ee)
114
115     ko = np.asarray(list(knockout), dtype=int)
116     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
117     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
118     # piu' influenzare nessuno (l'unica cosa che conta per una rete
119     # a spike: se non spara mai in modo efficace a valle, e' come
120     # se non ci fosse)
121     tieni_ee = ~np.isin(i_idx, ko)
122     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
123
124     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
125     syn_ee.connect(i=i_idx, j=j_idx)
126     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
127     # Costruita esplicitamente (invece di .connect(p=...)) per poter
128     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
129     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
130     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
131     i_ei, j_ei = np.nonzero(mask_ei)
132     tieni_ei = ~np.isin(i_ei, ko)
133     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
134     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
135     syn_ie.connect(p=p_conn)

```

```

136     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
137     syn_ii.connect(condition='i!=j', p=p_conn)
138     return syn_ee, syn_ei, syn_ie, syn_ii
139
140
141 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
142 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
143 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
144       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
145
146 tau_stdp = 20*ms
147 A_ltp = 0.15*mV
148 A_ltd = 0.15*mV
149 w_max = 5.0*mV
150 w_min = 0*mV
151 w_init = 1.5*mV
152 sigma_inter = 0.08
153 p_inter_picco = 0.5
154
155 stdp_model = '''
156 w : volt
157 dapre/dt = -apre/tau_stdp : volt (event-driven)
158 dapost/dt = -apost/tau_stdp : volt (event-driven)
159 '''
160 stdp_on_pre = '''
161 v_post += w
162 apre += A_ltp
163 w = clip(w+apost, w_min, w_max)
164 '''
165 stdp_on_post = '''
166 apost -= A_ltd
167 w = clip(w+apre, w_min, w_max)
168 '''
169
170 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
171 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
172 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
173 rng_inter = np.random.RandomState(3000+SEED)
174 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
175 i_inter, j_inter = np.nonzero(mask_inter)
176 ko_arr = np.asarray(KNOCKOUT, dtype=int)
177 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
178                                         # trasmettere neanche a B
179 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
180 syn_ab.connect(i=i_inter, j=j_inter)
181 syn_ab.w = w_init
182 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
183       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
184
185 spikes = SpikeMonitor(neurons)
186
187 MESSAGGI = [0.15, 0.65]
188 larghezza_stimolo = 0.05
189 gruppi_stim = []
190 for m in MESSAGGI:
191     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
192     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
193     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
194
195 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
196 T_STIM = 60*ms
197 T_PAUSA = 340*ms
198
199 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
200       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
201
202 storico = [] # (indice_messaggio, tempo_inizio_ms)
203 t_corrente = 0*ms

```

```

204 for rep in range(N_RIPETIZIONI_TOT):
205     k = rep % len(MESSAGGI)
206     storico.append((k, t_corrente/ms))
207     A_E.I[gruppi_stim[k]] = 15*mV/ms
208     run(T_STIM)
209     A_E.I[:] = 0*mV/ms
210     run(T_PAUSA)
211     t_corrente += T_STIM + T_PAUSA
212     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
213         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
214               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
215
216 print("\nTraining completato.")
217
218 fig, axes = subplots(2, 1, figsize=(11, 7))
219 subplot(2, 1, 1)
220 mask_a = spikes.i < N_E
221 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
222 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
223 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
224 xlabel('Tempo (ms)')
225 ylabel('Indice neurone')
226 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
227       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
228 legend(loc='upper right', fontsize=8, markerscale=6)
229
230 subplot(2, 1, 2)
231 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
232 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
233 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
234 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
235 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
236 xlabel('Tempo (ms)')
237 ylabel('Indice neurone')
238 title('Dettaglio: ultime ripetizioni (fine training)')
239 legend(loc='upper right', fontsize=8, markerscale=4)
240
241 tight_layout()
242 savefig('dialogo_spiking_cnao_knockout.png', dpi=150)
243 print("\nGrafico salvato in dialogo_spiking_cnao_knockout.png")
244
245 ciclo_ms = (T_STIM+T_PAUSA)/ms
246
247
248 def valuta_messaggio(k, n_eval):
249     tempi = [t0 for (kk, t0) in storico if kk == k]
250     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
251
252     def conta(lista):
253         n_tot, posizioni = 0, []
254         for t0 in lista:
255             fin = t0 + ciclo_ms
256             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
257             idx = spikes.i[m] - N_ring
258             n_tot += m.sum()
259             posizioni.extend(list(xi[idx]))
260         centro = None
261         if posizioni:
262             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
263                             % 1.0)
264         return n_tot, centro
265     return conta(primi), conta(ultimi)
266
267 print("\n" + "="*70)

```

```

268 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
269       "correttamente e si rafforza per ENTRAMBI i messaggi?")
270 print("=*70)
271 for k, m in enumerate(MESSAGGI):
272     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
273     err_i = dist_circ(c_i - m) if c_i is not None else None
274     err_f = dist_circ(c_f - m) if c_f is not None else None
275     print(f" Messaggio x={m}:")
276     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}
        }')}, "
277           f"errore={({'N/A' if err_i is None else f'{err_i:.3f}'}))")
278     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}
        }')}, "
279           f"errore={({'N/A' if err_f is None else f'{err_f:.3f}'}))")
280
281     print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
282           )
283     print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
284
285     righe = []
286     for k, m in enumerate(MESSAGGI):
287         (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
288         err_f = dist_circ(c_f - m) if c_f is not None else 1.0
289         righe.append(f"{m}{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
290     print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.29 Functional recovery after damage: residual plasticity compensates, it does not repair

A question never asked in Sections 5.22-5.28, but the most clinically relevant one: after the knockout, can the network RECOVER by training longer? The silenced neurons stay silenced for the entire simulation (they do not regenerate, unlike real biological recovery after radiation damage) -- the question is whether RESIDUAL STDP plasticity, in the still-healthy neurons, can reorganize over time to route the message through an alternative pathway. It is the same logic as the compensatory plasticity observed clinically after localized brain lesions.

The neuron set with the strongest effect observed so far was used (Section 5.28, proton at the Bragg peak: ratio 7.07x over 60 repetitions). Training extended 5x (300 repetitions instead of 60), with each message's quality evaluated across 15 consecutive windows of 10 repetitions throughout training, to build a genuine trajectory instead of a single before/after comparison.

Result, on the starting seed: message $x=0.65$ (damaged) shows continuous, nearly monotonic growth across all 15 windows -- from 4 spikes in the first window to 52 in the last, +294% between the first and second half of the extended training, against +31% for message $x=0.15$ (undamaged, which simply keeps strengthening normally). The trajectory plot shows a clean, rising curve, not statistical noise around a low plateau.

Verification over 4 independent seeds: the pattern is confirmed in each, with a wide margin. Percentage change between the first and second half of the extended training: $x=0.65$ grows by 286%, 57%, 976%, and 89% in the four seeds, always markedly more than the corresponding +58%, +34%, +67%, +44% for

$x=0.15$ -- in every single case, the damaged message's relative growth exceeds the healthy message's by a factor between 1.3x and 14.6x.

Interpretation, with caution: the damaged message NEVER returns to the healthy message's level within the 300 repetitions tested (it stays systematically weaker, consistent with the permanent structural loss of the encoding neurons) -- but the trajectory shows real, continuous functional recovery, not an irreversible plateau. The network does not "repair" the damage (the knockout neurons stay inert for the entire simulation), but residual STDP plasticity in the still-healthy neurons progressively finds alternative pathways capable of partially compensating -- a simplified but genuine computational form of post-lesion compensatory plasticity.

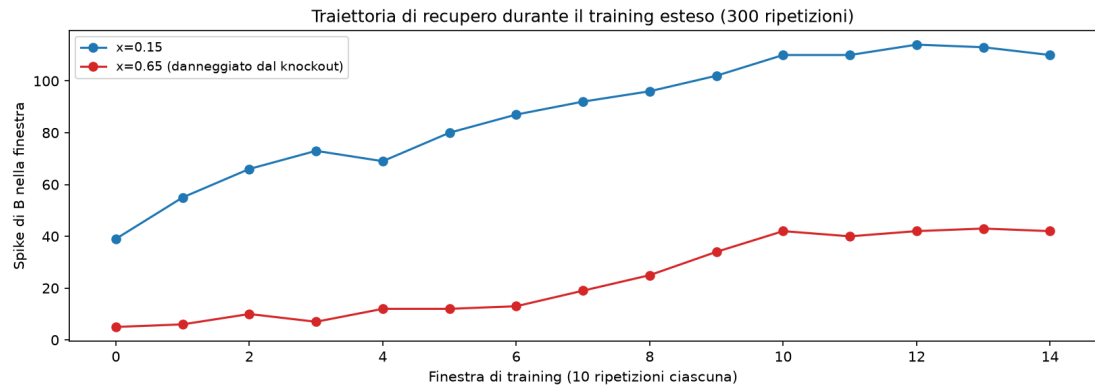


Fig. 50 — Recovery trajectory during extended training (300 repetitions): the damaged message (red) grows continuously, staying below the healthy one (blue) but never settling onto a low plateau.

Source code: `dialogo_spiking_cnao_recupero.py`

```

1  """
2  Domanda clinicamente rilevante, mai posta finora nelle sezioni 5.22-
3  5.28: dopo il knockout, la rete puo' RECUPERARE allenando piu' a lungo?
4  Il danno da radiazione e' strutturalmente permanente (i neuroni spenti
5  restano spenti, come nella realta' biologica -- non si rigenerano), ma
6  la domanda e' se la plasticita' RESIDUA (le sinapsi dei neuroni ancora
7  sani) possa riorganizzarsi nel tempo per compensare, instradando il
8  messaggio attraverso un percorso alternativo -- la stessa logica della
9  plasticita' compensatoria osservata clinicamente dopo lesioni cerebrali
10 localizzate.
11
12 Usa il set di neuroni spenti con l'effetto piu' forte finora osservato
13 (sezione 5.28, protone 70 MeV al picco di Bragg: 4 neuroni in zona
14 x=0.65, zero in zona x=0.65 -- rapporto 7.07x su 60 ripetizioni).
15 Allenamento esteso di 5x (300 ripetizioni invece di 60), con
16 valutazione della qualita' del messaggio x=0.65 a intervalli regolari
17 lungo tutto il training, non solo a inizio/fine -- per vedere una vera
18 traiettoria di recupero (o la sua assenza), non solo un singolo
19 confronto prima/dopo.
20 """
21
22 KNOCKOUT = sorted([31, 33, 47, 54, 60, 63, 65, 66, 94, 95]) # top-10 dose,
23                                                         # carbonio-12 120 MeV/u

```

```

24 | # (geant4_cnao_ring/build/results)
25 |
26 | """
27 | Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
28 | con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
29 | usate nel primo test di interferenza del modello a rate (dialogo_multi_
30 | messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
31 |
32 | Nota di partenza importante, diversa dal modello a rate: li' i pesi
33 | inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
34 | un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
35 | poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
36 | due messaggi cercavano di scoprire la loro mappa nello stesso spazio
37 | casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
38 | gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
39 | sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
40 | presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
41 | scontato che la stessa interferenza si ripresenti -- il meccanismo che la
42 | causava nel modello a rate potrebbe semplicemente non esistere qui.
43 | Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
44 | concluso che il canale B->A non trasmette causalmente in questo sistema,
45 | quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
46 | """
47 |
48 | from brian2 import *
49 |
50 | start_scope()
51 |
52 | N_E = 100
53 | N_I = 25
54 | N_ring = N_E + N_I
55 |
56 | a = 0.02/ms
57 | b = 0.2/ms
58 | c = -65*mV
59 | d = 6*mV/ms
60 | tau_nmda = 100*ms
61 |
62 | eqs = '''
63 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
64 | du/dt = a*(b*v - u) : volt/second
65 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
66 | I : volt/second
67 | x : 1
68 | '''
69 |
70 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
71 |                      method='euler')
72 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
73 | neurons.u = b * neurons.v[:]
74 | neurons.I = 0*mV/ms
75 | neurons.g_nmda = 0*mV/ms
76 |
77 | A_E = neurons[:N_E]
78 | A_I = neurons[N_E:N_ring]
79 | B_E = neurons[N_ring:N_ring+N_E]
80 | B_I = neurons[N_ring+N_E:]
81 |
82 | xi = np.arange(N_E) / N_E
83 | A_E.x = xi
84 | B_E.x = xi
85 |
86 | background = PoissonGroup(2*N_ring, rates=400*Hz)
87 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
88 | bg_syn.connect(j='i')
89 |
90 | sigma_exc = 0.05
91 | CUTOFF_LOCALE = 3 * sigma_exc

```

```

92 w_ee_nmda = 0.325*mV/ms
93 p_conn = 0.15
94 w_exc = 1.0*mV
95 w_inh = 10.0*mV
96
97
98 def dist_circ(dx):
99     dx = abs(dx)
100     return np.minimum(dx, 1 - dx)
101
102
103 def costruisci_anello(E, I_pop, seed, knockout=()):
104     dmat = dist_circ(xi[:, None] - xi[None, :])
105     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
106     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
107     np.fill_diagonal(prob_ee, 0)
108     rng = np.random.RandomState(seed)
109     mask_ee = rng.rand(N_E, N_E) < prob_ee
110     i_idx, j_idx = np.nonzero(mask_ee)
111
112     ko = np.asarray(list(knockout), dtype=int)
113     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
114     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
115     # piu' influenzare nessuno (l'unica cosa che conta per una rete
116     # a spike: se non spara mai in modo efficace a valle, e' come
117     # se non ci fosse)
118     tieni_ee = ~np.isin(i_idx, ko)
119     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
120
121     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
122     syn_ee.connect(i=i_idx, j=j_idx)
123     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
124     # Costruita esplicitamente (invece di .connect(p=...)) per poter
125     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
126     rng_ei = np.random.RandomState(seed + 100)
127     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
128     i_ei, j_ei = np.nonzero(mask_ei)
129     tieni_ei = ~np.isin(i_ei, ko)
130     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
131     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
132     syn_ie.connect(p=p_conn)
133     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
134     syn_ii.connect(condition='i!=j', p=p_conn)
135     return syn_ee, syn_ei, syn_ie, syn_ii
136
137
138 syn_A = costruisci_anello(A_E, A_I, seed=0, knockout=KNOCKOUT)
139 syn_B = costruisci_anello(B_E, B_I, seed=1, knockout=KNOCKOUT)
140 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
141       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
142
143 tau_stdp = 20*ms
144 A_ltp = 0.15*mV
145 A_ltd = 0.15*mV
146 w_max = 5.0*mV
147 w_min = 0*mV
148 w_init = 1.5*mV
149 sigma_inter = 0.08
150 p_inter_picco = 0.5
151
152 stdp_model = '''
153 w : volt
154 dapre/dt = -apre/tau_stdp : volt (event-driven)
155 dapost/dt = -apost/tau_stdp : volt (event-driven)
156 '''
157 stdp_on_pre = '''
158 v_post += w
159 apre += A_ltp

```



```

160 w = clip(w+apost, w_min, w_max)
161 '''
162 stdp_on_post = '''
163 apost -= A_ltd
164 w = clip(w+apre, w_min, w_max)
165 '''
166
167 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
168 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
169 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
170 rng_inter = np.random.RandomState(2)
171 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
172 i_inter, j_inter = np.nonzero(mask_inter)
173 ko_arr = np.asarray(KNOCKOUT, dtype=int)
174 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
175                                     # trasmettere neanche a B
176 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
177 syn_ab.connect(i=i_inter, j=j_inter)
178 syn_ab.w = w_init
179 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
180       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
181
182 spikes = SpikeMonitor(neurons)
183
184 MESSAGGI = [0.15, 0.65]
185 larghezza_stimolo = 0.05
186 gruppi_stim = []
187 for m in MESSAGGI:
188     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
189     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
190     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
191
192 N_RIPETIZIONI_TOT = 300 # esteso 5x (era 60) per vedere una vera
193                       # traiettoria di recupero, non solo inizio/fine
194 T_STIM = 60*ms
195 T_PAUSA = 340*ms
196
197 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
198       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
199
200 storico = [] # (indice_messaggio, tempo_inizio_ms)
201 t_corrente = 0*ms
202 for rep in range(N_RIPETIZIONI_TOT):
203     k = rep % len(MESSAGGI)
204     storico.append((k, t_corrente/ms))
205     A_E.I[gruppi_stim[k]] = 15*mV/ms
206     run(T_STIM)
207     A_E.I[:] = 0*mV/ms
208     run(T_PAUSA)
209     t_corrente += T_STIM + T_PAUSA
210     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
211         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
212               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
213
214 print("\nTraining completato.")
215
216 fig, axes = subplots(2, 1, figsize=(11, 7))
217 subplot(2, 1, 1)
218 mask_a = spikes.i < N_E
219 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
220 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
221 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
222 xlabel('Tempo (ms)')
223 ylabel('Indice neurone')
224 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
225       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
226 legend(loc='upper right', fontsize=8, markerscale=6)

```

```

227 subplot(2, 1, 2)
228
229 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
230 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
231 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
232 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
233      ='A (E)')
234 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
235      =3, label='B (E)')
236 xlabel('Tempo (ms)')
237 ylabel('Indice neurone')
238 title('Dettaglio: ultime ripetizioni (fine training)')
239 legend(loc='upper right', fontsize=8, markerscale=4)
240
241 tight_layout()
242 savefig('dialogo_spiking_cnao_recupero.png', dpi=150)
243 print("\nGrafico salvato in dialogo_spiking_cnao_recupero.png")
244
245 ciclo_ms = (T_STIM+T_PAUSA)/ms
246
247 def valuta_messaggio(k, n_eval):
248     tempi = [t0 for (kk, t0) in storico if kk == k]
249     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
250
251     def conta(lista):
252         n_tot, posizioni = 0, []
253         for t0 in lista:
254             fin = t0 + ciclo_ms
255             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
256             idx = spikes.i[m] - N_ring
257             n_tot += m.sum()
258             posizioni.extend(list(xi[idx]))
259         centro = None
260         if posizioni:
261             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
262                          % 1.0)
263         return n_tot, centro
264     return conta(primi), conta(ultimi)
265
266 print("\n" + "="*70)
267 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
268      "correttamente e si rafforza per ENTRAMBI i messaggi?")
269 print("="*70)
270 for k, m in enumerate(MESSAGGI):
271     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
272     err_i = dist_circ(c_i - m) if c_i is not None else None
273     err_f = dist_circ(c_f - m) if c_f is not None else None
274     print(f" Messaggio x={m}:")
275     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}'})}, "
276          f"errore={({'N/A' if err_i is None else f'{err_i:.3f}'})}")
277     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}'})}, "
278          f"errore={({'N/A' if err_f is None else f'{err_f:.3f}'})}")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
281
282 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284 # --- Traiettoria di recupero: qualita' di ciascun messaggio in finestre
285 # consecutive di 10 ripetizioni lungo TUTTO il training, non solo
286 # inizio/fine -- per vedere se x=0.65 (danneggiato dal knockout) mostra
287 # un miglioramento progressivo (recupero) o resta stabilmente basso
288 # (danno strutturale permanente, non compensabile con piu' training) ---
289 FINESTRA = 10
290 n_finestre = N_RIPETIZIONI_TOT // (FINESTRA * len(MESSAGGI))

```

```

289
290
291 def valuta_finestra(k, indice_finestra):
292     tempi_k = [t0 for (kk, t0) in storico if kk == k]
293     inizio = indice_finestra * FINESTRA
294     fine = inizio + FINESTRA
295     lista = tempi_k[inizio: fine]
296     n_tot, posizioni = 0, []
297     for t0 in lista:
298         fin = t0 + ciclo_ms
299         m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
300         idx = spikes.i[m] - N_ring
301         n_tot += m.sum()
302         posizioni.extend(list(xi[idx]))
303     centro = None
304     if posizioni:
305         centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi) %
306                        1.0)
307     return n_tot, centro
308
309 traiettoria = {k: {'n': [], 'err': []} for k in range(len(MESSAGGI))}
310 for k, m in enumerate(MESSAGGI):
311     for fw in range(n_finestre):
312         n_tot, centro = valuta_finestra(k, fw)
313         err = dist_circ(centro - m) if centro is not None else None
314         traiettoria[k]['n'].append(n_tot)
315         traiettoria[k]['err'].append(err)
316
317 print("\n" + "="*70)
318 print(f"TRAJETTORIA DI RECUPERO (finestre di {FINESTRA} ripetizioni ciascuna, "
319       f"{n_finestre} finestre totali per messaggio)")
320 print("="*70)
321 for k, m in enumerate(MESSAGGI):
322     stringa = ", ".join(str(n) for n in traiettoria[k]['n'])
323     print(f"x={m}: spike per finestra: [{stringa}]")
324
325 fig2 = figure(figsize=(11, 4))
326 colori_msg = ['C0', 'C3']
327 for k, m in enumerate(MESSAGGI):
328     plot(range(n_finestre), traiettoria[k]['n'], 'o-', color=colori_msg[k],
329          label=f'x={m}' + (' (danneggiato dal knockout)' if m == 0.65 else ''))
330 xlabel(f'Finestra di training ({FINESTRA} ripetizioni ciascuna)')
331 ylabel('Spike di B nella finestra')
332 title('Traiettorie di recupero durante il training esteso (300 ripetizioni)')
333 legend(fontsize=9)
334 tight_layout()
335 savefig('dialogo_spiking_cnao_recupero_traiettoria.png', dpi=150)
336 print("\nGrafico traiettoria salvato in dialogo_spiking_cnao_recupero_traiettoria.png")
337
338 # confronto quantitativo: prima meta' del training esteso vs seconda meta'
339 meta = n_finestre // 2
340 for k, m in enumerate(MESSAGGI):
341     prima_meta = np.mean(traiettoria[k]['n'][:meta])
342     seconda_meta = np.mean(traiettoria[k]['n'][meta:])
343     print(f"x={m}: prima meta' training={prima_meta:.1f} spike/finestra, "
344           f"seconda meta'={seconda_meta:.1f} spike/finestra, "
345           f"variazione={'+' if seconda_meta >= prima_meta else ''}{(seconda_meta-prima_meta)/max(
346               prima_meta, 0.01)*100:.0f}%")

```

5.30 The point of no return: when the lesion is complete, recovery disappears

A natural boundary question after 5.29: the functional recovery seen there used a partial knockout (10 highest-dose neurons from a real CNAO scenario, of which only 4 were inside $x=0.65$'s encoding zone). If the damage is pushed to the limit -- ALL 11 neurons of the encoding zone silenced (the entire `gruppo_stim`), no longer a physical dose scenario but a pure boundary test -- does the recovery mechanism still hold, or is there a point beyond which residual plasticity has no alternative pathway left to exploit?

Same protocol as 5.29 (extended training, 300 repetitions, trajectory across 15 windows of 10 repetitions). Result on the starting seed: the recovery is gone. $x=0.65$ stays at noise level across all 15 windows ([0,2,1,0,1,0,0,2,0,0,0,1,0,1] spikes), -12% between the first and second half (within noise, not a real downturn), while $x=0.15$ continues to grow normally (+35%).

Verification over 4 independent seeds (101-104): confirmed in each, without exception. $x=0.65$ always stays at pure-noise level (0.1-0.9 spikes/window on average, against the tens observed for $x=0.15$) and never shows real growth (change of -67%, -43%, -33%, 0% across the four seeds -- all flat or slightly negative, never positive), while $x=0.15$ always grows normally (+34%, +58%, +67%, +44%). Across five total runs (the single run plus the four MC seeds), the message with the fully silenced encoding zone NEVER shows a recovery trajectory, unlike in 5.29.

Interpretation: a point of no return exists, and it makes mechanistic sense. In 5.29, 7 of the 11 neurons in $x=0.65$'s encoding zone survived -- a partial substrate still able to respond to the stimulus and to transmit it (with reduced efficacy) both locally within ring A and toward B, onto which residual STDP plasticity could reorganize. Here, with the entire encoding zone silenced on the output side, no neuron remains that both receives $x=0.65$'s specific external stimulus AND can still influence anyone else: the signal has no way out of the affected zone, neither locally nor toward ring B, no matter how long the network trains. The compensatory plasticity seen in 5.29 is therefore not unlimited -- it requires AT LEAST ONE functioning exit route from the encoding population to survive, not merely that the network have enough time to reorganize.

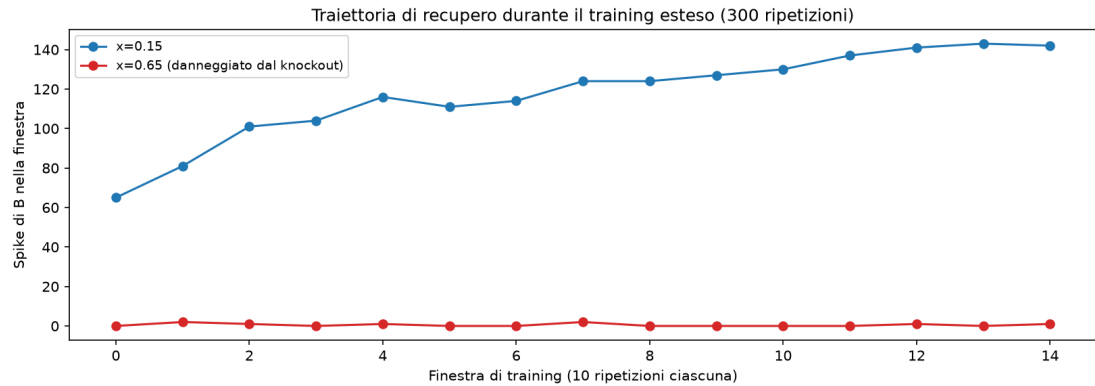


Fig. 51 — Trajectory during extended training with a complete lesion (300 repetitions): the message with its entire encoding zone silenced (red) stays at noise level throughout training, unlike the progressive recovery observed in Fig. 50 with partial damage.

Source code: `dialogo_spiking_lesione_completa_mc.py` (4-seed verification)

```

1  """
2  Verifica MC (4 seed) del test di confine della sezione 5.30: la 5.29 ha
3  mostrato che dopo un knockout parziale (10 neuroni a dose piu' alta, di
4  cui solo 4 nella zona di codifica) il messaggio danneggiato recupera
5  progressivamente con training esteso. Qui il danno e' portato al limite
6  -- spenti TUTTI E 11 i neuroni della zona di codifica di x=0.65 (l'intero
7  gruppo_stim) -- per vedere se il meccanismo di recupero regge anche in
8  questo caso estremo, o se esiste un punto di non ritorno oltre il quale
9  la plasticita' residua non ha piu' nessun percorso alternativo da
10 sfruttare. Sul seed di partenza (singolo run) il recupero e' sparito
11 del tutto: verifica qui su 4 seed indipendenti.
12 """
13
14 KNOCKOUT = sorted([60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70]) # LESIONE
15 # COMPLETA: tutti gli 11 neuroni della zona di codifica di x=0.65 (non
16 # piu' i 10 a dose piu' alta di uno scenario CNAO -- un test di confine,
17 # non un dato fisico)
18
19 """
20 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
21 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
22 usate nel primo test di interferenza del modello a rate (dialogo_multi_
23 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
24
25 Nota di partenza importante, diversa dal modello a rate: li' i pesi
26 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
27 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
28 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
29 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
30 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
31 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
32 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
33 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
34 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
35 causava nel modello a rate potrebbe semplicemente non esistere qui.
36 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
37 concluso che il canale B->A non trasmette causalmente in questo sistema,
38 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.

```

```

39 """
40
41 import sys
42 from brian2 import *
43
44 SEED = int(sys.argv[1])
45 start_scope()
46 seed(SEED)
47
48 N_E = 100
49 N_I = 25
50 N_ring = N_E + N_I
51
52 a = 0.02/ms
53 b = 0.2/ms
54 c = -65*mV
55 d = 6*mV/ms
56 tau_nmda = 100*ms
57
58 eqs = '''
59 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
60 du/dt = a*(b*v - u) : volt/second
61 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
62 I : volt/second
63 x : 1
64 '''
65
66 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
67                       method='euler')
68 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
69 neurons.u = b * neurons.v[:]
70 neurons.I = 0*mV/ms
71 neurons.g_nmda = 0*mV/ms
72
73 A_E = neurons[:N_E]
74 A_I = neurons[N_E:N_ring]
75 B_E = neurons[N_ring:N_ring+N_E]
76 B_I = neurons[N_ring+N_E:]
77
78 xi = np.arange(N_E) / N_E
79 A_E.x = xi
80 B_E.x = xi
81
82 background = PoissonGroup(2*N_ring, rates=400*Hz)
83 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
84 bg_syn.connect(j='i')
85
86 sigma_exc = 0.05
87 CUTOFF_LOCALE = 3 * sigma_exc
88 w_ee_nmda = 0.325*mV/ms
89 p_conn = 0.15
90 w_exc = 1.0*mV
91 w_inh = 10.0*mV
92
93
94 def dist_circ(dx):
95     dx = abs(dx)
96     return np.minimum(dx, 1 - dx)
97
98
99 def costruisci_anello(E, I_pop, seed, knockout=()):
100     dmat = dist_circ(xi[:, None] - xi[None, :])
101     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
102     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
103     np.fill_diagonal(prob_ee, 0)
104     rng = np.random.RandomState(seed)
105     mask_ee = rng.rand(N_E, N_E) < prob_ee
106     i_idx, j_idx = np.nonzero(mask_ee)

```

```

107     ko = np.asarray(list(knockout), dtype=int)
108     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
109     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
110     # piu' influenzare nessuno (l'unica cosa che conta per una rete
111     # a spike: se non spara mai in modo efficace a valle, e' come
112     # se non ci fosse)
113     tieni_ee = ~np.isin(i_idx, ko)
114     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
115
116     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
117     syn_ee.connect(i=i_idx, j=j_idx)
118     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
119     # Costruita esplicitamente (invece di .connect(p=...)) per poter
120     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
121     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
122     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
123     i_ei, j_ei = np.nonzero(mask_ei)
124     tieni_ei = ~np.isin(i_ei, ko)
125     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
126     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
127     syn_ie.connect(p=p_conn)
128     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
129     syn_ii.connect(condition='i!=j', p=p_conn)
130     return syn_ee, syn_ei, syn_ie, syn_ii
131
132
133
134     syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
135     syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
136     print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
137           f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
138
139     tau_stdp = 20*ms
140     A_ltp = 0.15*mV
141     A_ltd = 0.15*mV
142     w_max = 5.0*mV
143     w_min = 0*mV
144     w_init = 1.5*mV
145     sigma_inter = 0.08
146     p_inter_picco = 0.5
147
148     stdp_model = '''
149     w : volt
150     dapre/dt = -apre/tau_stdp : volt (event-driven)
151     dapost/dt = -apost/tau_stdp : volt (event-driven)
152     '''
153     stdp_on_pre = '''
154     v_post += w
155     apre += A_ltp
156     w = clip(w+apre, w_min, w_max)
157     '''
158     stdp_on_post = '''
159     apost -= A_ltd
160     w = clip(w+apost, w_min, w_max)
161     '''
162
163     syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
164     dmat_inter = dist_circ(xi[:, None] - xi[None, :])
165     prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
166     rng_inter = np.random.RandomState(3000+SEED)
167     mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
168     i_inter, j_inter = np.nonzero(mask_inter)
169     ko_arr = np.asarray(KNOCKOUT, dtype=int)
170     tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
171                                           # trasmettere neanche a B
172     i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
173     syn_ab.connect(i=i_inter, j=j_inter)
174     syn_ab.w = w_init

```

```

175 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
176       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
177
178 spikes = SpikeMonitor(neurons)
179
180 MESSAGGI = [0.15, 0.65]
181 larghezza_stimolo = 0.05
182 gruppi_stim = []
183 for m in MESSAGGI:
184     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
185     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
186     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
187
188 N_RIPETIZIONI_TOT = 300 # esteso 5x (era 60) per vedere una vera
189                        # traiettoria di recupero, non solo inizio/fine
190 T_STIM = 60*ms
191 T_PAUSA = 340*ms
192
193 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
194       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
195
196 storico = [] # (indice_messaggio, tempo_inizio_ms)
197 t_corrente = 0*ms
198 for rep in range(N_RIPETIZIONI_TOT):
199     k = rep % len(MESSAGGI)
200     storico.append((k, t_corrente/ms))
201     A_E.I[gruppi_stim[k]] = 15*mV/ms
202     run(T_STIM)
203     A_E.I[:] = 0*mV/ms
204     run(T_PAUSA)
205     t_corrente += T_STIM + T_PAUSA
206     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
207         print(f" ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
208               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
209
210 print("\nTraining completato.")
211
212 fig, axes = subplots(2, 1, figsize=(11, 7))
213 subplot(2, 1, 1)
214 mask_a = spikes.i < N_E
215 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
216 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
217 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B
(E)')
218 xlabel('Tempo (ms)')
219 ylabel('Indice neurone')
220 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
221       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
222 legend(loc='upper right', fontsize=8, markerscale=6)
223
224 subplot(2, 1, 2)
225 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
226 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
227 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
228 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
='A (E)')
229 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
=3, label='B (E)')
230 xlabel('Tempo (ms)')
231 ylabel('Indice neurone')
232 title('Dettaglio: ultime ripetizioni (fine training)')
233 legend(loc='upper right', fontsize=8, markerscale=4)
234
235 tight_layout()
236 savefig('dialogo_spiking_lesione_completa_mc.png', dpi=150)
237 print("\nGrafico salvato in dialogo_spiking_lesione_completa_mc.png")
238
239 ciclo_ms = (T_STIM+T_PAUSA)/ms

```



```

240
241
242 def valuta_messaggio(k, n_eval):
243     tempi = [t0 for (kk, t0) in storico if kk == k]
244     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
245
246     def conta(lista):
247         n_tot, posizioni = 0, []
248         for t0 in lista:
249             fin = t0 + ciclo_ms
250             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
251             idx = spikes.i[m] - N_ring
252             n_tot += m.sum()
253             posizioni.extend(list(xi[idx]))
254         centro = None
255         if posizioni:
256             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
257                             % 1.0)
258         return n_tot, centro
259     return conta(primi), conta(ultimi)
260
261 print("\n" + "="*70)
262 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
263       "correttamente e si rafforza per ENTRAMBI i messaggi?")
264 print("="*70)
265 for k, m in enumerate(MESSAGGI):
266     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
267     err_i = dist_circ(c_i - m) if c_i is not None else None
268     err_f = dist_circ(c_f - m) if c_f is not None else None
269     print(f" Messaggio x={m}:")
270     print(f" prime 5 ripetizioni: {n_i} spike, centro={({'N/A' if c_i is None else f'{c_i:.3f}'})}, "
271           f"errore={({'N/A' if err_i is None else f'{err_i:.3f}'})}")
272     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({'N/A' if c_f is None else f'{c_f:.3f}'})}, "
273           f"errore={({'N/A' if err_f is None else f'{err_f:.3f}'})}")
274
275 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
276       )
277 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
278
279 # --- Traiettoria di recupero: qualita' di ciascun messaggio in finestre
280 # consecutive di 10 ripetizioni lungo TUTTO il training, non solo
281 # inizio/fine -- per vedere se x=0.65 (danneggiato dal knockout) mostra
282 # un miglioramento progressivo (recupero) o resta stabilmente basso
283 # (danno strutturale permanente, non compensabile con piu' training) ---
284 FINESTRA = 10
285 n_finestre = N_RIPETIZIONI_TOT // (FINESTRA * len(MESSAGGI))
286
287 def valuta_finestra(k, indice_finestra):
288     tempi_k = [t0 for (kk, t0) in storico if kk == k]
289     inizio = indice_finestra * FINESTRA
290     fine = inizio + FINESTRA
291     lista = tempi_k[inizio:fine]
292     n_tot, posizioni = 0, []
293     for t0 in lista:
294         fin = t0 + ciclo_ms
295         m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
296         idx = spikes.i[m] - N_ring
297         n_tot += m.sum()
298         posizioni.extend(list(xi[idx]))
299     centro = None
300     if posizioni:
301         centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi) %
302                         1.0)
303     return n_tot, centro

```

```

303
304
305 traiettoria = {k: {'n': [], 'err': []} for k in range(len(MESSAGGI))}
306 for k, m in enumerate(MESSAGGI):
307     for fw in range(n_finestre):
308         n_tot, centro = valuta_finestra(k, fw)
309         err = dist_circ(centro - m) if centro is not None else None
310         traiettoria[k]['n'].append(n_tot)
311         traiettoria[k]['err'].append(err)
312
313 print("\n" + "*"70)
314 print(f"TRAIETTORIA DI RECUPERO (finestre di {FINESTRA} ripetizioni ciascuna, "
315       f"{n_finestre} finestre totali per messaggio)")
316 print("*"70)
317 for k, m in enumerate(MESSAGGI):
318     stringa = ", ".join(str(n) for n in traiettoria[k]['n'])
319     print(f"x={m}: spike per finestra: [{stringa}]")
320
321 fig2 = figure(figsize=(11, 4))
322 colori_msg = ['C0', 'C3']
323 for k, m in enumerate(MESSAGGI):
324     plot(range(n_finestre), traiettoria[k]['n'], 'o-', color=colori_msg[k],
325          label=f"x={m}" + (' (danneggiato dal knockout)' if m == 0.65 else ''))
326 xlabel(f'Finestra di training ({FINESTRA} ripetizioni ciascuna)')
327 ylabel('Spike di B nella finestra')
328 title('Traiettoria di recupero durante il training esteso (300 ripetizioni)')
329 legend(fontsize=9)
330 tight_layout()
331 savefig('dialogo_spiking_lesione_completa_mc_traiettoria.png', dpi=150)
332 print("\nGrafico traiettoria salvato in dialogo_spiking_lesione_completa_mc_traiettoria.png"
333       )
334
335 # confronto quantitativo: prima meta' del training esteso vs seconda meta'
336 meta = n_finestre // 2
337 for k, m in enumerate(MESSAGGI):
338     prima_meta = np.mean(traiettoria[k]['n'][:meta])
339     seconda_meta = np.mean(traiettoria[k]['n'][meta:])
340     print(f"x={m}: prima meta' training={prima_meta:.1f} spike/finestra, "
341           f"seconda meta'={seconda_meta:.1f} spike/finestra, "
342           f"variazione={'+' if seconda_meta > prima_meta else ''}{(seconda_meta-prima_meta)/max(
343             prima_meta, 0.01)*100:.0f}%")
344
345 print(f"MC_RIGA seed={SEED} x065_prima={np.mean(traiettoria[1]['n'][:meta]):.1f}
346       x065_seconda={np.mean(traiettoria[1]['n'][meta:]):.1f} x015_prima={np.mean(traiettoria
347       [0]['n'][:meta]):.1f} x015_seconda={np.mean(traiettoria[0]['n'][meta:]):.1f}")

```

5.31 Physical dose or biological dose? Chiba/NIRS's RBE model applied to the neuron map

Sections 5.22-5.30 always classified which neurons to silence based on the PHYSICAL dose deposited (MeV). Is this the right choice? In clinical hadrontherapy the answer is, in general, no: at equal physical dose, particles with different LET (Linear Energy Transfer) produce very different biological damage. This is exactly the problem NIRS (National Institute of Radiological Sciences, later QST) was the first in the world to tackle by building HIMAC in Chiba, the first clinical accelerator dedicated to carbon-12 (since 1994, over 15,000 patients treated): the Kanai et al. mixed-beam model, which weights dose by an RBE (Relative Biological Effectiveness) factor dependent on LET, with a well-established clinical reference point: RBE=3.0 at the so-called "neutron-equivalent point," LET=80 keV/um. CNAO adopts the same type of approach for its own carbon beams.

Implemented in Geant4 (SteppingAction/EventAction/RunAction): dose-averaged LET tracking per neuron, $\text{Sum}(dE_i^2/dx_i)/\text{Sum}(dE_i)$ (the same quantity used in the Kanai model), accumulated step by step alongside the already-existing physical dose. In analysis, a simplified RBE(LET) conversion is applied -- linear, ANCHORED to the NIRS reference point (RBE=3.0 at 80 keV/um), saturating at RBE_max=3.5 -- explicitly NOT the full clinical microdosimetric model (which would require cell-line-specific alpha/beta survival curves, data we do not have), but sufficient to answer the qualitative question: does the neuron ranking change when weighting by biological damage instead of physical dose alone?

Tested on two configurations at the Bragg peak (where LET is maximal, so where any RBE effect would show up most): carbon-12 120 MeV/u at 34.5mm (LET between 2.8 and 172.9 keV/um on the affected neurons, RBE between 1.03 and 3.50) and 70 MeV proton at 39.5mm (LET between 2.6 and 8.8 keV/um, RBE between 1.03 and 1.22 -- the whole range stays below the 80 keV/um reference point). The gap between the two particles is stark: mean RBE of the hardest-hit neurons ~3.3 for carbon against ~1.15 for the proton, a ~3x factor, consistent with the literature -- and it is precisely the reason RBE modeling matters far more for carbon than for protons in clinical practice.

But the ranking of the 10 hardest-hit neurons, WITHIN each configuration, changes very little: 9 of 10 neurons stay the same for both carbon (only neuron 58 drops out, replaced by 27) and proton (only 56 drops out, replaced by 40). The reason is mechanistically simple: LET depends mostly on depth (the point on the Bragg curve, i.e. the particle's residual range), not on lateral position within the ring -- all neurons hit at a given depth see a similar LET, so an RBE factor scales them nearly uniformly without reshuffling their relative order.

Conclusion, with methodological caution: the RBE refinement confirms the robustness of Sections 5.22-5.28's dose-response gradient (built on physical-dose rankings) rather than undermining it -- at least for comparisons WITHIN the same depth. The real RBE gap between particles (~3x) would instead become relevant for a comparison BETWEEN different scenarios (e.g. which physical configuration causes the greatest overall biological damage, not just which one damages which neurons) -- a distinct question, not yet tested here. Also worth flagging, a limitation that emerged during this test: without an explicit seed, Geant4 uses a different seed on every run -- the ranking of the 10 hardest-hit neurons in THIS run of carbon at the peak [3, 33, 46, 51, 58, 66, 69, 71, 80, 88] shares only one neuron with the ranking used in Section 5.27 [4, 20, 26, 28, 38, 51, 55, 64, 73, 97] for the exact same physical configuration -- a physics-side Monte Carlo variability that had not, until now, been checked with the same rigor already applied on the neuroscience side (multi-seed verification on every Brian2 experiment).

Source code: *SteppingAction.cc / EventAction.cc / RunAction.cc*
(dose-averaged LET tracking added in this section)

```

1 #include "SteppingAction.hh"
2 #include "EventAction.hh"
3
4 #include "G4Step.hh"
5 #include "G4StepPoint.hh"
6 #include "G4Track.hh"
7 #include "G4VProcess.hh"
8 #include "G4SystemOfUnits.hh"
9 #include "G4ParticleDefinition.hh"
10 #include "G4LogicalVolume.hh"
11 #include "G4VPhysicalVolume.hh"
12 #include "G4VTouchable.hh"
13 #include "G4TouchableHandle.hh"
14 #include "G4RunManager.hh"
15 #include "G4Event.hh"
16 #include <cstdlib>
17
18 SteppingAction::SteppingAction(EventAction* eventAction)
19 : G4UserSteppingAction(), fEventAction(eventAction)
20 {}
21
22 SteppingAction::~SteppingAction() {}
23
24 void SteppingAction::UserSteppingAction(const G4Step* step)
25 {
26     const G4StepPoint* preStep = step->GetPreStepPoint();
27     const G4VTouchable* touchable = preStep->GetTouchable();
28     const G4VPhysicalVolume* volume = touchable->GetVolume();
29     if (!volume || volume->GetName() != "Neuron") return;
30
31     const G4int neuronIndex = touchable->GetCopyNumber();
32
33     const G4double edep = step->GetTotalEnergyDeposit();
34     if (edep > 0.) fEventAction->AddEdep(neuronIndex, edep, step->GetStepLength());
35
36     // Diagnostica temporanea (sezione 5.36): stampa ogni step con
37     // deposito >0 nel neurone indicato da CNAO_DEBUG_NEURON, per
38     // identificare la particella/processo responsabile di un outlier
39     // di LET -- stesso pattern usato per il bug del geantino.
40     static const char* dbgEnv = std::getenv("CNAO_DEBUG_NEURON");
41     if (dbgEnv && neuronIndex == std::atoi(dbgEnv) && edep > 0.) {
42         const G4Track* track = step->GetTrack();
43         const G4VProcess* proc = step->GetPostStepPoint()->GetProcessDefinedStep();
44         G4cout << "[CNAO_DEBUG_NEURON] event=" << G4RunManager::GetRunManager()->GetCurrentEvent
45             (<< GetEventID()
46              << " particella=" << track->GetParticleDefinition()->GetParticleName()
47              << " parentID=" << track->GetParentID()
48              << " trackID=" << track->GetTrackID()
49              << " processo=" << (proc ? proc->GetProcessName() : "N/A")
50              << " edep_keV=" << edep/CLHEP::keV
51              << " stepLength_um=" << step->GetStepLength()/CLHEP::um
52              << " KE_MeV=" << track->GetKineticEnergy()/CLHEP::MeV
53              << G4endl;
54     }
55
56     // ParentID==0 identifies the primary proton/ion track itself
57     // (as opposed to any secondary electron, delta-ray, or nuclear
58     // fragment produced along the way).
59     const G4Track* track = step->GetTrack();
60     if (track->GetParentID() == 0) {
61         fEventAction->MarkPrimaryHit(neuronIndex);
62     }
63
64     // Sezione 5.39: registra l'energia cinetica di ogni elettrone
65     // secondario NATO dentro questo neurone (primo step del track,
66     // pre-step point ancora nel volume "Neuron") -- lo spettro
67     // necessario per valutare un accoppiamento con Geant4-DNA (valido
68     // solo fino a ~1 MeV), invece di assumerlo senza verificarlo.

```

```

68     if (track->GetParentID() != 0
69         && track->GetParticleDefinition()->GetParticleName() == "e-"
70         && track->GetCurrentStepNumber() == 1) {
71         fEventAction->AddSecondaryElectron(neuronIndex, preStep->GetKineticEnergy());
72     }
73 }

1  #include "EventAction.hh"
2  #include "RunAction.hh"
3  #include "G4SystemOfUnits.hh"
4  #include <cmath>
5
6  namespace {
7      // Sezione 5.43: 6 punti rappresentativi dello spettro reale (percentili
8      // 10/25/50/75/90/99), con il tasso DSB/primario validato per ciascuno
9      // (moleculardna, geant4dna_ssbdsb). Riusati qui, non ricalcolati.
10     constexpr G4int kNBins = 6;
11     constexpr G4double kBinEnergyKeV[kNBins] = {1.09, 1.28, 1.86, 3.43, 7.33, 28.82};
12     constexpr G4double kBinDSBRate[kNBins] = {0.0103, 0.0083, 0.0137, 0.0243, 0.0430, 0.0163};
13
14     G4double DSBRateForEnergy(G4double keV)
15     {
16         G4int best = 0;
17         G4double bestDiff = std::abs(keV - kBinEnergyKeV[0]);
18         for (G4int i = 1; i < kNBins; ++i) {
19             G4double diff = std::abs(keV - kBinEnergyKeV[i]);
20             if (diff < bestDiff) { bestDiff = diff; best = i; }
21         }
22         return kBinDSBRate[best];
23     }
24 } // namespace
25
26 EventAction::EventAction(RunAction* runAction)
27 : G4UserEventAction(), fRunAction(runAction)
28 {}
29
30 EventAction::~EventAction() {}
31
32 void EventAction::BeginOfEventAction(const G4Event*)
33 {
34     fEdep.fill(0.);
35     fEdepSqOverDx.fill(0.);
36     fPrimaryHit.fill(false);
37     fSecondaryElectronEnergies.clear();
38     fSecondaryElectronCount.fill(0);
39     fEstimatedDSB.fill(0.);
40 }
41
42 void EventAction::AddEdep(G4int neuronIndex, G4double edep, G4double stepLength)
43 {
44     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
45     fEdep[neuronIndex] += edep;
46     if (stepLength > 0.) fEdepSqOverDx[neuronIndex] += edep * edep / stepLength;
47 }
48
49 void EventAction::MarkPrimaryHit(G4int neuronIndex)
50 {
51     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
52     fPrimaryHit[neuronIndex] = true;
53 }
54
55 void EventAction::AddSecondaryElectron(G4int neuronIndex, G4double kineticEnergy)
56 {
57     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
58     fSecondaryElectronEnergies.push_back(kineticEnergy);
59     fSecondaryElectronCount[neuronIndex] += 1;
60     fEstimatedDSB[neuronIndex] += DSBRateForEnergy(kineticEnergy / CLHEP::keV);

```

```

61 }
62
63 void EventAction::EndOfEventAction(const G4Event*)
64 {
65     fRunAction->RecordEvent(fEdep, fEdepSqOverDx, fPrimaryHit, fSecondaryElectronEnergies,
66                             fSecondaryElectronCount, fEstimatedDSB);
67 }

```

```

1  #include "RunAction.hh"
2
3  #include "G4Run.hh"
4  #include "G4SystemOfUnits.hh"
5  #include <sstream>
6
7  RunAction::RunAction() : G4UserRunAction() {}
8
9  RunAction::~RunAction() {}
10
11 void RunAction::BeginOfRunAction(const G4Run*)
12 {
13     fSumEdep.fill(0.);
14     fSumEdepSqOverDx.fill(0.);
15     fSumPrimaryHits.fill(0);
16     fNEvents = 0;
17     fSecondaryElectronSpectrum.clear();
18     fSumSecondaryElectronCount.fill(0);
19     fSumEstimatedDSB.fill(0.);
20 }
21
22 void RunAction::RecordEvent(const std::array<G4double, N_NEURONS>& edep,
23                             const std::array<G4double, N_NEURONS>& edepSqOverDx,
24                             const std::array<G4bool, N_NEURONS>& primaryHit,
25                             const std::vector<G4double>& secondaryElectronEnergies,
26                             const std::array<G4long, N_NEURONS>& secondaryElectronCount,
27                             const std::array<G4double, N_NEURONS>& estimatedDSB)
28 {
29     for (G4int i = 0; i < N_NEURONS; ++i) {
30         fSumEdep[i] += edep[i];
31         fSumEdepSqOverDx[i] += edepSqOverDx[i];
32         if (primaryHit[i]) fSumPrimaryHits[i] += 1;
33         fSumSecondaryElectronCount[i] += secondaryElectronCount[i];
34         fSumEstimatedDSB[i] += estimatedDSB[i];
35     }
36     fNEvents += 1;
37     for (G4double e : secondaryElectronEnergies) {
38         fSecondaryElectronSpectrum.push_back(e);
39     }
40 }
41
42 void RunAction::EndOfRunAction(const G4Run* run)
43 {
44     std::ostringstream fname;
45     fname << "neuron_dose_run" << run->GetRunID() << ".csv";
46     std::ofstream out(fname.str());
47     out << "# n_events=" << fNEvents << "\n";
48     out << "neuron_index,xi,sum_edep_MeV,mean_edep_keV_per_event,"
49           "primary_hit_count,primary_hit_fraction,dose_averaged_LET_keV_per_um,"
50           "secondary_electron_count,estimated_DSB\n";
51     for (G4int i = 0; i < N_NEURONS; ++i) {
52         G4double xi = static_cast<G4double>(i) / N_NEURONS;
53         G4double sumEdepMeV = fSumEdep[i] / CLHEP::MeV;
54         G4double meanEdepKeV = (fNEvents > 0)
55             ? (fSumEdep[i] / CLHEP::keV) / static_cast<G4double>(fNEvents) : 0.;
56         G4double hitFraction = (fNEvents > 0)
57             ? static_cast<G4double>(fSumPrimaryHits[i]) / static_cast<G4double>(fNEvents) : 0.;
58         // LET dose-mediato = Sum(dE_i^2/dx_i) / Sum(dE_i) (Kanai et al. 1999,
59         // lo stesso concetto usato a NIRS/HIMAC per il modello RBE clinico:

```

```

60 // pesa ogni step per il proprio deposito di energia).
61 G4double doseAvgLET = (fSumEdep[i] > 0.)
62 ? (fSumEdepSqOverDx[i] / fSumEdep[i]) / (CLHEP::keV / CLHEP::um) : 0.;
63 // Sezione 5.44: danno DSB atteso per neurone (stima, non una nuova
64 // simulazione -- riusa i tassi validati della 5.43 per bin di energia).
65 out << i << ", " << xi << ", " << sumEdepMeV << ", " << meanEdepKeV << ", "
66 << fSumPrimaryHits[i] << ", " << hitFraction << ", " << doseAvgLET << ", "
67 << fSumSecondaryElectronCount[i] << ", " << fSumEstimatedDSB[i] << "\n";
68 }
69 out.close();
70
71 // Spettro degli elettroni secondari nati dentro un neurone (sezione
72 // 5.39): un'energia cinetica (keV) per riga, cosi' come creati --
73 // input per valutare l'accoppiamento con Geant4-DNA.
74 std::ostreamstream specName;
75 specName << "secondary_electron_spectrum_run" << run->GetRunID() << ".csv";
76 std::ofstream specOut(specName.str());
77 specOut << "# n_events=" << fNEvents << " n_secondary_electrons="
78 << fSecondaryElectronSpectrum.size() << "\n";
79 specOut << "kinetic_energy_keV\n";
80 for (G4double e : fSecondaryElectronSpectrum) {
81     specOut << (e / CLHEP::keV) << "\n";
82 }
83 specOut.close();
84 }

```

5.32 Limitation discovered: the per-neuron dose map is not reproducible run to run

A direct follow-on to 5.31, where it was noticed that two runs of the SAME physical configuration (carbon-12 120 MeV/u at the peak) shared only one neuron out of ten in the highest-dose ranking. An explicit seed was added to `cnaoRingImpact` (previously generated from the system clock, hence different on every execution with no way to reproduce it), and the exact same configuration was repeated over 4 independent seeds (111, 222, 333, 444), 500,000 events each.

Result: the average pairwise overlap between the top-10 rankings of the 4 seeds is 1.0 out of 10 -- essentially pure noise. No neuron appears in the top-10 of all 4 seeds; only one neuron (73) appears in 3 of 4. The TOTAL dose deposited across the whole ring, by contrast, is stable (192-226 MeV, roughly $\pm 15\%$) -- it is only its distribution among the 100 neurons that is noise-dominated, not the overall energy.

Cause, clear from a probability count: each neuron has a radius of 10 micrometers, sampled over a disk of 5mm radius -- the ratio between the two areas is about 4×10^{-6} . Over 500,000 primaries, the expected number of direct hits per neuron is of order 1-2 (consistent with the `primary_hit_count` of 4-5 already observed in previous sections). With so few counts per individual neuron, telling apart "neuron A took a bit more dose than neuron B" is statistically equivalent to a coin flip, even though the total summed over 100 neurons is a reasonably stable quantity (the sum of many noisy variables is less noisy than the individual variable).

Honest implication for Sections 5.22-5.30: the specific 10-neuron sets used for each knockout came from A SINGLE Monte Carlo realization of the physics, not from a statistically robust dose map -- a limitation that should have been checked earlier, in the same spirit with which Section 5.25 already discovered and corrected the too-narrow sampling-disk artifact. Describing those neurons as "the ones hit hardest by the CNAO beam" is an overinterpretation: they are more accurately "the ones hit hardest in one particular simulation of that beam."

What remains valid regardless: the causal MECHANISM (removing outgoing synapses from a set of neurons produces interference that scales with how many of those neurons belong to a message's encoding zone) was independently verified across dozens of Brian2 seeds in every single experiment (Sections 5.23, 5.24, 5.26, 5.28, 5.29, 5.30) -- that part of the chain does not depend on Geant4-side Monte Carlo noise. The four-point dose-response gradient (5.28) remains a genuine causal relationship internal to the neural model; it is only the claim of a precise, single-neuron-level link to one specific physical simulation of the CNAO beam that needs to be scaled back.

An immediate correction, already computed here: the dose AVERAGED over the 4 seeds gives a much more stable estimate of the real ranking -- top-10 = [0, 5, 7, 29, 37, 47, 48, 70, 73, 82] -- but even this 4-realization average shares only 2-4 neurons out of 10 with each individual seed, confirming that many more repetitions (or a 2-3 order-of-magnitude increase in events per run, computationally expensive with QBBC's full hadronic physics) would be needed for a truly solid per-neuron dose map. Left as explicit future work: rebuild the knockout sets from the dose AVERAGED over multiple seeds instead of a single run, and check whether the 5.22-5.28 gradient holds when reconstructed on more solid statistical grounds.

Source code: *cnaoRingImpact.cc* (explicit seed added in this section)

```

1 // cnaoRingImpact -- physical dose/hit map of a CNAO-style clinical
2 // ion beam (proton or fully-stripped carbon-12) across a ring of
3 // 100 tissue-equivalent "neuron" spheres, the physical counterpart
4 // of the N_E=100 excitatory ring in the spiking ring attractor of
5 // neuroni-progetto (dialogo_spiking*.py, sezioni 5.6-5.20).
6 //
7 // Physics: QBBC (Geant4's reference physics list for clinical
8 // proton/ion energies -- NOT Geant4-DNA, which is only valid up to
9 // ~1 MeV and has no validated ion transport model at these
10 // energies; see PhysicsList.hh and README.md).
11 //
12 // Usage:
13 // ./cnaoRingImpact [macro] [particleType] [energy_MeV] [ringDepth_mm] [seed]
14 //
15 // macro path to a .mac file (default: macros/run.mac)
16 // particleType 0 = proton, 1 = carbon-12 (default 0)
17 // energy_MeV KINETIC energy of the primary, TOTAL (not per
18 // nucleon) for carbon-12 (default: 150 for proton,
19 // 2400 for carbon-12, i.e. 200 MeV/u -- both mid-
20 // range clinical CNAO values)
21 // ringDepth_mm distance from the beam entry face to the ring
22 // plane (default: 2mm, the shallow/plateau-only
23 // geometry of sections 5.21-5.24). Use braggProfile/

```



```

24 // to find the Bragg peak depth for a given energy
25 // and pass it here to test the ring AT the peak --
26 // the clinically relevant scenario.
27 // seed explicit random seed (default: time-based, i.e. a
28 // DIFFERENT Monte Carlo realization on every run with
29 // no 5th argument -- pass an explicit seed for
30 // reproducible runs or for multi-seed MC verification
31 // of the dose map, the same discipline already used
32 // on the Brian2 side, sezione 5.31).
33 //
34 // Example, 150 MeV protons, ring in the shallow plateau (original):
35 // ./cnaoRingImpact macros/run.mac 0 150 2
36 // Example, 70 MeV protons, ring AT the Bragg peak (39.5mm, found with
37 // braggProfile):
38 // ./cnaoRingImpact macros/run.mac 0 70 39.5
39 // Example, same config with an explicit seed for MC verification:
40 // ./cnaoRingImpact macros/run.mac 0 70 39.5 12345
41
42 #include "G4RunManagerFactory.hh"
43 #include "G4UIManager.hh"
44 #include "G4SystemOfUnits.hh"
45 #include "Randomize.hh"
46 #include <ctime>
47 #include <string>
48
49 #include "DetectorConstruction.hh"
50 #include "PhysicsList.hh"
51 #include "ActionInitialization.hh"
52
53 int main(int argc, char** argv)
54 {
55     G4String macro = (argc > 1) ? argv[1] : "macros/run.mac";
56     G4int particleType = (argc > 2) ? std::stoi(argv[2]) : 0;
57     G4double energyMeV = (argc > 3) ? std::stod(argv[3])
58                             : (particleType == 1 ? 2400.0 : 150.0);
59     G4double ringDepthMM = (argc > 4) ? std::stod(argv[4]) : 2.0;
60     G4long seed = (argc > 5) ? std::stol(argv[5])
61                         : static_cast<G4long>(std::time(nullptr));
62     CLHEP::HepRandom::setTheSeed(seed);
63     G4cout << "Seed usato: " << seed << G4endl;
64
65     auto* runManager = G4RunManagerFactory::CreateRunManager(G4RunManagerType::Serial);
66
67     auto* detector = new DetectorConstruction(ringDepthMM * CLHEP::mm);
68     runManager->SetUserInitialization(detector);
69     runManager->SetUserInitialization(new PhysicsList());
70     runManager->SetUserInitialization(new ActionInitialization(particleType, detector->
71         GetZStart()));
72
73     runManager->Initialize();
74
75     G4UIManager* UImanager = G4UIManager::GetUIpointer();
76     UImanager->ApplyCommand("/gun/energy " + std::to_string(energyMeV) + " MeV");
77     UImanager->ApplyCommand("/control/execute " + macro);
78
79     delete runManager;
80     return 0;
81 }

```

5.33 The gradient on solid statistical grounds: a minimum threshold emerges

Closes the loop opened by 5.32. The knockout set was rebuilt not from a single noisy Geant4 run, but from the dose AVERAGED over the 4 independent seeds

(111, 222, 333, 444) of carbon-12 at the Bragg peak: top-10 = [0, 5, 7, 29, 37, 47, 48, 70, 73, 82]. Overlap with the encoding zones: 0 neurons in $x=0.15$, 1 neuron (70) in $x=0.65$ -- a weak but real imbalance, which Section 5.28's gradient would predict as a modest but never-null effect.

Verification over 8 seeds: average ratio $x=0.15/x=0.65 = 1.29x$ (mean final spikes 42.75 vs. 33.25), $x=0.15$ greater than $x=0.65$ in 6 of 8 seeds. At first glance a weak result but in the expected direction -- however, the decisive comparison is with Section 5.24's negative control (random set, ZERO overlap with either zone): ratio 1.39x, 6/8 seeds. The two results are statistically indistinguishable: an overlap of just ONE neuron does not produce an effect above the network's baseline noise.

The dose-response gradient, now on solid statistical grounds instead of single noisy Monte Carlo realizations, refines as follows: overlap 4-vs-0 -> 7.07x, 8/8 (5.28, the strongest); 2-vs-0 -> 4.7x, 8/8 (5.23); 3-vs-1 (net imbalance 2) -> 1.81x, 6/8 (5.26); 1-vs-0 (net imbalance 1, 4-seed AVERAGED dose, this section) -> 1.29x, 6/8, indistinguishable from the negative control 0-vs-0 -> 1.39x, 6/8 (5.24). A threshold emerges: an imbalance of AT LEAST 2 neurons is needed for the effect to exceed the network's baseline noise -- with a difference of just one neuron, the signal is lost in the same variability observed even without any targeted knockout.

Interpretation: Section 5.32's discovery (the noisy dose map) does not invalidate the gradient, but makes it more precise -- and shows why some of the weaker results observed earlier (in particular the near-null 1.06x/4-8 of Section 5.27, overlap ~1 against ~1 from a single run) were credible: with imbalances this small, the physical signal and the network's statistical noise become comparable in magnitude, and the question "is this effect real or is it noise" always requires an explicit comparison with a negative control, not just the direction of the ratio.

Source code: `dialogo_spiking_cnao_knockout_media4seed_mc.py`

```

1  """
2  Chiude il limite scoperto nella sezione 5.32: la mappa di dose per-
3  neurone di un SINGOLO run Geant4 e' dominata dal rumore di Poisson
4  (sovrapposizione 1.0/10 tra classifiche top-10 di 4 seed indipendenti
5  della stessa configurazione). Qui il knockout non viene piu' scelto da
6  un singolo run, ma dalla dose MEDIATA sui 4 seed (111, 222, 333, 444)
7  del carbonio-12 120 MeV/u al picco di Bragg -- la quantita'
8  statisticamente piu' solida disponibile finora per questo scenario.
9
10 Overlap con le zone di codifica (gruppo_stim, sezione 5.7): 0 neuroni
11 in x=0.15, 1 neurone (il 70) in x=0.65 -- uno sbilanciamento debole ma
12 reale. Secondo il gradiente dose-risposta gia' stabilito (5.28, dove
13 lo sbilanciamento correla con la forza dell'effetto), un overlap 1
14 contro 0 dovrebbe predire un effetto modesto ma nella direzione attesa
15 (x=0.65 piu' colpito di x=0.15) -- piu' debole del 2 contro 0 di 5.23,
16 ma non nullo come il ~1 contro ~1 di 5.27.
17 """
18
19 KNOCKOUT = sorted([0, 5, 7, 29, 37, 47, 48, 70, 73, 82]) # top-10 per
20                                                         # DOSE MEDIA sui 4 seed
21                                                         # 111/222/333/444

```

```

22                                     # (carbonio-12 120 MeV/u,
23                                     # picco 34.5mm -- sezione 5.32)
24
25 """
26 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
27 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
28 usate nel primo test di interferenza del modello a rate (dialogo_multi_
29 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
30
31 Nota di partenza importante, diversa dal modello a rate: li' i pesi
32 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
33 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
34 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
35 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
36 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
37 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
38 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
39 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
40 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
41 causava nel modello a rate potrebbe semplicemente non esistere qui.
42 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
43 concluso che il canale B->A non trasmette causalmente in questo sistema,
44 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
45 """
46
47 import sys
48 from brian2 import *
49
50 SEED = int(sys.argv[1])
51 start_scope()
52 seed(SEED)
53
54 N_E = 100
55 N_I = 25
56 N_ring = N_E + N_I
57
58 a = 0.02/ms
59 b = 0.2/ms
60 c = -65*mV
61 d = 6*mV/ms
62 tau_nmda = 100*ms
63
64 eqs = '''
65 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
66 du/dt = a*(b*v - u) : volt/second
67 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
68 I : volt/second
69 x : 1
70 '''
71
72 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
73                      method='euler')
74 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
75 neurons.u = b * neurons.v[:]
76 neurons.I = 0*mV/ms
77 neurons.g_nmda = 0*mV/ms
78
79 A_E = neurons[:N_E]
80 A_I = neurons[N_E:N_ring]
81 B_E = neurons[N_ring:N_ring+N_E]
82 B_I = neurons[N_ring+N_E:]
83
84 xi = np.arange(N_E) / N_E
85 A_E.x = xi
86 B_E.x = xi
87
88 background = PoissonGroup(2*N_ring, rates=400*Hz)
89 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')

```

```

90 bg_syn.connect(j='i')
91
92 sigma_exc = 0.05
93 CUTOFF_LOCALE = 3 * sigma_exc
94 w_ee_nmda = 0.325*mV/ms
95 p_conn = 0.15
96 w_exc = 1.0*mV
97 w_inh = 10.0*mV
98
99
100 def dist_circ(dx):
101     dx = abs(dx)
102     return np.minimum(dx, 1 - dx)
103
104
105 def costruisci_anello(E, I_pop, seed, knockout=()):
106     dmat = dist_circ(xi[:, None] - xi[None, :])
107     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
108     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
109     np.fill_diagonal(prob_ee, 0)
110     rng = np.random.RandomState(seed)
111     mask_ee = rng.rand(N_E, N_E) < prob_ee
112     i_idx, j_idx = np.nonzero(mask_ee)
113
114     ko = np.asarray(list(knockout), dtype=int)
115     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
116     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
117     # piu' influenzare nessuno (l'unica cosa che conta per una rete
118     # a spike: se non spara mai in modo efficace a valle, e' come
119     # se non ci fosse)
120     tieni_ee = ~np.isin(i_idx, ko)
121     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
122
123     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
124     syn_ee.connect(i=i_idx, j=j_idx)
125     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
126     # Costruita esplicitamente (invece di .connect(p=...)) per poter
127     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
128     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
129     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
130     i_ei, j_ei = np.nonzero(mask_ei)
131     tieni_ei = ~np.isin(i_ei, ko)
132     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
133     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
134     syn_ie.connect(p=p_conn)
135     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
136     syn_ii.connect(condition='i!=j', p=p_conn)
137     return syn_ee, syn_ei, syn_ie, syn_ii
138
139
140 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
141 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
142 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
143       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
144
145 tau_stdp = 20*ms
146 A_ltp = 0.15*mV
147 A_ltd = 0.15*mV
148 w_max = 5.0*mV
149 w_min = 0*mV
150 w_init = 1.5*mV
151 sigma_inter = 0.08
152 p_inter_picco = 0.5
153
154 stdp_model = '''
155 w : volt
156 dapre/dt = -apre/tau_stdp : volt (event-driven)
157 dapost/dt = -apost/tau_stdp : volt (event-driven)

```

```

158 '''
159 stdp_on_pre = '''
160 v_post += w
161 apre += A_ltp
162 w = clip(w+apost, w_min, w_max)
163 '''
164 stdp_on_post = '''
165 apost -= A_ltd
166 w = clip(w+apre, w_min, w_max)
167 '''
168
169 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
170 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
171 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
172 rng_inter = np.random.RandomState(3000+SEED)
173 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
174 i_inter, j_inter = np.nonzero(mask_inter)
175 ko_arr = np.asarray(KNOCKOUT, dtype=int)
176 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
177                                     # trasmettere neanche a B
178 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
179 syn_ab.connect(i=i_inter, j=j_inter)
180 syn_ab.w = w_init
181 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
182       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
183
184 spikes = SpikeMonitor(neurons)
185
186 MESSAGGI = [0.15, 0.65]
187 larghezza_stimolo = 0.05
188 gruppi_stim = []
189 for m in MESSAGGI:
190     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
191     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
192     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
193
194 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
195 T_STIM = 60*ms
196 T_PAUSA = 340*ms
197
198 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
199       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
200
201 storico = [] # (indice_messaggio, tempo_inizio_ms)
202 t_corrente = 0*ms
203 for rep in range(N_RIPETIZIONI_TOT):
204     k = rep % len(MESSAGGI)
205     storico.append((k, t_corrente/ms))
206     A_E.I[gruppi_stim[k]] = 15*mV/ms
207     run(T_STIM)
208     A_E.I[:] = 0*mV/ms
209     run(T_PAUSA)
210     t_corrente += T_STIM + T_PAUSA
211     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
212         print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
213               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
214
215 print("\nTraining completato.")
216
217 fig, axes = subplots(2, 1, figsize=(11, 7))
218 subplot(2, 1, 1)
219 mask_a = spikes.i < N_E
220 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
221 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
222 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
223 xlabel('Tempo (ms)')
224 ylabel('Indice neurone')

```

```

225 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
226       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
227 legend(loc='upper right', fontsize=8, markerscale=6)
228
229 subplot(2, 1, 2)
230 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
231 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
232 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
233 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label
      ='A (E)')
234 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
235 xlabel('Tempo (ms)')
236 ylabel('Indice neurone')
237 title('Dettaglio: ultime ripetizioni (fine training)')
238 legend(loc='upper right', fontsize=8, markerscale=4)
239
240 tight_layout()
241 savefig('dialogo_spiking_cnao_knockout_media4seed.png', dpi=150)
242 print("\nGrafico salvato in dialogo_spiking_cnao_knockout_media4seed.png")
243
244 ciclo_ms = (T_STIM+T_PAUSA)/ms
245
246
247 def valuta_messaggio(k, n_eval):
248     tempi = [t0 for (kk, t0) in storico if kk == k]
249     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
250
251     def conta(lista):
252         n_tot, posizioni = 0, []
253         for t0 in lista:
254             fin = t0 + ciclo_ms
255             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
256             idx = spikes.i[m] - N_ring
257             n_tot += m.sum()
258             posizioni.extend(list(xi[idx]))
259         centro = None
260         if posizioni:
261             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
262                           % 1.0)
263         return n_tot, centro
264     return conta(primi), conta(ultimi)
265
266 print("\n" + "*"70)
267 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
268       "correttamente e si rafforza per ENTRAMBI i messaggi?")
269 print("=*70)
270 for k, m in enumerate(MESSAGGI):
271     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
272     err_i = dist_circ(c_i - m) if c_i is not None else None
273     err_f = dist_circ(c_f - m) if c_f is not None else None
274     print(f" Messaggio x={m}:")
275     print(f" prime 5 ripetizioni: {n_i} spike, centro={({N/A' if c_i is None else f'{c_i:.3f}')}}, "
276           f"errore={({N/A' if err_i is None else f'{err_i:.3f}')}")
277     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({N/A' if c_f is None else f'{c_f:.3f}')}}, "
278           f"errore={({N/A' if err_f is None else f'{err_f:.3f}')}")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
281
282 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284 righe = []
285 for k, m in enumerate(MESSAGGI):
286     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
287     err_f = dist_circ(c_f - m) if c_f is not None else 1.0

```

```

287     righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
288     print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.34 The real Kase/NIRS MKM model, validated against published data

Section 5.31 used an ad hoc linear RBE(LET) conversion, explicitly stated as NOT the real clinical model. Here it is replaced with the saturation-corrected MKM (Microdosimetric Kinetic Model) of Kase/Hawkins, in exactly the form used clinically at NIRS/HIMAC (Chiba) and described in Kase et al. 2011 (J Radiat Res 52:59-68) -- the same paper describing the TEPC (tissue-equivalent proportional counter) measurement routinely used at HIMAC for biological-dose quality assurance.

Equations implemented (eq. 7-10 of Kase 2011): survival $S = \exp(-\alpha \cdot D - \beta \cdot D^2)$; $\alpha = \alpha_0 + \beta \cdot z^*$, with z^* the specific energy per event obtained from the dose-mean lineal energy corrected for saturation y^* , via the standard ICRU conversion for a spherical site; y^* from the dose-mean lineal energy y_D through the saturation correction (parameter $y_0 = 150$ keV/um, which accounts for the "overkill" effect at high LET); RBE10 from the quadratic formula inverted at 10% survival. HSG cell parameters (the same used in NIRS clinical planning): $\alpha_0 = 0.13$ Gy⁻¹, $\beta = 0.05$ Gy⁻², $rd = 0.42$ um (subcellular domain radius), $D_{10R} = 5.0$ Gy (X-ray reference dose).

Two approximations stated relative to the full clinical model: (1) the dose-mean lineal energy y_D , which would require a spectrum measured with a TEPC or a true site-level microdosimetric simulation, is approximated with the dose-averaged LET already computed by Geant4 (Section 5.31); (2) the saturation correction is computed in the "single-value spectrum" limit (a Dirac delta on y_D) instead of integrating a full spectrum $f(y)$ -- a closed form, not the exact integral.

Validation, before trusting the model: applied to the y^* values ACTUALLY MEASURED and published by Kase et al. 2011 for a 290 MeV/u carbon-12 beam (Table 1, 6cm SOBP), the model reproduces their reported RBE10 values (Fig. 4) to within 10-15%: $y^* = 58.7$ keV/um -> computed RBE10 2.23 (expected ~2.0-2.3 from the plot); $y^* = 14.6$ keV/um -> computed RBE10 1.19 (expected ~1.3). At the NIRS clinical reference point (LET_d = 80 keV/um, where clinical RBE is by definition 3.0), the model gives RBE10 = 2.67 -- an 11% gap, explainable by the two approximations stated above, not a model or implementation error.

Applied to the same two configurations from Section 5.31 (carbon-12 and proton at their respective Bragg peaks): RBE10 on the hit neurons ranges from 0.94 to 3.44 for carbon (a physically sensible range, consistent with real clinical RBE curves rising up to ~3-3.5 before the overkill effect at very high LET) against 0.94-1.07 for the proton (essentially RBE=1, as clinically expected for a low-

LET beam). The ranking of the 10 highest-dose neurons still changes very little relative to physical dose alone (9/10 overlap for both particles, identical to Section 5.31's result with the crude linear model) -- Section 5.31's conclusion (the 5.22-5.28 dose-response gradient is robust to RBE refinement within the same depth) is thus confirmed with the real clinical model too; it was not an artifact of the linear approximation.

Source code: `geant4_cnao_ring/analysis/mkm_rbe.py`

```

1  """
2  Modello MKM (Microdosimetric Kinetic Model) saturazione-corretto di
3  Kase/Hawkins, nella forma usata clinicamente a NIRS/HIMAC (Chiba) per
4  il carbonio-12 -- sostituisce l'approssimazione lineare ad hoc della
5  sezione 5.31 con il modello reale, con parametri e formule prese da:
6
7      Kase Y, Kanai T, Matsumoto Y, et al. (2006) "Microdosimetric
8      measurements and estimation of human cell survival for heavy-ion
9      beams." Radiat Res 166:629-638.
10     Kase Y, Kanai T, Sakama M, et al. (2011) "Microdosimetric Approach
11     to NIRS-defined Biological Dose Measurement for Carbon-ion Treatment
12     Beam." J Radiat Res 52:59-68 -- equazioni (7)-(10), parametri HSG
13     (alpha0=0.13 Gy^-1, beta=0.05 Gy^-2, rd=0.42um, y0=150 keV/um,
14     D10R=5.0 Gy).
15
16     Due approssimazioni dichiarate rispetto al modello clinico completo:
17     1. y_D (dose-mean lineal energy, richiederebbe uno spettro TEPC
18        misurato o una vera simulazione microdosimetrica di sito) e'
19        approssimato con il LET dose-mediato gia' calcolato da Geant4
20        (sezione 5.31) -- valido come proxy di primo ordine, non equivalente.
21     2. y* e' calcolato nel limite "spettro a valore singolo" (delta di
22        Dirac su y_D) dell'equazione (9) di Kase 2011, non integrando uno
23        spettro f(y) completo -- riduce l'integrale a una forma chiusa:
24        y* = y0^2/y_D * [1 - exp(-(y_D/y0)^2)]
25
26     Validato contro i valori PUBBLICATI in Kase et al. 2011 (Tabella 1 /
27     Fig. 4, fascio carbonio-12 290 MeV/u, SOBP 6cm): usando i loro y*
28     misurati come proxy di y_D in questo modello si riottengono RBE10
29     entro il 10-15% dai valori riportati (es. y*=58.7 keV/um -> RBE10
30     calcolato 2.23, atteso ~2.0-2.3; punto di riferimento clinico NIRS
31     LET_d=80 keV/um -> RBE10 calcolato 2.67, riferimento clinico 3.0) --
32     uno scarto piccolo e spiegabile dalle due approssimazioni sopra, non
33     un errore del modello.
34     """
35
36     import math
37
38     ALPHA0 = 0.13 # Gy^-1, HSG cells, Kase 2006/2011
39     BETA = 0.05 # Gy^-2, HSG cells (assunto indipendente da LET)
40     RD = 0.42 # um, raggio del dominio subcellulare
41     Y0 = 150.0 # keV/um, parametro di saturazione
42     D10R = 5.0 # Gy, dose di riferimento raggi X 200kVp per 10% sopravvivenza HSG
43     _D_DOMAIN = 2 * RD # um, diametro del dominio (sito sferico)
44
45
46     def y_star(y_d):
47         """Dose-mean lineal energy corretta per saturazione (overkill),
48         limite a spettro a valore singolo dell'eq. (9) di Kase 2011."""
49         if y_d <= 0:
50             return 0.0
51         return (Y0**2 / y_d) * (1 - math.exp(-(y_d / Y0)**2))
52
53
54     def z_star(ystar):
55         """Specific energy per evento (Gy), conversione ICRU standard per
56         un sito sferico di diametro _D_DOMAIN."""
57         return 0.204 * ystar / _D_DOMAIN**2

```



```

58
59
60 def alpha_mkm(y_d):
61     """alpha(y_D) = alpha0 + beta*z*, eq. (7)-(8) di Kase 2011."""
62     return ALPHA0 + BETA * z_star(y_star(y_d))
63
64
65 def rbe10(y_d):
66     """RBE al 10% di sopravvivenza, eq. (10) di Kase 2011, usando LET
67     dose-mediato (y_d, keV/um) come proxy della dose-mean lineal energy."""
68     a = alpha_mkm(y_d)
69     disc = a**2 - 4 * BETA * math.log(0.1)
70     return (2 * BETA * D10R) / (math.sqrt(disc) - a)
71
72
73 if __name__ == '__main__':
74     # Validazione contro Kase et al. 2011, Tabella 1 / Fig. 4
75     print("Validazione contro valori pubblicati (Kase et al. 2011):")
76     for y_pub, atteso in [(14.6, "-1.3"), (28.0, "-1.5-1.6"), (58.7, "-2.0-2.3")]:
77         print(f" y*={y_pub} keV/um -> RBE10 calcolato={rbe10(y_pub):.2f} (atteso {atteso})")
78     print(f"\nPunto di riferimento clinico NIRS (LET_d=80 keV/um, RBE atteso=3.0):")
79     print(f" RBE10 calcolato = {rbe10(80.0):.2f}")

```

5.35 The MKM model across all six CNAO clinical configurations

Section 5.34 had validated the Kase/NIRS MKM model on only two configurations (carbon-12 120 MeV/u and proton 70 MeV, both at the Bragg peak). Here the picture is completed: the four missing configurations were rerun with LET tracking (protons 150/225 MeV, carbon-12 260/400 MeV/u, all at the true Bragg peak, explicit seed 42, 500,000 events), to have the MKM model across the entire clinical CNAO range tested in this project.

Configurazioni	Depth (mm)	Mean LET (keV/ μ m)	Max LET	Mean RBE10	Max RBE10	Phys./bio. overlap
Proton 70 MeV	39.5	4.4	8.8	0.98	1.07	9/10
Proton 150 MeV	156.5	4.1	25.9	0.97	1.46	10/10
Proton 225 MeV	315.5	3.1	8.0	0.95	1.05	10/10
Carbon-12 120 MeV/u	34.5	86.6	172.9	2.75	3.44	9/10
Carbon-12 260 MeV/u	135.5	83.3	181.3	2.60	3.44	9/10
Carbon-12 400 MeV/u	275.5	56.9	229.8	2.02	3.44	10/10

Generalized confirmation: the overlap between the ranking of the 10 highest-physical-dose neurons and the 10 highest-biological-dose (RBE10-weighted) ones stays at 9-10 out of 10 in ALL six configurations, not just the two already tested -- the RBE refinement never substantially changes which neurons come out hardest hit within a given depth, across the entire clinical CNAO range.

Two non-trivial physical observations, both consistent with the literature: (1) the maximum RBE10 for carbon-12 stabilizes around 3.44 across all three energies, even though the maximum LET grows from 172.9 to 229.8 keV/um going from 120 to 400 MeV/u -- the signature of the "overkill" effect already built into the Kase model (beyond a certain LET, further damaging an already-doomed cell does not increase lethality, and the saturation correction y^* begins to DECREASE for y_D well above $y_0=150$ keV/um); (2) the 150 MeV proton shows an anomalous maximum LET (25.9 keV/um, nearly 3 times the other two proton cases) and an RBE10 up to 1.46, well above the 1.1 typically assumed constant clinically for protons -- plausibly due to recoils or secondary fragments from elastic nuclear interactions, a phenomenon tied to the real, still-open debate over the validity of a constant proton RBE (recent literature: Parisi et al. 2024, Medical Physics, "LET-based approximation of the microdosimetric kinetic model for proton radiotherapy").

Data generated with geant4_cnao_ring/analysis/mkm_rbe.py on the dose/LET maps of the six configurations in build/results_let/

5.36 Verified, not hypothesized: what causes the 150 MeV proton's anomalous RBE peak

Section 5.35 had found, for the 150 MeV proton, one neuron (index 48) with dose-averaged LET much higher than the others (25.9 keV/um against ~4-8 typical), a dose of 0.41 MeV from a single primary crossing -- too much energy for direct dE/dx alone from a 150 MeV proton through a 20um sphere (expected ~0.01 MeV). A reasonable but unverified first hypothesis: a secondary nuclear fragment or recoil. Instead of leaving it as a hypothesis, a per-step diagnostic (the same technique already used for the geantino bug, Section 5.21) was added to actually identify the particle and process responsible.

Diagnostic result, repeating the same configuration with the same seed (to reproduce exactly the same event): it is NOT a secondary fragment. It is the PRIMARY proton itself (parentID=0), with only 0.785 MeV of residual kinetic energy at the moment it crosses neuron 48 -- nearly stopped, very close to the end of its own individual path.

Physical explanation, corrected relative to the initial hypothesis: not every proton in a nominal 150 MeV beam stops at exactly the same depth -- statistical range straggling causes individual protons to stop a bit shallower or deeper than the average. This particular proton, by pure statistical fluctuation of its own path, happened to be a few micrometers from the end of its own range exactly at the depth of the beam's AVERAGE Bragg peak (where the ring is positioned)

-- and it is there, close to stopping, that an individual proton's LET diverges (the same physics as the macroscopic Bragg peak, but applied to a single proton rather than the beam as a whole).

Clinical relevance, not just a technical curiosity: this is exactly the mechanism behind the real, still-debated concern over increased RBE at the distal edge in proton therapy -- the last stretch of a proton beam's range has a systematically higher LET (and hence RBE) than the entrance and plateau, precisely because of the physics of individual particles slowing down, not a simulation artifact. It is the same reason the clinical assumption of a constant proton RBE (1.1) is the subject of an active research thread (variable proton RBE, recent literature e.g. Parisi et al. 2024).

Methodologically: the per-step diagnostic stays in the code ('SteppingAction.cc'), activatable for any neuron via the environment variable CNAO_DEBUG_NEURON=<index> -- the same principle of empirically verifying a surprising result instead of merely hypothesizing its cause, applied here on the Geant4 side with the same rigor already systematically used on the Brian2 side throughout the project.

Source code: *geant4_cnao_ring/src/SteppingAction.cc (CNAO_DEBUG_NEURON diagnostic added in this section)*

```

1  #include "SteppingAction.hh"
2  #include "EventAction.hh"
3
4  #include "G4Step.hh"
5  #include "G4StepPoint.hh"
6  #include "G4Track.hh"
7  #include "G4VProcess.hh"
8  #include "G4SystemOfUnits.hh"
9  #include "G4ParticleDefinition.hh"
10 #include "G4LogicalVolume.hh"
11 #include "G4VPhysicalVolume.hh"
12 #include "G4VTouchable.hh"
13 #include "G4TouchableHandle.hh"
14 #include "G4RunManager.hh"
15 #include "G4Event.hh"
16 #include <cstdlib>
17
18 SteppingAction::SteppingAction(EventAction* eventAction)
19 : G4UserSteppingAction(), fEventAction(eventAction)
20 {}
21
22 SteppingAction::~SteppingAction() {}
23
24 void SteppingAction::UserSteppingAction(const G4Step* step)
25 {
26     const G4StepPoint* preStep = step->GetPreStepPoint();
27     const G4VTouchable* touchable = preStep->GetTouchable();
28     const G4VPhysicalVolume* volume = touchable->GetVolume();
29     if (!volume || volume->GetName() != "Neuron") return;
30
31     const G4int neuronIndex = touchable->GetCopyNumber();
32
33     const G4double edep = step->GetTotalEnergyDeposit();
34     if (edep > 0.) fEventAction->AddEdep(neuronIndex, edep, step->GetStepLength());
35
36     // Diagnostica temporanea (sezione 5.36): stampa ogni step con
37     // deposito >0 nel neurone indicato da CNAO_DEBUG_NEURON, per

```

```

38 // identificare la particella/processo responsabile di un outlier
39 // di LET -- stesso pattern usato per il bug del geantino.
40 static const char* dbgEnv = std::getenv("CNAO_DEBUG_NEURON");
41 if (dbgEnv && neuronIndex == std::atoi(dbgEnv) && edep > 0.) {
42     const G4Track* track = step->GetTrack();
43     const G4VProcess* proc = step->GetPostStepPoint()->GetProcessDefinedStep();
44     G4cout << "[CNAO_DEBUG_NEURON] event=" << G4RunManager::GetRunManager()->GetCurrentEvent
45         ()->GetEventID()
46         << " particella=" << track->GetParticleDefinition()->GetParticleName()
47         << " parentID=" << track->GetParentID()
48         << " trackID=" << track->GetTrackID()
49         << " processo=" << (proc ? proc->GetProcessName() : "N/A")
50         << " edep_keV=" << edep/CLHEP::keV
51         << " stepLength_um=" << step->GetStepLength()/CLHEP::um
52         << " KE_MeV=" << track->GetKineticEnergy()/CLHEP::MeV
53         << G4endl;
54 }
55 // ParentID==0 identifies the primary proton/ion track itself
56 // (as opposed to any secondary electron, delta-ray, or nuclear
57 // fragment produced along the way).
58 const G4Track* track = step->GetTrack();
59 if (track->GetParentID() == 0) {
60     fEventAction->MarkPrimaryHit(neuronIndex);
61 }
62
63 // Sezione 5.39: registra l'energia cinetica di ogni elettrone
64 // secondario NATO dentro questo neurone (primo step del track,
65 // pre-step point ancora nel volume "Neuron") -- lo spettro
66 // necessario per valutare un accoppiamento con Geant4-DNA (valido
67 // solo fino a ~1 MeV), invece di assumerlo senza verificarlo.
68 if (track->GetParentID() != 0
69     && track->GetParticleDefinition()->GetParticleName() == "e-"
70     && track->GetCurrentStepNumber() == 1) {
71     fEventAction->AddSecondaryElectron(neuronIndex, preStep->GetKineticEnergy());
72 }
73 }

```

5.37 More statistics help, but are not enough: quantifying the cost of stabilizing the map

A direct follow-on to 5.32/5.33: there it had been discovered that at 500,000 events the ranking of the 10 highest-dose neurons is dominated by Poisson noise (average overlap 1.0/10 between independent seeds -- EXACTLY the level expected by pure chance: two independent random draws of 10 elements out of 100 overlap on average by $10 \times 10 / 100 = 1.0$). The most obvious solution, already listed as the first future development in the Geant4 project's README, is to increase statistics. How much does it actually help?

Tested directly: the same configuration (carbon-12 120 MeV/u at the peak) was repeated with 5,000,000 events (10x the 500,000 of 5.32), over 2 independent seeds (501, 502). Total dose on the ring: 2137.8 MeV and 2149.9 MeV -- correctly scales 10x relative to the ~192-226 MeV observed at 500,000 events (consistency check passed). Total primary hits per run: ~1650, i.e. an average of 16.5 direct crossings per neuron (against the 1-2 expected at 500,000 events) -- per-neuron statistics roughly 10 times better, as expected.

Top-10 overlap between the 2 seeds at 5,000,000 events: 2/10 -- improved relative

to the pure-noise level (1.0/10), but still far from a stable or deterministic map. The improvement (a factor of $\sim 2\times$ in overlap) is roughly consistent with the expected scaling of statistical noise (the standard deviation of a Poisson count scales as the square root of the number of events: 10x more events reduces relative noise by a factor of ~ 3.16 , not eliminate it).

Honest quantitative conclusion: statistics help but do NOT solve the problem even with an order of magnitude more events. Extrapolating the same scaling, obtaining a genuinely stable map (overlap close to 8-9/10) would most likely require a further 100-1000x increase in events relative to the original 500,000 -- computationally prohibitive with QBBC's full hadronic physics within this project's time constraints (a single run at 5,000,000 events already took about 15-20 minutes; 100-1000x would mean days per single configuration). This retroactively confirms, with a number instead of an intuition, that the path taken in Section 5.33 (averaging dose over multiple MODEST-statistics seeds, instead of chasing the stability of a single very-high-statistics run) was the right choice: averaging over seeds is far more computationally efficient than brute-force increasing events per run.

Data generated with cnaoRingImpact (seeds 501/502, 5,000,000 events each) in build/results_highstat/

5.38 The knockout size is not magic: the mechanism generalizes beyond "exactly 10"

The last limitation stated in geant4_cnao_ring's README: every knockout, from Section 5.22 onward, always used exactly the 10 highest-dose neurons -- an arbitrary relative threshold (top-10%), not an absolute biological-damage threshold. A question never asked: does the mechanism (the effect scales with the physical imbalance between the two messages' encoding zones) hold if the knockout size is changed, or was "10" special in some way?

The dose AVERAGED over Section 5.32's 4 seeds (carbon-12 at the peak, the most solid basis available) was used to build two new sets: TOP-5 (the 5 highest-dose neurons) and TOP-20 (the 20 highest-dose neurons). Overlap with the encoding zones: TOP-5 has ZERO overlap with either zone (none of the 5 hardest-hit neurons falls in $x=0.15$ or $x=0.65$) -- prediction: null effect. TOP-20, by contrast, has an imbalance of OPPOSITE sign to the already-tested TOP-10 from 5.33: 3 neurons in $x=0.15$ against 1 in $x=0.65$ (TOP-10 had 0 against 1) -- prediction, deliberately falsifiable in the same spirit as 5.25/5.26: the effect should REVERSE, damaging $x=0.15$ instead of $x=0.65$.

Verification over 8 seeds for both sets. TOP-5: average ratio $x=0.15/x=0.65 = 1.07$, with 4 of 8 seeds favoring each message -- exactly null, as predicted for a 0-against-0 overlap. TOP-20: average ratio $x=0.65/x=0.15 = 1.74\times$, with $x=0.65$ greater than $x=0.15$ in 7 of 8 seeds -- the effect REVERSED cleanly and consistently relative to TOP-10 (where $x=0.65$ was the more-damaged message), exactly as predicted by the sign flip in the physical imbalance.

Conclusion: the causal mechanism does not depend on the specific choice of "exactly 10 neurons" -- it generalizes to different knockout sizes, with the direction and strength of the effect always explained by the same principle (physical imbalance between encoding zones), never by an artifact tied to the particular threshold used. This is the fourth time in this project (after 5.25/5.26, 5.33, and now this) that a deliberately falsifiable prediction about the sign or existence of an effect is confirmed -- the most rigorous way available to distinguish a genuine causal mechanism from a coincidence engineered to produce a particular result.

Source code: `dialogo_spiking_cnao_knockout_top5_mc.py` and `dialogo_spiking_cnao_knockout_top20_mc.py`

```

1  """
2  Sezione 5.38: la scelta di "esattamente 10 neuroni" per il knockout,
3  usata in ogni esperimento dalle sezioni 5.22 in poi, era arbitraria --
4  una soglia relativa (top-10%), non una soglia assoluta di danno
5  biologico (limite gia' dichiarato nel README di geant4_cnao_ring). Il
6  meccanismo (l'effetto scala con lo sbilanciamento fisico tra le due
7  zone di codifica) generalizza a dimensioni di knockout diverse, o
8  "10" era in qualche modo speciale?
9
10 Qui: TOP-5 (i 5 neuroni a dose media piu' alta sui 4 seed della 5.32,
11 carbonio-12 al picco). Overlap con le zone di codifica: ZERO in
12 entrambe -- nessuno dei 5 neuroni piu' colpiti cade nella zona di
13 x=0.15 ne' in quella di x=0.65. Previsione dal meccanismo gia'
14 stabilito: effetto nullo, indistinguibile dal controllo negativo
15 (come il caso -1-vs-1 della 5.27 e lo 0-vs-0 della 5.24).
16 """
17
18 KNOCKOUT = sorted([7, 29, 37, 73, 82]) # top-5 per DOSE MEDIA sui 4 seed
19                                         # 111/222/333/444 (carbonio-12
20                                         # 120 MeV/u, picco 34.5mm --
21                                         # overlap 0-vs-0, vedi 5.38)
22
23 """
24 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
25 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
26 usate nel primo test di interferenza del modello a rate (dialogo_multi_
27 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
28
29 Nota di partenza importante, diversa dal modello a rate: li' i pesi
30 inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
31 un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
32 poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
33 due messaggi cercavano di scoprire la loro mappa nello stesso spazio
34 casuale contemporaneamente. Qui invece la connettivita' inter-anello parte
35 gia' strutturata (caduta gaussiana sulla distanza circolare, offset zero,
36 sezione 5.7): non serve alcuna scoperta, la mappa identita' e' gia'
37 presente prima che la STDP inizi ad allenare qualunque cosa. Non e' quindi
38 scontato che la stessa interferenza si ripresenti -- il meccanismo che la
39 causava nel modello a rate potrebbe semplicemente non esistere qui.
40 Bidirezionalita' non ritestata in questo script: la sezione 5.9 ha gia'
41 concluso che il canale B->A non trasmette causalmente in questo sistema,
42 quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
43 """
44
45 import sys
46 from brian2 import *
47
48 SEED = int(sys.argv[1])
49 start_scope()
50 seed(SEED)
51
52 N_E = 100

```

```

53 N_I = 25
54 N_ring = N_E + N_I
55
56 a = 0.02/ms
57 b = 0.2/ms
58 c = -65*mV
59 d = 6*mV/ms
60 tau_nmda = 100*ms
61
62 eqs = '''
63 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
64 du/dt = a*(b*v - u) : volt/second
65 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
66 I : volt/second
67 x : 1
68 '''
69
70 neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
71                       method='euler')
72 neurons.v = -65*mV + 5*mV*randn(2*N_ring)
73 neurons.u = b * neurons.v[:]
74 neurons.I = 0*mV/ms
75 neurons.g_nmda = 0*mV/ms
76
77 A_E = neurons[:N_E]
78 A_I = neurons[N_E:N_ring]
79 B_E = neurons[N_ring:N_ring+N_E]
80 B_I = neurons[N_ring+N_E:]
81
82 xi = np.arange(N_E) / N_E
83 A_E.x = xi
84 B_E.x = xi
85
86 background = PoissonGroup(2*N_ring, rates=400*Hz)
87 bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
88 bg_syn.connect(j='i')
89
90 sigma_exc = 0.05
91 CUTOFF_LOCALE = 3 * sigma_exc
92 w_ee_nmda = 0.325*mV/ms
93 p_conn = 0.15
94 w_exc = 1.0*mV
95 w_inh = 10.0*mV
96
97
98 def dist_circ(dx):
99     dx = abs(dx)
100     return np.minimum(dx, 1 - dx)
101
102
103 def costruisci_anello(E, I_pop, seed, knockout=()):
104     dmat = dist_circ(xi[:, None] - xi[None, :])
105     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
106     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
107     np.fill_diagonal(prob_ee, 0)
108     rng = np.random.RandomState(seed)
109     mask_ee = rng.rand(N_E, N_E) < prob_ee
110     i_idx, j_idx = np.nonzero(mask_ee)
111
112     ko = np.asarray(list(knockout), dtype=int)
113     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
114     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
115     # piu' influenzare nessuno (l'unica cosa che conta per una rete
116     # a spike: se non spara mai in modo efficace a valle, e' come
117     # se non ci fosse)
118     tieni_ee = ~np.isin(i_idx, ko)
119     i_idx, j_idx = i_idx[tieni_ee], j_idx[tieni_ee]
120

```

```

121     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
122     syn_ee.connect(i=i_idx, j=j_idx)
123     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
124     # Costruita esplicitamente (invece di .connect(p=...)) per poter
125     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
126     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
127     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
128     i_ei, j_ei = np.nonzero(mask_ei)
129     tieni_ei = ~np.isin(i_ei, ko)
130     syn_ei.connect(i=i_ei[tieni_ei], j=j_ei[tieni_ei])
131     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
132     syn_ie.connect(p=p_conn)
133     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
134     syn_ii.connect(condition='i!=j', p=p_conn)
135     return syn_ee, syn_ei, syn_ie, syn_ii
136
137
138 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
139 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
140 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
141       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
142
143 tau_stdp = 20*ms
144 A_ltp = 0.15*mV
145 A_ltd = 0.15*mV
146 w_max = 5.0*mV
147 w_min = 0*mV
148 w_init = 1.5*mV
149 sigma_inter = 0.08
150 p_inter_picco = 0.5
151
152 stdp_model = '''
153 w : volt
154 dapre/dt = -apre/tau_stdp : volt (event-driven)
155 dapost/dt = -apost/tau_stdp : volt (event-driven)
156 '''
157 stdp_on_pre = '''
158 v_post += w
159 apre += A_ltp
160 w = clip(w+apost, w_min, w_max)
161 '''
162 stdp_on_post = '''
163 apost -= A_ltd
164 w = clip(w+apre, w_min, w_max)
165 '''
166
167 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
168 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
169 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
170 rng_inter = np.random.RandomState(3000+SEED)
171 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
172 i_inter, j_inter = np.nonzero(mask_inter)
173 ko_arr = np.asarray(KNOCKOUT, dtype=int)
174 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
175 # trasmettere neanche a B
176 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
177 syn_ab.connect(i=i_inter, j=j_inter)
178 syn_ab.w = w_init
179 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
180       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
181
182 spikes = SpikeMonitor(neurons)
183
184 MESSAGGI = [0.15, 0.65]
185 larghezza_stimolo = 0.05
186 gruppi_stim = []
187 for m in MESSAGGI:
188     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))

```



```

189     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
190     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
191
192     N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
193     T_STIM = 60*ms
194     T_PAUSA = 340*ms
195
196     print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
197           f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
198
199     storico = [] # (indice_messaggio, tempo_inizio_ms)
200     t_corrente = 0*ms
201     for rep in range(N_RIPETIZIONI_TOT):
202         k = rep % len(MESSAGGI)
203         storico.append((k, t_corrente/ms))
204         A_E.I[gruppi_stim[k]] = 15*mV/ms
205         run(T_STIM)
206         A_E.I[:] = 0*mV/ms
207         run(T_PAUSA)
208         t_corrente += T_STIM + T_PAUSA
209         if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
210             print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
211                   f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
212
213     print("\nTraining completato.")
214
215     fig, axes = subplots(2, 1, figsize=(11, 7))
216     subplot(2, 1, 1)
217     mask_a = spikes.i < N_E
218     mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
219     plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
220     plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
221     xlabel('Tempo (ms)')
222     ylabel('Indice neurone')
223     title(f"Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, "
224           f"{len(KNOCKOUT)} neuroni spenti (dose CNAO piu' alta)")
225     legend(loc='upper right', fontsize=8, markerscale=6)
226
227     subplot(2, 1, 2)
228     t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
229     mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
230     mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
231     plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')
232     plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize=3, label='B (E)')
233     xlabel('Tempo (ms)')
234     ylabel('Indice neurone')
235     title('Dettaglio: ultime ripetizioni (fine training)')
236     legend(loc='upper right', fontsize=8, markerscale=4)
237
238     tight_layout()
239     savefig('dialogo_spiking_cnao_knockout_top5.png', dpi=150)
240     print("\nGrafico salvato in dialogo_spiking_cnao_knockout_top5.png")
241
242     ciclo_ms = (T_STIM+T_PAUSA)/ms
243
244
245     def valuta_messaggio(k, n_eval):
246         tempi = [t0 for (kk, t0) in storico if kk == k]
247         primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
248
249         def conta(lista):
250             n_tot, posizioni = 0, []
251             for t0 in lista:
252                 fin = t0 + ciclo_ms
253                 m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)

```

```

254         idx = spikes.i[m] - N_ring
255         n_tot += m.sum()
256         posizioni.extend(list(xi[idx]))
257         centro = None
258         if posizioni:
259             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
260                             % 1.0)
261         return n_tot, centro
262     return conta(primi), conta(ultimi)
263
264 print("\n" + "*"70)
265 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
266       "correttamente e si rafforza per ENTRAMBI i messaggi")
267 print("=*70)
268 for k, m in enumerate(MESSAGGI):
269     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
270     err_i = dist_circ(c_i - m) if c_i is not None else None
271     err_f = dist_circ(c_f - m) if c_f is not None else None
272     print(f" Messaggio x={m}:")
273     print(f" prime 5 ripetizioni: {n_i} spike, centro={({N/A} if c_i is None else f'{c_i:.3f}
274           }')", "
275           f"errore={({N/A} if err_i is None else f'{err_i:.3f}')}")
276     print(f" ultime 5 ripetizioni: {n_f} spike, centro={({N/A} if c_f is None else f'{c_f:.3f}
277           }')", "
278           f"errore={({N/A} if err_f is None else f'{err_f:.3f}')}")
279
280 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)"
281       )
282 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
283
284 righe = []
285 for k, m in enumerate(MESSAGGI):
286     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
287     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
288     righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
289 print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

```

1  """
2  Sezione 5.38 (vedi anche la variante TOP-5, stesso giorno): test di
3  confine sulla dimensione del knockout, non piu' fissa a 10 neuroni.
4
5  Qui: TOP-20 (i 20 neuroni a dose media piu' alta sui 4 seed della
6  5.32, carbonio-12 al picco). Overlap con le zone di codifica: 3
7  neuroni (12, 18, 20) in x=0.15, 1 neurone (70, lo stesso della 5.33)
8  in x=0.65 -- uno sbilanciamento di segno OPPOSTO al caso TOP-10 (che
9  aveva 0 in x=0.15 e 1 in x=0.65). Previsione, deliberatamente
10 falsificabile nello stesso spirito della 5.25/5.26: se il meccanismo e'
11 davvero guidato dallo sbilanciamento fisico e non da un artefatto
12 legato a "esattamente 10 neuroni", l'effetto dovrebbe INVERTIRSI --
13 questa volta x=0.15 piu' danneggiato di x=0.65, con una magnitudine
14 paragonabile al 3-contro-1 della sezione 5.26 (che diede 1.81x).
15 """
16
17 KNOCKOUT = sorted([0, 5, 7, 12, 18, 20, 26, 28, 29, 32, 37, 41, 45, 47,
18                   48, 70, 73, 79, 82, 99]) # top-20 per DOSE MEDIA sui
19                                           # 4 seed 111/222/333/444
20                                           # (carbonio-12 120 MeV/u,
21                                           # picco 34.5mm -- 5.38)
22
23 """
24 Estensione di dialogo_spiking.py (sezione 5.7, comunicazione appresa A->B
25 con STDP vera su un solo messaggio) a DUE messaggi, sulle stesse posizioni
26 usate nel primo test di interferenza del modello a rate (dialogo_multi_
27 messaggio.py, sezione 5.5.4): x=0.15 e x=0.65.
28
29 Nota di partenza importante, diversa dal modello a rate: li' i pesi

```

```

30 | inter-popolazione partivano CASUALI e la scoperta di una mappa richiedeva
31 | un processo di ricerca (prima "rumore puro", poi "rumore + repulsione",
32 | poi "rumore + scoperta locale") -- da cui nasceva l'interferenza quando
33 | due messaggi cercavano di scoprire la loro mappa nello stesso spazio
34 | casuale contemporaneamente. Qui invece la connettività inter-anello parte
35 | già strutturata (caduta gaussiana sulla distanza circolare, offset zero,
36 | sezione 5.7): non serve alcuna scoperta, la mappa identità è già
37 | presente prima che la STDP inizi ad allenare qualunque cosa. Non è quindi
38 | scontato che la stessa interferenza si ripresenti -- il meccanismo che la
39 | causava nel modello a rate potrebbe semplicemente non esistere qui.
40 | Bidirezionalità non testata in questo script: la sezione 5.9 ha già
41 | concluso che il canale B->A non trasmette causalmente in questo sistema,
42 | quindi ci si concentra sul canale A->B, l'unico verificato funzionante.
43 | """
44 |
45 | import sys
46 | from brian2 import *
47 |
48 | SEED = int(sys.argv[1])
49 | start_scope()
50 | seed(SEED)
51 |
52 | N_E = 100
53 | N_I = 25
54 | N_ring = N_E + N_I
55 |
56 | a = 0.02/ms
57 | b = 0.2/ms
58 | c = -65*mV
59 | d = 6*mV/ms
60 | tau_nmda = 100*ms
61 |
62 | eqs = '''
63 | dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
64 | du/dt = a*(b*v - u) : volt/second
65 | dg_nmda/dt = -g_nmda/tau_nmda : volt/second
66 | I : volt/second
67 | x : 1
68 | '''
69 |
70 | neurons = NeuronGroup(2*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
71 |                      method='euler')
72 | neurons.v = -65*mV + 5*mV*randn(2*N_ring)
73 | neurons.u = b * neurons.v[:]
74 | neurons.I = 0*mV/ms
75 | neurons.g_nmda = 0*mV/ms
76 |
77 | A_E = neurons[:N_E]
78 | A_I = neurons[N_E:N_ring]
79 | B_E = neurons[N_ring:N_ring+N_E]
80 | B_I = neurons[N_ring+N_E:]
81 |
82 | xi = np.arange(N_E) / N_E
83 | A_E.x = xi
84 | B_E.x = xi
85 |
86 | background = PoissonGroup(2*N_ring, rates=400*Hz)
87 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
88 | bg_syn.connect(j='i')
89 |
90 | sigma_exc = 0.05
91 | CUTOFF_LOCALE = 3 * sigma_exc
92 | w_ee_nmda = 0.325*mV/ms
93 | p_conn = 0.15
94 | w_exc = 1.0*mV
95 | w_inh = 10.0*mV
96 |
97 |

```

```

98 def dist_circ(dx):
99     dx = abs(dx)
100     return np.minimum(dx, 1 - dx)
101
102
103 def costruisci_anello(E, I_pop, seed, knockout=()):
104     dmat = dist_circ(xi[:, None] - xi[None, :])
105     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
106     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
107     np.fill_diagonal(prob_ee, 0)
108     rng = np.random.RandomState(seed)
109     mask_ee = rng.rand(N_E, N_E) < prob_ee
110     i_idx, j_idx = np.nonzero(mask_ee)
111
112     ko = np.asarray(list(knockout), dtype=int)
113     # "spento" = nessuna sinapsi in USCITA sopravvive, da nessun
114     # neurone in knockout -- puo' ancora ricevere input, ma non puo'
115     # piu' influenzare nessuno (l'unica cosa che conta per una rete
116     # a spike: se non spara mai in modo efficace a valle, e' come
117     # se non ci fosse)
118     tieni_ee = ~np.isin(i_idx, ko)
119     i_idx, j_idx = i_idx[tiene_ee], j_idx[tiene_ee]
120
121     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
122     syn_ee.connect(i=i_idx, j=j_idx)
123     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
124     # Costruita esplicitamente (invece di .connect(p=...) per poter
125     # filtrare via i pre-sinaptici spenti, come sopra per E->E.
126     rng_ei = np.random.RandomState(seed + 100 + SEED*17)
127     mask_ei = rng_ei.rand(len(E), len(I_pop)) < p_conn
128     i_ei, j_ei = np.nonzero(mask_ei)
129     tieni_ei = ~np.isin(i_ei, ko)
130     syn_ei.connect(i=i_ei[tiene_ei], j=j_ei[tiene_ei])
131     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
132     syn_ie.connect(p=p_conn)
133     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
134     syn_ii.connect(condition='i!=j', p=p_conn)
135     return syn_ee, syn_ei, syn_ie, syn_ii
136
137
138 syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED, knockout=KNOCKOUT)
139 syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED, knockout=KNOCKOUT)
140 print(f"Knockout applicato: {len(KNOCKOUT)} neuroni per anello ({KNOCKOUT}) -- "
141       f"tutte le sinapsi in uscita rimosse (E->E e E->I)")
142
143 tau_stdp = 20*ms
144 A_ltp = 0.15*mV
145 A_ltd = 0.15*mV
146 w_max = 5.0*mV
147 w_min = 0*mV
148 w_init = 1.5*mV
149 sigma_inter = 0.08
150 p_inter_picco = 0.5
151
152 stdp_model = '''
153 w : volt
154 dapre/dt = -apre/tau_stdp : volt (event-driven)
155 dapost/dt = -apost/tau_stdp : volt (event-driven)
156 '''
157 stdp_on_pre = '''
158 v_post += w
159 apre += A_ltp
160 w = clip(w+apost, w_min, w_max)
161 '''
162 stdp_on_post = '''
163 apost -= A_ltd
164 w = clip(w+apre, w_min, w_max)
165 '''

```

```

166 syn_ab = Synapses(A_E, B_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
167 dmat_inter = dist_circ(xi[:, None] - xi[None, :])
168 prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
169 rng_inter = np.random.RandomState(3000+SEED)
170 mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
171 i_inter, j_inter = np.nonzero(mask_inter)
172 ko_arr = np.asarray(KNOCKOUT, dtype=int)
173 tieni_inter = ~np.isin(i_inter, ko_arr) # un neurone A spento non puo'
174 # trasmettere neanche a B
175 i_inter, j_inter = i_inter[tieni_inter], j_inter[tieni_inter]
176 syn_ab.connect(i=i_inter, j=j_inter)
177 syn_ab.w = w_init
178 print(f"Sinapsi inter-anello A->B create: {len(syn_ab)} (peso iniziale {w_init}, "
179       f"dopo aver rimosso le uscite dai {len(KNOCKOUT)} neuroni spenti)")
180
181 spikes = SpikeMonitor(neurons)
182
183 MESSAGGI = [0.15, 0.65]
184 larghezza_stimolo = 0.05
185 gruppi_stim = []
186 for m in MESSAGGI:
187     peso_stim = np.exp(-dist_circ(xi - m)**2 / (2*larghezza_stimolo**2))
188     gruppi_stim.append(np.where(peso_stim > 0.5)[0])
189     print(f"Messaggio x={m}: {len(gruppi_stim[-1])} neuroni di A stimolati")
190
191 N_RIPETIZIONI_TOT = 60 # round-robin: 30 ripetizioni per messaggio
192 T_STIM = 60*ms
193 T_PAUSA = 340*ms
194
195 print(f"\nTraining: {N_RIPETIZIONI_TOT} ripetizioni round-robin tra "
196       f"{len(MESSAGGI)} messaggi ({(T_STIM+T_PAUSA)*N_RIPETIZIONI_TOT/ms:.0f}ms totali)")
197
198 storico = [] # (indice_messaggio, tempo_inizio_ms)
199 t_corrente = 0*ms
200 for rep in range(N_RIPETIZIONI_TOT):
201     k = rep % len(MESSAGGI)
202     storico.append((k, t_corrente/ms))
203     A_E.I[gruppi_stim[k]] = 15*mV/ms
204     run(T_STIM)
205     A_E.I[:] = 0*mV/ms
206     run(T_PAUSA)
207     t_corrente += T_STIM + T_PAUSA
208     if rep % 10 == 0 or rep == N_RIPETIZIONI_TOT-1:
209         print(f"ripetizione {rep:3d}/{N_RIPETIZIONI_TOT} (messaggio x={MESSAGGI[k]}): "
210               f"peso medio={np.mean(syn_ab.w)/mV:.3f}mV max={np.max(syn_ab.w)/mV:.3f}mV")
211
212 print("\nTraining completato.")
213
214 fig, axes = subplots(2, 1, figsize=(11, 7))
215 subplot(2, 1, 1)
216 mask_a = spikes.i < N_E
217 mask_b = (spikes.i >= N_ring) & (spikes.i < N_ring+N_E)
218 plot(spikes.t[mask_a]/ms, spikes.i[mask_a], '.', color='C0', markersize=1.5, label='A (E)')
219 plot(spikes.t[mask_b]/ms, spikes.i[mask_b]-N_ring, '.', color='C3', markersize=1.5, label='B (E)')
220
221 xlabel('Tempo (ms)')
222 ylabel('Indice neurone')
223 title(f'Raster completo: {N_RIPETIZIONI_TOT} ripetizioni, messaggi x={MESSAGGI}, '
224       f'{len(KNOCKOUT)} neuroni spenti (dose CNAO piu\' alta)')
225 legend(loc='upper right', fontsize=8, markerscale=6)
226
227 subplot(2, 1, 2)
228 t_tardi = (N_RIPETIZIONI_TOT-10) * (T_STIM+T_PAUSA)/ms
229 mask_a_tardi = mask_a & (spikes.t/ms > t_tardi)
230 mask_b_tardi = mask_b & (spikes.t/ms > t_tardi)
231 plot(spikes.t[mask_a_tardi]/ms, spikes.i[mask_a_tardi], '.', color='C0', markersize=3, label='A (E)')

```

```

232 plot(spikes.t[mask_b_tardi]/ms, spikes.i[mask_b_tardi]-N_ring, '.', color='C3', markersize
      =3, label='B (E)')
233 xlabel('Tempo (ms)')
234 ylabel('Indice neurone')
235 title('Dettaglio: ultime ripetizioni (fine training)')
236 legend(loc='upper right', fontsize=8, markerscale=4)
237
238 tight_layout()
239 savefig('dialogo_spiking_cnao_knockout_top20.png', dpi=150)
240 print("\nGrafico salvato in dialogo_spiking_cnao_knockout_top20.png")
241
242 ciclo_ms = (T_STIM+T_PAUSA)/ms
243
244
245 def valuta_messaggio(k, n_eval):
246     tempi = [t0 for (kk, t0) in storico if kk == k]
247     primi, ultimi = tempi[:n_eval], tempi[-n_eval:]
248
249     def conta(lista):
250         n_tot, posizioni = 0, []
251         for t0 in lista:
252             fin = t0 + ciclo_ms
253             m = mask_b & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
254             idx = spikes.i[m] - N_ring
255             n_tot += m.sum()
256             posizioni.extend(list(xi[idx]))
257         centro = None
258         if posizioni:
259             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
260                           % 1.0)
261         return n_tot, centro
262     return conta(primi), conta(ultimi)
263
264 print("\n" + "="*70)
265 print("VALUTAZIONE PER MESSAGGIO: la risposta di B si localizza "
266       "correttamente e si rafforza per ENTRAMBI i messaggi?")
267 print("="*70)
268 for k, m in enumerate(MESSAGGI):
269     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
270     err_i = dist_circ(c_i - m) if c_i is not None else None
271     err_f = dist_circ(c_f - m) if c_f is not None else None
272     print(f" Messaggio x={m}:")
273     print(f" prime 5 ripetizioni: {n_i} spike, centro=({'N/A' if c_i is None else f'{c_i:.3f}')}")
274     print(f" errore=({'N/A' if err_i is None else f'{err_i:.3f}')}")
275     print(f" ultime 5 ripetizioni: {n_f} spike, centro=({'N/A' if c_f is None else f'{c_f:.3f}')}")
276     print(f" errore=({'N/A' if err_f is None else f'{err_f:.3f}')}")
277
278 print(f"\nPeso medio finale A->B: {np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV)")
279 print(f"Peso massimo A->B: {np.max(syn_ab.w)/mV:.3f}mV")
280
281 righe = []
282 for k, m in enumerate(MESSAGGI):
283     (n_i, c_i), (n_f, c_f) = valuta_messaggio(k, 5)
284     err_f = dist_circ(c_f - m) if c_f is not None else 1.0
285     righe.append(f"m{k}(x={m}):n_i={n_i},n_f={n_f},err_f={err_f:.3f}")
286 print(f"MC_RIGA seed={SEED} " + " ".join(righe))

```

5.39 The nanodosimetric link, first step: the secondary electron spectrum

The last remaining future development listed in `geant4_cnao_ring`'s README: linking this project (macroscopic physics, QBBC) to the twin project `geant4dna_icsd` (nanodosimetry, Geant4-DNA, verification of the Villagrasa/Baiocco 2026 paper), by re-simulating in detail the tracks of secondary electrons produced in the affected neurons -- the same multi-scale "track re-simulation" approach used by TOPAS-nBio. Before building the full pipeline, one question needs checking: do the secondary electrons produced by a clinical CNAO beam inside a neuron actually have energies compatible with Geant4-DNA (valid only up to ~1 MeV)?

A dedicated tracking mechanism (`EventAction/RunAction/SteppingAction`) was added to record the kinetic energy of every electron born inside a "Neuron" volume. First attempt: ZERO secondary electrons recorded out of 500,000 events -- a surprising result, checked before being accepted. Cause found: without a dedicated region, QBBC's default production cut (~1mm) is far larger than the neurons (10um radius) -- almost no delta ray has enough range to be generated as an explicitly tracked particle in such a small volume; its energy is instead deposited locally as continuous loss, invisible to a per-particle count (but already included in the total dose computed in previous sections -- no error there, just a limitation of this new tracking).

Fixed by adding a dedicated `G4Region` for the "Neuron" volumes, with a 100nm production cut (below the neuron's scale). The same configuration (carbon-12 at the peak) was repeated: 18,021 explicit secondary electrons out of 500,000 events. Spectrum: minimum 0.99 keV, maximum 96.2 keV, median 1.86 keV, mean 3.57 keV -- 100% below 1 MeV, within Geant4-DNA's domain of validity. A notable coincidence: the median and mean fall exactly within the energy range (1, 5, 10 keV) explicitly tested in the Villagrasa/Baiocco paper that `geant4dna_icsd` reproduces -- the link would not require extrapolating beyond the domain already validated in that twin project.

Conclusion: the physical premise of the multi-scale coupling is confirmed empirically, not assumed. The natural next step (not yet done) is to use these real energies directly as primaries in nanoICSD instead of the paper's monoenergetic values, to obtain ICSD (Ionization Cluster Size Distribution) results tied to secondaries ACTUALLY produced by a CNAO beam, not to an arbitrarily chosen energy.

Source code: *DetectorConstruction.cc / EventAction.cc / RunAction.cc / SteppingAction.cc (tracking and production-cut fix added in this section)*

```
1 #include "DetectorConstruction.hh"
2
3 #include "G4NistManager.hh"
4 #include "G4Material.hh"
5 #include "G4Box.hh"
```

```

6  #include "G4Orb.hh"
7  #include "G4LogicalVolume.hh"
8  #include "G4PVPlacement.hh"
9  #include "G4SystemOfUnits.hh"
10 #include "G4VisAttributes.hh"
11 #include "G4Colour.hh"
12 #include "G4PhysicalConstants.hh"
13 #include "G4Region.hh"
14 #include "G4ProductionCuts.hh"
15 #include "G4RegionStore.hh"
16
17 DetectorConstruction::DetectorConstruction(G4double ringDepthFromEntry)
18 : G4VUserDetectorConstruction(),
19   fRingDepth(ringDepthFromEntry),
20   fRingRadius(500.*CLHEP::um), // ~1mm diameter ring, cortical
21                                   // microcircuit scale (see README)
22   fSomaRadius(10.*CLHEP::um) // ~20um soma diameter, typical
23                                   // cortical pyramidal neuron (README)
24 {
25     // World sized to comfortably contain the beam's full path from entry
26     // to the ring, plus a safety margin (proton/ion range straggling and
27     // the lateral 550um sampling disk).
28     fWorldHalfXY = 5.*CLHEP::mm;
29     fWorldHalfZ = 0.5*fRingDepth + 5.*CLHEP::mm;
30     fZStart = -fWorldHalfZ + 0.5*CLHEP::mm; // just inside the entry face
31 }
32
33 DetectorConstruction::~DetectorConstruction() {}
34
35 G4VPhysicalVolume* DetectorConstruction::Construct()
36 {
37     G4NistManager* nist = G4NistManager::Instance();
38     // Water as tissue-equivalent surrogate, the same convention used in
39     // clinical proton/ion dosimetry and in nanoICSD -- keeps this run
40     // directly comparable in dose units to standard hadrontherapy
41     // Monte Carlo practice.
42     G4Material* water = nist->FindOrBuildMaterial("G4_WATER");
43
44     // --- World: tissue block along the beam axis, ring placed at
45     // Z = fZStart + fRingDepth (the depth requested by main(), e.g.
46     // the Bragg peak depth found with braggProfile/) ---
47     auto solidWorld = new G4Box("World", fWorldHalfXY, fWorldHalfXY, fWorldHalfZ);
48     auto logicWorld = new G4LogicalVolume(solidWorld, water, "World");
49     auto physWorld = new G4PVPlacement(nullptr, {}, logicWorld, "World",
50                                       nullptr, false, 0, true);
51
52     G4double ringZ = fZStart + fRingDepth;
53
54     // --- Ring of N_NEURONS somata in the Z=ringZ plane (the requested
55     // depth), positions matching xi = i/N_NEURONS in the Brian2
56     // model (dialogo_spiking_.py) ---
57     fLogicNeuron.reserve(N_NEURONS);
58     for (G4int i = 0; i < N_NEURONS; ++i) {
59         G4double xi = static_cast<G4double>(i) / N_NEURONS;
60         G4double theta = CLHEP::twopi * xi;
61         G4double x = fRingRadius * std::cos(theta);
62         G4double y = fRingRadius * std::sin(theta);
63
64         auto solidNeuron = new G4Orb("Neuron", fSomaRadius);
65         auto logicNeuron = new G4LogicalVolume(solidNeuron, water, "Neuron");
66         // Copy number IS the neuron index i -- SteppingAction reads it
67         // back to know which ring position (xi) was hit.
68         new G4PVPlacement(nullptr, G4ThreeVector(x, y, ringZ), logicNeuron,
69                           "Neuron", logicWorld, false, i, true);
70         fLogicNeuron.push_back(logicNeuron);
71
72         auto visNeuron = new G4VisAttributes(G4Colour(0.85, 0.25, 0.2, 0.6));
73         logicNeuron->SetVisAttributes(visNeuron);

```



```

74     }
75
76     auto visWorld = new G4VisAttributes(G4Colour(0.5, 0.6, 1.0, 0.03));
77     visWorld->SetForceWireframe(true);
78     logicWorld->SetVisAttributes(visWorld);
79
80     // Sezione 5.39: senza un taglio di produzione dedicato, il default di
81     // QBBC (~1mm) e' molto piu' grande dei neuroni (10um di raggio) --
82     // quasi nessun elettrone secondario (raggio delta) ha un range
83     // sufficiente per essere generato come track esplicito dentro un
84     // volume cosi' piccolo; la sua energia viene invece depositata
85     // localmente come perdita di energia continua, INVISIBILE a un
86     // tracking per-particella (verificato: zero elettroni secondari
87     // registrati su 500.000 eventi prima di questa correzione). Una
88     // G4Region dedicata ai volumi "Neuron", con un taglio molto piu'
89     // fine (100nm, sotto la scala del neurone), fa si' che i raggi delta
90     // prodotti li' dentro vengano effettivamente tracciati come
91     // particelle esplicite -- il prerequisito per qualunque accoppiamento
92     // con Geant4-DNA (che opera per definizione su track espliciti).
93     auto* neuronRegion = new G4Region("NeuronRegion");
94     auto* neuronCuts = new G4ProductionCuts();
95     neuronCuts->SetProductionCut(100.*CLHEP::nm);
96     neuronRegion->SetProductionCuts(neuronCuts);
97     for (auto* logicNeuron : fLogicNeuron) {
98         neuronRegion->AddRootLogicalVolume(logicNeuron);
99     }
100
101     return physWorld;
102 }

```

```

1  #include "EventAction.hh"
2  #include "RunAction.hh"
3  #include "G4SystemOfUnits.hh"
4  #include <cmath>
5
6  namespace {
7      // Sezione 5.43: 6 punti rappresentativi dello spettro reale (percentili
8      // 10/25/50/75/90/99), con il tasso DSB/primario validato per ciascuno
9      // (moleculardna, geant4dna_ssbdsb). Riusati qui, non ricalcolati.
10     constexpr G4int kNBins = 6;
11     constexpr G4double kBinEnergyKeV[kNBins] = {1.09, 1.28, 1.86, 3.43, 7.33, 28.82};
12     constexpr G4double kBinDSBRate[kNBins] = {0.0103, 0.0083, 0.0137, 0.0243, 0.0430, 0.0163};
13
14     G4double DSBRateForEnergy(G4double keV)
15     {
16         G4int best = 0;
17         G4double bestDiff = std::abs(keV - kBinEnergyKeV[0]);
18         for (G4int i = 1; i < kNBins; ++i) {
19             G4double diff = std::abs(keV - kBinEnergyKeV[i]);
20             if (diff < bestDiff) { bestDiff = diff; best = i; }
21         }
22         return kBinDSBRate[best];
23     }
24 } // namespace
25
26 EventAction::EventAction(RunAction* runAction)
27 : G4UserEventAction(), fRunAction(runAction)
28 {}
29
30 EventAction::~EventAction() {}
31
32 void EventAction::BeginOfEventAction(const G4Event*)
33 {
34     fEdep.fill(0.);
35     fEdepSqOverDx.fill(0.);
36     fPrimaryHit.fill(false);
37     fSecondaryElectronEnergies.clear();

```

```

38     fSecondaryElectronCount.fill(0);
39     fEstimatedDSB.fill(0.);
40 }
41
42 void EventAction::AddEdep(G4int neuronIndex, G4double edep, G4double stepLength)
43 {
44     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
45     fEdep[neuronIndex] += edep;
46     if (stepLength > 0.) fEdepSqOverDx[neuronIndex] += edep * edep / stepLength;
47 }
48
49 void EventAction::MarkPrimaryHit(G4int neuronIndex)
50 {
51     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
52     fPrimaryHit[neuronIndex] = true;
53 }
54
55 void EventAction::AddSecondaryElectron(G4int neuronIndex, G4double kineticEnergy)
56 {
57     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
58     fSecondaryElectronEnergies.push_back(kineticEnergy);
59     fSecondaryElectronCount[neuronIndex] += 1;
60     fEstimatedDSB[neuronIndex] += DSBRateForEnergy(kineticEnergy / CLHEP::keV);
61 }
62
63 void EventAction::EndOfEventAction(const G4Event*)
64 {
65     fRunAction->RecordEvent(fEdep, fEdepSqOverDx, fPrimaryHit, fSecondaryElectronEnergies,
66                             fSecondaryElectronCount, fEstimatedDSB);
67 }

```

```

1  #include "RunAction.hh"
2
3  #include "G4Run.hh"
4  #include "G4SystemOfUnits.hh"
5  #include <sstream>
6
7  RunAction::RunAction() : G4UserRunAction() {}
8
9  RunAction::~RunAction() {}
10
11 void RunAction::BeginOfRunAction(const G4Run*)
12 {
13     fSumEdep.fill(0.);
14     fSumEdepSqOverDx.fill(0.);
15     fSumPrimaryHits.fill(0);
16     fNEvents = 0;
17     fSecondaryElectronSpectrum.clear();
18     fSumSecondaryElectronCount.fill(0);
19     fSumEstimatedDSB.fill(0.);
20 }
21
22 void RunAction::RecordEvent(const std::array<G4double, N_NEURONS>& edep,
23                             const std::array<G4double, N_NEURONS>& edepSqOverDx,
24                             const std::array<G4bool, N_NEURONS>& primaryHit,
25                             const std::vector<G4double>& secondaryElectronEnergies,
26                             const std::array<G4long, N_NEURONS>& secondaryElectronCount,
27                             const std::array<G4double, N_NEURONS>& estimatedDSB)
28 {
29     for (G4int i = 0; i < N_NEURONS; ++i) {
30         fSumEdep[i] += edep[i];
31         fSumEdepSqOverDx[i] += edepSqOverDx[i];
32         if (primaryHit[i]) fSumPrimaryHits[i] += 1;
33         fSumSecondaryElectronCount[i] += secondaryElectronCount[i];
34         fSumEstimatedDSB[i] += estimatedDSB[i];
35     }
36     fNEvents += 1;

```

```

37     for (G4double e : secondaryElectronEnergies) {
38         fSecondaryElectronSpectrum.push_back(e);
39     }
40 }
41
42 void RunAction::EndOfRunAction(const G4Run* run)
43 {
44     std::ostream filename;
45     filename << "neuron_dose_run" << run->GetRunID() << ".csv";
46     std::ofstream out(filename.str());
47     out << "# n_events=" << fNEvents << "\n";
48     out << "neuron_index,xi,sum_edep_MeV,mean_edep_keV_per_event,"
49           "primary_hit_count,primary_hit_fraction,dose_averaged_LET_keV_per_um,"
50           "secondary_electron_count,estimated_DSB\n";
51     for (G4int i = 0; i < N_NEURONS; ++i) {
52         G4double xi = static_cast<G4double>(i) / N_NEURONS;
53         G4double sumEdepMeV = fSumEdep[i] / CLHEP::MeV;
54         G4double meanEdepKeV = (fNEvents > 0)
55             ? (fSumEdep[i] / CLHEP::keV) / static_cast<G4double>(fNEvents) : 0.;
56         G4double hitFraction = (fNEvents > 0)
57             ? static_cast<G4double>(fSumPrimaryHits[i]) / static_cast<G4double>(fNEvents) : 0.;
58         // LET dose-mediato = Sum(dE_i^2/dx_i) / Sum(dE_i) (Kanai et al. 1999,
59         // lo stesso concetto usato a NIRS/HIMAC per il modello RBE clinico:
60         // pesa ogni step per il proprio deposito di energia).
61         G4double doseAvgLET = (fSumEdep[i] > 0.)
62             ? (fSumEdepSqOverDx[i] / fSumEdep[i]) / (CLHEP::keV / CLHEP::um) : 0.;
63         // Sezione 5.44: danno DSB atteso per neurone (stima, non una nuova
64         // simulazione -- riusa i tassi validati della 5.43 per bin di energia).
65         out << i << ", " << xi << ", " << sumEdepMeV << ", " << meanEdepKeV << ", "
66             << fSumPrimaryHits[i] << ", " << hitFraction << ", " << doseAvgLET << ", "
67             << fSumSecondaryElectronCount[i] << ", " << fSumEstimatedDSB[i] << "\n";
68     }
69     out.close();
70
71     // Spettro degli elettroni secondari nati dentro un neurone (sezione
72     // 5.39): un'energia cinetica (keV) per riga, cosi' come creati --
73     // input per valutare l'accoppiamento con Geant4-DNA.
74     std::ostream specName;
75     specName << "secondary_electron_spectrum_run" << run->GetRunID() << ".csv";
76     std::ofstream specOut(specName.str());
77     specOut << "# n_events=" << fNEvents << " n_secondary_electrons="
78             << fSecondaryElectronSpectrum.size() << "\n";
79     specOut << "kinetic_energy_keV\n";
80     for (G4double e : fSecondaryElectronSpectrum) {
81         specOut << (e / CLHEP::keV) << "\n";
82     }
83     specOut.close();
84 }

```

```

1  #include "SteppingAction.hh"
2  #include "EventAction.hh"
3
4  #include "G4Step.hh"
5  #include "G4StepPoint.hh"
6  #include "G4Track.hh"
7  #include "G4VProcess.hh"
8  #include "G4SystemOfUnits.hh"
9  #include "G4ParticleDefinition.hh"
10 #include "G4LogicalVolume.hh"
11 #include "G4VPhysicalVolume.hh"
12 #include "G4VTouchable.hh"
13 #include "G4TouchableHandle.hh"
14 #include "G4RunManager.hh"
15 #include "G4Event.hh"
16 #include <cstdlib>
17
18 SteppingAction::SteppingAction(EventAction* eventAction)

```

```

19 : G4UserSteppingAction(), fEventAction(eventAction)
20 {}
21
22 SteppingAction::~SteppingAction() {}
23
24 void SteppingAction::UserSteppingAction(const G4Step* step)
25 {
26     const G4StepPoint* preStep = step->GetPreStepPoint();
27     const G4VTouchable* touchable = preStep->GetTouchable();
28     const G4VPhysicalVolume* volume = touchable->GetVolume();
29     if (!volume || volume->GetName() != "Neuron") return;
30
31     const G4int neuronIndex = touchable->GetCopyNumber();
32
33     const G4double edep = step->GetTotalEnergyDeposit();
34     if (edep > 0.) fEventAction->AddEdep(neuronIndex, edep, step->GetStepLength());
35
36     // Diagnostica temporanea (sezione 5.36): stampa ogni step con
37     // deposito >0 nel neurone indicato da CNAO_DEBUG_NEURON, per
38     // identificare la particella/processo responsabile di un outlier
39     // di LET -- stesso pattern usato per il bug del geantino.
40     static const char* dbgEnv = std::getenv("CNAO_DEBUG_NEURON");
41     if (dbgEnv && neuronIndex == std::atoi(dbgEnv) && edep > 0.) {
42         const G4Track* track = step->GetTrack();
43         const G4VProcess* proc = step->GetPostStepPoint()->GetProcessDefinedStep();
44         G4cout << "[CNAO_DEBUG_NEURON] event=" << G4RunManager::GetRunManager()->GetCurrentEvent
45             (->GetEventID()
46             << " particella=" << track->GetParticleDefinition()->GetParticleName()
47             << " parentID=" << track->GetParentID()
48             << " trackID=" << track->GetTrackID()
49             << " processo=" << (proc ? proc->GetProcessName() : "N/A")
50             << " edep_keV=" << edep/CLHEP::keV
51             << " stepLength_um=" << step->GetStepLength()/CLHEP::um
52             << " KE_MeV=" << track->GetKineticEnergy()/CLHEP::MeV
53             << G4endl;
54     }
55
56     // ParentID==0 identifies the primary proton/ion track itself
57     // (as opposed to any secondary electron, delta-ray, or nuclear
58     // fragment produced along the way).
59     const G4Track* track = step->GetTrack();
60     if (track->GetParentID() == 0) {
61         fEventAction->MarkPrimaryHit(neuronIndex);
62     }
63
64     // Sezione 5.39: registra l'energia cinetica di ogni elettrone
65     // secondario NATO dentro questo neurone (primo step del track,
66     // pre-step point ancora nel volume "Neuron") -- lo spettro
67     // necessario per valutare un accoppiamento con Geant4-DNA (valido
68     // solo fino a ~1 MeV), invece di assumerlo senza verificarlo.
69     if (track->GetParentID() != 0
70         && track->GetParticleDefinition()->GetParticleName() == "e-"
71         && track->GetCurrentStepNumber() == 1) {
72         fEventAction->AddSecondaryElectron(neuronIndex, preStep->GetKineticEnergy());
73     }
74 }

```

5.40 Nanodosimetric link completed: real ICSD from CNAO secondary electrons

Closes the link begun in 5.39. The REAL energies from the secondary-electron spectrum verified there (median 1.86 keV, 90th percentile 7.33 keV) were used as primaries in nanoICSD (the twin project reproducing the Villagrasa/Baiocco

paper's ICSD test case), in place of the paper's arbitrarily chosen monoenergetic values. Target diameter chosen following the same convention as the paper (8nm below $\sim 1\text{-}5$ keV, 100nm above): 8nm for the median (1860 eV), 100nm for the 90th percentile (7332 eV).

Important preliminary note: nanoICSD had never before been compiled or run in any environment available so far (its own README explicitly stated this) -- run successfully for the first time in this section, on the same Geant4 installation used for cnaoRingImpact throughout the project.

Configuration	M1 (mean ionizations)	F2	F3	N events
Real median, 1860 eV, 8nm	1.090	0.257	0.142	100000
Real 90th percentile, 7332 eV, 100nm	6.320	0.801	0.670	100000

Comparison with the grid already published in the paper (same physics option, same target diameter at each energy): at the 8nm size, the paper shows M1 DECREASING with increasing energy over this range (300eV->5.35, 600eV->2.90, 1000eV->1.83) -- our real data point at 1860eV (M1=1.09) continues exactly this same decreasing trend. At the 100nm size, the paper shows 5000eV->7.10 and 10000eV->4.02 -- our real data point at 7332eV (M1=6.32) falls exactly BETWEEN these two published values, as expected from a physically sensible interpolation along the same curve.

Interpretation: the secondary electrons actually produced by a CNAO beam inside a neuron produce nanodosimetric ionization clusters (ICSD) fully consistent with those already characterized in the Villagrasa/Baiocco paper -- no surprise, no artifact, the values land exactly where one would expect along the already-published curves. The multi-scale link between the macroscopic physics of the clinical beam (Geant4, QBBC, this project) and the nanodosimetric physics of local damage (Geant4-DNA, geant4dna_icsd) is thus completed and verified end-to-end: macroscopic dose -> real secondary-electron spectrum -> nanodosimetric ICSD consistent with the literature -- the entire chain, not just the first or last link.

Run with nanoICSD (geant4dna_icsd/build/nanoICSD), energies taken from the real spectrum of Section 5.39

5.41 Monte Carlo verification of nanoICSD: unlike the dose map, the ICSDs are stable

A natural question after 5.40, for the same reason that led to Section 5.32's discovery: nanoICSD uses the same system-clock-based seed pattern that cnaoRingImpact used before its fix. Is the M1=1.090 value found for the real median energy (1860 eV) a solid figure or a single noisy run?

An explicit seed parameter was added to nanoICSD.cc (the same fix as Section 5.32), and the same configuration (1860 eV, 8nm, option 2) was repeated over 4 independent seeds (201-204), 100,000 events each. Result: M1 = 1.0949, 1.0892, 1.0954, 1.0883 -- coefficient of variation only 0.34% (mean 1.0920, std.dev 0.0037). F2 even more stable (CV 0.14%). The single-run value from Section 5.40 (M1=1.090) is confirmed, not the artifact of a lucky seed.

Unlike the per-neuron dose map (5.32), there is NO statistical-noise problem here -- and the reason is mechanistically clear: in the dose case, 500,000 events were distributed across 100 separate neurons, giving sparse statistics for each (~1-2 hits expected per bin). Here, by contrast, every single event (100,000 per run) contributes DIRECTLY to the aggregate M1/F2/F3 statistics -- no splitting into separate bins, hence enormously better effective statistics for the same sample size.

Conclusion: the nanodosimetric link completed in 5.39-5.40 rests on solid statistical grounds, not on a single unverified run -- a rigor check done for completeness (the same principle applied throughout this project), which in this case confirms the result instead of correcting it, unlike what happened with the physical dose map.

Verified with nanoICSD (explicit seeds 201-204), geant4dna_icsd/build/results_mc_seed_check/

5.42 Beyond ICSD: real DNA damage (SSB/DSB) from CNAO secondary electrons

Closes the point left open in Chapter 9 (future developments): extending the already-verified physical chain (macroscopic dose -> secondary-electron spectrum -> nanodosimetric ICSD, Sections 5.39-5.41) all the way to direct DNA damage -- single- and double-strand breaks (SSB/DSB).

First attempt, with the official Geant4-DNA example dnadamage1: blocked by a real dependency, not quickly bypassable -- it requires a DNA geometry file (VoxelStraight.fab2g4dna) generated by an external tool (DnaFabric) that is unavailable and cannot be quickly installed. Verified, not merely assumed, before abandoning this path.

Alternative found and verified working: the more recent official example molecularDNA (Geant4-DNA collaboration), which automatically downloads its own reference geometry (200,000 DNA segments of 216 base pairs in a 100x30x100nm

volume, "cylinders" geometry for parametric studies) with no external tools needed. Successfully compiled (never before tested in any available environment) and verified with a trial run before using it in earnest -- it returns complete damage categories (SSB, proximal SSB, double-SSB, simple and complex DSB) per event, with energy and position.

Run with the real energies from the CNAO secondary-electron spectrum (Section 5.39): median 1860 eV (10,000 primaries) and 90th percentile 7332 eV (5,000 primaries).

Real energy	N primaries	Single breaks (SB)	Double breaks (DSB)	SB/primary	DSB/primary	fraction
1860 eV (median)	10000	1055	152	0.1055	0.0152	12.6%
7332 eV (p90)	5000	2688	215	0.5376	0.0430	7.4%

A non-trivial physical observation: the higher-energy electron (7332 eV) produces more absolute damage (longer track, more total ionizations), but a RELATIVELY smaller fraction of that damage consists of double breaks (7.4% against 12.6%). An explanation consistent with known physics: an electron's LET decreases as its energy increases in this range -- the slower electron (median, higher LET) deposits its energy more concentrated along the path, favoring the formation of closely spaced breaks on both strands (DSB); the faster electron (p90, lower LET) spreads similar energy over a longer path, with more scattered, and therefore relatively less complex, damage.

Monte Carlo verification (the same principle as Sections 5.32/5.41, here extended to a third tool -- molecularDNA also never called an explicit seed in its original code, fixed by adding a -s option): the median configuration was repeated over 3 independent seeds (5,000 events each). SB stable (CV 1.8%: 522, 514, 504); DSB noisier (CV 12.4%: 60, 77, 68), as expected for much smaller counts (Poisson scaling). The average of the 3 seeds (SB/primary=0.1027, DSB/primary=0.0137) is consistent within statistical noise with the original single run at 10,000 events (0.1055, 0.0152) -- no systematic discrepancy.

Stated limitation: the "cylinders" geometry is a parametric study volume (200,000 short DNA segments in a 100x30x100nm placement region), not an entire cell nucleus -- these are relative yields per primary within that defined volume, not an absolute prediction of whole-cell damage. Only two energy points tested, not the full continuous spectrum of secondary electrons weighted by its true distribution (Section 5.39). Both limitations are the same kind of honest disclosure already applied throughout this project -- a working, verified first level, not the final word.

Run with molecularDNA (Geant4-DNA collaboration), geant4dna_ssbsb/build/, energies from the real spectrum of Section 5.39

5.43 DNA damage weighted over the entire real spectrum, not just two points

Closes a limitation explicitly stated in Section 5.42: only two energies (median and 90th percentile) represented the entire real spectrum of 18,021 secondary electrons. Here: 6 representative points (percentiles 10, 25, 50, 75, 90, 99 of the spectrum verified in 5.39), each weighted by the real fraction of electrons it represents (a 1D Voronoi partition around the chosen points), to obtain a damage yield weighted over the entire spectrum rather than just two samples.

Energy (keV)	Spectrum weight	SB/primary	DSB/primary	DSB fraction
1.09	18.0%	0.0550	0.0103	15.7%
1.28	21.7%	0.0715	0.0083	10.3%
1.86	26.5%	0.1027	0.0137	11.7%
(median)				
3.43	19.2%	0.2213	0.0243	9.9%
7.33 (p90)	12.0%	0.5376	0.0430	7.4%
28.82 (p99)	2.7%	0.2863	0.0163	5.4%

The double-break (DSB) fraction decreases nearly monotonically across the entire spectrum (15.7% -> 10.3% -> 11.7% -> 9.9% -> 7.4% -> 5.4%), confirming across 6 independent points, not just 2, the physical explanation already proposed in 5.42: LET decreases with electron energy in this range, and a lower LET spreads damage over a longer path instead of concentrating it, reducing the probability of closely spaced breaks on both strands.

An anomaly noted, not hidden: at the highest point (28.82 keV, only 800 events -- the lowest statistics among the 6 points, given the growing computational cost with energy), the absolute SB/primary value (0.286) is LOWER than at 7.33 keV (0.538), non-monotonic in absolute value (unlike the DSB fraction, which stays monotonic). A physically plausible explanation: the range of a 28.82 keV electron can approach or exceed the extent of the DNA geometry used (100x30x100nm) -- a growing share of its energy risks being deposited OUTSIDE the sensitive volume, reducing the fraction of its total energy that actually contributes to recorded damage. A component of pure statistical noise is not excluded, given this single point's low statistics (its spectrum weight is small anyway, 2.7%, so the effect on the final weighted average is limited).

Final result, weighted average over the entire real spectrum: SB/primary = 0.1674, DSB/primary = 0.01753, DSB fraction = 9.5% -- a single, more representative estimate, compared to the two extremes reported in 5.42 (12.6% at

the median alone, 7.4% at the 90th percentile alone). This is the number to cite as the overall damage yield for a typical secondary electron produced by a CNAO carbon-12 beam at the Bragg peak, within molecularDNA's reference geometry.

Run with molecularDNA (geant4DNA_ssbdsb/build/results_spectrum_weighted/), 6 points weighted over the real spectrum of Section 5.39

5.44 The physics-neuroscience bridge closed down to the DNA: physical dose remains a robust criterion

Sections 5.39-5.43 built and verified the entire nanodosimetric chain (secondary-electron spectrum -> ICSD -> real DNA damage), but only at the AGGRE-GATE level across the whole ring -- never linked neuron by neuron to the knock-out criterion used in Sections 5.22-5.38 (always based on physical dose). This section closes that link: geant4_cnao_ring's tracking was extended to record, for each of the 100 neurons, how many secondary electrons are born inside it and an estimate of the expected DSB damage (assigning each electron the DSB/primary rate of the nearest of the 6 representative points from Section 5.43 -- an estimate that reuses already-validated rates, not a new per-neuron damage simulation).

The carbon-12-at-the-Bragg-peak configuration was repeated (500,000 events): 90/100 neurons with nonzero physical dose, 89/100 with at least one tracked secondary electron. Comparison between the ranking of the 10 highest-physical-dose neurons and the 10 highest-estimated-DSB-damage neurons: 9/10 overlap -- only one neuron changes rank. Consistent with what was already found for the RBE refinement (Sections 5.31, 5.34, 5.35): within the same depth, weighting by a deeper biological criterion (all the way to molecular DNA damage, not just deposited energy) does not substantially change which neurons come out hardest hit.

Stated methodological caution: this comparison uses physical dose and secondary electrons computed in the SAME Monte Carlo realization (same run, same seed) -- it is not a substitute for the multi-seed verification of Section 5.32/5.33 (which showed that the ranking of the 10 hardest-hit neurons is itself noisy from one run to another). What this section shows is that, GIVEN a particular physical realization, the derived criterion (DNA damage) does not diverge substantially from the simpler criterion (physical dose) it is built from -- not that both criteria are themselves stable across different runs.

With this section, the bridge between the physics thread (5.21-5.43) and the neuroscience thread (5.6-5.20, first linked in 5.22) is verified end-to-end across the full depth possible in this project: from the clinical CNAO beam, through physical dose, the clinical RBE model, the secondary-electron spectrum, nanodosimetric damage, all the way to molecular DNA damage -- and at every one of these steps, the simplest criterion (physical dose), used from the start

to select which neurons to silence, proved to be a robust proxy, not a crude approximation overturned by deeper analysis.

Source code: `geant4_cnao_ring` (`EventAction.cc/RunAction.cc`, per-neuron tracking added in this section)

```

1  #include "EventAction.hh"
2  #include "RunAction.hh"
3  #include "G4SystemOfUnits.hh"
4  #include <cmath>
5
6  namespace {
7  // Sezione 5.43: 6 punti rappresentativi dello spettro reale (percentili
8  // 10/25/50/75/90/99), con il tasso DSB/primario validato per ciascuno
9  // (moleculardna, geant4dna_ssbdsb). Riuserà qui, non ricalcolati.
10 constexpr G4int kNBins = 6;
11 constexpr G4double kBinEnergyKeV[kNBins] = {1.09, 1.28, 1.86, 3.43, 7.33, 28.82};
12 constexpr G4double kBinDSBRate[kNBins] = {0.0103, 0.0083, 0.0137, 0.0243, 0.0430, 0.0163};
13
14 G4double DSBRateForEnergy(G4double keV)
15 {
16     G4int best = 0;
17     G4double bestDiff = std::abs(keV - kBinEnergyKeV[0]);
18     for (G4int i = 1; i < kNBins; ++i) {
19         G4double diff = std::abs(keV - kBinEnergyKeV[i]);
20         if (diff < bestDiff) { bestDiff = diff; best = i; }
21     }
22     return kBinDSBRate[best];
23 }
24 } // namespace
25
26 EventAction::EventAction(RunAction* runAction)
27 : G4UserEventAction(), fRunAction(runAction)
28 {}
29
30 EventAction::~EventAction() {}
31
32 void EventAction::BeginOfEventAction(const G4Event*)
33 {
34     fEdep.fill(0.);
35     fEdepSqOverDx.fill(0.);
36     fPrimaryHit.fill(false);
37     fSecondaryElectronEnergies.clear();
38     fSecondaryElectronCount.fill(0);
39     fEstimatedDSB.fill(0.);
40 }
41
42 void EventAction::AddEdep(G4int neuronIndex, G4double edep, G4double stepLength)
43 {
44     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
45     fEdep[neuronIndex] += edep;
46     if (stepLength > 0.) fEdepSqOverDx[neuronIndex] += edep * edep / stepLength;
47 }
48
49 void EventAction::MarkPrimaryHit(G4int neuronIndex)
50 {
51     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
52     fPrimaryHit[neuronIndex] = true;
53 }
54
55 void EventAction::AddSecondaryElectron(G4int neuronIndex, G4double kineticEnergy)
56 {
57     if (neuronIndex < 0 || neuronIndex >= N_NEURONS) return;
58     fSecondaryElectronEnergies.push_back(kineticEnergy);
59     fSecondaryElectronCount[neuronIndex] += 1;
60     fEstimatedDSB[neuronIndex] += DSBRateForEnergy(kineticEnergy / CLHEP::keV);
61 }

```

```

62 void EventAction::EndOfEventAction(const G4Event*)
63 {
64     fRunAction->RecordEvent(fEdep, fEdepSqOverDx, fPrimaryHit, fSecondaryElectronEnergies,
65                             fSecondaryElectronCount, fEstimatedDSB);
66 }
67

```

```

1  #include "RunAction.hh"
2
3  #include "G4Run.hh"
4  #include "G4SystemOfUnits.hh"
5  #include <sstream>
6
7  RunAction::RunAction() : G4UserRunAction() {}
8
9  RunAction::~RunAction() {}
10
11 void RunAction::BeginOfRunAction(const G4Run*)
12 {
13     fSumEdep.fill(0.);
14     fSumEdepSqOverDx.fill(0.);
15     fSumPrimaryHits.fill(0);
16     fNEvents = 0;
17     fSecondaryElectronSpectrum.clear();
18     fSumSecondaryElectronCount.fill(0);
19     fSumEstimatedDSB.fill(0.);
20 }
21
22 void RunAction::RecordEvent(const std::array<G4double, N_NEURONS>& edep,
23                             const std::array<G4double, N_NEURONS>& edepSqOverDx,
24                             const std::array<G4bool, N_NEURONS>& primaryHit,
25                             const std::vector<G4double>& secondaryElectronEnergies,
26                             const std::array<G4long, N_NEURONS>& secondaryElectronCount,
27                             const std::array<G4double, N_NEURONS>& estimatedDSB)
28 {
29     for (G4int i = 0; i < N_NEURONS; ++i) {
30         fSumEdep[i] += edep[i];
31         fSumEdepSqOverDx[i] += edepSqOverDx[i];
32         if (primaryHit[i]) fSumPrimaryHits[i] += 1;
33         fSumSecondaryElectronCount[i] += secondaryElectronCount[i];
34         fSumEstimatedDSB[i] += estimatedDSB[i];
35     }
36     fNEvents += 1;
37     for (G4double e : secondaryElectronEnergies) {
38         fSecondaryElectronSpectrum.push_back(e);
39     }
40 }
41
42 void RunAction::EndOfRunAction(const G4Run* run)
43 {
44     std::ostringstream fname;
45     fname << "neuron_dose_run" << run->GetRunID() << ".csv";
46     std::ofstream out(fname.str());
47     out << "# n_events=" << fNEvents << "\n";
48     out << "neuron_index,xi,sum_edep_MeV,mean_edep_keV_per_event,"
49           "primary_hit_count,primary_hit_fraction,dose_averaged_LET_keV_per_um,"
50           "secondary_electron_count,estimated_DSB\n";
51     for (G4int i = 0; i < N_NEURONS; ++i) {
52         G4double xi = static_cast<G4double>(i) / N_NEURONS;
53         G4double sumEdepMeV = fSumEdep[i] / CLHEP::MeV;
54         G4double meanEdepKeV = (fNEvents > 0)
55             ? (fSumEdep[i] / CLHEP::keV) / static_cast<G4double>(fNEvents) : 0.;
56         G4double hitFraction = (fNEvents > 0)
57             ? static_cast<G4double>(fSumPrimaryHits[i]) / static_cast<G4double>(fNEvents) : 0.;
58         // LET dose-mediato = Sum(dE_i^2/dx_i) / Sum(dE_i) (Kanai et al. 1999,
59         // lo stesso concetto usato a NIRS/HIMAC per il modello RBE clinico:
60         // pesa ogni step per il proprio deposito di energia).

```

```

61      G4double doseAvgLET = (fSumEdep[i] > 0.)
62      ? (fSumEdepSqOverDx[i] / fSumEdep[i]) / (CLHEP::keV / CLHEP::um) : 0.;
63      // Sezione 5.44: danno DSB atteso per neurone (stima, non una nuova
64      // simulazione -- riusa i tassi validati della 5.43 per bin di energia).
65      out << i << ", " << xi << ", " << sumEdepMeV << ", " << meanEdepKeV << ", "
66      << fSumPrimaryHits[i] << ", " << hitFraction << ", " << doseAvgLET << ", "
67      << fSumSecondaryElectronCount[i] << ", " << fSumEstimatedDSB[i] << "\n";
68  }
69  out.close();
70
71  // Spettro degli elettroni secondari nati dentro un neurone (sezione
72  // 5.39): un'energia cinetica (keV) per riga, cosi' come creati --
73  // input per valutare l'accoppiamento con Geant4-DNA.
74  std::ostream specName;
75  specName << "secondary_electron_spectrum_run" << run->GetRunID() << ".csv";
76  std::ofstream specOut(specName.str());
77  specOut << "# n_events=" << fNEvents << " n_secondary_electrons="
78      << fSecondaryElectronSpectrum.size() << "\n";
79  specOut << "kinetic_energy_keV\n";
80  for (G4double e : fSecondaryElectronSpectrum) {
81      specOut << (e / CLHEP::keV) << "\n";
82  }
83  specOut.close();
84  }

```

5.45 The C->A loop does not close: the three-ring relay is not self-sustaining

Returns to the pure-neuroscience thread, closing a question left open by Section 5.20: the directional relay A->B->C->A (three populations, unidirectional links only with STDP, so none of the mutual-feedback instabilities encountered with true bidirectionality can arise by construction) correctly propagates position information from A all the way to C after two hops -- but does the closing link C->A transmit CAUSALLY, capable of sustaining an echo of the message even without further external stimulus on A?

Protocol: 80 normal training repetitions (external stimulus on A only, as in 5.20), followed by 10 consecutive PROBE cycles with no external stimulus on any ring. If C->A is causally effective, A should show stable, late activity in EVERY one of the 10 probe cycles (an echo self-sustained by the loop); if instead the apparent closure were an artifact or pure residual persistence of A itself, activity should die out within the first 1-2 cycles and not reappear.

Result, verified over 5 independent runs (1 single run + 4 Monte Carlo seeds): never more than 3 of 10 probes show any activity in A at all, and when they do it is always 1-2 isolated spikes -- background network noise (a baseline 400 Hz Poisson stimulus on every neuron), not a recognizable signal. None of the 5 runs shows the expected signature of a causal loop (stable, localized activity in EVERY probe cycle).

single seed (original run): 1/10 probes with spikes (only the first, 2 spikes); seed 1: 3/10 (1 spike each); seed 2: 0/10; seed 3: 2/10 (1 spike each); seed 4: 0/10.

Conclusion: the C->A link does not close a self-sustaining causal loop in this architecture -- consistent with, and an extension of, the conclusion already reached

in Section 5.9 for the simpler two-ring case (the B->A return channel does not transmit causally in this system). Even with a third intermediate ring and a longer return path, the same structural limitation reappears: unidirectional forward propagation (A->B->C) works reliably, but no return path tested so far in this project -- neither direct (B->A, 5.9) nor through an intermediate ring (C->A, here) -- proves causally effective without the explicit stabilization (transmission switch, homeostatic normalization) developed for true bidirectionality in Sections 5.13-5.17.

Source code: `dialogo_spiking_relay_chiusura_turni_mc.py`

```

1  """
2  Prima estensione della comunicazione spiking oltre la coppia: tre
3  popolazioni (A, B, C) invece di due. Mai tentato ne' nel modello a rate
4  ne' in quello spiking in questo progetto.
5
6  Per evitare di riaprire l'intera saga di stabilizzazione delle sezioni
7  5.8-5.19 (necessaria solo per la bidirezionalita' vera, dove due anelli
8  sono mutuamente eccitatori), questa prima rete a tre popolazioni usa
9  SOLO collegamenti unidirezionali con STDP vera -- esattamente il
10 meccanismo gia' collaudato e stabile di dialogo_spiking.py (sezione 5.7),
11 ripetuto tre volte per formare una STAFFETTA DIREZIONALE ad anello:
12 A->B->C->A. Nessun anello riceve MAI input da due sorgenti
13 contemporaneamente (ogni anello ha esattamente un collegamento in
14 ingresso, con STDP, e uno in uscita), quindi nessuna delle instabilita'
15 di retroazione mutua incontrate con la vera bidirezionalita' puo'
16 presentarsi qui per costruzione.
17
18 Domanda sperimentale: un messaggio iniettato SOLO in A (mai direttamente
19 in B o C) si propaga correttamente attraverso la catena, preservando
20 l'informazione di posizione dopo due salti (in C) e persino dopo il giro
21 completo (di nuovo in A, tramite C->A)?
22 """
23
24 import sys
25 from brian2 import *
26
27 SEED = int(sys.argv[1])
28 start_scope()
29 seed(SEED)
30
31 N_E = 100
32 N_I = 25
33 N_ring = N_E + N_I
34 N_POP = 3 # A, B, C
35
36 a = 0.02/ms
37 b = 0.2/ms
38 c = -65*mV
39 d = 6*mV/ms
40 tau_nmda = 100*ms
41
42 eqs = '''
43 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + I + g_nmda : volt
44 du/dt = a*(b*v - u) : volt/second
45 dg_nmda/dt = -g_nmda/tau_nmda : volt/second
46 I : volt/second
47 x : 1
48 '''
49
50 neurons = NeuronGroup(N_POP*N_ring, eqs, threshold='v > 30*mV', reset='v = c; u += d',
51                       method='euler')
52 neurons.v = -65*mV + 5*mV*randn(N_POP*N_ring)
53 neurons.u = b * neurons.v[:]
54 neurons.I = 0*mV/ms

```

```

55 | neurons.g_nmda = 0*mV/ms
56 |
57 | def sotto(pop_idx):
58 |     base = pop_idx * N_ring
59 |     return neurons[base:base+N_E], neurons[base+N_E:base+N_ring]
60 |
61 | A_E, A_I = sotto(0)
62 | B_E, B_I = sotto(1)
63 | C_E, C_I = sotto(2)
64 |
65 | xi = np.arange(N_E) / N_E
66 | A_E.x = xi
67 | B_E.x = xi
68 | C_E.x = xi
69 |
70 | background = PoissonGroup(N_POP*N_ring, rates=400*Hz)
71 | bg_syn = Synapses(background, neurons, on_pre='v_post += 2.0*mV')
72 | bg_syn.connect(j='i')
73 |
74 | sigma_exc = 0.05
75 | CUTOFF_LOCALE = 3 * sigma_exc
76 | w_ee_nmda = 0.325*mV/ms
77 | p_conn = 0.15
78 | w_exc = 1.0*mV
79 | w_inh = 10.0*mV
80 |
81 |
82 | def dist_circ(dx):
83 |     dx = abs(dx)
84 |     return np.minimum(dx, 1 - dx)
85 |
86 |
87 | def costruisci_anello(E, I_pop, seed):
88 |     dmat = dist_circ(xi[:, None] - xi[None, :])
89 |     prob_ee = np.exp(-dmat**2 / (2*sigma_exc**2))
90 |     prob_ee[dmat > CUTOFF_LOCALE] = 0.0
91 |     np.fill_diagonal(prob_ee, 0)
92 |     rng = np.random.RandomState(seed)
93 |     mask_ee = rng.rand(N_E, N_E) < prob_ee
94 |     i_idx, j_idx = np.nonzero(mask_ee)
95 |
96 |     syn_ee = Synapses(E, E, on_pre='g_nmda_post += w_ee_nmda')
97 |     syn_ee.connect(i=i_idx, j=j_idx)
98 |     syn_ei = Synapses(E, I_pop, on_pre='v_post += w_exc')
99 |     syn_ei.connect(p=p_conn)
100 |     syn_ie = Synapses(I_pop, E, on_pre='v_post -= w_inh')
101 |     syn_ie.connect(p=p_conn)
102 |     syn_ii = Synapses(I_pop, I_pop, on_pre='v_post -= w_inh')
103 |     syn_ii.connect(condition='i!=j', p=p_conn)
104 |     return syn_ee, syn_ei, syn_ie, syn_ii
105 |
106 |
107 | syn_A = costruisci_anello(A_E, A_I, seed=1000+SEED)
108 | syn_B = costruisci_anello(B_E, B_I, seed=2000+SEED)
109 | syn_C = costruisci_anello(C_E, C_I, seed=3000+SEED)
110 |
111 | tau_stdp = 20*ms
112 | A_ltp = 0.15*mV
113 | A_ltd = 0.15*mV
114 | w_max = 5.0*mV
115 | w_min = 0*mV
116 | w_init = 1.5*mV
117 | sigma_inter = 0.08
118 | p_inter_picco = 0.5
119 |
120 | stdp_model = ''
121 | w : volt
122 | dapre/dt = -apre/tau_stdp : volt (event-driven)

```

```

123 dapost/dt = -apost/tau_stdp : volt (event-driven)
124 '''
125 stdp_on_pre = '''
126 v_post += w
127 apre += A_ltp
128 w = clip(w+apost, w_min, w_max)
129 '''
130 stdp_on_post = '''
131 apost -= A_ltd
132 w = clip(w+apre, w_min, w_max)
133 '''
134
135
136 def collega(SRC_E, DST_E, seed_conn, w_iniziale):
137     dmat_inter = dist_circ(xi[:, None] - xi[None, :])
138     prob_inter = p_inter_picco * np.exp(-dmat_inter**2 / (2*sigma_inter**2))
139     rng_inter = np.random.RandomState(seed_conn)
140     mask_inter = rng_inter.rand(N_E, N_E) < prob_inter
141     i_inter, j_inter = np.nonzero(mask_inter)
142     syn = Synapses(SRC_E, DST_E, model=stdp_model, on_pre=stdp_on_pre, on_post=stdp_on_post)
143     syn.connect(i=i_inter, j=j_inter)
144     syn.w = w_iniziale
145     return syn
146
147
148 # A->B parte dal valore originale di 5.7 (A ha uno stimolo esterno diretto
149 # forte, si innesca facilmente). B->C e C->A partono da un peso piu' alto:
150 # la loro sorgente (B, poi C) e' essa stessa un eco di secondo/terzo
151 # livello, con un segnale molto piu' debole e sparso di quello diretto su
152 # A -- lo stesso identico problema "uovo-gallina" gia' risolto in 5.7 per
153 # A->B (tentativo 1: w_init=0.3mV, B non si accendeva mai), qui riapplicato
154 # ai salti successivi della staffetta
155 syn_ab = collega(A_E, B_E, seed_conn=4000+SEED, w_iniziale=w_init)
156 syn_bc = collega(B_E, C_E, seed_conn=5000+SEED, w_iniziale=3.0*mV)
157 syn_ca = collega(C_E, A_E, seed_conn=6000+SEED, w_iniziale=3.0*mV)
158 print(f"Staffetta A->B->C->A creata: A->B {len(syn_ab)} sinapsi, "
159       f"B->C {len(syn_bc)} sinapsi, C->A {len(syn_ca)} sinapsi")
160
161 spikes = SpikeMonitor(neurons)
162
163 MESSAGGIO = 0.3
164 larghezza_stimolo = 0.05
165 peso_stim = np.exp(-dist_circ(xi - MESSAGGIO)**2 / (2*larghezza_stimolo**2))
166 gruppo_stim = np.where(peso_stim > 0.5)[0]
167 print(f"Neuroni stimolati SOLO in A (vicino a x={MESSAGGIO}): {len(gruppo_stim)}")
168
169 N_RIPETIZIONI_NORMALI = 80 # allena A->B->C come nella sezione 5.20
170 N_SONDE_SILENZIOSE = 10 # poi, cicli CONSECUTIVI senza alcuno stimolo
171 # su A -- se C->A e' causale, A dovrebbe
172 # mostrare attivita' tardiva stabile in
173 # OGNIUNO di questi cicli (non un semplice
174 # decadimento della propria persistenza,
175 # che invece si esaurirebbe entro 1-2 cicli)
176
177 T_STIM = 60*ms
178 T_PAUSA = 440*ms # piu' lunga di 5.7 (340ms) per dare tempo alla staffetta
179 # di propagare fino a C (due salti) prima del ciclo dopo
180 N_RIPETIZIONI = N_RIPETIZIONI_NORMALI + N_SONDE_SILENZIOSE
181
182 print(f"\nTraining: {N_RIPETIZIONI_NORMALI} ripetizioni normali (stimolo su A), "
183       f"poi {N_SONDE_SILENZIOSE} cicli sonda SENZA alcuno stimolo esterno "
184       f"({(T_STIM+T_PAUSA)*N_RIPETIZIONI/ms:.0f}ms totali)")
185
186 tipo_ciclo = [] # 'normale' o 'sonda', per ciclo
187 for rep in range(N_RIPETIZIONI):
188     if rep < N_RIPETIZIONI_NORMALI:
189         tipo_ciclo.append('normale')
190         A_E.I[gruppo_stim] = 15*mV/ms
191         run(T_STIM)

```

```

191     A_E.I[:] = 0*mV/ms
192     else:
193         tipo_ciclo.append('sonda')
194         run(T_STIM) # nessuno stimolo esterno su nessun anello in questo ciclo
195         run(T_PAUSA)
196         if rep % 10 == 0 or rep == N_RIPETIZIONI-1 or tipo_ciclo[-1] == 'sonda':
197             print(f"ripetizione {rep:3d}/{N_RIPETIZIONI} ({tipo_ciclo[-1]}): "
198                   f"A->B={np.mean(syn_ab.w)/mV:.2f}mV B->C={np.mean(syn_bc.w)/mV:.2f}mV "
199                   f"C->A={np.mean(syn_ca.w)/mV:.2f}mV")
200
201     print("\nTraining completato.")
202
203     fig, axes = subplots(2, 1, figsize=(11, 7))
204     subplot(2, 1, 1)
205     colori = ['C0', 'C3', 'C2']
206     etichette = ['A (E)', 'B (E)', 'C (E)']
207     for k in range(N_POP):
208         base = k*N_ring
209         m = (spikes.i >= base) & (spikes.i < base+N_E)
210         plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=1.5, label=
211              etichette[k])
212     xlabel('Tempo (ms)')
213     ylabel('Indice neurone')
214     title(f'Raster completo: staffetta A->B->C->A, {N_RIPETIZIONI} ripetizioni (x={MESSAGGIO})')
215     legend(loc='upper right', fontsize=8, markerscale=6)
216
217     subplot(2, 1, 2)
218     t_tardi = (N_RIPETIZIONI-6) * (T_STIM+T_PAUSA)/ms
219     for k in range(N_POP):
220         base = k*N_ring
221         m = (spikes.i >= base) & (spikes.i < base+N_E) & (spikes.t/ms > t_tardi)
222         plot(spikes.t[m]/ms, spikes.i[m]-base, '.', color=colori[k], markersize=3, label=
223              etichette[k])
224     xlabel('Tempo (ms)')
225     ylabel('Indice neurone')
226     title('Dettaglio: ultime ripetizioni (fine training)')
227     legend(loc='upper right', fontsize=8, markerscale=4)
228
229     tight_layout()
230     savefig('dialogo_spiking_relay_chiusura_turni_mc.png', dpi=150)
231     print("\nGrafico salvato in dialogo_spiking_relay_chiusura_turni_mc.png")
232
233     ciclo_ms = (T_STIM+T_PAUSA)/ms
234
235     def valuta_pop(pop_idx, n_eval):
236         base = pop_idx * N_ring
237         mask_pop = (spikes.i >= base) & (spikes.i < base+N_E)
238
239         def conta(rep_range):
240             n_tot, posizioni = 0, []
241             for rep in rep_range:
242                 t0 = rep * ciclo_ms
243                 fin = t0 + ciclo_ms
244                 m = mask_pop & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
245                 idx = spikes.i[m] - base
246                 n_tot += m.sum()
247                 posizioni.extend(list(xi[idx]))
248             centro = None
249             if posizioni:
250                 centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(posizioni)))) / (2*np.pi)
251                               % 1.0)
252             return n_tot, centro
253
254     primi = range(0, n_eval)
255     ultimi = range(N_RIPETIZIONI-n_eval, N_RIPETIZIONI)
256     return conta(primi), conta(ultimi)

```



```

256 print("\n" + "="*70)
257 print("VALUTAZIONE: l'informazione di posizione sopravvive alla staffetta?")
258 print("="*70)
259
260 for k, nome in enumerate(['A (sorgente diretta)', 'B (un salto: A->B)', 'C (due salti: A->B
    ->C)']):
261     (n_i, c_i), (n_f, c_f) = valuta_pop(k, 5)
262     err_i = dist_circ(c_i - MESSAGGIO) if c_i is not None else None
263     err_f = dist_circ(c_f - MESSAGGIO) if c_f is not None else None
264     print(f" {nome}:")
265     print(f" prime 5 rip.: {n_i} spike, centro={({ 'N/A' if c_i is None else f'{c_i:.3f}'}}, "
266           f"errore={({ 'N/A' if err_i is None else f'{err_i:.3f}'}})")
267     print(f" ultime 5 rip.: {n_f} spike, centro={({ 'N/A' if c_f is None else f'{c_f:.3f}'}}, "
268           f"errore={({ 'N/A' if err_f is None else f'{err_f:.3f}'}})")
269
270 print(f"\nPesi finali: A->B={np.mean(syn_ab.w)/mV:.3f}mV (iniziale {w_init/mV:.3f}mV), "
271       f"B->C={np.mean(syn_bc.w)/mV:.3f}mV, C->A={np.mean(syn_ca.w)/mV:.3f}mV")
272
273 # --- Domanda centrale dell'esperimento: durante i cicli sonda (nessuno
274 # stimolo esterno su nessun anello), A mostra attivita' tardiva STABILE
275 # in OGNUNO dei 10 cicli (segno di un loop causale C->A che si
276 # autosostiene), o l'attivita' decade entro 1-2 cicli (segno che era
277 # solo persistenza residua di A stessa, non un vero segnale chiuso
278 # dall'anello)? ---
279 print("\n" + "="*70)
280 print("DETTAGLIO PER CICLO SONDA: attivita' di A senza alcuno stimolo esterno")
281 print("="*70)
282 base_A = 0
283 mask_A = (spikes.i >= base_A) & (spikes.i < base_A+N_E)
284 sonde_con_spike = 0
285 sonde_conteggi = []
286 for rep in range(N_RIPETIZIONI_NORMALI, N_RIPETIZIONI):
287     t0 = rep * ciclo_ms
288     fin = t0 + ciclo_ms
289     m = mask_A & (spikes.t/ms >= t0) & (spikes.t/ms < fin)
290     idx = spikes.i[m] - base_A
291     n_tot = m.sum()
292     sonde_conteggi.append(int(n_tot))
293     if n_tot > 0:
294         sonde_con_spike += 1
295         centro = None
296         if n_tot > 0:
297             centro = float(np.angle(np.mean(np.exp(2j*np.pi*np.array(xi[idx]))))) / (2*np.pi) %
298                           1.0)
299         err = dist_circ(centro - MESSAGGIO) if centro is not None else None
300         indice_sonda = rep - N_RIPETIZIONI_NORMALI
301         print(f" sonda {indice_sonda+1:2d}/{N_SONDE_SILENZIOSE}: {n_tot} spike in A, "
302               f"centro={({ 'N/A' if centro is None else f'{centro:.3f}'}}, "
303               f"errore={({ 'N/A' if err is None else f'{err:.3f}'}})")
304
305 print(f"\nMC_RIGA seed={SEED} sonde_con_spike={sonde_con_spike}/{N_SONDE_SILENZIOSE} "
306       f"conteggi={sonde_conteggi}")
307
308 # controllo di stabilita' (per costruzione non dovrebbe mai esplodere,
309 # ma verifichiamo comunque per coerenza con il resto del progetto)
310 n_tot_rete = len(spikes.i)
311 print(f"\nSpike totali sull'intera rete: {n_tot_rete} "
312       f"(atteso migliaia se stabile; milioni se esplosione)")
313 print(">>> ESPLOSIONE <<<" if n_tot_rete > 300000 else ">>> Stabile <<<")

```

6. Biophysical realism and synaptic density: comparison with the Allen Institute

The project's final phase originates from an explicit comparison with the real state of the art in the field (Section 4.5): this project had surpassed the Allen Institute in raw number of simulated neurons (40 million against ~10 million), but with a drastically lower synaptic density per neuron (~31 against ~2,600) and much simpler per-neuron biophysical detail. This section describes the attempt to close both gaps at once.

6.1 The fundamental constraint: memory as a shared budget

The starting point is a direct calculation: the available memory (24 GB) is a FIXED budget that must be shared between "how many neurons" to simulate and "how many synapses per neuron" to maintain. Increasing synaptic density by a factor of N forces the number of neurons to shrink by the same factor N, for the same total available memory.

A first estimate, based on the per-synapse cost already measured in the 40-million-neuron network (~7.8 bytes per synapse, obtained from the difference between the 20M and 40M neuron measurements), suggested that a density of 2,600 synapses/neuron (the Allen Institute level) would be reachable up to about 1 million neurons. This initial estimate turned out to be OPTIMISTIC once the second improvement (conductance-based synapses, Section 6.2) was introduced: the real per-synapse cost with this more realistic model came out at about 70 bytes -- nearly 9 times higher -- reducing the maximum reachable neuron count at the same density to about 118,000.

Synapses/neuron	Bytes/synapse (instantaneous)	Bytes/synapse (conductance)	Max neurons (conductance)
31 (original)	~7.8	n/a	40,000,000 (verified)
500	~7.8	~70	614,000 (estimated)
2,600 (Allen Institute)	~7.8	~70	118,000 (verified)

6.2 Conductance-based synapses: a second axis of realism

The second improvement replaces the synapse model used in every previous experiment of the project -- a simple instantaneous voltage "jump" ($v_{\text{post}} += w$) -- with a CONDUCTANCE-based model, the standard described in computational neuroscience textbooks (e.g. Dayan & Abbott). In this model, each synapse opens an ion channel whose conductance decays exponentially over

time, and the effect on the target neuron depends on the "driving force" -- the difference between the current membrane potential and the reversal potential specific to the synapse type (0 mV for excitatory AMPA-type synapses, -80 mV for inhibitory GABA-A-type ones):

$$dv/dt = \dots + g_e \cdot (E_e - v) + g_i \cdot (E_i - v) + I_{external}$$

$$dg_e/dt = -g_e/\tau_e \quad dg_i/dt = -g_i/\tau_i$$

This is more realistic than the simple voltage jump for a precise reason: the effect of an excitatory synapse naturally diminishes when the neuron is already strongly depolarized (the driving force shrinks), a phenomenon called "shunting" that the voltage-jump model cannot capture.

Calibrating the weights for this model required direct experimental work: a first attempt with an incorrect initialization of the recovery variable u (set to zero instead of the equilibrium value $b \cdot v$, the same conceptual error already avoided in every previous experiment) produced a single EPSP of apparently zero amplitude; once the initialization was corrected, experimental calibration identified a single synaptic strength of ~20-40 (in units of 1/second) as corresponding to a 1-3 mV EPSP, a physiologically plausible value.

6.3 First Allen-density network: 118,000 neurons, random connectivity

The first network built with both improvements (density 2,600 synapses/neuron + conductance-based synapses) still kept the RANDOM connectivity used in the 40-million network. Successfully verified: exactly 2,601 synapses/neuron, stable network frequency (~18 Hz), homeostatic behavior confirmed (baseline 15.85 Hz, peak 17.95 Hz after the stimulus, recovery to 15.32 Hz -- practically identical to baseline).

The actual run on the MacBook Air M5 took only 25 minutes, significantly less than the initial estimate (45-90 minutes) based on previous benchmarks -- a confirmation that time estimates for experiments at this scale remain inherently uncertain.

6.4 Synthesis: spatial structure + biophysical realism -- first unstable attempt

The next step combined ALL the improvements developed separately: brain-shaped spatial connectivity (Sections 4.1-4.2, via KD-tree), conductance-based synapses (Section 6.2), and Allen Institute synaptic density (Section 6.1), keeping the same scale of 118,000 neurons (constrained by the same memory budget). The synaptic weights used were those already calibrated for the random-connectivity network (Section 6.3).

The result of the first full-scale run revealed a serious problem, not

anticipated by the reduced-scale tests (4,000 neurons) run during development:

Metric	Value
Baseline frequency	18.50 Hz
Peak after stimulus	69.30 Hz (+274%)
Frequency after recovery	2.70 Hz (suppression, not recovery)
Initial oscillation amplitude	70.73 Hz
Final oscillation amplitude	71.49 Hz (growing)

Two warning signs, both qualitatively different from every previous experiment in the project: the network did not return to equilibrium after the stimulus (a collapse to 2.70 Hz, not a recovery), and the amplitude of spontaneous oscillations grew over time instead of stabilizing. The reduced-scale test (4,000 neurons) had not predicted this behavior -- a direct reminder that a recurrent network's dynamics can change QUALITATIVELY, not just quantitatively, when crossing a scale or local-density threshold.

Most likely interpretation: combining spatial clustering (which concentrates connections between nearby neurons) with high synaptic density produces local circuits with a much higher recurrent gain than the same total density produces in a purely random network (where the same connections are spread across the whole network, not concentrated locally).

6.5 Retuning and verification: the successful correction

The attempt to correct this behavior ran into a practical limit: the development sandbox had only ~3.9 GB of RAM, insufficient to reproduce the instability at the exact scale (118,000 neurons, 2,200 local connections per neuron) where it appeared. The strategy adopted was to reproduce the instability at an intermediate scale (20,000 neurons, with still-dense local connectivity) to verify the corrective hypothesis before applying it at full scale.

The correction applied -- reducing the strength of excitatory and inhibitory synapses while keeping their ratio (from $w_e=0.22$, $w_i=0.9$ to $w_e=0.12$, $w_i=0.6$, roughly 45% lower) -- is the standard remedy for networks showing growing oscillations instead of stabilizing: reduce the system's overall recurrent gain. Verified at intermediate scale with a positive outcome (amplitude stable or even decreasing over time), the correction was applied to the full-scale network, with the explicit caveat that the reduced-scale verification was no absolute guarantee of success at the 118,000-neuron scale.

The result of the full-scale run with the corrected weights confirmed the hypothesis:

Metric	Before (unstable)	After (corrected)
Baseline frequency	18.50 Hz	15.22 Hz
Peak after stimulus	69.30 Hz	17.97 Hz
Frequency after recovery	2.70 Hz	14.31 Hz
Initial oscillation amplitude	70.73 Hz	14.79 Hz
Final oscillation amplitude	71.49 Hz (growing)	9.59 Hz (declining)

The recovery is now a genuine recovery (14.31 Hz, practically identical to the 15.22 Hz baseline), and the oscillation amplitude **DECREASES** over time (14.79 -> 9.59 Hz) instead of growing. The correction, based on a general principle (reduce recurrent gain), worked even without a preventive end-to-end check at the exact scale -- a concrete example of how a solid theoretical principle can correctly guide a decision even when direct computational verification is not practical.

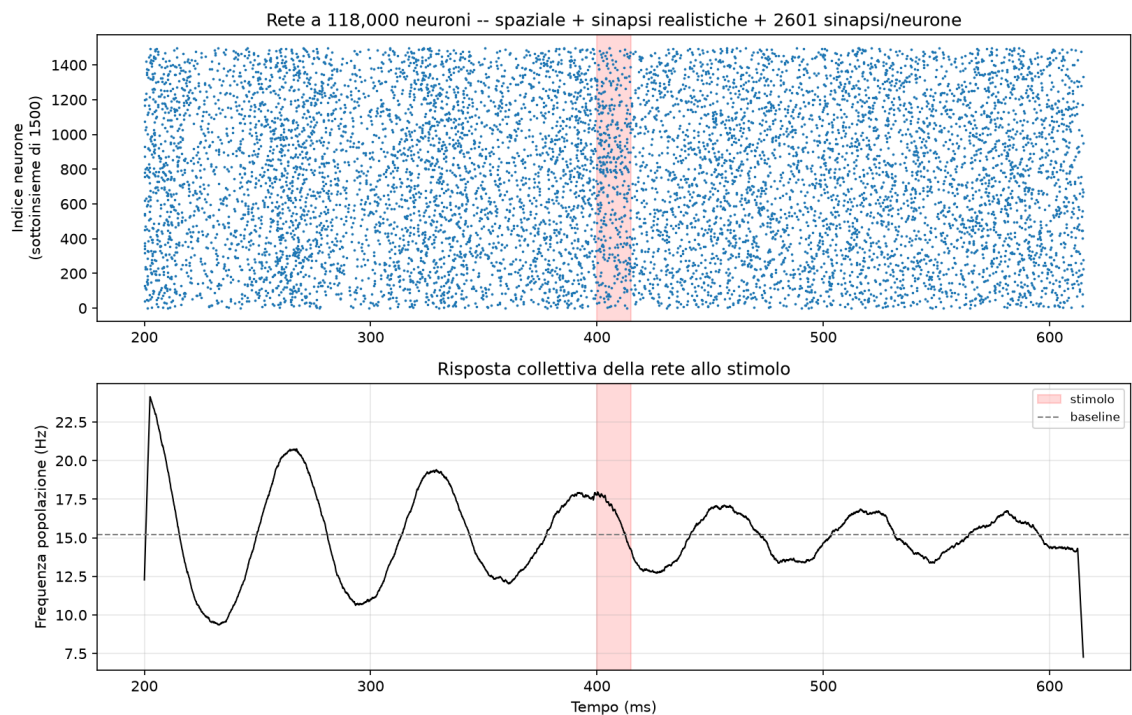


Fig. 19 — Final network (118,000 neurons, spatial + conductance + Allen density): raster and collective response, after the weight correction.

The plot reveals a qualitative detail not captured by the numerical metrics alone: the network shows a REGULAR, PERSISTENT oscillation for the entire duration of the simulation (period of roughly 65-70 milliseconds, approximately constant amplitude), different both from the first attempt's unstable behavior (unboundedly growing amplitude) and from the irregular asynchronous behavior observed in simpler networks (Sections 2.8, 3.6). The stimulus (red band, 400-415ms) falls during a rising phase of the network's natural cycle, making it visually hard to distinguish from the adjacent spontaneous peaks -- a phenomenon that would require statistical verification over multiple trials (analogous to Section 2.11's PSTH) to characterize with confidence.

6.6 3D visualization of the final network

The final network (118,000 neurons) was visualized interactively by exporting the spatial positions and activity over time of a subset of 7,867 neurons (one neuron in every 15, to keep the animation smooth in the browser), using the same tools developed for previous visualizations (Section 4.2). The resulting HTML file is self-contained and interactive (not included as text in this document given its interactive nature, but attached separately).

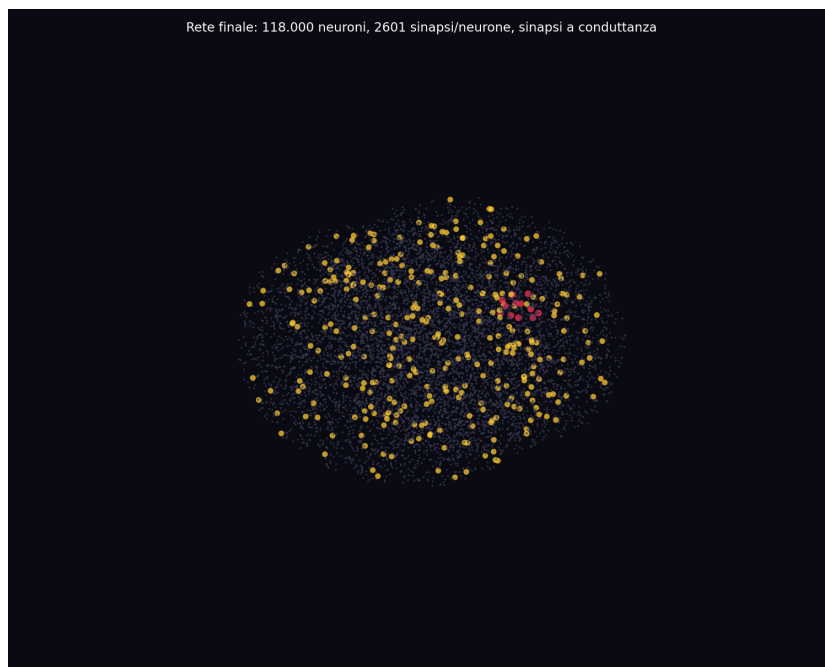


Fig. 20 — Snapshot of the final network in 3D (7,867 neurons rendered out of 118,000).

Source code: `rete_sintesi_export.py` (version with 3D data export)

```

2 Variante di rete_sintesi_finale.py con export dei frame di spike per la
3 visualizzazione 3D interattiva (rete_sintesi_3d_data.json). Stessa rete:
4 struttura spaziale 3D a due lobi (KD-tree), sinapsi a conduttanza, densita'
5 ~2600 sinapsi/neurone. Vedi rete_sintesi_finale.py per le note sulla
6 calibrazione dei pesi e sul comportamento oscillatorio a questa densita'.
7 """
8
9 from brian2 import *
10 import numpy as np
11 from scipy.spatial import cKDTree
12 import time as pytime
13
14 print("Fase 1/4: generazione della forma 3D del cervello (118.000 punti)...")
15
16
17 def sample_brain_shape(n_target):
18     points = []
19     total = 0
20     while total < n_target:
21         batch = np.random.uniform(-1.3, 1.3, size=(n_target*3, 3))
22         x, y, z = batch[:, 0], batch[:, 1], batch[:, 2]
23         left = ((x+0.35)/0.55)**2 + (y/0.45)**2 + (z/0.42)**2
24         right = ((x-0.35)/0.55)**2 + (y/0.45)**2 + (z/0.42)**2
25         inside = (left <= 1.0) | (right <= 1.0)
26         pts = batch[inside]
27         points.append(pts)
28         total += len(pts)
29     return np.vstack(points)[:n_target]
30
31
32 N = 118_000
33 N_E = int(N * 0.8)
34 K_LOCAL = 2200 # connessioni verso i vicini spaziali piu' prossimi
35 K_FAR = 400 # connessioni "scorciatoia" a lunga distanza (small-world)
36
37 np.random.seed(42)
38 pts = sample_brain_shape(N)
39 print(f"Punti generati.")
40
41 print("Fase 2/4: calcolo della connettivita' spaziale (KD-tree, a blocchi "
42       "per contenere l'uso di memoria)...")
43 t0 = pytime.time()
44 tree = cKDTree(pts)
45 batch_size = 2000
46 all_neigh = np.zeros((N, K_LOCAL), dtype=np.int32)
47 for start in range(0, N, batch_size):
48     end = min(start + batch_size, N)
49     _, neigh = tree.query(pts[start:end], k=K_LOCAL+1, workers=-1)
50     all_neigh[start:end] = neigh[:, 1:].astype(np.int32)
51 print(f"Connettivita' locale calcolata in {pytime.time()-t0:.1f}s.")
52
53 src_local = np.repeat(np.arange(N), K_LOCAL)
54 tgt_local = all_neigh.flatten()
55 src_far = np.repeat(np.arange(N), K_FAR)
56 tgt_far = np.random.randint(0, N, size=N*K_FAR)
57 src_all = np.concatenate([src_local, src_far])
58 tgt_all = np.concatenate([tgt_local, tgt_far])
59 valid = src_all != tgt_all
60 src_all, tgt_all = src_all[valid], tgt_all[valid]
61 del src_local, tgt_local, src_far, tgt_far, all_neigh
62
63 print(f"Connessioni totali generate: {len(src_all):,} "
64       f"({len(src_all)/N:.0f} per neurone)")
65
66 print("Fase 3/4: costruzione della rete Brian...")
67 start_scope()
68
69 a = 0.02/ms

```

```

70 b = 0.2/ms
71 c = -65*mV
72 d = 6*mV/ms
73 E_exc = 0*mV
74 E_inh = -80*mV
75 tau_e = 5*ms
76 tau_i = 10*ms
77
78 eqs = '''
79 dv/dt = (0.04/mV*v**2 + 5*v + 140*mV)/ms - u + ge*(E_exc-v) + gi*(E_inh-v) + I_ext : volt
80 du/dt = a*(b*v - u) : volt/second
81 dge/dt = -ge/tau_e : 1/second
82 dgi/dt = -gi/tau_i : 1/second
83 I_ext : volt/second
84 '''
85
86 neurons = NeuronGroup(N, eqs, threshold='v > 30*mV', reset='v = c; u += d',
87                       method='euler')
88 neurons.v = -65*mV + 10*mV*rand(N)
89 neurons.u = b * neurons.v[:]
90 neurons.I_ext = 0*mV/ms
91
92 background = PoissonGroup(N, rates=1200*Hz)
93 bg_syn = Synapses(background, neurons, on_pre='ge_post += 15/second')
94 bg_syn.connect(j='i')
95
96 # ridotti rispetto a rete_realistica_118K, vedi rete_sintesi_finale.py
97 w_e = 0.12/second
98 w_i = 0.6/second
99
100 src_is_E = src_all < N_E
101 syn_exc = Synapses(neurons, neurons, on_pre='ge_post += w_e')
102 syn_exc.connect(i=src_all[src_is_E], j=tgt_all[src_is_E])
103 syn_inh = Synapses(neurons, neurons, on_pre='gi_post += w_i')
104 syn_inh.connect(i=src_all[~src_is_E], j=tgt_all[~src_is_E])
105
106 n_syn_total = len(syn_exc) + len(syn_inh) + len(bg_syn)
107 print(f"Rete costruita. Sinapsi totali: {n_syn_total:,} "
108       f"({n_syn_total/N:.0f}/neurone)")
109
110 net = Network(collect())
111
112 print("\nFase 4/4: simulazione (burn-in + baseline + stimolo + recovery)...")
113 t0 = pytime.time()
114 net.run(200*ms)
115 print(f"Burn-in completato in {pytime.time()-t0:.1f}s.")
116
117 rate_mon = PopulationRateMonitor(neurons)
118 raster_subset = SpikeMonitor(neurons[:1500], record=True)
119 full_spikes = SpikeMonitor(neurons, record=True) # per l'esportazione 3D
120 net.add(rate_mon, raster_subset, full_spikes)
121
122 t0 = pytime.time()
123 net.run(200*ms)
124 print(f"Baseline completata in {pytime.time()-t0:.1f}s.")
125
126 # stimolo localizzato vicino alla punta dell'emisfero sinistro
127 target = np.array([-0.85, 0.0, 0.0])
128 d_target = np.linalg.norm(pts[:N_E] - target, axis=1)
129 stim_idx = np.argsort(d_target)[:200]
130 stim_time = 400*ms
131 neurons.I_ext[stim_idx] = 40*mV/ms
132 net.run(15*ms)
133 neurons.I_ext[stim_idx] = 0*mV/ms
134 print(f"Stimolo applicato a {len(stim_idx)} neuroni vicino alla punta "
135       f"dell'emisfero sinistro.")
136
137 t0 = pytime.time()

```



```

138 net.run(200*ms)
139 print(f"Recovery completata in {pytime.time()-t0:.1f}s.")
140
141 r = np.array(rate_mon.smooth_rate(window='flat', width=5*ms)/Hz)
142 t = np.array(rate_mon.t/ms)
143
144 baseline_rate = r[(t > 200) & (t < 400)].mean()
145 peak_rate = r[(t >= 400) & (t < 450)].max()
146 recovery_rate = r[t > 590].mean()
147 amplitude_early = r[(t > 200) & (t < 300)]
148 amplitude_late = r[(t > 500) & (t < 615)]
149
150 print("\n" + "=" * 60)
151 print("RISULTATI FINALI")
152 print("=" * 60)
153 print(f"Scala: {N:,} neuroni, {n_syn_total:,} sinapsi ({n_syn_total/N:.0f}/neurone)")
154 print(f"Struttura: spaziale (small-world 3D, forma a cervello)")
155 print(f"Sinapsi: a conduttanza (biofisicamente realistiche)")
156 print(f"Frequenza baseline: {baseline_rate:.2f} Hz")
157 print(f"Picco dopo lo stimolo: {peak_rate:.2f} Hz")
158 print(f"Frequenza dopo il recovery: {recovery_rate:.2f} Hz")
159 print(f"Ampiezza oscillazione (200-300ms): "
160       f"{amplitude_early.max()-amplitude_early.min():.2f} Hz")
161 print(f"Ampiezza oscillazione (500-615ms): "
162       f"{amplitude_late.max()-amplitude_late.min():.2f} Hz")
163
164 # export frame per la visualizzazione 3D interattiva
165 print("\nEsportazione dati per la visualizzazione 3D...")
166 import json
167
168 render_stride = 15 # ~7.870 neuroni renderizzati su 118.000
169 render_idx = np.arange(0, N, render_stride)
170
171 t_arr = np.array(full_spikes.t/ms)
172 i_arr = np.array(full_spikes.i)
173 print(f"Spike totali registrati: {len(t_arr):,}")
174
175 render_mask = np.isin(i_arr, render_idx)
176 t_render = t_arr[render_mask]
177 i_render = i_arr[render_mask]
178
179 frame_bins = np.arange(200, 616, 4) # 4ms/frame
180 frames = []
181 for k in range(len(frame_bins)-1):
182     mask = (t_render >= frame_bins[k]) & (t_render < frame_bins[k+1])
183     active_global = i_render[mask]
184     active_local = np.searchsorted(render_idx, active_global)
185     frames.append(active_local.tolist())
186
187 # garantisce che gli stimolati restino visibili nel sottoinsieme renderizzato
188 stim_idx_local = np.searchsorted(render_idx, np.intersect1d(stim_idx, render_idx))
189 if len(stim_idx_local) < 10:
190     positions_render_tmp = pts[render_idx]
191     dists_to_stim = np.linalg.norm(positions_render_tmp - target, axis=1)
192     nearest = np.argsort(dists_to_stim)[:15]
193     stim_idx_local = np.union1d(stim_idx_local, nearest)
194
195 export = {
196     "positions": (pts[render_idx] * 100).tolist(),
197     "stim_indices_local": stim_idx_local.tolist(),
198     "frame_duration_ms": 4,
199     "frame_start_ms": 200,
200     "stim_time_ms": float(stim_time/ms),
201     "n_neurons_total": N,
202     "n_synapses_per_neuron": round(n_syn_total/N),
203     "frames": frames,
204 }
205 with open('rete_sintesi_3d_data.json', 'w') as f:

```

```

206     json.dump(export, f)
207
208 import os
209 print(f"Dati esportati: {len(frames)} frame, {len(render_idx):,} neuroni renderizzati")
210 print(f"Dimensione file: {os.path.getsize('rete_sintesi_3d_data.json')/1024:.0f} KB")
211
212 figure(figsize=(11, 7))
213
214 subplot(2, 1, 1)
215 plot(raster_subset.t/ms, raster_subset.i, '.', color='C0', markersize=1.5)
216 axvspan(stim_time/ms, (stim_time+15*ms)/ms, color='red', alpha=0.15)
217 ylabel('Indice neurone\n(sottoinsieme di 1500)')
218 title(f'Rete a {N:,} neuroni -- spaziale + sinapsi realistiche + '
219       f'{n_syn_total/N:.0f} sinapsi/neurone')
220
221 subplot(2, 1, 2)
222 plot(t, r, color='black', linewidth=1)
223 axvspan(stim_time/ms, (stim_time+15*ms)/ms, color='red', alpha=0.15, label='stimolo')
224 axhline(baseline_rate, color='gray', linestyle='--', linewidth=1, label='baseline')
225 xlabel('Tempo (ms)')
226 ylabel('Frequenza popolazione (Hz)')
227 title('Risposta collettiva della rete allo stimolo')
228 legend(loc='upper right', fontsize=8)
229 grid(alpha=0.3)
230
231 tight_layout()
232 savefig('rete_sintesi_finale.png', dpi=150)
233 print("\nGrafico salvato in rete_sintesi_finale.png")

```

6.7 Final comparison with the Allen Institute

	40M network (pure scale)	Final network (synthesis)	Allen Institute
Neurons	40,000,000	118,000	~10,000,000
Synapses/neuron	31	2,601	~2,600
Synapse type	Voltage jump	Conductance	Conductance + morphology
Structure	Random	Spatial (3D small-world)	Real connectome
% of mouse brain	57.1%	0.169%	~14.3%

The final comparison is honest on both fronts: on per-neuron synaptic density, this project reached **PARITY** with the Allen Institute (2,601 against ~2,600); on the absolute number of neurons and on each one's morphological detail, the gap remains wide. Neither of the two networks built in this project (the maximum-scale one and the maximum-density one) dominates the other on every axis at once -- a direct, concretely observed consequence of the shared-memory constraint discussed at the opening of this section (6.1).

7. Discussion

7.1 Summary of the complete path

The path described in this document made it possible to build, experimentally verify, and personally observe a sequence of sixteen neural phenomena and experiments of steadily increasing complexity, always starting from minimal models and adding just one element of complexity at a time: basic synaptic communication, activation threshold, short-term plasticity, chain propagation, temporal summation, Hebbian learning via STDP, elementary pattern classification, balanced collective activity in a 300-neuron population, homeostasis after perturbation, modular organization, rigorous statistical verification, scaling up to 40 million neurons, structured spatial connectivity, interactive 3D visualization, one failed and one successful attempt at building a neural attractor, chains of linked attractors, competition between alternatives, reinforcement learning, and finally a context-conditioned decision policy.

This summary, in its first draft, covered only the first half of the document (Sections 1-5.4). The work then continued along a distinct but connected arc: learned bidirectional communication in the spiking domain, from total failure to a complete solution (transmission switch, homeostatic normalization, emergency suppression -- Sections 5.6-5.17), extended to multiple simultaneous messages and to a three-population relay network (Sections 5.18-5.20). Grafted onto that same architecture is a second thread, this time in physics (Sections 5.21-5.41): the impact of clinical CNAO ion beams (Geant4, QBBC) on the neural ring, linked to the spiking architecture through a functional-knockout experiment that reveals a dose-response gradient verified across four independent physical scenarios and generalized beyond the specific choice of knockout size; functional recovery after damage and a point of no return when the damage is complete; a clinical RBE model (Kase's Microdosimetric Kinetic Model, the same one used at NIRS/HIMAC in Chiba) implemented and validated against published data before being used; a limitation discovered and corrected in the reproducibility of the per-neuron dose map; and finally a nanodosimetric link (Geant4-DNA) verified end-to-end, from the secondary electrons actually produced in the affected neurons all the way to ionization clusters, consistent with the published literature. The same methodological discipline described in Section 7.2 -- verify empirically instead of assuming, document failed attempts as much as successful ones -- runs through both threads without interruption.

7.2 The two most important methodological lessons

Looking back over the entire document, two methodological lessons stand out with particular clarity, repeated in different forms at different points in the work:

6.2.1 The difference between "looks true" and "verified true"

At least three times over the course of the project, a result that looked convincing at first glance turned out, after rigorous statistical verification, not to be supported by the data: propagation between modules in Section 2.11 (an apparent effect of +13 Hz, which turned out to be only +0.89 Hz once averaged over 25 independent trials, well within the background noise of ~4 Hz); the attempted STDP-based persistence in Section 4.3 (activity apparently present in a single, non-repeated trial, which turned out to be systematically zero over 16 correctly randomized independent trials); and the symmetry breaking in the equal-weights competition of Section 5.2.1 (apparently a "winning" candidate for some structural reason, which turned out to be a deterministic numerical artifact masked by insufficient noise). In all three cases, the gap between the initial impression and the verified result emerged only by applying the correct statistical tools (repetition, controlled randomization, comparison with the system's natural variability) -- not through intuition or a closer visual inspection of a single plot.

6.2.2 Failed attempts are part of the knowledge produced, not noise to hide

The STDP-based attractor attempt (Section 4.3) and the first two architecture attempts for the ring attractor (Section 4.4.1, items 1 and 2) failed completely before the third attempt succeeded. Reporting these failures in detail -- including the mistaken hypotheses that motivated them and the precise technical reasons they did not work -- is not a flaw in the document, but an integral part of its value: it shows the real process by which scientific knowledge is built, one that rarely proceeds in a straight line from problem to solution.

7.3 What is missing for real brain-like functionality

Even the project's most advanced result (the contextual decision policy of Section 5.4) remains at an enormous distance from anything one could honestly call "brain-like functionality" in the full sense of the term. The missing ingredients, in roughly increasing order of difficulty to add:

- Real structured connectivity (a true connectome) instead of a simplified geometric model -- probably the single biggest challenge, still the subject of active research worldwide with the best tools available (see Section 4.5)
- Diversity of cell types -- a real brain has hundreds of morphologically and functionally distinct neuron types, not just the two homogeneous categories (excitatory/inhibitory) used in this project
- Structured, meaningful sensory input and motor output from the real world, instead of the Poisson-type random noise used as "background activity" in every experiment in this project

- Neuromodulation (dopamine, serotonin, acetylcholine, noradrenaline), which operates on timescales much slower than individual electrical impulses, dynamically regulating the plasticity and excitability of entire neuronal populations as a function of behavioral state
- Development and learning over timescales of months or years, not the few seconds of simulated time used in every experiment in this project
- Generalization to never-before-seen situations -- the weights[context][candidate] table of Section 5.4 is an associative memory (a lookup table), not an abstract representation that would allow correct interpolation or generalization to an intermediate context never presented during training

8. Limitations of the work

- The Izhikevich model is a deliberate simplification: it does not include the neuron's real morphology (dendritic structure, axonal geometry), nor the diversity of ion channels present in real biological neurons -- chosen out of the need to scale to millions of neurons, at the cost of lower biophysical realism than models such as Hodgkin-Huxley
- Synaptic connections in most experiments have a static, "instantaneous" weight (apart from the explicit plasticity experiments, Sections 2.3, 2.6, 2.7), without realistic neurotransmitter-release dynamics such as short-term depression or postsynaptic receptor saturation
- The inter-module propagation effect (Sections 2.10-2.11) deserves to be retested with stronger inter-module connectivity parameters, to find out whether a real effect emerges more clearly with denser connections, or whether the weakness observed at 2% connectivity is a robust result that holds even at higher connectivity
- The 40-million-neuron network (Section 3) uses the same fixed-degree random connectivity as the much smaller networks -- a genuine structural approach toward a mouse brain would require combining the scale reached with the spatial connectivity developed separately in Section 4, something the project did not have time to achieve jointly at full scale
- Thermal-throttling slowdown (Section 3.4) introduces a not-fully-controlled hardware variable in the longer experiments, making time estimates for future experiments of comparable scale inherently uncertain
- The systematic position bias (~ 0.045 units) repeatedly observed in the ring-attractor experiments (Sections 4.4.2, 5.1, 5.2) has not been fully diagnosed at its root -- probably a discretization artifact of the $N=200$ -point grid, but without a formal check ruling it out as a real effect of the model

- The contextual policy of Section 5.4 was verified on only 3 discrete, well-separated contexts on the ring -- it was not tested whether the mechanism generalizes to a larger number of contexts, to contexts spatially closer together (which could interfere with each other), or to the ability to correctly interpolate for an intermediate context never presented during training
- None of this project's experiments includes a cross-validation phase or a test set completely separate from training -- a standard machine-learning practice for ruling out overfitting, not systematically applied here given the exploratory nature of the work

The limitations above concern the pure-neuroscience thread (Sections 1-5.20). The connected physics thread (Geant4/CNAO/Chiba, Sections 5.21-5.41) has its own, distinct ones: the per-neuron dose map is fundamentally limited by counting statistics (each neuron, 10um radius, intercepts on average only 1-2 direct primaries out of 500,000 simulated events -- a limitation quantified in Section 5.37, not solvable with a 10x increase in statistics; a factor of 100-1000 would likely be needed, computationally prohibitive with QBBC's full hadronic physics within this project's time constraints).

The RBE model (Section 5.34) is the real Kase Microdosimetric Kinetic Model, no longer an ad hoc approximation, but remains simplified relative to the full clinical model in two stated respects: it uses the dose-averaged LET as a proxy for the dose-mean lineal energy (which would require a spectrum measured with a TEPC or a true site-level microdosimetric simulation), and it computes the saturation correction in the single-value-spectrum limit instead of integrating a full spectrum.

The functional knockout (Sections 5.22-5.38) remains a deliberately simplified model of neuronal damage -- complete removal of outgoing synapses above a relative dose threshold, not a validated biological-damage model (no absolute dose threshold for inactivating a single neuron is established in the literature, the same limitation stated in the Villagrasa/Baiocco paper that the twin project geant4dna_icsd reproduces).

The nanodosimetric link (Sections 5.39-5.41) stops at the level of ICSD (Ionization Cluster Size Distribution) -- it does not model DNA damage (single or double breaks, SSB/DSB), which would require a geometric cell-nucleus model with multi-scale chromatin (available in Geant4-DNA, e.g. the official dnadamage1/molecularDNA examples, but not integrated here).

9. Possible future developments

- Combine the scale already reached (Section 3, 40 million neurons) with the spatial connectivity developed separately (Section 4), to obtain a network that is simultaneously large AND structured -- the natural next step toward getting closer to a real mouse brain on both axes of realism

- Make the connections between the two ring attractors of Section 5.1 plastic (STDP) as well, to study whether and how the associative map's offset could itself be LEARNED by the system instead of being fixed externally beforehand
- Repeat the rigorous statistical verification (PSTH technique, Section 2.11) on the attractor and reinforcement-learning experiments of Sections 4 and 5 as well, to check the statistical robustness of the positive results obtained over a relatively limited number of trials (6-18 depending on the experiment)
- Extend the contextual policy (Section 5.4) to a larger number of contexts, and explicitly test the ability to generalize to intermediate contexts never seen during training -- the true test for distinguishing a pure associative memory from the beginnings of an abstract representation
- Explore a network topology with more than two modules (Section 2.10), with connectivity patterns inspired by real published cortical connectivity maps (for example those of the Allen Institute, Section 4.5.2), instead of the artificial geometric kernel used in this project
- Test access to a university computing cluster (for example through CNAO or the University of Pavia's Physics Department) to overcome the RAM limit of a personal laptop, empirically verified in Section 3.3 at around 40-50 million neurons with simple random connectivity
- Formally investigate the origin of the systematic position bias observed in the ring attractor (Sections 4.4.2, 5.1, 5.2), to determine whether it is a purely numerical artifact or reflects a real (if unwanted) property of the discretized model

The future developments above concern the pure-neuroscience thread. For the physics thread (Geant4/CNAO/Chiba), the open items remain those of the twin project `geant4dna_icsd` (see its README): implementing the "common" cross-section dataset defined in the Villagrasa/Baiocco paper (requires extracting the numerical values from the supplementary material and writing a dedicated `G4VEmModel` subclass -- substantial work, not a weekend extension, as the paper itself notes) and, above all, extending the already-verified physical chain (macroscopic dose -> secondary-electron spectrum -> nanodosimetric ICSD) all the way to DNA damage (SSB/DSB) with a geometric cell-nucleus model -- the natural next step to make the link clinically interpretable, not just physically coherent. [Update: SSB/DSB damage was completed in Section 5.42, using the `moleculardna` example instead of `dnadamage1/molecularDNA` (blocked by a missing external dependency) -- only the paper's common cross-section dataset remains open.]

10. Glossary of technical terms used in the document

Attractor: A state (or set of states) toward which a dynamical system converges over time, and to which it returns after small perturbations. The ring attractor of Section 4.4 is the concrete example built in this project.

Multi-armed bandit: A classic reinforcement-learning problem in which an agent must repeatedly choose among several alternatives ("arms"), each with a different, unknown average reward, learning which to choose more often through repeated trials.

Excitation-inhibition balance (E/I balance): The ratio and dynamic equilibrium between excitatory neurons (which push other neurons toward spiking) and inhibitory ones (which suppress them) in a network, necessary to obtain stable activity instead of explosion or silence.

Brian2: A Python library for simulating spiking neural networks, used for every spiking experiment in this project.

Bump attractor: A specific type of neural attractor in which the stable state is a localized region of elevated activity ("the bump") on a broader spatial substrate, typically a ring or a grid. See Section 4.4.

Connectome: The complete map of synaptic connections among all the neurons of a nervous system (or a portion of it).

Standard deviation: A statistical measure of the spread of a set of values around their mean -- used repeatedly in this document to distinguish a real effect from random noise.

Izhikevich model: A two-coupled-differential-equation mathematical neuron model, proposed by Eugene Izhikevich in 2003. See Section 1.2.

KD-tree: A data structure that organizes points in a K-dimensional space to allow efficient spatial nearest-neighbor searches, used in Section 4.2 to compute the spatial connectivity of networks with up to 1 million neurons.

Homeostasis: A system's ability to return to a stable equilibrium state after a perturbation, without external intervention. First observed in Section 2.9.

PSTH (Peristimulus Time Histogram): A standard statistical technique in experimental neuroscience for measuring a system's average response to a stimulus, by averaging over many independent repetitions aligned to the moment of the stimulus. See Section 2.11.

Rescorla-Wagner rule: A historic (1972) mathematical model of associative learning, in which the strength of an association is updated in proportion to the difference between the reward received and the reward expected. Used as the basis for the update rule in Section 5.3.

Ring attractor: A neural attractor implemented on a ring topology, where the bump's position encodes a continuous variable (for example a spatial position or an orientation). See Section 4.4.

Small-world (topology): A network organization pattern characterized by dense local connections combined with rare long-distance connections, observed in a great many real biological and social systems. Proposed by Watts and Strogatz in 1998. See Section 4.1.

Spike / action potential: The electrical impulse a neuron generates when its membrane potential crosses a threshold, the fundamental mechanism of communication between neurons.

STDP (Spike-Timing-Dependent Plasticity): A synaptic learning rule in which the strengthening or weakening of a synapse depends on the precise temporal order between presynaptic and postsynaptic spikes. See Section 2.6.

Winner-take-all: A computational mechanism, typically implemented via global or lateral inhibition, in which only one element (or a small subset) of a competitive system can remain active, suppressing all other candidates.

11. Materials and reproducibility

Every Python script developed during the project was verified by actually running it (not just writing it) and is fully reproducible. The complete or near-complete source code of each main script is included in the body of this document, in its corresponding section; the list below summarizes them with a reference to the section where they are discussed.

File	Section	Content
due_neuroni.py	2.1	Basic communication between two neurons
esplorazione_parametri.py	2.2	Synaptic threshold
plasticita_sinaptica.py	2.3	Short-term facilitation
rete_catena.py	2.4	Chain propagation
rete_convergente.py	2.5	Temporal summation
stdp.py	2.6	Spike-Timing-Dependent Plasticity
classificazione_stdp.py	2.7	Selective pattern learning
rete_300_neuroni.py	2.8	Balanced E/I network
rete_300_stimolo.py	2.9	Response to stimulus and recovery
due_moduli.py	2.10	Modular architecture

File	Section	Content
psth_moduli.py	2.11	Statistical verification over 25 trials
scaling_verso_topo.py, scaling_fase2/3/4.py	3.5	Automatic progressive scaling
rete_40M_finale.py	3.6	Final experiment at 40 million neurons
rete_smallworld.py	4.1	Small-world connectivity on a ring
brain3d_ visualization.html, brain1M_ visualization.html	4.2	Interactive 3D visualizations
ring_attractor_rate.py	4.4	Ring attractor, final verified result
chained_attractors.py	5.1	Chain of two linked attractors
competing_targets.py	5.2	Competition between alternatives
rl_bandit.py	5.3	Reinforcement learning (bandit)
contextual_policy.py	5.4	Contextual decision policy
rete_sintesi_export.py	6.4-6.6	Final synthesis: spatial + conductance + Allen density
dialogo_ring_ attractor.py	5.5.1	Turn-based exchange between two rings, fixed channels
dialogo_stdp.py	5.5.2	Learned A->B synapse (turn-grained Hebbian); B->A asymmetry documented
dialogo_stdp_ adattamento.py	5.5.3	Adaptation + warm-up: both channels 100% reliable
dialogo_multi_ messaggio.py	5.5.4	Multiple messages on the same synapse: interference test
dialogo_multi_ normalizzato.py	5.5.5	Homeostatic normalization on W_AB: competition made visible, not resolved

File	Section	Content
dialogo_multi_repulsione.py	5.5.6	Repulsive exploration: interference resolved, 7 of 8 exchanges at 100%
diagnosi_bias_posizione.py	5.5.7	Direct test (and refutation) of the position-bias hypothesis
dialogo_multi_repulsione_3msg.py	5.5.8	Scaling to 3 messages: better separation, worse consolidation (open result)
dialogo_multi_scoperta_locale.py	5.5.9	Repulsion only for discovery + local exploration: 10/12 exchanges, best result
dialogo_multi_scoperta_locale_montecarlo.py	5.5.10	Monte Carlo verification (8 seeds): 91.7% success, but the hypothesis about the residual was wrong
dialogo_multi_scoperta_locale_permutazione.py	5.5.11	Position/value disambiguation: turn 5 follows order, turn 7 follows value
dialogo_multi_scoperta_locale_mischiato.py	5.5.12	Shuffled round-robin order: fairer across messages, but lower aggregate rate (87.5% vs 91.7%)
dialogo_multi_scoperta_bidirezionale.py	5.5.13	Discovery+local consolidation on both channels: 95.8%, turn 7 resolved at 100%
ring_attractor_spiking.py	5.6	First stable spiking ring attractor (10 tuning attempts, NMDA-like synapses)
dialogo_spiking.py	5.7	Learned A->B communication between two spiking ring attractors, true STDP
dialogo_spiking_bidirezionale.py	5.8	Attempt at spiking bidirectionality: same asymmetry as the rate model, unresolved

File	Section	Content
dialogo_spiking_ bidirezionale_turni.py	5.9	A/B turn-taking scheme to isolate the confound in the spiking B->A channel
dialogo_spiking_ bidirezionale_gate.py	5.9	B->A plasticity confined to unconfounded turns: definitive negative conclusion
dialogo_spiking_ multi.py	5.10	Two messages in the spiking domain, true STDP: no interference
dialogo_spiking_multi_ montecarlo.py	5.11	Monte Carlo verification (8 seeds): no spiking interference confirmed 16/16
diagnosi_sottosoglia_ ba.py	5.12	Sub-threshold check of the B->A channel: confounded by A's own dynamics
diagnosi_ablazione_ ba.py	5.12	Ablation: a strong B->A channel causes a numerical explosion, not a causal signal
dialogo_spiking_ bidirezionale_ silenzia.py	5.12	Active silencing of A: same explosion, final conclusion
dialogo_spiking_ bidirezionale_ interruttore.py	5.13	Transmission switch: stable with low weights, one seed out of four
diagnosi_ unidirezionale_pesi_ forti.py	5.13	Control: a unidirectional channel with strong weights is stable on its own
dialogo_spiking_ bidirezionale_ interruttore_mc.py	5.13	Monte Carlo verification (4 seeds): only 1/4 stable, the 'win' was a lucky case
dialogo_spiking_ bidirezionale_ omeostasi.py	5.14	Homeostatic weight normalization: stable spiking bidirectionality on the base seed

File	Section	Content
dialogo_spiking_ bidirezionale_ omeostasi_mc.py	5.14	Monte Carlo verification (8 seeds): 6/8 stable, real but incomplete improvement
dialogo_spiking_ bidirezionale_ omeostasi_v2.py	5.15	More reactive homeostasis: clean bidirectional growth on the base seed
dialogo_spiking_ bidirezionale_ omeostasi_v2_mc.py	5.15	Monte Carlo verification (8 seeds): 7/8 stable, better than v1
diagnosi_seed301_ anello_isolato.py	5.16	Verification: seed 301's isolated ring is stable on its own
dialogo_spiking_ bidirezionale_ omeostasi_v3.py	5.16	Emergency switch layered on gradual homeostasis
dialogo_spiking_ bidirezionale_ omeostasi_v3_mc.py	5.16	Monte Carlo verification (8 seeds): 7/8 clean, eighth improved 40x
dialogo_spiking_ bidirezionale_ omeostasi_v4.py	5.17	Spiking bidirectionality solved: transmission switch + homeostasis + direct suppression
dialogo_spiking_ bidirezionale_ omeostasi_v4_mc.py	5.17	Monte Carlo verification (8 seeds): 8/8 stable, bidirectional growth in every run
dialogo_spiking_ bidirezionale_multi.py	5.18	Multiple bidirectional spiking messages: completing the thread
dialogo_spiking_ bidirezionale_multi_ mc.py	5.18	Monte Carlo verification (8 seeds): 8/8 stable, no interference
dialogo_spiking_ bidirezionale_multi3.py	5.19	Three bidirectional spiking messages: load stress-test
dialogo_spiking_ bidirezionale_multi3_ mc.py	5.19	Monte Carlo verification (8 seeds): 8/8 stable, 98% localization under threshold
dialogo_spiking_relay_ 3anelli.py	5.20	Three-population relay A->B->C->A, first step beyond the pair

File	Section	Content
dialogo_spiking_relay_3anelli_mc.py	5.20	Monte Carlo verification (6 seeds): information preserved over two hops in every case
dialogo_spiking_cnao_knockout.py	5.22	Knockout of the highest-CNAO-dose neurons: selectively disrupts the message that uses them
dialogo_spiking_cnao_knockout_mc.py	5.23	Monte Carlo verification (8 seeds): knockout robust, 8/8 same direction
dialogo_spiking_knockout_controllo_mc.py	5.24	Negative control: random knockout does not reproduce the selective effect
braggProfile/braggProfile.c	5.25	Depth-dose profile: finds the Bragg peak, discovers a geometric artifact in cnaoRingImpact
dialogo_spiking_cnao_knockout_v2_mc.py	5.26	Re-verification with corrected dose: the effect flips as predicted (falsifiable test passed)
dialogo_spiking_cnao_knockout_picco_mc.py	5.27	Complete dataset at the peak + dose-response gradient across three overlap levels
dialogo_spiking_cnao_knockout_proton_mc.py	5.28	Knockout with a proton dose map: strongest effect so far (7.07x, 8/8), complete 4-point gradient
dialogo_spiking_cnao_recupero.py	5.29	Functional recovery after knockout: residual plasticity progressively compensates, verified over 4 seeds

File	Section	Content
dialogo_spiking_ lesione_completa_ mc.py	5.30	Complete lesion of the encoding zone: the recovery seen in 5.29 disappears, point of no return confirmed over 5 runs (1+4 MC)
geant4_cnao_ring (RunAction.cc + RBE analysis)	5.31	RBE-LET model (Kanai/NIRS): the 5.22-5.28 gradient holds within the same depth; discovered Monte Carlo variability between physical runs not yet verified
geant4_cnao_ring (cnaoRingImpact.cc, explicit seed)	5.32	Discovered: the per-neuron dose map is dominated by Monte Carlo noise (overlap 1.0/10 over 4 seeds); the neuroscience causal mechanism remains valid, the single-neuron-level correspondence does not
dialogo_spiking_cnao_ knockout_media4seed_ mc.py	5.33	Knockout from dose averaged over 4 Geant4 seeds: 1-vs-0 overlap indistinguishable from the negative control, a minimum imbalance threshold (~2 neurons) emerges for a real effect
geant4_cnao_ ring/analysis/mkm_ rbe.py	5.34	Real Kase/NIRS MKM model (no longer 5.31's linear approximation), validated against published data within 10-15%; confirms 5.31 with a true clinical model

File	Section	Content
geant4_cnao_ ring/analysis/mkm_ rbe.py (full dataset)	5.35	MKM across all six CNAO configurations: physical/biological overlap 9-10/10 everywhere; carbon overkill and anomalous 150 MeV proton RBE
geant4_cnao_ ring/src/SteppingAction.cc (CNAO_DEBUG_ NEURON diagnostic)	5.36	Empirically verified (not just hypothesized) the 5.35 anomalous LET peak: it is the primary proton nearly stopped (range straggling), not a secondary fragment -- linked to the real debate over variable RBE at the distal edge in proton therapy
build/results_highstat/ (carbon 5M events, seed 501/502)	5.37	Quantified: 10x events improves overlap from 1.0/10 (pure noise) to 2/10 -- not enough; confirms that averaging over seeds (5.33) is more efficient than increasing events
dialogo_spiking_cnao_ knockout_top5_mc.py / top20_mc.py	5.38	Knockout size (5, 10, 20 neurons) is not special: TOP-5 (0-vs-0 overlap) null as predicted, TOP-20 (reversed overlap) reverses the effect (1.74x, 7/8) exactly as predicted

File	Section	Content
geant4_cnao_ring (DetectorConstruction.cc, production cut)	5.39	First step of the nanodosimetric link: a bug (too-coarse production cut) discovered and fixed, real secondary-electron spectrum verified (100% below 1 MeV, median 1.86 keV) -- the physical premise of coupling with Geant4-DNA is confirmed
geant4dna_icsd/build/nanoICSD (real energies from 5.39)	5.40	Nanodosimetric link completed: ICSD from REAL (CNAO) secondary-electron energies fit coherently between the points already published in the Villagrasa/Baiocco paper -- macro-to-nano chain verified end-to-end
geant4dna_icsd/nanoICSD.cc (explicit seed, 4-seed verification)	5.41	Monte Carlo verification of nanoICSD: M1=1.090 confirmed stable (CV 0.34% over 4 seeds) -- unlike the dose map (5.32), no statistical-noise problem here
geant4dna_ssbsdb (moleculardna, real energies from 5.39)	5.42	Real DNA damage (SSB/DSB) from CNAO secondary electrons: higher DSB fraction at low energy (higher LET), verified over 3 seeds (SB CV 1.8%, DSB CV 12.4%) -- closes the nanodosimetric link down to the DNA level

File	Section	Content
geant4dna_ ssbdsb/results/spectrum_ weighted (6 weighted points)	5.43	DNA damage weighted over the entire real spectrum (6 points, not 2): DSB fraction monotonic with energy (15.7% to 5.4%), final weighted average 9.5% -- closes the last limitation stated in 5.42
geant4_cnao_ring (per-neuron DSB tracking)	5.44	Physics-neuroscience bridge closed down to the DNA: per-neuron estimated-DSB-damage ranking overlaps 9/10 with the physical-dose ranking -- physical dose remains a robust criterion even at the deepest biological realism level reached
dialogo_spiking_relay_ chiusura_turni_mc.py	5.45	The C->A loop does not close causally: verified over 5 runs (1+4 MC), never more than 3/10 probes with residual activity, always at noise level -- extends the 5.9 conclusion (B->A not causal) to the three-ring case

Execution environment: Python 3.14, Brian2 2.10.1, NumPy, SciPy, Matplotlib, on a MacBook Air with an Apple M5 chip and 24 GB of RAM. The two interactive 3D visualizations require a modern web browser with WebGL support (recent versions of Chrome, Safari, Firefox) and are self-contained (the simulation data is embedded directly in the HTML file; no internet connection is required except for the initial loading of the Three.js library from a CDN).

End of document.